
AtomX Portal

Complete Source Code Deposit

WORK	AtomX Portal — integrated web application suite
OWNER	AtomX
DEPLOYED AT	https://portal.atomx.in
COMPONENTS	Access Portal · Dashboard · Tag Series · shared packages
LANGUAGE	JavaScript (JSX), CSS
FRAMEWORK	Next.js 16.2.6 / React 19.2.0, npm workspaces + Turborepo
SOURCE FILES	162
LINES OF CODE	23,015
DEPOSIT PAGES	445
PREPARED	22 August 2026

This document contains the source code of AtomX Portal, an original literary work in the form of a computer programme. Generated build artefacts, third-party dependencies (*node_modules*), lockfiles and environment configuration files are excluded; the deposit contains only source authored for this work.

Table of Contents

FILE	LINES	PAGE
jsconfig.json	22	1
turbo.json	21	2
package.json	38	3
apps/access_portal/components/Add_Role/Add_Role.js	91	4
apps/access_portal/components/AssignmentCard/AssignmentCard.js	84	6
apps/access_portal/components/HeaderBar/HeaderBar.js	98	8
apps/access_portal/components/HeaderBar/HeaderBar.module.css	41	10
apps/access_portal/components/LoginScreen/LoginScreen.js	399	11
apps/access_portal/components/QuickActionCard/QuickActionCard.js	24	18
apps/access_portal/components/QuickActionCard/QuickActionCard.module.css	38	19
apps/access_portal/components/QuickActionsPanel/QuickActionsPanel.js	26	20
apps/access_portal/components/QuickActionsPanel/QuickActionsPanel.module.css	31	21
apps/access_portal/components/Session/SessionGuard.js	268	22
apps/access_portal/components/UserMenu/UserMenu.js	95	26
apps/access_portal/components/WelcomePanel/WelcomePanel.js	28	28
apps/access_portal/next.config.mjs	23	29
apps/access_portal/package.json	20	30
apps/access_portal/pages/_app.js	17	31
apps/access_portal/pages/_document.js	22	32
apps/access_portal/pages/access.js	1330	33
apps/access_portal/pages/index.js	29	53
apps/access_portal/postcss.config.mjs	7	54
apps/access_portal/styles/globals.css	180	55
apps/access_portal/tailwind.config.mjs	16	58
apps/dashboard/jsconfig.json	7	59
apps/dashboard/next.config.mjs	29	60
apps/dashboard/package.json	24	61
apps/dashboard/postcss.config.mjs	7	62
apps/dashboard/src/app/Blocked/components/BlockedCards.js	116	63
apps/dashboard/src/app/Blocked/page.js	60	65
apps/dashboard/src/app/Config/components/Access_Gate_Config.js	84	66
apps/dashboard/src/app/Config/components/Access_Gate_Master.js	80	68
apps/dashboard/src/app/Config/components/Add_Device.js	538	70
apps/dashboard/src/app/Config/components/ConfigIcons.js	79	79
apps/dashboard/src/app/Config/components/ConfigTransition.js	27	81
apps/dashboard/src/app/Config/components/CreateAccessGateMasterModal.js	117	82
apps/dashboard/src/app/Config/components/CreateStallModal.js	408	84
apps/dashboard/src/app/Config/components/CreateVendorModal.js	434	91
apps/dashboard/src/app/Config/components/EditAccessGateConfigModal.js	150	98
apps/dashboard/src/app/Config/components/EditAccessGateMasterModal.js	150	101
apps/dashboard/src/app/Config/components/EditAccessXCategoryModal.js	202	104
apps/dashboard/src/app/Config/components/EditStallModal.js	478	108
apps/dashboard/src/app/Config/components/EditVendorModal.js	469	116
apps/dashboard/src/app/Config/components/VendorConfigurationContent.js	1388	124
apps/dashboard/src/app/Config/components/stallPayload.js	60	145
apps/dashboard/src/app/Config/components/vendorPayload.js	74	146

Table of Contents

FILE	LINES	PAGE
apps/dashboard/src/app/Config/menu/components/CategoryDetailPanel.js	142	148
apps/dashboard/src/app/Config/menu/components/CategoryTabs.js	31	151
apps/dashboard/src/app/Config/menu/components/MenuBar.js	83	152
apps/dashboard/src/app/Config/menu/components/MenuContent.js	358	154
apps/dashboard/src/app/Config/menu/components/MenuIcons.js	121	160
apps/dashboard/src/app/Config/menu/components/MenuItemsTable.js	416	162
apps/dashboard/src/app/Config/menu/page.js	28	169
apps/dashboard/src/app/Config/page.js	24	170
apps/dashboard/src/app/Reports/components/ReportFilters.js	397	171
apps/dashboard/src/app/Reports/components/ReportHistoryTable.js	177	178
apps/dashboard/src/app/Reports/components/ReportsContent.js	159	181
apps/dashboard/src/app/Reports/page.js	27	184
apps/dashboard/src/app/admin/AdminClient.js	632	185
apps/dashboard/src/app/admin/AdminClientNoSSR.js	13	195
apps/dashboard/src/app/admin/Create_event/components/AdminRoleManager.js	625	196
apps/dashboard/src/app/admin/Create_event/page.js	18	206
apps/dashboard/src/app/admin/page.js	5	207
apps/dashboard/src/app/apk_upload/components/RecentUploads.js	96	208
apps/dashboard/src/app/apk_upload/components/UploadDropzone.js	41	210
apps/dashboard/src/app/apk_upload/components/UploadSummaryCard.js	27	211
apps/dashboard/src/app/apk_upload/page.js	76	212
apps/dashboard/src/app/components/Buttons/SaveButton.js	19	214
apps/dashboard/src/app/components/Header.js	247	215
apps/dashboard/src/app/components/Popups/Toast.js	31	219
apps/dashboard/src/app/components/PostHogInit.js	12	220
apps/dashboard/src/app/components/ProfileMenu.js	370	221
apps/dashboard/src/app/components/Session/SessionGuard.js	384	227
apps/dashboard/src/app/components/Session/SessionPrompt.js	62	233
apps/dashboard/src/app/components/SideDrawer.js	198	234
apps/dashboard/src/app/device/components/AddPersoDeviceModal.js	293	238
apps/dashboard/src/app/device/components/DeviceList.js	413	243
apps/dashboard/src/app/device/components/PersoDevicesTable.js	402	250
apps/dashboard/src/app/device/page.js	27	257
apps/dashboard/src/app/device_masterlist/components/DeviceBankCredentials.js	161	258
apps/dashboard/src/lib/apiConfig.js	3	261
packages/lib/src/apiConfig.js	5	262
apps/dashboard/src/app/device_masterlist/components/AddDeviceModal.js	171	263
apps/dashboard/src/app/device_masterlist/components/DeviceFormModal.js	255	266
apps/dashboard/src/app/device_masterlist/components/DeviceMasterIcons.js	39	271
apps/dashboard/src/app/device_masterlist/components/DeviceMasterList.js	311	272
apps/dashboard/src/app/device_masterlist/components/EditDeviceModal.js	229	277
apps/dashboard/src/app/device_masterlist/page.js	16	281
apps/dashboard/src/app/event-edit/components/EventEditContent.js	839	282
apps/dashboard/src/app/event-edit/components/EventEditPrimitives.js	210	295
apps/dashboard/src/app/event-edit/page.js	25	299
apps/dashboard/src/app/globals.css	139	300

Table of Contents

FILE	LINES	PAGE
apps/dashboard/src/app/layout.js	44	302
apps/dashboard/src/app/page.js	17	303
apps/dashboard/src/app/patcha-ny-track/page.js	17	304
apps/dashboard/src/app/tapx/components/TapXTransactions.js	415	305
apps/dashboard/src/app/tapx/page.js	16	312
apps/dashboard/src/app/timeline/components/SearchBar.js	31	313
apps/dashboard/src/app/timeline/components/TimelineEmptyState.js	31	314
apps/dashboard/src/app/timeline/page.js	26	315
apps/dashboard/src/app/transactions/components/TransactionFilters.js	1223	316
apps/dashboard/src/app/transactions/page.js	33	336
apps/dashboard/src/app/whitelist/components/WhitelistContent.js	326	337
apps/dashboard/src/app/whitelist/page.js	69	343
apps/dashboard/src/lib/apiClient.js	241	344
apps/dashboard/src/lib/dashboardApi.js	612	348
apps/dashboard/src/lib/idb.js	58	357
apps/dashboard/src/lib/useFormAutosave.js	109	358
apps/dashboard/src/store/dashboardStore.js	124	360
apps/tag_series/api/api.js	135	362
apps/tag_series/app/(auth)/login/page.js	15	364
apps/tag_series/app/Admin/AddFormFactor/page.js	55	365
apps/tag_series/app/Admin/AddProduct/page.js	43	366
apps/tag_series/app/Admin/View/page.js	236	367
apps/tag_series/app/generate/page.js	1063	371
apps/tag_series/app/globals.css	69	387
apps/tag_series/app/layout.js	24	388
apps/tag_series/app/page.js	459	389
apps/tag_series/components/AnalyticsList.js	19	396
apps/tag_series/components/PostHogInit.js	12	397
apps/tag_series/lib/devices.js	31	398
apps/tag_series/lib/setupStorage.js	23	399
apps/tag_series/next.config.mjs	29	400
apps/tag_series/package.json	25	401
apps/tag_series/postcss.config.mjs	7	402
apps/tag_series/utils/thresholds.js	8	403
packages/api/package.json	7	404
packages/api/src/createApiClient.js	46	405
packages/api/src/index.js	2	406
packages/api/src/routes.js	8	407
packages/auth/package.json	7	408
packages/auth/src/config.js	20	409
packages/auth/src/createAuthClient.js	30	410
packages/auth/src/index.js	2	411
packages/global-components/loaders/AtomXLoader.js	28	412
packages/global-components/loaders/index.js	1	413
packages/global-components/package.json	13	414
packages/global-components/src/AppShell.js	34	415

Table of Contents

FILE	LINES	PAGE
packages/global-components/src/AuthPanel.js	15	416
packages/global-components/src/GlobalFooter.js	16	417
packages/global-components/src/TokenGate.js	50	418
packages/global-components/src/UniversalLoginPage.js	231	419
packages/global-components/src/index.js	7	423
packages/global-components/src/posthog.js	186	424
packages/lib/package.json	7	427
packages/lib/src/environment.js	7	428
packages/lib/src/index.js	4	429
packages/lib/src/projects.js	19	430
packages/lib/src/token.js	30	431
packages/public-assets/package.json	7	432
packages/public-assets/src/index.js	65	433
packages/shared-ui/package.json	7	434
packages/shared-ui/src/Button.js	26	435
packages/shared-ui/src/Card.js	15	436
packages/shared-ui/src/SectionHeading.js	19	437
packages/shared-ui/src/index.js	3	438
packages/utils/package.json	7	439
packages/utils/src/featureFlags.js	13	440
packages/utils/src/formatters.js	9	441
packages/utils/src/index.js	2	442
scripts/export-all.js	95	443
scripts/sync-public-assets.js	46	445

FILE: jsconfig.json

```
1 {
2   "compilerOptions": {
3     "baseUrl": ".",
4     "ignoreDeprecations": "6.0",
5     "paths": {
6       "@atomx/shared-ui": ["packages/shared-ui/src"],
7       "@atomx/shared-ui/*": ["packages/shared-ui/src/*"],
8       "@atomx/global-components": ["packages/global-components/src"],
9       "@atomx/global-components/*": ["packages/global-components/src/*"],
10      "@atomx/auth": ["packages/auth/src"],
11      "@atomx/auth/*": ["packages/auth/src/*"],
12      "@atomx/api-client": ["packages/api/src"],
13      "@atomx/api-client/*": ["packages/api/src/*"],
14      "@atomx/lib": ["packages/lib/src"],
15      "@atomx/lib/*": ["packages/lib/src/*"],
16      "@atomx/utils": ["packages/utils/src"],
17      "@atomx/utils/*": ["packages/utils/src/*"],
18      "@tag_series/*": ["apps/tag_series/*"]
19    }
20  },
21  "exclude": ["node_modules", ".next", "out"]
22 }
```

FILE: turbo.json

```
1 {
2   "$schema": "https://turbo.build/schema.json",
3   "tasks": {
4     "dev": {
5       "cache": false
6     },
7     "build": {
8       "dependsOn": [
9         "^build"
10      ],
11      "outputs": [
12        ".next/**",
13        "out/**",
14        "dist/**"
15      ]
16    },
17    "lint": {
18      "outputs": []
19    }
20  }
21 }
```

FILE: package.json

```
1 {
2   "name": "atomx_dashboard_v3",
3   "version": "0.1.0",
4   "private": true,
5   "workspaces": [
6     "apps/*",
7     "packages/*"
8   ],
9   "scripts": {
10    "dev": "turbo dev",
11    "dev:tag_series": "turbo dev --filter=@atomx/tag_series",
12    "dev:access": "turbo dev --filter=@atomx/access-portal",
13    "dev:dashboard": "turbo dev --filter=dashboard",
14    "build": "turbo build",
15    "build:out": "node scripts/export-all.js",
16    "build:tag_series": "turbo build --filter=@atomx/tag_series",
17    "build:access": "turbo build --filter=@atomx/access-portal",
18    "build:dashboard": "turbo build --filter=dashboard",
19    "lint": "turbo lint",
20    "sync:public": "node scripts/sync-public-assets.js",
21    "postinstall": "npm run sync:public || true"
22  },
23  "packageManager": "npm@10.9.0",
24  "devDependencies": {
25    "babel-plugin-react-compiler": "1.0.0",
26    "baseline-browser-mapping": "^2.10.0",
27    "dotenv": "^16.4.5",
28    "turbo": "^2.6.0"
29  },
30  "overrides": {
31    "react": "19.2.0",
32    "react-dom": "19.2.0",
33    "next": {
34      "react": "19.2.0",
35      "react-dom": "19.2.0"
36    }
37  }
38 }
```


FILE: apps/access_portal/components/Add_Role/Add_Role.js

```

1  "use client";
2
3  import { useEffect, useMemo, useState } from "react";
4
5  const ORANGE = "#e65a24";
6  const ORANGE_DARK = "#d94c14";
7
8  export function Add_Role({ labels = ["Admin", "Operator"], onClick }) {
9    const items = useMemo(() => {
10      const list = Array.isArray(labels) ? labels.filter(Boolean) : [];
11      return list.length ? list : ["Admin"];
12    }, [labels]);
13    const hasLoop = items.length > 1;
14    const [current, setCurrent] = useState(0);
15    const [next, setNext] = useState(() => (items.length > 1 ? 1 : 0));
16    const [sliding, setSliding] = useState(false);
17
18    useEffect(() => {
19      // reset indices when labels change
20      setCurrent(0);
21      setNext(items.length > 1 ? 1 : 0);
22    }, [items]);
23
24    useEffect(() => {
25      if (!hasLoop) return undefined;
26      const id = setInterval(() => setSliding(true), 3000);
27      return () => clearInterval(id);
28    }, [hasLoop]);
29
30    useEffect(() => {
31      if (!sliding) return undefined;
32      const timeout = setTimeout(() => {
33        setCurrent((prev) => (prev + 1) % items.length);
34        setNext((prev) => (prev + 1) % items.length);
35        setSliding(false);
36      }, 450);
37      return () => clearTimeout(timeout);
38    }, [sliding, items.length]);
39
40    return (
41      <button
42        type="button"
43        onClick={onClick}
44        className="group inline-flex items-center justify-center gap-2 rounded-full border border-white/60
bg-[radial-gradient(circle_at_25%_30%,rgba(255,255,255,0.18),transparent_55%),linear-gradient(135deg,#ee7
237,#e04420)] px-3.5 py-2 text-white shadow-[0_12px_24px_rgba(224,68,32,0.22)] transition hover:translate
-y-[-1px] hover:brightness-105 active:translate-y-[0px]"
45        style={{
46          backgroundColor: ORANGE,
47          borderColor: ORANGE_DARK
48        }}
49      >
50        <span className="flex h-8 w-8 items-center justify-center text-base font-bold text-white">
51          <svg
52            aria-hidden="true"
53            focusable="false"
54            className="h-5 w-5"
55            viewBox="0 0 24 24"
56            fill="none"
57            stroke="currentColor"
58            strokeWidth="3"
59            strokeLinecap="round"
60            strokeLinejoin="round"
61          >
62            <path d="M12 5v14M5 12h14" />
63          </svg>
64        </span>
65        <span
66          className="relative h-5 overflow-hidden text-sm font-semibold leading-none tracking-wide text-whi

```

```
te"
67     style={{ minWidth: "86px" }}
68     aria-live="polite"
69   >
70     <span
71       className="absolute inset-0 flex items-center justify-start transition-transform duration-500 ease-out will-change-transform"
72       style={{
73         transform: sliding ? "translateY(100%)" : "translateY(0%)"
74       }}
75     >
76       {items[current]}
77     </span>
78     {hasLoop && (
79       <span
80         className="absolute inset-0 flex items-center justify-start transition-transform duration-500 ease-out will-change-transform"
81         style={{
82           transform: sliding ? "translateY(0%)" : "translateY(-100%)"
83         }}
84       >
85         {items[next]}
86       </span>
87     )}
88   </span>
89 </button>
90 );
91 }
```

FILE: apps/access_portal/components/AssignmentCard/AssignmentCard.js

```

1 export function AssignmentCard({ assignment, onPermissionClick }) {
2   return (
3     <section className="overflow-hidden rounded-2xl bg-white shadow-[0_10px_32px_rgba(15,23,42,0.08)]">
4       <div className="flex items-center justify-between bg-[#f88c43] px-5 py-3 text-white">
5         <div>
6           <p className="m-0 text-base font-bold leading-tight">Current Assignment</p>
7           <p className="m-0 pt-1 text-sm opacity-90">Your active event assignment details</p>
8         </div>
9         <span className="rounded-full border border-white/40 bg-white/15 px-3 py-1 text-sm font-semibold
leading-none">
10           Live
11         </span>
12       </div>
13
14       <div className="flex flex-col gap-4 px-5 py-4">
15         <div className="flex flex-wrap gap-5 sm:gap-8">
16           <div>
17             <p className="m-0 text-xs font-semibold uppercase tracking-[0.16em] text-slate-500">
18               Event
19             </p>
20             <p className="mt-1 text-lg font-semibold">{assignment.event}</p>
21           </div>
22           <div>
23             <p className="m-0 text-xs font-semibold uppercase tracking-[0.16em] text-slate-500">
24               Email
25             </p>
26             <p className="mt-1 text-lg font-semibold">{assignment.email}</p>
27           </div>
28         </div>
29
30         <div className="flex flex-col gap-2">
31           <p className="m-0 text-xs font-semibold uppercase tracking-[0.16em] text-slate-500">
32             Assigned Period
33           </p>
34           <div className="flex items-center gap-3 rounded-[14px] bg-[#e6f4f7] px-4 py-3 font-semibold tex
t-[#0f9ca3]">
35             <CalendarIcon />
36             <span>{assignment.period}</span>
37           </div>
38         </div>
39
40         <div className="flex flex-col gap-2">
41           <p className="m-0 text-xs font-semibold uppercase tracking-[0.16em] text-slate-500">
42             Access Permissions
43           </p>
44           <div className="flex flex-wrap gap-2">
45             {assignment.permissions.map((permission) => {
46               const label = typeof permission === "string" ? permission : permission.label;
47               return (
48                 <button
49                   type="button"
50                   key={label}
51                   className="rounded-full bg-[#0f9ca3] px-3.5 py-2 text-sm font-bold text-white shadow-[0
_8px_20px_rgba(15,156,163,0.18)] transition hover:-translate-y-[1px] hover:shadow-[0_12px_28px_rgba(15,15
6,163,0.25)] hover:brightness-105"
52                   onClick={() => onPermissionClick?.(permission)}
53                 >
54                   {label}
55                 </button>
56               );
57             })}
58           </div>
59         </div>
60       </div>
61     </section>
62   );
63 }
64
65 function CalendarIcon() {

```

```
66   return (
67     <svg
68       width="20"
69       height="20"
70       viewBox="0 0 24 24"
71       aria-hidden
72       className="text-[#0f9ca3]"
73     >
74       <path
75         d="M7 4V2m10 2V2m-9 8h8m-9 12h10a3 3 0 0 0 3-3V7a3 3 0 0 0 0-3-3H8a3 3 0 0 0 0-3 3v12a3 3 0 0 0 3 3Z"
76         fill="none"
77         stroke="currentColor"
78         strokeWidth="1.6"
79         strokeLinecap="round"
80         strokeLinejoin="round"
81       />
82     </svg>
83   );
84 }
```

FILE: apps/access_portal/components/HeaderBar/HeaderBar.js

```

1 import { useEffect, useState } from "react";
2 import Image from "next/image";
3 import { UserMenu } from "../UserMenu/UserMenu";
4
5 const LOGO_SRC = "/shared/logos/AtomX_Logo.svg";
6 const THEME_KEY = "atomx.theme";
7
8 function useTheme() {
9   const [theme, setTheme] = useState("light");
10
11   useEffect(() => {
12     if (typeof document === "undefined") return;
13     const current =
14       document.documentElement.getAttribute("data-atx") === "dark"
15         ? "dark"
16         : "light";
17     setTheme(current);
18   }, []);
19
20   const toggleTheme = () => {
21     setTheme((prev) => {
22       const next = prev === "dark" ? "light" : "dark";
23       if (typeof document !== "undefined") {
24         if (next === "dark") {
25           document.documentElement.setAttribute("data-atx", "dark");
26         } else {
27           document.documentElement.removeAttribute("data-atx");
28         }
29       }
30       try {
31         window.localStorage.setItem(THEME_KEY, next);
32       } catch (err) {
33         /* ignore storage failures */
34       }
35       return next;
36     });
37   };
38
39   return { theme, toggleTheme };
40 }
41
42 export function HeaderBar({ user, onSignOut, pageTitle = "Workspace" }) {
43   const { theme, toggleTheme } = useTheme();
44   const isDark = theme === "dark";
45
46   return (
47     <div
48       style={{
49         "--header-h": "58px",
50         "--crumb-h": "0px",
51         "--header-total-h": "var(--header-h)"
52       }}
53     >
54     <div className="fixed left-0 right-0 top-0 z-40">
55       <header
56         className="w-full border-b border-(--line) bg-(--surface) text-(--text) shadow-(--shadow)"
57         style={{ height: "var(--header-h)" }}
58       >
59       <div className="flex h-full items-center gap-3 px-4 md:px-5">
60         <div className="flex min-w-0 items-center gap-3">
61           <span className="relative block h-[40px] w-[100px] shrink-0 overflow-hidden">
62             <Image
63               src={LOGO_SRC}
64               alt="AtomX logo"
65               width={150}
66               height={150}
67               priority
68               className="absolute -left-[28px] -top-[52px] h-[132px] w-[150px] max-w-none"
69             />

```

```

70         </span>
71         <div className="hidden h-9 w-px bg-(--line) sm:block" aria-hidden />
72         <div className="font-chillax flex min-w-0 items-center gap-2 text-[1.3rem] font-semibold le
ading-none sm:text-[1.45rem]">
73             <span className="hidden truncate text-(--text) sm:inline">Portal</span>
74             <span className="hidden text-(--faint) sm:inline"></span>
75             <span className="truncate text-(--orange)">{pageTitle}</span>
76         </div>
77     </div>
78     <div className="flex-1" />
79     <div className="flex items-center gap-2.5">
80         <button
81             type="button"
82             onClick={toggleTheme}
83             title={isDark ? "Switch to light mode" : "Switch to dark mode"}
84             aria-label={isDark ? "Switch to light mode" : "Switch to dark mode"}
85             aria-pressed={isDark}
86             className="font-vcr flex h-[34px] w-[34px] items-center justify-center rounded-[10px] bor
der border-(--line) bg-(--surface) text-[11px] tracking-[0.06em] text-(--muted) transition hover:border-(
--orange) hover:text-(--text)"
87             >
88                 {isDark ? "LT" : "DK"}
89             </button>
90             <UserMenu user={user} onSignOut={onSignOut} />
91         </div>
92     </div>
93 </header>
94 </div>
95 <div style={{ height: "var(--header-total-h)" }} />
96 </div>
97 );
98 }

```

FILE: apps/access_portal/components/HeaderBar/HeaderBar.module.css

```
1 .header {
2   display: flex;
3   align-items: center;
4   justify-content: space-between;
5   padding: 0.8rem 1.2rem;
6   border-radius: 16px;
7   background: var(--surface);
8   box-shadow: 0 6px 24px rgba(15, 23, 42, 0.07);
9   gap: 1.1rem;
10 }
11
12 .logoStack {
13   display: flex;
14   align-items: center;
15   gap: 0.75rem;
16 }
17
18 .logoStack img {
19   height: 52px;
20   width: auto;
21 }
22
23 .meta {
24   display: flex;
25   align-items: center;
26   gap: 0.75rem;
27   padding-left: 0.75rem;
28   border-left: 1px solid #dcdfe3;
29   font-size: 1.9rem;
30   font-weight: 800;
31   letter-spacing: 0.02em;
32   color: #e35f1a;
33 }
34
35 @media (max-width: 768px) {
36   .header {
37     flex-direction: column;
38     align-items: flex-start;
39     padding: 0.9rem 1rem;
40   }
41 }
```

FILE: apps/access_portal/components/LoginScreen/LoginScreen.js

```

1  "use client";
2
3  import { useCallback, useEffect, useMemo, useState } from "react";
4  import { useRouter } from "next/router";
5  import { decodeJwt } from "@atomx/lib";
6  import {
7    capturePostHogEvent,
8    getAnalyticsConsent,
9    identifyPostHogUser,
10   initPostHog,
11   setAnalyticsConsent
12 } from "@atomx/global-components";
13
14 const DEFAULT_AUTH_URL = `${
15   (process.env.NEXT_PUBLIC_BASE_URL ?? "https://dapi.atomx.in/").replace(/\/$/, "")
16 }/auth/google/start`;
17
18 function buildAuthUrl(authUrl, appId, redirect) {
19   try {
20     const url = new URL(authUrl);
21     if (redirect) url.searchParams.set("redirect", redirect);
22     if (appId) url.searchParams.set("app", appId);
23     return url.toString();
24   } catch {
25     return authUrl;
26   }
27 }
28
29 function IconGoogle(props) {
30   return (
31     <svg viewBox="0 0 24 24" aria-hidden {...props}>
32       <path
33         fill="#4285F4"
34         d="M23.49 12.27c0-.79-.07-1.54-.2-2.27H12v4.3h6.45a5.52 5.52 0 01-2.39 3.62v2.99h3.86c2.26-2.08 3.57-5.14 3.57-8.64z"
35       />
36       <path
37         fill="#34A853"
38         d="M12 24c3.24 0 5.96-1.07 7.95-2.91-3.86-2.99c-1.07-7.72-2.44 1.15-4.09 1.15-3.14 0-5.8-2.12-6.75-4.98H1.26v3.11A12 12 0 0012 24z"
39       />
40       <path
41         fill="#FBBC05"
42         d="M5.25 14.27a7.22 7.22 0 010-4.54V6.62H1.26a12 12 0 000 10.76l3.99-3.11z"
43       />
44       <path
45         fill="#EA4335"
46         d="M12 4.74c1.76 0 3.34.61 4.58 1.81l3.44-3.44C17.95 1.19 15.24 0 12 0 7.31 0 3.23 2.69 1.26 6.62l3.99 3.11c.95-2.86 3.61-4.99 6.75-4.99z"
47       />
48     </svg>
49   );
50 }
51
52 export default function LoginScreen({
53   appId = "portal",
54   redirectPath = "/access",
55   authUrl = DEFAULT_AUTH_URL
56 }) {
57   const router = useRouter();
58   const [status, setStatus] = useState("idle");
59   const [error, setError] = useState(null);
60   const [profile, setProfile] = useState(null);
61   const [loginUrl, setLoginUrl] = useState(authUrl);
62   const [devToken, setDevToken] = useState(null);
63   const [activeSlide, setActiveSlide] = useState(0);
64   const [consentOpen, setConsentOpen] = useState(false);
65   const sliderItems = useMemo(
66     () => [

```



```

67     {
68       src: "/images/1.avif",
69       title: "CashlessX",
70       description: "Enable fast, secure NFC payments for high-volume event counters."
71     },
72     {
73       src: "/images/2.avif",
74       title: "AccessX",
75       description: "Control gates and scan entries in real time with reliable validation."
76     },
77     {
78       src: "/images/3.avif",
79       title: "InventoryX",
80       description: "Track stock movement live and keep every stall inventory synchronized."
81     }
82   ],
83   []
84 );
85
86 useEffect(() => {
87   if (typeof window === "undefined") return;
88   let redirectTarget = redirectPath || "/";
89   if (!/^https?:\/\//i.test(redirectTarget)) {
90     try {
91       redirectTarget = new URL(redirectTarget, window.location.origin).toString();
92     } catch {
93       redirectTarget = window.location.origin;
94     }
95   }
96   setLoginUrl(buildAuthUrl(authUrl, appId, redirectTarget));
97 }, [appId, authUrl, redirectPath]);
98
99 useEffect(() => {
100   if (typeof window === "undefined") return;
101   const consent = getAnalyticsConsent();
102   if (consent === "granted") {
103     setConsentOpen(false);
104     initPostHog("access_portal");
105     return;
106   }
107   if (consent === "denied") {
108     setConsentOpen(false);
109     return;
110   }
111   setConsentOpen(true);
112 }, []);
113
114 const processToken = useCallback(
115   (token, delay = 1200) => {
116     if (!token) return;
117     try {
118       const decoded = decodeJwt(token);
119       const userEmail = decoded?.email || null;
120       const userId = decoded?.sub ?? decoded?.id ?? userEmail;
121       identifyPostHogUser(userId, {
122         email: userEmail,
123         name: decoded?.name ?? null,
124         role_type: decoded?.type ?? null,
125         app: "access_portal"
126       });
127       capturePostHogEvent("login_success", {
128         app: "access_portal",
129         app_id: appId,
130         user_email: userEmail,
131         role_type: decoded?.type ?? null,
132         has_roles: Array.isArray(decoded?.roles) && decoded?.roles.length > 0
133       });
134       setProfile(decoded);
135       setStatus("success");
136       if (typeof window !== "undefined") {

```

```

137 window.localStorage.setItem(`atomx.auth.${appId}`, token);
138 if (appId === "portal") {
139   window.localStorage.setItem("atomx.portal.token", token);
140 }
141
142 const url = new URL(window.location.href);
143 url.searchParams.delete("token");
144 window.history.replaceState(
145   window.history.state,
146   "",
147   `${url.pathname}${url.search}${url.hash}`
148 );
149
150 window.setTimeout(() => {
151   if (/^https?:\/\//i.test(redirectPath)) {
152     window.location.assign(redirectPath);
153   } else {
154     router.push(redirectPath || "/");
155   }
156 }, delay);
157 }
158 } catch (err) {
159   console.error(err);
160   capturePostHogEvent("login_failure", {
161     app: "access_portal",
162     app_id: appId,
163     stage: "token_decode",
164     error_message: err?.message || "token_decode_failed"
165   });
166   setError("We could not verify your login token. Please try again.");
167   setStatus("error");
168 }
169 },
170 [appId, redirectPath, router]
171 );
172
173 useEffect(() => {
174   if (
175     typeof window === "undefined" ||
176     process.env.NODE_ENV !== "development" ||
177     status !== "idle"
178   ) {
179     return;
180   }
181
182   setDevToken(process.env.NEXT_PUBLIC_DEV_PORTAL_TOKEN || null);
183 }, [status]);
184
185 useEffect(() => {
186   if (!sliderItems.length) return;
187   const timer = window.setInterval(() => {
188     setActiveSlide((prev) => (prev + 1) % sliderItems.length);
189   }, 4300);
190   return () => window.clearInterval(timer);
191 }, [sliderItems]);
192
193 useEffect(() => {
194   if (!router.isReady) return;
195   const raw = router.query.token;
196   const fromQuery = Array.isArray(raw) ? raw[0] : raw;
197   const tokenFromUrl =
198     fromQuery ||
199     (typeof window !== "undefined"
200       ? new URLSearchParams(window.location.search).get("token")
201       : null);
202   if (tokenFromUrl) {
203     processToken(tokenFromUrl);
204   }
205 }, [router.isReady, router.query.token, processToken]);
206

```

```

207 const showDevButton =
208   process.env.NODE_ENV === "development" &&
209   process.env.NEXT_PUBLIC_DEV_TOKEN_BUTTON === "true" &&
210   devToken &&
211   status === "idle" &&
212   !profile;
213
214 const handleDevTokenLogin = () => {
215   if (!showDevButton) return;
216   processToken(devToken, 450);
217 };
218
219 const handleGoogleSignIn = useCallback(
220   (event) => {
221     event.preventDefault();
222     if (status === "loading" || status === "success") return;
223     setError(null);
224     setStatus("loading");
225     capturePostHogEvent("login_start", {
226       app: "access_portal",
227       app_id: appId,
228       provider: "google",
229       redirect_path: redirectPath
230     });
231     if (typeof window !== "undefined") {
232       window.setTimeout(() => {
233         window.location.assign(loginUrl);
234       }, 1200);
235     }
236   },
237   [loginUrl, status]
238 );
239
240 const helperText = useMemo(() => {
241   if (status === "success") return "Authenticated. Redirecting...";
242   if (status === "loading") return "Verifying account. Redirecting you to Google...";
243   if (status === "error") return error;
244   return "Access your event dashboard, manage cashless payments, and view real-time analytics.";
245 }, [status, error]);
246
247 const activeSlideItem = sliderItems[activeSlide] || sliderItems[0];
248
249 const handleAcceptAnalytics = () => {
250   setAnalyticsConsent(true, "access_portal");
251   setConsentOpen(false);
252 };
253
254 const handleRejectAnalytics = () => {
255   setAnalyticsConsent(false, "access_portal");
256   setConsentOpen(false);
257 };
258
259 return (
260   <main className="relative min-h-screen overflow-hidden bg-[#031026]">
261     {sliderItems.map((item, index) => (
262       <img
263         key={item.src}
264         src={item.src}
265         alt={`AtomX sign in visual ${index + 1}`}
266         className={`absolute inset-0 h-full w-full object-cover brightness-[1.04] contrast-[1.03] satur
267 ate-[1.06] transition-opacity duration-[1200ms] ease-out ${
268           index === activeSlide ? "opacity-100" : "opacity-0"
269         }}
270       />
271     ))}
272     <div className="absolute inset-0 bg-[linear-gradient(102deg,rgba(2,8,20,0.06)_0%,rgba(2,8,20,0.03)_
273 44%,rgba(2,8,20,0.2)_78%,rgba(2,8,20,0.28)_100%)]" />
274     <div className="absolute bottom-9 left-8 right-[42%] z-10 hidden text-white md:block lg:left-12">
275       <h2 className="max-w-[460px] text-[3.1rem] font-semibold leading-[1.05] drop-shadow-[0_6px_14px_r

```

```

275     gba(0,0,0,0.45))">
276       {activeSlideItem?.title}
277     </h2>
278     <p className="mt-5 max-w-[560px] text-xl leading-relaxed text-white/86">
279       {activeSlideItem?.description}
280     </p>
281   </div>
282
283   <section className="relative z-10 flex min-h-screen items-end justify-center px-5 py-8 md:items-cen
284     ter md:justify-end md:pl-10 md:pr-4 lg:pl-14 lg:pr-6">
285     <div className="w-full max-w-[460px]">
286       <div className="flex rounded-[8px] border border-white/28 bg-black/32 p-7 shadow-[0_26px_60px_r
287         gba(2,8,20,0.46)] backdrop-blur-[10px] md:min-h-[600px] md:px-9 md:py-10">
288         <div className="flex w-full flex-col">
289           
295           <div className="mt-2 h-px w-full max-w-[210px] bg-[linear-gradient(90deg,#F88C43_0%,#1495AB_5
296             2%,#FFFFFF_100%)]" />
297           <h1 className="mt-7 text-[2.25rem] font-semibold leading-[1.08] text-white md:text-[3rem]">
298             Sign in to{" "}
299             <span className="text-[#1495AB] drop-shadow-[0_2px_2px_rgba(15,23,42,0.45)]">
300               Portal
301             </span>
302           </h1>
303           <p className="mt-5 max-w-[640px] text-[1.08rem] leading-relaxed text-white/65 md:text-[1.22re
304             m]">
305             {helperText}
306           </p>
307           {status === "error" ? (
308             <div className="mt-6 rounded-2xl border border-red-300/40 bg-red-400/10 px-4 py-3 text-base
309               text-red-100">
310               {error}
311             </div>
312           ) : null}
313           {profile ? (
314             <div className="mt-7 rounded-2xl border border-white/25 bg-white/10 px-4 py-3 text-base tex
315               t-white/90">
316               Authenticated as <span className="font-semibold text-white">{profile.email}</span>. Redir
317               ecting...
318             </div>
319           ) : (
320             <a
321               href={loginUrl}
322               onClick={handleGoogleSignIn}
323               className={`mt-8 flex h-14 w-full items-center justify-center gap-3 rounded-[26px] bg-[#f
324                 88c43] px-7 text-[1.2rem] font-semibold text-white [text-shadow:0_1px_2px_rgba(0,0,0,0.28)] shadow-[0_18p
325                 x_38px_rgba(248,140,67,0.45)] transition focus-visible:outline-none focus-visible:ring-2 focus-visible:ri
326                 ng-[#f88c43]/45`}
327               status === "loading"
328               ? "pointer-events-none opacity-95"
329               : "hover:brightness-105"
330             >
331               <IconGoogle className="h-7 w-7" />
332               <span className="leading-none">
333                 {status === "loading" ? "Verifying..." : "Continue with Google"}
334               </span>
335             </a>
336           )}
337           {showDevButton ? (
338             <button

```

```

333         type="button"
334         onClick={handleDevTokenLogin}
335         className="mt-3 w-full rounded-full border border-dashed border-[#ffbb92] bg-[#fff3ea] px
-4 py-2 text-sm font-semibold text-[#d24f10]"
336     >
337         Use dev session token
338     </button>
339 ) : null}
340
341     <p className="mt-auto pt-9 text-[0.96rem] leading-relaxed text-white/66 md:text-[1rem]">
342         By continuing, you agree to AtomX&apos;s{" "}
343         <a href="/legal/terms" className="text-[#f88c43] underline underline-offset-2 hover:text-[#
ff9f5f]">
344             Terms
345         </a>{" "}
346         and{" "}
347         <a href="/legal/privacy" className="text-[#f88c43] underline underline-offset-2 hover:text-
[#ff9f5f]">
348             Privacy Policy
349         </a>
350         .
351     </p>
352     <button
353         type="button"
354         onClick={() => setConsentOpen(true)}
355         className="mt-3 self-start text-sm text-white/70 underline underline-offset-2 transition ho
ver:text-white"
356     >
357         Cookie settings
358     </button>
359 </div>
360 </div>
361 </div>
362 </section>
363
364     <div className="pointer-events-none absolute bottom-7 left-1/2 z-10 hidden -translate-x-1/2 items-c
enter gap-2 md:flex">
365         {sliderItems.map((_, index) => (
366             <span
367                 key={`slide-dot-${index}`}
368                 className={`h-2 rounded-full transition-all duration-300 ${
369                     index === activeSlide ? "w-7 bg-white" : "w-2 bg-white/35"
370                 }`}
371             />
372         ))}
373     </div>
374
375     {consentOpen ? (
376         <div className="fixed inset-x-4 bottom-4 z-30 mx-auto w-full max-w-[620px] rounded-2xl border bor
der-white/30 bg-black/55 p-5 text-white shadow-[0_24px_46px_rgba(2,8,20,0.55)] backdrop-blur-xl md:inset-
x-auto md:right-6 md:mx-0">
377             <h3 className="text-lg font-semibold tracking-tight">Cookie Preferences</h3>
378
379             <div className="mt-4 flex flex-wrap items-center gap-2">
380                 <button
381                     type="button"
382                     onClick={handleAcceptAnalytics}
383                     className="rounded-full bg-[#f88c43] px-4 py-2 text-sm font-semibold text-white shadow-[0_1
0px_20px_rgba(248,140,67,0.35)] hover:brightness-105"
384                 >
385                     Accept all cookies
386                 </button>
387                 <button
388                     type="button"
389                     onClick={handleRejectAnalytics}
390                     className="rounded-full border border-white/35 bg-transparent px-4 py-2 text-sm font-semibo
ld text-white/90 hover:bg-white/10"
391                 >
392                     Accept only essential cookies
393                 </button>

```

```
394         </div>
395     </div>
396     ) : null}
397 </main>
398 );
399 }
```

FILE: apps/access_portal/components/QuickActionCard/QuickActionCard.js

```
1 import styles from "./QuickActionCard.module.css";
2
3 export function QuickActionCard({ title, description, iconColor, onClick }) {
4   return (
5     <button type="button" className={styles.card} onClick={onClick}>
6       <div className={styles.icon} style={{ backgroundColor: iconColor }}>
7         <svg width="24" height="24" viewBox="0 0 24 24" aria-hidden>
8           <path
9             d="M12 12.75c1.795 0 3.25-1.455 3.25-3.25S13.795 6.25 12 6.25 8.75 7.705 8.75 9.5s1.455 3.25
10            3.25 3.25Zm0 0c-3.175 0-5.75 2.575-5.75 5.75h11.5c0-3.175-2.575-5.75-5.75-5.75Z"
11            stroke="white"
12            strokeWidth="1.5"
13            strokeLinecap="round"
14            strokeLinejoin="round"
15            fill="none"
16          />
17        </svg>
18      </div>
19      <p className={styles.title}>{title}</p>
20      <p className={styles.description}>{description}</p>
21    </div>
22  </button>
23 );
24 }
```

FILE: apps/access_portal/components/QuickActionCard/QuickActionCard.module.css

```
1 .card {
2   width: 100%;
3   border: 1px solid var(--border);
4   border-radius: 14px;
5   padding: 1rem 1.1rem;
6   display: flex;
7   align-items: center;
8   gap: 0.9rem;
9   background: var(--surface);
10  cursor: pointer;
11  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.04);
12  transition: transform 0.15s ease, box-shadow 0.15s ease;
13 }
14
15 .card:hover {
16   transform: translateY(-2px);
17   box-shadow: 0 14px 30px rgba(20, 149, 171, 0.12);
18 }
19
20 .icon {
21   width: 46px;
22   height: 46px;
23   border-radius: 12px;
24   display: grid;
25   place-items: center;
26 }
27
28 .title {
29   margin: 0;
30   font-weight: 700;
31   font-size: 1rem;
32 }
33
34 .description {
35   margin: 0.12rem 0 0;
36   color: var(--muted);
37   font-size: 0.88rem;
38 }
```


FILE: apps/access_portal/components/QuickActionsPanel/QuickActionsPanel.js

```
1 import styles from "../QuickActionsPanel.module.css";
2 import { QuickActionCard } from "../QuickActionCard/QuickActionCard";
3
4 export function QuickActionsPanel({ actions, onAction }) {
5   return (
6     <section className={styles.panel}>
7       <div className={styles.banner}>
8         <div>
9           <p className={styles.title}>Quick Actions</p>
10          <p className={styles.subtitle}>Access your assigned modules</p>
11        </div>
12      </div>
13      <div className={styles.grid}>
14        {actions.map((action) => (
15          <QuickActionCard
16            key={action.title}
17            title={action.title}
18            description={action.description}
19            iconColor={action.color}
20            onClick={() => onAction?.(action)}
21          </>
22        ))}
23      </div>
24    </section>
25  );
26 }
```

FILE: apps/access_portal/components/QuickActionsPanel/QuickActionsPanel.module.css

```
1 .panel {
2   margin-top: 1rem;
3   border-radius: 18px;
4   background: var(--surface);
5   box-shadow: 0 10px 32px rgba(15, 23, 42, 0.08);
6   overflow: hidden;
7 }
8
9 .banner {
10  background: var(--teal);
11  padding: 1rem 1.35rem;
12  color: white;
13 }
14
15 .title {
16  margin: 0;
17  font-weight: 700;
18  font-size: 1.1rem;
19 }
20
21 .subtitle {
22  margin: 0.25rem 0 0;
23  opacity: 0.85;
24 }
25
26 .grid {
27  padding: 1.2rem 1.35rem 1.6rem;
28  display: grid;
29  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
30  gap: 0.85rem;
31 }
```

FILE: apps/access_portal/components/Session/SessionGuard.js

```

1 import { useCallback, useEffect, useMemo, useRef, useState } from "react";
2 import { useRouter } from "next/router";
3 import { decodeJwt } from "@atomx/lib";
4
5 const WARNING_WINDOW_MS = 10 * 60 * 1000;
6 const REAUTH_CONTEXT_KEY = "atomx.portal.reauth";
7 const REAUTH_CONTEXT_TTL_MS = 24 * 60 * 60 * 1000;
8
9 function sanitizeReturnTo(value) {
10   try {
11     const parsed = new URL(value, window.location.origin);
12     parsed.searchParams.delete("returnTo");
13     return parsed.toString();
14   } catch (err) {
15     return value;
16   }
17 }
18
19 function getLoginUrl(returnTo) {
20   const baseUrl = new URL("/", window.location.origin);
21   baseUrl.searchParams.delete("returnTo");
22   if (returnTo) {
23     const sanitized = sanitizeReturnTo(returnTo);
24     const baseId = `${baseUrl.origin}${baseUrl.pathname}`;
25     if (sanitized && sanitized !== baseId) {
26       baseUrl.searchParams.set("returnTo", sanitized);
27     }
28   }
29   return baseUrl.toString();
30 }
31
32 function normalizeRoleType(value) {
33   return String(value || "")
34     .trim()
35     .toLowerCase()
36     .replace(/[_\s]+/g, "-");
37 }
38
39 function mapServiceParam(type) {
40   const normalized = normalizeRoleType(type);
41   if (normalized.includes("tag-series") || normalized.includes("tagseries")) {
42     return "tag-series";
43   }
44   if (normalized.includes("cashless")) return "cashless";
45   if (normalized.includes("inventory")) return "inventory";
46   if (normalized.includes("admin")) return "admin";
47   return normalized || null;
48 }
49
50 function coerceId(value) {
51   if (value === null || value === undefined || value === "") return null;
52   const num = Number(value);
53   return Number.isFinite(num) ? num : value;
54 }
55
56 function formatCountdown(ms) {
57   if (ms <= 0) return "0:00";
58   const totalSeconds = Math.floor(ms / 1000);
59   const minutes = Math.floor(totalSeconds / 60);
60   const seconds = String(totalSeconds % 60).padStart(2, "0");
61   return `${minutes}:${seconds}`;
62 }
63
64 function clearPortalAuthCache() {
65   if (typeof window === "undefined") return;
66   window.localStorage.removeItem("atomx.portal.token");
67   window.localStorage.removeItem("atomx.dashboard.token");
68   window.localStorage.removeItem("atomx.dashboard.store");
69   const keysToRemove = [];

```

```

70   for (let i = 0; i < window.localStorage.length; i += 1) {
71     const key = window.localStorage.key(i);
72     if (key && key.startsWith("atomx.auth.")) {
73       keysToRemove.push(key);
74     }
75   }
76   keysToRemove.forEach((key) => window.localStorage.removeItem(key));
77 }
78
79 export default function SessionGuard() {
80   const router = useRouter();
81   const isLoginRoute = router.pathname === "/";
82   const [token, setToken] = useState(null);
83   const [warningOpen, setWarningOpen] = useState(false);
84   const [expiredOpen, setExpiredOpen] = useState(false);
85   const [countdown, setCountdown] = useState("");
86   const timersRef = useRef({ warn: null, expire: null, tick: null });
87   const reloginRef = useRef(false);
88
89   useEffect(() => {
90     if (typeof window === "undefined") return;
91     const stored = window.localStorage.getItem("atomx.portal.token");
92     if (stored) setToken(stored);
93
94     const onStorage = (event) => {
95       if (event.key === "atomx.portal.token") {
96         setToken(event.newValue || null);
97         reloginRef.current = false;
98         setWarningOpen(false);
99         setExpiredOpen(false);
100      }
101    };
102    window.addEventListener("storage", onStorage);
103    return () => window.removeEventListener("storage", onStorage);
104  }, []);
105
106   const expiresAt = useMemo(() => {
107     if (!token) return null;
108     try {
109       const decoded = decodeJwt(token);
110       if (!decoded?.exp) return null;
111       return decoded.exp * 1000;
112     } catch (err) {
113       console.error("Failed to decode token", err);
114       return null;
115     }
116   }, [token]);
117
118   const startRelogin = useCallback(() => {
119     if (reloginRef.current) return;
120     const returnTo = sanitizeReturnTo(window.location.href);
121     let contextType = null;
122     let contextAdminId = null;
123     let contextEventId = null;
124     if (token) {
125       try {
126         const decoded = decodeJwt(token);
127         const roles = Array.isArray(decoded?.roles) ? decoded.roles : [];
128         const ctxType = decoded?.ctx?.type;
129         const normalizedCtxType = normalizeRoleType(ctxType);
130         const roleMatch = roles.find(
131           (role) => normalizeRoleType(role?.type) === normalizedCtxType
132         );
133         contextType = roleMatch?.type || ctxType || null;
134         contextAdminId = coerceId(decoded?.ctx?.adminId ?? roleMatch?.adminId);
135         contextEventId = coerceId(decoded?.ctx?.eventId ?? roleMatch?.eventId);
136       } catch (err) {
137         console.error("Failed to decode token for reauth context", err);
138       }
139     }

```

```
140     const payload = {
141       version: 1,
142       createdAt: Date.now(),
143       expiresAt: Date.now() + REAUTH_CONTEXT_TTL_MS,
144       returnTo,
145       type: contextType,
146       service: mapServiceParam(contextType),
147       adminId: contextAdminId,
148       eventId: contextEventId
149     };
150     try {
151       window.localStorage.setItem(REAUTH_CONTEXT_KEY, JSON.stringify(payload));
152     } catch (err) {
153       console.error("Failed to persist reauth context", err);
154     }
155     const loginUrl = getLoginUrl(returnTo);
156     reloginRef.current = true;
157     clearPortalAuthCache();
158     window.location.assign(loginUrl);
159   }, [token]);
160
161   const scheduleTimers = useCallback(() => {
162     if (!expiresAt || isLoginRoute) return;
163     const now = Date.now();
164     const warnAt = expiresAt - WARNING_WINDOW_MS;
165     const warnDelay = Math.max(warnAt - now, 0);
166     const expireDelay = Math.max(expiresAt - now, 0);
167
168     if (timersRef.current.warn) clearTimeout(timersRef.current.warn);
169     if (timersRef.current.expire) clearTimeout(timersRef.current.expire);
170     if (timersRef.current.tick) clearInterval(timersRef.current.tick);
171
172     timersRef.current.warn = setTimeout(() => {
173       setWarningOpen(true);
174     }, warnDelay);
175
176     timersRef.current.expire = setTimeout(() => {
177       setExpiredOpen(true);
178     }, expireDelay);
179
180     timersRef.current.tick = setInterval(() => {
181       const remaining = expiresAt - Date.now();
182       if (remaining <= 0) {
183         setCountdown("0:00");
184         return;
185       }
186       setCountdown(formatCountdown(remaining));
187     }, 1000);
188   }, [expiresAt, isLoginRoute]);
189
190   useEffect(() => {
191     if (!isLoginRoute) return;
192     setWarningOpen(false);
193     setExpiredOpen(false);
194     reloginRef.current = false;
195     if (token && expiresAt && expiresAt <= Date.now()) {
196       clearPortalAuthCache();
197       setToken(null);
198     }
199   }, [isLoginRoute, token, expiresAt]);
200
201   useEffect(() => {
202     if (!expiresAt) return;
203     scheduleTimers();
204     return () => {
205       if (timersRef.current.warn) clearTimeout(timersRef.current.warn);
206       if (timersRef.current.expire) clearTimeout(timersRef.current.expire);
207       if (timersRef.current.tick) clearInterval(timersRef.current.tick);
208     };
209   }, [expiresAt, scheduleTimers]);
```

```

210
211   if (!expiresAt || isLoginRoute) return null;
212
213   return (
214     <
215     {(warningOpen || expiredOpen) && (
216       <div className="fixed inset-0 z-[999] flex items-center justify-center bg-black/30 px-4">
217         <div className="w-full max-w-md rounded-xl border border-[#f2d9c8] bg-white p-6 shadow-[0_18px_
40px_rgba(15,23,42,0.25)]">
218           <div className="flex items-start justify-between gap-3">
219             <div>
220               <h2 className="text-base font-semibold text-slate-800">
221                 {expiredOpen ? "Session expired" : "Session expiring soon"}
222               </h2>
223               <p className="mt-2 text-sm text-slate-600">
224                 {expiredOpen
225                   ? "Please re-login to continue."
226                   : "Your session will expire in"}
227                 {!expiredOpen && countdown ? (
228                   <span className="ml-1 text-slate-800 font-semibold">{countdown}</span>
229                 ) : null}
230               </p>
231             </div>
232             {!expiredOpen ? (
233               <button
234                 type="button"
235                 onClick={() => setWarningOpen(false)}
236                 className="text-slate-400 hover:text-slate-600"
237                 aria-label="Close"
238               >
239                 <svg
240                   viewBox="0 0 24 24"
241                   className="h-4 w-4"
242                   fill="none"
243                   stroke="currentColor"
244                   strokeWidth="2"
245                   strokeLinecap="round"
246                   strokeLinejoin="round"
247                 >
248                   <path d="M18 6L6 18" />
249                   <path d="M6 6L12 12" />
250                 </svg>
251               </button>
252             ) : null}
253           </div>
254           <div className="mt-5 flex items-center justify-end">
255             <button
256               type="button"
257               onClick={startReLogin}
258               className="rounded-md bg-[#f88c43] px-4 py-2 text-sm font-semibold text-white shadow-[0_6
px_12px_rgba(248,140,67,0.25)]"
259             >
260               {expiredOpen ? "Login" : "Re-login"}
261             </button>
262           </div>
263         </div>
264       </div>
265     )}
266   </>
267 );
268 }

```

FILE: apps/access_portal/components/UserMenu/UserMenu.js

```

1  "use client";
2
3  import { useState } from "react";
4
5  function formatRole(value, long = false) {
6    const raw = String(value || "member").trim();
7    if (!raw) return long ? "Member" : "Member";
8    if (raw.toLowerCase() === "admin") return long ? "Administrator" : "Admin";
9    return raw
10     .replace(/[_-]+/g, " ")
11     .replace(/\b\w/g, (char) => char.toUpperCase());
12  }
13
14  export function UserMenu({ user, onSignOut = () => {} }) {
15    const [open, setOpen] = useState(false);
16    const avatarText = String(user.initials || user.name || "A").slice(0, 1).toUpperCase();
17
18    const handleSignOut = () => {
19      setOpen(false);
20      onSignOut();
21    };
22
23    return (
24      <div className="relative">
25        <button
26          type="button"
27          className="flex h-[42px] items-center gap-2 rounded-lg border border-(--line) bg-(--surface) px-2
28            .5 text-left shadow-(--shadow) transition hover:-translate-y-[1px] hover:border-(--orange)"
29          onClick={() => setOpen((prev) => !prev)}
30          aria-expanded={open}
31          >
32          <div className="hidden min-w-0 flex-col leading-tight sm:flex">
33            <span className="font-vcr max-w-[6.5rem] truncate text-[0.62rem] uppercase tracking-[0.12em] te
34              xt-(--muted)">
35              {formatRole(user.role)}
36            </span>
37            <span className="max-w-[7rem] truncate text-[0.86rem] font-semibold text-(--text)">
38              {user.name}
39            </span>
40          </div>
41          <div className="flex h-8 w-8 shrink-0 items-center justify-center overflow-hidden rounded-lg bg-[
42            linear-gradient(135deg,#e04420,#2f1ec7)] text-[0.85rem] font-semibold text-white">
43            {user.picture ? (
44              // eslint-disable-next-line @next/next/no-img-element
45              <img src={user.picture} alt={user.name} className="h-full w-full object-cover" />
46            ) : (
47              avatarText
48            )}
49          </div>
50          <svg
51            width="16"
52            height="16"
53            viewBox="0 0 24 24"
54            className={`text-(--muted) transition-transform ${open ? "rotate-180" : ""}`}
55            aria-hidden
56          >
57            <path
58              fill="none"
59              stroke="currentColor"
60              strokeWidth="2"
61              strokeLinecap="round"
62              strokeLinejoin="round"
63              d="M6 9l6 6 6-6"
64            />
65          </svg>
66        </button>
67        <div && (
68          <div className="absolute right-0 top-full z-50 mt-2 w-[236px] rounded-lg border border-(--line) b
69            g-(--surface) p-2.5 shadow-(--shadowUp)">

```

```

66         <div className="flex items-center gap-2.5 px-1 py-1">
67             <div className="flex h-9 w-9 shrink-0 items-center justify-center overflow-hidden rounded-full
        l bg-[linear-gradient(135deg,#e04420,#2f1ec7)] text-sm font-semibold text-white">
68                 {user.picture ? (
69                     // eslint-disable-next-line @next/next/no-img-element
70                     <img src={user.picture} alt={user.name} className="h-full w-full object-cover" />
71                 ) : (
72                     avatarText
73                 )}
74             </div>
75             <div className="min-w-0 leading-tight">
76                 <p className="m-0 truncate text-[0.9rem] font-semibold text-(--text)">
77                     {user.name}
78                 </p>
79                 <small className="text-[0.74rem] font-medium text-(--muted)">
80                     {formatRole(user.role, true)}
81                 </small>
82             </div>
83         </div>
84         <button
85             type="button"
86             className="mt-2 w-full rounded-md bg-(--text) px-3 py-2.5 text-left text-[0.84rem] font-semib
        old text-(--bg) transition hover:opacity-90"
87             onClick={handleSignOut}
88         >
89             Logout
90         </button>
91     </div>
92     )}
93 </div>
94 );
95 }

```


FILE: apps/access_portal/components/WelcomePanel/WelcomePanel.js

```
1 export function WelcomePanel({ user, actions = [] }) {
2   return (
3     <section className="flex flex-col gap-1">
4       <div>
5         <h1 className="font-chillax m-0 text-[30px] font-semibold leading-[1.06] tracking-[-0.02em] text-
6           (---text) md:text-[38px] lg:text-[44px]">
7           <span className="text-(---orange)">Welcome</span>, {user.name}.
8         </h1>
9         <p className="m-0 pt-2.5 text-[15px] font-light text-(---muted)">
10           Choose where you want to continue.
11         </p>
12       </div>
13       {actions.length > 0 && (
14         <div className="flex flex-wrap gap-2">
15           {actions.map((action) => (
16             <span
17               key={action.label}
18               className={`rounded-full px-3 py-1 text-sm font-semibold text-white shadow-sm ${
19                 action.variant === "orange" ? "bg-[#f88c43]" : "bg-[#1495ab]"
20               }`}
21               >
22                 {action.label}
23             </span>
24           ))}
25         </div>
26       )}
27     </section>
28   );
29 }
```

FILE: apps/access_portal/next.config.mjs

```
1 import path from "node:path";
2 import dotenv from "dotenv";
3
4 dotenv.config({ path: path.resolve(process.cwd(), "..", "..", ".env") });
5
6 /** @type {import('next').NextConfig} */
7 const config = {
8   reactCompiler: true,
9   output: "export",
10  images: {
11    unoptimized: true
12  },
13  transpilePackages: [
14    "@atomx/shared-ui",
15    "@atomx/global-components",
16    "@atomx/lib"
17  ],
18  experimental: {
19    externalDir: true
20  }
21 };
22
23 export default config;
```

FILE: apps/access_portal/package.json

```
1 {
2   "name": "@atomx/access-portal",
3   "private": true,
4   "scripts": {
5     "dev": "next dev -p 3003",
6     "build": "next build",
7     "start": "next start"
8   },
9   "dependencies": {
10    "@atomx/global-components": "0.1.0",
11    "@atomx/lib": "0.1.0",
12    "next": "16.2.6",
13    "react": "19.2.0",
14    "react-dom": "19.2.0"
15  },
16  "devDependencies": {
17    "@tailwindcss/postcss": "^4",
18    "tailwindcss": "^4"
19  }
20 }
```

FILE: apps/access_portal/pages/_app.js

```
1 import "../styles/globals.css";
2 import { useEffect } from "react";
3 import { initPostHog } from "@atomx/global-components";
4 import SessionGuard from "../components/Session/SessionGuard";
5
6 export default function App({ Component, pageProps }) {
7   useEffect(() => {
8     initPostHog("access_portal");
9   }, []);
10
11   return (
12     <>
13       <Component {...pageProps} />
14       <SessionGuard />
15     </>
16   );
17 }
```

FILE: apps/access_portal/pages/_document.js

```
1 import { Html, Head, Main, NextScript } from "next/document";
2
3 // Applies the saved theme before first paint to avoid a light/dark flash.
4 const THEME_BOOT = `
5 try {
6   var t = localStorage.getItem('atomx.theme');
7   if (t === 'dark') document.documentElement.setAttribute('data-atx', 'dark');
8 } catch (e) {}
9 `;
10
11 export default function Document() {
12   return (
13     <Html lang="en">
14       <Head />
15       <body>
16         <script dangerouslySetInnerHTML={{ __html: THEME_BOOT }} />
17         <Main />
18         <NextScript />
19       </body>
20     </Html>
21   );
22 }
```

FILE: apps/access_portal/pages/access.js

```

1 import { useEffect, useMemo, useRef, useState } from "react";
2 import Head from "next/head";
3 import { useRouter } from "next/router";
4 import { decodeJwt, getBaseUrl, getInitials } from "@atomx/lib";
5 import { AtomXLoader, capturePostHogEvent } from "@atomx/global-components";
6 import { HeaderBar } from "../components/HeaderBar/HeaderBar";
7 import { WelcomePanel } from "../components/WelcomePanel/WelcomePanel";
8
9 const moduleLinks = {
10   livelink: process.env.NEXT_PUBLIC_LIVELINK_URL ?? "/livelink",
11   "tag-series": process.env.NEXT_PUBLIC_TAG_SERIES_URL ?? "/tag_series"
12 };
13 const dashboardBase = (process.env.NEXT_PUBLIC_DASHBOARD_URL ?? "").replace(/\/$/, "");
14 const dashboardConfigPath = "/Config/";
15 const REAUTH_CONTEXT_KEY = "atomx.portal.reauth";
16 const REAUTH_CONTEXT_FALLBACK_TTL_MS = 24 * 60 * 60 * 1000;
17 const BOOTSTRAP_TOKEN_COOKIE = "atomx_bootstrap_token";
18 const DASHBOARD_SELECTED_TOKEN_KEY = "atomx.dashboard.token";
19 const DASHBOARD_API_KEY =
20   process.env.NEXT_PUBLIC_DASHBOARD_API_KEY ??
21   "pZebJlF_.dv3_prod.Iu7Zitu3X30C2R6-bVZtRXRu0DeiHY-j";
22
23 function ensureTrailingSlashForRoute(value) {
24   if (!value) return value;
25   try {
26     const isAbsolute = /^https?:\/\//i.test(value);
27     const parsed = isAbsolute ? new URL(value) : new URL(value, "http://atomx.local");
28     const pathname = parsed.pathname || "/";
29     const lastSegment = pathname.split("/").pop() || "";
30     const hasFileExtension = lastSegment.includes(".");
31     if (!hasFileExtension && !pathname.endsWith("/")) {
32       parsed.pathname = `${pathname}/`;
33     }
34     return isAbsolute
35       ? parsed.toString()
36       : `${parsed.pathname}${parsed.search}${parsed.hash}`;
37   } catch {
38     return value;
39   }
40 }
41
42 const accessAdminUrl = ensureTrailingSlashForRoute(
43   process.env.NEXT_PUBLIC_ACCESS_ADMIN_URL ?? `${dashboardBase}/admin`
44 );
45
46 const MODULE_CATALOG = {
47   "tag-series": {
48     title: "Tag Series",
49     description: "Provision NFC tags, manage telemetry, and orchestrate IoT fleets.",
50     color: "#1495ab",
51     variant: "teal",
52     href: moduleLinks["tag-series"]
53   },
54   livelink: {
55     title: "LiveLink",
56     description: "Route live content, moderate pipelines, and automate streaming.",
57     color: "#f88c43",
58     variant: "orange",
59     href: moduleLinks.livelink
60   }
61 };
62
63 const FALLBACK_MODULE = {
64   title: "Coming Soon",
65   description: "This module will be available shortly.",
66   color: "#d1d5db",
67   variant: "teal"
68 };
69

```

```

70 function formatSessionExpiry(expirySeconds) {
71   if (!expirySeconds) return null;
72   const expiryDate = new Date(expirySeconds * 1000);
73   if (Number.isNaN(expiryDate.getTime())) return null;
74
75   const now = new Date();
76   const startOfToday = new Date(now.getFullYear(), now.getMonth(), now.getDate());
77   const startOfTomorrow = new Date(now.getFullYear(), now.getMonth(), now.getDate() + 1);
78   const startOfYesterday = new Date(now.getFullYear(), now.getMonth(), now.getDate() - 1);
79
80   let prefix = "";
81   if (expiryDate >= startOfToday && expiryDate < startOfTomorrow) {
82     prefix = "Today";
83   } else if (expiryDate >= startOfTomorrow && expiryDate < new Date(startOfTomorrow.getTime() + 86400000))
84 ) {
85   prefix = "Tomorrow";
86 } else if (expiryDate >= startOfYesterday && expiryDate < startOfToday) {
87   prefix = "Yesterday";
88 }
89
90 const formatted = new Intl.DateTimeFormat("en-IN", {
91   dateStyle: "medium",
92   timeStyle: "short"
93 }).format(expiryDate);
94
95 return prefix ? `${prefix}, ${formatted}` : formatted;
96 }
97
98 function sanitizeReturnTo(value) {
99   try {
100     const parsed = new URL(value, window.location.origin);
101     parsed.searchParams.delete("returnTo");
102     parsed.searchParams.delete("token");
103     return parsed.toString();
104   } catch (err) {
105     return value;
106   }
107 }
108
109 function normalizeRoleType(value) {
110   return String(value || "")
111     .trim()
112     .toLowerCase()
113     .replace(/[_\s]+/g, "-");
114 }
115
116 function mapServiceParam(type) {
117   const normalized = normalizeRoleType(type);
118   if (normalized.includes("tag-series") || normalized.includes("tagseries")) {
119     return "tag-series";
120   }
121   if (normalized.includes("cashless")) return "cashless";
122   if (normalized.includes("inventory")) return "inventory";
123   return normalized;
124 }
125
126 function sanitizeModules(roles = []) {
127   const seen = new Set();
128   return roles.reduce((acc, role) => {
129     if (!role?.type || seen.has(role.type)) return acc;
130     seen.add(role.type);
131     const meta = MODULE_CATALOG[role.type] ?? {
132       ...FALLBACK_MODULE,
133       title: role.type,
134       typeLabel: role.type
135     };
136     acc.push({
137       ...meta,
138       type: role.type,
139       expiryAt: role.expiryAt ?? null,

```

```
139     adminId: role.adminId ?? null,
140     rawRole: role
141   });
142   return acc;
143 }, []);
144 }
145
146 function findRoleMatch(roles, type, eventId, adminId) {
147   if (!Array.isArray(roles) || !type) return null;
148   const typeNorm = normalizeRoleType(type);
149   const eventIdNorm =
150     eventId === null || eventId === undefined || eventId === ""
151       ? null
152       : String(eventId);
153   const adminIdNorm =
154     adminId === null || adminId === undefined || adminId === ""
155       ? null
156       : String(adminId);
157
158   const typedRoles = roles.filter((role) => normalizeRoleType(role?.type) === typeNorm);
159   if (!typedRoles.length) return null;
160
161   if (eventIdNorm) {
162     const byEvent = typedRoles.find((role) => String(role?.eventId) === eventIdNorm);
163     if (byEvent) return byEvent;
164   }
165
166   if (adminIdNorm) {
167     const byAdmin = typedRoles.find((role) => String(role?.adminId) === adminIdNorm);
168     if (byAdmin) return byAdmin;
169   }
170
171   return null;
172 }
173
174 function readReauthContext() {
175   if (typeof window === "undefined") return null;
176   let parsed = null;
177   try {
178     const raw = window.localStorage.getItem(REAUTH_CONTEXT_KEY);
179     if (!raw) return null;
180     parsed = JSON.parse(raw);
181   } catch (err) {
182     console.error("Failed to parse reauth context", err);
183     window.localStorage.removeItem(REAUTH_CONTEXT_KEY);
184     return null;
185   }
186
187   if (!parsed || typeof parsed !== "object") {
188     window.localStorage.removeItem(REAUTH_CONTEXT_KEY);
189     return null;
190   }
191
192   const now = Date.now();
193   const expiresAt = Number(parsed.expiresAt);
194   const createdAt = Number(parsed.createdAt);
195   const hasExpiry = Number.isFinite(expiresAt);
196   const hasCreatedAt = Number.isFinite(createdAt);
197   const isExpired = hasExpiry
198     ? expiresAt <= now
199     : hasCreatedAt
200       ? now - createdAt > REAUTH_CONTEXT_FALLBACK_TTL_MS
201       : false;
202
203   if (isExpired) {
204     window.localStorage.removeItem(REAUTH_CONTEXT_KEY);
205     return null;
206   }
207
208   return parsed;
```



```

209 }
210
211 function readCookie(name) {
212     if (typeof document === "undefined") return null;
213     const escaped = name.replace(/[\.*?^$()|[\]\$\s]/g, "\\$&");
214     const match = document.cookie.match(new RegExp(`(?:^|; )${escaped}=( [^;]*)`));
215     return match ? decodeURIComponent(match[1]) : null;
216 }
217
218 function setBootstrapTokenCookie(tokenValue) {
219     if (typeof window === "undefined" || !tokenValue) return;
220     const secure = window.location.protocol === "https:" ? "; Secure" : "";
221     document.cookie = `${BOOTSTRAP_TOKEN_COOKIE}=${encodeURIComponent(
222         tokenValue
223     )}; Path=/; Max-Age=1800; SameSite=Lax${secure}`;
224 }
225
226 function clearBootstrapTokenCookie() {
227     if (typeof document === "undefined") return;
228     document.cookie = `${BOOTSTRAP_TOKEN_COOKIE}=; Path=/; Max-Age=0; SameSite=Lax`;
229 }
230
231 function findTokenInResponse(value, seen = new Set(), allowRawString = true) {
232     if (!value) return null;
233     if (typeof value === "string") {
234         return allowRawString && value.split(".").length >= 3 ? value : null;
235     }
236     if (typeof value !== "object" || seen.has(value)) return null;
237     seen.add(value);
238
239     const tokenKeys = [
240         "token",
241         "accessToken",
242         "access_token",
243         "jwt",
244         "idToken",
245         "id_token",
246         "authToken",
247         "auth_token",
248         "bearerToken",
249         "bearer_token",
250         "selectedToken",
251         "selected_token",
252         "sessionToken",
253         "session_token"
254     ];
255     for (const key of tokenKeys) {
256         const token = value[key];
257         if (typeof token === "string" && token) return token;
258     }
259
260     for (const key of ["data", "result", "value"]) {
261         const token = value[key];
262         if (typeof token === "string" && token.split(".").length >= 3) return token;
263     }
264
265     for (const item of Object.values(value)) {
266         const token = findTokenInResponse(item, seen, false);
267         if (token) return token;
268     }
269     return null;
270 }
271
272 function persistSelectedAuthToken(token, { type, service }) {
273     if (typeof window === "undefined" || !token) return;
274     const keys = new Set([
275         DASHBOARD_SELECTED_TOKEN_KEY,
276         type ? `atomx.auth.${type}` : null,
277         type ? `atomx.auth.${normalizeRoleType(type)}` : null,
278         service ? `atomx.auth.${service}` : null

```

```

279 });
280 if (service === "tag-series") {
281     keys.add("atomx.auth.tag_series");
282 }
283 keys.forEach((key) => {
284     if (key) window.localStorage.setItem(key, token);
285 });
286 }
287
288 async function fetchSelectedEventDetails({ apiBase, eventId, token }) {
289     if (!apiBase || !eventId) {
290         throw new Error("Missing data to load event details.");
291     }
292
293     const res = await fetch(
294         `${apiBase}/v1/Events/Details/${encodeURIComponent(eventId)}`,
295         {
296             method: "GET",
297             headers: {
298                 ...(DASHBOARD_API_KEY ? { "x-api-key": DASHBOARD_API_KEY } : {}),
299                 ...(token ? { Authorization: `Bearer ${token}` } : {}),
300             },
301             credentials: "include",
302             cache: "no-store"
303         }
304     );
305
306     if (!res.ok) {
307         const text = await res.text().catch(() => "");
308         throw new Error(text || `Unable to load event details (${res.status})`);
309     }
310
311     const data = await res.json().catch(() => null);
312     return data?.event ?? data?.data?.event ?? null;
313 }
314
315 function formatSectionCount(count) {
316     return String(count).padStart(2, "0");
317 }
318
319 function formatChipLabel(type) {
320     const parts = String(type || "").
321         .split(/[-_\s]+/)
322         .filter(Boolean);
323     if (!parts.length) return "Access";
324     return parts
325         .map((part, index) =>
326             index === 0
327                 ? part.charAt(0).toUpperCase() + part.slice(1).toLowerCase()
328                 : part.toLowerCase()
329         )
330         .join(" · ");
331 }
332
333 // tile = icon chip fill, text = type label color, border = subtle gradient card border
334 const DEFAULT_ROLE_THEME = {
335     tile: "linear-gradient(140deg,#341CD6,#E04420)",
336     text: "#341CD6",
337     border: "linear-gradient(135deg,rgba(52,28,214,.55),rgba(224,68,32,.45))"
338 };
339
340 const ROLE_THEMES = {
341     admin: {
342         tile: "linear-gradient(140deg,#341CD6,#E04420)",
343         text: "#341CD6",
344         border: "linear-gradient(135deg,rgba(52,28,214,.6),rgba(224,68,32,.5) 55%,rgba(52,28,214,.12))"
345     },
346     cashless: {
347         tile: "#E04420",
348         text: "#E04420",

```

```

349     border: "linear-gradient(135deg, rgba(224,68,32,.6), rgba(224,68,32,.14))"
350   },
351   inventory: {
352     tile: "#00A9F2",
353     text: "#0284c7",
354     border: "linear-gradient(135deg, rgba(0,169,242,.6), rgba(0,169,242,.14))"
355   },
356   "tag-series": {
357     tile: "#8B5CF6",
358     text: "#8B5CF6",
359     border: "linear-gradient(135deg, rgba(139,92,246,.6), rgba(139,92,246,.14))"
360   },
361   "timeline-readonly": {
362     tile: "#341CD6",
363     text: "#341CD6",
364     border: "linear-gradient(135deg, rgba(52,28,214,.6), rgba(0,169,242,.18))"
365   },
366   "timeline-admin": {
367     tile: "#1C1C1C",
368     text: "#1C1C1C",
369     border: "linear-gradient(135deg, rgba(224,68,32,.45), rgba(52,28,214,.4))"
370   }
371 };
372
373 function getRoleTheme(type) {
374   return ROLE_THEMES[normalizeRoleType(type)] ?? DEFAULT_ROLE_THEME;
375 }
376
377 function RoleGlyph({ type }) {
378   const normalized = normalizeRoleType(type);
379   const iconClass = "h-3.5 w-3.5";
380
381   if (normalized === "admin") {
382     return (
383       <svg
384         viewBox="0 0 24 24"
385         className={iconClass}
386         fill="none"
387         stroke="currentColor"
388         strokeWidth="1.9"
389         strokeLinecap="round"
390         strokeLinejoin="round"
391         aria-hidden
392       >
393         <path d="M12 12a4 4 0 1 0 0-8 4 4 0 0 0 0 8Z" />
394         <path d="M4 21a8 8 0 0 1 16 0" />
395       </svg>
396     );
397   }
398
399   if (normalized.includes("cashless")) {
400     return (
401       <svg
402         viewBox="0 0 24 24"
403         className={iconClass}
404         fill="none"
405         stroke="currentColor"
406         strokeWidth="1.9"
407         strokeLinecap="round"
408         strokeLinejoin="round"
409         aria-hidden
410       >
411         <rect x="4" y="6" width="16" height="12" rx="2" />
412         <path d="M4 10h16" />
413         <path d="M8 15h4" />
414       </svg>
415     );
416   }
417
418   if (normalized.includes("inventory")) {

```

```

419     return (
420       <svg
421         viewBox="0 0 24 24"
422         className={iconClass}
423         fill="none"
424         stroke="currentColor"
425         strokeWidth="1.9"
426         strokeLinecap="round"
427         strokeLinejoin="round"
428         aria-hidden
429       >
430         <path d="m12 3 8 4.5v9L12 21l-8-4.5v-9L12 3Z" />
431         <path d="m4.5 8 7.5 4 7.5-4" />
432         <path d="M12 12v8.5" />
433       </svg>
434     );
435   }
436
437   if (normalized.includes("tag")) {
438     return (
439       <svg
440         viewBox="0 0 24 24"
441         className={iconClass}
442         fill="none"
443         stroke="currentColor"
444         strokeWidth="1.9"
445         strokeLinecap="round"
446         strokeLinejoin="round"
447         aria-hidden
448       >
449         <path d="M20 13 13 20 4 11V4h7l9 9Z" />
450         <path d="M7.5 7.5h.01" />
451       </svg>
452     );
453   }
454
455   if (normalized.includes("timeline")) {
456     return (
457       <svg
458         viewBox="0 0 24 24"
459         className={iconClass}
460         fill="none"
461         stroke="currentColor"
462         strokeWidth="1.9"
463         strokeLinecap="round"
464         strokeLinejoin="round"
465         aria-hidden
466       >
467         <circle cx="12" cy="12" r="8.5" />
468         <path d="M12 7.5V12l3 2" />
469       </svg>
470     );
471   }
472
473   return (
474     <svg
475       viewBox="0 0 24 24"
476       className={iconClass}
477       fill="none"
478       stroke="currentColor"
479       strokeWidth="1.9"
480       strokeLinecap="round"
481       strokeLinejoin="round"
482       aria-hidden
483     >
484       <rect x="5" y="5" width="14" height="14" rx="3" />
485       <path d="M9 9h6v6H9z" />
486     </svg>
487   );
488 }

```

```

489
490 function OpenArrowGlyph() {
491   return (
492     <svg
493       viewBox="0 0 24 24"
494       className="h-3.5 w-3.5"
495       fill="none"
496       stroke="currentColor"
497       strokeWidth="2"
498       strokeLinecap="round"
499       strokeLinejoin="round"
500       aria-hidden
501     >
502     <path d="M7 17 17 7" />
503     <path d="M9 7h8v8" />
504   </svg>
505 );
506 }
507
508 export default function AccessPage() {
509   const router = useRouter();
510   const [profile, setProfile] = useState(null);
511   const [modules, setModules] = useState([]);
512   const [status, setStatus] = useState("loading");
513   const [error, setError] = useState(null);
514   const [modalApp, setModalApp] = useState(null);
515   const [selecting, setSelecting] = useState(null);
516   const [selectError, setSelectError] = useState(null);
517   const [hideErrorBanner, setHideErrorBanner] = useState(false);
518   const [hideEmptyBanner, setHideEmptyBanner] = useState(false);
519   const [searchQuery, setSearchQuery] = useState("");
520   const [activeFilter, setActiveFilter] = useState("all");
521   const reauthHandledRef = useRef(false);
522
523   useEffect(() => {
524     if (!router.isReady) return;
525     const queryTokenRaw = router.query.token;
526     const queryToken = Array.isArray(queryTokenRaw) ? queryTokenRaw[0] : queryTokenRaw;
527     let tokenCandidate = queryToken;
528
529     if (!tokenCandidate && typeof window !== "undefined") {
530       const directToken = new URLSearchParams(window.location.search).get("token");
531       if (directToken) {
532         tokenCandidate = directToken;
533       }
534     }
535
536     if (!tokenCandidate && typeof window !== "undefined") {
537       tokenCandidate = window.localStorage.getItem("atomx.portal.token");
538     }
539
540     if (!tokenCandidate) {
541       setStatus("empty");
542       router.replace("/");
543       return;
544     }
545
546     try {
547       const decoded = decodeJwt(tokenCandidate);
548       const sanitized = sanitizeModules(decoded.roles);
549       setProfile(decoded);
550       setModules(sanitized);
551       setBootstrapTokenCookie(tokenCandidate);
552       if (typeof window !== "undefined") {
553         window.localStorage.setItem("atomx.portal.token", tokenCandidate);
554         sanitized.forEach((module) => {
555           window.localStorage.setItem(`atomx.auth.${module.type}`, tokenCandidate);
556         });
557       }
558       setStatus("ready");

```

```

559     if (tokenCandidate && typeof window !== "undefined") {
560         const url = new URL(window.location.href);
561         if (url.searchParams.has("token")) {
562             url.searchParams.delete("token");
563             router.replace(url.pathname, undefined, { shallow: true });
564         }
565     }
566 } catch (err) {
567     console.error(err);
568     setStatus("error");
569     setError("We could not verify your access token. Please sign in again.");
570 }
571 }, [router]);
572
573 useEffect(() => {
574     setHideErrorBanner(false);
575     setHideEmptyBanner(false);
576 }, [status, error]);
577
578 const user = useMemo(() => {
579     if (!profile) {
580         return {
581             name: "AtomX Operator",
582             role: "Awaiting Login",
583             email: "login@atomx.in",
584             initials: "A0",
585             picture: null
586         };
587     }
588     return {
589         name: profile.name ?? "AtomX Operator",
590         role: profile.type ?? "member",
591         email: profile.email ?? "unknown@atomx.in",
592         initials: getInitials(profile.name),
593         picture: profile.picture ?? profile.image ?? null
594     };
595 }, [profile]);
596
597 const roleCards = useMemo(() => {
598     if (!Array.isArray(profile?.roles)) return [];
599     return profile.roles.map((role, index) => ({
600         id: role?.id ?? `${role?.type ?? "role"}-${index}`,
601         type: role?.type ?? "Unknown",
602         expiryAt: role?.expiryAt ?? null,
603         eventName: role?.eventName ?? null,
604         eventId: role?.eventId ?? null,
605         adminName: role?.adminName ?? null,
606         adminId: role?.adminId ?? null,
607         appName: role?.appName ?? role?.moduleName ?? role?.applicationName ?? null,
608         appId: role?.appId ?? role?.moduleId ?? role?.applicationId ?? null
609     }));
610 }, [profile]);
611
612 const adminRoles = useMemo(() => {
613     return roleCards.filter(
614         (role) => String(role.type || "").toLowerCase() === "admin"
615     );
616 }, [roleCards]);
617
618 const eventRoles = useMemo(() => {
619     return roleCards.filter(
620         (role) =>
621             String(role.type || "").toLowerCase() !== "admin" &&
622             !role.appId &&
623             !role.appName
624     );
625 }, [roleCards]);
626
627 const appRoles = useMemo(() => {
628     return roleCards.filter(

```

```

629     (role) =>
630         String(role.type || "").toLowerCase() !== "admin" &&
631         (role.appId || role.appName)
632     );
633 }, [roleCards]);
634
635 // Everything that is not an admin workspace lives under "Events" in the design.
636 const eventSectionRoles = useMemo(
637     () => [...eventRoles, ...appRoles],
638     [eventRoles, appRoles]
639 );
640
641 const filters = useMemo(() => {
642     const counts = new Map();
643     const order = [];
644     roleCards.forEach((role) => {
645         const key = normalizeRoleType(role.type);
646         if (!key) return;
647         if (!counts.has(key)) {
648             counts.set(key, { key, label: formatChipLabel(role.type), count: 0 });
649             order.push(key);
650         }
651         counts.get(key).count += 1;
652     });
653     return [
654         { key: "all", label: "All access", count: roleCards.length },
655         ...order.map((key) => counts.get(key))
656     ];
657 }, [roleCards]);
658
659 // Keep the active filter valid when the available roles change.
660 useEffect(() => {
661     if (!filters.some((filter) => filter.key === activeFilter)) {
662         setActiveFilter("all");
663     }
664 }, [filters, activeFilter]);
665
666 const matchesFilters = useMemo(() => {
667     const query = searchQuery.trim().toLowerCase();
668     return (role) => {
669         if (activeFilter !== "all" && normalizeRoleType(role.type) !== activeFilter) {
670             return false;
671         }
672         if (!query) return true;
673         const haystack = [
674             role.type,
675             role.eventName,
676             role.adminName,
677             role.eventId,
678             role.adminId
679         ]
680             .filter((value) => value !== null && value !== undefined && value !== "")
681             .join(" ")
682             .toLowerCase();
683         return haystack.includes(query);
684     };
685 }, [activeFilter, searchQuery]);
686
687 const visibleAdminRoles = useMemo(
688     () => adminRoles.filter(matchesFilters),
689     [adminRoles, matchesFilters]
690 );
691 const visibleEventRoles = useMemo(
692     () => eventSectionRoles.filter(matchesFilters),
693     [eventSectionRoles, matchesFilters]
694 );
695 const hasVisibleRoles = visibleAdminRoles.length > 0 || visibleEventRoles.length > 0;
696
697 const highlightActions = useMemo(() => [], []);
698

```

```

699 const handlePermissionClick = async (permission) => {
700   if (typeof window === "undefined") return;
701   if (selecting) return;
702   setSelectError(null);
703   const apiBase = getBaseUrl();
704   const moduleMeta =
705     modules.find((module) => module.type === permission.type) ??
706     modules.find((module) => module.title === permission.label);
707   const destination =
708     permission.destination ?? permission.href ?? moduleMeta?.href ?? moduleLinks[permission.type];
709   const roleMatch = findRoleMatch(
710     profile?.roles,
711     permission.type ?? permission.label,
712     permission.eventId,
713     permission.adminId
714   );
715   const adminId = permission.adminId ?? roleMatch?.adminId;
716   const apiType = permission.type ?? roleMatch?.type;
717   const normalizedType = normalizeRoleType(apiType || "");
718   const isAdminType = normalizedType === "admin";
719   const needsEventId =
720     normalizedType &&
721     normalizedType !== "tag-series" &&
722     normalizedType !== "tag_series" &&
723     normalizedType !== "tag series" &&
724     !isAdminType;
725   const eventId = permission.eventId ?? roleMatch?.eventId ?? null;
726   const userEmail = profile?.email ?? null;
727
728   capturePostHogEvent("module_role_selected", {
729     app: "access_portal",
730     user_email: userEmail,
731     module_type: apiType ?? null,
732     selected_label: permission.label ?? null,
733     admin_id: adminId ?? null,
734     event_id: eventId ?? null
735   });
736
737   if (!apiBase) {
738     setSelectError("Missing API base URL (NEXT_PUBLIC_BASE_URL)");
739     return;
740   }
741   if (!apiType) {
742     setSelectError("Module type is missing.");
743     return;
744   }
745   if (!roleMatch) {
746     setSelectError("Please select the workspace again. No matching access role was found.");
747     return;
748   }
749   if (needsEventId && !eventId) {
750     setSelectError("Event ID is required for this module.");
751     return;
752   }
753   if (!needsEventId && !adminId) {
754     setSelectError("Admin ID is required for this module.");
755     return;
756   }
757
758   const shouldFetchEventDetails = Boolean(
759     (permission.requireEventDetails ??
760       (needsEventId &&
761         normalizedType !== "tag-series" &&
762         normalizedType !== "tag_series" &&
763         normalizedType !== "tag series")) &&
764     eventId
765   );
766
767   let didNavigateAway = false;
768   try {

```



```

769     setSelecting(permission.type || permission.label || "switching");
770     const payload = needsEventId
771       ? { type: apiType, eventId }
772       : { type: apiType, adminId };
773     const bootstrapToken = readCookie(BOOTSTRAP_TOKEN_COOKIE);
774     const res = await fetch(`${apiBase}/auth/select`, {
775       method: "POST",
776       headers: {
777         "Content-Type": "application/json",
778         ...(bootstrapToken ? { Authorization: `Bearer ${bootstrapToken}` } : {})
779       },
780       credentials: "include",
781       body: JSON.stringify(payload)
782     });
783
784     if (!res.ok) {
785       const text = await res.text();
786       throw new Error(text || `Request failed (${res.status})`);
787     }
788     const selectionData = await res.json().catch(() => null);
789     const selectedToken = findTokenInResponse(selectionData);
790     const service = permission.service ?? mapServiceParam(apiType);
791     if (selectedToken) {
792       persistSelectedAuthToken(selectedToken, {
793         type: apiType,
794         service
795       });
796     } else if (typeof window !== "undefined") {
797       window.localStorage.removeItem(DASHBOARD_SELECTED_TOKEN_KEY);
798       window.localStorage.removeItem("atomx.dashboard.store");
799     }
800     clearBootstrapTokenCookie();
801
802     if (typeof window !== "undefined") {
803       window.localStorage.removeItem(REAUTH_CONTEXT_KEY);
804     }
805
806     let eventDetails = null;
807     if (shouldFetchEventDetails) {
808       eventDetails = await fetchSelectedEventDetails({
809         apiBase,
810         eventId,
811         token: selectedToken
812       });
813     }
814
815     capturePostHogEvent("module_selected", {
816       app: "access_portal",
817       user_email: userEmail,
818       service,
819       module_type: apiType ?? null,
820       admin_id: adminId ?? null,
821       event_id: eventId ?? null,
822       destination: destination ?? null
823     });
824
825     if (destination) {
826       const target = destination.startsWith("http")
827         ? new URL(destination)
828         : new URL(destination, window.location.origin);
829       if (permission.returnMode === "message" && window.opener && !window.opener.closed) {
830         try {
831           window.opener.postMessage(
832             {
833               type: "atomx.auth",
834               service: service ?? permission.type,
835               token: selectedToken,
836               eventId
837             },
838             target.origin

```

```

839         );
840         window.close();
841         return;
842     } catch (err) {
843         console.error("Failed to post auth token", err);
844     }
845 }
846 if (service) {
847     target.searchParams.set("service", service);
848 }
849 if (selectedToken) {
850     target.searchParams.set("token", selectedToken);
851 }
852 if (shouldFetchEventDetails) {
853     const resolvedEventId = eventDetails?.id ?? eventId;
854     const resolvedEventName = eventDetails?.name ?? eventDetails?.eventName ?? null;
855     const resolvedVenue = eventDetails?.venue ?? null;
856     const resolvedCity =
857         eventDetails?.locationCity ?? eventDetails?.city ?? null;
858
859     if (resolvedEventId) {
860         target.searchParams.set("eventId", String(resolvedEventId));
861     }
862     if (resolvedEventName) {
863         target.searchParams.set("eventName", resolvedEventName);
864     }
865     if (resolvedVenue) {
866         target.searchParams.set("venue", resolvedVenue);
867     }
868     if (resolvedCity) {
869         target.searchParams.set("city", resolvedCity);
870     }
871 }
872 didNavigateAway = true;
873 window.location.assign(target.toString());
874 return;
875 }
876 } catch (err) {
877     console.error(err);
878     capturePostHogEvent("module_selection_failed", {
879         app: "access_portal",
880         user_email: userEmail,
881         module_type: apiType ?? null,
882         service: permission.service ?? mapServiceParam(apiType),
883         admin_id: adminId ?? null,
884         event_id: eventId ?? null,
885         error_message: err?.message || "service_selection_failed"
886     });
887     setSelectError(err.message || "Failed to switch module");
888     setModalApp({
889         title: permission.label,
890         description: "We could not refresh your access token. Please try again."
891     });
892 } finally {
893     clearBootstrapTokenCookie();
894     if (!didNavigateAway) {
895         setSelecting(null);
896     }
897 }
898 };
899
900 useEffect(() => {
901     if (status !== "ready" || !profile || reauthHandledRef.current) return;
902     if (typeof window === "undefined") return;
903
904     const returnToParamRaw = router.query.returnTo;
905     const returnToParam = Array.isArray(returnToParamRaw)
906         ? returnToParamRaw[0]
907         : returnToParamRaw;
908     const sanitizedReturnToParam =

```

```

909     returnToParam && typeof window !== "undefined"
910     ? sanitizeReturnTo(returnToParam)
911     : returnToParam;
912
913     const context = readReauthContext();
914     const rawReturnTo = context?.returnTo || sanitizedReturnToParam || null;
915     const returnTo = rawReturnTo ? sanitizeReturnTo(rawReturnTo) : null;
916     const type = context?.type || null;
917     const eventId = context?.eventId ?? null;
918     const adminId = context?.adminId ?? null;
919     const service = context?.service || mapServiceParam(type);
920     const currentUrl = sanitizeReturnTo(window.location.href);
921
922     if (type) {
923         const hasRole = Boolean(findRoleMatch(profile?.roles, type, eventId, adminId));
924         if (hasRole) {
925             reauthHandledRef.current = true;
926             handlePermissionClick({
927                 type,
928                 label: type,
929                 eventId,
930                 adminId,
931                 destination: returnTo,
932                 service
933             });
934             return;
935         }
936         window.localStorage.removeItem(REAUTH_CONTEXT_KEY);
937     }
938
939     if (returnTo && returnTo !== currentUrl) {
940         reauthHandledRef.current = true;
941         window.localStorage.removeItem(REAUTH_CONTEXT_KEY);
942         window.location.assign(returnTo);
943     }
944 }, [status, profile, router.query.returnTo, handlePermissionClick]);
945
946 const handleSignOut = () => {
947     if (typeof window !== "undefined") {
948         try {
949             window.localStorage.removeItem("atomx.portal.token");
950             window.localStorage.removeItem(DASHBOARD_SELECTED_TOKEN_KEY);
951             window.localStorage.removeItem("atomx.dashboard.store");
952             window.localStorage.removeItem(REAUTH_CONTEXT_KEY);
953             const keysToRemove = [];
954             for (let i = 0; i < window.localStorage.length; i += 1) {
955                 const key = window.localStorage.key(i);
956                 if (!key) continue;
957                 if (key.startsWith("atomx.auth.")) {
958                     keysToRemove.push(key);
959                 }
960             }
961             keysToRemove.forEach((key) => window.localStorage.removeItem(key));
962             if (window.sessionStorage) {
963                 window.sessionStorage.clear();
964             }
965         } catch (err) {
966             console.error("Failed to clear auth cache", err);
967         }
968         window.location.assign("/");
969         return;
970     }
971     setProfile(null);
972     setModules([]);
973     setStatus("empty");
974     clearBootstrapTokenCookie();
975     router.replace("/");
976 };
977
978 const handleEventRoleClick = (role) => {

```

```

979     const service = mapServiceParam(role?.type);
980     const isTagSeries = service === "tag-series";
981     const destination = isTagSeries
982       ? ensureTrailingSlashForRoute(moduleLinks["tag-series"])
983       : `${dashboardBase}${dashboardConfigPath}`;
984     handlePermissionClick({
985       type: role.type,
986       label: role.type,
987       destination,
988       service,
989       eventId: role.eventId,
990       adminId: role.adminId,
991       requireEventDetails: !isTagSeries
992     });
993   };
994
995   const handleAdminRoleClick = (role) => {
996     handlePermissionClick({
997       type: role.type,
998       label: role.type,
999       destination: accessAdminUrl,
1000       adminId: role.adminId
1001     });
1002   };
1003
1004   const renderSectionHeader = (count, label, caption) => (
1005     <div className="mb-4 flex items-center gap-3">
1006       <span className="font-vcr flex h-7 w-7 shrink-0 items-center justify-center rounded-full bg-(--text
1007 ) text-[10.5px] tracking-[0.04em] text-(--bg)">
1008         {formatSectionCount(count)}
1009       </span>
1010       <h2 className="font-chillax m-0 shrink-0 text-[1.35rem] font-medium tracking-[-0.01em] text-(--text
1011 )">
1012         {label}
1013       </h2>
1014       <span className="h-px flex-1 bg-(--line2)" aria-hidden />
1015       {caption ? (
1016         <span className="font-vcr hidden shrink-0 text-[9px] uppercase tracking-[0.14em] text-(--faint) s
1017 m:block">
1018           {caption}
1019         </span>
1020       ) : null}
1021     </div>
1022   );
1023
1024   const renderRoleCards = (roles, emptyLabel, options = {}) => {
1025     const { onCardClick, showOpenIcon = false } = options;
1026     const isAdminSection = options.section === "admin";
1027
1028     if (!roles.length) {
1029       return (
1030         <div className="rounded-[14px] border border-dashed border-(--line) bg-(--surface) px-5 py-4 text
1031 -sm text-(--muted)">
1032           {emptyLabel}
1033         </div>
1034       );
1035     }
1036
1037     return (
1038       <section
1039         style={{
1040           display: "grid",
1041           gap: "clamp(12px,1.4vw,18px)",
1042           gridTemplateColumns: "repeat(auto-fill,minmax(min(100%,270px),1fr))"
1043         }}
1044       >
1045         {roles.map((role) => {
1046           const theme = getRoleTheme(role.type);
1047           return (
1048             <button

```

```

1045         key={role.id}
1046         type="button"
1047         className={`group relative flex flex-col overflow-hidden rounded-[14px] text-left shadow-(
--shadow) transition duration-200 ease-out hover:-translate-y-[3px] hover:shadow-[var(--shadowUp),0_0_1p
x_rgba(224,68,32,0.30)]`} ${
1048         onCardClick ? "cursor-pointer" : "cursor-default"
1049       }`
1050       style={{
1051         border: "1px solid transparent",
1052         background: `linear-gradient(var(--surface),var(--surface)) padding-box, ${theme.border}
border-box`,
1053         "--accent": theme.text
1054       }}
1055       onClick={() => {
1056         if (!onCardClick || !role?.type) return;
1057         onCardClick(role);
1058       }}
1059       disabled={!onCardClick}
1060       aria-label={`Open ${role.type}`}
1061     >
1062     <div className="flex items-start gap-3 px-4 pb-2.5 pt-4">
1063       <span
1064         className="flex h-[34px] w-[34px] shrink-0 items-center justify-center rounded-[10px] t
ext-white"
1065         style={{ background: theme.tile }}
1066       >
1067         <RoleGlyph type={role.type} />
1068       </span>
1069       <div className="min-w-0 flex-1">
1070         <p
1071           className="font-vcr m-0 truncate text-[9.5px] tracking-[0.15em]"
1072           style={{ color: theme.text }}
1073         >
1074           {String(role.type || "-").toUpperCase()}
1075         </p>
1076         <p className="m-0 mt-1 truncate text-[11.5px] font-light text-(--muted)">
1077           {role.expiryAt ? formatSessionExpiry(role.expiryAt) : "No expiry"}
1078         </p>
1079       </div>
1080       {showOpenIcon ? (
1081         <span className="flex h-[26px] w-[26px] shrink-0 items-center justify-center rounded-[8
px] border border-(--line) text-(--muted) transition duration-200 group-hover:border-(--accent) group-hov
er:text-(--accent)">
1082           <OpenArrowGlyph />
1083         </span>
1084       ) : null}
1085     </div>
1086     <div className="px-4 pb-3">
1087       <p className="font-chillax m-0 truncate text-[16.5px] font-medium tracking-[-0.01em] text
-(--text)">
1088         {isAdminSection ? role.adminName || "-" : role.eventName || "-"}
1089       </p>
1090     </div>
1091     <div className="mt-auto flex items-center justify-between gap-2.5 border-t border-(--line2)
bg-(--surface2) px-4 py-[9px]">
1092       <span className="font-vcr text-[9px] tracking-[0.14em] text-(--faint)">
1093         {isAdminSection ? "ADMIN ID" : "EVENT ID"}
1094       </span>
1095       <span className="font-vcr text-[13px] tracking-[0.02em] text-(--text)">
1096         {isAdminSection ? role.adminId ?? "-" : role.eventId ?? "-"}
1097       </span>
1098     </div>
1099   </button>
1100 );
1101 }}}
1102 </section>
1103 );
1104 };
1105 };

```

```

1107
1108   return (
1109     <>
1110     <Head>
1111       <title>AtomX Access Portal</title>
1112       <meta
1113         name="description"
1114         content="Pick the AtomX module you need in one click."
1115       />
1116     </Head>
1117     <main className="min-h-[calc(100vh-58px)] w-full bg-(--bg) px-4 pb-20 md:px-8">
1118       <HeaderBar
1119         user={user}
1120         onSignOut={handleSignOut}
1121         pageTitle="Workspace"
1122       />
1123
1124       {status === "error" && !hideErrorBanner && (
1125         <div className="relative mt-5 rounded-lg border border-red-200 bg-red-50 px-5 py-4 pr-12 text-s
1126           m text-red-700">
1127           {error}
1128           <button
1129             type="button"
1130             className="absolute right-3 top-3 flex h-7 w-7 items-center justify-center rounded-md borde
1131             r border-red-200 text-red-500 transition hover:bg-red-100"
1132             onClick={() => setHideErrorBanner(true)}
1133             aria-label="Dismiss error"
1134           >
1135             <svg
1136               viewBox="0 0 24 24"
1137               className="h-4 w-4"
1138               fill="none"
1139               stroke="currentColor"
1140               strokeWidth="2"
1141               strokeLinecap="round"
1142               strokeLinejoin="round"
1143               aria-hidden
1144             >
1145               <path d="M18 6L6 18" />
1146               <path d="M6 6L12 12" />
1147             </svg>
1148           </button>
1149         </div>
1150       )}
1151
1152       {status === "empty" && !hideEmptyBanner && (
1153         <div className="relative mt-5 rounded-lg border border-amber-200 bg-amber-50 px-5 py-4 pr-12 te
1154         xt-sm text-amber-700">
1155         No active session detected. Please log in first.
1156         <button
1157           type="button"
1158           className="absolute right-3 top-3 flex h-7 w-7 items-center justify-center rounded-md borde
1159           r border-amber-200 text-amber-600 transition hover:bg-amber-100"
1160           onClick={() => setHideEmptyBanner(true)}
1161           aria-label="Dismiss notice"
1162         >
1163           <svg
1164             viewBox="0 0 24 24"
1165             className="h-4 w-4"
1166             fill="none"
1167             stroke="currentColor"
1168             strokeWidth="2"
1169             strokeLinecap="round"
1170             strokeLinejoin="round"
1171             aria-hidden
1172           >

```

```

1173     </div>
1174   })
1175
1176   <div className="mx-auto w-full max-w-[1680px]">
1177     <div className="mt-8 flex flex-wrap items-end justify-between gap-4">
1178       <WelcomePanel user={user} actions={highlightActions} />
1179       <div className="flex h-10 w-full min-w-[210px] items-center gap-2 rounded-[11px] border borde
1180 r-(--line) bg-(--surface) px-3 md:w-auto md:max-w-[600px] md:flex-1">
1181         <span className="pointer-events-none shrink-0 text-(--faint)">
1182           <svg
1183             viewBox="0 0 16 16"
1184             className="h-3.5 w-3.5"
1185             fill="none"
1186             stroke="currentColor"
1187             strokeWidth="1.5"
1188             strokeLinecap="round"
1189             aria-hidden
1190           >
1191             <circle cx="7" cy="7" r="4.6" />
1192             <path d="M10.5 10.5 14 14" />
1193           </svg>
1194         </span>
1195         <input
1196           type="text"
1197           value={searchQuery}
1198           onChange={(event) => setSearchQuery(event.target.value)}
1199           placeholder="Search workspaces & events"
1200           aria-label="Search workspaces and events"
1201           className="w-full border-0 bg-transparent p-0 text-[12.5px] text-(--text) outline-none pl
1202 aceholder:text-(--faint)"
1203         />
1204       </div>
1205     </div>
1206
1207     {filters.length > 1 ? (
1208       <div className="mt-6 flex flex-wrap gap-2 pb-0.5 max-sm:flex-nowrap max-sm:overflow-x-auto ma
1209 x-sm:[-ms-overflow-style:none] max-sm:[scrollbar-width:none] max-sm:[&::-webkit-scrollbar]:hidden">
1210         {filters.map((filter) => {
1211           const isActive = filter.key === activeFilter;
1212           return (
1213             <button
1214               key={filter.key}
1215               type="button"
1216               onClick={() => setActiveFilter(filter.key)}
1217               className={`flex shrink-0 items-center gap-[7px] rounded-full border px-[13px] py-[7p
1218 x] text-[11.5px] font-medium transition ${
1219               isActive
1220               ? "border-(--text) bg-(--text) text-(--bg)"
1221               : "border-(--line) bg-(--surface) text-(--muted) hover:border-(--orange) hover:te
1222 xt-(--text)"
1223             }`}
1224             aria-pressed={isActive}
1225           >
1226             <span>{filter.label}</span>
1227             <span className="font-vcr text-[9px] opacity-60">
1228               {formatSectionCount(filter.count)}
1229             </span>
1230           </button>
1231         );
1232       )}
1233     </div>
1234   ) : null}
1235
1236   <div className="mt-[22px] h-px w-full bg-(--line)" />
1237
1238   {!roleCards.length ? (
1239     <div className="mt-6 rounded-[14px] border border-dashed border-(--line) bg-(--surface) px-5
1240 py-4 text-sm text-(--muted)">
1241       No roles assigned yet.
1242     </div>
1243   ) : null}

```

```

1237         ) : !hasVisibleRoles ? (
1238             <div className="mt-6 rounded-[14px] border border-dashed border-(--line) bg-(--surface) px-5
py-8 text-center text-sm text-(--muted)">
1239                 No workspaces or events match your search.
1240             </div>
1241         ) : (
1242             <div className="mt-[30px] flex flex-col gap-[30px]">
1243                 {visibleAdminRoles.length ? (
1244                     <div className="flex flex-col">
1245                         {renderSectionHeader(
1246                             visibleAdminRoles.length,
1247                             "Admin Workspaces",
1248                             "Full platform control"
1249                         )}
1250                         {renderRoleCards(visibleAdminRoles, "No admin roles assigned yet.", {
1251                             onCardClick: handleAdminRoleClick,
1252                             showOpenIcon: true,
1253                             section: "admin"
1254                         })}
1255                     </div>
1256                 ) : null}
1257
1258                 {visibleEventRoles.length ? (
1259                     <div className="flex flex-col">
1260                         {renderSectionHeader(
1261                             visibleEventRoles.length,
1262                             "Events",
1263                             "Scoped module access"
1264                         )}
1265                         {renderRoleCards(visibleEventRoles, "No event roles assigned yet.", {
1266                             onCardClick: handleEventRoleClick,
1267                             showOpenIcon: true,
1268                             section: "event"
1269                         })}
1270                     </div>
1271                 ) : null}
1272             </div>
1273         )}
1274     </div>
1275 </main>
1276
1277     {selecting && (
1278         <div className="fixed inset-0 z-50 flex items-center justify-center bg-[color-mix(in_srgb,var(--b
g)_72%,transparent)] backdrop-blur-sm">
1279             <div className="rounded-2xl bg-(--surface) px-6 py-4 text-(--text) shadow-(--shadowUp)">
1280                 <AtomXLoader label="Switching access..." size={62} />
1281             </div>
1282         </div>
1283     )}
1284
1285     {selectError && !modalApp && (
1286         <div className="fixed bottom-4 left-1/2 z-40 w-[90%] max-w-md -translate-x-1/2 rounded-2xl bg-red
-50 px-4 py-3 text-center text-sm font-medium text-red-700 shadow-lg">
1287             {selectError}
1288             <button
1289                 type="button"
1290                 className="absolute right-3 top-3 flex h-7 w-7 items-center justify-center rounded-full borde
r border-red-200 text-red-500 transition hover:bg-red-100"
1291                 onClick={() => setSelectError(null)}
1292                 aria-label="Dismiss error"
1293             >
1294                 <svg
1295                     viewBox="0 0 24 24"
1296                     className="h-4 w-4"
1297                     fill="none"
1298                     stroke="currentColor"
1299                     strokeWidth="2"
1300                     strokeLinecap="round"
1301                     strokeLinejoin="round"
1302                     aria-hidden

```



```

1303         >
1304         <path d="M18 6L6 18" />
1305         <path d="M6 6L12 12" />
1306         </svg>
1307     </button>
1308 </div>
1309 }}
1310
1311     {modalApp && (
1312     <div className="fixed inset-0 z-50 flex items-center justify-center bg-slate-900/60 px-4 backdrop
-blur-sm">
1313         <div className="w-full max-w-md rounded-2xl bg-(--surface) p-6 text-center text-(--text) shadow
-[0_32px_65px_rgba(0,0,0,0.35)]">
1314             <h2 className="m-0 text-xl font-semibold">{modalApp.title}</h2>
1315             <p className="mt-3 text-base text-(--muted)">
1316                 {modalApp.description || "This module is going live soon. We'll notify you when access is r
eady."}
1317             </p>
1318             <button
1319                 type="button"
1320                 className="mt-5 w-full rounded-full bg-[#e04420] px-4 py-3 text-sm font-semibold text-white
shadow-[0_15px_40px_rgba(224,68,32,0.3)] transition hover:brightness-105"
1321                 onClick={() => setModalApp(null)}
1322             >
1323                 Got it
1324             </button>
1325         </div>
1326     </div>
1327     )}
1328 </>
1329 );
1330 }

```

FILE: apps/access_portal/pages/index.js

```
1 import { useMemo } from "react";
2 import { useRouter } from "next/router";
3 import LoginScreen from "../components/LoginScreen/LoginScreen";
4
5 function sanitizeReturnTo(value) {
6   try {
7     const parsed = new URL(value, window.location.origin);
8     parsed.searchParams.delete("returnTo");
9     return parsed.toString();
10  } catch (err) {
11    return value;
12  }
13 }
14
15 export default function PortalLogin() {
16   const router = useRouter();
17   const returnTo = useMemo(() => {
18     const raw = router.query.returnTo;
19     const value = Array.isArray(raw) ? raw[0] : raw;
20     if (!value || typeof window === "undefined") return value;
21     return sanitizeReturnTo(value);
22   }, [router.query.returnTo]);
23
24   const redirectPath = returnTo
25     ? ` /access?returnTo=${encodeURIComponent(returnTo)}`
26     : "/access";
27
28   return <LoginScreen appId="portal" redirectPath={redirectPath} />;
29 }
```

FILE: apps/access_portal/postcss.config.mjs

```
1 const config = {
2   plugins: {
3     "@tailwindcss/postcss": {}
4   }
5 };
6
7 export default config;
```

FILE: apps/access_portal/styles/globals.css

```
1 @import "tailwindcss";
2
3 @font-face {
4   font-family: "Poppins";
5   src: url("/shared/fonts/Poppins/Poppins-Thin.ttf") format("truetype");
6   font-display: swap;
7   font-weight: 100;
8   font-style: normal;
9 }
10
11 @font-face {
12   font-family: "Poppins";
13   src: url("/shared/fonts/Poppins/Poppins-Light.ttf") format("truetype");
14   font-display: swap;
15   font-weight: 300;
16   font-style: normal;
17 }
18
19 @font-face {
20   font-family: "Poppins";
21   src: url("/shared/fonts/Poppins/Poppins-Regular.ttf") format("truetype");
22   font-display: swap;
23   font-weight: 400;
24   font-style: normal;
25 }
26
27 @font-face {
28   font-family: "Poppins";
29   src: url("/shared/fonts/Poppins/Poppins-MediumItalic.ttf") format("truetype");
30   font-display: swap;
31   font-weight: 500;
32   font-style: italic;
33 }
34
35 @font-face {
36   font-family: "Poppins";
37   src: url("/shared/fonts/Poppins/Poppins-SemiBold.ttf") format("truetype");
38   font-display: swap;
39   font-weight: 600;
40   font-style: normal;
41 }
42
43 @font-face {
44   font-family: "Poppins";
45   src: url("/shared/fonts/Poppins/Poppins-Bold.ttf") format("truetype");
46   font-display: swap;
47   font-weight: 700;
48   font-style: normal;
49 }
50
51 /* AtomX primary / headline font (brand guidelines) */
52 @font-face {
53   font-family: "Chillax";
54   src: url("/shared/fonts/Chillex/Chillax-Light.woff2") format("woff2");
55   font-display: swap;
56   font-weight: 300;
57   font-style: normal;
58 }
59
60 @font-face {
61   font-family: "Chillax";
62   src: url("/shared/fonts/Chillex/Chillax-Regular.woff2") format("woff2");
63   font-display: swap;
64   font-weight: 400;
65   font-style: normal;
66 }
67
68 @font-face {
69   font-family: "Chillax";
```

```

70  src: url("/shared/fonts/Chillex/Chillax-Medium.woff2") format("woff2");
71  font-display: swap;
72  font-weight: 500;
73  font-style: normal;
74 }
75
76 @font-face {
77   font-family: "Chillax";
78   src: url("/shared/fonts/Chillex/Chillax-Semibold.woff2") format("woff2");
79   font-display: swap;
80   font-weight: 600;
81   font-style: normal;
82 }
83
84 @font-face {
85   font-family: "Chillax";
86   src: url("/shared/fonts/Chillex/Chillax-Bold.woff2") format("woff2");
87   font-display: swap;
88   font-weight: 700;
89   font-style: normal;
90 }
91
92 /* AtomX accent font (brand guidelines) – used for technical/data labels */
93 @font-face {
94   font-family: "VCR";
95   src: url("/shared/fonts/VCR/VCR_OSD_MONO_1.001.ttf") format("truetype");
96   font-display: swap;
97   font-weight: 400;
98   font-style: normal;
99 }
100
101 .font-chillax {
102   font-family: "Chillax", "Poppins", "Segoe UI", system-ui, sans-serif;
103 }
104
105 .font-vcr {
106   font-family: "VCR", "VCR OSD Mono", ui-monospace, "SFMono-Regular", monospace;
107 }
108
109 :root {
110   /* legacy aliases (still referenced by a few module CSS files) */
111   --orange-soft: #ffd5b3;
112   --teal: #1495ab;
113   --teal-soft: #d4eff4;
114
115   /* AtomX semantic tokens – light theme */
116   --bg: #f2f1ee;
117   --surface: #ffffff;
118   --surface2: #f8f7f5;
119   --text: #1c1c1c;
120   --muted: #71706e;
121   --faint: #9b9a97;
122   --border: #ebe7e1;
123   --line: rgba(28, 28, 28, 0.12);
124   --line2: rgba(28, 28, 28, 0.06);
125   --chip: rgba(28, 28, 28, 0.045);
126
127   /* brand accents (constant across themes, except purple) */
128   --orange: #e04420;
129   --blue: #341cd6;
130   --lblue: #00a9f2;
131   --purple: #8b5cf6;
132   --egg: #ebebeb;
133
134   --shadow: 0 1px 2px rgba(28, 28, 28, 0.04), 0 10px 30px -18px rgba(28, 28, 28, 0.25);
135   --shadowUp: 0 2px 4px rgba(28, 28, 28, 0.05), 0 18px 40px -20px rgba(28, 28, 28, 0.35);
136
137   color-scheme: light;
138 }
139

```

```
140 html[data-atx="dark"] {
141   --bg: #0c0c0c;
142   --surface: #181818;
143   --surface2: #212120;
144   --text: #ebebcb;
145   --muted: #8e8d8a;
146   --faint: #6b6a67;
147   --border: #2a2a2a;
148   --line: rgba(235, 235, 235, 0.14);
149   --line2: rgba(235, 235, 235, 0.07);
150   --chip: rgba(235, 235, 235, 0.06);
151
152   --purple: #d5b7ff;
153
154   --shadow: 0 1px 2px rgba(0, 0, 0, 0.5), 0 10px 30px -18px rgba(0, 0, 0, 0.9);
155   --shadowUp: 0 2px 6px rgba(0, 0, 0, 0.6), 0 18px 40px -20px rgba(0, 0, 0, 1);
156
157   color-scheme: dark;
158 }
159
160 * {
161   box-sizing: border-box;
162 }
163
164 body {
165   margin: 0;
166   font-family: "Poppins", "Inter", "Segoe UI", system-ui, -apple-system,
167     BlinkMacSystemFont, "Helvetica Neue", Arial, sans-serif;
168   background: var(--bg);
169   color: var(--text);
170   min-height: 100vh;
171 }
172
173 a {
174   color: inherit;
175   text-decoration: none;
176 }
177
178 button {
179   font-family: inherit;
180 }
```

FILE: apps/access_portal/tailwind.config.mjs

```
1  /** @type {import('tailwindcss').Config} */
2  const config = {
3    content: [
4      "./pages/**/*.{js,jsx}",
5      "./components/**/*.{js,jsx}",
6      "./styles/**/*.{js,jsx,css}",
7      "../../packages/global-components/src/**/*.{js,jsx}",
8      "../../packages/shared-ui/src/**/*.{js,jsx}"
9    ],
10   theme: {
11     extend: {}
12   },
13   plugins: []
14 };
15
16 export default config;
```

FILE: apps/dashboard/jsconfig.json

```
1 {  
2   "compilerOptions": {  
3     "paths": {  
4       "@/*": [".src/*"]  
5     }  
6   }  
7 }
```


FILE: apps/dashboard/next.config.mjs

```
1 import path from "node:path";
2 import dotenv from "dotenv";
3
4 dotenv.config({ path: path.resolve(process.cwd(), "..", "..", ".env") });
5
6 /** @type {import('next').NextConfig} */
7 const basePathEnv = process.env.NEXT_PUBLIC_DASHBOARD_BASEPATH ?? "/";
8 const basePath = basePathEnv === "/" ? "" : basePathEnv;
9
10 const nextConfig = {
11   reactCompiler: true,
12   output: "export",
13   basePath,
14   assetPrefix: basePath || undefined,
15   images: {
16     unoptimized: true
17   },
18   trailingSlash: true,
19   transpilePackages: [
20     "@atomx/shared-ui",
21     "@atomx/global-components",
22     "@atomx/lib"
23   ],
24   experimental: {
25     externalDir: true
26   }
27 };
28
29 export default nextConfig;
```

FILE: apps/dashboard/package.json

```
1 {
2   "name": "dashboard",
3   "version": "0.1.0",
4   "private": true,
5   "scripts": {
6     "dev": "next dev",
7     "build": "next build",
8     "start": "next start"
9   },
10  "dependencies": {
11    "@atomx/global-components": "0.1.0",
12    "@atomx/lib": "0.1.0",
13    "next": "16.2.6",
14    "react": "19.2.0",
15    "react-dom": "19.2.0",
16    "xlsx": "^0.18.5",
17    "zustand": "^4.5.5"
18  },
19  "devDependencies": {
20    "@tailwindcss/postcss": "^4",
21    "babel-plugin-react-compiler": "1.0.0",
22    "tailwindcss": "^4"
23  }
24 }
```

FILE: apps/dashboard/postcss.config.mjs

```
1 const config = {
2   plugins: {
3     "@tailwindcss/postcss": {},
4   },
5 };
6
7 export default config;
```

FILE: apps/dashboard/src/app/Blocked/components/BlockedCards.js

```

1  "use client";
2
3  import { useMemo, useState } from "react";
4
5  const BLOCKED_CARDS = [
6    { id: 1, cardId: "24", status: "ACTIVE" },
7    { id: 2, cardId: "240060131", status: "ACTIVE" },
8    { id: 3, cardId: "111111", status: "ACTIVE" },
9    { id: 4, cardId: "99889988", status: "ACTIVE" }
10 ];
11
12 function SearchIcon() {
13   return (
14     <svg viewBox="0 0 24 24" className="h-4 w-4" fill="none" stroke="currentColor" strokeWidth="2" stroke
Linecap="round" strokeLinejoin="round">
15       <circle cx="11" cy="11" r="7" />
16       <path d="m20 20-3.5-3.5" />
17     </svg>
18   );
19 }
20
21 function LockIcon() {
22   return (
23     <svg viewBox="0 0 24 24" className="h-4 w-4" fill="none" stroke="currentColor" strokeWidth="2" stroke
Linecap="round" strokeLinejoin="round">
24       <rect x="5" y="11" width="14" height="9" rx="2" />
25       <path d="M8 11V8a4 4 0 0 1 8 0v3" />
26     </svg>
27   );
28 }
29
30 function EmptyIcon() {
31   return (
32     <svg viewBox="0 0 24 24" className="h-6 w-6" fill="none" stroke="currentColor" strokeWidth="1.8" stro
keLinecap="round" strokeLinejoin="round">
33       <circle cx="12" cy="12" r="9" />
34       <path d="M7 7l10 10" />
35     </svg>
36   );
37 }
38
39 function BlockedCard({ item }) {
40   return (
41     <article className="min-h-[126px] rounded-lg border border-[#efb9d9] bg-white p-3 shadow-[0_10px_24px
_rgba(15,23,42,0.035)] transition duration-200 hover:-translate-y-0.5 hover:border-[#ded4ff] hover:shadow
-[0_16px_30px_rgba(52,28,214,0.09)] max-[640px]:min-h-[108px] max-[640px]:p-2.5">
42       <div className="flex items-start justify-between gap-3">
43         <span className="text-[0.8rem] font-bold text-[#E04420] max-[640px]:text-[0.68rem]">#{item.id}</s
pan>
44         <span className="rounded-full bg-[#e4f6ff] px-2.5 py-1 text-[0.62rem] font-bold uppercase text-[#
0285bf] max-[640px]:px-2 max-[640px]:py-0.5 max-[640px]:text-[0.52rem]">
45           {item.status}
46         </span>
47       </div>
48
49       <div className="mt-3">
50         <p className="text-[0.62rem] font-bold uppercase tracking-[0.16em] text-[#929292] max-[640px]:tex
t-[0.52rem]">
51           Card ID
52         </p>
53         <p className="mt-1 text-[1.05rem] font-bold leading-tight text-[#202020] max-[640px]:text-[0.86re
m]">
54           {item.cardId}
55         </p>
56       </div>
57
58       <button
59         type="button"
60         className="mt-3 inline-flex h-9 w-full items-center justify-center gap-2 rounded-lg bg-[#1c1c1c]

```

```

    text-[0.74rem] font-bold text-white shadow-[0_10px_20px_rgba(28,28,28,0.10)] transition duration-200 hover:
    r:bg-[#E04420] max-[640px]:mt-2 max-[640px]:h-8 max-[640px]:text-[0.64rem]"
61     >
62     <LockIcon />
63     Unblock
64   </button>
65 </article>
66   );
67 }
68
69 export default function BlockedCards() {
70   const [query, setQuery] = useState("");
71
72   const filteredCards = useMemo(() => {
73     const normalizedQuery = query.trim().toLowerCase();
74     if (!normalizedQuery) return BLOCKED_CARDS;
75
76     return BLOCKED_CARDS.filter((item) =>
77       [item.id, item.cardId, item.status].some((value) =>
78         String(value).toLowerCase().includes(normalizedQuery)
79       )
80     );
81   }, [query]);
82
83   return (
84     <section className="rounded-xl border border-[#ded4ff] border-l-[4px] border-l-[#E04420] bg-white p-4
85       shadow-[0_18px_52px_rgba(15,23,42,0.09)] max-[640px]:rounded-lg max-[640px]:p-3">
86       <div className="flex flex-col gap-3 border-b border-[#e5e5e5] pb-3 lg:flex-row lg:items-center lg:ju
87         ustify-between max-[640px]:gap-2 max-[640px]:pb-2.5">
88         <div className="flex items-center gap-2.5">
89           <span className="grid h-8 w-8 place-items-center rounded-lg bg-[linear-gradient(135deg,#E04420_
90             0%,#A9379E_48%,#341CD6_100%)] text-[0.82rem] font-bold text-white shadow-[0_10px_22px_rgba(52,28,214,0.20
91             )] max-[640px]:h-7 max-[640px]:w-7 max-[640px]:text-[0.66rem]">
92             4
93           </span>
94           <h2 className="text-[1.05rem] font-semibold text-[#1f1f1f] max-[640px]:text-[0.95rem]">Cards</h
95             2>
96         </div>
97
98         <label className="flex h-9 w-full items-center gap-3 border-b border-[#cfcfcf] px-1 text-[#8f80ff
99           ] focus-within:border-[#E04420] lg:max-w-[720px] max-[640px]:gap-2">
100           <SearchIcon />
101           <input
102             value={query}
103             onChange={(event) => setQuery(event.target.value)}
104             placeholder="Search Card"
105             className="h-full min-w-0 flex-1 bg-transparent text-[0.84rem] font-normal text-[#1f2937] out
106               line-none placeholder:text-[#8e98ad] max-[640px]:text-[0.7rem]"
107           />
108         </label>
109       </div>
110
111       <div className="grid gap-2.5 pt-3 sm:grid-cols-2 lg:grid-cols-4 xl:max-w-[960px] max-[640px]:grid-c
112         ols-2 max-[640px]:gap-2">
113         {filteredCards.map((item) => (
114           <BlockedCard key={item.cardId} item={item} />
115         ))}
116       </div>
117
118       <div className="flex min-h-[92px] flex-col items-center justify-center gap-2 text-[#8b96aa] max-[64
119         0px]:min-h-[70px]">
120         <EmptyIcon />
121         <p className="text-[0.82rem] font-normal max-[640px]:text-[0.68rem]">No more blocked IDs to show<
122         /p>
123       </div>
124     </section>
125   );
126 }

```

FILE: apps/dashboard/src/app/Blocked/page.js

```

1 import Header from "../components/Header";
2 import BlockedCards from "../components/BlockedCards";
3
4 export default function BlockedPage() {
5   return (
6     <main
7       className="blocked-page-font min-h-screen bg-[color:rgb(var(--color-bg))] pb-10"
8       style={{ fontFamily: '"AtomX Blocked Poppins", Poppins, sans-serif' }}
9     >
10      <style>{`
11        @font-face {
12          font-family: "AtomX Blocked Poppins";
13          src: url("/shared/fonts/Poppins/Poppins-Regular.ttf") format("truetype");
14          font-weight: 400;
15          font-style: normal;
16          font-display: swap;
17        }
18        @font-face {
19          font-family: "AtomX Blocked Poppins";
20          src: url("/shared/fonts/Poppins/Poppins-SemiBold.ttf") format("truetype");
21          font-weight: 600;
22          font-style: normal;
23          font-display: swap;
24        }
25        @font-face {
26          font-family: "AtomX Blocked Poppins";
27          src: url("/shared/fonts/Poppins/Poppins-Bold.ttf") format("truetype");
28          font-weight: 700;
29          font-style: normal;
30          font-display: swap;
31        }
32        .blocked-page-font,
33        .blocked-page-font * {
34          font-family: "AtomX Blocked Poppins", Poppins, sans-serif;
35        }
36      `}</style>
37      <Header areaLabel="Blocked" variant="portal" />
38
39      <div className="w-full pr-4 pl-[72px] pt-5 md:pr-7 md:pl-[88px] max-[900px]:px-3 max-[640px]:pt-4">
40        <section className="mb-4 flex flex-col gap-4 border-b border-[#d8d8d8] pb-4 sm:flex-row sm:items-
41        start sm:justify-between max-[640px]:mb-3 max-[640px]:gap-3 max-[640px]:pb-3">
42          <div>
43            <h1 className="text-[1.75rem] font-semibold leading-none text-[#111827] md:text-[1.9rem] max-
44            [640px]:text-[1.25rem]">
45              Blocked IDs
46            </h1>
47            <p className="mt-3 text-[0.9rem] font-normal text-[#777777] max-[640px]:mt-2 max-[640px]:text
48            -[0.7rem]">
49              Search, review, and unblock IDs for the active event.
50            </p>
51          </div>
52          <div className="flex h-[54px] min-w-[92px] items-center justify-center gap-2 rounded-lg border
53          border-[#e8e8e8] bg-white px-3 shadow-[0_10px_24px_rgba(15,23,42,0.05)] max-[640px]:h-10 max-[640px]:min-
54          w-[72px] max-[640px]:self-start">
55            <span className="text-[1.25rem] font-bold leading-none text-[#E04420] max-[640px]:text-[0.95r
56            em]">4</span>
57            <span className="text-[0.82rem] font-normal text-[#777777] max-[640px]:text-[0.66rem]">listed
58          </span>
59        </div>
60      </section>
61
62      <BlockedCards />
63    </main>
64  );
65 }

```

FILE: apps/dashboard/src/app/Config/components/Access_Gate_Config.js

```

1  "use client";
2
3  function EditIcon() {
4    return (
5      <svg
6        viewBox="0 0 16 16"
7        className="h-[15px] w-[15px]"
8        fill="none"
9        stroke="currentColor"
10       strokeWidth="1.4"
11       strokeLinecap="round"
12       strokeLinejoin="round"
13       aria-hidden="true"
14     >
15       <path d="M11.2 2.4l2.4 2.4-8 8H3.2v-2.4z" />
16     </svg>
17   );
18 }
19
20 function gateName(gate) {
21   return gate?.gatesmaster_name ?? gate?.gateName ?? gate?.name ?? "-";
22 }
23
24 function categoryName(gate) {
25   return gate?.category_name ?? gate?.categoryName ?? gate?.category ?? "-";
26 }
27
28 export default function Access_Gate_Config({ gates, onEdit }) {
29   return (
30     <div className="max-h-[333px] overflow-auto rounded-[11px] border border-(--line)">
31       <table className="w-full min-w-[520px] border-collapse">
32         <thead className="font-vcr sticky top-0 z-[1] bg-(--surface2)">
33           <tr>
34             <th className="w-[64px] border-b border-(--line) px-3 py-2.5 text-left text-[9.5px] tracking-[0.15em] text-(--orange)">
35               NO.
36             </th>
37             <th className="border-b border-(--line) px-3 py-2.5 text-left text-[9.5px] tracking-[0.15em] text-(--orange)">
38               GATE
39             </th>
40             <th className="border-b border-(--line) px-3 py-2.5 text-left text-[9.5px] tracking-[0.15em] text-(--orange)">
41               CATEGORY
42             </th>
43             <th className="w-[64px] border-b border-(--line) px-3 py-2.5 text-center text-[9.5px] tracking-[0.15em] text-(--orange)">
44               EDIT
45             </th>
46           </tr>
47         </thead>
48         <tbody>
49           {gates.map((gate, index) => (
50             <tr
51               key={gate?.id ?? `${gateName(gate)}-${categoryName(gate)}-${index}`}
52               className="transition hover:bg-(--surface2)"
53             >
54               <td className="font-vcr border-b border-(--line2) px-3 py-2.5 text-[12px] text-(--orange)">
55                 {index + 1}
56               </td>
57               <td className="border-b border-(--line2) px-3 py-2.5">
58                 <span className="block truncate text-[13px] font-semibold text-(--text)">
59                   {gateName(gate)}
60                 </span>
61               </td>
62               <td className="border-b border-(--line2) px-3 py-2.5">
63                 <span className="block truncate text-[13px] font-medium text-(--muted)">
64                   {categoryName(gate)}
65                 </span>

```

```
66         </td>
67         <td className="border-b border-(--line2) px-3 py-2.5 text-center">
68             <button
69                 type="button"
70                 onClick={() => onEdit?.(gate)}
71                 className="inline-flex h-[30px] w-[30px] items-center justify-center rounded-[8px] border border-(--line) bg-(--surface) text-(--muted) transition hover:border-(--orange) hover:text-(--orange)
72                 "
73                 aria-label={`Edit ${gateName(gate)} ${categoryName(gate)}`}
74                 title={`Edit ${gateName(gate)} ${categoryName(gate)}`}
75                 >
76                 <EditIcon />
77             </button>
78         </td>
79     </tr>
80 </tbody>
81 </table>
82 </div>
83 );
84 }
```


FILE: apps/dashboard/src/app/Config/components/Access_Gate_Master.js

```

1  "use client";
2
3  function EditIcon() {
4    return (
5      <svg
6        viewBox="0 0 16 16"
7        className="h-[15px] w-[15px]"
8        fill="none"
9        stroke="currentColor"
10       strokeWidth="1.4"
11       strokeLinecap="round"
12       strokeLinejoin="round"
13       aria-hidden="true"
14     >
15       <path d="M11.2 2.4l2.4 2.4-8 8H3.2v-2.4z" />
16     </svg>
17   );
18 }
19
20 function gateCategoryName(gate) {
21   return gate?.name ?? gate?.gateName ?? gate?.gate_name ?? gate?.title ?? "-";
22 }
23
24 export default function Access_Gate_Master({ gates, onEdit }) {
25   return (
26     <div className="max-h-[333px] overflow-auto rounded-[11px] border border-(--line)">
27       <table className="w-full min-w-[470px] border-collapse">
28         <thead className="font-vcr sticky top-0 z-[1] bg-(--surface2)">
29           <tr>
30             <th className="w-[64px] border-b border-(--line) px-3 py-2.5 text-left text-[9.5px] tracking-[0.15em] text-(--orange)">
31               NO.
32             </th>
33             <th className="w-[90px] border-b border-(--line) px-3 py-2.5 text-left text-[9.5px] tracking-[0.15em] text-(--orange)">
34               GATE
35             </th>
36             <th className="border-b border-(--line) px-3 py-2.5 text-left text-[9.5px] tracking-[0.15em] text-(--orange)">
37               CATEGORY NAME
38             </th>
39             <th className="w-[64px] border-b border-(--line) px-3 py-2.5 text-center text-[9.5px] tracking-[0.15em] text-(--orange)">
40               EDIT
41             </th>
42           </tr>
43         </thead>
44         <tbody>
45           {gates.map((gate, index) => (
46             <tr
47               key={gate?.id ?? `${gateCategoryName(gate)}-${index}`}
48               className="transition hover:bg-(--surface2)"
49             >
50               <td className="font-vcr border-b border-(--line2) px-3 py-2.5 text-[12px] text-(--orange)">
51                 {index + 1}
52               </td>
53               <td className="border-b border-(--line2) px-3 py-2.5">
54                 <span className="font-vcr text-[12.5px] text-(--blue)">
55                   {gate?.gateDay ?? "-"}
56                 </span>
57               </td>
58               <td className="border-b border-(--line2) px-3 py-2.5">
59                 <span className="block truncate text-[13px] font-semibold text-(--text)">
60                   {gateCategoryName(gate)}
61                 </span>
62               </td>
63               <td className="border-b border-(--line2) px-3 py-2.5 text-center">
64                 <button
65                   type="button"

```

```
66         onClick={() => onEdit?.(gate)}
67         className="inline-flex h-[30px] w-[30px] items-center justify-center rounded-[8px] border border-(--line) bg-(--surface) text-(--muted) transition hover:border-(--orange) hover:text-(--orange)"
68         aria-label={`Edit ${gateCategoryName(gate)}`}
69         title={`Edit ${gateCategoryName(gate)}`}
70     >
71         <EditIcon />
72     </button>
73 </td>
74 </tr>
75 )}
76 </tbody>
77 </table>
78 </div>
79 );
80 }
```

FILE: apps/dashboard/src/app/Config/components/Add_Device.js

```

1  "use client";
2
3  import { useEffect, useMemo, useRef, useState } from "react";
4  import { searchDeviceMasterlist } from "../../../lib/dashboardApi";
5  import { useDashboardStore } from "../../../store/dashboardStore";
6  import { SearchIcon } from "./ConfigIcons";
7
8  const DEVICE_TYPES = [
9    "MENU",
10   "TICKET MENU",
11   "INVOICE",
12   "SALE ITEMS",
13   "SALE KIOSK",
14   "INVENTORY COUNT",
15   "PICKUP",
16   "CHARGE",
17   "TOPUP CASH",
18   "TOPUP CARD",
19   "TOPUP COMPLIMENTARY",
20   "TOPUP ITEMS",
21   "ACCESS",
22   "TOPUP CORPORATE",
23   "TOPUP EMPLOYEE",
24   "TOPUP SPECIAL",
25   "TOPUP KIOSK",
26   "TOPUP KIOSK ONLINE",
27   "TOPUP KIOSK OFFLINE",
28   "TOPUP HOME DELIVERY",
29   "TOPUP PICKUP",
30   "DISCOUNT COUPON ISSUANCE",
31   "RETURN",
32   "MERGE",
33   "MASTER RETURN",
34   "ISSUANCE",
35   "STALL INFO SHARING"
36 ];
37
38 const PRINTER_OPTIONS = [
39   "NONE",
40   "WISEPOS+",
41   "EZETAP",
42   "MF919",
43   "MOSAMBEE | DX8000",
44   "WORLDLINE",
45   "SUNMIPAY",
46   "AIPAY",
47   "PINELABS",
48   "RUGTEK",
49   "BT-2INCH",
50   "SPRIN-3INCH",
51   "SPRIN-2INCH",
52   "PANDA-3INCH",
53   "USB-3INCH"
54 ];
55
56 function CloseIcon({ className = "h-3.5 w-3.5" }) {
57   return (
58     <svg viewBox="0 0 14 14" className={className} fill="none" stroke="currentColor" strokeWidth="1.6" strokeLinecap="round">
59       <path d="M3.5 3.5l7 7" />
60     </svg>
61   );
62 }
63
64 function InfoIcon() {
65   return (
66     <svg viewBox="0 0 24 24" className="h-4 w-4" fill="none" stroke="currentColor" strokeWidth="1.6" strokeLinecap="round" strokeLinejoin="round">
67       <circle cx="12" cy="12" r="9" />

```

```

68     <path d="M12 10v6" />
69     <path d="M12 7h.01" />
70   </svg>
71   );
72 }
73
74 function ChevronIcon() {
75   return (
76     <svg viewBox="0 0 12 12" className="h-2.5 w-2.5" fill="none" stroke="currentColor" strokeWidth="1.5"
strokeLinecap="round">
77       <path d="M2.5 4.5 6 8l3.5-3.5" />
78     </svg>
79   );
80 }
81
82 function SelectField({ label, value, onChange, options }) {
83   return (
84     <label className="block">
85       <span className="font-vcr mb-1.5 block text-[8.5px] uppercase tracking-[0.16em] text-(--muted)">{label}</span>
86       <span className="relative block">
87         <select
88           value={value}
89           onChange={(event) => onChange(event.target.value)}
90           className="h-11 w-full cursor-pointer appearance-none rounded-[10px] border border-(--line) bg-(
--surface) px-3.5 pr-9 text-[13px] font-medium uppercase tracking-[0.04em] text-(--text) outline-none tr
ansition focus:border-(--orange) focus:shadow-[0_0_3px_rgba(224,68,32,0.12)] max-[640px]:h-10"
91         >
92           {options.map((option) => (
93             <option key={option} value={option}>
94               {option}
95             </option>
96           ))}
97         </select>
98         <span className="pointer-events-none absolute inset-y-0 right-3.5 flex items-center text-(--muted
) opacity-60">
99           <ChevronIcon />
100         </span>
101       </span>
102     </label>
103   );
104 }
105
106 function DetailChip({ label, value }) {
107   return (
108     <div className="rounded-[10px] border border-(--line) bg-(--surface2) px-3.5 py-2.5">
109       <div className="font-vcr text-[7.5px] uppercase tracking-[0.14em] text-(--faint)">{label}</div>
110       <div className="mt-1 truncate text-[13px] font-semibold text-(--text)">{value || ""}</div>
111     </div>
112   );
113 }
114
115 function SectionDivider() {
116   return (
117     <div className="flex items-center gap-3 py-3 text-(--faint)">
118       <div className="h-px flex-1 bg-(--line2)" />
119       <span className="grid h-7 w-7 place-items-center rounded-full border border-(--line) bg-(--surface)
">
120         <InfoIcon />
121       </span>
122       <div className="h-px flex-1 bg-(--line2)" />
123     </div>
124   );
125 }
126
127 function getStallName(stall) {
128   return stall?.name ?? stall?.stallName ?? stall?.stall ?? "-";
129 }
130
131 function getVendorName(stall) {

```

```
132   return stall?.vendorName ?? stall?.vendor?.name ?? stall?.vendor ?? "-";
133 }
134
135 function getStallType(stall) {
136   return String(stall?.type ?? stall?.category ?? "-").toUpperCase();
137 }
138
139 function getPayMode(stall) {
140   const raw =
141     stall?.payOptions ??
142     stall?.pay_modes ??
143     stall?.payModes ??
144     stall?.paymentModes ??
145     stall?.payOption;
146   if (raw === 0 || raw === "0") return "ONLY NFC";
147   if (Array.isArray(raw)) return raw.join(", ").toUpperCase();
148   return String(raw || "ONLY NFC").replaceAll(",", " ").toUpperCase();
149 }
150
151 function getBankMode(stall) {
152   return String(stall?.bankOption ?? stall?.bankMode ?? stall?.bank ?? "NONE").toUpperCase();
153 }
154
155 function normalizeDeviceList(list) {
156   if (Array.isArray(list)) {
157     return list.filter((device) => device !== null && device !== undefined && String(device).trim() !== "")
158   };
159
160   if (typeof list === "string") {
161     return list
162       .split(",")
163       .map((device) => device.trim())
164       .filter(Boolean);
165   }
166
167   if (list === null || list === undefined || list === "") {
168     return [];
169   }
170
171   return [list];
172 }
173
174 function getStallDevices(stall) {
175   const list =
176     stall?.devices_list ??
177     stall?.device_list ??
178     stall?.devicesList ??
179     stall?.devices ??
180     stall?.device ??
181     [];
182   return normalizeDeviceList(list);
183 }
184
185 function getDeviceLabel(device) {
186   if (device === null || device === undefined) return "-";
187   if (typeof device === "string" || typeof device === "number") return String(device);
188   return String(
189     device.printId ??
190     device.deviceId ??
191     device.id ??
192     device.reference ??
193     device.androidId ??
194     device.hardwareId ??
195     "-"
196   );
197 }
198
199 function getDeviceKey(device) {
200   if (device === null || device === undefined) return "";
```

```

201   if (typeof device === "string" || typeof device === "number") return String(device);
202   return String(
203     device.id ??
204     device.printId ??
205     device.deviceId ??
206     device.reference ??
207     device.androidId ??
208     device.hardwareId ??
209     ""
210   );
211 }
212
213 function getDevicePayloadId(device) {
214   const value =
215     typeof device === "string" || typeof device === "number"
216       ? device
217       : device?.printId ?? device?.deviceId ?? device?.id ?? device?.reference;
218   if (value === null || value === undefined || value === "") return null;
219   const numericValue = Number(value);
220   return Number.isFinite(numericValue) ? numericValue : value;
221 }
222
223 function getStallId(stall) {
224   return stall?.id ?? stall?.stallId ?? stall?.stall_id ?? null;
225 }
226
227 function getSearchResultLabel(device) {
228   return [device?.printId, device?.description, device?.bank, device?.type].filter(Boolean).join(" | ");
229 }
230
231 function toApiTypeValue(value) {
232   return String(value ?? "")
233     .trim()
234     .toLowerCase()
235     .replace(/^[a-z0-9]+/g, "_")
236     .replace(/^[_+|_+$/g, "");
237 }
238
239 function toApiPrinterValue(value) {
240   return String(value ?? "")
241     .trim()
242     .toLowerCase()
243     .replace(/s*\|s*/g, "_")
244     .replace(/s+/g, "_");
245 }
246
247 export default function Add_Device({ stall, onClose, onConfirm }) {
248   const token = useDashboardStore((state) => state.token);
249   const [type, setType] = useState("RETURN");
250   const [printer, setPrinter] = useState("WISEPOS+");
251   const [query, setQuery] = useState("");
252   const [devices, setDevices] = useState([]);
253   const [selectedDevices, setSelectedDevices] = useState([]);
254   const [loading, setLoading] = useState(false);
255   const [error, setError] = useState("");
256   const [submitError, setSubmitError] = useState("");
257   const [submitting, setSubmitting] = useState(false);
258   const overlayRef = useRef(null);
259   const requestIdRef = useRef(0);
260   const stallDevices = useMemo(() => getStallDevices(stall), [stall]);
261
262   useEffect(() => {
263     const previousOverflow = document.body.style.overflow;
264     const handleKeyDown = (event) => {
265       if (event.key === "Escape") onClose?();
266     };
267     document.body.style.overflow = "hidden";
268     document.addEventListener("keydown", handleKeyDown);
269     return () => {
270       document.body.style.overflow = previousOverflow;

```

```
271     document.removeEventListener("keydown", handleKeyDown);
272   };
273 }, [onClose]);
274
275 useEffect(() => {
276   const search = query.trim();
277   requestIdRef.current += 1;
278   const requestId = requestIdRef.current;
279
280   if (search.length < 3) {
281     setDevices([]);
282     setLoading(false);
283     setError("");
284     return undefined;
285   }
286
287   setLoading(true);
288   setError("");
289   const timeoutId = window.setTimeout(async () => {
290     try {
291       const list = await searchDeviceMasterlist({ search, token, dedupe: false });
292       if (requestId === requestIdRef.current) {
293         setDevices(Array.isArray(list) ? list : []);
294       }
295     } catch (searchError) {
296       console.error("Device search failed", searchError);
297       if (requestId === requestIdRef.current) {
298         setDevices([]);
299         setError("Unable to search devices.");
300       }
301     } finally {
302       if (requestId === requestIdRef.current) {
303         setLoading(false);
304       }
305     }
306   }, 450);
307
308   return () => window.clearTimeout(timeoutId);
309 }, [query, token]);
310
311 const addSelectedDevice = (device) => {
312   const key = getDeviceKey(device);
313   if (!key) return;
314   setSelectedDevices((current) => {
315     if (current.some((item) => getDeviceKey(item) === key)) return current;
316     return [...current, device];
317   });
318 };
319
320 const removeSelectedDevice = (device) => {
321   const key = getDeviceKey(device);
322   setSelectedDevices((current) => current.filter((item) => getDeviceKey(item) !== key));
323 };
324
325 const handleConfirm = async () => {
326   const stallId = getStallId(stall);
327   const selectedDeviceIds = selectedDevices
328     .map(getDevicePayloadId)
329     .filter((deviceId) => deviceId !== null && deviceId !== undefined && deviceId !== "");
330
331   if (!stallId) {
332     setSubmitError("Missing stall ID.");
333     return;
334   }
335
336   if (selectedDeviceIds.length === 0) {
337     setSubmitError("Select at least one device.");
338     return;
339   }
340 }
```

```

341     setSubmitting(true);
342     setSubmitError("");
343
344     try {
345         await onConfirm?({
346             type: toApiTypeValue(type),
347             devices: selectedDeviceIds,
348             printer: toApiPrinterValue(printer),
349             stallId
350         });
351     } catch (confirmError) {
352         console.error("Add devices failed", confirmError);
353         setSubmitError("Unable to add devices.");
354     } finally {
355         setSubmitting(false);
356     }
357 };
358
359 return (
360     <div
361         ref={overlayRef}
362         onMouseDown={(event) => {
363             if (event.target === overlayRef.current) onClose?();
364         }}
365         className="fixed inset-0 z-[220] flex items-center justify-center bg-[rgba(12,12,12,0.5)] px-4 py-6
366         backdrop-blur-[3px] max-[640px]:p-0"
367     >
368         <div className="flex max-h-[calc(100dvh-48px)] w-full max-w-[820px] flex-col overflow-hidden rounded-
369         d-[16px] border border-(--line) bg-(--surface) shadow-(--shadowUp) max-[640px]:h-full max-[640px]:max-h-f
370         ull max-[640px]:rounded-none">
371             <div
372                 className="relative shrink-0 overflow-hidden px-6 py-4 max-[640px]:px-4"
373                 style={{ background: "linear-gradient(120deg,#1C1C1C 0%,#341CD6 62%,#E04420 130%)" }}
374             >
375                 <div
376                     className="pointer-events-none absolute inset-0 opacity-[0.12]"
377                     style={{ background: "repeating-linear-gradient(90deg,#fff 0 1px,transparent 1px 46px)" }}
378                     aria-hidden
379                 />
380                 <div className="relative flex items-start justify-between gap-3">
381                     <div className="min-w-0">
382                         <p className="font-vcr text-[9px] uppercase tracking-[0.2em] text-(--purple)">Add Devices</
383                         p>
384                         <h2 className="font-chillax m-0 mt-1 truncate text-[clamp(18px,2.3vw,25px)] font-semibold t
385                         racking-[-0.01em] text-white">
386                             {getStallName(stall)}
387                         </h2>
388                         <p className="m-0 mt-1 text-[12px] font-light text-white/70">Vendor: {getVendorName(stall)}
389                     </p>
390                 </div>
391                 <button
392                     type="button"
393                     onClick={onClose}
394                     className="grid h-8 w-8 shrink-0 place-items-center rounded-[9px] border border-white/20 te
395                     xt-white/80 transition hover:border-(--orange) hover:bg-(--orange)"
396                     aria-label="Close add device modal"
397                 >
398                     <CloseIcon />
399                 </button>
400             </div>
401             <div>
402                 <div className="min-h-0 flex-1 overflow-y-auto px-6 py-5 max-[640px]:px-4">
403                     <div className="grid gap-5 lg:grid-cols-[minmax(0,1fr)_290px]">
404                         <div className="space-y-4">
405                             <div className="grid gap-3 sm:grid-cols-2">
406                                 <SelectField label="Type" value={type} onChange={setType} options={DEVICE_TYPES} />
407                                 <SelectField label="Printer" value={printer} onChange={setPrinter} options={PRINTER_OPTIO
408                                 NS} />
409                             </div>
410                         </div>
411                     </div>
412                 </div>
413             </div>
414         </div>
415     );

```



```

403
404         <label className="block">
405             <span className="font-vcr mb-1.5 block text-[8.5px] uppercase tracking-[0.16em] text-(--m
uted)">
406                 Devices ({selectedDevices.length})
407             </span>
408             <div className="flex h-11 items-center gap-2 rounded-[10px] border border-(--line) bg-(--
surface) px-3.5 text-(--muted) transition focus-within:border-(--orange) max-[640px]:h-10">
409                 <SearchIcon className="h-4 w-4 shrink-0 opacity-60" />
410                 <input
411                     value={query}
412                     onChange={(event) => setQuery(event.target.value)}
413                     placeholder="Type 3 digits to search"
414                     className="h-full min-w-0 flex-1 bg-transparent text-[13px] font-medium text-(--text)
outline-none placeholder:text-(--faint)"
415                 />
416             </div>
417         </label>
418
419         {selectedDevices.length > 0 && (
420             <div className="flex flex-wrap gap-2 rounded-[12px] border border-(--line) bg-(--surface2
) p-2.5">
421                 {selectedDevices.map((device) => (
422                     <span
423                         key={getDeviceKey(device)}
424                         className="font-vcr inline-flex h-8 items-center gap-2 rounded-full border border-(
--line) bg-(--surface) pl-3 pr-1.5 text-[11px] text-(--text)"
425                     >
426                         {getDeviceLabel(device)}
427                     <button
428                         type="button"
429                         onClick={() => removeSelectedDevice(device)}
430                         className="grid h-5 w-5 place-items-center rounded-full bg-(--text) text-(--bg) t
ransition hover:bg-(--orange)"
431                         aria-label={`Remove device ${getDeviceLabel(device)}`}
432                     >
433                         <CloseIcon className="h-2.5 w-2.5" />
434                     </button>
435                     </span>
436                 )]}
437             </div>
438         )}
439
440         {query.trim().length >= 3 && (
441             <div className="rounded-[12px] border border-(--line) bg-(--surface2) p-3">
442                 {loading ? (
443                     <p className="m-0 text-[13px] font-medium text-(--muted)">Searching devices...</p>
444                 ) : error ? (
445                     <p className="m-0 text-[13px] font-semibold text-(--orange)">{error}</p>
446                 ) : devices.length ? (
447                     <div className="grid max-h-[180px] gap-2 overflow-y-auto pr-1">
448                         {devices.map((device, index) => {
449                             const key = device?.id ?? device?.printId ?? device?.androidId ?? index;
450                             const active = selectedDevices.some((item) => getDeviceKey(item) === getDeviceKey
(device));
451                             return (
452                                 <button
453                                     key={key}
454                                     type="button"
455                                     onClick={() => addSelectedDevice(device)}
456                                     className={`rounded-[10px] border px-3 py-2 text-left text-[13px] font-semibo
ld transition ${{
457                                         active
458                                             ? "border-(--orange) bg-[rgba(224,68,32,0.06)] text-(--text)"
459                                             : "border-(--line) bg-(--surface) text-(--muted) hover:border-(--orange)"
460                                         }}
461                                     >
462                                     <span className="flex items-center justify-between gap-3">
463                                         <span>{getSearchResultLabel(device) || getDeviceLabel(device)}</span>
464                                         <span className="font-vcr shrink-0 text-[9px] uppercase tracking-[0.12em] t

```

```

    ext(--orange)">
465         {active ? "Selected" : "Add"}
466       </span>
467     </span>
468   </button>
469   );
470   }}}}
471 </div>
472 ) : (
473   <p className="m-0 text-[13px] font-medium text-(--muted)">No matching devices found.<
/p>
474   )}
475 </div>
476 )}
477
478 {submitError && (
479   <p className="m-0 rounded-[10px] border border-[rgba(224,68,32,0.25)] bg-[rgba(224,68,32,
0.06)] px-3 py-2 text-[12px] font-semibold text-(--orange)">
480     {submitError}
481   </p>
482 )}
483 </div>
484
485 <div className="rounded-[12px] border border-(--line) bg-(--surface) p-4 shadow-(--shadow) ma
x-[640px]:p-3">
486   <h3 className="font-chillax m-0 text-[14px] font-semibold text-(--text)">Details</h3>
487   <div className="mt-3 grid gap-2">
488     <DetailChip label="STALL TYPE" value={getStallType(stall)} />
489     <DetailChip label="PAY MODES" value={getPayMode(stall)} />
490     <DetailChip label="BANK MODE" value={getBankMode(stall)} />
491   </div>
492 </div>
493 </div>
494
495 <SectionDivider />
496
497 <section>
498   <h3 className="font-vcr m-0 text-[10px] uppercase tracking-[0.15em] text-(--muted)">
499     Stall Devices ( {stallDevices.length} )
500   </h3>
501   <div className="mt-3 flex flex-wrap gap-2">
502     {stallDevices.length ? (
503       stallDevices.map((device, index) => (
504         <span
505           key={` ${getDeviceLabel(device)}-${index}`}
506           className="font-vcr rounded-full border border-(--line) bg-(--surface2) px-3.5 py-1.5
text-[11px] text-(--muted)"
507         >
508           {getDeviceLabel(device)}
509         </span>
510       )
511     ) : (
512       <span className="text-[13px] font-medium text-(--faint)">No stall devices assigned.</span>
513     )}
514   </div>
515 </section>
516 </div>
517
518 <div className="flex shrink-0 justify-end gap-2.5 border-t border-(--line) bg-(--surface2) px-6 p
y-4 max-[640px]:grid max-[640px]:grid-cols-2 max-[640px]:px-4">
519   <button
520     type="button"
521     onClick={onClose}
522     className="flex h-11 items-center justify-center rounded-[10px] border border-(--line) bg-(--
surface) px-6 text-[13.5px] font-semibold text-(--muted) transition hover:border-(--orange) hover:text-(
-orange)"
523   >
524     Cancel
525   </button>

```

```
526         <button
527             type="button"
528             onClick={handleConfirm}
529             disabled={submitting}
530             className="flex h-11 items-center justify-center rounded-[10px] bg-(--text) px-8 text-[13.5px
] font-semibold text-(--bg) transition hover:bg-(--orange) disabled:cursor-not-allowed disabled:opacity-6
0"
531         >
532             {submitting ? "Adding..." : "Confirm"}
533         </button>
534     </div>
535 </div>
536 </div>
537 );
538 }
```

FILE: apps/dashboard/src/app/Config/components/ConfigIcons.js

```

1  "use client";
2
3  const baseProps = {
4    fill: "none",
5    stroke: "currentColor",
6    strokeWidth: "2",
7    strokeLinecap: "round",
8    strokeLinejoin: "round",
9    "aria-hidden": true
10 };
11
12 export function SearchIcon({ className = "h-5 w-5" }) {
13   return (
14     <svg viewBox="0 0 24 24" className={className} {...baseProps}>
15       <circle cx="11" cy="11" r="7" />
16       <path d="m20 20-3.5-3.5" />
17     </svg>
18   );
19 }
20
21 export function PlusIcon({ className = "h-4 w-4" }) {
22   return (
23     <svg viewBox="0 0 24 24" className={className} {...baseProps}>
24       <path d="M12 5v14" />
25       <path d="M5 12h14" />
26     </svg>
27   );
28 }
29
30 export function GridIcon({ className = "h-4 w-4" }) {
31   return (
32     <svg viewBox="0 0 24 24" className={className} {...baseProps}>
33       <rect x="4" y="4" width="6" height="6" rx="1.4" />
34       <rect x="14" y="4" width="6" height="6" rx="1.4" />
35       <rect x="4" y="14" width="6" height="6" rx="1.4" />
36       <rect x="14" y="14" width="6" height="6" rx="1.4" />
37     </svg>
38   );
39 }
40
41 export function LinkIcon({ className = "h-4 w-4" }) {
42   return (
43     <svg viewBox="0 0 24 24" className={className} {...baseProps}>
44       <path d="M10 13a5 5 0 0 7.07 0l2.12-2.12a5 5 0 0 0-7.07-7.07L11 4.93" />
45       <path d="M14 11a5 5 0 0 0-7.07 0L4.81 13.12a5 5 0 0 7.07 7.07L13 19.07" />
46     </svg>
47   );
48 }
49
50 export function EditIcon({ className = "h-4 w-4" }) {
51   return (
52     <svg viewBox="0 0 24 24" className={className} {...baseProps}>
53       <path d="M12 20h9" />
54       <path d="M16.5 3.5a2.1 2.1 0 0 1 3 3L7 19l-4 1 1-4 12.5-12.5Z" />
55     </svg>
56   );
57 }
58
59 export function DeviceIcon({ className = "h-4 w-4" }) {
60   return (
61     <svg viewBox="0 0 24 24" className={className} {...baseProps}>
62       <rect x="7" y="3" width="10" height="18" rx="2" />
63       <path d="M11 17h2" />
64     </svg>
65   );
66 }
67
68 export function ListIcon({ className = "h-4 w-4" }) {
69   return (

```

```
70     <svg viewBox="0 0 24 24" className={className} {...baseProps}>
71       <path d="M8 6h12" />
72       <path d="M8 12h12" />
73       <path d="M8 18h12" />
74       <path d="M4 6h.01" />
75       <path d="M4 12h.01" />
76       <path d="M4 18h.01" />
77     </svg>
78   );
79 }
```

FILE: apps/dashboard/src/app/Config/components/ConfigTransition.js

```
1 "use client";
2
3 import { useEffect, useState } from "react";
4
5 export default function ConfigTransition({ children }) {
6   const [mounted, setMounted] = useState(false);
7
8   useEffect(() => {
9     if (mounted) return;
10    const timer = setTimeout(() => setMounted(true), 20);
11    return () => clearTimeout(timer);
12  }, [mounted]);
13
14  return (
15    <div
16      // NOTE: once mounted we drop the transform utilities entirely. A lingering
17      // non-none `transform` here would make any `position: fixed` descendant
18      // (the Config/Menu modals) resolve against this box instead of the
19      // viewport, dropping them to the bottom of the page.
20      className={`transition-all duration-300 ease-out ${
21        mounted ? "opacity-100" : "opacity-100 translate-x-6 scale-[0.98]"
22      }`}
23    >
24      {children}
25    </div>
26  );
27 }
```

FILE: apps/dashboard/src/app/Config/components/CreateAccessGateMasterModal.js

```

1  "use client";
2
3  import { useEffect, useState } from "react";
4
5  function CloseIcon() {
6    return (
7      <svg viewBox="0 0 14 14" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="1.6"
strokeLinecap="round" aria-hidden="true">
8        <path d="M3.5 3.5l7 7M10.5 3.5l-7 7" />
9      </svg>
10   );
11 }
12
13 export default function CreateAccessGateMasterModal({ onClose, onConfirm }) {
14   const [name, setName] = useState("");
15   const [saving, setSaving] = useState(false);
16   const [error, setError] = useState("");
17
18   useEffect(() => {
19     const handleKeyDown = (event) => {
20       if (event.key === "Escape" && !saving) onClose?();
21     };
22     document.addEventListener("keydown", handleKeyDown);
23     return () => document.removeEventListener("keydown", handleKeyDown);
24   }, [onClose, saving]);
25
26   const handleSubmit = async (event) => {
27     event.preventDefault();
28     const normalizedName = name.trim();
29     if (!normalizedName) return;
30
31     setSaving(true);
32     setError("");
33     try {
34       await onConfirm?({ name: normalizedName });
35     } catch {
36       setError("Unable to create this gate master. Please try again.");
37     } finally {
38       setSaving(false);
39     }
40   };
41
42   return (
43     <div
44       className="fixed inset-0 z-[200] flex items-center justify-center bg-[rgba(12,12,12,0.5)] p-4 backd
rop-blur-[3px]"
45     >
46       <div
47         onMouseDown={(event) => {
48           if (event.target === event.currentTarget && !saving) onClose?();
49         }}
50       >
51         <section
52           role="dialog"
53           aria-modal="true"
54           aria-labelledby="create-access-gate-master-title"
55           className="w-full max-w-[440px] overflow-hidden rounded-[16px] border border-(--line) bg-(--surfa
ce) shadow-(--shadowUp)"
56         >
57           <header
58             className="relative overflow-hidden px-5 py-4"
59             style={{ background: "linear-gradient(120deg,#1C1C1C 0%,#341CD6 62%,#E04420 130%)" }}
60           >
61             <div className="relative flex items-start justify-between gap-3">
62               <div>
63                 <p className="font-vcr text-[9px] uppercase tracking-[0.2em] text-(--purple)">AccessX</p>
64                 <h2 id="create-access-gate-master-title" className="font-chillax mt-1 text-[clamp(18px,2.2v
w,24px)] font-semibold tracking-[-0.01em] text-white">
65                   Add Gate Master
66                 </h2>
67               </div>
68             </div>
69           </header>
70         </section>
71       </div>
72     </div>
73   );
74 }

```

```

66         <button
67             type="button"
68             onClick={onClose}
69             disabled={saving}
70             className="grid h-8 w-8 shrink-0 place-items-center rounded-[9px] border border-white/20 te
xt-white/80 transition hover:border-(--orange) hover:bg-(--orange) disabled:opacity-40"
71             aria-label="Close gate master form"
72         >
73             <CloseIcon />
74         </button>
75     </div>
76 </header>
77
78     <form onSubmit={handleSubmit} className="p-5 max-[640px]:p-4">
79         <label className="block">
80             <span className="font-vcr mb-1.5 block text-[8.5px] uppercase tracking-[0.16em] text-(--muted
)">Name</span>
81             <input
82                 autoFocus
83                 value={name}
84                 onChange={(event) => setName(event.target.value)}
85                 placeholder="Enter gate master name"
86                 className="h-11 w-full rounded-[10px] border border-(--line) bg-(--surface) px-3.5 text-[13
px] font-medium text-(--text) outline-none transition placeholder:text-(--faint) focus:border-(--orange)
focus:shadow-[0_0_3px_rgba(224,68,32,0.12)]"
87             />
88         </label>
89
90         {error ? (
91             <p className="mt-3 rounded-[10px] border border-[rgba(224,68,32,0.25)] bg-[rgba(224,68,32,0.0
6)] px-3 py-2 text-[12px] font-semibold text-(--orange)">
92                 {error}
93             </p>
94         ) : null}
95
96         <div className="mt-5 flex justify-end gap-2.5 border-t border-(--line) pt-4 max-[640px]:grid ma
x-[640px]:grid-cols-2">
97             <button
98                 type="button"
99                 onClick={onClose}
100                 disabled={saving}
101                 className="flex h-11 items-center justify-center rounded-[10px] border border-(--line) bg-(
--surface) px-5 text-[13px] font-semibold text-(--muted) transition hover:border-(--orange) hover:text-(
--orange) disabled:opacity-50"
102             >
103                 Cancel
104             </button>
105             <button
106                 type="submit"
107                 disabled={saving || !name.trim()}
108                 className="flex h-11 items-center justify-center rounded-[10px] bg-(--text) px-6 text-[13px
] font-semibold text-(--bg) transition hover:bg-(--orange) disabled:cursor-not-allowed disabled:opacity-5
0"
109             >
110                 {saving ? "Creating..." : "Confirm"}
111             </button>
112         </div>
113     </form>
114 </section>
115 </div>
116 );
117 }

```


FILE: apps/dashboard/src/app/Config/components/CreateStallModal.js

```

1  "use client";
2
3  import { useEffect, useRef, useState } from "react";
4
5  const STALL_TYPES = ["TOPUP", "SALE", "ACCESSX", "INVENTORY", "STOCKMASTER", "TABLES"];
6  const SCAN_MODES = ["NONE", "MENU", "TICKET"];
7  const PAYMENT_MODE_OPTIONS = ["cash", "card", "coupon"];
8  const NFC_SETTINGS = ["LOGIC"];
9  const QR_SETTINGS = ["OFF"];
10
11 const DEFAULT_FORM = {
12   stallName: "",
13   type: "SALE",
14   acceptAllModes: false,
15   paymentModes: ["cash", "card", "coupon"],
16   grnMode: false,
17   cashDisabled: false,
18   kotLan: false,
19   modeInfoMandatory: false,
20   scanMode: "NONE",
21   showInTapX: false,
22   showPrice: false,
23   addToCart: false,
24   nfcSetting: "LOGIC",
25   qrSetting: "OFF",
26   locationId: "",
27   eventMatchId: "",
28   useOnlineTopups: false,
29   useInOutLogic: false,
30   fetchDetails: false,
31   checkOnlineUnique: false,
32 };
33
34 const FIELD_CLASS =
35   "h-11 w-full rounded-[10px] border border-(--line) bg-(--surface) px-3.5 text-[13px] font-medium text-(
36     --text) outline-none transition placeholder:text-(--faint) focus:border-(--orange) focus:shadow-[0_0_3p
37     x_rgba(224,68,32,0.12)] max-[640px]:h-10";
38
39 function CloseIcon() {
40   return (
41     <svg viewBox="0 0 14 14" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="1.6"
42       strokeLinecap="round">
43       <path d="M3.5 3.5l7 7" />
44     </svg>
45   );
46 }
47
48 function ChevronIcon() {
49   return (
50     <svg viewBox="0 0 24 24" className="h-4 w-4 shrink-0" fill="none" stroke="currentColor" strokeWidth="
51       2" strokeLinecap="round" strokeLinejoin="round">
52       <path d="m6 9 6 6 6-6" />
53     </svg>
54   );
55 }
56
57 function TapXIcon() {
58   return (
59     <svg viewBox="0 0 24 24" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="1.6"
60       strokeLinecap="round" strokeLinejoin="round">
61       <path d="M12 3v4" />
62       <path d="M12 17v4" />
63       <path d="M3 12h4" />
64       <path d="M17 12h4" />
65       <circle cx="12" cy="12" r="3" />
66     </svg>
67   );
68 }

```

```

65 function AccessXIcon() {
66   return (
67     <svg viewBox="0 0 24 24" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="1.6"
strokeLinecap="round" strokeLinejoin="round">
68       <path d="M12 3 4 7v6c0 5 3.4 7.4 8 8 4.6-.6 8-3 8-8V7l-8-4Z" />
69       <path d="m9 12 2 2 4-5" />
70     </svg>
71   );
72 }
73
74 function CheckIcon() {
75   return (
76     <svg viewBox="0 0 24 24" className="h-3 w-3" fill="none" stroke="currentColor" strokeWidth="3" stroke
Linecap="round" strokeLinejoin="round">
77       <path d="m5 12 4 4L19 6" />
78     </svg>
79   );
80 }
81
82 function SegmentedChoice({ active, leftLabel, rightLabel, onChange }) {
83   return (
84     <div className="grid h-11 grid-cols-2 overflow-hidden rounded-full border border-(--line) bg-(--surfa
ce2) p-1 max-[640px]:h-10">
85       <button
86         type="button"
87         onClick={() => onChange(false)}
88         className={`rounded-full px-3 text-[12.5px] font-semibold transition ${
89           !active ? "bg-(--surface) text-(--orange) shadow-sm" : "text-(--muted) hover:text-(--text)"
90         }`}
91       >
92         {leftLabel}
93       </button>
94       <button
95         type="button"
96         onClick={() => onChange(true)}
97         className={`rounded-full px-3 text-[12.5px] font-semibold transition ${
98           active ? "bg-[linear-gradient(135deg,#E04420,#341CD6)] text-white" : "text-(--muted) hover:text
99           -(--text)"
100         }`}
101       >
102         {rightLabel}
103       </button>
104     </div>
105   );
106 }
107
108 function SettingChip({ label, active, onToggle }) {
109   return (
110     <button
111       type="button"
112       onClick={onToggle}
113       aria-pressed={active}
114       className={`flex h-11 items-center justify-between gap-3 rounded-[10px] border px-3.5 text-left tra
115       nsition max-[640px]:h-10 ${
116         active
117         ? "border-(--orange) bg-[rgba(224,68,32,0.06)] text-(--text)"
118         : "border-(--line) bg-(--surface) text-(--muted) hover:border-(--orange)"
119       }`}
120     >
121       <span className="truncate text-[12.5px] font-semibold">{label}</span>
122       <span
123         className={`flex h-5 w-5 shrink-0 items-center justify-center rounded-md border transition ${
124           active ? "border-(--orange) bg-(--orange) text-white" : "border-(--line) bg-(--surface2) text-t
ransparent"
125         }`}
126       >
127         <CheckIcon />
128       </span>
129     </button>
130   );

```

```

129 }
130
131 function PaymentModeOptions({ selected, onToggle }) {
132   return (
133     <div className="grid grid-cols-3 gap-2.5 max-sm:grid-cols-1">
134       {PAYMENT_MODE_OPTIONS.map((mode) => {
135         const active = selected.includes(mode);
136         const label = mode.charAt(0).toUpperCase() + mode.slice(1);
137         return <SettingChip key={mode} label={label} active={active} onToggle={() => onToggle(mode)} />;
138       })}
139     </div>
140   );
141 }
142
143 function StallDropdown({ value, onChange, options }) {
144   const [open, setOpen] = useState(false);
145   const ref = useRef(null);
146
147   useEffect(() => {
148     const handler = (e) => {
149       if (!ref.current?.contains(e.target)) setOpen(false);
150     };
151     if (open) document.addEventListener("mousedown", handler);
152     return () => document.removeEventListener("mousedown", handler);
153   }, [open]);
154
155   return (
156     <div ref={ref} className="relative">
157       <button
158         type="button"
159         onClick={() => setOpen((prev) => !prev)}
160         className="flex h-11 w-full items-center justify-between rounded-[10px] bg-[linear-gradient(135deg,#E04420,#341CD6)] px-4 text-[13px] font-semibold tracking-[0.06em] text-white transition hover:brightness-105 max-[640px]:h-10"
161       >
162         <span>{value}</span>
163         <ChevronIcon />
164       </button>
165       {open && (
166         <div className="absolute left-0 top-full z-50 mt-1.5 w-full overflow-hidden rounded-[11px] border border-(--line) bg-(--surface) shadow-(--shadowUp)">
167           {options.map((option) => (
168             <button
169               key={option}
170               type="button"
171               onClick={() => {
172                 onChange(option);
173                 setOpen(false);
174               }}
175               className={`block w-full px-4 py-2.5 text-left text-[13px] font-semibold tracking-[0.04em] transition hover:bg-(--surface2) ${
176                 value === option ? "text-(--orange)" : "text-(--text)"
177               }`}
178             >
179               {option}
180             </button>
181           ))}
182         </div>
183       )}
184     </div>
185   );
186 }
187
188 function SectionDivider({ icon, label }) {
189   return (
190     <div className="flex items-center gap-3 py-1.5">
191       <div className="h-px flex-1 bg-gradient-to-r from-transparent via-(--orange)/25 to-(--purple)/40" /
192       >
193       <span className="font-vcr flex items-center gap-1.5 text-[9px] uppercase tracking-[0.16em] text-(--muted)">

```

```

193     {icon}
194     {label}
195   </span>
196   <div className="h-px flex-1 bg-gradient-to-r from-(--purple)/40 via-(--orange)/25 to-transparent" /
197 >
198   </div>
199   );
200 }
201 function FormRow({ label, children }) {
202   return (
203     <div className="grid grid-cols-[170px_1fr] items-center gap-4 max-sm:grid-cols-1 max-sm:gap-1.5">
204       <span className="text-right text-[13px] font-medium text-(--muted) max-sm:text-left">
205         {label}
206       </span>
207       <div>{children}</div>
208     </div>
209   );
210 }
211
212 function SettingsGrid({ children }) {
213   return <div className="grid grid-cols-2 gap-2.5 max-sm:grid-cols-1">{children}</div>;
214 }
215
216 export default function CreateStallModal({ vendorName = "", vendorType = "", onClose, onConfirm }) {
217   const [form, setForm] = useState({ ...DEFAULT_FORM, stallName: vendorName });
218   const overlayRef = useRef(null);
219   const isAccessXVendor =
220     String(vendorType || "").toUpperCase() === "ACCESSX" ||
221     String(vendorName || "").toUpperCase() === "ACCESSX";
222
223   const set = (key, value) => setForm((prev) => ({ ...prev, [key]: value }));
224   const setAcceptAllModes = (value) => {
225     setForm((prev) => ({
226       ...prev,
227       acceptAllModes: value,
228       paymentModes: value ? [...PAYMENT_MODE_OPTIONS] : prev.paymentModes,
229     }));
230   };
231   const togglePaymentMode = (mode) => {
232     setForm((prev) => {
233       const hasMode = prev.paymentModes.includes(mode);
234       if (hasMode && prev.paymentModes.length === 1) return prev;
235       return {
236         ...prev,
237         paymentModes: hasMode
238           ? prev.paymentModes.filter((item) => item !== mode)
239           : [...prev.paymentModes, mode],
240       };
241     });
242   };
243   const toggleTapX = () => {
244     setForm((prev) => ({
245       ...prev,
246       showInTapX: !prev.showInTapX,
247       showPrice: prev.showInTapX ? false : prev.showPrice,
248       addToCart: prev.showInTapX ? false : prev.addToCart,
249     }));
250   };
251   const toggleShowPrice = () => {
252     setForm((prev) => ({
253       ...prev,
254       showPrice: !prev.showPrice,
255       addToCart: prev.showPrice ? false : prev.addToCart,
256     }));
257   };
258
259   useEffect(() => {
260     const handler = (e) => {
261       if (e.key === "Escape") onClose?();

```

```

262     };
263     const previousOverflow = document.body.style.overflow;
264
265     document.body.style.overflow = "hidden";
266     document.addEventListener("keydown", handler);
267     return () => {
268         document.body.style.overflow = previousOverflow;
269         document.removeEventListener("keydown", handler);
270     };
271 }, [onClose]);
272
273 return (
274     <div
275         ref={overlayRef}
276         onMouseDown={(e) => {
277             if (e.target === overlayRef.current) onClose?();
278         }}
279         className="fixed inset-0 z-[200] flex items-center justify-center overscroll-none bg-[rgba(12,12,12
,0.5)] px-4 py-6 backdrop-blur-[3px] max-[640px]:p-0"
280     >
281         <div className="flex max-h-[calc(100dvh-48px)] w-full max-w-[760px] flex-col overflow-hidden rounded-[16px] border border-(--line) bg-(--surface) shadow-(--shadowUp) max-[640px]:h-full max-[640px]:max-h-full max-[640px]:rounded-none">
282             <div
283                 className="relative shrink-0 overflow-hidden px-7 py-4 max-[640px]:px-4"
284                 style={{ background: "linear-gradient(120deg,#1C1C1C 0%,#341CD6 62%,#E04420 130%)" }}
285             >
286                 <div
287                     className="pointer-events-none absolute inset-0 opacity-[0.12]"
288                     style={{ background: "repeating-linear-gradient(90deg,#fff 0 1px,transparent 1px 46px)" }}
289                     aria-hidden
290                 />
291                 <div className="relative flex items-start justify-between gap-3">
292                     <div className="min-w-0">
293                         <p className="font-vcr m-0 text-[9px] uppercase tracking-[0.2em] text-(--purple)">Stall Set
up</p>
294                         <h2 className="font-chillax m-0 mt-1 truncate text-[clamp(20px,2.4vw,26px)] font-semibold tracking-[-0.01em] text-white">
                Create Stall{vendorName ? ` for ${vendorName}` : ""}
295
296                     </h2>
297                 </div>
298                 <button
299                     type="button"
300                     onClick={onClose}
301                     className="grid h-8 w-8 shrink-0 place-items-center rounded-[9px] border border-white/20 text-white/80 transition hover:border-(--orange) hover:bg-(--orange)"
302                     aria-label="Close create stall modal"
303                 >
304                     <CloseIcon />
305                 </button>
306             </div>
307         </div>
308
309         <div className="min-h-0 flex-1 space-y-3.5 overflow-y-auto px-7 py-5 max-[640px]:px-4">
310             <FormRow label="Stall Name">
311                 <input type="text" value={form.stallName} onChange={(e) => set("stallName", e.target.value)}
className={FIELD_CLASS} />
312             </FormRow>
313
314             <FormRow label="Type">
315                 <StallDropdown value={form.type} onChange={(value) => set("type", value)} options={STALL_TYPE
S} />
316             </FormRow>
317
318             <FormRow label="Payment Modes">
319                 <SegmentedChoice active={form.acceptAllModes} leftLabel="Only NFC" rightLabel="Accept All Mod
es" onChange={setAcceptAllModes} />
320             </FormRow>
321
322             {form.acceptAllModes && (

```

```

323         <FormRow label="Mode Options">
324             <PaymentModeOptions selected={form.paymentModes} onToggle={togglePaymentMode} />
325         </FormRow>
326     )}
327
328     <FormRow label="Stall Settings">
329         <SettingsGrid>
330             <SettingChip label="GRN Mode" active={form.grnMode} onToggle={() => set("grnMode", !form.gr
nMode)} />
331             <SettingChip label="Cash Disabled" active={form.cashDisabled} onToggle={() => set("cashDisa
bled", !form.cashDisabled)} />
332             <SettingChip label="KOT LAN" active={form.kotLan} onToggle={() => set("kotLan", !form.kotLa
n)} />
333             <SettingChip label="Mode Info Mandatory" active={form.modeInfoMandatory} onToggle={() => se
t("modeInfoMandatory", !form.modeInfoMandatory)} />
334         </SettingsGrid>
335     </FormRow>
336
337     <FormRow label="Scan Mode">
338         <StallDropdown value={form.scanMode} onChange={(value) => set("scanMode", value)} options={SC
AN_MODES} />
339     </FormRow>
340
341     <SectionDivider icon={<TapXIcon />} label="TapX Items" />
342
343     <FormRow label="TapX">
344         <SettingsGrid>
345             <SettingChip label="Show in TapX" active={form.showInTapX} onToggle={toggleTapX} />
346             {form.showInTapX && (
347                 <SettingChip label="Show Price" active={form.showPrice} onToggle={toggleShowPrice} />
348             )}
349             {form.showInTapX && form.showPrice && (
350                 <SettingChip label="Add to Cart" active={form.addToCart} onToggle={() => set("addToCart",
!form.addToCart)} />
351             )}
352         </SettingsGrid>
353     </FormRow>
354
355     {isAccessXVendor && (
356         <>
357             <SectionDivider icon={<AccessXIcon />} label="AccessX Settings" />
358
359             <div className="grid grid-cols-2 gap-3 max-sm:grid-cols-1">
360                 <FormRow label="NFC Setting">
361                     <StallDropdown value={form.nfcSetting} onChange={(value) => set("nfcSetting", value)} o
ptions={NFC_SETTINGS} />
362                 </FormRow>
363                 <FormRow label="QR Setting">
364                     <StallDropdown value={form.qrSetting} onChange={(value) => set("qrSetting", value)} opt
ions={QR_SETTINGS} />
365                 </FormRow>
366             </div>
367
368             <div className="grid grid-cols-2 gap-3 max-sm:grid-cols-1">
369                 <FormRow label="Location ID">
370                     <input type="text" value={form.locationId} onChange={(e) => set("locationId", e.target.
value)} className={FIELD_CLASS} />
371                 </FormRow>
372                 <FormRow label="Event/Match ID">
373                     <input type="text" value={form.eventMatchId} onChange={(e) => set("eventMatchId", e.tar
get.value)} className={FIELD_CLASS} />
374                 </FormRow>
375             </div>
376
377             <FormRow label="Access Controls">
378                 <SettingsGrid>
379                     <SettingChip label="Use Online Topups" active={form.useOnlineTopups} onToggle={() => se
t("useOnlineTopups", !form.useOnlineTopups)} />
380                     <SettingChip label="Use In-Out Logic" active={form.useInOutLogic} onToggle={() => set("
useInOutLogic", !form.useInOutLogic)} />

```

```

381         <SettingChip label="Fetch Details" active={form.fetchDetails} onToggle={() => set("fetc
hDetails", !form.fetchDetails)} />
382         <SettingChip label="Check Online Unique" active={form.checkOnlineUnique} onToggle={() =
> set("checkOnlineUnique", !form.checkOnlineUnique)} />
383     </SettingsGrid>
384 </FormRow>
385 </>
386     )}
387 </div>
388
389     <div className="flex shrink-0 items-center justify-end gap-2.5 border-t border-(--line) bg(--sur
face2) px-7 py-4 max-[640px]:grid max-[640px]:grid-cols-2 max-[640px]:px-4">
390         <button
391             type="button"
392             onClick={onClose}
393             className="flex h-11 items-center justify-center rounded-[10px] border border-(--line) bg(--
surface) px-6 text-[13.5px] font-semibold text-(--muted) transition hover:border-(--orange) hover:text-(
-orange)"
394             >
395             Cancel
396         </button>
397         <button
398             type="button"
399             onClick={() => onConfirm?.(form)}
400             className="flex h-11 items-center justify-center rounded-[10px] bg-(--text) px-7 text-[13.5px
] font-semibold text-(--bg) transition hover:bg-(--orange)"
401             >
402             Confirm
403         </button>
404     </div>
405 </div>
406 </div>
407 );
408 }

```

FILE: apps/dashboard/src/app/Config/components/CreateVendorModal.js

```

1  "use client";
2
3  import { useEffect, useRef, useState } from "react";
4  import {
5    buildVendorPayload,
6    DEFAULT_VENDOR_FORM,
7    numericOnly,
8    VENDOR_TYPES
9  } from "../vendorPayload";
10
11  const FIELD_CLASS =
12    "h-11 w-full rounded-[10px] border border-(--line) bg-(--surface) px-3.5 text-[13px] font-medium text-(
    --text) outline-none transition placeholder:text-(--faint) focus:border-(--orange) focus:shadow-[0_0_3px
    x_rgba(224,68,32,0.12)] max-[640px]:h-10";
13
14  function CloseIcon() {
15    return (
16      <svg viewBox="0 0 14 14" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="1.6"
    strokeLinecap="round">
17        <path d="M3.5 3.5l7 7" />
18      </svg>
19    );
20  }
21
22  function ChevronIcon() {
23    return (
24      <svg viewBox="0 0 24 24" className="h-4 w-4 shrink-0" fill="none" stroke="currentColor" strokeWidth="
    2" strokeLinecap="round" strokeLinejoin="round">
25        <path d="m6 9 6 6 6-6" />
26      </svg>
27    );
28  }
29
30  function EyeIcon({ hidden }) {
31    if (hidden) {
32      return (
33        <svg viewBox="0 0 24 24" className="h-[17px] w-[17px]" fill="none" stroke="currentColor" strokeWidt
    h="2" strokeLinecap="round" strokeLinejoin="round">
34          <path d="M17.94 17.94A10.07 10.07 0 0 1 12 20c-7 0-11-8-11-8a18.45 18.45 0 0 1 5.06-5.94" />
35          <path d="M9.9 4.24A9.12 9.12 0 0 1 12 4c7 0 11 8 11 8a18.5 18.5 0 0 1-2.16 3.19" />
36          <path d="M1 1l22 22" />
37        </svg>
38      );
39    }
40
41    return (
42      <svg viewBox="0 0 24 24" className="h-[17px] w-[17px]" fill="none" stroke="currentColor" strokeWidth=
    "2" strokeLinecap="round" strokeLinejoin="round">
43        <path d="M1 12s4-8 11-8 11 8 11 8-4 8-11 8-11-8-11-8z" />
44        <circle cx="12" cy="12" r="3" />
45      </svg>
46    );
47  }
48
49  function SectionDivider({ icon, label }) {
50    return (
51      <div className="flex items-center gap-3 py-1.5">
52        <div className="h-px flex-1 bg-gradient-to-r from-transparent via-(--orange)/25 to-(--purple)/40" /
    >
53        <span className="font-vcr flex items-center gap-1.5 text-[9px] uppercase tracking-[0.16em] text-(--
    muted)">
54          {icon}
55          {label}
56        </span>
57        <div className="h-px flex-1 bg-gradient-to-r from-(--purple)/40 via-(--orange)/25 to-transparent" /
    >
58      </div>
59    );
60  }

```



```

61
62 function Toggle({ active, onToggle }) {
63   return (
64     <button
65       type="button"
66       onClick={onToggle}
67       aria-pressed={active}
68       className={`flex h-7 w-[54px] shrink-0 items-center rounded-full p-1 transition-all duration-200 ${
69         active ? "bg-[linear-gradient(135deg,#E04420,#341CD6)]" : "bg-(--line)"
70       }`}
71     >
72       <span
73         className={`h-5 w-5 rounded-full bg-white shadow-sm transition-transform duration-200 ${
74           active ? "translate-x-[26px]" : "translate-x-0"
75         }`}
76       />
77     </button>
78   );
79 }
80
81 function Typedropdown({ value, onChange, options }) {
82   const [open, setOpen] = useState(false);
83   const ref = useRef(null);
84
85   useEffect(() => {
86     const handler = (e) => {
87       if (!ref.current?.contains(e.target)) setOpen(false);
88     };
89     if (open) document.addEventListener("mousedown", handler);
90     return () => document.removeEventListener("mousedown", handler);
91   }, [open]);
92
93   return (
94     <div ref={ref} className="relative">
95       <button
96         type="button"
97         onClick={() => setOpen((prev) => !prev)}
98         className="flex h-11 w-full items-center justify-between rounded-[10px] bg-[linear-gradient(135deg,#E04420,#341CD6)] px-4 text-[13px] font-semibold tracking-[0.06em] text-white transition hover:brightness-105 max-[640px]:h-10"
99       >
100         <span>{value}</span>
101         <ChevronIcon />
102       </button>
103       {open && (
104         <div className="absolute left-0 top-full z-50 mt-1.5 w-full overflow-hidden rounded-[11px] border border-(--line) bg-(--surface) shadow-(--shadowUp)">
105           {options.map((option) => (
106             <button
107               key={option}
108               type="button"
109               onClick={() => {
110                 onChange(option);
111                 setOpen(false);
112               }}
113               className={`block w-full px-4 py-2.5 text-left text-[13px] font-semibold tracking-[0.04em] transition hover:bg-(--surface2) ${
114                 value === option ? "text-(--orange)" : "text-(--text)"
115               }`}
116             >
117               {option}
118             </button>
119           ))}
120         </div>
121       )}
122     </div>
123   );
124 }
125
126 function BoxInput({ value, onChange, type = "text", placeholder = "" }) {

```

```

127   return (
128     <input type={type} value={value} onChange={onChange} placeholder={placeholder} className={FIELD_CLASS}
129   ) />
129   );
130 }
131
132 function NumberInput({ value, onChange, placeholder = "" }) {
133   return (
134     <input
135       inputMode="decimal"
136       value={value}
137       onChange={(event) => onChange(numericOnly(event.target.value))}
138       placeholder={placeholder}
139       className={FIELD_CLASS}
140     />
141   );
142 }
143
144 function PasswordInput({ value, onChange }) {
145   const [show, setShow] = useState(false);
146
147   return (
148     <div className="relative">
149       <input type={show ? "text" : "password"} value={value} onChange={onChange} className={FIELD_CLASS}
150     />
151     <button
152       type="button"
153       onClick={() => setShow((prev) => !prev)}
154       className="absolute right-3 top-1/2 flex h-6 w-6 -translate-y-1/2 items-center justify-center rounded-md text-(--muted) transition hover:bg-(--chip) hover:text-(--text)"
155       aria-label={show ? "Hide password" : "Show password"}
156     >
157       <EyeIcon hidden={show} />
158     </button>
159   </div>
160 );
161 }
162
163 function CharTextarea({ value, onChange, maxLength = 128, placeholder = "" }) {
164   return (
165     <div className="relative">
166       <textarea
167         value={value}
168         onChange={onChange}
169         maxLength={maxLength}
170         placeholder={placeholder}
171         rows={3}
172         className="w-full resize-y rounded-[10px] border border-(--line) bg-(--surface) px-3.5 py-2.5 pr-14 text-[13px] font-medium text-(--text) outline-none transition placeholder:text-(--faint) focus:border-(--orange) focus:shadow-[0_0_3px_rgba(224,68,32,0.12)]"
173       />
174       <span className="font-vcr absolute bottom-2.5 right-3 text-[9px] text-(--faint)">
175         {value.length}/{maxLength}
176       </span>
177     </div>
178 );
179 }
180
181 function FormRow({ label, children }) {
182   return (
183     <div className="grid grid-cols-[170px_1fr] items-start gap-4 max-sm:grid-cols-1 max-sm:gap-1.5">
184       <span className="pt-2.5 text-right text-[13px] font-medium text-(--muted) max-sm:pt-0 max-sm:text-left">
185         {label}
186       </span>
187       <div>{children}</div>
188     </div>
189 );
190 }

```

```

191 function FieldGrid({ children }) {
192   return <div className="grid grid-cols-2 gap-3 max-sm:grid-cols-1">{children}</div>;
193 }
194
195 function FieldGroup({ label, children }) {
196   return (
197     <div>
198       <div className="font-vcr mb-1.5 text-[8.5px] uppercase tracking-[0.16em] text-(--muted)">
199         {label}
200       </div>
201       {children}
202     </div>
203   );
204 }
205
206 function FormatSelector({ value, onChange }) {
207   return (
208     <div className="grid grid-cols-2 overflow-hidden rounded-[10px] border border-(--line) bg-(--surface)
209       p-1">
210       [
211         ["format1", "Default"],
212         ["format2", "Format2"]
213       ].map(([option, label]) => (
214         <button
215           key={option}
216           type="button"
217           onClick={() => onChange(option)}
218           className={`h-8 rounded-md text-[12px] font-semibold transition ${
219             value === option ? "bg-(--text) text-(--bg)" : "text-(--muted) hover:bg-(--surface2)"
220           }`}
221           >
222             {label}
223           </button>
224         ))
225     </div>
226   );
227 }
228
229 function ToggleRow({ label, children }) {
230   return (
231     <div className="grid grid-cols-[170px_1fr] items-center gap-4 max-sm:grid-cols-1 max-sm:gap-1.5">
232       <span className="text-right text-[13px] font-medium text-(--muted) max-sm:text-left">
233         {label}
234       </span>
235       <div>{children}</div>
236     </div>
237   );
238 }
239
240 function DashboardIcon() {
241   return (
242     <svg viewBox="0 0 24 24" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="1.6"
243       strokeLinecap="round" strokeLinejoin="round">
244       <circle cx="12" cy="12" r="10" />
245       <path d="M2 12h20" />
246       <path d="M12 2a15.3 15.3 0 0 1 4 10 15.3 15.3 0 0 1-4 10 15.3 15.3 0 0 1-10 15.3 15.3 0 0 1-10 15.3 0 0 1-4 10 15.3 15.3 0 0 1-10 15.3 0 0 1-4 10" />
247     </svg>
248   );
249 }
250
251 function PrintIcon() {
252   return (
253     <svg viewBox="0 0 24 24" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="1.6"
254       strokeLinecap="round" strokeLinejoin="round">
255       <path d="M6 9v2h12v7" />
256       <path d="M6 18h4a2 2 0 0 1-2-2v-5a2 2 0 0 1 2-2h16a2 2 0 0 1 2 2v5a2 2 0 0 1-2 2h-2" />
257       <path d="M6 14h12v8H6z" />
258     </svg>
259   );

```

```

257 }
258
259 export default function CreateVendorModal({ eventId, onClose, onConfirm }) {
260   const [form, setForm] = useState({ ...DEFAULT_VENDOR_FORM });
261   const overlayRef = useRef(null);
262
263   const set = (key, value) => setForm((prev) => ({ ...prev, [key]: value }));
264
265   useEffect(() => {
266     const handler = (e) => {
267       if (e.key === "Escape") onClose?();
268     };
269     const previousOverflow = document.body.style.overflow;
270
271     document.body.style.overflow = "hidden";
272     document.addEventListener("keydown", handler);
273     return () => {
274       document.body.style.overflow = previousOverflow;
275       document.removeEventListener("keydown", handler);
276     };
277   }, [onClose]);
278
279   return (
280     <div
281       ref={overlayRef}
282       onMouseDown={(e) => {
283         if (e.target === overlayRef.current) onClose?();
284       }}
285       className="fixed inset-0 z-[200] flex items-center justify-center overscroll-none bg-[rgba(12,12,12
286 ,0.5)] px-4 py-6 backdrop-blur-[3px] max-[640px]:p-0"
287       >
288         <div className="flex max-h-[calc(100dvh-48px)] w-full max-w-[760px] flex-col overflow-hidden rounded-
289 d-[16px] border border-(--line) bg-(--surface) shadow-(--shadowUp) max-[640px]:h-full max-[640px]:max-h-f
290 ull max-[640px]:rounded-none">
291           <div
292             className="relative shrink-0 overflow-hidden px-7 py-4 max-[640px]:px-4"
293             style={{ background: "linear-gradient(120deg,#1C1C1C 0%,#341CD6 62%,#E04420 130%)" }}
294             >
295             <div
296               className="pointer-events-none absolute inset-0 opacity-[0.12]"
297               style={{ background: "repeating-linear-gradient(90deg,#fff 0 1px,transparent 1px 46px)" }}
298               aria-hidden=""
299             />
300             <div className="relative flex items-start justify-between gap-3">
301               <div>
302                 <p className="font-vcr m-0 text-[9px] uppercase tracking-[0.2em] text-(--purple)">
303                   Vendor Setup
304                 </p>
305                 <h2 className="font-chillax m-0 mt-1 text-[clamp(20px,2.4vw,26px)] font-semibold tracking-[
306 -0.01em] text-white">
307                   Create Vendor
308                 </h2>
309               </div>
310               <button
311                 type="button"
312                 onClick={onClose}
313                 className="grid h-8 w-8 shrink-0 place-items-center rounded-[9px] border border-white/20 te
314 xt-white/80 transition hover:border-(--orange) hover:bg-(--orange)"
315                 aria-label="Close create vendor modal"
316               >
317                 <CloseIcon />
318               </button>
319             </div>
320             <div className="min-h-0 flex-1 space-y-3.5 overflow-y-auto px-7 py-5 max-[640px]:px-4">
321               <FormRow label="Vendor Name">
322                 <BoxInput value={form.vendorName} onChange={(e) => set("vendorName", e.target.value)} />
323               </FormRow>
324             </div>
325           </div>
326         </div>
327       </div>
328     </div>
329   );
330 }

```

```

322     <FormRow label="Type">
323         <TypeDropdown value={form.type} onChange={(value) => set("type", value)} options={VENDOR_TYPE
S} />
324     </FormRow>
325
326     <FormRow label="Rev Share">
327         <NumberInput value={form.revShare} onChange={(value) => set("revShare", value)} />
328     </FormRow>
329
330     <FormRow label="Tax Details">
331         <FieldGrid>
332             <FieldGroup label="GSTIN">
333                 <BoxInput value={form.gstin} onChange={(e) => set("gstin", e.target.value)} />
334             </FieldGroup>
335             <FieldGroup label="PAN">
336                 <BoxInput value={form.pan} onChange={(e) => set("pan", e.target.value)} />
337             </FieldGroup>
338         </FieldGrid>
339     </FormRow>
340
341     <FormRow label="Charges">
342         <FieldGrid>
343             <FieldGroup label="Service Charge">
344                 <NumberInput value={form.serviceCharge} onChange={(value) => set("serviceCharge", value)}
/>
345             </FieldGroup>
346             <FieldGroup label="Service Charge Tax">
347                 <NumberInput value={form.serviceChargeTax} onChange={(value) => set("serviceChargeTax", v
alue)} />
348             </FieldGroup>
349         </FieldGrid>
350     </FormRow>
351
352     <ToggleRow label="Show Print Details">
353         <Toggle active={form.showPrintDetails} onToggle={() => set("showPrintDetails", !form.showPrin
tDetails)} />
354     </ToggleRow>
355
356     {form.showPrintDetails && (
357         <div className="space-y-3.5 rounded-[12px] border border-(--line) bg-(--surface2) px-4 py-4">
358             <SectionDivider icon={<PrintIcon />} label="Print Details" />
359
360             <FormRow label="Mobile">
361                 <BoxInput type="tel" value={form.mobile} onChange={(e) => set("mobile", numericOnly(e.tar
get.value))} />
362             </FormRow>
363
364             <FormRow label="Address">
365                 <CharTextarea value={form.address} onChange={(e) => set("address", e.target.value)} maxLe
ngth={128} />
366             </FormRow>
367
368             <FormRow label="Thank You Note">
369                 <CharTextarea value={form.thankYouNote} onChange={(e) => set("thankYouNote", e.target.val
ue)} maxLength={128} />
370             </FormRow>
371
372             <ToggleRow label="SAC">
373                 <Toggle active={form.sac} onToggle={() => set("sac", !form.sac)} />
374             </ToggleRow>
375             {form.sac && (
376                 <FormRow label="SAC Value">
377                     <BoxInput value={form.sacText} onChange={(e) => set("sacText", e.target.value)} />
378                 </FormRow>
379             )}
380
381             <ToggleRow label="Auto Print">
382                 <Toggle active={form.autoPrint} onToggle={() => set("autoPrint", !form.autoPrint)} />
383             </ToggleRow>
384

```

```

385         <FormRow label="Max Invoices">
386             <NumberInput value={form.maxInvoices} onChange={(value) => set("maxInvoices", value)} />
387         </FormRow>
388
389         <ToggleRow label="KOT">
390             <Toggle active={form.kot} onToggle={() => set("kot", !form.kot)} />
391         </ToggleRow>
392         {form.kot && (
393             <FormRow label="KOT Count">
394                 <NumberInput value={form.kotCount} onChange={(value) => set("kotCount", value)} />
395             </FormRow>
396         )}
397
398         <ToggleRow label="Invoice Format">
399             <FormatSelector value={form.invoiceFormat} onChange={(value) => set("invoiceFormat", valu
e)} />
400         </ToggleRow>
401
402         <FormRow label="Re-Print Password">
403             <PasswordInput value={form.reprintPassword} onChange={(e) => set("reprintPassword", e.tar
get.value)} />
404         </FormRow>
405     </div>
406 )}
407
408     <SectionDivider icon={<DashboardIcon />} label="Dashboard" />
409
410     <FormRow label="Dashboard Password">
411         <PasswordInput value={form.dashboardPassword} onChange={(e) => set("dashboardPassword", e.tar
get.value)} />
412     </FormRow>
413 </div>
414
415     <div className="flex shrink-0 items-center justify-end gap-2.5 border-t border-(--line) bg-(--sur
face2) px-7 py-4 max-[640px]:grid max-[640px]:grid-cols-2 max-[640px]:px-4">
416         <button
417             type="button"
418             onClick={onClose}
419             className="flex h-11 items-center justify-center rounded-[10px] border border-(--line) bg-(--
surface) px-6 text-[13.5px] font-semibold text-(--muted) transition hover:border-(--orange) hover:text-(
--orange)"
420             >
421             Cancel
422         </button>
423         <button
424             type="button"
425             onClick={() => onConfirm?.(buildVendorPayload(form, eventId))}
426             className="flex h-11 items-center justify-center rounded-[10px] bg-(--text) px-7 text-[13.5px
] font-semibold text-(--bg) transition hover:bg-(--orange)"
427             >
428             Confirm
429         </button>
430     </div>
431 </div>
432 </div>
433 );
434 }

```

FILE: apps/dashboard/src/app/Config/components/EditAccessGateConfigModal.js

```

1  "use client";
2
3  import { useEffect, useState } from "react";
4
5  function CloseIcon() {
6    return (
7      <svg viewBox="0 0 14 14" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="1.6"
strokeLinecap="round" aria-hidden="true">
8        <path d="M3.5 3.5l7 7M10.5 3.5l-7 7" />
9      </svg>
10   );
11 }
12
13 export default function EditAccessGateConfigModal({ gateConfig, onClose, onConfirm }) {
14   const [name, setName] = useState(gateConfig?.gatesmaster_name ?? "");
15   const [useCatgCounts, setUseCatgCounts] = useState(Number(gateConfig?.gatesmaster_useCatgCounts) === 1)
;
16   const [saving, setSaving] = useState(false);
17   const [error, setError] = useState("");
18
19   useEffect(() => {
20     setName(gateConfig?.gatesmaster_name ?? "");
21     setUseCatgCounts(Number(gateConfig?.gatesmaster_useCatgCounts) === 1);
22     setError("");
23   }, [gateConfig]);
24
25   useEffect(() => {
26     const handleKeyDown = (event) => {
27       if (event.key === "Escape" && !saving) onClose?();
28     };
29     document.addEventListener("keydown", handleKeyDown);
30     return () => document.removeEventListener("keydown", handleKeyDown);
31   }, [onClose, saving]);
32
33   const handleSubmit = async (event) => {
34     event.preventDefault();
35     const normalizedName = name.trim();
36     if (!normalizedName) return;
37
38     setSaving(true);
39     setError("");
40     try {
41       await onConfirm?.({ name: normalizedName, useCatgCounts: useCatgCounts ? 1 : 0 });
42     } catch {
43       setError("Unable to update this gate configuration. Please try again.");
44     } finally {
45       setSaving(false);
46     }
47   };
48
49   return (
50     <div
51       className="fixed inset-0 z-[200] flex items-center justify-center bg-[rgba(12,12,12,0.5)] p-4 backd
rop-blur-[3px]"
52     >
53       <section
54         role="dialog"
55         aria-modal="true"
56         aria-labelledby="edit-access-gate-config-title"
57         className="w-full max-w-[480px] overflow-hidden rounded-[16px] border border-(--line) bg-(--surfa
ce) shadow-(--shadowUp)"
58       >
59         <header
60           className="relative overflow-hidden px-5 py-4"
61           style={{ background: "linear-gradient(120deg,#1C1C1C 0%,#341CD6 62%,#E04420 130%)" }}
62         >

```

```

66     <div className="relative flex items-start justify-between gap-3">
67       <div>
68         <p className="font-vcr text-[9px] uppercase tracking-[0.2em] text-(--purple)">AccessX</p>
69         <h2 id="edit-access-gate-config-title" className="font-chillax mt-1 text-[clamp(18px,2.2vw,
24px)] font-semibold tracking-[-0.01em] text-white">
70           Edit Gate Config
71         </h2>
72       </div>
73       <button
74         type="button"
75         onClick={onClose}
76         disabled={saving}
77         className="grid h-8 w-8 shrink-0 place-items-center rounded-[9px] border border-white/20 te
xt-white/80 transition hover:border-(--orange) hover:bg-(--orange) disabled:opacity-40"
78         aria-label="Close gate configuration editor"
79       >
80         <CloseIcon />
81       </button>
82     </div>
83   </header>
84
85   <form onSubmit={handleSubmit} className="p-5 max-[640px]:p-4">
86     <div className="grid gap-4 sm:grid-cols-2 max-[640px]:gap-3">
87       <label className="block sm:col-span-2">
88         <span className="font-vcr mb-1.5 block text-[8.5px] uppercase tracking-[0.16em] text-(--mut
ed)">Name</span>
89         <input
90           autoFocus
91           value={name}
92           onChange={(event) => setName(event.target.value)}
93           placeholder="Enter gate master name"
94           className="h-11 w-full rounded-[10px] border border-(--line) bg-(--surface) px-3.5 text-[
13px] font-medium text-(--text) outline-none transition placeholder:text-(--faint) focus:border-(--orange
) focus:shadow-[0_0_0_3px_rgba(224,68,32,0.12)]"
95         />
96       </label>
97
98       <div>
99         <span className="font-vcr mb-1.5 block text-[8.5px] uppercase tracking-[0.16em] text-(--mut
ed)">Position</span>
100        <div className="font-vcr flex h-11 items-center rounded-[10px] border border-(--line) bg-(--
-surface2) px-3.5 text-[13px] text-(--faint)">
101          {gateConfig?.gatesmaster_position ?? "-"}
102        </div>
103      </div>
104
105      <div>
106        <span className="font-vcr mb-1.5 block text-[8.5px] uppercase tracking-[0.16em] text-(--mut
ed)">Use Category Counts</span>
107        <button
108          type="button"
109          onClick={() => setUseCatgCounts(current) => !current}
110          className="flex h-11 w-full items-center justify-between rounded-[10px] border border-(--
line) bg-(--surface) px-3.5"
111          aria-pressed={useCatgCounts}
112        >
113          <span className="text-[12.5px] font-semibold text-(--muted)">
114            {useCatgCounts ? "Enabled" : "Disabled"}
115          </span>
116          <span className={`relative h-5 w-9 rounded-full transition ${useCatgCounts ? "bg-(--orang
e)" : "bg-(--line)"}`}>
117            <span className={`absolute top-0.5 h-4 w-4 rounded-full bg-white shadow transition ${us
eCatgCounts ? "left-[18px]" : "left-0.5"}`> />
118          </span>
119        </button>
120      </div>
121    </div>
122
123    {error ? (
124      <p className="mt-4 rounded-[10px] border border-[rgba(224,68,32,0.25)] bg-[rgba(224,68,32,0.0

```



```

6)] px-3 py-2 text-[12px] font-semibold text-(--orange)">
125     {error}
126   </p>
127   ) : null}
128
129   <div className="mt-5 flex justify-end gap-2.5 border-t border-(--line) pt-4 max-[640px]:grid ma
x-[640px]:grid-cols-2">
130     <button
131       type="button"
132       onClick={onClose}
133       disabled={saving}
134       className="flex h-11 items-center justify-center rounded-[10px] border border-(--line) bg-(
--surface) px-5 text-[13px] font-semibold text-(--muted) transition hover:border-(--orange) hover:text-(
--orange) disabled:opacity-50"
135     >
136       Cancel
137     </button>
138     <button
139       type="submit"
140       disabled={saving || !name.trim()}
141       className="flex h-11 items-center justify-center rounded-[10px] bg-(--text) px-6 text-[13px
] font-semibold text-(--bg) transition hover:bg-(--orange) disabled:cursor-not-allowed disabled:opacity-5
0"
142     >
143       {saving ? "Saving..." : "Confirm"}
144     </button>
145   </div>
146 </form>
147 </section>
148 </div>
149 );
150 }

```

FILE: apps/dashboard/src/app/Config/components/EditAccessGateMasterModal.js

```

1  "use client";
2
3  import { useEffect, useState } from "react";
4
5  function CloseIcon() {
6    return (
7      <svg viewBox="0 0 14 14" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="1.6"
strokeLinecap="round" aria-hidden="true">
8        <path d="M3.5 3.5l7 7M10.5 3.5l-7 7" />
9      </svg>
10   );
11 }
12
13 export default function EditAccessGateMasterModal({ gateMaster, onClose, onConfirm }) {
14   const [name, setName] = useState(gateMaster?.name ?? "");
15   const [useCatgCounts, setUseCatgCounts] = useState(Number(gateMaster?.useCatgCounts) === 1);
16   const [saving, setSaving] = useState(false);
17   const [error, setError] = useState("");
18
19   useEffect(() => {
20     setName(gateMaster?.name ?? "");
21     setUseCatgCounts(Number(gateMaster?.useCatgCounts) === 1);
22     setError("");
23   }, [gateMaster]);
24
25   useEffect(() => {
26     const handleKeyDown = (event) => {
27       if (event.key === "Escape" && !saving) onClose?();
28     };
29     document.addEventListener("keydown", handleKeyDown);
30     return () => document.removeEventListener("keydown", handleKeyDown);
31   }, [onClose, saving]);
32
33   const handleSubmit = async (event) => {
34     event.preventDefault();
35     const normalizedName = name.trim();
36     if (!normalizedName) return;
37
38     setSaving(true);
39     setError("");
40     try {
41       await onConfirm?.({ name: normalizedName, useCatgCounts: useCatgCounts ? 1 : 0 });
42     } catch {
43       setError("Unable to update this gate master. Please try again.");
44     } finally {
45       setSaving(false);
46     }
47   };
48
49   return (
50     <div
51       className="fixed inset-0 z-[200] flex items-center justify-center bg-[rgba(12,12,12,0.5)] p-4 backd
rop-blur-[3px]"
52     >
53       <section
54         role="dialog"
55         aria-modal="true"
56         aria-labelledby="edit-access-gate-master-title"
57         className="w-full max-w-[480px] overflow-hidden rounded-[16px] border border-(--line) bg-(--surfa
ce) shadow-(--shadowUp)"
58       >
59         <header
60           className="relative overflow-hidden px-5 py-4"
61           style={{ background: "linear-gradient(120deg,#1C1C1C 0%,#341CD6 62%,#E04420 130%)" }}
62         >
63           <div className="relative flex items-start justify-between gap-3">

```

```

67         <div>
68             <p className="font-vcr text-[9px] uppercase tracking-[0.2em] text-(--purple)">AccessX</p>
69             <h2 id="edit-access-gate-master-title" className="font-chillax mt-1 text-[clamp(18px,2.2vw,
24px)] font-semibold tracking-[-0.01em] text-white">
70                 Edit Gate Master
71             </h2>
72         </div>
73         <button
74             type="button"
75             onClick={onClose}
76             disabled={saving}
77             className="grid h-8 w-8 shrink-0 place-items-center rounded-[9px] border border-white/20 te
xt-white/80 transition hover:border-(--orange) hover:bg-(--orange) disabled:opacity-40"
78             aria-label="Close gate master editor"
79         >
80             <CloseIcon />
81         </button>
82     </div>
83 </header>
84
85     <form onSubmit={handleSubmit} className="p-5 max-[640px]:p-4">
86         <div className="grid gap-4 sm:grid-cols-2 max-[640px]:gap-3">
87             <label className="block sm:col-span-2">
88                 <span className="font-vcr mb-1.5 block text-[8.5px] uppercase tracking-[0.16em] text-(--mut
ed)">Name</span>
89                 <input
90                     autoFocus
91                     value={name}
92                     onChange={(event) => setName(event.target.value)}
93                     placeholder="Enter gate master name"
94                     className="h-11 w-full rounded-[10px] border border-(--line) bg-(--surface) px-3.5 text-[
13px] font-medium text-(--text) outline-none transition placeholder:text-(--faint) focus:border-(--orange
) focus:shadow-[0_0_3px_rgba(224,68,32,0.12)]"
95                 />
96             </label>
97
98             <div>
99                 <span className="font-vcr mb-1.5 block text-[8.5px] uppercase tracking-[0.16em] text-(--mut
ed)">Position</span>
100                 <div className="font-vcr flex h-11 items-center rounded-[10px] border border-(--line) bg-(--
surface2) px-3.5 text-[13px] text-(--faint)">
101                     {gateMaster?.position ?? "-"}
102                 </div>
103             </div>
104
105             <div>
106                 <span className="font-vcr mb-1.5 block text-[8.5px] uppercase tracking-[0.16em] text-(--mut
ed)">Use Category Counts</span>
107                 <button
108                     type="button"
109                     onClick={() => setUseCatgCounts((current) => !current)}
110                     className="flex h-11 w-full items-center justify-between rounded-[10px] border border-(--
line) bg-(--surface) px-3.5"
111                     aria-pressed={useCatgCounts}
112                 >
113                     <span className="text-[12.5px] font-semibold text-(--muted)">
114                         {useCatgCounts ? "Enabled" : "Disabled"}
115                     </span>
116                     <span className={`relative h-5 w-9 rounded-full transition ${useCatgCounts ? "bg-(--orang
e)" : "bg-(--line)"}`}>
117                         <span className={`absolute top-0.5 h-4 w-4 rounded-full bg-white shadow transition ${us
eCatgCounts ? "left-[18px]" : "left-0.5"}`}> />
118                     </span>
119                 </button>
120             </div>
121         </div>
122
123         {error ? (
124             <p className="mt-4 rounded-[10px] border border-[rgba(224,68,32,0.25)] bg-[rgba(224,68,32,0.0
6)] px-3 py-2 text-[12px] font-semibold text-(--orange)">

```

```

125         {error}
126     </p>
127     ) : null}
128
129     <div className="mt-5 flex justify-end gap-2.5 border-t border-(--line) pt-4 max-[640px]:grid ma
x-[640px]:grid-cols-2">
130         <button
131             type="button"
132             onClick={onClose}
133             disabled={saving}
134             className="flex h-11 items-center justify-center rounded-[10px] border border-(--line) bg-(
--surface) px-5 text-[13px] font-semibold text-(--muted) transition hover:border-(--orange) hover:text-(
--orange) disabled:opacity-50"
135             >
136             Cancel
137         </button>
138         <button
139             type="submit"
140             disabled={saving || !name.trim()}
141             className="flex h-11 items-center justify-center rounded-[10px] bg-(--text) px-6 text-[13px
] font-semibold text-(--bg) transition hover:bg-(--orange) disabled:cursor-not-allowed disabled:opacity-5
0"
142             >
143             {saving ? "Saving..." : "Confirm"}
144         </button>
145     </div>
146 </form>
147 </section>
148 </div>
149 );
150 }

```

FILE: apps/dashboard/src/app/Config/components/EditAccessXCategoryModal.js

```

1  "use client";
2
3  import { useEffect, useState } from "react";
4
5  const FIELD_CLASS =
6    "h-11 w-full rounded-[10px] border border-(--line) bg-(--surface) px-3.5 text-[13px] font-medium text-(
    --text) outline-none transition placeholder:text-(--faint) focus:border-(--orange) focus:shadow-[0_0_0_3p
    x_rgba(224,68,32,0.12)]";
7
8  const ALLOW_OPTIONS = [
9    { label: "Always Allow", value: 0 },
10   ...Array.from({ length: 9 }, (_, index) => ({
11     label: `${index + 1} ${index === 0 ? "Time" : "Times"}`,
12     value: index + 1
13   }))
14 ];
15
16 function valueOf(value) {
17   return value === null || value === undefined ? "" : String(value);
18 }
19
20 function createForm(category) {
21   return {
22     name: valueOf(category?.name),
23     allowCount: Number(category?.allowCount) || 0,
24     position: valueOf(category?.position),
25     qrLogicIssuerId: valueOf(category?.qrLogicIssuerId),
26     qrLogicSectorId: valueOf(category?.qrLogicSectorId),
27     qrLogicDiscountId: valueOf(category?.qrLogicDiscountId),
28     qrLogicTicketType: valueOf(category?.qrLogicTicketType),
29     qrLogicTktAccred: valueOf(category?.qrLogicTktAccred)
30   };
31 }
32
33 function CloseIcon() {
34   return (
35     <svg viewBox="0 0 14 14" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="1.6"
    strokeLinecap="round" aria-hidden="true">
36       <path d="M3.5 3.5l7 7" />
37     </svg>
38   );
39 }
40
41 function FormField({ label, value, onChange, type = "text", placeholder = "" }) {
42   return (
43     <label className="block min-w-0">
44       <span className="font-vcr mb-1.5 block text-[8.5px] uppercase tracking-[0.16em] text-(--muted)">{la
    bel}</span>
45       <input
46         type={type}
47         value={value}
48         onChange={(event) => onChange(event.target.value)}
49         placeholder={placeholder}
50         className={FIELD_CLASS}
51       />
52     </label>
53   );
54 }
55
56 export default function EditAccessXCategoryModal({ category, mode = "edit", onClose, onConfirm }) {
57   const [form, setForm] = useState(() => createForm(category));
58   const [saving, setSaving] = useState(false);
59   const [error, setError] = useState("");
60   const isCreate = mode === "create";
61
62   useEffect(() => {
63     setForm(createForm(category));
64     setError("");
65   }, [category]);

```

```

66
67   useEffect(() => {
68     const handleKeyDown = (event) => {
69       if (event.key === "Escape") onClose?();
70     };
71     document.addEventListener("keydown", handleKeyDown);
72     return () => document.removeEventListener("keydown", handleKeyDown);
73   }, [onClose]);
74
75   const setField = (key, value) => {
76     setForm((current) => ({ ...current, [key]: value }));
77   };
78
79   const handleSubmit = async (event) => {
80     event.preventDefault();
81     setSaving(true);
82     setError("");
83     try {
84       await onConfirm?({
85         ...form,
86         allowCount: Number(form.allowCount) || 0,
87         position: form.position === "" ? null : Number(form.position),
88         qrLogicIssuerId: form.qrLogicIssuerId || null,
89         qrLogicSectorId: form.qrLogicSectorId || null,
90         qrLogicDiscountId: form.qrLogicDiscountId || null,
91         qrLogicTicketType: form.qrLogicTicketType || null,
92         qrLogicTktAccred: form.qrLogicTktAccred || null
93       });
94     } catch {
95       setError("Unable to save this category. Please try again.");
96     } finally {
97       setSaving(false);
98     }
99   };
100
101   return (
102     <div
103       className="fixed inset-0 z-[200] flex items-center justify-center bg-[rgba(12,12,12,0.5)] p-4 backd
104       rop-blur-[3px] max-[640px]:p-0"
105       onMouseDown={(event) => {
106         if (event.target === event.currentTarget) onClose?();
107       }}
108     >
109       <section
110         role="dialog"
111         aria-modal="true"
112         aria-labelledby="edit-accesssx-category-title"
113         className="flex max-h-[calc(100dvh-48px)] w-full max-w-[680px] flex-col overflow-hidden rounded-[
114         16px] border border-(--line) bg-(--surface) shadow-(--shadowUp) max-[640px]:h-full max-[640px]:max-h-full
115         max-[640px]:rounded-none"
116       >
117         <header
118           className="relative shrink-0 overflow-hidden px-5 py-4"
119           style={{ background: "linear-gradient(120deg,#1C1C1C 0%,#341CD6 62%,#E04420 130%)" }}
120         >
121           <div className="relative flex items-start justify-between gap-3">
122             <div className="min-w-0">
123               <p className="font-vcr text-[9px] uppercase tracking-[0.2em] text-(--purple)">AccessX Categ
124             ory</p>
125             <h2 id="edit-accesssx-category-title" className="font-chillax mt-1 truncate text-[clamp(18px
126             ,2.2vw,24px)] font-semibold tracking-[-0.01em] text-white">
127               {isCreate ? "Add Category" : `Edit ${category?.name || "Category"}`}
128             </h2>
129             </div>
130             <button
131               type="button"
132               onClick={onClose}
133               className="grid h-8 w-8 shrink-0 place-items-center rounded-[9px] border border-white/20 te
134               xt-white/80 transition hover:border-(--orange) hover:bg-(--orange)"
135               aria-label="Close category editor"

```

```

130         >
131         <CloseIcon />
132     </button>
133 </div>
134 </header>
135
136     <form onSubmit={handleSubmit} className="flex min-h-0 flex-1 flex-col">
137         <div className="min-h-0 flex-1 overflow-y-auto p-5 max-[640px]:p-4">
138             <div className="grid gap-4 sm:grid-cols-2 max-[640px]:gap-3">
139                 <div className="sm:col-span-2">
140                     <FormField label="Name" value={form.name} onChange={(value) => setField("name", value)} placeholder="Category name" />
141                 </div>
142
143                 <label className="block min-w-0">
144                     <span className="font-vcr mb-1.5 block text-[8.5px] uppercase tracking-[0.16em] text-(--muted)">Access Allow Count</span>
145                     <div className="relative">
146                         <select
147                             value={form.allowCount}
148                             onChange={(event) => setField("allowCount", Number(event.target.value))}
149                             className={` ${FIELD_CLASS} cursor-pointer appearance-none pr-9`}
150                         >
151                             {ALLOW_OPTIONS.map((option) => (
152                                 <option key={option.value} value={option.value}>
153                                     {option.label}
154                                 </option>
155                             ))}
156                         </select>
157                         <svg viewBox="0 0 12 12" className="pointer-events-none absolute right-3.5 top-1/2 h-2.5 w-2.5 -translate-y-1/2 opacity-50" fill="none" stroke="currentColor" strokeWidth="1.5" strokeLinecap="round">
158                             <path d="M2.5 4.5 6 8 3.5-3.5" />
159                         </svg>
160                     </div>
161                 </label>
162
163                 <FormField label="Position" type="number" value={form.position} onChange={(value) => setField("position", value)} placeholder="Position" />
164
165                 <FormField label="Issuer ID" value={form.qrLogicIssuerId} onChange={(value) => setField("qrLogicIssuerId", value)} placeholder="Issuer ID" />
166                 <FormField label="Sector ID" value={form.qrLogicSectorId} onChange={(value) => setField("qrLogicSectorId", value)} placeholder="Sector ID" />
167                 <FormField label="Discount ID" value={form.qrLogicDiscountId} onChange={(value) => setField("qrLogicDiscountId", value)} placeholder="Discount ID" />
168                 <FormField label="Ticket Type" value={form.qrLogicTicketType} onChange={(value) => setField("qrLogicTicketType", value)} placeholder="Ticket type" />
169                 <div className="sm:col-span-2">
170                     <FormField label="Ticket / Accred" value={form.qrLogicTktAccred} onChange={(value) => setField("qrLogicTktAccred", value)} placeholder="Ticket or accreditation" />
171                 </div>
172             </div>
173
174             {error ? (
175                 <p className="mt-4 rounded-[10px] border border-[rgba(224,68,32,0.25)] bg-[rgba(224,68,32,0.06)] px-3 py-2 text-[12px] font-semibold text-(--orange)">
176                     {error}
177                 </p>
178             ) : null}
179         </div>
180
181         <div className="flex shrink-0 justify-end gap-2.5 border-t border-(--line) bg-(--surface2) px-5 py-4 max-[640px]:grid max-[640px]:grid-cols-2">
182             <button
183                 type="button"
184                 onClick={onClose}
185                 disabled={saving}
186                 className="flex h-11 items-center justify-center rounded-[10px] border border-(--line) bg-(--surface) px-5 text-[13px] font-semibold text-(--muted) transition hover:border-(--orange) hover:text-(--muted)" />

```

```
-orange)"
187         >
188         Cancel
189     </button>
190     <button
191         type="submit"
192         disabled={saving || !form.name.trim()}
193         className="flex h-11 items-center justify-center rounded-[10px] bg-(--text) px-6 text-[13px
] font-semibold text-(--bg) transition hover:bg-(--orange) disabled:cursor-not-allowed disabled:opacity-5
0"
194     >
195         {saving ? "Saving..." : isCreate ? "Add Category" : "Confirm"}
196     </button>
197 </div>
198 </form>
199 </section>
200 </div>
201 );
202 }
```


FILE: apps/dashboard/src/app/Config/components/EditStallModal.js

```

1  "use client";
2
3  import { useEffect, useRef, useState } from "react";
4
5  const STALL_TYPES = ["SALE", "STOCKROOM", "BAR", "FOOD", "BEVERAGE", "KITCHEN"];
6  const SCAN_MODES = ["NONE", "BARCODE", "QR", "NFC", "RFID", "MENU"];
7  const DEVICE_VIEWS = ["GRID VIEW", "LIST VIEW"];
8  const BANK_PAYMENTS = ["NONE", "MSWIPE", "PAYTM", "RAZORPAY", "STRIPE"];
9  const MODE_OPTIONS = ["CASH", "CARD", "UPI", "COUPON", "NFC", "WALLET"];
10
11 const DEFAULT_FORM = {
12   stallName: "",
13   vendorName: "",
14   type: "SALE",
15   acceptAllModes: false,
16   modeOptions: ["CASH", "CARD", "UPI", "COUPON"],
17   grnMode: false,
18   cashDisabled: false,
19   bankPayment: "MSWIPE",
20   kotLan: false,
21   modeInfoMandatory: false,
22   scanMode: "NONE",
23   deviceMenuView: "GRID VIEW",
24   showInTapX: false,
25 };
26
27 const FIELD_CLASS =
28   "h-11 w-full rounded-[10px] border border-(--line) bg-(--surface) px-3.5 text-[13px] font-medium text-(
  --text) outline-none transition placeholder:text-(--faint) focus:border-(--orange) focus:shadow-[0_0_0_3p
  x_rgba(224,68,32,0.12)] disabled:cursor-not-allowed disabled:bg-(--surface2) disabled:text-(--faint) max-
  [640px]:h-10";
29
30 function getStallName(stall) {
31   return stall?.name ?? stall?.stallName ?? stall?.stall ?? "";
32 }
33
34 function getStallVendor(stall) {
35   return stall?.vendorName ?? stall?.vendor?.name ?? stall?.vendor ?? "";
36 }
37
38 function getStallType(stall) {
39   return String(stall?.type ?? "SALE").toUpperCase();
40 }
41
42 function CloseIcon() {
43   return (
44     <svg viewBox="0 0 14 14" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="1.6"
  strokeLinecap="round">
45       <path d="M3.5 3.5l7 7" />
46     </svg>
47   );
48 }
49
50 function ChevronIcon() {
51   return (
52     <svg viewBox="0 0 24 24" className="h-4 w-4 shrink-0" fill="none" stroke="currentColor" strokeWidth="
  2" strokeLinecap="round" strokeLinejoin="round">
53       <path d="m6 9 6 6-6-6" />
54     </svg>
55   );
56 }
57
58 function TapXIcon() {
59   return (
60     <svg viewBox="0 0 24 24" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="1.6"
  strokeLinecap="round" strokeLinejoin="round">
61       <path d="M12 3v4" />
62       <path d="M12 17v4" />
63       <path d="M3 12h4" />

```

```

64     <path d="M17 12h4" />
65     <circle cx="12" cy="12" r="3" />
66   </svg>
67   );
68 }
69
70 function CheckIcon() {
71   return (
72     <svg viewBox="0 0 24 24" className="h-3 w-3" fill="none" stroke="currentColor" strokeWidth="3" stroke
LineCap="round" strokeLinejoin="round">
73       <path d="m5 12 4 4L19 6" />
74     </svg>
75   );
76 }
77
78 function ImageIcon() {
79   return (
80     <svg viewBox="0 0 24 24" className="h-5 w-5" fill="none" stroke="currentColor" strokeWidth="1.8" stro
keLineCap="round" strokeLinejoin="round">
81       <rect x="3" y="3" width="18" height="18" rx="2" />
82       <circle cx="8.5" cy="8.5" r="1.5" />
83       <path d="m21 15-5-5L5 21" />
84     </svg>
85   );
86 }
87
88 function Dropdown({ value, onChange, options }) {
89   const [open, setOpen] = useState(false);
90   const ref = useRef(null);
91
92   useEffect(() => {
93     const handler = (e) => {
94       if (!ref.current?.contains(e.target)) setOpen(false);
95     };
96     if (open) document.addEventListener("mousedown", handler);
97     return () => document.removeEventListener("mousedown", handler);
98   }, [open]);
99
100   return (
101     <div ref={ref} className="relative">
102       <button
103         type="button"
104         onClick={() => setOpen((prev) => !prev)}
105         className="flex h-11 w-full items-center justify-between rounded-[10px] bg-[linear-gradient(135deg,#E04420,#341CD6)] px-4 text-[13px] font-semibold tracking-[0.06em] text-white transition hover:brightne
ss-105 max-[640px]:h-10"
106       >
107         <span>{value}</span>
108         <ChevronIcon />
109       </button>
110       {open && (
111         <div className="absolute left-0 top-full z-50 mt-1.5 w-full overflow-hidden rounded-[11px] border
border-(--line) bg-(--surface) shadow-(--shadowUp)">
112           {options.map((option) => (
113             <button
114               key={option}
115               type="button"
116               onClick={() => {
117                 onChange(option);
118                 setOpen(false);
119               }}
120               className={`block w-full px-4 py-2.5 text-left text-[13px] font-semibold tracking-[0.04em]
transition hover:bg-(--surface2) ${
121                 value === option ? "text-(--orange)" : "text-(--text)"
122               }`}
123             >
124               {option}
125             </button>
126           ))}
127         </div>

```

```

128     })
129   </div>
130   );
131 }
132
133 function ModeOptionsDropdown({ value = [], onChange }) {
134   const [open, setOpen] = useState(false);
135   const ref = useRef(null);
136
137   useEffect(() => {
138     const handler = (e) => {
139       if (!ref.current?.contains(e.target)) setOpen(false);
140     };
141     if (open) document.addEventListener("mousedown", handler);
142     return () => document.removeEventListener("mousedown", handler);
143   }, [open]);
144
145   const toggle = (option) => {
146     onChange(value.includes(option) ? value.filter((item) => item !== option) : [...value, option]);
147   };
148
149   return (
150     <div ref={ref} className="relative">
151       <button
152         type="button"
153         onClick={() => setOpen((prev) => !prev)}
154         className="flex min-h-11 w-full items-center gap-2 rounded-[10px] border border-(--line) bg-(--surface) px-3 text-left text-(--text) transition hover:border-(--orange) focus:border-(--orange) focus:outline-none"
155       >
156         <div className="flex flex-1 flex-wrap gap-1.5 py-1.5">
157           {value.length === 0 ? (
158             <span className="text-[13px] font-medium text-(--faint)">Select modes</span>
159           ) : (
160             value.map((option) => (
161               <span
162                 key={option}
163                 className="font-vcr flex items-center gap-1 rounded-full bg-[rgba(224,68,32,0.08)] px-2.5 py-0.5 text-[10px] text-(--orange)"
164               >
165                 {option}
166                 <span
167                   role="button"
168                   tabIndex={0}
169                   onClick={(e) => {
170                     e.stopPropagation();
171                     toggle(option);
172                   }}
173                   onKeyDown={(e) => {
174                     if (e.key === "Enter" || e.key === " ") {
175                       e.stopPropagation();
176                       toggle(option);
177                     }
178                   }}
179                   className="text-(--orange)/70 hover:text-(--orange)"
180                 >
181                   x
182                 </span>
183               </span>
184             ))
185           )}
186         </div>
187         <ChevronIcon />
188       </button>
189       {open && (
190         <div className="absolute left-0 top-full z-50 mt-1.5 w-full overflow-hidden rounded-[11px] border border-(--line) bg-(--surface) shadow-(--shadowUp)">
191           {MODE_OPTIONS.map((option) => {
192             const checked = value.includes(option);
193             return (

```

```

194         <button
195             key={option}
196             type="button"
197             onClick={() => toggle(option)}
198             className="flex w-full items-center gap-3 px-4 py-2.5 text-left text-[13px] font-semibold
199             transition hover:bg-(--surface2)"
200         >
201             <span
202                 className={`flex h-5 w-5 shrink-0 items-center justify-center rounded-md border transit
ion ${
203                 checked ? "border-(--orange) bg-(--orange) text-white" : "border-(--line) bg-(--surfa
ce2) text-transparent"
204                 }`}
205             >
206                 <CheckIcon />
207             </span>
208             <span className={checked ? "text-(--orange)" : "text-(--text)"}>{option}</span>
209         </button>
210     );
211 }
212 </div>
213 </div>
214 );
215 }
216
217 function SegmentedChoice({ active, leftLabel, rightLabel, onChange }) {
218     return (
219         <div className="grid h-11 grid-cols-2 overflow-hidden rounded-full border border-(--line) bg-(--surfa
ce2) p-1 max-[640px]:h-10">
220             <button
221                 type="button"
222                 onClick={() => onChange(false)}
223                 className={`rounded-full px-3 text-[12.5px] font-semibold transition ${
224                 !active ? "bg-(--surface) text-(--orange) shadow-sm" : "text-(--muted) hover:text-(--text)"
225                 }`}
226             >
227                 {leftLabel}
228             </button>
229             <button
230                 type="button"
231                 onClick={() => onChange(true)}
232                 className={`rounded-full px-3 text-[12.5px] font-semibold transition ${
233                 active ? "bg-[linear-gradient(135deg,#E04420,#341CD6)] text-white" : "text-(--muted) hover:text
-(--text)"
234                 }`}
235             >
236                 {rightLabel}
237             </button>
238         </div>
239     );
240 }
241
242 function SettingChip({ label, active, onToggle }) {
243     return (
244         <button
245             type="button"
246             onClick={onToggle}
247             aria-pressed={active}
248             className={`flex h-11 items-center justify-between gap-3 rounded-[10px] border px-3.5 text-left tra
nsition max-[640px]:h-10 ${
249                 active
250                 ? "border-(--orange) bg-[rgba(224,68,32,0.06)] text-(--text)"
251                 : "border-(--line) bg-(--surface) text-(--muted) hover:border-(--orange)"
252             }`}
253         >
254             <span className="truncate text-[12.5px] font-semibold">{label}</span>
255             <span
256                 className={`flex h-5 w-5 shrink-0 items-center justify-center rounded-md border transition ${
257                 active ? "border-(--orange) bg-(--orange) text-white" : "border-(--line) bg-(--surface2) text-t

```

```

ransparent"
258   `}`
259   >
260   <CheckIcon />
261   </span>
262   </button>
263   );
264 }
265
266 function SectionDivider({ icon, label }) {
267   return (
268     <div className="flex items-center gap-3 py-1.5">
269       <div className="h-px flex-1 bg-gradient-to-r from-transparent via-(--orange)/25 to-(--purple)/40" /
270     >
271       <span className="font-vcr flex items-center gap-1.5 text-[9px] uppercase tracking-[0.16em] text-(--muted)">
272         {icon}
273         {label}
274       </span>
275       <div className="h-px flex-1 bg-gradient-to-r from-(--purple)/40 via-(--orange)/25 to-transparent" /
276     >
277   </div>
278   );
279 }
280
281 function FormRow({ label, children }) {
282   return (
283     <div className="grid grid-cols-[170px_1fr] items-center gap-4 max-sm:grid-cols-1 max-sm:gap-1.5">
284       <span className="text-right text-[13px] font-medium text-(--muted) max-sm:text-left">
285         {label}
286       </span>
287       <div>{children}</div>
288     </div>
289   );
290 }
291
292 function FieldGrid({ children }) {
293   return <div className="grid grid-cols-2 gap-3 max-md:grid-cols-1">{children}</div>;
294 }
295
296 function FieldGroup({ label, children }) {
297   return (
298     <div>
299       <div className="font-vcr mb-1.5 text-[8.5px] uppercase tracking-[0.16em] text-(--muted)">
300         {label}
301       </div>
302       {children}
303     </div>
304   );
305 }
306
307 function SettingsGrid({ children }) {
308   return <div className="grid grid-cols-2 gap-2.5 max-sm:grid-cols-1">{children}</div>;
309 }
310
311 function ImageUploadRow() {
312   const [preview, setPreview] = useState(null);
313   const inputRef = useRef(null);
314
315   const handleFile = (e) => {
316     const file = e.target.files?.[0];
317     if (!file) return;
318     setPreview(URL.createObjectURL(file));
319   };
320
321   return (
322     <button
323       type="button"
324       onClick={() => inputRef.current?.click()}
325       className="flex h-11 w-full items-center gap-3 rounded-[10px] border border-dashed border-(--line)

```

```

    bg(--surface2) px-3.5 text-[13px] font-semibold text(--muted) transition hover:border-(--orange) hover:
    text-(--orange)"
324   >
325     <input ref={inputRef} type="file" accept="image/*" className="hidden" onChange={handleFile} />
326     {preview ? (
327       // eslint-disable-next-line @next/next/no-img-element
328       <img src={preview} alt="Stall banner preview" className="h-7 w-12 rounded-md border border-(--lin
e) object-cover" />
329     ) : (
330       <span className="text-(--orange)">
331         <ImageIcon />
332       </span>
333     )}
334     <span>{preview ? "Change Image" : "Upload Image"}</span>
335   </button>
336 );
337 }
338
339 export default function EditStallModal({ stall, onClose, onConfirm }) {
340   const [form, setForm] = useState({
341     ...DEFAULT_FORM,
342     stallName: getStallName(stall),
343     vendorName: getStallVendor(stall),
344     type: getStallType(stall),
345   });
346   const overlayRef = useRef(null);
347
348   const set = (key, value) => setForm((prev) => ({ ...prev, [key]: value }));
349
350   useEffect(() => {
351     const handler = (e) => {
352       if (e.key === "Escape") onClose?();
353     };
354     const previousOverflow = document.body.style.overflow;
355
356     document.body.style.overflow = "hidden";
357     document.addEventListener("keydown", handler);
358     return () => {
359       document.body.style.overflow = previousOverflow;
360       document.removeEventListener("keydown", handler);
361     };
362   }, [onClose]);
363
364   return (
365     <div
366       ref={overlayRef}
367       onMouseDown={(e) => {
368         if (e.target === overlayRef.current) onClose?();
369       }}
370       className="fixed inset-0 z-[200] flex items-center justify-center overscroll-none bg-[rgba(12,12,12
,0.5)] px-4 py-6 backdrop-blur-[3px] max-[640px]:p-0"
371     >
372       <div className="flex max-h-[calc(100dvh-48px)] w-full max-w-[820px] flex-col overflow-hidden rounde
d-[16px] border border-(--line) bg(--surface) shadow-(--shadowUp) max-[640px]:h-full max-[640px]:max-h-f
ull max-[640px]:rounded-none">
373         <div
374           className="relative shrink-0 overflow-hidden px-7 py-4 max-[640px]:px-4"
375           style={{ background: "linear-gradient(120deg,#1C1C1C 0%,#341CD6 62%,#E04420 130%)" }}
376         >
377           <div
378             className="pointer-events-none absolute inset-0 opacity-[0.12]"
379             style={{ background: "repeating-linear-gradient(90deg,#fff 0 1px,transparent 1px 46px)" }}
380             aria-hidden
381           />
382           <div className="relative flex items-start justify-between gap-3">
383             <div className="min-w-0">
384               <p className="font-vcr m-0 text-[9px] uppercase tracking-[0.2em] text-(--purple)">Edit Stal
l</p>
385               <h2 className="font-chillax m-0 mt-1 truncate text-[clamp(20px,2.4vw,26px)] font-semibold t
racking-[-0.01em] text-white">

```

```

386         {form.stallName || "Stall"}
387     </h2>
388 </div>
389 <button
390     type="button"
391     onClick={onClose}
392     className="grid h-8 w-8 shrink-0 place-items-center rounded-[9px] border border-white/20 te
xt-white/80 transition hover:border-(--orange) hover:bg-(--orange)"
393     aria-label="Close edit stall modal"
394 >
395     <CloseIcon />
396 </button>
397 </div>
398 </div>
399
400 <div className="min-h-0 flex-1 space-y-4 overflow-y-auto px-7 py-5 max-[640px]:px-4">
401     <FieldGrid>
402         <FieldGroup label="Vendor">
403             <input type="text" value={form.vendorName} onChange={() => {}} disabled className={FIELD_CL
ASS} />
404         </FieldGroup>
405         <FieldGroup label="Stall Name">
406             <input type="text" value={form.stallName} onChange={(e) => set("stallName", e.target.value)
} className={FIELD_CLASS} />
407         </FieldGroup>
408         <FieldGroup label="Stall Banner">
409             <ImageUploadRow />
410         </FieldGroup>
411         <FieldGroup label="Type">
412             <Dropdown value={form.type} onChange={(value) => set("type", value)} options={STALL_TYPES}
/>
413         </FieldGroup>
414     </FieldGrid>
415
416     <FormRow label="Payment Modes">
417         <SegmentedChoice
418             active={form.acceptAllModes}
419             leftLabel="Only NFC"
420             rightLabel="Accept All Modes"
421             onChange={(value) => set("acceptAllModes", value)}
422         />
423     </FormRow>
424
425     <FormRow label="Mode Options">
426         <ModeOptionsDropdown value={form.modeOptions} onChange={(value) => set("modeOptions", value)}
/>
427     </FormRow>
428
429     <FormRow label="Stall Settings">
430         <SettingsGrid>
431             <SettingChip label="GRN Mode" active={form.grnMode} onToggle={() => set("grnMode", !form.gr
nMode)} />
432             <SettingChip label="Cash Disabled" active={form.cashDisabled} onToggle={() => set("cashDisa
bled", !form.cashDisabled)} />
433             <SettingChip label="KOT LAN" active={form.kotLan} onToggle={() => set("kotLan", !form.kotLa
n)} />
434             <SettingChip label="Mode Info Mandatory" active={form.modeInfoMandatory} onToggle={() => se
t("modeInfoMandatory", !form.modeInfoMandatory)} />
435         </SettingsGrid>
436     </FormRow>
437
438     <FieldGrid>
439         <FieldGroup label="Bank Payment">
440             <Dropdown value={form.bankPayment} onChange={(value) => set("bankPayment", value)} options=
{BANK_PAYMENTS} />
441         </FieldGroup>
442         <FieldGroup label="Scan Mode">
443             <Dropdown value={form.scanMode} onChange={(value) => set("scanMode", value)} options={SCAN_
MODES} />
444         </FieldGroup>

```

```

445         <FieldGroup label="Device Menu View">
446             <Dropdown value={form.deviceMenuView} onChange={(value) => set("deviceMenuView", value)} op
tions={DEVICE_VIEWS} />
447         </FieldGroup>
448     </FieldGrid>
449
450     <SectionDivider icon={<TapXIcon />} label="TapX Items" />
451
452     <FormRow label="TapX">
453         <SettingsGrid>
454             <SettingChip label="Show in TapX" active={form.showInTapX} onToggle={() => set("showInTapX"
, !form.showInTapX)} />
455         </SettingsGrid>
456     </FormRow>
457 </div>
458
459     <div className="flex shrink-0 items-center justify-end gap-2.5 border-t border-(--line) bg-(--sur
face2) px-7 py-4 max-[640px]:grid max-[640px]:grid-cols-2 max-[640px]:px-4">
460         <button
461             type="button"
462             onClick={onClose}
463             className="flex h-11 items-center justify-center rounded-[10px] border border-(--line) bg-(--
surface) px-6 text-[13.5px] font-semibold text-(--muted) transition hover:border-(--orange) hover:text-(
--orange)"
464             >
465             Cancel
466         </button>
467         <button
468             type="button"
469             onClick={() => onConfirm?.(form)}
470             className="flex h-11 items-center justify-center rounded-[10px] bg-(--text) px-7 text-[13.5px
] font-semibold text-(--bg) transition hover:bg-(--orange)"
471             >
472             Confirm
473         </button>
474     </div>
475 </div>
476 </div>
477 );
478 }

```


FILE: apps/dashboard/src/app/Config/components/EditVendorModal.js

```

1  "use client";
2
3  import { useEffect, useRef, useState } from "react";
4  import {
5    buildVendorPayload,
6    DEFAULT_VENDOR_FORM,
7    numericOnly,
8    VENDOR_TYPES
9  } from "../vendorPayload";
10
11  const FIELD_CLASS =
12    "h-11 w-full rounded-[10px] border border-(--line) bg-(--surface) px-3.5 text-[13px] font-medium text-(
    --text) outline-none transition placeholder:text-(--faint) focus:border-(--orange) focus:shadow-[0_0_3p
    x_rgba(224,68,32,0.12)] max-[640px]:h-10";
13
14  function CloseIcon() {
15    return (
16      <svg viewBox="0 0 14 14" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="1.6"
    strokeLinecap="round">
17        <path d="M3.5 3.5l7 7M10.5 3.5l-7 7" />
18      </svg>
19    );
20  }
21
22  function ChevronIcon() {
23    return (
24      <svg viewBox="0 0 24 24" className="h-4 w-4 shrink-0" fill="none" stroke="currentColor" strokeWidth="
    2" strokeLinecap="round" strokeLinejoin="round">
25        <path d="m6 9 6 6 6-6" />
26      </svg>
27    );
28  }
29
30  function EyeIcon({ hidden }) {
31    if (hidden) {
32      return (
33        <svg viewBox="0 0 24 24" className="h-[17px] w-[17px]" fill="none" stroke="currentColor" strokeWidt
    h="2" strokeLinecap="round" strokeLinejoin="round">
34          <path d="M17.94 17.94A10.07 10.07 0 0 1 12 20c-7 0-11-8-11-8a18.45 18.45 0 0 1 5.06-5.94" />
35          <path d="M9.9 4.24A9.12 9.12 0 0 1 12 4c7 0 11 8 11 8a18.5 18.5 0 0 1-2.16 3.19" />
36          <path d="M1 1l22 22" />
37        </svg>
38      );
39    }
40    return (
41      <svg viewBox="0 0 24 24" className="h-[17px] w-[17px]" fill="none" stroke="currentColor" strokeWidth=
    "2" strokeLinecap="round" strokeLinejoin="round">
42        <path d="M1 12s4-8 11-8 11 8 11 8-4 8-11 8-11-8-11-8z" />
43        <circle cx="12" cy="12" r="3" />
44      </svg>
45    );
46  }
47
48  function PrintIcon() {
49    return (
50      <svg viewBox="0 0 24 24" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="1.6"
    strokeLinecap="round" strokeLinejoin="round">
51        <path d="M6 9V2h12v7" />
52        <path d="M6 18H4a2 2 0 0 1-2-2v-5a2 2 0 0 1 2-2h16a2 2 0 0 1 2 2v5a2 2 0 0 1-2 2h-2" />
53        <path d="M6 14h12v8H6z" />
54      </svg>
55    );
56  }
57
58  function DashboardIcon() {
59    return (
60      <svg viewBox="0 0 24 24" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="1.6"
    strokeLinecap="round" strokeLinejoin="round">
61        <circle cx="12" cy="12" r="10" />

```

```

62     <path d="M2 12h20" />
63     <path d="M12 2a15.3 15.3 0 0 1 4 10 15.3 15.3 0 0 1-4 10 15.3 15.3 0 0 1-4-10 15.3 15.3 0 0 1 4-10z
" />
64   </svg>
65   );
66 }
67
68 function CheckIcon() {
69   return (
70     <svg viewBox="0 0 24 24" className="h-3 w-3" fill="none" stroke="currentColor" strokeWidth="3" stroke
Linecap="round" strokeLinejoin="round">
71       <path d="m5 12 4 4L19 6" />
72     </svg>
73   );
74 }
75
76 function Toggle({ active, onToggle }) {
77   return (
78     <button
79       type="button"
80       onClick={onToggle}
81       aria-pressed={active}
82       className={`flex h-7 w-[54px] shrink-0 items-center rounded-full p-1 transition-all duration-200 ${
83         active ? "bg-[linear-gradient(135deg,#E04420,#341CD6)]" : "bg(--line)"
84       }`}
85     >
86       <span
87         className={`h-5 w-5 rounded-full bg-white shadow-sm transition-transform duration-200 ${
88           active ? "translate-x-[26px]" : "translate-x-0"
89         }`}
90       />
91     </button>
92   );
93 }
94
95 function SettingChip({ label, active, onToggle }) {
96   return (
97     <button
98       type="button"
99       onClick={onToggle}
100       aria-pressed={active}
101       className={`flex h-11 items-center justify-between gap-3 rounded-[10px] border px-3.5 text-left tra
nsition max-[640px]:h-10 ${
102         active
103         ? "border(--orange) bg-[rgba(224,68,32,0.06)] text(--text)"
104         : "border(--line) bg(--surface) text(--muted) hover:border(--orange)"
105       }`}
106     >
107       <span className="truncate text-[12.5px] font-semibold">{label}</span>
108       <span
109         className={`flex h-5 w-5 shrink-0 items-center justify-center rounded-md border transition ${
110           active ? "border(--orange) bg(--orange) text-white" : "border(--line) bg(--surface2) text-t
ransparent"
111         }`}
112       >
113         <CheckIcon />
114       </span>
115     </button>
116   );
117 }
118
119 function TypeDropdown({ value, onChange, options }) {
120   const [open, setOpen] = useState(false);
121   const ref = useRef(null);
122
123   useEffect(() => {
124     const handler = (e) => {
125       if (!ref.current?.contains(e.target)) setOpen(false);
126     };
127     if (open) document.addEventListener("mousedown", handler);

```

```

128     return () => document.removeEventListener("mousedown", handler);
129 }, [open]);
130
131 return (
132   <div ref={ref} className="relative">
133     <button
134       type="button"
135       onClick={() => setOpen((prev) => !prev)}
136       className="flex h-11 w-full items-center justify-between rounded-[10px] bg-[linear-gradient(135deg, #E04420, #341CD6)] px-4 text-[13px] font-semibold tracking-[0.06em] text-white transition hover:brightness-105 max-[640px]:h-10"
137     >
138       <span>{value}</span>
139       <ChevronIcon />
140     </button>
141     {open && (
142       <div className="absolute left-0 top-full z-50 mt-1.5 w-full overflow-hidden rounded-[11px] border border-(--line) bg-(--surface) shadow-(--shadowUp)">
143         {options.map((option) => (
144           <button
145             key={option}
146             type="button"
147             onClick={() => {
148               onChange(option);
149               setOpen(false);
150             }}
151             className={`block w-full px-4 py-2.5 text-left text-[13px] font-semibold tracking-[0.04em] transition hover:bg-(--surface2) ${
152               value === option ? "text-(--orange)" : "text-(--text)"
153             }`}
154           >
155             {option}
156           </button>
157         ))}
158       </div>
159     )}
160   </div>
161 );
162 }
163
164 function SectionDivider({ icon, label }) {
165   return (
166     <div className="flex items-center gap-3 py-1.5">
167       <div className="h-px flex-1 bg-gradient-to-r from-transparent via-(--orange)/25 to-(--purple)/40" /
168     >
169       <span className="font-vcr flex items-center gap-1.5 text-[9px] uppercase tracking-[0.16em] text-(--muted)">
170         {icon}
171         {label}
172       </span>
173       <div className="h-px flex-1 bg-gradient-to-r from-(--purple)/40 via-(--orange)/25 to-transparent" /
174     >
175   </div>
176 );
177 }
178
179 function BoxInput({ value, onChange, type = "text", placeholder = "" }) {
180   return <input type={type} value={value} onChange={onChange} placeholder={placeholder} className={FIELD_CLASS} />;
181 }
182
183 function NumberInput({ value, onChange, placeholder = "" }) {
184   return (
185     <input
186       inputMode="decimal"
187       value={value}
188       onChange={(event) => onChange(numericOnly(event.target.value))}
189       placeholder={placeholder}
190       className={FIELD_CLASS}
191     />
192 )

```

```

190   );
191 }
192
193 function PasswordInput({ value, onChange }) {
194   const [show, setShow] = useState(false);
195
196   return (
197     <div className="relative">
198       <input type={show ? "text" : "password"} value={value} onChange={onChange} className={` ${FIELD_CLAS
199 } pr-10`} />
200       <button
201         type="button"
202         onClick={() => setShow(prev => !prev)}
203         className="absolute right-3 top-1/2 flex h-6 w-6 -translate-y-1/2 items-center justify-center rounded-md text-(--muted) transition hover:bg-(--chip) hover:text-(--text)"
204         aria-label={show ? "Hide password" : "Show password"}
205       >
206         <EyeIcon hidden={show} />
207       </button>
208     </div>
209   );
210 }
211
212 function CharTextarea({ value, onChange, maxLength = 128, placeholder = "" }) {
213   return (
214     <div className="relative">
215       <textarea
216         value={value}
217         onChange={onChange}
218         maxLength={maxLength}
219         placeholder={placeholder}
220         rows={3}
221         className="w-full resize-y rounded-[10px] border border-(--line) bg-(--surface) px-3.5 py-2.5 pr-14 text-[13px] font-medium text-(--text) outline-none transition placeholder:text-(--faint) focus:border-(--orange) focus:shadow-[0_0_3px_rgba(224,68,32,0.12)]"
222       />
223       <span className="font-vcr absolute bottom-2.5 right-3 text-[9px] text-(--faint)">
224         {String(value ?? "").length}/{maxLength}
225       </span>
226     </div>
227   );
228 }
229
230 function FormRow({ label, children }) {
231   return (
232     <div className="grid grid-cols-[170px_1fr] items-start gap-4 max-sm:grid-cols-1 max-sm:gap-1.5">
233       <span className="pt-2.5 text-right text-[13px] font-medium text-(--muted) max-sm:pt-0 max-sm:text-left">
234         {label}
235       </span>
236       <div>{children}</div>
237     </div>
238   );
239 }
240
241 function FieldGrid({ children }) {
242   return <div className="grid grid-cols-3 gap-3 max-md:grid-cols-1">{children}</div>;
243 }
244
245 function SettingsGrid({ children }) {
246   return <div className="grid grid-cols-2 gap-2.5 max-sm:grid-cols-1">{children}</div>;
247 }
248
249 function FieldGroup({ label, children }) {
250   return (
251     <div>
252       <div className="font-vcr mb-1.5 text-[8.5px] uppercase tracking-[0.16em] text-(--muted)">
253         {label}
254       </div>
255       {children}
256     </div>
257   );
258 }

```

```

255     </div>
256   );
257 }
258
259 function FormatSelector({ value, onChange }) {
260   return (
261     <div className="grid grid-cols-2 overflow-hidden rounded-[10px] border border-(--line) bg-(--surface)
262       p-1">
263       {[
264         ["format1", "Default"],
265         ["format2", "Format2"]
266       ].map(([, option, label]) => (
267         <button
268           key={option}
269           type="button"
270           onClick={() => onChange(option)}
271           className={`h-8 rounded-md text-[12px] font-semibold transition ${
272             value === option ? "bg-(--text) text-(--bg)" : "text-(--muted) hover:bg-(--surface2)"
273           }`}
274           >
275             {label}
276           </button>
277         ))}
278     </div>
279   );
280 }
281
282 export default function EditVendorModal({ vendor, eventId, onClose, onConfirm }) {
283   const vendorName = vendor?.name ?? vendor?.vendorName ?? vendor?.title ?? "";
284   const vendorType = String(vendor?.type ?? vendor?.category ?? "FNB").toUpperCase();
285
286   const [form, setForm] = useState({
287     ...DEFAULT_VENDOR_FORM,
288     vendorName,
289     type: VENDOR_TYPES.includes(vendorType) ? vendorType : "INVENTORY",
290     revShare: vendor?.revShare ?? "",
291     gstin: vendor?.gstin ?? "",
292     pan: vendor?.pan ?? "",
293     serviceCharge: vendor?.sc ?? vendor?.serviceCharge ?? "",
294     serviceChargeTax: vendor?.scTax ?? vendor?.serviceChargeTax ?? "",
295     dashboardPassword: vendor?.password ?? "",
296     mobile: vendor?.mobile ?? "",
297     address: vendor?.address ?? "",
298     thankYouNote: vendor?.thankyouNote ?? vendor?.thankYouNote ?? DEFAULT_VENDOR_FORM.thankYouNote,
299     sac: Boolean(vendor?.showSac),
300     sacText: vendor?.sac ?? "",
301     autoPrint: Boolean(vendor?.autoPrint),
302     maxInvoices: vendor?.invoicePrintCount ?? "",
303     kot: Boolean(vendor?.kotPrint),
304     kotCount: vendor?.kotPrintCount ?? "",
305     invoiceFormat: vendor?.printFormat ?? "format1",
306     reprintPassword: vendor?.rePrintPassword ?? "",
307   });
308   const overlayRef = useRef(null);
309
310   const set = (key, value) => setForm((prev) => ({ ...prev, [key]: value }));
311
312   useEffect(() => {
313     const handler = (e) => {
314       if (e.key === "Escape") onClose?();
315     };
316     const previousOverflow = document.body.style.overflow;
317     document.body.style.overflow = "hidden";
318     document.addEventListener("keydown", handler);
319     return () => {
320       document.body.style.overflow = previousOverflow;
321       document.removeEventListener("keydown", handler);
322     };
323   }, [onClose]);

```

```

324
325   return (
326     <div
327       ref={overlayRef}
328       onMouseDown={(e) => {
329         if (e.target === overlayRef.current) onClose?();
330       }}
331       className="fixed inset-0 z-[200] flex items-center justify-center overscroll-none bg-[rgba(12,12,12
,0.5)] px-4 py-6 backdrop-blur-[3px] max-[640px]:p-0"
332     >
333       <div className="flex max-h-[calc(100dvh-48px)] w-full max-w-[860px] flex-col overflow-hidden rounded
d-[16px] border border-(--line) bg-(--surface) shadow-(--shadowUp) max-[640px]:h-full max-[640px]:max-h-f
ull max-[640px]:rounded-none">
334         <div
335           className="relative shrink-0 overflow-hidden px-7 py-4 max-[640px]:px-4"
336           style={{ background: "linear-gradient(120deg,#1C1C1C 0%,#341CD6 62%,#E04420 130%)" }}
337         >
338           <div
339             className="pointer-events-none absolute inset-0 opacity-[0.12]"
340             style={{ background: "repeating-linear-gradient(90deg,#fff 0 1px,transparent 1px 46px)" }}
341             aria-hidden
342           />
343           <div className="relative flex items-start justify-between gap-3">
344             <div className="min-w-0">
345               <p className="font-vcr m-0 text-[9px] uppercase tracking-[0.2em] text-(--purple)">
346                 Edit Vendor
347               </p>
348               <h2 className="font-chillax m-0 mt-1 truncate text-[clamp(20px,2.4vw,26px)] font-semibold t
racking-[-0.01em] text-white">
349                 {vendorName || "Vendor"}
350               </h2>
351             </div>
352             <button
353               type="button"
354               onClick={onClose}
355               className="grid h-8 w-8 shrink-0 place-items-center rounded-[9px] border border-white/20 te
xt-white/80 transition hover:border-(--orange) hover:bg-(--orange)"
356               aria-label="Close edit vendor modal"
357             >
358               <CloseIcon />
359             </button>
360           </div>
361         </div>
362
363         <div className="min-h-0 flex-1 space-y-4 overflow-y-auto px-7 py-5 max-[640px]:px-4">
364           <FormRow label="Vendor Name">
365             <BoxInput value={form.vendorName} onChange={(e) => set("vendorName", e.target.value)} />
366           </FormRow>
367
368           <FieldGrid>
369             <FieldGroup label="Type">
370               <TypeDropdown value={form.type} onChange={(value) => set("type", value)} options={VENDOR_TY
PES} />
371             </FieldGroup>
372             <FieldGroup label="Rev Share">
373               <BoxInput value={form.revShare} onChange={(e) => set("revShare", numericOnly(e.target.value
))} />
374             </FieldGroup>
375             <FieldGroup label="GSTIN">
376               <BoxInput value={form.gstin} onChange={(e) => set("gstin", e.target.value)} />
377             </FieldGroup>
378             <FieldGroup label="PAN">
379               <BoxInput value={form.pan} onChange={(e) => set("pan", e.target.value)} />
380             </FieldGroup>
381             <FieldGroup label="Service Charge">
382               <BoxInput value={form.serviceCharge} onChange={(e) => set("serviceCharge", numericOnly(e.ta
rget.value))} />
383             </FieldGroup>
384             <FieldGroup label="Service Charge Tax">
385               <BoxInput value={form.serviceChargeTax} onChange={(e) => set("serviceChargeTax", numericOnl

```

```

    y(e.target.value))) />
386     </FieldGroup>
387   </FieldGrid>
388
389   <FormRow label="Show Print Details">
390     <Toggle active={form.showPrintDetails} onToggle={() => set("showPrintDetails", !form.showPrintDetails)} />
391   </FormRow>
392
393   {form.showPrintDetails && (
394     <div className="space-y-4 rounded-[12px] border border-(--line) bg-(--surface2) px-4 py-4">
395       <SectionDivider icon={<PrintIcon />} label="Print Details" />
396
397       <FieldGrid>
398         <FieldGroup label="Mobile">
399           <BoxInput type="tel" value={form.mobile} onChange={(e) => set("mobile", numericOnly(e.target.value))} />
400         </FieldGroup>
401         <FieldGroup label="Address">
402           <CharTextarea value={form.address} onChange={(e) => set("address", e.target.value)} maxLength={128} />
403         </FieldGroup>
404         <FieldGroup label="Thank You Note">
405           <CharTextarea value={form.thankYouNote} onChange={(e) => set("thankYouNote", e.target.value)} maxLength={128} />
406         </FieldGroup>
407       </FieldGrid>
408
409       <FormRow label="Print Settings">
410         <SettingsGrid>
411           <SettingChip label="SAC" active={form.sac} onToggle={() => set("sac", !form.sac)} />
412           <SettingChip label="Auto Print" active={form.autoPrint} onToggle={() => set("autoPrint", !form.autoPrint)} />
413           <SettingChip label="KOT" active={form.kot} onToggle={() => set("kot", !form.kot)} />
414         </SettingsGrid>
415       </FormRow>
416
417       {form.sac && (
418         <FormRow label="SAC Value">
419           <BoxInput value={form.sacText} onChange={(e) => set("sacText", e.target.value)} />
420         </FormRow>
421       )}
422
423       <FieldGrid>
424         <FieldGroup label="Max Invoices">
425           <NumberInput value={form.maxInvoices} onChange={(value) => set("maxInvoices", value)} />
426         </FieldGroup>
427         {form.kot && (
428           <FieldGroup label="KOT Count">
429             <NumberInput value={form.kotCount} onChange={(value) => set("kotCount", value)} />
430           </FieldGroup>
431         )}
432         <FieldGroup label="Re-Print Password">
433           <PasswordInput value={form.reprintPassword} onChange={(e) => set("reprintPassword", e.target.value)} />
434         </FieldGroup>
435       </FieldGrid>
436
437       <FormRow label="Invoice Format">
438         <FormatSelector value={form.invoiceFormat} onChange={(value) => set("invoiceFormat", value)} />
439       </FormRow>
440     </div>
441   )}
442
443   <SectionDivider icon={<DashboardIcon />} label="Dashboard" />
444
445   <FormRow label="Dashboard Password">
446     <PasswordInput value={form.dashboardPassword} onChange={(e) => set("dashboardPassword", e.target.value)} />

```

```
    get.value)) />
447     </FormRow>
448   </div>
449
450   <div className="flex shrink-0 items-center justify-end gap-2.5 border-t border-(--line) bg-(--sur
face2) px-7 py-4 max-[640px]:grid max-[640px]:grid-cols-2 max-[640px]:px-4">
451     <button
452       type="button"
453       onClick={onClose}
454       className="flex h-11 items-center justify-center rounded-[10px] border border-(--line) bg-(--
surface) px-6 text-[13.5px] font-semibold text-(--muted) transition hover:border-(--orange) hover:text-(
--orange)"
455     >
456       Cancel
457     </button>
458     <button
459       type="button"
460       onClick={() => onConfirm?.(buildVendorPayload(form, eventId))}
461       className="flex h-11 items-center justify-center rounded-[10px] bg-(--text) px-7 text-[13.5px
] font-semibold text-(--bg) transition hover:bg-(--orange)"
462     >
463       Confirm
464     </button>
465   </div>
466 </div>
467 </div>
468 );
469 }
```


FILE: apps/dashboard/src/app/Config/components/VendorConfigurationContent.js

```

1  "use client";
2
3  import { useCallback, useEffect, useMemo, useRef, useState } from "react";
4  import { AtomXLoader } from "@atomx/global-components";
5  import { useRouter } from "next/navigation";
6  import {
7    addDevicesToStall,
8    createAccessXCategory,
9    createAccessXGateMaster,
10   createStall,
11   createVendor,
12   fetchAccessXCategories,
13   fetchAccessXGateMasters,
14   fetchAccessXGates,
15   fetchStalls,
16   fetchVendors,
17   updateAccessXCategory,
18   updateAccessXGateMaster,
19   updateVendor
20 } from "../../lib/dashboardApi";
21 import { useDashboardStore } from "../../store/dashboardStore";
22 import {
23   DeviceIcon,
24   EditIcon,
25   GridIcon,
26   LinkIcon,
27   ListIcon,
28   PlusIcon,
29   SearchIcon
30 } from "../ConfigIcons";
31 import CreateStallModal from "../CreateStallModal";
32 import EditVendorModal from "../EditVendorModal";
33 import CreateVendorModal from "../CreateVendorModal";
34 import EditStallModal from "../EditStallModal";
35 import EditAccessXCategoryModal from "../EditAccessXCategoryModal";
36 import Access_Gate_Master from "../Access_Gate_Master";
37 import Access_Gate_Config from "../Access_Gate_Config";
38 import CreateAccessGateMasterModal from "../CreateAccessGateMasterModal";
39 import EditAccessGateMasterModal from "../EditAccessGateMasterModal";
40 import EditAccessGateConfigModal from "../EditAccessGateConfigModal";
41 import AddDevice from "../Add_Device";
42 import { buildCreateStallPayload } from "../stallPayload";
43
44 function getVendorId(vendor, index) {
45   return vendor?.id ?? vendor?.vendorId ?? index + 1;
46 }
47
48 function getVendorName(vendor) {
49   return vendor?.name ?? vendor?.vendorName ?? vendor?.title ?? "-";
50 }
51
52 function getVendorType(vendor) {
53   return String(vendor?.type ?? vendor?.category ?? "-").toUpperCase();
54 }
55
56 function getVendorLogin(vendor) {
57   return (
58     vendor?.loginCode ??
59     vendor?.login ??
60     vendor?.code ??
61     vendor?.password ??
62     vendor?.devicePassword ??
63     vendor?.id ??
64     "-"
65   );
66 }
67
68 function getCreateStallVendorId(vendor, index) {
69   return (

```

```
70     vendor?.loginCode ??
71     vendor?.login ??
72     vendor?.code ??
73     vendor?.password ??
74     vendor?.vendorId ??
75     vendor?.id ??
76     index + 1
77   );
78 }
79
80 function getVendorLink(vendor) {
81   return vendor?.link ?? vendor?.url ?? vendor?.linkUrl ?? vendor?.website ?? "";
82 }
83
84 function getStallId(stall, index) {
85   return stall?.id ?? stall?.stallId ?? index + 1;
86 }
87
88 function getStallApiId(stall) {
89   return stall?.id ?? stall?.stallId ?? stall?.stall_id ?? null;
90 }
91
92 function getStallVendor(stall) {
93   return stall?.vendorName ?? stall?.vendor?.name ?? stall?.vendor ?? "-";
94 }
95
96 function getStallName(stall) {
97   return stall?.name ?? stall?.stallName ?? stall?.stall ?? "-";
98 }
99
100 function getStallType(stall) {
101   return String(stall?.type ?? stall?.category ?? "").trim().toLowerCase();
102 }
103
104 function isStockmasterStall(stall) {
105   return getStallType(stall) === "stockmaster";
106 }
107
108 function isTableStall(stall) {
109   return getStallType(stall) === "tables";
110 }
111
112 function isAccessXStall(stall) {
113   return getStallType(stall) === "accessx";
114 }
115
116 function isGroupedStall(stall) {
117   return isStockmasterStall(stall) || isTableStall(stall) || isAccessXStall(stall);
118 }
119
120 function filterStallsBySearch(stallList, query) {
121   if (!query) return stallList;
122   return stallList.filter((stall, index) =>
123     [getStallId(stall, index), getStallVendor(stall), getStallName(stall), getStallType(stall)]
124       .join(" ")
125       .toLowerCase()
126       .includes(query)
127   );
128 }
129
130 function normalizeAccessXCategories(response) {
131   const categories =
132     response?.categories ??
133     response?.data?.categories ??
134     response?.data ??
135     response?.list ??
136     [];
137   return Array.isArray(categories) ? categories : [];
138 }
139
```

```
140 function normalizeAccessXGateMasters(response) {
141   const gateMasters =
142     response?.gatesmasters ??
143     response?.gateMasters ??
144     response?.gatesMasters ??
145     response?.gatesMaster ??
146     response?.masters ??
147     response?.gates ??
148     response?.data?.gateMasters ??
149     response?.data?.gatesmasters ??
150     response?.data?.gatesMasters ??
151     response?.data?.gatesMaster ??
152     response?.data?.masters ??
153     response?.data?.gates ??
154     response?.data ??
155     response?.list ??
156     [];
157   return Array.isArray(gateMasters) ? gateMasters : [];
158 }
159
160 function normalizeAccessXGates(response) {
161   const gates =
162     response?.gates ??
163     response?.data?.gates ??
164     response?.data ??
165     response?.list ??
166     [];
167   return Array.isArray(gates) ? gates : [];
168 }
169
170 function getGateMasterName(gate) {
171   return gate?.name ?? gate?.gateName ?? gate?.gate_name ?? gate?.title ?? "-";
172 }
173
174 function getCategoryAllowLabel(category) {
175   const allowCount = Number(category?.allowCount);
176   return allowCount === 0 ? "ALL" : String(allowCount);
177 }
178
179 function normalizeDeviceList(list) {
180   if (Array.isArray(list)) {
181     return list.filter((device) => device !== null && device !== undefined && String(device).trim() !== "
182   ");
183   }
184   if (typeof list === "string") {
185     return list
186       .split(",")
187       .map((device) => device.trim())
188       .filter(Boolean);
189   }
190   if (list === null || list === undefined || list === "") {
191     return [];
192   }
193   return [list];
194 }
195
196 function getDeviceCount(stall) {
197   const devices = normalizeDeviceList(
198     stall?.devices_list ??
199     stall?.device_list ??
200     stall?.devicesList ??
201     stall?.devices ??
202     stall?.device
203   );
204   if (devices.length > 0) {
205     return devices.length;
206   }
207 }
208
```

```

209   }
210
211   const count =
212     stall?.deviceCount ??
213     stall?.devicesCount ??
214     stall?.devices_count ??
215     stall?.device_count;
216
217   return typeof count === "number" ? count : 0;
218 }
219
220 function EmptyState({ children }) {
221   return (
222     <div className="rounded-[11px] border border-dashed border-(--line) px-4 py-8 text-center text-[13px]
223       font-medium text-(--muted)">
224       {children}
225     </div>
226   );
227 }
228
229 function LoadingState({ label }) {
230   return (
231     <div className="flex min-h-[118px] items-center justify-center">
232       <AtomXLoader label={label} size={46} />
233     </div>
234   );
235 }
236
237 function ActionButton({ children, label, active = false, as: Component = "button", ...props }) {
238   return (
239     <Component
240       type={Component === "button" ? "button" : undefined}
241       aria-label={label}
242       title={label}
243       className={`inline-flex h-[30px] w-[30px] shrink-0 items-center justify-center rounded-[8px] transi
244         tion ${
245           active
246             ? "bg-(--text) text-(--bg) hover:bg-(--orange)"
247             : "border border-(--line) bg-(--surface) text-(--muted) hover:border-(--orange) hover:text-(--o
248             range)"
249         }`
250       {...props}
251     >
252       {children}
253     </Component>
254   );
255 }
256
257 function ConfigSearchField({ value, onChange, placeholder }) {
258   return (
259     <label className="flex min-w-[160px] flex-1 items-center gap-2 border-b border-(--line) pb-1.5 text-(
260       --muted) transition focus-within:border-(--orange) max-[640px]:min-w-0">
261       <span className="sr-only">{placeholder}</span>
262       <SearchIcon className="h-4 w-4 shrink-0 opacity-60" />
263       <input
264         value={value}
265         onChange={(event) => onChange(event.target.value)}
266         placeholder={placeholder}
267         className="min-w-0 flex-1 bg-transparent text-[13px] font-medium text-(--text) outline-none place
268         holder:text-(--faint)"
269       />
270     </label>
271   );
272 }
273
274 function ConfigPanel({
275   title,
276   count,
277   searchValue,
278   onSearchChange,

```

```

274   searchPlaceholder,
275   action,
276   children,
277   theme = "default"
278 }) {
279   const themes = {
280     default: { accent: "#e04420", tile: "linear-gradient(140deg,#e04420,#8b5cf6)" },
281     stockroom: { accent: "#00a9f2", tile: "linear-gradient(140deg,#00a9f2,#341cd6)" },
282     tables: { accent: "#e08a20", tile: "linear-gradient(140deg,#e04420,#e08a20)" },
283     accessx: { accent: "#341cd6", tile: "linear-gradient(140deg,#8b5cf6,#341cd6)" }
284   };
285   const palette = themes[theme] || themes.default;
286
287   return (
288     <section
289       className="rounded-[15px] border border-(--line) border-l-[3px] bg-(--surface) px-4 py-4 shadow-(--
shadow) max-[640px]:px-3 max-[640px]:py-3"
290       style={{ borderLeftColor: palette.accent }}
291     >
292       <div className="flex flex-wrap items-center gap-3 border-b border-(--line2) pb-3">
293         <div className="flex min-w-fit items-center gap-2.5">
294           <span
295             className="font-vcr flex h-9 w-9 items-center justify-center rounded-[10px] text-[12px] text-
white"
296             style={{ background: palette.tile }}
297           >
298             {String(count).padStart(2, "0")}
299           </span>
300           <h2 className="font-chillax text-[18px] font-semibold tracking-[-0.01em] text-(--text)">
301             {title}
302           </h2>
303         </div>
304         <ConfigSearchField
305           value={searchValue}
306           onChange={onSearchChange}
307           placeholder={searchPlaceholder}
308         />
309         {action ? <div className="ml-auto max-[640px]:ml-0 max-[640px]:w-full">{action}</div> : null}
310       </div>
311       <div className="pt-3">{children}</div>
312     </section>
313   );
314 }
315
316 function ScrollRows({ children }) {
317   return (
318     <div className="max-h-[333px] space-y-2.5 overflow-y-auto pr-1 max-[640px]:max-h-[360px] max-[640px]:
space-y-2">
319       {children}
320     </div>
321   );
322 }
323
324 function VendorRow({ vendor, index, onAddStall, onEditVendor }) {
325   const link = getVendorLink(vendor);
326
327   return (
328     <div
329       className="rounded-[11px] border border-transparent p-px transition duration-200 hover:shadow-(--sh
adow)"
330       style={{
331         background:
332           "linear-gradient(var(--surface),var(--surface)) padding-box, linear-gradient(135deg,rgba(224,68
,32,.34),rgba(139,92,246,.26) 52%,rgba(0,169,242,.22)) border-box"
333       }}
334     >
335       <div className="flex flex-wrap items-center gap-x-3 gap-y-2 rounded-[10px] bg-(--surface) px-3 py-2
.5">
336         <span className="font-vcr grid h-9 w-9 shrink-0 place-items-center rounded-[9px] bg-(--surface2)
text-[12px] text-(--orange)">

```

```

337     #{index + 1}
338   </span>
339   <div className="min-w-0 flex-[1_1_140px]">
340     <div className="font-vcr text-[7.5px] tracking-[0.15em] text-(--faint)">NAME</div>
341     <div className="mt-0.5 truncate text-[13.5px] font-semibold text-(--text)">
342       {getVendorName(vendor)}
343     </div>
344   </div>
345   <div className="min-w-0 flex-[1_1_80px]">
346     <div className="font-vcr text-[7.5px] tracking-[0.15em] text-(--faint)">TYPE</div>
347     <div className="mt-0.5 truncate text-[13px] font-semibold text-(--text)">
348       {getVendorType(vendor)}
349     </div>
350   </div>
351   <div className="min-w-0 flex-[1_1_80px] max-[640px]:hidden">
352     <div className="font-vcr text-[7.5px] tracking-[0.15em] text-(--faint)">LOGIN</div>
353     <div className="font-vcr mt-0.5 truncate text-[12.5px] text-(--muted)">
354       {getVendorLogin(vendor)}
355     </div>
356   </div>
357   <div className="ml-auto flex shrink-0 items-center gap-1.5">
358     <ActionButton label="Open items">
359       <GridIcon />
360     </ActionButton>
361     {link ? (
362       <ActionButton as="a" href={link} target="_blank" rel="noreferrer" label="Open vendor link">
363         <LinkIcon />
364       </ActionButton>
365     ) : (
366       <ActionButton label="No vendor link">
367         <LinkIcon />
368       </ActionButton>
369     )}
370     <ActionButton label="Add stall" active onClick={onAddStall}>
371       <PlusIcon />
372     </ActionButton>
373     <ActionButton label="Edit vendor" onClick={onEditVendor}>
374       <EditIcon />
375     </ActionButton>
376   </div>
377 </div>
378 </div>
379 );
380 }
381
382 function StallRow({ stall, index, onEditStall, onAddDevice, onOpenMenu }) {
383   return (
384     <div
385       className="rounded-[11px] border border-transparent p-px transition duration-200 hover:shadow-(--shadow)"
386       style={{
387         background:
388           "linear-gradient(var(--surface),var(--surface)) padding-box, linear-gradient(135deg, rgba(224,68
389           ,32,.34), rgba(139,92,246,.26) 52%, rgba(0,169,242,.22)) border-box"
390       }}
391     >
392       <div className="flex flex-wrap items-center gap-x-3 gap-y-2 rounded-[10px] bg-(--surface) px-3 py-2
393       .5">
394         <span className="font-vcr shrink-0 text-[13px] text-(--orange)">#{getStallId(stall, index)}</span>
395         <div className="min-w-0 flex-[1_1_100px]">
396           <div className="font-vcr text-[7.5px] tracking-[0.15em] text-(--faint)">VENDOR</div>
397           <div className="mt-0.5 truncate text-[13px] font-semibold text-(--text)">
398             {getStallVendor(stall)}
399           </div>
400         </div>
401         <div className="min-w-0 flex-[1_1_100px]">
402           <div className="font-vcr text-[7.5px] tracking-[0.15em] text-(--faint)">STALL</div>
403           <div className="mt-0.5 truncate text-[13px] font-semibold text-(--text)">
404             {getStallName(stall)}

```

```

403         </div>
404     </div>
405     <div className="ml-auto flex shrink-0 items-center gap-1.5">
406         <button
407             type="button"
408             onClick={onAddDevice}
409             className="inline-flex h-[30px] items-center gap-1.5 rounded-[8px] bg-(--text) px-2.5 text-[1
2px] font-semibold text-(--bg) transition hover:bg-(--orange)"
410             aria-label={`Add device to ${getStallName(stall)}`}
411             title="Add device"
412         >
413             <span className="font-vcr">{getDeviceCount(stall)}</span>
414             <DeviceIcon />
415             <span className="h-4 w-px bg-white/25" />
416             <PlusIcon />
417         </button>
418         <ActionButton label="Open menu" onClick={onOpenMenu}>
419             <ListIcon />
420         </ActionButton>
421         <ActionButton label="Edit stall" onClick={onEditStall}>
422             <EditIcon />
423         </ActionButton>
424     </div>
425 </div>
426 </div>
427 );
428 }
429
430 function AccessXCategoryTable({ categories, onEdit }) {
431     return (
432         <div className="max-h-[333px] overflow-auto rounded-[11px] border border-(--line)">
433             <table className="w-full min-w-[430px] border-collapse">
434                 <thead className="font-vcr sticky top-0 z-[1] bg-(--surface2)">
435                     <tr>
436                         <th className="w-[64px] border-b border-(--line) px-3 py-2.5 text-left text-[9.5px] tracking-
[0.15em] text-(--orange)">
437                             NO.
438                         </th>
439                         <th className="border-b border-(--line) px-3 py-2.5 text-left text-[9.5px] tracking-[0.15em]
text-(--orange)">
440                             CATEGORY
441                         </th>
442                         <th className="w-[90px] border-b border-(--line) px-3 py-2.5 text-left text-[9.5px] tracking-
[0.15em] text-(--orange)">
443                             ALLOW
444                         </th>
445                         <th className="w-[64px] border-b border-(--line) px-3 py-2.5 text-center text-[9.5px] trackin
g-[0.15em] text-(--orange)">
446                             EDIT
447                         </th>
448                     </tr>
449                 </thead>
450                 <tbody>
451                     {categories.map((category, index) => (
452                         <tr key={category?.id ?? `${category?.name}-${index}`} className="transition hover:bg-(--surf
ace2)">
453                             <td className="font-vcr border-b border-(--line2) px-3 py-2.5 text-[12px] text-(--orange)">
454                                 {index + 1}
455                             </td>
456                             <td className="border-b border-(--line2) px-3 py-2.5">
457                                 <span className="block truncate text-[13px] font-semibold text-(--text)">
458                                     {category?.name || "-"}
459                                 </span>
460                             </td>
461                             <td className="border-b border-(--line2) px-3 py-2.5">
462                                 <span className="font-vcr inline-flex min-w-8 items-center justify-center rounded-[6px] b
g-(--chip) px-2 py-1 text-[10px] text-(--blue)">
463                                     {getCategoryAllowLabel(category)}
464                                 </span>
465                             </td>

```

```

466         <td className="border-b border-(--line2) px-3 py-2.5 text-center">
467             <ActionButton label={`Edit ${category?.name} | "category"}`} onClick={() => onEdit(catego
ry)}}>
468                 <EditIcon />
469             </ActionButton>
470         </td>
471     </tr>
472 }}
473 </tbody>
474 </table>
475 </div>
476 );
477 }
478
479 export default function VendorConfigurationContent() {
480     const router = useRouter();
481     const token = useDashboardStore((state) => state.token);
482     const eventMeta = useDashboardStore((state) => state.eventMeta);
483     const eventDetails = useDashboardStore((state) => state.eventDetails);
484     const eventId = eventMeta?.eventId ?? eventDetails?.id;
485     const eventName =
486         eventDetails?.name ??
487         eventMeta?.eventName ??
488         eventMeta?.name ??
489         "";
490     const cachedVendors = useDashboardStore((state) =>
491         eventId ? state.vendorsByEventId?.[eventId] : undefined
492     );
493     const cachedStalls = useDashboardStore((state) =>
494         eventId ? state.stallsByEventId?.[eventId] : undefined
495     );
496     const setVendorsForEvent = useDashboardStore((state) => state.setVendorsForEvent);
497     const setStallsForEvent = useDashboardStore((state) => state.setStallsForEvent);
498     const [vendors, setVendors] = useState(() => cachedVendors || []);
499     const [stalls, setStalls] = useState(() => cachedStalls || []);
500     const [vendorQuery, setVendorQuery] = useState("");
501     const [stallQuery, setStallQuery] = useState("");
502     const [categoryQuery, setCategoryQuery] = useState("");
503     const [gateMasterQuery, setGateMasterQuery] = useState("");
504     const [gateConfigQuery, setGateConfigQuery] = useState("");
505     const [accessXCategories, setAccessXCategories] = useState([]);
506     const [accessXGateMasters, setAccessXGateMasters] = useState([]);
507     const [accessXGates, setAccessXGates] = useState([]);
508     const [createStallFor, setCreateStallFor] = useState(null);
509     const [editVendor, setEditVendor] = useState(null);
510     const [showCreateVendor, setShowCreateVendor] = useState(false);
511     const [editStall, setEditStall] = useState(null);
512     const [addDeviceStall, setAddDeviceStall] = useState(null);
513     const [editAccessXCategory, setEditAccessXCategory] = useState(null);
514     const [showCreateAccessXCategory, setShowCreateAccessXCategory] = useState(false);
515     const [showCreateAccessGateMaster, setShowCreateAccessGateMaster] = useState(false);
516     const [editAccessGateMaster, setEditAccessGateMaster] = useState(null);
517     const [editAccessGateConfig, setEditAccessGateConfig] = useState(null);
518     const [vendorsLoading, setVendorsLoading] = useState(false);
519     const [stallsLoading, setStallsLoading] = useState(false);
520     const [accessXCategoriesLoading, setAccessXCategoriesLoading] = useState(false);
521     const [accessXGateMastersLoading, setAccessXGateMastersLoading] = useState(false);
522     const [accessXGatesLoading, setAccessXGatesLoading] = useState(false);
523     const [vendorsError, setVendorsError] = useState("");
524     const [stallsError, setStallsError] = useState("");
525     const [accessXCategoriesError, setAccessXCategoriesError] = useState("");
526     const [accessXGateMastersError, setAccessXGateMastersError] = useState("");
527     const [accessXGatesError, setAccessXGatesError] = useState("");
528     const requestedAccessXEventIds = useRef(new Set());
529     const activeAccessXEventId = useRef("");
530
531     const loadVendors = useCallback(
532         async ({ force = false } = {}) => {
533             if (!eventId) return;
534             setVendorsLoading(true);

```



```

535     setVendorsError("");
536     try {
537         const list = await fetchVendors({ eventId, token, dedupe: !force });
538         const normalized = Array.isArray(list) ? list : [];
539         setVendors(normalized);
540         setVendorsForEvent(eventId, normalized);
541     } catch (error) {
542         console.error("Failed to load vendors", error);
543         setVendors([]);
544         setVendorsError("Unable to load vendors.");
545     } finally {
546         setVendorsLoading(false);
547     }
548 },
549 [eventId, token, setVendorsForEvent]
550 );
551
552 const loadStalls = useCallback(
553     async ({ force = false } = {}) => {
554         if (!eventId) return;
555         setStallsLoading(true);
556         setStallsError("");
557         try {
558             const list = await fetchStalls({ eventId, token, dedupe: !force });
559             const normalized = Array.isArray(list) ? list : [];
560             setStalls(normalized);
561             setStallsForEvent(eventId, normalized);
562         } catch {
563             setStalls([]);
564             setStallsError("Unable to load stalls.");
565         } finally {
566             setStallsLoading(false);
567         }
568     },
569     [eventId, token, setStallsForEvent]
570 );
571
572 const handleOpenStallMenu = useCallback(
573     (stall) => {
574         const stallId = getStallApiId(stall);
575         if (!stallId) {
576             console.error("Unable to load stall menu: missing stall ID", stall);
577             return;
578         }
579
580         const params = new URLSearchParams({
581             stallId: String(stallId),
582             stallName: getStallName(stall)
583         });
584         router.push(`/Config/menu?${params.toString()}`);
585     },
586     [router]
587 );
588
589 useEffect(() => {
590     loadVendors();
591     loadStalls();
592 }, [loadVendors, loadStalls]);
593
594 useEffect(() => {
595     if (!eventId) return;
596
597     const requestKey = String(eventId);
598     if (requestedAccessXEventIds.current.has(requestKey)) return;
599     requestedAccessXEventIds.current.add(requestKey);
600     activeAccessXEventId.current = requestKey;
601     setAccessXCategories([]);
602     setAccessXGateMasters([]);
603     setAccessXGates([]);
604     setAccessXCategoriesError("");

```

```
605   setAccessXGateMastersError("");
606   setAccessXGatesError("");
607   setAccessXCategoriesLoading(true);
608   setAccessXGateMastersLoading(true);
609   setAccessXGatesLoading(true);
610
611   Promise.allSettled([
612     fetchAccessXCategories({ eventId, token }),
613     fetchAccessXGateMasters({ eventId, token }),
614     fetchAccessXGates({ eventId, token })
615   ]).then((results) => {
616     if (activeAccessXEventId.current !== requestKey) return;
617
618     const categoryResult = results[0];
619     if (categoryResult.status === "fulfilled") {
620       setAccessXCategories(normalizeAccessXCategories(categoryResult.value));
621     } else {
622       setAccessXCategoriesError("Unable to load AccessX categories.");
623     }
624     setAccessXCategoriesLoading(false);
625
626     const gateMasterResult = results[1];
627     if (gateMasterResult.status === "fulfilled") {
628       setAccessXGateMasters(normalizeAccessXGateMasters(gateMasterResult.value));
629     } else {
630       setAccessXGateMastersError("Unable to load Access Gate Masterlist.");
631     }
632     setAccessXGateMastersLoading(false);
633
634     const gatesResult = results[2];
635     if (gatesResult.status === "fulfilled") {
636       setAccessXGates(normalizeAccessXGates(gatesResult.value));
637     } else {
638       setAccessXGatesError("Unable to load Access Gate Config.");
639     }
640     setAccessXGatesLoading(false);
641
642     results.forEach((result, index) => {
643       if (result.status === "rejected") {
644         const labels = ["categories", "gate masters", "gates"];
645         console.error(`Failed to load AccessX ${labels[index]}`, result.reason);
646       }
647     });
648   });
649 }, [eventId, token]);
650
651 const filteredVendors = useMemo(() => {
652   const query = vendorQuery.trim().toLowerCase();
653   if (!query) return vendors;
654   return vendors.filter((vendor) =>
655     [
656       getVendorId(vendor),
657       getVendorName(vendor),
658       getVendorType(vendor),
659       getVendorLogin(vendor)
660     ]
661     .join(" ")
662     .toLowerCase()
663     .includes(query)
664   );
665 }, [vendorQuery, vendors]);
666
667 const filteredStalls = useMemo(() => {
668   const query = stallQuery.trim().toLowerCase();
669   const regularStalls = stalls.filter((stall) => !isGroupedStall(stall));
670   return filterStallsBySearch(regularStalls, query);
671 }, [stallQuery, stalls]);
672
673 const filteredStockrooms = useMemo(() => {
674   const query = stallQuery.trim().toLowerCase();
```

```

675     const stockrooms = stalls.filter(isStockmasterStall);
676     return filterStallsBySearch(stockrooms, query);
677 }, [stallQuery, stalls]);
678
679 const filteredTableStalls = useMemo(() => {
680     const query = stallQuery.trim().toLowerCase();
681     const tableStalls = stalls.filter(isTableStall);
682     return filterStallsBySearch(tableStalls, query);
683 }, [stallQuery, stalls]);
684
685 const filteredAccessXStalls = useMemo(() => {
686     const query = stallQuery.trim().toLowerCase();
687     const accessXStalls = stalls.filter(isAccessXStall);
688     return filterStallsBySearch(accessXStalls, query);
689 }, [stallQuery, stalls]);
690
691 const filteredAccessXCategories = useMemo(() => {
692     const query = categoryQuery.trim().toLowerCase();
693     if (!query) return accessXCategories;
694     return accessXCategories.filter((category) =>
695         [category?.name, category?.allowCount, category?.position]
696             .join(" ")
697             .toLowerCase()
698             .includes(query)
699     );
700 }, [accessXCategories, categoryQuery]);
701
702 const filteredAccessXGateMasters = useMemo(() => {
703     const query = gateMasterQuery.trim().toLowerCase();
704     if (!query) return accessXGateMasters;
705     return accessXGateMasters.filter((gate) =>
706         [gate?.id, gate?.gateDay, getGateMasterName(gate)]
707             .join(" ")
708             .toLowerCase()
709             .includes(query)
710     );
711 }, [accessXGateMasters, gateMasterQuery]);
712
713 const filteredAccessXGates = useMemo(() => {
714     const query = gateConfigQuery.trim().toLowerCase();
715     if (!query) return accessXGates;
716     return accessXGates.filter((gate) =>
717         [
718             gate?.id,
719             gate?.gatesmaster_name,
720             gate?.category_name,
721             gate?.gateId,
722             gate?.categoryId
723         ]
724             .join(" ")
725             .toLowerCase()
726             .includes(query)
727     );
728 }, [accessXGates, gateConfigQuery]);
729
730 const addVendorButton = (
731     <button
732         type="button"
733         onClick={() => setShowCreateVendor(true)}
734         className="inline-flex h-10 items-center gap-2 rounded-[10px] bg-(--text) px-4 text-[13.5px] font-s
735     >
736         <PlusIcon className="h-4 w-4" />
737         Add Vendor
738     </button>
739 );
740
741 const addCategoryButton = (
742     <button
743         type="button"

```

```

744     onClick={() => setShowCreateAccessXCategory(true)}
745     className="inline-flex h-8 items-center gap-1.5 rounded-[8px] bg(--text) px-3 text-[12px] font-sem
ibold text(--bg) transition hover:bg(--orange) max-[640px]:w-full max-[640px]:justify-center"
746   >
747     <PlusIcon className="h-3.5 w-3.5" />
748     Add
749   </button>
750 );
751
752 const addGateMasterButton = (
753   <button
754     type="button"
755     onClick={() => setShowCreateAccessGateMaster(true)}
756     className="inline-flex h-8 items-center gap-1.5 rounded-[8px] bg(--text) px-3 text-[12px] font-sem
ibold text(--bg) transition hover:bg(--orange) max-[640px]:w-full max-[640px]:justify-center"
757   >
758     <PlusIcon className="h-3.5 w-3.5" />
759     Add
760   </button>
761 );
762
763 const addGateConfigButton = (
764   <button
765     type="button"
766     title="Access Gate Config create API pending"
767     className="inline-flex h-8 items-center gap-1.5 rounded-[8px] bg(--text) px-3 text-[12px] font-sem
ibold text(--bg) transition hover:bg(--orange) max-[640px]:w-full max-[640px]:justify-center"
768   >
769     <PlusIcon className="h-3.5 w-3.5" />
770     Add
771   </button>
772 );
773
774 if (!eventId) {
775   return (
776     <ConfigPanel
777       title="Vendors"
778       count={0}
779       searchValue={vendorQuery}
780       onSearchChange={setVendorQuery}
781       searchPlaceholder="Search vendor"
782       action={addVendorButton}
783     >
784       <EmptyState>Select an event to load configuration.</EmptyState>
785     </ConfigPanel>
786   );
787 }
788
789 return (
790   <>
791     <div className="space-y-4">
792       <div className="grid items-start gap-4 xl:grid-cols-[minmax(0,1.38fr)_minmax(390px,1fr)]">
793         <ConfigPanel
794           title="Vendors"
795           count={vendors.length}
796           searchValue={vendorQuery}
797           onSearchChange={setVendorQuery}
798           searchPlaceholder="Search vendor"
799           action={addVendorButton}
800         >
801           <ScrollRows>
802             {vendorsLoading ? (
803               <LoadingState label="Loading vendors..." />
804             ) : vendorsError ? (
805               <EmptyState>{vendorsError}</EmptyState>
806             ) : filteredVendors.length === 0 ? (
807               <EmptyState>No vendors found.</EmptyState>
808             ) : (
809               filteredVendors.map((vendor, index) => (
810                 <VendorRow

```

```

811         key={vendor?.id ?? vendor?.vendorId ?? `${getVendorName(vendor)}-${index}`}
812         vendor={vendor}
813         index={index}
814         onAddStall={() =>
815             setCreateStallFor({
816                 vendorId: getCreateStallVendorId(vendor, index),
817                 vendorName: getVendorName(vendor),
818                 vendorType: getVendorType(vendor),
819             })
820         }
821         onEditVendor={() => setEditVendor(vendor)}
822     />
823 ))
824 }}
825 </ScrollRows>
826 </ConfigPanel>
827
828 <ConfigPanel
829     title="Stall"
830     count={stalls.filter((stall) => !isGroupedStall(stall)).length}
831     searchValue={stallQuery}
832     onSearchChange={setStallQuery}
833     searchPlaceholder="Search stall"
834 >
835     <ScrollRows>
836         {stallsLoading ? (
837             <LoadingState label="Loading stalls..." />
838         ) : stallsError ? (
839             <EmptyState>{stallsError}</EmptyState>
840         ) : filteredStalls.length === 0 ? (
841             <EmptyState>No stalls found.</EmptyState>
842         ) : (
843             filteredStalls.map((stall, index) => (
844                 <StallRow
845                     key={stall?.id ?? stall?.stallId ?? `${getStallName(stall)}-${index}`}
846                     stall={stall}
847                     index={index}
848                     onAddDevice={() => setAddDeviceStall(stall)}
849                     onOpenMenu={() => handleOpenStallMenu(stall)}
850                     onEditStall={() => setEditStall(stall)}
851                 />
852             ))
853         )}
854     </ScrollRows>
855 </ConfigPanel>
856 </div>
857
858 <div className="gap-4 xl:columns-2 [&section]:mb-4 [&section]:break-inside-avoid">
859     <ConfigPanel
860         title="Stockroom"
861         count={stalls.filter(isStockmasterStall).length}
862         searchValue={stallQuery}
863         onSearchChange={setStallQuery}
864         searchPlaceholder="Search stockroom"
865         theme="stockroom"
866     >
867         <ScrollRows>
868             {stallsLoading ? (
869                 <LoadingState label="Loading stockrooms..." />
870             ) : stallsError ? (
871                 <EmptyState>{stallsError}</EmptyState>
872             ) : filteredStockrooms.length === 0 ? (
873                 <EmptyState>No stockrooms found.</EmptyState>
874             ) : (
875                 filteredStockrooms.map((stall, index) => (
876                     <StallRow
877                         key={stall?.id ?? stall?.stallId ?? `${getStallName(stall)}-${index}`}
878                         stall={stall}
879                         index={index}
880                         onAddDevice={() => setAddDeviceStall(stall)}

```

```

881         onOpenMenu={() => handleOpenStallMenu(stall)}
882         onEditStall={() => setEditStall(stall)}
883     />
884 ))
885 }}
886 </ScrollRows>
887 </ConfigPanel>
888
889 <ConfigPanel
890   title="Tables"
891   count={stalls.filter(isTableStall).length}
892   searchValue={stallQuery}
893   onSearchChange={setStallQuery}
894   searchPlaceholder="Search tables"
895   theme="tables"
896 >
897   <ScrollRows>
898     {stallsLoading ? (
899       <LoadingState label="Loading tables..." />
900     ) : stallsError ? (
901       <EmptyState>{stallsError}</EmptyState>
902     ) : filteredTableStalls.length === 0 ? (
903       <EmptyState>No table stalls found.</EmptyState>
904     ) : (
905       filteredTableStalls.map((stall, index) => (
906         <StallRow
907           key={stall?.id ?? stall?.stallId ?? `${getStallName(stall)}-${index}`}
908           stall={stall}
909           index={index}
910           onAddDevice={() => setAddDeviceStall(stall)}
911           onOpenMenu={() => handleOpenStallMenu(stall)}
912           onEditStall={() => setEditStall(stall)}
913         />
914       ))
915     )}
916   </ScrollRows>
917 </ConfigPanel>
918
919 <ConfigPanel
920   title="AccessX"
921   count={stalls.filter(isAccessXStall).length}
922   searchValue={stallQuery}
923   onSearchChange={setStallQuery}
924   searchPlaceholder="Search AccessX"
925   theme="accessx"
926 >
927   <ScrollRows>
928     {stallsLoading ? (
929       <LoadingState label="Loading AccessX..." />
930     ) : stallsError ? (
931       <EmptyState>{stallsError}</EmptyState>
932     ) : filteredAccessXStalls.length === 0 ? (
933       <EmptyState>No AccessX stalls found.</EmptyState>
934     ) : (
935       filteredAccessXStalls.map((stall, index) => (
936         <StallRow
937           key={stall?.id ?? stall?.stallId ?? `${getStallName(stall)}-${index}`}
938           stall={stall}
939           index={index}
940           onAddDevice={() => setAddDeviceStall(stall)}
941           onOpenMenu={() => handleOpenStallMenu(stall)}
942           onEditStall={() => setEditStall(stall)}
943         />
944       ))
945     )}
946   </ScrollRows>
947 </ConfigPanel>
948
949 <ConfigPanel
950   title="AccessX Categories"

```

```

951         count={accessXCategories.length}
952         searchValue={categoryQuery}
953         onSearchChange={setCategoryQuery}
954         searchPlaceholder="Search category"
955         action={addCategoryButton}
956         theme="accessx"
957     >
958     {accessXCategoriesLoading ? (
959         <LoadingState label="Loading categories..." />
960     ) : accessXCategoriesError ? (
961         <EmptyState>{accessXCategoriesError}</EmptyState>
962     ) : filteredAccessXCategories.length === 0 ? (
963         <EmptyState>No AccessX categories found.</EmptyState>
964     ) : (
965         <AccessXCategoryTable
966             categories={filteredAccessXCategories}
967             onEdit={setEditAccessXCategory}
968         />
969     )}
970 </ConfigPanel>
971
972 <ConfigPanel
973     title="Access_Gate_Master"
974     count={accessXGateMasters.length}
975     searchValue={gateMasterQuery}
976     onSearchChange={setGateMasterQuery}
977     searchPlaceholder="Search gate"
978     action={addGateMasterButton}
979     theme="accessx"
980 >
981     {accessXGateMastersLoading ? (
982         <LoadingState label="Loading gate masterlist..." />
983     ) : accessXGateMastersError ? (
984         <EmptyState>{accessXGateMastersError}</EmptyState>
985     ) : filteredAccessXGateMasters.length === 0 ? (
986         <EmptyState>No gate masters found.</EmptyState>
987     ) : (
988         <Access_Gate_Master
989             gates={filteredAccessXGateMasters}
990             onEdit={setEditAccessGateMaster}
991         />
992     )}
993 </ConfigPanel>
994
995 <ConfigPanel
996     title="Access Gate Config"
997     count={accessXGates.length}
998     searchValue={gateConfigQuery}
999     onSearchChange={setGateConfigQuery}
1000     searchPlaceholder="Search gate or category"
1001     action={addGateConfigButton}
1002     theme="accessx"
1003 >
1004     {accessXGatesLoading ? (
1005         <LoadingState label="Loading gate configuration..." />
1006     ) : accessXGatesError ? (
1007         <EmptyState>{accessXGatesError}</EmptyState>
1008     ) : filteredAccessXGates.length === 0 ? (
1009         <EmptyState>No gate configuration found.</EmptyState>
1010     ) : (
1011         <Access_Gate_Config
1012             gates={filteredAccessXGates}
1013             onEdit={setEditAccessGateConfig}
1014         />
1015     )}
1016 </ConfigPanel>
1017 </div>
1018 </div>
1019
1020 {editStall && (

```

```

1021     <EditStallModal
1022       stall={editStall}
1023       onClose={() => setEditStall(null)}
1024       onConfirm={(data) => {
1025         console.log("Edit stall", data);
1026         setEditStall(null);
1027       }}
1028     />
1029   )}
1030
1031   {editAccessXCategory && (
1032     <EditAccessXCategoryModal
1033       category={editAccessXCategory}
1034       onClose={() => setEditAccessXCategory(null)}
1035       onConfirm={async (data) => {
1036         const numericEventId = Number(eventId);
1037         const payload = {
1038           allowCount: data.allowCount,
1039           eventId: Number.isNaN(numericEventId) ? eventId : numericEventId,
1040           id: editAccessXCategory.id,
1041           name: data.name,
1042           description: editAccessXCategory.description ?? null,
1043           externalLink: editAccessXCategory.externalLink ?? null,
1044           position: data.position,
1045           qrLogicDiscountId: data.qrLogicDiscountId,
1046           qrLogicIssuerId: data.qrLogicIssuerId,
1047           qrLogicSectorId: data.qrLogicSectorId,
1048           qrLogicTicketType: data.qrLogicTicketType,
1049           qrLogicTktAccred: data.qrLogicTktAccred,
1050           status: editAccessXCategory.status ?? "active"
1051         };
1052
1053         try {
1054           const response = await updateAccessXCategory({ token, category: payload });
1055           const updatedCategory =
1056             response?.category ??
1057             response?.data?.category ??
1058             response?.data ??
1059             payload;
1060           setAccessXCategories((current) =>
1061             current.map((category) =>
1062               category?.id === editAccessXCategory?.id
1063                 ? { ...category, ...payload, ...updatedCategory }
1064                 : category
1065             )
1066           );
1067           setEditAccessXCategory(null);
1068         } catch (error) {
1069           console.error("Edit AccessX category failed", error);
1070           throw error;
1071         }
1072       }}
1073     />
1074   )}
1075
1076   {showCreateAccessXCategory && (
1077     <EditAccessXCategoryModal
1078       category={null}
1079       mode="create"
1080       onClose={() => setShowCreateAccessXCategory(false)}
1081       onConfirm={async (data) => {
1082         const numericEventId = Number(eventId);
1083         const payload = {
1084           allowCount: data.allowCount,
1085           eventId: Number.isNaN(numericEventId) ? eventId : numericEventId,
1086           name: data.name.trim(),
1087           description: eventName || null,
1088           externalLink: data.name.trim(),
1089           qrLogicDiscountId: data.qrLogicDiscountId,
1090           qrLogicIssuerId: data.qrLogicIssuerId,

```



```
1091         qrLogicSectorId: data.qrLogicSectorId,
1092         qrLogicTicketType: data.qrLogicTicketType,
1093         qrLogicTktAccred: data.qrLogicTktAccred
1094     };
1095
1096     try {
1097         const response = await createAccessXCategory({ token, category: payload });
1098         try {
1099             const refreshed = await fetchAccessXCategories({
1100                 eventId,
1101                 token,
1102                 dedupe: false
1103             });
1104             setAccessXCategories(normalizeAccessXCategories(refreshed));
1105         } catch (refreshError) {
1106             console.error("Reload AccessX categories failed", refreshError);
1107             const createdCategory =
1108                 response?.category ??
1109                 response?.data?.category ??
1110                 response?.data ??
1111                 payload;
1112             setAccessXCategories((current) => [
1113                 ...current,
1114                 { ...payload, ...createdCategory }
1115             ]);
1116         }
1117         setShowCreateAccessXCategory(false);
1118     } catch (error) {
1119         console.error("Create AccessX category failed", error);
1120         throw error;
1121     }
1122 }}
1123 />
1124 }}
1125
1126 {showCreateAccessGateMaster && (
1127     <CreateAccessGateMasterModal
1128         onClose={() => setShowCreateAccessGateMaster(false)}
1129         onConfirm={async ({ name }) => {
1130             const numericEventId = Number(eventId);
1131             const payload = {
1132                 name,
1133                 eventId: Number.isNaN(numericEventId) ? eventId : numericEventId,
1134                 gateDay: 1,
1135                 useCatgCounts: 1,
1136                 position: 1,
1137                 hide: 0
1138             };
1139
1140             try {
1141                 const response = await createAccessXGateMaster({
1142                     token,
1143                     gateMaster: payload
1144                 });
1145                 try {
1146                     const refreshed = await fetchAccessXGateMasters({
1147                         eventId,
1148                         token,
1149                         dedupe: false
1150                     });
1151                     setAccessXGateMasters(normalizeAccessXGateMasters(refreshed));
1152                 } catch (refreshError) {
1153                     console.error("Reload Access Gate Masterlist failed", refreshError);
1154                     const createdGateMaster =
1155                         response?.gatesmaster ??
1156                         response?.gateMaster ??
1157                         response?.data?.gatesmaster ??
1158                         response?.data?.gateMaster ??
1159                         response?.data ??
1160                         payload;
```

```

1161         setAccessXGateMasters((current) => [
1162             ...current,
1163             { ...payload, ...createdGateMaster }
1164         ]);
1165     }
1166     setShowCreateAccessGateMaster(false);
1167 } catch (error) {
1168     console.error("Create Access Gate Master failed", error);
1169     throw error;
1170 }
1171 }}
1172 />
1173 }}
1174
1175 {editAccessGateMaster && (
1176     <EditAccessGateMasterModal
1177         gateMaster={editAccessGateMaster}
1178         onClose={() => setEditAccessGateMaster(null)}
1179         onConfirm={async ({ name, useCatgCounts }) => {
1180             const numericEventId = Number(eventId);
1181             const numericPosition = Number(editAccessGateMaster.position);
1182             const payload = {
1183                 id: editAccessGateMaster.id,
1184                 name,
1185                 eventId: Number.isNaN(numericEventId) ? eventId : numericEventId,
1186                 useCatgCounts,
1187                 position: Number.isNaN(numericPosition)
1188                     ? editAccessGateMaster.position
1189                     : numericPosition
1190             };
1191
1192             try {
1193                 const response = await updateAccessXGateMaster({
1194                     token,
1195                     gateMaster: payload
1196                 });
1197                 const updatedGateMaster =
1198                     response?.gatesmaster ??
1199                     response?.gateMaster ??
1200                     response?.data?.gatesmaster ??
1201                     response?.data?.gateMaster ??
1202                     response?.data ??
1203                     payload;
1204                 setAccessXGateMasters((current) =>
1205                     current.map((gateMaster) =>
1206                         gateMaster?.id === editAccessGateMaster?.id
1207                             ? { ...gateMaster, ...payload, ...updatedGateMaster }
1208                             : gateMaster
1209                     )
1210                 );
1211                 setEditAccessGateMaster(null);
1212             } catch (error) {
1213                 console.error("Edit Access Gate Master failed", error);
1214                 throw error;
1215             }
1216         }}
1217     />
1218 }}
1219
1220 {editAccessGateConfig && (
1221     <EditAccessGateConfigModal
1222         gateConfig={editAccessGateConfig}
1223         onClose={() => setEditAccessGateConfig(null)}
1224         onConfirm={async ({ name, useCatgCounts }) => {
1225             const numericEventId = Number(eventId);
1226             const numericGateMasterId = Number(editAccessGateConfig.gateId);
1227             const numericPosition = Number(editAccessGateConfig.gatesmaster_position);
1228             const gateMasterId = Number.isNaN(numericGateMasterId)
1229                 ? editAccessGateConfig.gateId
1230                 : numericGateMasterId;

```

```
1231     const position = Number.isNaN(numericPosition)
1232       ? editAccessGateConfig.gatesmaster_position
1233       : numericPosition;
1234     const payload = {
1235       id: gateMasterId,
1236       name,
1237       eventId: Number.isNaN(numericEventId) ? eventId : numericEventId,
1238       useCatgCounts,
1239       position
1240     };
1241
1242     try {
1243       await updateAccessXGateMaster({
1244         token,
1245         gateMaster: payload
1246       });
1247
1248       setAccessXGates((current) =>
1249         current.map((gate) =>
1250           String(gate?.gateId) === String(gateMasterId)
1251             ? {
1252               ...gate,
1253               gatesmaster_name: name,
1254               gatesmaster_useCatgCounts: useCatgCounts,
1255               gatesmaster_position: position
1256             }
1257           : gate
1258         )
1259       );
1260       setAccessXGateMasters((current) =>
1261         current.map((gateMaster) =>
1262           String(gateMaster?.id) === String(gateMasterId)
1263             ? {
1264               ...gateMaster,
1265               name,
1266               useCatgCounts,
1267               position
1268             }
1269           : gateMaster
1270         )
1271       );
1272
1273       const [gateMastersResult, gatesResult] = await Promise.allSettled([
1274         fetchAccessXGateMasters({
1275           eventId,
1276           token,
1277           dedupe: false
1278         }),
1279         fetchAccessXGates({
1280           eventId,
1281           token,
1282           dedupe: false
1283         })
1284       ]);
1285
1286       if (gateMastersResult.status === "fulfilled") {
1287         setAccessXGateMasters(
1288           normalizeAccessXGateMasters(gateMastersResult.value)
1289         );
1290       } else {
1291         console.error(
1292           "Reload Access Gate Masterlist failed",
1293           gateMastersResult.reason
1294         );
1295       }
1296
1297       if (gatesResult.status === "fulfilled") {
1298         setAccessXGates(normalizeAccessXGates(gatesResult.value));
1299       } else {
1300         console.error(
```

```
1301         "Reload Access Gate Config failed",
1302         gatesResult.reason
1303     );
1304 }
1305
1306     setEditAccessGateConfig(null);
1307 } catch (error) {
1308     console.error("Edit Access Gate Config failed", error);
1309     throw error;
1310 }
1311 }}
1312 />
1313 }}
1314
1315 {addDeviceStall && (
1316     <AddDevice
1317         stall={addDeviceStall}
1318         onClose={() => setAddDeviceStall(null)}
1319         onConfirm={async (payload) => {
1320             try {
1321                 await addDevicesToStall({ token, payload });
1322                 setAddDeviceStall(null);
1323                 loadStalls({ force: true });
1324             } catch (error) {
1325                 console.error("Add devices to stall failed", error);
1326                 throw error;
1327             }
1328         }}
1329     />
1330 )}
1331
1332 {showCreateVendor && (
1333     <CreateVendorModal
1334         eventId={eventId}
1335         onClose={() => setShowCreateVendor(false)}
1336         onConfirm={async (data) => {
1337             try {
1338                 await createVendor({ token, vendor: data });
1339                 setShowCreateVendor(false);
1340                 loadVendors({ force: true });
1341             } catch (error) {
1342                 console.error("Create vendor failed", error);
1343             }
1344         }}
1345     />
1346 )}
1347
1348 {editVendor && (
1349     <EditVendorModal
1350         vendor={editVendor}
1351         eventId={eventId}
1352         onClose={() => setEditVendor(null)}
1353         onConfirm={async (data) => {
1354             try {
1355                 await updateVendor({
1356                     vendorId: getVendorId(editVendor),
1357                     token,
1358                     payload: data
1359                 });
1360                 setEditVendor(null);
1361                 loadVendors({ force: true });
1362             } catch (error) {
1363                 console.error("Edit vendor failed", error);
1364             }
1365         }}
1366     />
1367 )}
1368
1369 {createStallFor && (
1370     <CreateStallModal
```

```
1371     vendorName={createStallFor.vendorName}
1372     vendorType={createStallFor.vendorType}
1373     onClose={() => setCreateStallFor(null)}
1374     onConfirm={async (data) => {
1375         try {
1376             const payload = buildCreateStallPayload(data, createStallFor.vendorId);
1377             await createStall({ token, stall: payload });
1378             setCreateStallFor(null);
1379             loadStalls({ force: true });
1380         } catch (error) {
1381             console.error("Create stall failed", error);
1382         }
1383     }}
1384     />
1385   )}
1386   </>
1387 );
1388 }
```

FILE: apps/dashboard/src/app/Config/components/stallPayload.js

```
1  const DEFAULT_PAY_OPTIONS = ["cash", "card", "coupon"];
2
3  function enabled(value) {
4    return value ? 1 : 0;
5  }
6
7  function lower(value, fallback = "") {
8    return String(value || fallback).trim().toLowerCase();
9  }
10
11 function hasValue(value) {
12   return String(value ?? "").trim() !== "";
13 }
14
15 function normalizedId(value) {
16   const text = String(value ?? "").trim();
17   return /^\\d+$/\\.test(text) ? Number(text) : value;
18 }
19
20 export function buildCreateStallPayload(form, vendorId) {
21   const payOptions =
22     form.acceptAllModes && Array.isArray(form.paymentModes) && form.paymentModes.length > 0
23     ? form.paymentModes
24     : DEFAULT_PAY_OPTIONS;
25
26   const payload = {
27     name: String(form.stallName || "").trim(),
28     type: lower(form.type, "sale"),
29     bankOption: "none",
30     scanMode: lower(form.scanMode, "none"),
31     smsFormat: "default",
32     kotLan: enabled(form.kotLan),
33     addDiscountInCard: 0,
34     accessxSettings: {
35       nfcSettings: lower(form.nfcSetting, "logic"),
36       qrSettings: String(form.qrSetting || "OFF").trim().toUpperCase(),
37       useOnlineTopups: enabled(form.useOnlineTopups),
38       checkOnlineUnique: enabled(form.checkOnlineUnique),
39       useFetchDetails: enabled(form.fetchDetails),
40       useInOut: enabled(form.useInOutLogic),
41     },
42     payOptions: payOptions.map((option) => lower(option)).join(","),
43     vendorId: normalizedId(vendorId),
44     payOption: 0,
45     useGrn: enabled(form.grnMode),
46     cashDisabled: enabled(form.cashDisabled),
47     modeInfoMandatory: enabled(form.modeInfoMandatory),
48     showStall: enabled(form.showInTapX),
49   };
50
51   if (hasValue(form.locationId)) {
52     payload.accessxSettings.locationId = String(form.locationId).trim();
53   }
54
55   if (hasValue(form.eventMatchId)) {
56     payload.accessxSettings.eventMatchId = String(form.eventMatchId).trim();
57   }
58
59   return payload;
60 }
```

FILE: apps/dashboard/src/app/Config/components/vendorPayload.js

```
1 export const VENDOR_TYPES = ["FNB", "PASS", "TICKET", "SALE", "INVENTORY", "TABLES", "ACCESSX"];
2
3 export const DEFAULT_VENDOR_FORM = {
4   vendorName: "",
5   type: "INVENTORY",
6   revShare: "",
7   gstin: "",
8   pan: "",
9   serviceCharge: "",
10  serviceChargeTax: "",
11  showPrintDetails: false,
12  dashboardPassword: "1234",
13  mobile: "",
14  address: "",
15  thankYouNote: "Thank You for using AtomX POS",
16  sac: false,
17  sacText: "",
18  autoPrint: false,
19  maxInvoices: "",
20  kot: false,
21  kotCount: "",
22  invoiceFormat: "format1",
23  reprintPassword: "2026"
24 };
25
26 export function numericOnly(value) {
27   return String(value ?? "").replace(/[^\d.]/g, "").replace(/(\.\.*)"./g, "$1");
28 }
29
30 function hasValue(value) {
31   return String(value ?? "").trim() !== "";
32 }
33
34 function numberValue(value) {
35   const normalized = numericOnly(value);
36   return normalized === "" ? undefined : Number(normalized);
37 }
38
39 export function buildVendorPayload(form, eventId) {
40   const payload = {
41     type: String(form.type || "INVENTORY").toLowerCase(),
42     password: hasValue(form.dashboardPassword) ? form.dashboardPassword.trim() : "1234",
43     rePrintPassword: hasValue(form.reprintPassword)
44       ? Number(form.reprintPassword)
45       : 2026,
46     thankyouNote: hasValue(form.thankYouNote)
47       ? form.thankYouNote.trim()
48       : "Thank You for using AtomX POS",
49     address: hasValue(form.address) ? form.address.trim() : "",
50     eventId: Number(eventId) || eventId
51   };
52
53   if (hasValue(form.vendorName)) payload.name = form.vendorName.trim();
54   if (hasValue(form.revShare)) payload.revShare = numberValue(form.revShare);
55   if (hasValue(form.gstin)) payload.gstin = form.gstin.trim();
56   if (hasValue(form.pan)) payload.pan = form.pan.trim();
57   if (hasValue(form.serviceCharge)) payload.sc = numberValue(form.serviceCharge);
58   if (hasValue(form.serviceChargeTax)) payload.scTax = numberValue(form.serviceChargeTax);
59   if (hasValue(form.mobile)) payload.mobile = form.mobile.trim();
60
61   if (form.sac) {
62     payload.showSac = 1;
63     if (hasValue(form.sacText)) payload.sac = form.sacText.trim();
64   }
65   if (form.autoPrint) payload.autoPrint = 1;
66   if (form.kot) {
67     payload.kotPrint = 1;
68     if (hasValue(form.kotCount)) payload.kotPrintCount = numberValue(form.kotCount);
69   }
70 }
```

```
70  if (hasValue(form.maxInvoices)) payload.invoicePrintCount = numberValue(form.maxInvoices);
71  if (form.showPrintDetails && form.invoiceFormat) payload.printFormat = form.invoiceFormat;
72
73  return payload;
74 }
```


FILE: apps/dashboard/src/app/Config/menu/components/CategoryDetailPanel.js

```

1  "use client";
2
3  import { useEffect, useRef, useState } from "react";
4  import { GearIcon, ChevronDownIcon } from "../MenuIcons";
5
6  const GST_OPTIONS = [0, 5, 12, 18, 28];
7
8  function Toggle({ active, onToggle }) {
9    return (
10     <button
11       type="button"
12       onClick={onToggle}
13       aria-pressed={active}
14       className={`flex h-7 w-14 shrink-0 items-center rounded-full p-1 transition-all duration-200 ${
15         active ? "bg-[linear-gradient(135deg,#E04420,#341CD6)]" : "bg-(--line)"
16       }`}
17     >
18       <span
19         className={`h-5 w-5 rounded-full bg-white shadow-sm transition-transform duration-200 ${
20           active ? "translate-x-7" : "translate-x-0"
21         }`}
22       />
23     </button>
24   );
25 }
26
27 function GstDropdown({ value, onChange }) {
28   const [open, setOpen] = useState(false);
29   const ref = useRef(null);
30
31   useEffect(() => {
32     const handler = (e) => {
33       if (!ref.current?.contains(e.target)) setOpen(false);
34     };
35     if (open) document.addEventListener("mousedown", handler);
36     return () => document.removeEventListener("mousedown", handler);
37   }, [open]);
38
39   return (
40     <div ref={ref} className="relative">
41       <button
42         type="button"
43         onClick={() => setOpen((prev) => !prev)}
44         className="flex h-10 items-center gap-1.5 rounded-[10px] bg-[linear-gradient(135deg,#E04420,#341CD6)] px-4 text-[13px] font-semibold text-white transition hover:brightness-105"
45       >
46         {value}
47         <ChevronDownIcon />
48       </button>
49       {open && (
50         <div className="absolute left-0 top-full z-30 mt-1.5 w-[90px] overflow-hidden rounded-[10px] border border-(--line) bg-(--surface) shadow-(--shadowUp)">
51           {GST_OPTIONS.map((opt) => (
52             <button
53               key={opt}
54               type="button"
55               onClick={() => { onChange(opt); setOpen(false); }}
56               className={`block w-full px-3 py-2 text-left text-[13px] font-semibold transition hover:bg-(--surface2) ${
57                 value === opt ? "text-(--orange)" : "text-(--text)"
58               }`}
59             >
60               {opt}%
61             </button>
62           ))}
63         </div>
64       )}
65     </div>
66   );

```

```

67 }
68
69 export default function CategoryDetailPanel({ category, onUpdate }) {
70   if (!category) return null;
71
72   return (
73     <div className="border-b border-(--line2) bg-(--surface) px-5 py-4">
74       <div className="grid items-start gap-5 lg:grid-cols-[minmax(0,2fr)_minmax(0,1.1fr)_minmax(0,1.35fr)]">
75         { /* NAME */ }
76         <div className="flex flex-col gap-3">
77           <div className="font-vcr flex items-center gap-2 text-[8.5px] uppercase tracking-[0.18em] text-(--muted)">
78             <GearIcon className="h-3.5 w-3.5 text-(--orange)" />
79             <span>NAME</span>
80           </div>
81           <div className="flex items-center gap-3">
82             <div className="flex flex-1 items-center gap-2 rounded-[10px] border border-(--line) bg-(--surface2) px-3 py-2.5 transition focus-within:border-(--orange)">
83               <GearIcon className="h-3.5 w-3.5 shrink-0 text-(--faint)" />
84               <input
85                 type="text"
86                 value={category.name}
87                 onChange={(e) => onUpdate?.({ name: e.target.value })}
88                 placeholder="Category name"
89                 className="min-w-0 flex-1 bg-transparent text-[13px] font-medium text-(--text) outline-none placeholder:text-(--faint)"
90               />
91             </div>
92             <Toggle active={category.active} onToggle={() => onUpdate?.({ active: !category.active })} />
93           </div>
94         </div>
95
96         { /* VAT */ }
97         <div className="flex flex-col gap-3">
98           <div className="font-vcr flex items-center gap-2 text-[8.5px] uppercase tracking-[0.18em] text-(--muted)">
99             <GearIcon className="h-3.5 w-3.5 text-(--orange)" />
100             <span>VAT</span>
101           </div>
102           <div className="flex items-center gap-2.5">
103             <div className="flex w-[86px] items-center gap-1.5 rounded-[10px] border border-(--line) bg-(--surface2) px-2.5 py-2.5 transition focus-within:border-(--orange)">
104               <GearIcon className="h-3.5 w-3.5 shrink-0 text-(--faint)" />
105               <input
106                 type="number"
107                 value={category.vat ?? 0}
108                 onChange={(e) => onUpdate?.({ vat: Number(e.target.value) })}
109                 min={0}
110                 className="w-full min-w-0 bg-transparent text-[13px] font-medium text-(--text) outline-none"
111               />
112             </div>
113             <button
114               type="button"
115               className="rounded-[10px] bg-(--text) px-4 py-2.5 text-[12.5px] font-semibold text-(--bg) transition hover:bg-(--orange)"
116             >
117               Apply
118             </button>
119           </div>
120         </div>
121
122         { /* GST */ }
123         <div className="flex flex-col gap-3">
124           <div className="font-vcr flex items-center gap-2 text-[8.5px] uppercase tracking-[0.18em] text-(--muted)">
125             <GearIcon className="h-3.5 w-3.5 text-(--orange)" />
126             <span>GST</span>
127           </div>

```

```
128         <div className="flex items-center gap-2">
129             <GstDropdown value={category.gst ?? 0} onChange={(val) => onUpdate?.({ gst: val })} />
130             <button
131                 type="button"
132                 onClick={() => onUpdate?.({ gstInclusive: !category.gstInclusive })}
133                 className="flex h-10 items-center gap-1 rounded-[10px] bg-(--text) px-4 text-[12.5px] font-
semibold text-(--bg) transition hover:bg-(--orange)"
134                 >
135                 {category.gstInclusive ? "INCLU" : "EXCLU"}
136             </button>
137         </div>
138     </div>
139 </div>
140 </div>
141 );
142 }
```

FILE: apps/dashboard/src/app/Config/menu/components/CategoryTabs.js

```

1 "use client";
2
3 export default function CategoryTabs({ categories, activeId, onSelect }) {
4   return (
5     <div className="flex items-end gap-1 overflow-x-auto border-b border-(--line) bg-(--surface) px-4 [-ms-overflow-style:none] [scrollbar-width:none] [&::-webkit-scrollbar]:hidden">
6       {categories.map((cat) => {
7         const isActive = activeId === cat.id;
8         return (
9           <button
10             key={cat.id}
11             type="button"
12             onClick={() => onSelect(cat.id)}
13             className={`relative flex shrink-0 items-center gap-2 px-4 py-3 text-[13px] font-semibold transition-colors ${
14               isActive ? "text-(--text)" : "text-(--muted) hover:text-(--text)"
15             }`}
16           >
17             <span>{cat.name}</span>
18             <span
19               className={`font-vcr flex h-5 min-w-[20px] items-center justify-center rounded-full px-1.5 text-[9.5px] leading-none ${
20                 isActive ? "bg-(--text) text-(--bg)" : "bg-(--chip) text-(--faint)"
21               }`}
22             >
23               {cat.count}
24             </span>
25             {isActive && <span className="absolute bottom-0 left-0 right-0 h-[2px] rounded-t-sm bg-(--orange)" />}
26           </button>
27         );
28       })}
29     </div>
30   );
31 }

```

FILE: apps/dashboard/src/app/Config/menu/components/MenuBar.js

```

1  "use client";
2
3  import { DownloadIcon, PlusIcon, CloudUploadIcon, SaveIcon } from "./MenuIcons";
4
5  function Toggle({ active, onToggle }) {
6    return (
7      <button
8        type="button"
9        onClick={onToggle}
10       aria-pressed={active}
11       className={`flex h-6 w-11 shrink-0 items-center rounded-full p-0.5 transition-all duration-200 ${
12         active ? "bg-[linear-gradient(135deg,#E04420,#341CD6)]" : "bg-(--line)"
13       }`}
14     >
15       <span
16         className={`h-5 w-5 rounded-full bg-white shadow-sm transition-transform duration-200 ${
17           active ? "translate-x-5" : "translate-x-0"
18         }`}
19       />
20     </button>
21   );
22 }
23
24 export default function MenuActionBar({
25   stallName = "Stall",
26   inactiveCategories,
27   onToggleInactiveCategories,
28   onDownload,
29   onAddCategory
30 }) {
31   return (
32     <div className="flex flex-wrap items-center gap-3 rounded-[15px] border border-(--line) border-l-[3px]
33 ] border-l-(--orange) bg-(--surface) px-4 py-3 shadow-(--shadow)">
34       <span className="font-chillax shrink-0 text-[15px] font-semibold text-(--text)">
35         {stallName} <span className="text-(--orange)">MENU</span>
36       </span>
37
38       <div className="h-5 w-px shrink-0 bg-(--line)" />
39
40       <div className="flex items-center gap-2 text-[12.5px] font-semibold text-(--muted)">
41         <span className="shrink-0">Inactive Categories:</span>
42         <Toggle active={inactiveCategories} onToggle={onToggleInactiveCategories} />
43       </div>
44
45       <div className="flex-1" />
46
47       <div className="flex flex-wrap items-center gap-2">
48         <button
49           type="button"
50           onClick={onDownload}
51           className="flex h-9 items-center gap-1.5 rounded-[8px] border border-(--line) bg-(--surface) px
52 -3 text-[12.5px] font-semibold text-(--muted) transition hover:border-(--orange) hover:text-(--orange)"
53         >
54           <DownloadIcon className="h-3.5 w-3.5" />
55           Download
56         </button>
57
58         <button
59           type="button"
60           className="flex h-9 items-center gap-1.5 rounded-[8px] bg-[linear-gradient(135deg,#E04420,#341C
61 D6)] px-3 text-[12.5px] font-semibold text-white transition hover:brightness-105"
62         >
63           <CloudUploadIcon className="h-3.5 w-3.5" />
64           Menu
65         </button>
66
67         <button
68           type="button"
69           onClick={onAddCategory}

```

```
67         className="flex h-9 items-center gap-1.5 rounded-[8px] border border-(--line) bg-(--surface) px
-3 text-[12.5px] font-semibold text-(--muted) transition hover:border-(--orange) hover:text-(--orange)"
68     >
69         <PlusIcon className="h-3.5 w-3.5" />
70         Category
71     </button>
72
73     <button
74         type="button"
75         className="flex h-9 items-center gap-1.5 rounded-[8px] bg-(--text) px-3 text-[12.5px] font-semi
bold text-(--bg) transition hover:bg-(--orange)"
76     >
77         <SaveIcon className="h-3.5 w-3.5" />
78         Save
79     </button>
80 </div>
81 </div>
82 );
83 }
```

FILE: apps/dashboard/src/app/Config/menu/components/MenuContent.js

```
1 "use client";
2
3 import { useEffect, useState } from "react";
4 import * as XLSX from "xlsx";
5 import { useSearchParams } from "next/navigation";
6 import { fetchStallItems } from "../../../../../lib/dashboardApi";
7 import { useDashboardStore } from "../../../../../store/dashboardStore";
8 import MenuActionBar from "../MenuActionBar";
9 import CategoryTabs from "../CategoryTabs";
10 import CategoryDetailPanel from "../CategoryDetailPanel";
11 import MenuItemTable from "../MenuItemTable";
12
13 function asNumber(value, fallback = 0) {
14   const numberValue = Number(value);
15   return Number.isFinite(numberValue) ? numberValue : fallback;
16 }
17
18 function asText(value) {
19   return value == null ? "" : String(value).trim();
20 }
21
22 function normalizeTags(value) {
23   if (Array.isArray(value)) {
24     return value.map((tag) => asText(tag).toUpperCase()).filter(Boolean);
25   }
26
27   return asText(value)
28     .split(/[;,]/)
29     .map((tag) => tag.trim().toUpperCase())
30     .filter(Boolean);
31 }
32
33 function normalizeMenuItem(item, index) {
34   return {
35     id: item?.id ?? `item-${index}`,
36     name: asText(item?.name),
37     price: asNumber(item?.price),
38     happy: asNumber(item?.happyPrice),
39     hsn: asText(item?.hsn),
40     barcode: asText(item?.barcode),
41     epc: asText(item?.epc),
42     type: asText(item?.type).toUpperCase() || "OTHER",
43     tags: normalizeTags(item?.tags),
44     active: asText(item?.status).toLowerCase() === "active",
45     image: item?.imagePath ?? null,
46     supplierCode: asText(item?.supplierCode),
47     groupId: asText(item?.groupId),
48     variant: asText(item?.variant),
49     colour: asText(item?.colour),
50     position: asNumber(item?.position, index)
51   };
52 }
53
54 function getResponseArray(response, key) {
55   const candidates = [
56     response?.[key],
57     response?.data?.[key],
58     response?.result?.[key],
59     response?.data?.data?.[key]
60   ];
61
62   return candidates.find(Array.isArray) ?? [];
63 }
64
65 function normalizeMenuResponse(response) {
66   const categories = getResponseArray(response, "categories");
67   const allItems = getResponseArray(response, "items");
68   const nestedMenu = getResponseArray(response, "menu");
69   const menu = categories.length > 0 ? categories : nestedMenu;
```

```

70
71   if (menu.length === 0) return [];
72
73   const itemsByCategory = allItems.reduce((groupedItems, item) => {
74     if (item?.categoryId == null) return groupedItems;
75
76     const categoryKey = String(item.categoryId);
77     const categoryItems = groupedItems.get(categoryKey) ?? [];
78     categoryItems.push(item);
79     groupedItems.set(categoryKey, categoryItems);
80     return groupedItems;
81   }, new Map());
82
83   return menu.map((category, index) => {
84     const categoryItems = Array.isArray(category?.items)
85       ? category.items
86       : itemsByCategory.get(String(category?.id)) ?? [];
87     const items = categoryItems.map(normalizeMenuItem);
88
89     return {
90       id: category?.id ?? `category-${index}`,
91       name: asText(category?.name) || `Category ${index + 1}`,
92       count: items.length,
93       active: asText(category?.status).toLowerCase() === "active",
94       vat: asNumber(category?.vat),
95       gst: asNumber(category?.gstSlab),
96       gstInclusive: asText(category?.gstType).toLowerCase() === "inclusive",
97       items
98     };
99   });
100 }
101
102 export default function MenuContent() {
103   const searchParams = useSearchParams();
104   const stallId = searchParams.get("stallId");
105   const stallName = searchParams.get("stallName") || "Stall";
106   const token = useDashboardStore((state) => state.token);
107   const [categories, setCategories] = useState([]);
108   const [activeCategoryId, setActiveCategoryId] = useState(null);
109   const [inactiveCategories, setInactiveCategories] = useState(true);
110   const [inactiveItems, setInactiveItems] = useState(true);
111   const [loading, setLoading] = useState(Boolean(stallId));
112   const [loadError, setLoadError] = useState("");
113
114   useEffect(() => {
115     let active = true;
116
117     if (!stallId) {
118       setCategories([]);
119       setActiveCategoryId(null);
120       setLoading(false);
121       setLoadError("Stall ID is unavailable.");
122       return () => {
123         active = false;
124       };
125     }
126
127     setLoading(true);
128     setLoadError("");
129     fetchStallItems({ stallId, token, dedupe: false })
130       .then((response) => {
131         if (!active) return;
132         const nextCategories = normalizeMenuResponse(response);
133         setCategories(nextCategories);
134         setActiveCategoryId(nextCategories[0]?.id ?? null);
135       })
136       .catch((error) => {
137         if (!active) return;
138         console.error('Unable to load menu for stall ${stallId}', error);
139         setCategories([]);

```



```
140     setActiveCategoryId(null);
141     setLoadError("Unable to load this stall menu.");
142   })
143   .finally(() => {
144     if (active) setLoading(false);
145   });
146
147   return () => {
148     active = false;
149   };
150 }, [stallId, token]);
151
152 const activeCategory = categories.find((c) => c.id === activeCategoryId) ?? null;
153
154 const updateCategory = (updates) => {
155   setCategories((prev) =>
156     prev.map((c) => (c.id === activeCategoryId ? { ...c, ...updates } : c))
157   );
158 };
159
160 const updateItem = (itemId, updates) => {
161   setCategories((prev) =>
162     prev.map((c) =>
163       c.id === activeCategoryId
164         ? {
165           ...c,
166           items: c.items.map((item) =>
167             item.id === itemId ? { ...item, ...updates } : item
168           )
169         }
170       : c
171     )
172   );
173 };
174
175 const reorderItems = (sourceId, targetId, placement = "before") => {
176   if (sourceId === targetId) return;
177
178   setCategories((prev) =>
179     prev.map((category) => {
180       if (category.id !== activeCategoryId) return category;
181
182       const sourceIndex = category.items.findIndex((item) => item.id === sourceId);
183       const targetIndex = category.items.findIndex((item) => item.id === targetId);
184       if (sourceIndex < 0 || targetIndex < 0) return category;
185
186       const reorderedItems = [...category.items];
187       const [movedItem] = reorderedItems.splice(sourceIndex, 1);
188       let insertIndex = targetIndex + (placement === "after" ? 1 : 0);
189       if (sourceIndex < insertIndex) insertIndex -= 1;
190       reorderedItems.splice(insertIndex, 0, movedItem);
191
192       return {
193         ...category,
194         items: reorderedItems.map((item, index) => ({
195           ...item,
196           position: index + 1
197         })))
198     };
199   });
200 };
201
202 const addCategory = () => {
203   const newId = Date.now();
204   const newName = `cat${categories.length + 1}`;
205   const newItemId = newId + 1;
206   setCategories((prev) => [
207     ...prev,
208     {
209
```

```

210     id: newId,
211     name: newName,
212     count: 1,
213     active: true,
214     vat: 0,
215     gst: 0,
216     gstInclusive: true,
217     items: [
218       {
219         id: newItemId,
220         name: "",
221         price: 0,
222         happy: 0,
223         hsn: "",
224         barcode: "",
225         epc: "",
226         type: "FOOD",
227         tags: [],
228         active: true,
229         image: null,
230         supplierCode: "",
231         groupId: "",
232         variant: "",
233         colour: ""
234       }
235     ]
236   }
237   });
238   setActiveCategoryId(newId);
239 };
240
241 const downloadExcel = () => {
242   const header = [
243     "CATEGORY", "CATEGORY STATUS", "ITEM", "COST", "MRP",
244     "PRICE", "HAPPY PRICE", "BARCODE", "EPC", "QUANTITY",
245     "ITEM CODE", "DESCRIPTION", "ITEM STATUS", "TAGS"
246   ];
247   const rows = [header];
248   for (const cat of categories) {
249     for (const item of cat.items) {
250       rows.push([
251         cat.name,
252         cat.active ? "active" : "inactive",
253         item.name,
254         0,
255         0,
256         item.price,
257         item.happy ?? 0,
258         item.barcode ?? "",
259         item.epc ?? "",
260         0,
261         item.supplierCode ?? "",
262         "",
263         item.active ? "active" : "inactive",
264         (item.tags ?? []).join(",")
265       ]);
266     }
267   }
268   const ws = XLSX.utils.aoa_to_sheet(rows);
269   const wb = XLSX.utils.book_new();
270   XLSX.utils.book_append_sheet(wb, ws, "Menu");
271   XLSX.writeFile(wb, `${stallName} Inventory.xlsx`);
272 };
273
274 const addItem = () => {
275   if (!activeCategoryId) return;
276   const newId = Date.now();
277   setCategories((prev) =>
278     prev.map((c) =>
279       c.id === activeCategoryId

```

```

280     ? {
281         ...c,
282         count: c.count + 1,
283         items: [
284             ...c.items,
285             {
286                 id: newId,
287                 name: "",
288                 price: 0,
289                 happy: 0,
290                 hsn: "",
291                 barcode: "",
292                 epc: "",
293                 type: "FOOD",
294                 tags: [],
295                 active: true,
296                 image: null,
297                 supplierCode: "",
298                 groupId: "",
299                 variant: "",
300                 colour: ""
301             }
302         ]
303     }
304     : c
305 )
306 );
307 };
308
309 const visibleItems = (activeCategory?.items ?? []).filter(
310     (item) => inactiveItems || item.active
311 );
312
313 return (
314     <div className="flex flex-col gap-4">
315         <MenuBar
316             stallName={stallName}
317             inactiveCategories={inactiveCategories}
318             onToggleInactiveCategories={() => setInactiveCategories((p) => !p)}
319             onDownload={downloadExcel}
320             onAddCategory={addCategory}
321         />
322
323         <div className="overflow-hidden rounded-[15px] border border-(--line) border-l-[3px] border-l-(--orange) bg-(--surface) shadow-(--shadow)">
324             {loading ? (
325                 <div className="flex min-h-[220px] items-center justify-center text-[13px] font-medium text-(--muted)">
326                     Loading menu...
327                 </div>
328             ) : loadError ? (
329                 <div className="flex min-h-[220px] items-center justify-center px-5 text-center text-[13px] font-semibold text-(--orange)">
330                     {loadError}
331                 </div>
332             ) : (
333                 <>
334                     <CategoryTabs
335                         categories={categories}
336                         activeId={activeCategoryId}
337                         onSelect={setActiveCategoryId}
338                     />
339
340                     <CategoryDetailPanel
341                         category={activeCategory}
342                         onUpdate={updateCategory}
343                     />
344
345                     <MenuItemsTable
346                         items={visibleItems}

```

```
347         onItemUpdate={updateItem}
348         onReorderItems={reorderItems}
349         onAddItem={addItem}
350         inactiveItems={inactiveItems}
351         onToggleInactiveItems={() => setInactiveItems((p) => !p)}
352     />
353 </>
354     )}
355 </div>
356 </div>
357 );
358 }
```

FILE: apps/dashboard/src/app/Config/menu/components/MenuIcons.js

```

1  "use client";
2
3  const base = {
4    fill: "none",
5    stroke: "currentColor",
6    strokeWidth: "2",
7    strokeLinecap: "round",
8    strokeLinejoin: "round",
9    "aria-hidden": true,
10 };
11
12 export function DownloadIcon({ className = "h-4 w-4" }) {
13   return (
14     <svg viewBox="0 0 24 24" className={className} {...base}>
15       <path d="M12 3v12" />
16       <path d="M8 11l4 4-4-4" />
17       <rect x="4" y="17" width="16" height="4" rx="2" />
18     </svg>
19   );
20 }
21
22 export function PlusIcon({ className = "h-4 w-4" }) {
23   return (
24     <svg viewBox="0 0 24 24" className={className} {...base}>
25       <path d="M12 5v14" />
26       <path d="M5 12h14" />
27     </svg>
28   );
29 }
30
31 export function CloudUploadIcon({ className = "h-4 w-4" }) {
32   return (
33     <svg viewBox="0 0 24 24" className={className} {...base}>
34       <polyline points="16 16 12 12 8 16" />
35       <line x1="12" y1="12" x2="12" y2="21" />
36       <path d="M20.39 18.39A5 5 0 018 9h-1.26A8 8 0 103 16.3" />
37     </svg>
38   );
39 }
40
41 export function SaveIcon({ className = "h-4 w-4" }) {
42   return (
43     <svg viewBox="0 0 24 24" className={className} {...base}>
44       <path d="M19 21H5a2 2 0 01-2-2V5a2 2 0 012-2h11l5 5v11a2 2 0 01-2 2z" />
45       <polyline points="17 21 17 13 7 13 7 21" />
46       <polyline points="7 3 7 8 15 8" />
47     </svg>
48   );
49 }
50
51 export function GearIcon({ className = "h-4 w-4" }) {
52   return (
53     <svg viewBox="0 0 24 24" className={className} {...base}>
54       <circle cx="12" cy="12" r="3" />
55       <path d="M19.4 15a1.65 1.65 0 00-.33 1.82l.06.06a2 2 0 010 2.83 2 2 0 01-2.83 0l-.06-.06a1.65 1.65 0 00-1.82-.33 1.65 1.65 0 00-1 1.51V21a2 2 0 01-4 0v-.09a1.65 1.65 0 009 19.4a1.65 1.65 0 00-1.82.33l-.06.06a2 2 0 01-2.83-.06-.06A1.65 1.65 0 004.6 9a1.65 1.65 0 00-.33-1.82l-.06-.06a2 2 0 012.83-2.83l.06.06A1.65 1.65 0 009 4.68a1.65 1.65 0 001-1.51V3a2 2 0 014 0v.09a1.65 1.65 0 001 1.51 1.65 1.65 0 001.82-.33l.06-.06a2 2 0 012.83 2.83l-.06.06A1.65 1.65 0 0019.4 9a1.65 1.65 0 001.51 1H21a2 2 0 010 4h-.09a1.65 1.65 0 00-1.51 1z" />
56     </svg>
57   );
58 }
59
60 export function BarcodeIcon({ className = "h-4 w-4" }) {
61   return (
62     <svg viewBox="0 0 24 24" className={className} {...base}>
63       <path d="M3 5v14" />
64       <path d="M6 5v14" />

```

```

65     <path d="M9 5v14" />
66     <path d="M12 5v14" />
67     <path d="M16 5v14" />
68     <path d="M20 5v14" />
69   </svg>
70   );
71 }
72
73 export function ImageUploadIcon({ className = "h-5 w-5" }) {
74   return (
75     <svg viewBox="0 0 24 24" className={className} {...base}>
76       <rect x="3" y="3" width="18" height="18" rx="2" />
77       <circle cx="8.5" cy="8.5" r="1.5" />
78       <polyline points="21 15 16 10 5 21" />
79       <path d="M14 9l2-2 2 2" />
80     </svg>
81   );
82 }
83
84 export function TagIcon({ className = "h-4 w-4" }) {
85   return (
86     <svg viewBox="0 0 24 24" className={className} {...base}>
87       <path d="M20.59 13.41l-7.17 7.17a2 2 0 01-2.83 0L2 12V2h10l8.59 8.59a2 2 0 010 2.82z" />
88       <line x1="7" y1="7" x2="7.01" y2="7" />
89     </svg>
90   );
91 }
92
93 export function SearchIcon({ className = "h-4 w-4" }) {
94   return (
95     <svg viewBox="0 0 24 24" className={className} {...base}>
96       <circle cx="11" cy="11" r="7" />
97       <path d="m20 20-3.5-3.5" />
98     </svg>
99   );
100 }
101
102 export function ChevronDownIcon({ className = "h-3.5 w-3.5" }) {
103   return (
104     <svg viewBox="0 0 24 24" className={className} {...base}>
105       <polyline points="6 9 12 15 18 9" />
106     </svg>
107   );
108 }
109
110 export function GripVerticalIcon({ className = "h-4 w-4" }) {
111   return (
112     <svg viewBox="0 0 24 24" className={className} {...base}>
113       <circle cx="9" cy="5" r="1" fill="currentColor" stroke="none" />
114       <circle cx="15" cy="5" r="1" fill="currentColor" stroke="none" />
115       <circle cx="9" cy="12" r="1" fill="currentColor" stroke="none" />
116       <circle cx="15" cy="12" r="1" fill="currentColor" stroke="none" />
117       <circle cx="9" cy="19" r="1" fill="currentColor" stroke="none" />
118       <circle cx="15" cy="19" r="1" fill="currentColor" stroke="none" />
119     </svg>
120   );
121 }

```

FILE: apps/dashboard/src/app/Config/menu/components/MenuItemsTable.js

```

1  "use client";
2
3  import { useEffect, useRef, useState } from "react";
4  import {
5    PlusIcon,
6    SearchIcon,
7    BarcodeIcon,
8    ImageUploadIcon,
9    ChevronDownIcon,
10   TagIcon,
11   CloudUploadIcon,
12   DownloadIcon,
13   GripVerticalIcon
14 } from "../MenuIcons";
15
16 const ITEM_TYPES = ["FOOD", "BEVERAGE", "NON-FOOD", "COMBO", "OTHER"];
17 const ITEM_TAGS = ["VEG", "NON-VEG", "VEGAN", "SPICY", "GLUTEN-FREE"];
18
19 const INPUT_CLASS =
20   "rounded-[8px] border border-(--line) bg-(--surface) px-2.5 py-1.5 text-[13px] font-medium text-(--text) outline-none transition placeholder:text-(--faint) focus:border-(--orange) focus:shadow-[0_0_3px_rgba(224,68,32,0.12)]";
21
22 const SUB_INPUT_CLASS =
23   "font-vcv rounded-[8px] border border-(--line) bg-(--surface2) px-2.5 py-1.5 text-[11px] uppercase tracking-[0.06em] text-(--muted) outline-none transition placeholder:text-(--faint) focus:border-(--orange) focus:shadow-[0_0_3px_rgba(224,68,32,0.12)]";
24
25 const TH_CLASS = "font-vcv pb-2.5 pr-3 text-[9.5px] tracking-[0.15em] text-(--orange)";
26
27 function SmallToggle({ active, onToggle }) {
28   return (
29     <button
30       type="button"
31       onClick={onToggle}
32       aria-pressed={active}
33       className={`flex h-6 w-11 shrink-0 items-center rounded-full p-0.5 transition-colors duration-200 $
34 {
35   active ? "bg-[linear-gradient(135deg,#E04420,#341CD6)]" : "bg-(--line)"
36 }`}
37     >
38     <span
39       className={`h-5 w-5 rounded-full bg-white shadow-sm transition-transform duration-200 ${
40         active ? "translate-x-5" : "translate-x-0"
41       }`}
42     />
43   </button>
44 );
45 }
46
47 function TypeDropdown({ value, onChange }) {
48   const [open, setOpen] = useState(false);
49   const ref = useRef(null);
50
51   useEffect(() => {
52     const handler = (e) => { if (!ref.current?.contains(e.target)) setOpen(false); };
53     if (open) document.addEventListener("mousedown", handler);
54     return () => document.removeEventListener("mousedown", handler);
55   }, [open]);
56
57   return (
58     <div ref={ref} className="relative">
59       <button
60         type="button"
61         onClick={() => setOpen((p) => !p)}
62         className="flex h-7 items-center gap-1 rounded-full bg-(--orange) px-2.5 text-[11px] font-semibol
63         d text-white transition hover:brightness-105"
64       >
65         <span className="max-w-[68px] truncate">{value}</span>

```

```

64     <ChevronDownIcon className="h-3 w-3 shrink-0" />
65   </button>
66   {open && (
67     <div className="absolute left-0 top-full z-30 mt-1 w-[110px] overflow-hidden rounded-[10px] borde
68 r border-(--line) bg-(--surface) shadow-(--shadowUp)">
69       {ITEM_TYPES.map((type) => (
70         <button
71           key={type}
72           type="button"
73           onClick={() => { onChange(type); setOpen(false); }}
74           className={`block w-full px-3 py-2 text-left text-[12.5px] font-semibold transition hover:b
75 g-(--surface2) ${
76           value === type ? "text-(--orange)" : "text-(--text)"
77         }`}
78         >
79           {type}
80         </button>
81       ))}
82     </div>
83   );
84 }
85
86 function TagsDropdown({ value = [], onChange }) {
87   const [open, setOpen] = useState(false);
88   const ref = useRef(null);
89
90   useEffect(() => {
91     const handler = (e) => { if (!ref.current?.contains(e.target)) setOpen(false); };
92     if (open) document.addEventListener("mousedown", handler);
93     return () => document.removeEventListener("mousedown", handler);
94   }, [open]);
95
96   const label = value.length > 0 ? value[0] : "Select";
97
98   return (
99     <div ref={ref} className="relative">
100       <button
101         type="button"
102         onClick={() => setOpen((p) => !p)}
103         className="flex h-7 items-center gap-1 rounded-full bg-(--orange) px-2.5 text-[11px] font-semibol
104 d text-white transition hover:brightness-105"
105       >
106         <span className="max-w-[60px] truncate">{label}</span>
107         <ChevronDownIcon className="h-3 w-3 shrink-0" />
108       </button>
109       {open && (
110         <div className="absolute left-0 top-full z-30 mt-1 w-[140px] overflow-hidden rounded-[10px] borde
111 r border-(--line) bg-(--surface) shadow-(--shadowUp)">
112           {ITEM_TAGS.map((tag) => {
113             const checked = value.includes(tag);
114             return (
115               <button
116                 key={tag}
117                 type="button"
118                 onClick={() => {
119                   onChange(checked ? value.filter((t) => t !== tag) : [...value, tag]);
120                 }}
121                 className="flex w-full items-center gap-2 px-3 py-2 text-left text-[12.5px] font-semibold
122 transition hover:bg-(--surface2)"
123               >
124                 <span
125                   className={`flex h-3.5 w-3.5 shrink-0 items-center justify-center rounded border ${
126                     checked ? "border-(--orange) bg-(--orange)" : "border-(--line)"
127                   }`}
128                 >
129                   {checked && (
130                     <svg viewBox="0 0 12 12" className="h-2.5 w-2.5 text-white" fill="none" stroke="curre
131 ntColor" strokeWidth="2.5">

```


[illegible]

```

192     </td>
193     <td className="py-3 pr-3">
194       <input type="text" value={item.epc ?? ""} onChange={(e) => onUpdate({ epc: e.target.value })} p
195       laceholder="epc" className={`w-24 ${INPUT_CLASS}`} />
196     </td>
197     <td className="py-3 pr-3">
198       <TypeDropdown value={item.type ?? "FOOD"} onChange={(val) => onUpdate({ type: val })} />
199     </td>
200     <td className="py-3 pr-3">
201       <TagsDropdown value={item.tags ?? []} onChange={(val) => onUpdate({ tags: val })} />
202     </td>
203     <td className="py-3 pr-3">
204       <SmallToggle active={item.active} onToggle={() => onUpdate({ active: !item.active })} />
205     </td>
206     <td className="py-3">
207       <button
208         type="button"
209         className="flex h-8 w-8 items-center justify-center rounded-[8px] border border-dashed border
210         (---line) bg-(---surface) text-(---faint) transition hover:border-(---orange) hover:text-(---orange)"
211         aria-label="Upload image"
212       >
213         <ImageUploadIcon className="h-4 w-4" />
214       </button>
215     </td>
216   </tr>
217   <tr className={`border-b border-(---line2) ${dropPosition === "after" ? "border-b-2 border-b-(---oran
218   ge)" : ""}`}>
219     <td colspan={10} className="pb-3 pt-0">
220       <div className="flex flex-wrap items-center gap-2 pl-1">
221         <input type="text" value={item.supplierCode ?? ""} onChange={(e) => onUpdate({ supplierCode:
222         e.target.value })} placeholder="SUPPLIER CODE" className={`w-[130px] ${SUB_INPUT_CLASS}`} />
223         <input type="text" value={item.groupId ?? ""} onChange={(e) => onUpdate({ groupId: e.target.v
224         alue })} placeholder="GROUP-ID" className={`w-[100px] ${SUB_INPUT_CLASS}`} />
225         <input type="text" value={item.variant ?? ""} onChange={(e) => onUpdate({ variant: e.target.v
226         alue })} placeholder="VARIANT" className={`w-[100px] ${SUB_INPUT_CLASS}`} />
227         <input type="text" value={item.colour ?? ""} onChange={(e) => onUpdate({ colour: e.target.val
228         ue })} placeholder="COLOUR" className={`w-[100px] ${SUB_INPUT_CLASS}`} />
229         <button
230           type="button"
231           className="flex h-9 w-9 items-center justify-center rounded-[10px] bg-[rgba(224,68,32,0.08)
232           ] text-(---orange) transition hover:bg-[rgba(224,68,32,0.14)]"
233           aria-label="Upload item image"
234         >
235           <ImageUploadIcon className="h-5 w-5" />
236         </button>
237       </div>
238     </td>
239   </tr>
240 </tbody>
241 </table>
242 );
243 }
244
245 export default function MenuItemsTable({
246   items,
247   onItemUpdate,
248   onReorderItems,
249   onAddItem,
250   inactiveItems,
251   onToggleInactiveItems
252 }) {
253   const [search, setSearch] = useState("");
254   const [draggedItemId, setDraggedItemId] = useState(null);
255   const [dropTarget, setDropTarget] = useState(null);
256
257   const filtered = (items ?? []).filter(
258     (item) => !search || item.name.toLowerCase().includes(search.toLowerCase())
259   );
260
261   const clearDragState = () => {

```

```

254     setDraggedItemId(null);
255     setDropTarget(null);
256   };
257
258   const handleDragStart = (event, itemId) => {
259     setDraggedItemId(itemId);
260     event.dataTransfer.effectAllowed = "move";
261     event.dataTransfer.setData("text/plain", String(itemId));
262   };
263
264   const handleDragOver = (event, itemId) => {
265     event.preventDefault();
266     if (draggedItemId == null || draggedItemId === itemId) {
267       setDropTarget(null);
268       return;
269     }
270
271     event.dataTransfer.dropEffect = "move";
272     const bounds = event.currentTarget.getBoundingClientRect();
273     const placement = event.clientY < bounds.top + bounds.height / 2 ? "before" : "after";
274     setDropTarget({ itemId, placement });
275   };
276
277   const handleDrop = (event, targetId) => {
278     event.preventDefault();
279     const rawSourceId = event.dataTransfer.getData("text/plain");
280     const sourceItem = (items ?? []).find((item) => String(item.id) === rawSourceId);
281     const sourceId = sourceItem?.id ?? draggedItemId;
282     const placement = dropTarget?.itemId === targetId ? dropTarget.placement : "before";
283
284     if (sourceId != null && sourceId !== targetId) {
285       onReorderItems?.(sourceId, targetId, placement);
286     }
287     clearDragState();
288   };
289
290   return (
291     <div className="flex flex-col">
292       { /* Items action bar */ }
293       <div className="flex flex-wrap items-center gap-3 border-b border-(--line2) px-5 py-3">
294         <div className="flex flex-wrap items-center gap-2">
295           <button
296             type="button"
297             onClick={onAddItem}
298             className="flex h-9 items-center gap-1.5 rounded-[8px] bg-(--text) px-3 text-[12.5px] font-se
299             mibold text-(--bg) transition hover:bg-(--orange)"
300           >
301             <PlusIcon className="h-3.5 w-3.5" />
302             ITEM
303           </button>
304
305           <div className="flex items-center gap-2 text-[12.5px] font-semibold text-(--muted)">
306             Inactive Items:
307             <button
308               type="button"
309               onClick={onToggleInactiveItems}
310               aria-pressed={inactiveItems}
311               className={`flex h-6 w-11 shrink-0 items-center rounded-full p-0.5 transition-colors durati
312               on-200 ${
313                 inactiveItems ? "bg-[linear-gradient(135deg,#E04420,#341CD6)]" : "bg-(--line)"
314               }`}
315             >
316               <span
317                 className={`h-5 w-5 rounded-full bg-white shadow-sm transition-transform duration-200 ${
318                   inactiveItems ? "translate-x-5" : "translate-x-0"
319                 }`}
320               />
321             </button>
322           </div>
323         </div>

```

```

322         <button
323             type="button"
324             className="flex h-9 items-center gap-1.5 rounded-[8px] bg(--text) px-3 text-[12.5px] font-se
mibold text(--bg) transition hover:bg(--orange)"
325         >
326             <TagIcon className="h-3.5 w-3.5" />
327             SET TYPE
328         </button>
329     </div>
330
331     <div className="flex-1" />
332
333     <div className="flex items-center gap-2">
334         <button
335             type="button"
336             className="flex h-9 items-center gap-1.5 rounded-[8px] bg-[linear-gradient(135deg,#E04420,#34
1CD6)] px-3 text-[12.5px] font-semibold text-white transition hover:brightness-105"
337         >
338             <CloudUploadIcon className="h-3.5 w-3.5" />
339             Menu
340         </button>
341         <button
342             type="button"
343             className="flex h-9 items-center gap-1.5 rounded-[8px] border border(--orange) bg(--surface
) px-3 text-[12.5px] font-semibold text(--orange) transition hover:bg-[rgba(224,68,32,0.06)]"
344         >
345             <DownloadIcon className="h-3.5 w-3.5" />
346             Sample
347         </button>
348     </div>
349 </div>
350
351     {/ * Search */}
352     <div className="px-5 py-3">
353         <div className="flex items-center gap-2.5 rounded-[10px] border border(--line) bg(--surface) px
-3 py-2.5 transition focus-within:border(--orange)">
354             <SearchIcon className="h-4 w-4 shrink-0 text(--muted) opacity-60" />
355             <input
356                 type="text"
357                 value={search}
358                 onChange={(e) => setSearch(e.target.value)}
359                 placeholder="Search Items"
360                 className="min-w-0 flex-1 bg-transparent text-[13px] font-medium text(--text) outline-none p
laceholder:text(--faint)"
361             />
362         </div>
363     </div>
364
365     {/ * Table */}
366     <div className="overflow-x-auto px-5 pb-5">
367         <table className="min-w-full text-left">
368             <thead>
369                 <tr className="border-b border(--line)">
370                     <th className={TH_CLASS}>ITEM</th>
371                     <th className={TH_CLASS}>PRICE</th>
372                     <th className={TH_CLASS}>HAPPY</th>
373                     <th className={TH_CLASS}>HSN</th>
374                     <th className="pb-2.5 pr-3">
375                         <BarcodeIcon className="h-4 w-4 text(--orange)" />
376                     </th>
377                     <th className="pb-2.5 pr-3">
378                         <div className="font-vcr flex items-center gap-1 text-[9.5px] tracking-[0.15em] text(--o
range)">
379                             <BarcodeIcon className="h-4 w-4" />
380                             EPC
381                         </div>
382                     </th>
383                     <th className={TH_CLASS}>TYPE</th>
384                     <th className={TH_CLASS}>TAGS</th>
385                     <th className={TH_CLASS}>STATUS</th>

```

```

386         <th className="font-vcr pb-2.5 text-[9.5px] tracking-[0.15em] text-(--orange)">IMAGE</th>
387     </tr>
388 </thead>
389 {filtered.length === 0 ? (
390     <tbody>
391         <tr>
392             <td colSpan={10} className="py-12 text-center text-[13px] font-medium text-(--muted)">
393                 No items found. Click <span className="text-(--orange)">+ ITEM</span> to add one.
394             </td>
395         </tr>
396     </tbody>
397 ) : (
398     filtered.map((item) => (
399         <ItemRow
400             key={item.id}
401             item={item}
402             onUpdate={({updates}) => onItemUpdate?.(item.id, updates)}
403             onDragStart={handleDragStart}
404             onDragOver={handleDragOver}
405             onDrop={handleDrop}
406             onDragEnd={clearDragState}
407             isDragging={draggedItemId === item.id}
408             dropPosition={dropTarget?.itemId === item.id ? dropTarget.placement : null}
409         </ItemRow>
410     ))
411 )}
412 </table>
413 </div>
414 </div>
415 );
416 }

```

FILE: apps/dashboard/src/app/Config/menu/page.js

```
1 import { Suspense } from "react";
2 import Header from "../../components/Header";
3 import ConfigTransition from "../../components/ConfigTransition";
4 import MenuContent from "../components/MenuContent";
5
6 export default function MenuPage() {
7   return (
8     <main
9       className="min-h-screen bg-(--bg) pb-10"
10       style={{ fontFamily: '"Poppins", system-ui, sans-serif' }}
11     >
12       <Header areaLabel="Menu" variant="portal" />
13       <ConfigTransition>
14         <div className="mx-auto w-full max-w-[1780px] pr-4 pl-[72px] pt-6 md:pr-7 md:pl-[88px] max-[900px]
15           :px-3">
16           <Suspense
17             fallback={(
18               <div className="flex min-h-[220px] items-center justify-center rounded-[15px] border border
19                 -(--line) bg-(--surface) text-[13px] font-medium text-(--muted)">
20                 Loading menu...
21               </div>
22             )}>
23             <MenuContent />
24           </Suspense>
25         </div>
26       </ConfigTransition>
27     </main>
28   );
29 }
```

FILE: apps/dashboard/src/app/Config/page.js

```
1 import Header from "../components/Header";
2 import ConfigTransition from "../components/ConfigTransition";
3 import VendorConfigurationContent from "../components/VendorConfigurationContent";
4
5 export default function ConfigPage() {
6   return (
7     <main
8       className="min-h-screen bg-(--bg) pb-10"
9       style={{ fontFamily: "'Poppins', system-ui, sans-serif" }}
10    >
11      <Header
12        areaLabel="Configuration"
13        breadcrumb="Admin"
14        variant="portal"
15        showEditEventButton
16      />
17      <ConfigTransition>
18        <div className="mx-auto w-full max-w-[1780px] pr-4 pl-[72px] pt-6 md:pr-7 md:pl-[88px] max-[900px
19]:px-3">
20          <VendorConfigurationContent />
21        </div>
22      </ConfigTransition>
23    </main>
24  );
25 }
```

FILE: apps/dashboard/src/app/Reports/components/ReportFilters.js

```

1  "use client";
2
3  import { Fragment, useEffect, useMemo, useState } from "react";
4  import { startReportBuild } from "../../lib/dashboardApi";
5
6  const EVENT_REPORT_VALUE = "event";
7  const EVENT_REPORT_LABEL = "Event Report";
8  const EVENT_ID_TYPE = "event";
9  const VENDOR_ID_TYPE = "vendor";
10 const VENDOR_VALUE_PREFIX = "vendor:";
11
12 function getVendorId(vendor) {
13   return vendor?.id ?? vendor?.vendorId ?? vendor?.vendor_id ?? null;
14 }
15
16 function getVendorName(vendor) {
17   return (
18     vendor?.name ??
19     vendor?.vendorName ??
20     vendor?.vendor_name ??
21     vendor?.title ??
22     "Unnamed vendor"
23   );
24 }
25
26 function toVendorValue(vendorId) {
27   return `${VENDOR_VALUE_PREFIX}${vendorId}`;
28 }
29
30 const REPORT_TYPE_OPTIONS = [
31   "SUMMARY",
32   "REDEMPTION-ACCESS SUMMARY",
33   "COMP TRANSACTION DUMP",
34   "PERSO DUMP",
35   "INVENTORY STOCK SUMMARY",
36   "STALLS STOCK SUMMARY",
37   "ACCESSX DUMP",
38   "ACCESSX SUMMARY",
39   "WHITELIST PERSONS DUMP",
40   "BOOKING ITEMS DUMP",
41   "CONSENT DUMP",
42 ];
43
44 function parseEventDays(value) {
45   const rawDays = Array.isArray(value) ? value : String(value ?? "").split(",");
46
47   return rawDays
48     .map((day) => String(day ?? "").trim())
49     .filter(Boolean)
50     .filter((day, index, list) => list.indexOf(day) === index);
51 }
52
53 function toDayPayloadValue(day) {
54   const numericDay = Number(day);
55   return Number.isNaN(numericDay) ? day : numericDay;
56 }
57
58 function createRequestId() {
59   if (typeof crypto !== "undefined" && typeof crypto.randomUUID === "function") {
60     return crypto.randomUUID();
61   }
62
63   return `${Date.now()}-${Math.random().toString(16).slice(2)}`;
64 }
65
66 function DownloadIcon({ className = "h-4 w-4" }) {
67   return (
68     <svg viewBox="0 0 24 24" className={className} fill="none" stroke="currentColor" strokeWidth="1.8" strokeLinecap="round" strokeLinejoin="round">

```



```

69     <path d="M12 3v12" />
70     <path d="m7 10 5 5 5-5" />
71     <path d="M5 21h14" />
72   </svg>
73   );
74 }
75
76 function ClockIcon() {
77   return (
78     <svg viewBox="0 0 24 24" className="h-4 w-4" fill="none" stroke="currentColor" strokeWidth="1.8" strokeLinecap="round" strokeLinejoin="round">
79       <circle cx="12" cy="12" r="9" />
80       <path d="M12 7v5l3 2" />
81     </svg>
82   );
83 }
84
85 function CompassIcon() {
86   return (
87     <svg viewBox="0 0 24 24" className="h-4 w-4" fill="none" stroke="currentColor" strokeWidth="1.6" strokeLinecap="round" strokeLinejoin="round">
88       <path d="m12 2 4 8 6 2-8 4-2 6-4-8-6-2 8-4z" />
89       <circle cx="12" cy="12" r="2" />
90     </svg>
91   );
92 }
93
94 function TypeIcon() {
95   return (
96     <svg viewBox="0 0 24 24" className="h-4 w-4" fill="none" stroke="currentColor" strokeWidth="1.8" strokeLinecap="round" strokeLinejoin="round">
97       <rect x="5" y="3" width="14" height="18" rx="2" />
98       <path d="M8 8h8" />
99       <path d="M8 12h8" />
100      <path d="M8 16h5" />
101    </svg>
102  );
103 }
104
105 function ChevronIcon() {
106   return (
107     <svg viewBox="0 0 24 24" className="h-4 w-4" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
108       <path d="m6 9 6 6 6-6" />
109     </svg>
110   );
111 }
112
113 function CheckBox() {
114   return <span className="h-4 w-4 rounded-[4px] border border-(--line) bg-(--surface)" />;
115 }
116
117 function LabelCell({ label, icon = null, withCheck = false }) {
118   return (
119     <div className="flex h-11 items-center gap-2 text-(--muted) max-[640px]:h-9">
120       {withCheck ? <CheckBox /> : icon}
121       <span className="font-vcr text-[10px] uppercase tracking-[0.14em]">{label}</span>
122     </div>
123   );
124 }
125
126 function SelectRow({
127   label,
128   value,
129   icon,
130   withCheck = false,
131   options = null,
132   optionGroups = null,
133   onChange = null,
134   emptyLabel = "No options available"

```

```

135 }) {
136   const isEmpty = Array.isArray(options) && options.length === 0;
137   const hasSelect = Boolean(options || optionGroups);
138   return (
139     <div className="grid min-h-[58px] grid-cols-[230px_1fr] items-center gap-4 max-[640px]:min-h-[46px] m
ax-[640px]:grid-cols-[110px_minmax(0,1fr)] max-[640px]:gap-2">
140       <LabelCell label={label} icon={icon} withCheck={withCheck} />
141       {hasSelect ? (
142         <div className="relative h-11 max-[640px]:h-10">
143           <select
144             aria-label={label}
145             value={value}
146             onChange={onChange}
147             disabled={isEmpty}
148             className="h-full w-full appearance-none rounded-[10px] border border-(--line) bg-(--surface)
px-3.5 pr-11 text-[13px] font-medium text-(--text) outline-none transition hover:border-(--orange) focus
:border-(--orange) disabled:cursor-not-allowed disabled:bg-(--surface2) disabled:text-(--faint) disabled:
hover:border-(--line)"
149             >
150             {optionGroups
151               ? optionGroups.map((group, groupIndex) => {
152                 const groupOptions = group.options.map((option) => (
153                   <option key={option.value} value={option.value} disabled={option.disabled}>
154                     {option.label}
155                   </option>
156                 ));
157
158                 return group.label ? (
159                   <optgroup key={group.label} label={group.label}>
160                     {groupOptions}
161                   </optgroup>
162                 ) : (
163                   <Fragment key={`group-${groupIndex}`}>{groupOptions}</Fragment>
164                 );
165               })
166               : isEmpty
167               ? <option value="">{emptyLabel}</option>
168               : options.map((option, index) => (
169                 <option key={`-${option}-${index}`} value={String(index)}>
170                   {option}
171                 </option>
172               ))}
173             </select>
174             <span className="pointer-events-none absolute inset-y-0 right-0 grid w-[42px] place-items-cente
r text-(--muted) opacity-60 max-[640px]:w-7">
175               <ChevronIcon />
176             </span>
177           </div>
178         ) : (
179           <button
180             type="button"
181             className="grid h-11 grid-cols-[1fr_42px] items-center rounded-[10px] border border-(--line) bg
-(--surface) text-left text-(--text) transition hover:border-(--orange) max-[640px]:h-10 max-[640px]:grid
-cols-[1fr_28px]"
182             >
183             <span className="truncate px-3.5 text-[13px] font-medium">{value}</span>
184             <span className="grid place-items-center text-(--muted) opacity-60">
185               <ChevronIcon />
186             </span>
187           </button>
188         )}
189       </div>
190     );
191 }
192
193 function DateRow({ startDate, endDate, onStartDateChange, onEndDateChange }) {
194   return (
195     <div className="grid min-h-[58px] grid-cols-[230px_minmax(280px,650px)] items-center gap-4 max-[640px]
:min-h-[46px] max-[640px]:grid-cols-[110px_minmax(0,1fr)] max-[640px]:gap-2">
196       <LabelCell label="Dates" withCheck />

```

```

197     <div className="grid h-11 grid-cols-[48px_1fr_42px_1fr] items-center overflow-hidden rounded-[10px]
border border-(--line) bg-(--surface2) max-[640px]:h-10 max-[640px]:grid-cols-[30px_1fr_20px_1fr]">
198     <span className="grid h-full place-items-center text-(--muted) opacity-70">
199       <ClockIcon />
200     </span>
201     <input
202       type="date"
203       aria-label="Report start date"
204       value={startDate}
205       max={endDate || undefined}
206       onChange={(event) => onStartDateChange(event.target.value)}
207       className="min-w-0 bg-transparent px-4 text-center text-[13px] text-(--text) outline-none max-[
640px]:px-1"
208     />
209     <span className="text-center text-[13px] text-(--faint)">--</span>
210     <input
211       type="date"
212       aria-label="Report end date"
213       value={endDate}
214       min={startDate || undefined}
215       onChange={(event) => onEndDateChange(event.target.value)}
216       className="min-w-0 bg-transparent px-4 text-center text-[13px] text-(--text) outline-none max-[
640px]:px-1"
217     />
218   </div>
219 </div>
220 );
221 }
222
223 export default function ReportFilters({
224   eventId,
225   token,
226   eventDays,
227   daysLoading = false,
228   vendors = [],
229   vendorsLoading = false,
230   onReportStarted
231 }) {
232   const [eventVendorValue, setEventVendorValue] = useState(EVENT_REPORT_VALUE);
233   const [reportTypeIndex, setReportTypeIndex] = useState("0");
234   const [dayIndex, setDayIndex] = useState("0");
235   const [startDate, setStartDate] = useState("");
236   const [endDate, setEndDate] = useState("");
237   const [submitting, setSubmitting] = useState(false);
238   const [submitError, setSubmitError] = useState("");
239   const [submitMessage, setSubmitMessage] = useState("");
240   const dayOptions = useMemo(() => parseEventDays(eventDays), [eventDays]);
241   const selectedDay = dayOptions[Number(dayIndex)] ?? dayOptions[0];
242
243   const vendorOptions = useMemo(
244     () =>
245       vendors
246         .map((vendor) => ({ id: getVendorId(vendor), name: getVendorName(vendor) }))
247         .filter((vendor) => vendor.id !== null && vendor.id !== ""),
248     [vendors]
249   );
250
251   const eventVendorGroups = useMemo(() => {
252     const vendorGroupOptions = vendorOptions.length
253       ? vendorOptions.map((vendor) => ({
254         value: toVendorValue(vendor.id),
255         label: vendor.name
256       }))
257       : [
258         {
259           value: "",
260           label: vendorsLoading ? "Loading vendors..." : "No vendors found",
261           disabled: true
262         }
263       ];

```

```

264
265     return [
266       { label: null, options: [{ value: EVENT_REPORT_VALUE, label: EVENT_REPORT_LABEL }] },
267       { label: "Vendors", options: vendorGroupOptions }
268     ];
269   }, [vendorOptions, vendorsLoading]);
270
271   const selectedVendor = useMemo(() => {
272     if (!eventVendorValue.startsWith(VENDOR_VALUE_PREFIX)) return null;
273     const vendorId = eventVendorValue.slice(VENDOR_VALUE_PREFIX.length);
274     return vendorOptions.find((vendor) => String(vendor.id) === vendorId) ?? null;
275   }, [eventVendorValue, vendorOptions]);
276
277   useEffect(() => {
278     setDayIndex("0");
279   }, [eventDays]);
280
281   useEffect(() => {
282     setEventVendorValue(EVENT_REPORT_VALUE);
283   }, [eventId]);
284
285   const handleDownload = async () => {
286     if (!startDate || !endDate) {
287       setSubmitMessage("");
288       setSubmitError("Select both start and end dates.");
289       return;
290     }
291
292     if (endDate < startDate) {
293       setSubmitMessage("");
294       setSubmitError("End date cannot be before the start date.");
295       return;
296     }
297
298     setSubmitting(true);
299     setSubmitError("");
300     setSubmitMessage("");
301
302     try {
303       const response = await startReportBuild({
304         eventId,
305         token,
306         dates: [startDate, endDate],
307         days: selectedDay ? [toDayPayloadValue(selectedDay)] : [0],
308         id: selectedVendor ? selectedVendor.id : eventId,
309         idType: selectedVendor ? VENDOR_ID_TYPE : EVENT_ID_TYPE,
310         type: REPORT_TYPE_OPTIONS[Number(reportTypeIndex)] ?? REPORT_TYPE_OPTIONS[0],
311         requestId: createRequestId()
312       });
313
314       if (response?.success === false) {
315         throw new Error(response?.message || "Unable to start the report.");
316       }
317
318       setSubmitMessage(response?.message || "Report generation started.");
319       onReportStarted?.(response);
320     } catch (error) {
321       console.error("Failed to start report build", error);
322       setSubmitError(error?.message || "Unable to start the report.");
323     } finally {
324       setSubmitting(false);
325     }
326   };
327
328   return (
329     <section className="rounded-[15px] border border-(--line) border-l-[3px] border-l-(--orange) bg-(--surface) p-4 shadow-(--shadow) max-[640px]:p-3">
330       <div className="flex flex-col gap-3 border-b border-(--line2) pb-3 lg:flex-row lg:items-center lg:justify-between">
331         <div className="flex items-center gap-2.5">

```

```

332         <span className="font-vcr grid h-9 w-9 place-items-center rounded-[10px] bg-[linear-gradient(14
0deg,#e04420,#8b5cf6)] text-[12px] text-white">
333         01
334         </span>
335         <h2 className="font-chillax text-[18px] font-semibold text-(--text)">Filter</h2>
336     </div>
337     <p className="text-[12.5px] font-light text-(--muted)">Select report scope before downloading.</p
>
338 </div>
339
340 <div className="grid pt-4 lg:grid-cols-[minmax(620px,920px)_1fr]">
341     <div>
342         <DateRow
343             startDate={startDate}
344             endDate={endDate}
345             onStartDateChange={setStartDate}
346             onEndDateChange={setEndDate}
347         />
348         <SelectRow
349             label="Days"
350             value={dayIndex}
351             withCheck
352             options={dayOptions}
353             emptyLabel={daysLoading ? "Loading days..." : "No days available"}
354             onChange={(event) => setDayIndex(event.target.value)}
355         />
356         <SelectRow
357             label="Event / Vendor"
358             value={eventVendorValue}
359             icon={<CompassIcon />}
360             optionGroups={eventVendorGroups}
361             onChange={(event) => setEventVendorValue(event.target.value)}
362         />
363         <SelectRow
364             label="Type"
365             value={reportTypeIndex}
366             icon={<TypeIcon />}
367             options={REPORT_TYPE_OPTIONS}
368             onChange={(event) => setReportTypeIndex(event.target.value)}
369         />
370     </div>
371     <div className="mt-3 grid min-h-[48px] grid-cols-[230px_minmax(280px,650px)] items-center gap-4
max-[640px]:min-h-[42px] max-[640px]:grid-cols-[110px_minmax(0,1fr)] max-[640px]:gap-2">
372         <span />
373         <button
374             type="button"
375             onClick={handleDownload}
376             disabled={submitting || eventId === "" || eventId == null}
377             className="inline-flex h-11 items-center justify-center gap-2 rounded-[10px] bg-(--text) px
-5 text-[13.5px] font-semibold text-(--bg) transition duration-200 hover:bg-(--orange) disabled:cursor-no
t-allowed disabled:opacity-50"
378         >
379             <DownloadIcon />
380             {submitting ? "Starting..." : "Download"}
381         </button>
382     </div>
383
384     {submitError || submitMessage ? (
385         <div className="grid grid-cols-[230px_minmax(280px,650px)] gap-4 pt-2 max-[640px]:grid-cols-[
110px_minmax(0,1fr)] max-[640px]:gap-2">
386             <span />
387             <p className="text-[11.5px] font-medium ${submitError ? "text-(--orange)" : "text-[#177657
]}"} role="status">
388                 {submitError || submitMessage}
389             </p>
390         </div>
391     ) : null}
392 </div>
393 <div className="hidden lg:block" />
394 </div>

```

```
395     </section>
396   );
397 }
```

FILE: apps/dashboard/src/app/Reports/components/ReportHistoryTable.js

```

1  "use client";
2
3  import { useMemo } from "react";
4
5  function DownloadIcon() {
6    return (
7      <svg viewBox="0 0 24 24" className="h-4 w-4" fill="none" stroke="currentColor" strokeWidth="1.8" strokeLinecap="round" strokeLinejoin="round" aria-hidden>
8        <path d="M12 3v12" />
9        <path d="m7 10 5 5 5-5" />
10       <path d="M5 21h14" />
11     </svg>
12   );
13 }
14
15 function EmptyIcon() {
16   return (
17     <svg viewBox="0 0 24 24" className="h-5 w-5" fill="none" stroke="currentColor" strokeWidth="1.7" strokeLinecap="round" strokeLinejoin="round" aria-hidden>
18       <path d="M6 3h9l3 3v15H6z" />
19       <path d="M14 3v4h4M9 12h6M9 16h4" />
20     </svg>
21   );
22 }
23
24 function formatReportName(value) {
25   const normalized = String(value || "Report")
26     .replace(/[_-]+/g, " ")
27     .replace(/\\s+/g, " ")
28     .trim();
29
30   return normalized.replace(/\\b\\w/g, (letter) => letter.toUpperCase());
31 }
32
33 function formatDateTime(value) {
34   if (!value) return "-";
35   const date = new Date(value);
36   if (Number.isNaN(date.getTime())) return String(value);
37
38   return date.toLocaleString("en-IN", {
39     day: "2-digit",
40     month: "short",
41     year: "numeric",
42     hour: "2-digit",
43     minute: "2-digit"
44   });
45 }
46
47 function getReportTime(report) {
48   const value = report?.reportCreatedAt ?? report?.reportUpdatedAt;
49   const timestamp = value ? new Date(value).getTime() : 0;
50   return Number.isNaN(timestamp) ? 0 : timestamp;
51 }
52
53 function StatusBadge({ status }) {
54   const normalized = String(status || "unknown").trim().toLowerCase();
55   const completed = normalized === "completed";
56   const failed = normalized === "failed" || normalized === "error";
57   const className = completed
58     ? "bg-[#e5f7ef] text-[#177657]"
59     : failed
60     ? "bg-[#fff0ec] text-[--orange]"
61     : "bg-[#eeeaf0] text-[#4935b8]";
62
63   return (
64     <span className={`font-vcr inline-flex rounded-[6px] px-2 py-1 text-[8.5px] uppercase tracking-[0.1em]`} ${className}>
65       {normalized || "unknown"}
66     </span>

```

```

67   );
68 }
69
70 export default function ReportHistoryTable({ reports, loading, error }) {
71   const orderedReports = useMemo(
72     () => [...reports].sort((left, right) => getReportTime(right) - getReportTime(left)),
73     [reports]
74   );
75
76   return (
77     <section className="overflow-hidden rounded-[15px] border border-(--line) border-l-[3px] border-l-(--
orange) bg-(--surface) shadow-(--shadow)">
78       <div className="flex flex-wrap items-center justify-between gap-3 border-b border-(--line2) px-4 py
-3.5">
79         <div className="flex items-center gap-2.5">
80           <span className="font-vcr grid h-9 min-w-9 place-items-center rounded-[10px] bg-[linear-gradien
t(140deg,#e04420,#8b5cf6)] px-2 text-[11px] text-white">
81             {String(orderedReports.length).padStart(2, "0")}
82           </span>
83           <div>
84             <h2 className="font-chillax text-[18px] font-semibold text-(--text)">Report History</h2>
85             <p className="mt-0.5 text-[11.5px] text-(--faint)">Latest requests appear first.</p>
86           </div>
87         </div>
88       </div>
89
90       {error ? (
91         <div className="border-b border-(--line2) bg-[rgba(224,68,32,0.05)] px-4 py-3 text-[12px] font-me
dium text-(--orange)">
92           {error}
93         </div>
94       ) : null}
95
96       <div className="overflow-x-auto">
97         <table className="w-full min-w-[940px] border-collapse text-left">
98           <thead className="bg-(--surface2)">
99             <tr className="border-b border-(--line2)">
100               {["ID", "Report", "Requested By", "Created", "Updated", "Status", "File"].map((heading) =>
101                 <th key={heading} className="font-vcr px-4 py-3 text-[9px] font-normal uppercase tracking
-[0.14em] text-(--faint)">
102                   {heading}
103                 </th>
104               )}
105             </tr>
106           </thead>
107           <tbody>
108             {loading ? (
109               Array.from({ length: 3 }, (_, index) => (
110                 <tr key={index} className="border-b border-(--line2) last:border-b-0">
111                   {Array.from({ length: 7 }, (_, cellIndex) => (
112                     <td key={cellIndex} className="px-4 py-3.5">
113                       <span className="block h-3 animate-pulse rounded-[4px] bg-(--line2)" />
114                     </td>
115                   ))}
116                 </tr>
117               ))
118             ) : orderedReports.length ? (
119               orderedReports.map((report, index) => (
120                 <tr key={report?.reportId ?? `${report?.reportName}-${index}`} className="border-b border
-(--line2) transition last:border-b-0 hover:bg-(--surface2)">
121                   <td className="px-4 py-3 text-[12px] font-semibold text-(--orange)">
122                     #{report?.reportId ?? index + 1}
123                   </td>
124                   <td className="px-4 py-3">
125                     <span className="block max-w-[230px] truncate text-[12.5px] font-semibold text-(--tex
t)" title={report?.reportName || "Report"}>
126                       {formatReportName(report?.reportName)}
127                     </span>
128                   </td>

```



```

129         <td className="px-4 py-3">
130             <span className="block max-w-[220px] truncate text-[12px] font-medium text-(--text)">
131                 {report?.opName || ""}</span>
132             <span className="mt-0.5 block max-w-[220px] truncate text-[10.5px] text-(--faint)">{r
133                 eport?.opEmail || ""}</span>
134             </td>
135             <td className="whitespace-nowrap px-4 py-3 text-[11.5px] text-(--muted)">{formatDateTim
136                 e(report?.reportCreatedAt)}</td>
137             <td className="whitespace-nowrap px-4 py-3 text-[11.5px] text-(--muted)">{formatDateTim
138                 e(report?.reportUpdatedAt)}</td>
139             <td className="px-4 py-3"><StatusBadge status={report?.reportStatus} /></td>
140             <td className="px-4 py-3">
141                 {report?.reportLink ? (
142                     <a
143                         href={report.reportLink}
144                         target="_blank"
145                         rel="noreferrer"
146                         aria-label={`Download ${formatReportName(report?.reportName)}`}
147                         title="Download report"
148                         className="grid h-8 w-8 place-items-center rounded-[8px] bg-(--text) text-(--bg)
149                         transition hover:bg-(--orange)"
150                     >
151                         <DownloadIcon />
152                     </a>
153                 ) : (
154                     <button
155                         type="button"
156                         disabled
157                         aria-label="Report file unavailable"
158                         title="Report file unavailable"
159                         className="grid h-8 w-8 cursor-not-allowed place-items-center rounded-[8px] borde
160                             r border-(--line) text-(--faint) opacity-55"
161                     >
162                         <DownloadIcon />
163                     </button>
164                 )}
165             </td>
166         </tr>
167     </tbody>
168 ) : (
169     <tr>
170         <td colspan={7} className="px-4 py-12 text-center">
171             <span className="mx-auto grid h-10 w-10 place-items-center rounded-[10px] border border
172                 -dashed border-(--line) text-(--faint)">
173                 <EmptyIcon />
174             </span>
175             <span className="mt-3 block text-[12.5px] font-medium text-(--muted)">No reports found
176             for this event.</span>
177         </td>
178     </tr>
179 )}
180 </tbody>
181 </table>
182 </div>
183 </section>
184 );
185 }

```

FILE: apps/dashboard/src/app/Reports/components/ReportsContent.js

```

1  "use client";
2
3  import { useEffect, useState } from "react";
4  import { fetchEventDetails, fetchReportsList, fetchVendors } from "../../lib/dashboardApi";
5  import { useDashboardStore } from "../../store/dashboardStore";
6  import ReportFilters from "../ReportFilters";
7  import ReportHistoryTable from "../ReportHistoryTable";
8
9  function getReports(response) {
10   const candidates = [
11     response?.reports,
12     response?.data?.reports,
13     response?.data,
14     response?.list
15   ];
16
17   return candidates.find(Array.isArray) ?? [];
18 }
19
20 export default function ReportsContent() {
21   const token = useDashboardStore((state) => state.token);
22   const eventMeta = useDashboardStore((state) => state.eventMeta);
23   const eventDetails = useDashboardStore((state) => state.eventDetails);
24   const setEventDetails = useDashboardStore((state) => state.setEventDetails);
25   const setVendorsForEvent = useDashboardStore((state) => state.setVendorsForEvent);
26   const eventId = eventMeta?.eventId ?? eventDetails?.id;
27   const cachedVendors = useDashboardStore((state) =>
28     eventId ? state.vendorsByEventId?.[eventId] : undefined
29   );
30   const detailsMatchEvent =
31     eventId !== null && eventId !== "" && String(eventDetails?.id ?? "") === String(eventId);
32   const eventDays = detailsMatchEvent ? eventDetails?.days : null;
33   const [reports, setReports] = useState([]);
34   const [loading, setLoading] = useState(false);
35   const [error, setError] = useState("");
36   const [daysLoading, setDaysLoading] = useState(false);
37   const [vendors, setVendors] = useState(() => cachedVendors || []);
38   const [vendorsLoading, setVendorsLoading] = useState(false);
39
40   const refreshReports = () => {
41     if (eventId === "" || eventId == null) return;
42
43     setLoading(true);
44     setError("");
45     fetchReportsList({ eventId, token, dedupe: false })
46       .then((response) => setReports(getReports(response)))
47       .catch((requestError) => {
48         console.error("Failed to refresh reports", requestError);
49         setError(requestError?.message || "Unable to refresh reports.");
50       })
51       .finally(() => setLoading(false));
52   };
53
54   useEffect(() => {
55     if (eventId === "" || eventId == null) {
56       setReports([]);
57       setLoading(false);
58       setError("Select an event to load its reports.");
59       return undefined;
60     }
61
62     let cancelled = false;
63     setLoading(true);
64     setError("");
65
66     fetchReportsList({ eventId, token })
67       .then((response) => {
68         if (!cancelled) setReports(getReports(response));
69       })

```

```
70     .catch((requestError) => {
71       console.error("Failed to load reports", requestError);
72       if (!cancelled) {
73         setReports([]);
74         setError(requestError?.message || "Unable to load reports.");
75       }
76     })
77     .finally(() => {
78       if (!cancelled) setLoading(false);
79     });
80
81   return () => {
82     cancelled = true;
83   };
84 }, [eventId, token]);
85
86 // Report day options come from the event's own `days` list, so details must be
87 // loaded for the current event before the Days dropdown can be populated.
88 useEffect(() => {
89   if (eventId === "" || eventId == null || detailsMatchEvent) return undefined;
90
91   let cancelled = false;
92   setDaysLoading(true);
93
94   fetchEventDetails({ eventId, token })
95     .then((details) => {
96       if (!cancelled && details) setEventDetails(details);
97     })
98     .catch((requestError) => {
99       console.error("Failed to load event details for report days", requestError);
100     })
101     .finally(() => {
102       if (!cancelled) setDaysLoading(false);
103     });
104
105   return () => {
106     cancelled = true;
107   };
108 }, [detailsMatchEvent, eventId, setEventDetails, token]);
109
110 // Vendor reports need the event's vendor list for the Event / Vendor selector.
111 useEffect(() => {
112   if (eventId === "" || eventId == null) {
113     setVendors([]);
114     return undefined;
115   }
116
117   if (cachedVendors?.length) {
118     setVendors(cachedVendors);
119     return undefined;
120   }
121
122   let cancelled = false;
123   setVendorsLoading(true);
124
125   fetchVendors({ eventId, token })
126     .then((list) => {
127       if (cancelled) return;
128       const normalized = Array.isArray(list) ? list : [];
129       setVendors(normalized);
130       setVendorsForEvent(eventId, normalized);
131     })
132     .catch((requestError) => {
133       console.error("Failed to load vendors for reports", requestError);
134       if (!cancelled) setVendors([]);
135     })
136     .finally(() => {
137       if (!cancelled) setVendorsLoading(false);
138     });
139 }
```

```
140     return () => {
141         cancelled = true;
142     };
143 }, [cachedVendors, eventId, setVendorsForEvent, token]);
144
145 return (
146     <div className="space-y-[clamp(14px,1.8vw,20px)]">
147         <ReportFilters
148             eventId={eventId}
149             token={token}
150             eventDays={eventDays}
151             daysLoading={daysLoading}
152             vendors={vendors}
153             vendorsLoading={vendorsLoading}
154             onReportStarted={refreshReports}
155         />
156         <ReportHistoryTable reports={reports} loading={loading} error={error} />
157     </div>
158 );
159 }
```

FILE: apps/dashboard/src/app/Reports/page.js

```
1 import Header from "../components/Header";
2 import ReportsContent from "../components/ReportsContent";
3
4 export default function ReportsPage() {
5   return (
6     <main
7       className="min-h-screen bg-(--bg) pb-10"
8       style={{ fontFamily: '"Poppins", system-ui, sans-serif' }}
9     >
10      <Header areaLabel="Reports" variant="portal" />
11
12      <div className="mx-auto w-full max-w-[1780px] pr-4 pl-[72px] pt-6 md:pr-7 md:pl-[88px] max-[900px]:
13 px-3">
14        <h1 className="font-chillax text-[clamp(24px,3vw,32px)] font-semibold leading-[1.05] tracking-[-0
15 .02em] text-(--text)">
16          Reports
17        </h1>
18        <p className="mt-2 text-[13.5px] font-light text-(--muted)">
19          Configure filters and download event, vendor, and summary reports.
20        </p>
21        <div className="mt-[clamp(16px,2vw,22px)] h-px w-full bg-(--line)" />
22
23        <div className="mt-[clamp(16px,2vw,22px)]">
24          <ReportsContent />
25        </div>
26      </div>
27    </main>
28  );
29 }
```

FILE: apps/dashboard/src/app/admin/AdminClient.js

```

1  "use client";
2
3  import { useEffect, useMemo, useState } from "react";
4  import { useRouter } from "next/navigation";
5  import Header from "../../components/Header";
6  import { AtomXLoader } from "@atomx/global-components";
7  import { useDashboardStore } from "../../store/dashboardStore";
8  import { fetchEventDetails, fetchEventsList } from "../../lib/dashboardApi";
9
10 const ACCESS_FALLBACK_PATH = "/access";
11 const ADMIN_BACK_GUARD_KEY = "__atomxAdminBackGuard";
12 const SEARCH_PLACEHOLDERS = [
13   "Search By Event",
14   "Name Try Sunburn Arena",
15   "Search by Event ID"
16 ];
17 const EVENT_ACTIONS = [
18   { id: "open", label: "Open dashboard" },
19   { id: "reports", label: "Reports" },
20   // { id: "analytics", label: "Analytics" },
21   { id: "devices", label: "Devices" }
22 ];
23
24 const STATUS_STYLES = {
25   past: {
26     a: "#3a3a38",
27     b: "#1c1c1c",
28     badgeBg: "rgba(255,255,255,0.9)",
29     badgeFg: "#1c1c1c",
30     dot: "#9b9a97",
31     pulse: false,
32     border: "linear-gradient(135deg, rgba(28,28,28,.22), rgba(28,28,28,.07))"
33   },
34   present: {
35     a: "#e04420",
36     b: "#341cd6",
37     badgeBg: "rgba(224,68,32,0.92)",
38     badgeFg: "#fff",
39     dot: "#fff",
40     pulse: true,
41     border: "linear-gradient(135deg, rgba(224,68,32,.65), rgba(52,28,214,.35))"
42   },
43   upcoming: {
44     a: "#341cd6",
45     b: "#00a9f2",
46     badgeBg: "rgba(52,28,214,0.9)",
47     badgeFg: "#fff",
48     dot: "#00a9f2",
49     pulse: false,
50     border: "linear-gradient(135deg, rgba(52,28,214,.5), rgba(0,169,242,.22))"
51   }
52 };
53
54 function parseDate(value) {
55   if (!value) return null;
56   const parsed = new Date(value);
57   return Number.isNaN(parsed.getTime()) ? null : parsed;
58 }
59
60 function classifyEvent(eventItem) {
61   const rawStatus = String(
62     eventItem?.status ??
63     eventItem?.eventStatus ??
64     eventItem?.state ??
65     eventItem?.eventState ??
66     ""
67   ).toLowerCase();
68
69   const startDate = parseDate(

```

```
70     eventItem?.startAt ??
71     eventItem?.startDate ??
72     eventItem?.eventStartDate ??
73     eventItem?.eventStart ??
74     eventItem?.start_date ??
75     eventItem?.start
76 );
77 const endDate = parseDate(
78     eventItem?.endAt ??
79     eventItem?.endDate ??
80     eventItem?.eventEndDate ??
81     eventItem?.eventEnd ??
82     eventItem?.end_date ??
83     eventItem?.end
84 );
85 const now = new Date();
86
87 if (endDate && endDate < now) return "past";
88 if (startDate && startDate > now) return "upcoming";
89 if (startDate && endDate && startDate <= now && endDate >= now) return "present";
90
91 if (/^(past|closed|complete|completed|ended|expired)/.test(rawStatus)) {
92     return "past";
93 }
94 if (/^(upcoming|upcomming|future|scheduled)/.test(rawStatus)) {
95     return "upcoming";
96 }
97 if (/^(present|live|running|ongoing)/.test(rawStatus)) {
98     return "present";
99 }
100
101 return "present";
102 }
103
104 function getEventSearchText(eventItem) {
105     return [
106         eventItem?.id,
107         eventItem?.eventId,
108         eventItem?.name,
109         eventItem?.eventName,
110         eventItem?.title,
111         eventItem?.venue,
112         eventItem?.city,
113         eventItem?.locationCity
114     ]
115         .filter(Boolean)
116         .join(" ")
117         .toLowerCase();
118 }
119
120 function getEventId(eventItem) {
121     return eventItem?.id ?? eventItem?.eventId ?? "";
122 }
123
124 function getEventName(eventItem) {
125     return eventItem?.name ?? eventItem?.eventName ?? eventItem?.title ?? "Untitled Event";
126 }
127
128 function getEventCity(eventItem) {
129     return eventItem?.locationCity ?? eventItem?.city ?? "-";
130 }
131
132 function getEventVenue(eventItem) {
133     return eventItem?.venue ?? "-";
134 }
135
136 function getEventCountry(eventItem) {
137     const country = eventItem?.country ?? "India";
138     return String(country)
139         .split(" ")
```

```

140     .filter(Boolean)
141     .map((word) => `${word.slice(0, 1).toUpperCase()}${word.slice(1).toLowerCase()}`)
142     .join(" ");
143 }
144
145 function getEventDate(eventItem) {
146     const eventDate = parseDate(
147         eventItem?.startAt ??
148         eventItem?.startDate ??
149         eventItem?.eventStartDate ??
150         eventItem?.eventStart ??
151         eventItem?.start_date ??
152         eventItem?.start ??
153         eventItem?.createdAt
154     );
155
156     if (!eventDate) return "-";
157     return new Intl.DateTimeFormat("en-GB", {
158         day: "2-digit",
159         month: "short",
160         year: "numeric"
161     }).format(eventDate);
162 }
163
164 function posterBackground(status, id) {
165     const style = STATUS_STYLES[status] ?? STATUS_STYLES.present;
166     const seed = Number(String(id).replace(/\D/g, "")) || 0;
167     const rot = 110 + (seed % 7) * 14;
168     return `linear-gradient(${rot}deg, ${style.a}, ${style.b})`;
169 }
170
171 function EventActionIcon({ type }) {
172     if (type === "reports") {
173         return (
174             <svg viewBox="0 0 16 16" className="h-[15px] w-[15px]" fill="none" stroke="currentColor" strokeWidth
h="1.35" strokeLinecap="round" strokeLinejoin="round" aria-hidden>
175                 <path d="M4 1.9h4.9L12.1 5.1v9H4z" />
176                 <path d="M8.9 1.9v3.2h3.2" />
177                 <path d="M6.1 8.7h4M6.1 11.1h2.9" />
178             </svg>
179         );
180     }
181     if (type === "analytics") {
182         return (
183             <svg viewBox="0 0 16 16" className="h-[15px] w-[15px]" fill="none" stroke="currentColor" strokeWidth
h="1.4" strokeLinecap="round" aria-hidden>
184                 <path d="M2.3 13.4h11.4" />
185                 <path d="M4.6 13.4V7.2M8 13.4V3.4M11.4 13.4V9.4" />
186             </svg>
187         );
188     }
189     if (type === "devices") {
190         return (
191             <svg viewBox="0 0 16 16" className="h-[15px] w-[15px]" fill="none" stroke="currentColor" strokeWidth
h="1.35" strokeLinecap="round" strokeLinejoin="round" aria-hidden>
192                 <rect x="4.4" y="1.9" width="7.2" height="12.2" rx="1.4" />
193                 <path d="M6.6 4.4h2.8M6.8 11.7h2.4" />
194             </svg>
195         );
196     }
197     return (
198         <svg viewBox="0 0 16 16" className="h-[15px] w-[15px]" fill="currentColor" aria-hidden>
199             <rect x="2" y="2" width="5" height="6.6" rx="1.2" />
200             <rect x="9" y="2" width="5" height="4.2" rx="1.2" opacity=".55" />
201             <rect x="2" y="10.2" width="5" height="3.8" rx="1.2" opacity=".55" />
202             <rect x="9" y="7.8" width="5" height="6.2" rx="1.2" />
203         </svg>
204     );
205 }
206

```



```

207 function EventPoster({ eventItem, status }) {
208   const style = STATUS_STYLES[status] ?? STATUS_STYLES.present;
209   const id = String(getEventId(eventItem));
210   const words = getEventName(eventItem)
211     .toUpperCase()
212     .split(/[ \-]+/)
213     .filter(Boolean)
214     .slice(0, 3);
215
216   return (
217     <div
218       className="relative overflow-hidden border-b border-(--line2)"
219       style={{ aspectRatio: "16 / 9", background: posterBackground(status, id) }}
220     >
221       <svg className="pointer-events-none absolute -right-6 -top-8 h-[150%] w-auto opacity-70" viewBox="0
0 300 300" fill="none" aria-hidden>
222         {[0, 1, 2].map((ring) => (
223           <circle
224             key={ring}
225             cx="230"
226             cy="70"
227             r={60 + ring * 42}
228             stroke="#fff"
229             strokeOpacity={0.16 - ring * 0.04}
230             strokeWidth="1.5"
231           />
232         ))}
233       </svg>
234
235       <div className="relative flex h-full flex-col justify-between p-3">
236         <div className="flex items-start justify-between">
237           <span
238             className="inline-flex items-center gap-1.5 rounded-full px-2.5 py-1.5 backdrop-blur"
239             style={{ background: style.badgeBg, color: style.badgeFg }}
240           >
241             <span
242               className="h-1.5 w-1.5 rounded-full"
243               style={{ background: style.dot, animation: style.pulse ? "atxLive 1.4s ease-in-out infinite
" : undefined }}
244             />
245             <span className="font-vcr text-[9px] tracking-[0.14em]">{status.toUpperCase()}</span>
246           </span>
247           <span className="font-vcr rounded-[7px] bg-[rgba(28,28,28,0.62)] px-2 py-1.5 text-[9px] trackin
g-[0.12em] text-[#ebebcb] backdrop-blur">
248             #{id || "-"}
249           </span>
250         </div>
251
252         <div>
253           <span className="mb-2 block h-1 w-10 rounded-full bg-white/85" />
254           {words.map((word, index) => (
255             <div
256               key={` ${word}-${index} `}
257               className="text-[22px] font-bold uppercase leading-[1.02] tracking-[-0.01em] text-white"
258               style={{ opacity: 1 - index * 0.22 }}
259             >
260               {word}
261             </div>
262           ))}
263           <div className="font-vcr mt-2 text-[9px] tracking-[0.24em] text-white/60">
264             ATOMX · {String(getEventCity(eventItem)).toUpperCase()}
265           </div>
266         </div>
267       </div>
268     </div>
269   );
270 }
271
272 export default function AdminClient() {
273   const router = useRouter();

```

```

274 const token = useDashboardStore((state) => state.token);
275 const setEventMeta = useDashboardStore((state) => state.setEventMeta);
276 const setEventDetails = useDashboardStore((state) => state.setEventDetails);
277 const [events, setEvents] = useState([]);
278 const [openingEventId, setOpeningEventId] = useState("");
279 const [loading, setLoading] = useState(false);
280 const [submitting, setSubmitting] = useState(false);
281 const [error, setError] = useState("");
282 const [showExitConfirm, setShowExitConfirm] = useState(false);
283 const [activeTab, setActiveTab] = useState("all");
284 const [searchQuery, setSearchQuery] = useState("");
285 const [searchPlaceholder, setSearchPlaceholder] = useState("");
286
287 useEffect(() => {
288   if (typeof window === "undefined") return;
289   const pushGuardState = () => {
290     window.history.pushState({ [ADMIN_BACK_GUARD_KEY]: true }, "", window.location.href);
291   };
292
293   pushGuardState();
294   const onPopState = (event) => {
295     if (event.state?.[ADMIN_BACK_GUARD_KEY]) return;
296     setShowExitConfirm(true);
297     pushGuardState();
298   };
299
300   window.addEventListener("popstate", onPopState);
301   return () => {
302     window.removeEventListener("popstate", onPopState);
303   };
304 }, []);
305
306 const handleBackConfirm = () => {
307   if (typeof window === "undefined") return;
308   try {
309     window.localStorage.removeItem("atomx.portal.token");
310     window.localStorage.removeItem("atomx.dashboard.token");
311     window.localStorage.removeItem("atomx.portal.reauth");
312     const keysToRemove = [];
313     for (let i = 0; i < window.localStorage.length; i += 1) {
314       const key = window.localStorage.key(i);
315       if (key && key.startsWith("atomx.auth.")) {
316         keysToRemove.push(key);
317       }
318     }
319     keysToRemove.forEach((key) => window.localStorage.removeItem(key));
320     if (window.sessionStorage) {
321       window.sessionStorage.clear();
322     }
323   } catch (err) {
324     console.error("Failed to clear session on exit", err);
325   }
326   const accessBase = process.env.NEXT_PUBLIC_ACCESS_PORTAL_URL || "/";
327   const target = new URL(accessBase, window.location.origin);
328   if (target.pathname === ACCESS_FALLBACK_PATH) {
329     target.pathname = "/";
330   }
331   window.location.assign(target.toString());
332 };
333
334 useEffect(() => {
335   let active = true;
336   const loadEvents = async () => {
337     setLoading(true);
338     setError("");
339     try {
340       const list = await fetchEventsList({ token });
341       if (!active) return;
342       setEvents(Array.isArray(list) ? list : []);
343     } catch (err) {

```

```
344     console.error("Failed to load events", err);
345     if (active) {
346         setError("Unable to load events.");
347         setEvents([]);
348     }
349     } finally {
350         if (active) setLoading(false);
351     }
352 };
353 loadEvents();
354 return () => {
355     active = false;
356 };
357 }, [token]);
358
359 useEffect(() => {
360     let phraseIndex = 0;
361     let charIndex = 0;
362     let deleting = false;
363     let pauseTicks = 0;
364
365     const timer = setInterval(() => {
366         const phrase = SEARCH_PLACEHOLDERS[phraseIndex];
367
368         if (pauseTicks > 0) {
369             pauseTicks -= 1;
370             return;
371         }
372
373         if (!deleting) {
374             charIndex += 1;
375             setSearchPlaceholder(phrase.slice(0, charIndex));
376             if (charIndex >= phrase.length) {
377                 deleting = true;
378                 pauseTicks = 10;
379             }
380             return;
381         }
382
383         charIndex -= 1;
384         setSearchPlaceholder(phrase.slice(0, Math.max(charIndex, 0)));
385         if (charIndex <= 0) {
386             deleting = false;
387             phraseIndex = (phraseIndex + 1) % SEARCH_PLACEHOLDERS.length;
388             pauseTicks = 3;
389         }
390     }, 90);
391
392     return () => clearInterval(timer);
393 }, []);
394
395 const tabCounts = useMemo(() => {
396     const counts = {
397         all: events.length,
398         past: 0,
399         present: 0,
400         upcoming: 0
401     };
402
403     events.forEach((eventItem) => {
404         const bucket = classifyEvent(eventItem);
405         counts[bucket] += 1;
406     });
407
408     return counts;
409 }, [events]);
410
411 const visibleEvents = useMemo(() => {
412     const query = searchQuery.trim().toLowerCase();
413     return events.filter((eventItem) => {
```

```

414     const bucket = classifyEvent(eventItem);
415     const matchesTab = activeTab === "all" || bucket === activeTab;
416     const matchesSearch = !query || getEventSearchText(eventItem).includes(query);
417     return matchesTab && matchesSearch;
418   });
419 }, [activeTab, events, searchQuery]);
420
421 const handleOpenEvent = async (eventId) => {
422   if (!eventId) {
423     setError("Event details are missing.");
424     return;
425   }
426   const selectedEventId = String(eventId);
427   setSubmitting(true);
428   setOpeningEventId(selectedEventId);
429   setError("");
430   try {
431     const details = await fetchEventDetails({
432       eventId: selectedEventId,
433       token
434     });
435     if (details) {
436       setEventDetails(details);
437       setEventMeta({
438         eventId: details.id ?? selectedEventId,
439         eventName: details.name ?? details.eventName ?? "",
440         venue: details.venue ?? "",
441         city: details.locationCity ?? ""
442       });
443     }
444     const params = new URLSearchParams();
445     if (details?.id ?? selectedEventId) params.set("eventId", details?.id ?? selectedEventId);
446     if (details?.name) params.set("eventName", details.name);
447     if (details?.venue) params.set("venue", details.venue);
448     if (details?.locationCity) params.set("city", details.locationCity);
449     router.push(`/Config?${params.toString()}`);
450   } catch (err) {
451     console.error("Failed to load event details", err);
452     setError("Unable to load event details.");
453   } finally {
454     setSubmitting(false);
455     setOpeningEventId("");
456   }
457 };
458
459 const tabs = [
460   { id: "all", label: "All", count: tabCounts.all },
461   { id: "past", label: "Past", count: tabCounts.past },
462   { id: "present", label: "Present", count: tabCounts.present },
463   { id: "upcoming", label: "Upcoming", count: tabCounts.upcoming }
464 ];
465 const shellClass = "mx-auto w-full max-w-[1680px] px-[clamp(16px,3vw,32px)]";
466
467 return (
468   <main className="min-h-screen bg-(--bg) pb-24">
469     <style>{@keyframes atxLive{0%,100%{opacity:1;transform:scale(1)}50%{opacity:.3;transform:scale(.7)}}@keyframes atxCardIn{from{opacity:0;transform:translateY(8px)}to{opacity:1;transform:none}}`}</style>
470     <Header areaLabel="Event List" variant="portal" hideNav />
471
472     <div className="sticky top-[58px] z-30 border-b border-(--line) bg-(--bg) pt-3">
473       <div className={ `${shellClass} flex flex-wrap items-center gap-4`}>
474         <div className="flex min-w-0 flex-1 gap-1 overflow-x-auto pb-0.5 [scrollbar-width:none] [&::-webkit-scrollbar]:hidden">
475           {tabs.map((tab) => {
476             const active = activeTab === tab.id;
477             return (
478               <button
479                 key={tab.id}
480                 type="button"
481                 onClick={() => setActiveTab(tab.id)}

```

```

482         className={`flex shrink-0 items-center gap-2 border-b-2 px-3.5 pb-2.5 pt-2 text-[13px]
transition ${
483             active
484             ? "border-(--orange) font-semibold text-(--text)"
485             : "border-transparent font-normal text-(--muted) hover:text-(--text)"
486         }`}
487     >
488     <span>{tab.label}</span>
489     <span
490         className={`font-vcr rounded-[6px] px-1.5 py-0.5 text-[9.5px] tracking-[0.06em] ${
491             active ? "bg-(--text) text-(--bg)" : "bg-(--chip) text-(--faint)"
492         }`}
493     >
494         {String(tab.count).padStart(2, "0")}
495     </span>
496 </button>
497 );
498 }}
499 </div>
500
501 <label className="mb-3 flex h-[38px] min-w-[210px] items-center gap-2 rounded-[11px] border bor
der-(--line) bg-(--surface) px-3 text-(--muted) focus-within:border-(--orange)">
502     <svg viewBox="0 0 16 16" className="h-3.5 w-3.5 shrink-0 opacity-50" fill="none" stroke="curr
entColor" strokeWidth="1.5" strokeLinecap="round" aria-hidden>
503         <circle cx="7" cy="7" r="4.6" />
504         <path d="M10.5 10.5 14 14" />
505     </svg>
506     <input
507         value={searchQuery}
508         onChange={(event) => setSearchQuery(event.target.value)}
509         className="h-full min-w-0 flex-1 bg-transparent text-[12.5px] text-(--text) outline-none pl
aceholder:text-(--faint)"
510         placeholder={searchPlaceholder || SEARCH_PLACEHOLDERS[0]}
511     />
512 </label>
513 </div>
514 </div>
515
516 <div className={` ${shellClass} pt-[clamp(18px,2.5vw,26px)]`}>
517     {loading ? (
518         <div className="rounded-[15px] border border-(--line) bg-(--surface) py-12 shadow-(--shadow)">
519             <AtomXLoader label="Loading events..." size={56} />
520         </div>
521     ) : null}
522
523     {error ? (
524         <div className="mb-4 rounded-[10px] border border-[rgba(224,68,32,0.25)] bg-[rgba(224,68,32,0.0
6)] px-4 py-3 text-sm font-medium text-(--orange)">
525             {error}
526         </div>
527     ) : null}
528
529     {!loading && visibleEvents.length === 0 ? (
530         <div className="rounded-[16px] border border-dashed border-(--line) px-6 py-14 text-center">
531             <div className="font-chillax text-[19px] font-medium text-(--text)">No events match</div>
532             <div className="mt-1.5 text-[12.5px] text-(--faint)">Try a different tab or clear the search.
</div>
533         </div>
534     ) : null}
535
536     {!loading && visibleEvents.length > 0 ? (
537         <section
538             className="grid gap-[clamp(12px,1.1vw,16px)]"
539             style={{ gridTemplateColumns: "repeat(auto-fill,minmax(min(100%,250px),1fr))" }}
540         >
541             {visibleEvents.map((eventItem, index) => {
542                 const eventId = String(getEventId(eventItem));
543                 const bucket = classifyEvent(eventItem);
544                 const isOpening = openingEventId === eventId;
545                 const meta = [

```

```

546      { label: "CITY", value: getEventCity(eventItem) },
547      { label: "VENUE", value: getEventVenue(eventItem) },
548      { label: "COUNTRY", value: getEventCountry(eventItem) },
549      { label: "DATE", value: getEventDate(eventItem) }
550    ];
551    const theme = STATUS_STYLES[bucket] ?? STATUS_STYLES.present;
552
553    return (
554      <article
555        key={eventId || getEventName(eventItem)}
556        className="flex flex-col overflow-hidden rounded-[15px] border border-transparent shado
w(--shadow) transition-shadow duration-200 hover:shadow-(--shadowUp)"
557        style={{
558          background: `linear-gradient(var(--surface),var(--surface)) padding-box, ${theme.bord
er} border-box`,
559          animation: "atxCardIn 0.35s both",
560          animationDelay: `${index * 0.035}s`
561        }}
562      >
563        <EventPoster eventItem={eventItem} status={bucket} />
564
565        <div className="px-[14px] pt-[14px]">
566          <div className="font-chillax truncate text-[15.5px] font-medium tracking-[-0.01em]">
567            {getEventName(eventItem)}
568          </div>
569        </div>
570
571        <div className="mx-[14px] mt-2.5 grid grid-cols-2 gap-px overflow-hidden rounded-[9px]
border border-(--line2) bg-(--line2)">
572          {meta.map((item) => (
573            <div key={item.label} className="min-w-0 bg-(--surface2) px-[9px] py-[7px]">
574              <div className="font-vcr text-[7.5px] tracking-[0.15em] text-(--faint)">{item.lab
el}</div>
575              <div className="mt-0.5 truncate text-[12.5px] font-medium">{item.value}</div>
576            </div>
577          ))}
578        </div>
579
580        <div className="mt-auto grid grid-cols-4 gap-1.5 px-[14px] pb-[14px] pt-[11px]">
581          {EVENT_ACTIONS.map((action) => (
582            <button
583              key={action.id}
584              type="button"
585              aria-label={action.label}
586              title={action.label}
587              onClick={() => handleOpenEvent(eventId)}
588              disabled={submitting || !eventId || action.id === "analytics"}
589              className="flex h-8 items-center justify-center rounded-[9px] border border-(--li
ne) bg-(--surface) text-(--muted) transition hover:border-(--orange) hover:bg-(--chip) hover:text-(--oran
ge) disabled:cursor-not-allowed disabled:opacity-55"
590            >
591              {isOpening && action.id === "open" ? (
592                <span className="h-4 w-4 animate-spin rounded-full border-2 border-(--orange) b
order-t-transparent" />
593              ) : (
594                <EventActionIcon type={action.id} />
595              )}
596            </button>
597          ))}
598        </div>
599      </article>
600    );
601  }})
602  </section>
603  ) : null}
604 </div>
605
606  {showExitConfirm ? (
607    <div className="fixed inset-0 z-[70] flex items-center justify-center bg-[rgba(12,12,12,0.5)] p-4
backdrop-blur-sm">

```

```

608         <div className="w-full max-w-md rounded-2xl border border-(--line) bg-(--surface) p-5 text-(--t
ext) shadow-(--shadowUp)">
609             <h3 className="font-chillax text-lg font-semibold">Take Exit?</h3>
610             <p className="mt-2 text-sm text-(--muted)">Do you really want to take exit?</p>
611             <div className="mt-5 flex justify-end gap-2">
612                 <button
613                     type="button"
614                     onClick={() => setShowExitConfirm(false)}
615                     className="rounded-[10px] border border-(--line) px-4 py-2 text-sm font-semibold text-(--
muted) transition hover:border-(--orange) hover:text-(--orange)"
616                 >
617                     No
618                 </button>
619                 <button
620                     type="button"
621                     onClick={handleBackConfirm}
622                     className="rounded-[10px] bg-(--text) px-4 py-2 text-sm font-semibold text-(--bg) transit
ion hover:bg-(--orange)"
623                 >
624                     Yes
625                 </button>
626             </div>
627         </div>
628     </div>
629     ) : null}
630 </main>
631 );
632 }

```

FILE: apps/dashboard/src/app/admin/AdminClientNoSSR.js

```
1 "use client";
2
3 import dynamic from "next/dynamic";
4 import { AtomXLoader } from "@atomx/global-components";
5
6 const AdminClient = dynamic(() => import("./AdminClient"), {
7   ssr: false,
8   loading: () => <AtomXLoader fullScreen label="Loading admin..." className="bg-[color:rgb(var(--color-bg
9   ))]" />
10 });
11
12 export default function AdminClientNoSSR() {
13   return <AdminClient />;
14 }
```


FILE: apps/dashboard/src/app/admin/Create_event/components/AdminRoleManager.js

```
1 "use client";
2
3 import { useCallback, useEffect, useMemo, useState } from "react";
4 import {
5   fetchOperatorsList,
6   linkAdmin,
7   linkOperator
8 } from "../../../lib/dashboardApi";
9 import { useDashboardStore } from "../../../store/dashboardStore";
10
11 const TABS = [
12   { id: "admin", label: "Admins" },
13   { id: "operator", label: "Operators" }
14 ];
15
16 const OPERATOR_TYPES = [
17   { value: "event", label: "Event" },
18   { value: "cashless", label: "Cashless" },
19   { value: "access", label: "Access" },
20   { value: "inventory", label: "Inventory" },
21   { value: "guest-ops", label: "Guest Ops" },
22   { value: "finance", label: "Finance" }
23 ];
24
25 function nowParts() {
26   const now = new Date();
27   return {
28     timestampMs: now.getTime(),
29     whenAdded: "Just now",
30     date: now.toLocaleDateString("en-IN", {
31       day: "2-digit",
32       month: "short",
33       year: "numeric"
34     }),
35     time: now.toLocaleTimeString("en-IN", {
36       hour: "2-digit",
37       minute: "2-digit"
38     })
39   };
40 }
41
42 function getTypeLabel(value) {
43   const normalized = String(value || "").trim().toLowerCase();
44   return (
45     OPERATOR_TYPES.find((option) => option.value === normalized)?.label ||
46     (normalized ? normalized.charAt(0).toUpperCase() + normalized.slice(1) : "-")
47   );
48 }
49
50 function idsMatch(value, expected) {
51   if (value == null || expected == null || String(expected).trim() === "") {
52     return false;
53   }
54
55   return String(value) === String(expected);
56 }
57
58 function formatOperatorRecord(record, index) {
59   const timestamp =
60     record?.createdAt ??
61     record?.created_at ??
62     record?.linkedAt ??
63     record?.linked_at ??
64     record?.updatedAt ??
65     record?.updated_at;
66   const parsedDate = timestamp ? new Date(timestamp) : null;
67   const hasValidDate = parsedDate && !Number.isNaN(parsedDate.getTime());
68
69   return {
```

```

70     id: record?.id ?? record?.operatorId ?? record?.operator_id ?? index,
71     email:
72       record?.email ??
73       record?.operatorEmail ??
74       record?.operator_email ??
75       record?.user?.email ??
76       record?.operator?.email ??
77     "_",
78     typeLabel: getTypeLabel(
79       record?.type ?? record?.operatorType ?? record?.operator_type ?? record?.role
80     ),
81     timestampMs: isValidDate ? parsedDate.getTime() : 0,
82     whenAdded: isValidDate ? "Recorded" : "-",
83     date: isValidDate
84       ? parsedDate.toLocaleDateString("en-IN", {
85         day: "2-digit",
86         month: "short",
87         year: "numeric"
88       })
89       : "-",
90     time: isValidDate
91       ? parsedDate.toLocaleTimeString("en-IN", {
92         hour: "2-digit",
93         minute: "2-digit"
94       })
95       : "-"
96   };
97 }
98
99 function groupOperatorRecords(records) {
100   const groups = new Map();
101
102   records.forEach((record) => {
103     const normalizedEmail = String(record?.email || "").trim().toLowerCase();
104     const key = normalizedEmail && normalizedEmail !== "-"
105       ? normalizedEmail
106       : `record-${record?.id}`;
107     const existing = groups.get(key) || [];
108     existing.push(record);
109     groups.set(key, existing);
110   });
111
112   return Array.from(groups.entries())
113     .map(([key, history]) => {
114       const sortedHistory = [...history].sort(
115         (left, right) => (right.timestampMs || 0) - (left.timestampMs || 0)
116       );
117       const latest = sortedHistory[0];
118
119       return {
120         ...latest,
121         id: `email-${key}`,
122         whenAdded: "Latest",
123         history: sortedHistory,
124         historyCount: sortedHistory.length
125       };
126     })
127     .sort((left, right) => (right.timestampMs || 0) - (left.timestampMs || 0));
128 }
129
130 function getInitialAdminId(profile, eventMeta, eventDetails) {
131   return (
132     profile?.ctx?.adminId ??
133     eventMeta?.adminId ??
134     eventDetails?.adminId ??
135     eventDetails?.admin?.id ??
136     eventDetails?.admin_id ??
137     eventDetails?.admin?.adminId ??
138     ""
139   );

```

```

140 }
141
142 function getInitialEventId(eventMeta, eventDetails) {
143   return eventMeta?.eventId ?? eventDetails?.id ?? "";
144 }
145
146 function RoleTabs({ activeTab, onChange }) {
147   return (
148     <div className="inline-flex rounded-[11px] border border-(--line) bg-(--surface) p-1">
149       {TABS.map((tab) => {
150         const active = activeTab === tab.id;
151         return (
152           <button
153             key={tab.id}
154             type="button"
155             onClick={() => onChange(tab.id)}
156             className={`h-9 rounded-[8px] px-4 text-[12.5px] font-semibold transition ${
157               active ? "bg-(--text) text-(--bg)" : "text-(--muted) hover:text-(--orange)"
158             }`}
159           >
160             {tab.label}
161           </button>
162         );
163       })}
164     </div>
165   );
166 }
167
168 function Field({ label, children }) {
169   return (
170     <label className="block">
171       <span className="font-vcr mb-1.5 block text-[8.5px] uppercase tracking-[0.16em] text-(--muted)">
172         {label}
173       </span>
174       {children}
175     </label>
176   );
177 }
178
179 function TextInput(props) {
180   return (
181     <input
182       {...props}
183       className="h-11 w-full rounded-[10px] border border-(--line) bg-(--surface) px-3.5 text-[13px] font-
184       medium text-(--text) outline-none transition placeholder:text-(--faint) focus:border-(--orange) focus:sh
185       adow-[0_0_0_3px_rgba(224,68,32,0.12)]"
186     />
187   );
188 }
189
190 function TypeSelect({ value, onChange }) {
191   return (
192     <div className="relative">
193       <select
194         value={value}
195         onChange={(event) => onChange(event.target.value)}
196         className="h-11 w-full cursor-pointer appearance-none rounded-[10px] border border-(--line) bg-(
197         --surface) px-3.5 pr-9 text-[13px] font-medium text-(--text) outline-none transition focus:border-(--orang
198         e) focus:shadow-[0_0_0_3px_rgba(224,68,32,0.12)]"
199       >
200         <option value="" disabled>
201           Select operator type
202         </option>
203         {OPERATOR_TYPES.map((option) => (
204           <option key={option.value} value={option.value}>
205             {option.label}
206           </option>
207         ))}
208       </select>
209     </div>
210   );
211 }

```

```

206         viewBox="0 0 16 16"
207         className="pointer-events-none absolute right-3 top-1/2 h-4 w-4 -translate-y-1/2 text-(--muted) opacity-60"
208         fill="none"
209         stroke="currentColor"
210         strokeWidth="1.5"
211         strokeLinecap="round"
212         strokeLinejoin="round"
213         aria-hidden
214     >
215     <path d="m4 6 4 4 4-4" />
216 </svg>
217 </div>
218 );
219 }
220
221 function AddedList({
222     title,
223     items,
224     emptyText,
225     showType = false,
226     loading = false,
227     loadError = ""
228 }) {
229     const [expandedItems, setExpandedItems] = useState({});
230
231     const toggleHistory = (id) => {
232         setExpandedItems((current) => ({ ...current, [id]: !current[id] }));
233     };
234
235     return (
236         <section className="rounded-[15px] border border-(--line) border-l-[3px] border-l-(--orange) bg-(--surface) p-4 shadow-(--shadow)">
237             <div className="mb-3 flex items-center gap-2.5">
238                 <span className="font-vcr flex h-9 min-w-9 items-center justify-center rounded-[10px] bg-[linear-gradient(140deg,#e04420,#8b5cf6)] px-2 text-[12px] text-white">
239                     {String(items.length).padStart(2, "0")}
240                 </span>
241                 <h2 className="font-chillax m-0 text-[18px] font-semibold text-(--text)">{title}</h2>
242             </div>
243             <div className="space-y-2">
244                 {loading ? (
245                     <div className="rounded-[11px] border border-dashed border-(--line) px-4 py-8 text-center text-[13px] font-medium text-(--muted)">
246                         Loading records...
247                     </div>
248                 ) : loadError ? (
249                     <div className="rounded-[11px] border border-[rgb(224,68,32,0.25)] bg-[rgb(224,68,32,0.06)] px-4 py-3 text-center text-[13px] font-semibold text-(--orange)">
250                         {loadError}
251                     </div>
252                 ) : items.length === 0 ? (
253                     <div className="rounded-[11px] border border-dashed border-(--line) px-4 py-8 text-center text-[13px] font-medium text-(--muted)">
254                         {emptyText}
255                     </div>
256                 ) : (
257                     items.map((item, index) => {
258                         const canExpand = (item.historyCount || 0) > 1;
259                         const isExpanded = Boolean(expandedItems[item.id]);
260
261                         return (
262                             <div
263                                 key={item.id ?? `${item.email}-${item.time}-${index}`}
264                                 className="overflow-hidden rounded-[11px] border border-transparent p-px"
265                                 style={{
266                                     background:
267                                         "linear-gradient(var(--surface),var(--surface)) padding-box, linear-gradient(135deg,rgba(224,68,32,.34),rgba(139,92,246,.26) 52%,rgba(0,169,242,.22)) border-box"
268                                 }}

```

```

269         >
270         <div
271             className={`grid items-center gap-3 rounded-[10px] bg-(--surface) max-lg:grid-cols-1 ${
272                 showType
273                 ? "grid-cols-[minmax(180px,1fr)_120px_120px_90px_90px_auto]"
274                 : "grid-cols-[minmax(180px,1fr)_120px_90px_90px_auto]"
275             }`}
276         >
277         <div className="contents max-lg:block">
278             <div className="rounded-[10px] bg-(--surface) px-3 py-2">
279                 <div className="font-vcr text-[7.5px] uppercase tracking-[0.15em] text-(--faint)">
280                     Email
281                 </div>
282                 <div className="mt-0.5 truncate text-[13.5px] font-semibold text-(--text)">
283                     {item.email}
284                 </div>
285             </div>
286             {showType ? (
287                 <div className="rounded-[10px] bg-(--surface) px-3 py-2">
288                     <div className="font-vcr text-[7.5px] uppercase tracking-[0.15em] text-(--faint)"
289                         Type
290                     </div>
291                     <div className="mt-0.5 text-[13px] font-semibold text-(--text)">
292                         {item.typeLabel}
293                     </div>
294                 </div>
295             ) : null}
296             <div className="rounded-[10px] bg-(--surface) px-3 py-2">
297                 <div className="font-vcr text-[7.5px] uppercase tracking-[0.15em] text-(--faint)">
298                     When Added
299                 </div>
300                 <div className="mt-0.5 text-[13px] font-semibold text-(--text)">
301                     {item.whenAdded}
302                 </div>
303             </div>
304             <div className="rounded-[10px] bg-(--surface) px-3 py-2">
305                 <div className="font-vcr text-[7.5px] uppercase tracking-[0.15em] text-(--faint)">
306                     Date
307                 </div>
308                 <div className="mt-0.5 text-[13px] font-semibold text-(--text)">
309                     {item.date}
310                 </div>
311             </div>
312             <div className="rounded-[10px] bg-(--surface) px-3 py-2">
313                 <div className="font-vcr text-[7.5px] uppercase tracking-[0.15em] text-(--faint)">
314                     Time
315                 </div>
316                 <div className="mt-0.5 text-[13px] font-semibold text-(--text)">
317                     {item.time}
318                 </div>
319             </div>
320         </div>
321         {canExpand ? (
322             <button
323                 type="button"
324                 onClick={() => toggleHistory(item.id)}
325                 aria-expanded={isExpanded}
326                 title={isExpanded ? "Hide history" : "Show history"}
327                 className={`font-vcr mr-2 flex h-8 items-center gap-1.5 rounded-[8px] border border-
328                     (--line) px-2.5 text-[11px] font-medium text-(--muted) transition hover:border-(--orange) hover:text-(--o
329                     range) max-lg:mb-2 max-lg:ml-3 max-lg:w-fit`}
330             >
331                 <span>{item.historyCount}</span>
332                 <svg
333                     viewBox="0 0 16 16"
334                     className={`h-3.5 w-3.5 transition ${isExpanded ? "rotate-180" : ""}`}
335                     fill="none"
336                     stroke="currentColor"
337                     strokeWidth="1.5"

```

```

336         strokeLinecap="round"
337         strokeLinejoin="round"
338         aria-hidden
339       >
340         <path d="m4 6 4 4 4-4" />
341       </svg>
342     </button>
343   ) : (
344     <span className="font-vcr mr-3 text-right text-[9px] uppercase tracking-[0.1em] text-
345     (---faint) max-lg:hidden">
346       1 record
347     </span>
348   )}
349 </div>
350 {isExpanded ? (
351   <div className="border-t border-(---line2) bg-(---surface2) px-5 py-4">
352     <div className="relative ml-1 pl-6">
353       <span className="absolute bottom-2 left-[5px] top-2 w-px bg-[linear-gradient(#e0442
354       0,#341cd6)]" />
355       <div className="space-y-4">
356         {item.history.map((historyItem, historyIndex) => (
357           <div
358             key={`_${historyItem.id}-${historyIndex}`}
359             className="relative flex flex-wrap items-center justify-between gap-x-5 gap-y
360             -1"
361             >
362               <span
363                 className={`absolute -left-[26px] top-1.5 h-3 w-3 rounded-full border-2 bor
364                 der-white shadow-[0_0_1px_rgba(52,28,214,0.18)] ${
365                   historyIndex === 0
366                     ? "bg-[#e04420]"
367                     : "bg-[#341cd6]"
368                 }`}
369               />
370               <div>
371                 <div className="text-[12.5px] font-semibold text-(---text)">
372                   {historyItem.typeLabel}
373                 </div>
374                 <div className="mt-0.5 text-[11px] font-medium text-(---muted)">
375                   {historyIndex === 0 ? "Latest access link" : "Previous access link"}
376                 </div>
377               </div>
378               <div className="text-right">
379                 <div className="font-vcr text-[12px] text-(---text)">
380                   {historyItem.date}
381                 </div>
382                 <div className="font-vcr mt-0.5 text-[11px] text-(---muted)">
383                   {historyItem.time}
384                 </div>
385               </div>
386             )}
387           </div>
388         </div>
389       ) : null}
390     </div>
391   );
392 })
393 </div>
394 </section>
395 );
396 }
397
398 export default function AdminRoleManager() {
399   const token = useDashboardStore((state) => state.token);
400   const profile = useDashboardStore((state) => state.profile);
401   const eventMeta = useDashboardStore((state) => state.eventMeta);

```

```
402 const eventDetails = useDashboardStore((state) => state.eventDetails);
403 const initialAdminId = useMemo(
404   () => getInitialAdminId(profile, eventMeta, eventDetails),
405   [profile, eventMeta, eventDetails]
406 );
407 const initialEventId = useMemo(
408   () => getInitialEventId(eventMeta, eventDetails),
409   [eventMeta, eventDetails]
410 );
411 const [activeTab, setActiveTab] = useState("admin");
412 const [adminEmail, setAdminEmail] = useState("");
413 const [operatorEmail, setOperatorEmail] = useState("");
414 const [operatorType, setOperatorType] = useState("");
415 const [admins, setAdmins] = useState([]);
416 const [operators, setOperators] = useState([]);
417 const [linksLoading, setLinksLoading] = useState(true);
418 const [linksError, setLinksError] = useState("");
419 const [submitting, setSubmitting] = useState(false);
420 const [error, setError] = useState("");
421
422 const loadOperators = useCallback(
423   async ({ force = false, quiet = false } = {}) => {
424     if (!quiet) {
425       setLinksLoading(true);
426     }
427     setLinksError("");
428
429     try {
430       const list = await fetchOperatorsList({ token, dedupe: !force });
431       const records = Array.isArray(list) ? list : [];
432       const adminRecords = records.filter((record) =>
433         idsMatch(record?.adminId ?? record?.admin_id, initialAdminId)
434       );
435       const operatorRecords = records.filter((record) => {
436         const recordAdminId = record?.adminId ?? record?.admin_id;
437         const matchesEvent = idsMatch(
438           record?.eventId ?? record?.event_id,
439           initialEventId
440         );
441         const matchesAdmin =
442           recordAdminId == null || idsMatch(recordAdminId, initialAdminId);
443
444         return matchesEvent && matchesAdmin;
445       });
446       const normalizedAdmins = groupOperatorRecords(
447         adminRecords.map((record, index) => formatOperatorRecord(record, index))
448       );
449       const normalizedOperators = groupOperatorRecords(
450         operatorRecords.map((record, index) => formatOperatorRecord(record, index))
451       );
452
453       setAdmins(normalizedAdmins);
454       setOperators(normalizedOperators);
455       return { admins: normalizedAdmins, operators: normalizedOperators };
456     } catch (err) {
457       console.error("Operator list failed", err);
458       if (!quiet) {
459         setLinksError(err?.message || "Unable to load linked users.");
460       }
461       return null;
462     } finally {
463       if (!quiet) {
464         setLinksLoading(false);
465       }
466     }
467   },
468   [initialAdminId, initialEventId, token]
469 );
470
471 useEffect(() => {
```

```

472     loadOperators();
473 }, [loadOperators]);
474
475 const addRole = async () => {
476     const isAdmin = activeTab === "admin";
477     const email = (isAdmin ? adminEmail : operatorEmail).trim();
478     const normalizedAdminId = String(initialAdminId || "").trim();
479     const normalizedEventId = String(initialEventId || "").trim();
480     if (!email) {
481         setError(`Enter the ${isAdmin ? "admin" : "operator"} email address.`);
482         return;
483     }
484
485     if (!isAdmin && !operatorType) {
486         setError("Select an operator type.");
487         return;
488     }
489
490     if ((isAdmin && !normalizedAdminId) || (!isAdmin && !normalizedEventId)) {
491         setError("Active workspace context is unavailable. Select a workspace and try again.");
492         return;
493     }
494
495     setSubmitting(true);
496     setError("");
497     try {
498         if (isAdmin) {
499             await linkAdmin({
500                 token,
501                 email,
502                 adminId: Number(normalizedAdminId) || normalizedAdminId
503             });
504         } else {
505             await linkOperator({
506                 token,
507                 email,
508                 eventId: Number(normalizedEventId) || normalizedEventId,
509                 type: operatorType.toLowerCase()
510             });
511         }
512         const selectedType = OPERATOR_TYPES.find((option) => option.value === operatorType);
513         const entry = {
514             id: `pending-${Date.now()}`,
515             email,
516             typeLabel: isAdmin ? "Admin" : selectedType?.label || operatorType,
517             ...nowParts()
518         };
519         if (isAdmin) {
520             setAdminEmail("");
521         } else {
522             setOperatorEmail("");
523             setOperatorType("");
524         }
525
526         const refreshed = await loadOperators({ force: true, quiet: true });
527         if (!refreshed) {
528             if (isAdmin) {
529                 setAdmins((prev) =>
530                     groupOperatorRecords([
531                         entry,
532                         ...prev.flatMap((item) => item.history || [item])
533                     ])
534                 );
535             } else {
536                 setOperators((prev) =>
537                     groupOperatorRecords([
538                         entry,
539                         ...prev.flatMap((item) => item.history || [item])
540                     ])
541                 );

```



```

542     }
543   }
544   } catch (err) {
545     console.error("Link role failed", err);
546     setError("Unable to add this role.");
547   } finally {
548     setSubmitting(false);
549   }
550 };
551
552 const isAdmin = activeTab === "admin";
553
554 return (
555   <div className="space-y-4">
556     <div className="flex flex-wrap items-start justify-between gap-4">
557       <div>
558         <h1 className="font-chillax m-0 text-[clamp(24px,3vw,32px)] font-semibold tracking-[-0.02em] text-(--text)">Admin</h1>
559         <p className="m-0 mt-1 text-[13.5px] font-light text-(--muted)">
560           Link admin access and event operators.
561         </p>
562       </div>
563       <RoleTabs activeTab={activeTab} onChange={setActiveTab} />
564     </div>
565
566     <section className="rounded-[15px] border border-(--line) border-l-[3px] border-l-(--orange) bg-(--surface) p-4 shadow-(--shadow)">
567       <div>
568         <div className={`grid items-end gap-3 max-sm:grid-cols-1 ${
569           isAdmin
570             ? "grid-cols-[minmax(260px,1fr)_auto]"
571             : "grid-cols-[minmax(260px,1fr)_minmax(180px,260px)_auto]"
572         }`}>
573           >
574             <Field label={isAdmin ? "Admin Email" : "Operator Email"}>
575               <TextInput
576                 type="email"
577                 value={isAdmin ? adminEmail : operatorEmail}
578                 onChange={(event) =>
579                   isAdmin ? setAdminEmail(event.target.value) : setOperatorEmail(event.target.value)
580                 }
581                 placeholder="name@atomx.in"
582               />
583             </Field>
584             {!isAdmin ? (
585               <Field label="Operator Type">
586                 <TypeSelect value={operatorType} onChange={setOperatorType} />
587               </Field>
588             ) : null}
589             <button
590               type="button"
591               onClick={addRole}
592               disabled={submitting}
593               className="h-11 rounded-[10px] bg-(--text) px-5 text-[13.5px] font-semibold text-(--bg) transition hover:bg-(--orange) disabled:cursor-not-allowed disabled:opacity-55"
594             >
595               {submitting ? "Adding..." : isAdmin ? "Add Admin" : "Add Operator"}
596             </button>
597           </div>
598           {error ? (
599             <div className="mt-3 rounded-[10px] border border-[rgba(224,68,32,0.25)] bg-[rgba(224,68,32,0.06)] px-3 py-2 text-[12.5px] font-semibold text-(--orange)">
600               {error}
601             </div>
602           ) : null}
603         </section>
604
605         {isAdmin ? (
606           <AddedList
607             title="Added Admins"

```

```
608         items={admins}
609         emptyText="No admins found."
610         loading={linksLoading}
611         loadError={linksError}
612     />
613 ) : (
614     <AddedList
615         title="Added Operators"
616         items={operators}
617         emptyText="No operators found."
618         showType
619         loading={linksLoading}
620         loadError={linksError}
621     />
622 )}
623 </div>
624 );
625 }
```

FILE: apps/dashboard/src/app/admin/Create_event/page.js

```
1 "use client";
2
3 import Header from "../../components/Header";
4 import AdminRoleManager from "./components/AdminRoleManager";
5
6 export default function CreateEventPage() {
7   return (
8     <main
9       className="min-h-screen bg-(--bg) pb-10"
10       style={{ fontFamily: '"Poppins", system-ui, sans-serif' }}
11     >
12       <Header areaLabel="Admin" />
13       <div className="mx-auto w-full max-w-[1780px] pl-[72px] pr-4 pt-6 md:pl-[88px] md:pr-6 max-[900px]:
px-3">
14         <AdminRoleManager />
15       </div>
16     </main>
17   );
18 }
```

FILE: apps/dashboard/src/app/admin/page.js

```
1 import AdminClientNoSSR from "./AdminClientNoSSR";
2
3 export default function AdminPage() {
4   return <AdminClientNoSSR />;
5 }
```

FILE: apps/dashboard/src/app/apk_upload/components/RecentUploads.js

```

1  "use client";
2
3  const uploads = [
4    {
5      id: 1,
6      name: "app-release-v1.2.3.apk",
7      meta: "Uploaded 2 hours ago • 25.4 MB",
8      status: "Success"
9    },
10   {
11     id: 2,
12     name: "app-debug-v1.2.2.apk",
13     meta: "Uploaded 1 day ago • 28.1 MB",
14     status: "Success"
15   }
16 ];
17
18 const FileIcon = () => (
19   <div className="grid h-11 w-11 place-items-center rounded-lg bg-[#ffe9e4] text-[#E04420] max-[640px]:h-
20     8 max-[640px]:w-8">
21     <svg
22       viewBox="0 0 24 24"
23       className="h-5 w-5"
24       fill="none"
25       stroke="currentColor"
26       strokeWidth="2"
27       strokeLinecap="round"
28       strokeLinejoin="round"
29     >
30       <path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2 2V8Z" />
31       <path d="M14 2v6h6" />
32       <path d="M9 13h6" />
33       <path d="M9 17h4" />
34     </svg>
35   </div>
36 );
37
38 const UploadEmptyIcon = () => (
39   <svg
40     viewBox="0 0 24 24"
41     className="h-7 w-7"
42     fill="none"
43     stroke="currentColor"
44     strokeWidth="1.8"
45     strokeLinecap="round"
46     strokeLinejoin="round"
47   >
48     <path d="M12 16V4" />
49     <path d="m7 9 5-5 5 5" />
50     <path d="M5 20h14" />
51   </svg>
52 );
53
54 export default function RecentUploads() {
55   return (
56     <section className="mt-5 rounded-xl border border-[#ded4ff] border-l-[4px] border-l-[#E04420] bg-white
57       p-5 shadow-[0_22px_65px_rgba(15,23,42,0.10)] max-[640px]:mt-3 max-[640px]:rounded-lg max-[640px]:p-3">
58       <div className="flex flex-col gap-2 border-b border-[#e5e5e5] pb-4 sm:flex-row sm:items-end sm:justify-between max-[640px]:pb-3">
59         <div>
60           <p className="text-[0.76rem] font-bold uppercase tracking-[0.24em] text-[#E04420] max-[640px]:text-[0.58rem] max-[640px]:tracking-[0.18em]">
61             Recent Uploads
62           </p>
63           <h2 className="mt-1 text-[1.32rem] font-bold leading-none text-[#1c1c1c] max-[640px]:text-[0.95rem]">
64             2 listed
65           </h2>
66         </div>

```

```

65     <p className="text-[0.9rem] font-medium text-[#8f8f8f] max-[640px]:text-[0.68rem]">
66       Signed packages and debug builds
67     </p>
68   </div>
69
70   <div className="mt-4 space-y-3 max-[640px]:mt-3 max-[640px]:space-y-2">
71     {uploads.map((item) => (
72       <div
73         key={item.id}
74         className="flex items-center justify-between gap-4 rounded-lg border border-[#e5e5e5] bg-white px-4 py-3 shadow-[0_8px_22px_rgba(15,23,42,0.04)] max-[640px]:gap-2 max-[640px]:px-3 max-[640px]:py-2.5"
75       >
76         <div className="flex min-w-0 items-center gap-4 max-[640px]:gap-2.5">
77           <FileIcon />
78           <div className="min-w-0">
79             <p className="truncate text-[1rem] font-semibold text-[#1f1f1f] max-[640px]:text-[0.74rem]">{item.name}</p>
80             <p className="text-[0.86rem] font-medium text-[#657391] max-[640px]:text-[0.62rem]">{item.meta}</p>
81           </div>
82         </div>
83         <span className="shrink-0 rounded-full bg-[#e4f6ff] px-4 py-2 text-[0.78rem] font-bold text-[#0285bf] max-[640px]:px-2.5 max-[640px]:py-1 max-[640px]:text-[0.58rem]">
84           {item.status}
85         </span>
86       </div>
87     )})
88   </div>
89
90   <div className="mt-10 flex flex-col items-center justify-center gap-2 pb-3 text-[#8b96aa] max-[640px]:mt-6">
91     <UploadEmptyIcon />
92     <p className="text-[0.9rem] font-medium max-[640px]:text-[0.68rem]">No more uploads to show</p>
93   </div>
94 </section>
95 );
96 }

```

FILE: apps/dashboard/src/app/apk_upload/components/UploadDropzone.js

```

1  "use client";
2
3  import React from "react";
4
5  const UploadIcon = () => (
6    <div className="grid h-12 w-12 place-items-center rounded-xl bg-[#ffe9e4] text-[#E04420] max-[640px]:h-
9    max-[640px]:w-9 max-[640px]:rounded-lg">
7      <svg
8        viewBox="0 0 24 24"
9        className="h-6 w-6"
10       fill="none"
11       stroke="currentColor"
12       strokeWidth="2"
13       strokeLinecap="round"
14       strokeLinejoin="round"
15     >
16       <path d="M12 4v12" />
17       <path d="m7 9 5-5 5 5" />
18       <path d="M5 20h14" />
19     </svg>
20   </div>
21 );
22
23 export default function UploadDropzone() {
24   return (
25     <div className="flex min-h-[198px] flex-col items-center justify-center rounded-lg border border-dash
ed border-[#b8a8ff] bg-[linear-gradient(135deg,#ffffff_0%,#fbfaff_100%)] px-6 py-6 text-center max-[640px
]:min-h-[138px] max-[640px]:px-4 max-[640px]:py-4">
26       <UploadIcon />
27       <p className="mt-5 text-[0.96rem] font-semibold text-[#1f1f1f] max-[640px]:mt-3 max-[640px]:text-[0
.78rem]">
28         Drop APK files here
29       </p>
30       <p className="mt-2 text-[0.82rem] font-normal text-[#848484] max-[640px]:mt-1 max-[640px]:text-[0.6
8rem]">
31         or browse from your device
32       </p>
33       <button
34         type="button"
35         className="mt-4 h-10 rounded-lg bg-[#1c1c1c] px-7 text-[0.8rem] font-semibold text-white shadow-[
0_10px_22px_rgba(28,28,28,0.13)] transition duration-200 hover:-translate-y-0.5 hover:bg-[#E04420] max-[6
40px]:mt-3 max-[640px]:h-8 max-[640px]:px-4 max-[640px]:text-[0.66rem]"
36       >
37         Browse Files
38       </button>
39     </div>
40   );
41 }

```

FILE: apps/dashboard/src/app/apk_upload/components/UploadSummaryCard.js

```

1 "use client";
2
3 export default function UploadSummaryCard() {
4   return (
5     <div className="relative min-h-[198px] overflow-hidden rounded-lg bg-[#1c1c1c] px-7 py-7 text-white shadow-[0_16px_42px_rgba(28,28,28,0.18)] max-[640px]:min-h-[138px] max-[640px]:px-4 max-[640px]:py-4">
6       <div className="absolute bottom-0 right-0 h-28 w-40 rounded-tl-full bg-[#341CD6]/42 blur-xl" aria-hidden />
7       <div className="relative">
8         <div className="grid h-10 w-10 place-items-center rounded-lg bg-[linear-gradient(135deg,#E04420_0%,#A9379E_48%,#341CD6_100%)] text-white shadow-[0_12px_28px_rgba(52,28,214,0.22)] max-[640px]:h-8 max-[640px]:w-8">
9           <svg viewBox="0 0 24 24" className="h-5 w-5" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
10             <path d="M12 22s8-4 8-10V5l-8-3-8 3v7c0 6 8 10 8 10Z" />
11             <path d="m9 12 2 2 4-4" />
12           </svg>
13         </div>
14         <p className="mt-5 text-[0.68rem] font-semibold uppercase tracking-[0.22em] text-white/64 max-[640px]:mt-3 max-[640px]:text-[0.56rem] max-[640px]:tracking-[0.18em]">
15           Uploaded
16         </p>
17         <p className="mt-2 text-[1.82rem] font-normal leading-tight text-white max-[640px]:text-[1.2rem]">
18           >
19             2 builds
20           </p>
21           <p className="mt-3 max-w-[370px] text-[0.82rem] font-normal leading-6 text-white/62 max-[640px]:mt-2 max-[640px]:text-[0.68rem] max-[640px]:leading-5">
22             Latest package is ready for rollout. New uploads should be APK files only.
23           </p>
24         </div>
25       </div>
26     );
27   }

```


FILE: apps/dashboard/src/app/apk_upload/page.js

```

1 import Header from "../components/Header";
2 import UploadDropzone from "../components/UploadDropzone";
3 import UploadSummaryCard from "../components/UploadSummaryCard";
4 import RecentUploads from "../components/RecentUploads";
5
6 export default function ApkUploadPage() {
7   return (
8     <main
9       className="apk-upload-font min-h-screen bg-[color:rgb(var(--color-bg))] pb-10"
10      style={{ fontFamily: '"AtomX APK Poppins", Poppins, sans-serif' }}
11    >
12      <style>{`
13        @font-face {
14          font-family: "AtomX APK Poppins";
15          src: url("/shared/fonts/Poppins/Poppins-Regular.ttf") format("truetype");
16          font-weight: 400;
17          font-style: normal;
18          font-display: swap;
19        }
20        @font-face {
21          font-family: "AtomX APK Poppins";
22          src: url("/shared/fonts/Poppins/Poppins-SemiBold.ttf") format("truetype");
23          font-weight: 600;
24          font-style: normal;
25          font-display: swap;
26        }
27        @font-face {
28          font-family: "AtomX APK Poppins";
29          src: url("/shared/fonts/Poppins/Poppins-Bold.ttf") format("truetype");
30          font-weight: 700;
31          font-style: normal;
32          font-display: swap;
33        }
34        .apk-upload-font,
35        .apk-upload-font * {
36          font-family: "AtomX APK Poppins", Poppins, sans-serif;
37        }
38      `}</style>
39      <Header areaLabel="APK Uploads" variant="portal" />
40
41      <div className="w-full pr-4 pl-[72px] pt-5 md:pr-7 md:pl-[88px] max-[900px]:px-3 max-[640px]:pt-4">
42        <section className="mb-4 flex flex-col gap-4 border-b border-[#d8d8d8] pb-4 sm:flex-row sm:items-
43        center sm:justify-between max-[640px]:mb-3 max-[640px]:gap-3 max-[640px]:pb-3">
44          <div>
45            <h1 className="text-[1.75rem] font-semibold leading-none text-[#111827] md:text-[1.9rem] max-
46            [640px]:text-[1.25rem]">
47              APK Uploads
48            </h1>
49            <p className="mt-3 text-[0.9rem] font-normal text-[#777777] max-[640px]:mt-2 max-[640px]:text
50            -[0.7rem]">
51              Upload Android builds and track recent release packages.
52            </p>
53          </div>
54
55          <button
56            type="button"
57            className="inline-flex h-10 items-center justify-center gap-3 rounded-lg bg-[#1c1c1c] px-5 te
58            xt-[0.86rem] font-semibold text-white shadow-[0_12px_26px_rgba(28,28,28,0.16)] transition duration-200 ho
59            ver:-translate-y-0.5 hover:bg-[#E04420] max-[640px]:h-9 max-[640px]:gap-2 max-[640px]:px-3 max-[640px]:te
60            xt-[0.72rem]"
61          >
62            <svg viewBox="0 0 24 24" className="h-[18px] w-[18px]" fill="none" stroke="currentColor" stro
63            keWidth="2" strokeLinecap="round" strokeLinejoin="round">
64              <path d="M12 4v12" />
65              <path d="m7 9 5-5 5 5" />
66              <path d="M5 20h14" />
67            </svg>
68            Upload APK
69          </button>

```

```
63         </section>
64
65         <section className="rounded-xl border border-[#ded4ff] border-l-[4px] border-l-[#E04420] bg-white
p-3 shadow-[0_18px_52px_rgba(15,23,42,0.09)] md:p-4 max-[640px]:rounded-lg">
66             <div className="grid gap-4 lg:grid-cols-[0.85fr_1.15fr] max-[640px]:gap-3">
67                 <UploadSummaryCard />
68                 <UploadDropzone />
69             </div>
70         </section>
71
72         <RecentUploads />
73     </div>
74 </main>
75 );
76 }
```

FILE: apps/dashboard/src/app/components/Buttons/SaveButton.js

```
1 "use client";
2
3 export default function SaveButton({
4   label = "Save Changes",
5   onClick,
6   disabled = false,
7   className = ""
8 }) {
9   return (
10     <button
11       type="button"
12       onClick={onClick}
13       disabled={disabled}
14       className={`rounded-full bg-[color:rgb(var(--color-orange))] px-6 py-3 text-sm font-semibold text-white shadow-[0_12px_24px_rgb(var(--color-orange)/0.35)] transition hover:bg-[#f57f2f] disabled:cursor-not-allowed disabled:opacity-60 ${className}`}
15     >
16       {label}
17     </button>
18   );
19 }
```

FILE: apps/dashboard/src/app/components/Header.js

```
1 "use client";
2
3 import { useEffect, useMemo, useState } from "react";
4 import Image from "next/image";
5 import { useRouter } from "next/navigation";
6 import { getInitials } from "@atomx/lib";
7 import SideDrawer from "../SideDrawer";
8 import ProfileMenu from "../ProfileMenu";
9 import { useDashboardStore } from "../../store/dashboardStore";
10
11 export default function Header({
12   eventId = null,
13   eventName = "SunBurun",
14   venue = "Mahalaxmi Race Cours",
15   city = "Mumbai, India",
16   areaLabel = "Configuration",
17   profileName = "Omkar",
18   profileInitials = "OD",
19   profileRole = "Admin",
20   profileEmail = "design@atomx.in",
21   breadcrumb = "Profile / Operations",
22   variant = "portal",
23   showEditEventButton = false,
24   hideNav = false
25 }) {
26   const router = useRouter();
27   const profile = useDashboardStore((state) => state.profile);
28   const storedEventMeta = useDashboardStore((state) => state.eventMeta);
29   const storedEventDetails = useDashboardStore((state) => state.eventDetails);
30   const selectedService = useDashboardStore((state) => state.selectedService);
31   const setEventMeta = useDashboardStore((state) => state.setEventMeta);
32   const setSelectedService = useDashboardStore((state) => state.setSelectedService);
33   const [isMobileDrawerOpen, setIsMobileDrawerOpen] = useState(false);
34   const [theme, setTheme] = useState("light");
35
36   useEffect(() => {
37     if (typeof document === "undefined") return;
38     setTheme(
39       document.documentElement.getAttribute("data-atx") === "dark" ? "dark" : "light"
40     );
41   }, []);
42
43   const toggleTheme = () => {
44     setTheme((prev) => {
45       const next = prev === "dark" ? "light" : "dark";
46       if (typeof document !== "undefined") {
47         if (next === "dark") {
48           document.documentElement.setAttribute("data-atx", "dark");
49         } else {
50           document.documentElement.removeAttribute("data-atx");
51         }
52       }
53       try {
54         window.localStorage.setItem("atomx.theme", next);
55       } catch (err) {
56         /* ignore storage failures */
57       }
58       return next;
59     });
60   };
61   const isDark = theme === "dark";
62
63   useEffect(() => {
64     if (typeof window === "undefined") return;
65     const params = new URLSearchParams(window.location.search);
66     const nextEventId = params.get("eventId");
67     const nextEventName = params.get("eventName");
68     const nextVenue = params.get("venue");
69     const nextCity = params.get("city");
```

```

70     const nextService = params.get("service");
71
72     if (nextEventId || nextEventName || nextVenue || nextCity) {
73         setEventMeta({
74             eventId: nextEventId || storedEventMeta?.eventId || eventId,
75             eventName: nextEventName || storedEventMeta?.eventName || eventName,
76             venue: nextVenue || storedEventMeta?.venue || venue,
77             city: nextCity || storedEventMeta?.city || city
78         });
79     }
80     if (nextService && nextService !== selectedService) {
81         setSelectedService(nextService);
82     }
83 }, [
84     storedEventMeta,
85     eventId,
86     eventName,
87     venue,
88     city,
89     selectedService,
90     setEventMeta,
91     setSelectedService
92 ]);
93
94 const resolvedInitials = useMemo(() => {
95     if (profile?.name) return getInitials(profile.name);
96     return profileInitials;
97 }, [profile, profileInitials]);
98
99 const resolvedRole = profile?.type ?? profileRole;
100 const resolvedEmail = profile?.email ?? profileEmail;
101 const resolvedName = profile?.name ?? profileName;
102 const resolvedPicture =
103     profile?.picture ?? profile?.image ?? profile?.avatar ?? profile?.photoURL ?? null;
104 const resolvedEventMeta = useMemo(() => {
105     return {
106         eventId:
107             storedEventMeta?.eventId ??
108             storedEventDetails?.id ??
109             profile?.ctx?.eventId ??
110             eventId,
111         eventName: storedEventMeta?.eventName ?? eventName,
112         venue: storedEventMeta?.venue ?? venue,
113         city: storedEventMeta?.city ?? city
114     };
115 }, [storedEventMeta, storedEventDetails, profile, eventId, eventName, venue, city]);
116
117 const headerHeight = "58px";
118 const crumbHeight = "0px";
119 return (
120     <div
121         style={{
122             "--header-h": headerHeight,
123             "--crumb-h": crumbHeight,
124             "--header-total-h": "var(--header-h)"
125         }}
126     >
127         <div className="fixed left-0 right-0 top-0 z-40">
128             <header
129                 className="w-full border-b border-(--line) bg-(--surface) text-(--text) shadow-(--shadow)"
130                 style={{ height: "var(--header-h)" }}
131             >
132                 <div className="flex h-full items-center gap-3 px-4 md:px-5 max-[900px]:gap-2 max-[900px]:px-3"
133 >
134                     <div className="flex min-w-0 items-center gap-3 max-[900px]:gap-2">
135                         {!hideNav ? (
136                             <button
137                                 type="button"
138                                 onClick={() => setIsMobileDrawerOpen(true)}
139                                 className="grid h-9 w-9 shrink-0 place-items-center rounded-lg border border-(--line) bg-

```

```

    (---surface) text-(---text) shadow-(---shadow) transition hover:border-(---orange) hover:text-(---orange) min-
    [901px]:hidden"
139     aria-label="Open navigation"
140   >
141     <svg
142       viewBox="0 0 24 24"
143       className="h-5 w-5"
144       fill="none"
145       stroke="currentColor"
146       strokeWidth="2.2"
147       strokeLinecap="round"
148       strokeLinejoin="round"
149       aria-hidden
150     >
151       <path d="M4 7h16" />
152       <path d="M4 12h16" />
153       <path d="M4 17h16" />
154     </svg>
155   </button>
156   ) : null}
157   <span className="relative block h-[40px] w-[100px] shrink-0 overflow-hidden max-[900px]:w-[
158     88px]">
159     <Image
160       src="/shared/logos/AtomX_Logo.svg"
161       alt="AtomX logo"
162       width={148}
163       height={148}
164       priority
165       className="absolute -left-[27px] -top-[52px] h-[132px] w-[150px] max-w-none"
166     />
167   </span>
168   <div className="hidden h-9 w-px bg-(---line) sm:block" aria-hidden />
169   <div className="font-chillax flex min-w-0 items-center gap-2 text-[1.3rem] font-semibold le
170     ading-none tracking-[-0.01em] sm:text-[1.4rem] max-[900px]:gap-1.5 max-[900px]:text-[1.05rem]">
171     <span className="truncate text-(---muted)">Portal</span>
172     <span className="text-(---faint)"></span>
173     <span className="truncate text-(---orange)">{areaLabel}</span>
174   </div>
175   </div>
176   <div className="flex-1" />
177   <span
178     className="font-vcr inline-flex shrink-0 items-center gap-2 text-[11px] font-semibold track
179     ing-[0.1em] text-(---muted)"
180     aria-label={`Event ID ${resolvedEventMeta.eventId ?? "not selected"}`}
181     title="Selected event ID"
182   >
183     <span className="hidden uppercase sm:inline">Event</span>
184     <span className="font-chillax text-[14px] font-bold tracking-[0.01em] text-(---text)">
185       #{resolvedEventMeta.eventId ?? "-"}
186     </span>
187   </span>
188   <button
189     type="button"
190     onClick={toggleTheme}
191     title={isDark ? "Switch to light mode" : "Switch to dark mode"}
192     aria-label={isDark ? "Switch to light mode" : "Switch to dark mode"}
193     aria-pressed={isDark}
194     className="mr-1 grid h-8 w-8 shrink-0 place-items-center rounded-[10px] border border-(---li
195     ne) bg-(---surface) text-(---muted) transition hover:border-(---orange) hover:text-(---orange)"
196   >
197     {isDark ? (
198       <svg viewBox="0 0 24 24" className="h-[17px] w-[17px]" fill="none" stroke="currentColor"
199       strokeWidth="1.8" strokeLinecap="round" strokeLinejoin="round" aria-hidden>
200         <circle cx="12" cy="12" r="4" />
201         <path d="M12 2v2M12 20v2M4.93 11.42 1.42M17.65 17.65 11.42 1.42M2 12h2M4.93
202         19.07 11.42 1.42M17.65 6.35 11.42 1.42" />
203       </svg>
204     ) : (
205       <svg viewBox="0 0 24 24" className="h-[17px] w-[17px]" fill="none" stroke="currentColor"
206       strokeWidth="1.8" strokeLinecap="round" strokeLinejoin="round" aria-hidden>

```

```

200         <path d="M20.5 14.2A8.4 8.4 0 0 1 9.8 3.5 8.5 8.5 0 1 0 20.5 14.2Z" />
201     </svg>
202     })
203 </button>
204 {showEditEventButton ? (
205     <button
206         type="button"
207         onClick={() => router.push("/event-edit")}
208         className="mr-1 inline-flex h-9 items-center gap-2 rounded-lg border border-(--line) bg-(
--surface) px-3 text-[0.82rem] font-bold text-(--text) shadow-(--shadow) transition hover:border-(--orang
e) hover:text-(--orange)"
209         aria-label="Edit event"
210     >
211         <svg
212             viewBox="0 0 24 24"
213             className="h-4 w-4"
214             fill="none"
215             stroke="currentColor"
216             strokeWidth="2"
217             strokeLinecap="round"
218             strokeLinejoin="round"
219             aria-hidden
220         >
221             <path d="M12 20h9" />
222             <path d="M16.5 3.5a2.1 2.1 0 0 1 3 3L7 19l-4 1 1-4 12.5-12.5Z" />
223         </svg>
224         <span>Event</span>
225     </button>
226 ) : null}
227 <ProfileMenu
228     initials={resolvedInitials}
229     name={resolvedName}
230     role={resolvedRole}
231     email={resolvedEmail}
232     picture={resolvedPicture}
233     variant="portal"
234 />
235 </div>
236 </header>
237 </div>
238 <div style={{ height: "var(--header-total-h)" }} />
239 {!hideNav ? (
240     <SideDrawer
241         mobileOpen={isMobileDrawerOpen}
242         onMobileClose={() => setIsMobileDrawerOpen(false)}
243     />
244 ) : null}
245 </div>
246 );
247 }

```

FILE: apps/dashboard/src/app/components/Popups/Toast.js

```
1 "use client";
2
3 import { useEffect } from "react";
4
5 export default function Toast({
6   open,
7   title,
8   message,
9   duration = 2400,
10  onClose
11 }) {
12   useEffect(() => {
13     if (!open) return;
14     const timer = setTimeout(() => {
15       onClose?();
16     }, duration);
17     return () => clearTimeout(timer);
18   }, [open, duration, onClose]);
19
20   if (!open) return null;
21
22   return (
23     <div className="fixed top-[92px] left-1/2 z-50 -translate-x-1/2">
24       <div className="flex items-center gap-3 rounded-full border border-slate-200 bg-white px-5 py-3 text-
25         t-sm text-slate-700 shadow-[0_12px_26px_rgba(15,23,42,0.18)]">
26         <span className="font-semibold text-slate-800">{title}</span>
27         <span className="text-slate-300">|</span>
28         <span className="text-slate-600">{message}</span>
29       </div>
30     );
31 }
```


FILE: apps/dashboard/src/app/components/PostHogInit.js

```
1 "use client";
2
3 import { useEffect } from "react";
4 import { initPostHog } from "@atomx/global-components";
5
6 export default function PostHogInit() {
7   useEffect(() => {
8     initPostHog("dashboard");
9   }, []);
10
11   return null;
12 }
```

FILE: apps/dashboard/src/app/components/ProfileMenu.js

```

1  "use client";
2
3  import { useEffect, useRef, useState } from "react";
4  import { decodeJwt } from "@atomx/lib";
5  import { useDashboardStore } from "../../store/dashboardStore";
6
7  const REAUTH_CONTEXT_KEY = "atomx.portal.reauth";
8  const REAUTH_CONTEXT_TTL_MS = 24 * 60 * 60 * 1000;
9
10 function normalizeRoleType(value) {
11   return String(value || "").
12     .trim()
13     .toLowerCase()
14     .replace(/[_\s]+/g, "-");
15 }
16
17 function mapServiceParam(type) {
18   const normalized = normalizeRoleType(type);
19   if (normalized.includes("tag-series") || normalized.includes("tagseries")) {
20     return "tag-series";
21   }
22   if (normalized.includes("cashless")) return "cashless";
23   if (normalized.includes("inventory")) return "inventory";
24   if (normalized.includes("admin")) return "admin";
25   return normalized || null;
26 }
27
28 function formatRole(value, long = false) {
29   const raw = String(value || "member").trim();
30   if (!raw) return long ? "Member" : "Member";
31   if (raw.toLowerCase() === "admin") return long ? "Administrator" : "Admin";
32   return raw
33     .replace(/[-_]+/g, " ")
34     .replace(/\b\w/g, (char) => char.toUpperCase());
35 }
36
37 function coerceId(value) {
38   if (value === null || value === undefined || value === "") return null;
39   const num = Number(value);
40   return Number.isFinite(num) ? num : value;
41 }
42
43 function sanitizeReturnTo(value) {
44   try {
45     const parsed = new URL(value, window.location.origin);
46     parsed.searchParams.delete("returnTo");
47     parsed.searchParams.delete("token");
48     return parsed.toString();
49   } catch {
50     return value;
51   }
52 }
53
54 function buildReturnToUrl(currentHref, eventMeta, selectedService) {
55   const sanitized = sanitizeReturnTo(currentHref);
56   try {
57     const parsed = new URL(sanitized, window.location.origin);
58     if (eventMeta?.eventId && !parsed.searchParams.get("eventId")) {
59       parsed.searchParams.set("eventId", String(eventMeta.eventId));
60     }
61     if (eventMeta?.eventName && !parsed.searchParams.get("eventName")) {
62       parsed.searchParams.set("eventName", eventMeta.eventName);
63     }
64     if (eventMeta?.venue && !parsed.searchParams.get("venue")) {
65       parsed.searchParams.set("venue", eventMeta.venue);
66     }
67     if (eventMeta?.city && !parsed.searchParams.get("city")) {
68       parsed.searchParams.set("city", eventMeta.city);
69     }

```

```

70     const service = mapServiceParam(selectedService);
71     if (service && !parsed.searchParams.get("service")) {
72         parsed.searchParams.set("service", service);
73     }
74     return parsed.toString();
75 } catch {
76     return sanitized;
77 }
78 }
79
80 function getLoginUrl(returnTo) {
81     const portalBase =
82         process.env.NEXT_PUBLIC_PORTAL_URL ||
83         process.env.NEXT_PUBLIC_ACCESS_PORTAL_URL ||
84         "/";
85     const baseUrl = portalBase.startsWith("http")
86         ? new URL(portalBase)
87         : new URL(portalBase, window.location.origin);
88     baseUrl.searchParams.delete("returnTo");
89     if (returnTo) {
90         const baseId = `${baseUrl.origin}${baseUrl.pathname}`;
91         if (returnTo !== baseId) {
92             baseUrl.searchParams.set("returnTo", returnTo);
93         }
94     }
95     return baseUrl.toString();
96 }
97
98 function clearAuthCache() {
99     if (typeof window === "undefined") return;
100     window.localStorage.removeItem("atomx.portal.token");
101     window.localStorage.removeItem("atomx.dashboard.token");
102     window.localStorage.removeItem("atomx.dashboard.store");
103     const keysToRemove = [];
104     for (let i = 0; i < window.localStorage.length; i += 1) {
105         const key = window.localStorage.key(i);
106         if (key && key.startsWith("atomx.auth.")) {
107             keysToRemove.push(key);
108         }
109     }
110     keysToRemove.forEach((key) => window.localStorage.removeItem(key));
111 }
112
113 const SERVICES = [
114     {
115         id: "cashless",
116         label: "Cashless Payments",
117         icon: (
118             <svg viewBox="0 0 24 24" className="h-6 w-6" fill="none" stroke="currentColor" strokeWidth="1.8">
119                 <rect x="3" y="6" width="18" height="12" rx="2" />
120                 <path d="M7 10h6" />
121                 <path d="M17 14h-4" />
122             </svg>
123         ),
124     },
125     {
126         id: "crew",
127         label: "Crew Meal",
128         icon: (
129             <svg viewBox="0 0 24 24" className="h-6 w-6" fill="none" stroke="currentColor" strokeWidth="1.8">
130                 <path d="M4 3v7a3 3 0 0 0 6 0V3" />
131                 <path d="M10 3v7a3 3 0 0 1 6 0V3" />
132                 <path d="M14 3v7" />
133                 <path d="M14 10a3 3 0 0 0 6 0V3" />
134                 <path d="M16 14v7" />
135             </svg>
136         ),
137     },
138     {
139         id: "access",

```

```

140     label: "Access Control",
141     icon: (
142       <svg viewBox="0 0 24 24" className="h-6 w-6" fill="none" stroke="currentColor" strokeWidth="1.8">
143         <circle cx="12" cy="12" r="3.2" />
144         <path d="M12 5v2" />
145         <path d="M12 17v2" />
146         <path d="M5 12h2" />
147         <path d="M17 12h2" />
148       </svg>
149     ),
150   },
151   {
152     id: "inventory",
153     label: "InventoryX",
154     icon: (
155       <svg viewBox="0 0 24 24" className="h-6 w-6" fill="none" stroke="currentColor" strokeWidth="1.8">
156         <path d="M3 7h18" />
157         <path d="M5 7l2-3h10l2 3" />
158         <rect x="4" y="7" width="16" height="12" rx="2" />
159         <path d="M9 11h6" />
160       </svg>
161     ),
162   },
163   {
164     id: "tag-series",
165     label: "Tag Series",
166     icon: (
167       <svg viewBox="0 0 24 24" className="h-6 w-6" fill="none" stroke="currentColor" strokeWidth="1.8">
168         <path d="M3 7h10l8 5-8 5H3z" />
169         <circle cx="7" cy="12" r="2" />
170       </svg>
171     ),
172   }
173 ];
174
175 export default function ProfileMenu({
176   initials = "OD",
177   name = "Omkar",
178   role = "Admin",
179   email = "design@atomx.in",
180   picture = null,
181   variant = "event"
182 }) {
183   const [open, setOpen] = useState(false);
184   const menuRef = useRef(null);
185   const token = useDashboardStore((state) => state.token);
186   const profile = useDashboardStore((state) => state.profile);
187   const eventMeta = useDashboardStore((state) => state.eventMeta);
188   const selectedService = useDashboardStore((state) => state.selectedService);
189   const setToken = useDashboardStore((state) => state.setToken);
190   const setSelectedService = useDashboardStore((state) => state.setSelectedService);
191   const isPortalVariant = variant === "portal";
192
193   useEffect(() => {
194     const handler = (event) => {
195       if (!menuRef.current?.contains(event.target)) {
196         setOpen(false);
197       }
198     };
199     if (open) {
200       document.addEventListener("mousedown", handler);
201     }
202     return () => document.removeEventListener("mousedown", handler);
203   }, [open]);
204
205   const handleLogout = () => {
206     if (typeof window === "undefined") return;
207     const returnUrl = buildReturnToUrl(window.location.href, eventMeta, selectedService);
208     let decodedProfile = profile;
209     if (!decodedProfile && token) {

```

```

210     try {
211       decodedProfile = decodeJwt(token);
212     } catch (err) {
213       console.error("Failed to decode token for logout context", err);
214     }
215   }
216   const roles = Array.isArray(decodedProfile?.roles) ? decodedProfile.roles : [];
217   const ctxType = decodedProfile?.ctx?.type;
218   const normalizedType = normalizeRoleType(ctxType || selectedService || "");
219   const roleMatch = roles.find(
220     (entry) => normalizeRoleType(entry?.type) === normalizedType
221   );
222   const payload = {
223     version: 1,
224     createdAt: Date.now(),
225     expiresAt: Date.now() + REAUTH_CONTEXT_TTL_MS,
226     returnTo,
227     type: roleMatch?.type || ctxType || selectedService || null,
228     service: mapServiceParam(roleMatch?.type || ctxType || selectedService),
229     eventId: coerceId(eventMeta?.eventId ?? decodedProfile?.ctx?.eventId ?? roleMatch?.eventId),
230     adminId: coerceId(decodedProfile?.ctx?.adminId ?? roleMatch?.adminId)
231   };
232   try {
233     window.localStorage.setItem(REAUTH_CONTEXT_KEY, JSON.stringify(payload));
234   } catch (err) {
235     console.error("Failed to save resume context before logout", err);
236   }
237   clearAuthCache();
238   setToken(null);
239   setSelectedService(null);
240   window.location.assign(getLoginUrl(returnTo));
241 };
242
243 return (
244   <div ref={menuRef} className="relative">
245     <button
246       type="button"
247       onClick={() => setOpen((prev) => !prev)}
248       className={
249         isPortalVariant
250         ? "flex h-[42px] items-center gap-2 rounded-[11px] border border-(--line) bg-(--surface2) px-
2.5 text-left text-(--text) transition hover:border-(--orange)"
251         : "flex h-9 items-center gap-2 rounded-full bg-white/18 px-3 text-xs font-semibold text-white
ring-1 ring-white/25"
252       }
253       aria-haspopup="menu"
254       aria-expanded={open}
255     >
256       {isPortalVariant ? (
257         <div className="hidden min-w-0 flex-col leading-tight sm:flex">
258           <span className="font-vcr max-w-[6.5rem] truncate text-[0.6rem] uppercase tracking-[0.14em] t
ext-(--muted)">
259             {formatRole(role)}
260           </span>
261           <span className="max-w-[7rem] truncate text-[0.82rem] font-semibold text-(--text)">
262             {name}
263           </span>
264         </div>
265       ) : null}
266       <span
267         className={
268           isPortalVariant
269           ? "font-chillax flex h-8 w-8 items-center justify-center overflow-hidden rounded-[9px] bg-[
linear-gradient(140deg,#341cd6,#e04420)] text-[0.85rem] font-semibold text-white"
270           : "flex h-7 w-7 items-center justify-center rounded-full bg-white text-[11px] font-bold tex
t-#[#258d9c]"
271         }
272       >
273         {picture && isPortalVariant ? (
274           // eslint-disable-next-line @next/next/no-img-element

```

```

275         <img src={picture} alt={name} className="h-full w-full object-cover" />
276     ) : (
277         initials
278     )}
279 </span>
280 <svg
281     viewBox="0 0 24 24"
282     className={`h-4 w-4 transition-transform ${
283         open ? "rotate-180" : ""
284     } ${isPortalVariant ? "text-(--muted)" : ""}`}
285     fill="none"
286     stroke="currentColor"
287     strokeWidth="2.2"
288 >
289     <polyline points="6 9 12 15 18 9" />
290 </svg>
291 </button>
292
293     {open && isPortalVariant && (
294         <div className="absolute right-0 top-full z-50 mt-2 w-[236px] rounded-[11px] border border-(--lin
295 e) bg-(--surface) p-2.5 shadow-(--shadowUp)">
296         <div className="flex items-center gap-2.5 px-1 py-1">
297             <div className="font-chillax flex h-9 w-9 shrink-0 items-center justify-center overflow-hidde
298 n rounded-full bg-[linear-gradient(140deg,#341cd6,#e04420)] text-sm font-semibold text-white">
299                 {picture ? (
300                     // eslint-disable-next-line @next/next/no-img-element
301                     <img src={picture} alt={name} className="h-full w-full object-cover" />
302                 ) : (
303                     initials
304                 )}
305             </div>
306             <div className="min-w-0 leading-tight">
307                 <p className="m-0 truncate text-[0.9rem] font-semibold text-(--text)">
308                     {name}
309                 </p>
310                 <small className="text-[0.74rem] font-medium text-(--muted)">
311                     {formatRole(role, true)}
312                 </small>
313             </div>
314         </div>
315         <button
316             type="button"
317             className="mt-2 w-full rounded-md bg-(--text) px-3 py-2.5 text-left text-[0.84rem] font-semib
318 old text-(--bg) transition hover:opacity-90"
319             onClick={handleLogout}
320         >
321             Logout
322         </button>
323     </div>
324 )}
325
326     {open && !isPortalVariant && (
327         <div className="absolute right-0 top-full z-50 mt-3 w-72 rounded-2xl border border-slate-200 bg-w
328 hite p-4 shadow-[0_18px_40px_rgba(15,23,42,0.18)]">
329         <div className="flex items-start justify-between">
330             <div>
331                 <div className="text-sm font-semibold text-slate-700">{role}</div>
332                 <div className="text-xs text-slate-500">{email}</div>
333             </div>
334             <button
335                 type="button"
336                 className="text-xs font-semibold text-slate-500 hover:text-[color:rgb(var(--color-orange))]"
337                 onClick={handleLogout}
338             >
339                 Log out
340             </button>
341         </div>
342     </div>
343
344     <div className="my-3 h-px bg-slate-200" />

```

```

340
341     <div className="grid grid-cols-2 gap-3">
342         {SERVICES.map((service) => {
343             const isSelected = selectedService === service.id;
344             return (
345                 <button
346                     key={service.label}
347                     type="button"
348                     className={`flex flex-col items-center gap-2 rounded-2xl border px-2 py-3 text-center tex
t-xs font-semibold transition ${
349                         isSelected
350                         ? "border-[#258d9c] bg-[#f0f8fa] text-[#258d9c]"
351                         : "border-slate-200 text-slate-600 hover:border-[#258d9c] hover:text-[#258d9c]"
352                     }`}
353                     aria-pressed={isSelected}
354                 >
355                     <span
356                         className={`flex h-12 w-12 items-center justify-center rounded-2xl ${
357                             isSelected ? "bg-[#dff3f6] text-[#258d9c]" : "bg-[#f2f6f9] text-[#258d9c]"
358                         }`}
359                     >
360                         {service.icon}
361                     </span>
362                     <span className="leading-tight">{service.label}</span>
363                 </button>
364             )}}
365         </div>
366     </div>
367 )}
368 </div>
369 );
370 }

```

FILE: apps/dashboard/src/app/components/Session/SessionGuard.js

```

1  "use client";
2
3  import { useCallback, useEffect, useMemo, useRef, useState } from "react";
4  import { decodeJwt } from "@atomx/lib";
5  import {
6    capturePostHogEvent,
7    identifyPostHogUser
8  } from "@atomx/global-components";
9  import { useDashboardStore } from "../../store/dashboardStore";
10 import SessionPrompt from "../SessionPrompt";
11
12 const WARNING_WINDOW_MS = 10 * 60 * 1000;
13 const REAUTH_CONTEXT_KEY = "atomx.portal.reauth";
14 const REAUTH_CONTEXT_TTL_MS = 24 * 60 * 60 * 1000;
15 const DASHBOARD_SELECTED_TOKEN_KEY = "atomx.dashboard.token";
16
17 function sanitizeReturnTo(value) {
18   try {
19     const parsed = new URL(value, window.location.origin);
20     parsed.searchParams.delete("returnTo");
21     parsed.searchParams.delete("token");
22     return parsed.toString();
23   } catch (err) {
24     return value;
25   }
26 }
27
28 function normalizeRoleType(value) {
29   return String(value || "").
30     .trim()
31     .toLowerCase()
32     .replace(/[_\s]+/g, "-");
33 }
34
35 function mapServiceParam(type) {
36   const normalized = normalizeRoleType(type);
37   if (normalized.includes("tag-series") || normalized.includes("tagseries")) {
38     return "tag-series";
39   }
40   if (normalized.includes("cashless")) return "cashless";
41   if (normalized.includes("inventory")) return "inventory";
42   if (normalized.includes("admin")) return "admin";
43   return normalized || null;
44 }
45
46 function coerceId(value) {
47   if (value === null || value === undefined || value === "") return null;
48   const num = Number(value);
49   return Number.isFinite(num) ? num : value;
50 }
51
52 function buildReturnToUrl(currentHref, eventMeta, selectedService) {
53   const sanitized = sanitizeReturnTo(currentHref);
54   try {
55     const parsed = new URL(sanitized, window.location.origin);
56     if (eventMeta?.eventId && !parsed.searchParams.get("eventId")) {
57       parsed.searchParams.set("eventId", String(eventMeta.eventId));
58     }
59     if (eventMeta?.eventName && !parsed.searchParams.get("eventName")) {
60       parsed.searchParams.set("eventName", eventMeta.eventName);
61     }
62     if (eventMeta?.venue && !parsed.searchParams.get("venue")) {
63       parsed.searchParams.set("venue", eventMeta.venue);
64     }
65     if (eventMeta?.city && !parsed.searchParams.get("city")) {
66       parsed.searchParams.set("city", eventMeta.city);
67     }
68     const serviceParam = mapServiceParam(selectedService);
69     if (serviceParam && !parsed.searchParams.get("service")) {

```



```
70     parsed.searchParams.set("service", serviceParam);
71   }
72   return parsed.toString();
73 } catch {
74   return sanitized;
75 }
76 }
77
78 function resolvePortalBase() {
79   const envBase =
80     process.env.NEXT_PUBLIC_PORTAL_URL || process.env.NEXT_PUBLIC_ACCESS_PORTAL_URL;
81   if (envBase) return envBase;
82   return "/";
83 }
84
85 function getLoginUrl(returnTo) {
86   const portalBase = resolvePortalBase();
87   const baseUrl = portalBase.startsWith("http")
88     ? new URL(portalBase)
89     : new URL(portalBase, window.location.origin);
90   baseUrl.searchParams.delete("returnTo");
91   if (returnTo) {
92     const sanitized = sanitizeReturnTo(returnTo);
93     const baseId = `${baseUrl.origin}${baseUrl.pathname}`;
94     if (sanitized && sanitized !== baseId) {
95       baseUrl.searchParams.set("returnTo", sanitized);
96     }
97   }
98   return baseUrl.toString();
99 }
100
101 function formatCountdown(ms) {
102   if (ms <= 0) return "0:00";
103   const totalSeconds = Math.floor(ms / 1000);
104   const minutes = Math.floor(totalSeconds / 60);
105   const seconds = String(totalSeconds % 60).padStart(2, "0");
106   return `${minutes}:${seconds}`;
107 }
108
109 function clearDashboardAuthCache() {
110   if (typeof window === "undefined") return;
111   window.localStorage.removeItem("atomx.portal.token");
112   window.localStorage.removeItem(DASHBOARD_SELECTED_TOKEN_KEY);
113   window.localStorage.removeItem("atomx.dashboard.store");
114   const keysToRemove = [];
115   for (let i = 0; i < window.localStorage.length; i += 1) {
116     const key = window.localStorage.key(i);
117     if (key && key.startsWith("atomx.auth.")) {
118       keysToRemove.push(key);
119     }
120   }
121   keysToRemove.forEach((key) => window.localStorage.removeItem(key));
122 }
123
124 function isDashboardAuthStorageKey(key) {
125   return key === DASHBOARD_SELECTED_TOKEN_KEY;
126 }
127
128 function persistDashboardToken(token) {
129   if (typeof window === "undefined" || !token) return;
130   try {
131     window.localStorage.setItem(DASHBOARD_SELECTED_TOKEN_KEY, token);
132     const params = new URLSearchParams(window.location.search);
133     const service = params.get("service");
134     if (service) {
135       window.localStorage.setItem(`atomx.auth.${service}`, token);
136     }
137   } catch (err) {
138     console.error("Failed to persist dashboard token", err);
139   }
140 }
```

```
140 }
141
142 function readTokenFromUrl() {
143   if (typeof window === "undefined") return null;
144   return new URLSearchParams(window.location.search).get("token");
145 }
146
147 function removeTokenFromUrl() {
148   if (typeof window === "undefined") return;
149   const url = new URL(window.location.href);
150   if (!url.searchParams.has("token")) return;
151   url.searchParams.delete("token");
152   window.history.replaceState(window.history.state, "", url.toString());
153 }
154
155 function readStoredDashboardToken() {
156   if (typeof window === "undefined") return null;
157   try {
158     const selectedToken = window.localStorage.getItem(DASHBOARD_SELECTED_TOKEN_KEY);
159     if (selectedToken) return selectedToken;
160   } catch (err) {
161     console.error("Failed to read dashboard auth token", err);
162   }
163   return null;
164 }
165
166 export default function SessionGuard() {
167   const token = useDashboardStore((state) => state.token);
168   const profile = useDashboardStore((state) => state.profile);
169   const selectedService = useDashboardStore((state) => state.selectedService);
170   const eventMeta = useDashboardStore((state) => state.eventMeta);
171   const setToken = useDashboardStore((state) => state.setToken);
172   const setSelectedService = useDashboardStore((state) => state.setSelectedService);
173   const [warningOpen, setWarningOpen] = useState(false);
174   const [expiredOpen, setExpiredOpen] = useState(false);
175   const [countdown, setCountdown] = useState("");
176   const timersRef = useRef({ warn: null, expire: null, tick: null });
177   const reloginRef = useRef(false);
178
179   useEffect(() => {
180     const urlToken = readTokenFromUrl();
181     if (urlToken) {
182       persistDashboardToken(urlToken);
183       removeTokenFromUrl();
184     }
185     const nextToken = urlToken || readStoredDashboardToken();
186     if (nextToken && nextToken !== token) {
187       setToken(nextToken);
188       return;
189     }
190     if (!nextToken && token) {
191       setToken(null);
192     }
193   }, [setToken, token]);
194
195   const expiresAt = useMemo(() => {
196     if (!token) return null;
197     try {
198       const decoded = decodeJwt(token);
199       if (!decoded?.exp) return null;
200       return decoded.exp * 1000;
201     } catch (err) {
202       console.error("Failed to decode token", err);
203       return null;
204     }
205   }, [token]);
206
207   useEffect(() => {
208     if (!token) return;
209     try {
```

```

210     const decoded = decodeJwt(token);
211     const email = decoded?.email ?? null;
212     const distinctId = decoded?.sub ?? decoded?.id ?? email;
213     identifyPostHogUser(distinctId, {
214         app: "dashboard",
215         email,
216         name: decoded?.name ?? null,
217         role_type: decoded?.ctx?.type ?? decoded?.type ?? null
218     });
219     capturePostHogEvent("session_identified", {
220         app: "dashboard",
221         user_email: email,
222         role_type: decoded?.ctx?.type ?? decoded?.type ?? null
223     });
224     } catch (err) {
225         console.error("Failed to identify dashboard user in PostHog", err);
226     }
227 }, [token]);
228
229 const startReLogin = useCallback(() => {
230     if (reloginRef.current) return;
231     if (typeof window === "undefined") return;
232     const returnTo = buildReturnToUrl(window.location.href, eventMeta, selectedService);
233     let decodedProfile = profile;
234     if (!decodedProfile && token) {
235         try {
236             decodedProfile = decodeJwt(token);
237         } catch (err) {
238             console.error("Failed to decode token for reauth context", err);
239         }
240     }
241     const roles = Array.isArray(decodedProfile?.roles) ? decodedProfile.roles : [];
242     const ctxType = decodedProfile?.ctx?.type;
243     const normalizedType = normalizeRoleType(ctxType || selectedService || "");
244     const roleMatch = roles.find(
245         (role) => normalizeRoleType(role?.type) === normalizedType
246     );
247     const payload = {
248         version: 1,
249         createdAt: Date.now(),
250         expiresAt: Date.now() + REAUTH_CONTEXT_TTL_MS,
251         returnTo,
252         type: roleMatch?.type || ctxType || selectedService || null,
253         service: mapServiceParam(roleMatch?.type || ctxType || selectedService),
254         eventId: coerceId(eventMeta?.eventId ?? decodedProfile?.ctx?.eventId ?? roleMatch?.eventId),
255         adminId: coerceId(decodedProfile?.ctx?.adminId ?? roleMatch?.adminId)
256     };
257     try {
258         window.localStorage.setItem(REAUTH_CONTEXT_KEY, JSON.stringify(payload));
259     } catch (err) {
260         console.error("Failed to persist reauth context", err);
261     }
262     const loginUrl = getLoginUrl(returnTo);
263     capturePostHogEvent("session_relogin_start", {
264         app: "dashboard",
265         service: mapServiceParam(payload.type),
266         event_id: payload.eventId ?? null,
267         admin_id: payload.adminId ?? null,
268         return_to: returnTo
269     });
270     relloginRef.current = true;
271     clearDashboardAuthCache();
272     setToken(null);
273     setSelectedService(null);
274     window.location.assign(loginUrl);
275 }, [eventMeta, profile, selectedService, setSelectedService, setToken, token]);
276
277 const scheduleTimers = useCallback(() => {
278     if (!expiresAt) return;
279     const now = Date.now();

```

```

280     const warnAt = expiresAt - WARNING_WINDOW_MS;
281     const warnDelay = Math.max(warnAt - now, 0);
282     const expireDelay = Math.max(expiresAt - now, 0);
283
284     if (timersRef.current.warn) clearTimeout(timersRef.current.warn);
285     if (timersRef.current.expire) clearTimeout(timersRef.current.expire);
286     if (timersRef.current.tick) clearInterval(timersRef.current.tick);
287
288     timersRef.current.warn = setTimeout(() => {
289       setWarningOpen(true);
290     }, warnDelay);
291
292     timersRef.current.expire = setTimeout(() => {
293       setExpiredOpen(true);
294     }, expireDelay);
295
296     timersRef.current.tick = setInterval(() => {
297       const remaining = expiresAt - Date.now();
298       if (remaining <= 0) {
299         setCountdown("0:00");
300         return;
301       }
302       setCountdown(formatCountdown(remaining));
303     }, 1000);
304   }, [expiresAt, startRelogin]);
305
306   useEffect(() => {
307     if (!expiresAt) return;
308     scheduleTimers();
309     return () => {
310       if (timersRef.current.warn) clearTimeout(timersRef.current.warn);
311       if (timersRef.current.expire) clearTimeout(timersRef.current.expire);
312       if (timersRef.current.tick) clearInterval(timersRef.current.tick);
313     };
314   }, [expiresAt, scheduleTimers]);
315
316   useEffect(() => {
317     const onStorage = (event) => {
318       if (isDashboardAuthStorageKey(event.key)) {
319         const nextToken = event.newValue || readStoredDashboardToken();
320         setToken(nextToken || null);
321         if (nextToken) {
322           capturePostHogEvent("session_relogin_success", {
323             app: "dashboard",
324             source: "storage"
325           });
326         }
327         reloginRef.current = false;
328         setWarningOpen(false);
329         setExpiredOpen(false);
330       }
331     };
332     window.addEventListener("storage", onStorage);
333     return () => window.removeEventListener("storage", onStorage);
334   }, [setToken]);
335
336   useEffect(() => {
337     const onMessage = (event) => {
338       if (event?.data?.type !== "atomx.auth") return;
339       const nextToken = event.data.token;
340       if (nextToken) {
341         persistDashboardToken(nextToken);
342         setToken(nextToken);
343       }
344       if (event.data.service) {
345         setSelectedService(event.data.service);
346       }
347       capturePostHogEvent("session_relogin_success", {
348         app: "dashboard",
349         source: "post_message",

```

```
350     service: event.data.service ?? null
351   });
352   reloginRef.current = false;
353   setWarningOpen(false);
354   setExpiredOpen(false);
355   };
356   window.addEventListener("message", onMessage);
357   return () => window.removeEventListener("message", onMessage);
358 }, [setToken, setSelectedService]);
359
360 if (!expiresAt) return null;
361
362 return (
363   <
364     <SessionPrompt
365       open={warningOpen && !expiredOpen}
366       title="Session expiring soon"
367       message="Your session will expire in"
368       countdown={countdown}
369       actionLabel="Re-login"
370       onAction={startRelogin}
371       onClose={() => setWarningOpen(false)}
372       allowDismiss
373     />
374     <SessionPrompt
375       open={expiredOpen}
376       title="Session expired"
377       message="Please re-login to continue."
378       actionLabel="Login"
379       onAction={startRelogin}
380       allowDismiss={false}
381     />
382   </>
383 );
384 }
```

FILE: apps/dashboard/src/app/components/Session/SessionPrompt.js

```

1  "use client";
2
3  export default function SessionPrompt({
4    open,
5    title,
6    message,
7    countdown,
8    actionLabel = "Re-login",
9    onAction,
10   onClose,
11   allowDismiss = true
12 }) {
13   if (!open) return null;
14
15   return (
16     <div className="fixed inset-0 z-[999] flex items-center justify-center bg-black/30 px-4">
17       <div className="w-full max-w-md rounded-xl border border-[#f2d9c8] bg-white p-6 shadow-[0_18px_40px_
18         _rgba(15,23,42,0.25)]">
19         <div className="flex items-start justify-between gap-3">
20           <div>
21             <h2 className="text-base font-semibold text-slate-800">{title}</h2>
22             <p className="mt-2 text-sm text-slate-600">
23               {message}
24               {countdown ? (
25                 <span className="ml-1 text-slate-800 font-semibold">{countdown}</span>
26               ) : null}
27             </p>
28           </div>
29           {allowDismiss ? (
30             <button
31               type="button"
32               onClick={onClose}
33               className="text-slate-400 hover:text-slate-600"
34               aria-label="Close"
35             >
36               <svg
37                 viewBox="0 0 24 24"
38                 className="h-4 w-4"
39                 fill="none"
40                 stroke="currentColor"
41                 strokeWidth="2"
42                 strokeLinecap="round"
43                 strokeLinejoin="round"
44               >
45                 <path d="M18 6L6 18" />
46                 <path d="M6 6L12 12" />
47               </svg>
48             </button>
49           ) : null}
50         </div>
51         <div className="mt-5 flex items-center justify-end gap-3">
52           <button
53             type="button"
54             onClick={onAction}
55             className="rounded-md bg-[color:rgb(var(--color-orange))] px-4 py-2 text-sm font-semibold tex
56             t-white shadow-[0_6px_12px_rgb(var(--color-orange)/0.25)]"
57           >
58             {actionLabel}
59           </button>
60         </div>
61       </div>
62     </div>
63   );
64 }

```

FILE: apps/dashboard/src/app/components/SideDrawer.js

```

1  "use client";
2
3  import { usePathname, useRouter } from "next/navigation";
4
5  const ITEMS = [
6    { id: "deviceMaster", label: "Device Master", href: "/device_masterlist", match: "/device_masterlist" }
7  ,
8    // { id: "analytics", label: "Analytics", disabled: true },
9    { id: "configuration", label: "Configuration", href: "/Config", match: "/Config" },
10   { id: "whitelist", label: "Whitelist", href: "/whitelist", match: "/whitelist" },
11   { id: "admin", label: "Admin", href: "/admin/Create_event", match: "/admin/Create_event" },
12   { id: "reports", label: "Reports", href: "/Reports", match: "/Reports" },
13   { id: "transactions", label: "Transactions", href: "/transactions", match: "/transactions" },
14   { id: "device", label: "Devices", href: "/device", match: "/device" },
15   { id: "blocked", label: "Blocked", href: "/Blocked", match: "/Blocked" },
16   // { id: "apk", label: "APK Uploads", href: "/apk_upload", match: "/apk_upload" },
17   // { id: "tapx", label: "TapX-Transactions", href: "/tapx", match: "/tapx" },
18   // { id: "patchaTrack", label: "Patcha-NY-Track", href: "/patcha-ny-track", match: "/patcha-ny-track" }
19 ];
20
21 const iconClass = "h-4 w-4";
22
23 const ICONS = {
24   deviceMaster: (
25     <svg viewBox="0 0 24 24" className={iconClass} fill="none" stroke="currentColor" strokeWidth="1.9" strokeLinecap="round" strokeLinejoin="round">
26       <rect x="5" y="3" width="14" height="18" rx="2" />
27       <path d="M9 8h6" />
28       <path d="M9 12h6" />
29       <path d="M9 16h3" />
30     </svg>
31   ),
32   analytics: (
33     <svg viewBox="0 0 24 24" className={iconClass} fill="none" stroke="currentColor" strokeWidth="1.9" strokeLinecap="round" strokeLinejoin="round">
34       <path d="M4 19h16" />
35       <path d="M7 16V9" />
36       <path d="M12 16V5" />
37       <path d="M17 16v-4" />
38     </svg>
39   ),
40   configuration: (
41     <svg viewBox="0 0 24 24" className={iconClass} fill="none" stroke="currentColor" strokeWidth="1.9" strokeLinecap="round" strokeLinejoin="round">
42       <path d="M4 7h4" />
43       <path d="M14 7h6" />
44       <path d="M10 5v4" />
45       <path d="M4 12h9" />
46       <path d="M19 12h1" />
47       <path d="M15 10v4" />
48       <path d="M4 17h2" />
49       <path d="M12 17h8" />
50       <path d="M8 15v4" />
51     </svg>
52   ),
53   whitelist: (
54     <svg viewBox="0 0 24 24" className={iconClass} fill="none" stroke="currentColor" strokeWidth="1.9" strokeLinecap="round" strokeLinejoin="round">
55       <path d="M12 3 20 6.3v5.9c0 4.7-3.2 7.8-8 9.3-4.8-1.5-8-4.6-8-9.3V6.3L12 3Z" />
56       <path d="m8.7 12.2 2.1 2.1 4.5-5" />
57     </svg>
58   ),
59   tapx: (
60     <svg viewBox="0 0 24 24" className={iconClass} fill="none" stroke="currentColor" strokeWidth="1.9" strokeLinecap="round" strokeLinejoin="round">
61       <rect x="7" y="3" width="10" height="18" rx="2" />
62       <path d="M10 7.5a3.5 3.5 0 0 1 3.5 3.5" />
63       <path d="M10 10.5a5.5 5.5 0 0 1 5.5 5.5" />
64       <path d="M11 17h2" />

```

```

64     </svg>
65   ),
66   patchaTrack: (
67     <svg viewBox="0 0 24 24" className={iconClass} fill="none" stroke="currentColor" strokeWidth="1.9" st
rokeLinecap="round" strokeLinejoin="round">
68       <circle cx="6" cy="18" r="2" />
69       <circle cx="18" cy="6" r="2" />
70       <path d="M8 18h3a3 3 0 0 3-3v-6a3 3 0 0 1 3-3" />
71       <path d="m9 8 2-2-2-2" />
72     </svg>
73   ),
74   admin: (
75     <svg viewBox="0 0 24 24" className={iconClass} fill="none" stroke="currentColor" strokeWidth="1.9" st
rokeLinecap="round" strokeLinejoin="round">
76       <path d="M12 12a4 4 0 1 0 0-8 4 4 0 0 0 8 0" />
77       <path d="M4 21a8 8 0 0 1 16 0" />
78     </svg>
79   ),
80   events: (
81     <svg viewBox="0 0 24 24" className={iconClass} fill="none" stroke="currentColor" strokeWidth="1.9" st
rokeLinecap="round" strokeLinejoin="round">
82       <rect x="4" y="5" width="16" height="15" rx="2" />
83       <path d="M8 3v4" />
84       <path d="M16 3v4" />
85       <path d="M4 10h16" />
86       <path d="M8 14h3" />
87       <path d="M13 14h3" />
88     </svg>
89   ),
90   reports: (
91     <svg viewBox="0 0 24 24" className={iconClass} fill="none" stroke="currentColor" strokeWidth="1.9" st
rokeLinecap="round" strokeLinejoin="round">
92       <path d="M7 3h7l5 5v13H7V3Z" />
93       <path d="M14 3v5h5" />
94       <path d="M10 13h6" />
95       <path d="M10 17h5" />
96     </svg>
97   ),
98   transactions: (
99     <svg viewBox="0 0 24 24" className={iconClass} fill="none" stroke="currentColor" strokeWidth="1.9" st
rokeLinecap="round" strokeLinejoin="round">
100       <path d="M7 7h11" />
101       <path d="m15 4 3 3-3 3" />
102       <path d="M17 17h6" />
103       <path d="m9 14-3 3 3 3" />
104     </svg>
105   ),
106   device: (
107     <svg viewBox="0 0 24 24" className={iconClass} fill="none" stroke="currentColor" strokeWidth="1.9" st
rokeLinecap="round" strokeLinejoin="round">
108       <rect x="7" y="3" width="10" height="18" rx="2" />
109       <path d="M11 17h2" />
110     </svg>
111   ),
112   blocked: (
113     <svg viewBox="0 0 24 24" className={iconClass} fill="none" stroke="currentColor" strokeWidth="1.9" st
rokeLinecap="round" strokeLinejoin="round">
114       <circle cx="12" cy="12" r="8" />
115       <path d="M7 7l10 10" />
116     </svg>
117   ),
118   apk: (
119     <svg viewBox="0 0 24 24" className={iconClass} fill="none" stroke="currentColor" strokeWidth="1.9" st
rokeLinecap="round" strokeLinejoin="round">
120       <path d="M12 4v12" />
121       <path d="m7 9 5-5 5 5" />
122       <path d="M5 20h14" />
123     </svg>
124   )
125 };

```



```

126
127 export default function SideDrawer({ mobileOpen = false, onMobileClose }) {
128   const pathname = usePathname();
129   const router = useRouter();
130
131   return (
132     <
133       {mobileOpen ? (
134         <button
135           type="button"
136           className="fixed inset-x-0 bottom-0 z-[49] bg-[#0c0c0c]/45 backdrop-blur-[2px] min-[901px]:hidd
en"
137           style={{ top: "var(--header-h)" }}
138           onClick={onMobileClose}
139           aria-label="Close navigation overlay"
140         />
141       ) : null}
142
143       <aside
144         className={`group fixed left-0 z-50 transition-transform duration-300 ease-out max-[900px]:z-[60]
${
145           mobileOpen ? "max-[900px]:translate-x-0" : "max-[900px]:-translate-x-full"
146         }}
147         style={{ top: "var(--header-h)", height: "calc(100vh - var(--header-h))" }}
148       >
149         <div className="flex h-full w-[60px] flex-col overflow-hidden bg-(--rail) text-(--railText) shado
w-[16px_0_42px_rgba(0,0,0,0.28)] transition-[width] duration-300 ease-out group-hover:w-[248px] max-[900p
x]:w-[248px]">
150           <nav className="flex flex-1 flex-col gap-1 overflow-y-auto overflow-x-hidden px-2.5 py-3.5">
151             {ITEMS.map((item) => {
152               const matchBase = item.match || item.href;
153               const isActive = item.exact
154                 ? pathname === matchBase
155                 : Boolean(matchBase && pathname?.startsWith(matchBase));
156
157               return (
158                 <button
159                   key={item.id}
160                   type="button"
161                   onClick={() => {
162                     if (!item.href || item.disabled) return;
163                     router.push(item.href);
164                     onMobileClose?.();
165                   }}
166                   disabled={item.disabled}
167                   title={item.label}
168                   className={`flex h-10 w-full items-center gap-3.5 rounded-[10px] px-[9px] text-left tex
t-[0.84rem] transition-colors duration-200 ${
169                     isActive
170                       ? "bg-(--surface) font-semibold text-(--text)"
171                       : item.disabled
172                         ? "cursor-default font-normal text-(--railText)"
173                         : "font-normal text-(--railText) hover:bg-white/[0.09]"
174                   }}
175                 >
176                   <span className="flex h-5 w-5 shrink-0 items-center justify-center">
177                     {ICONS[item.id]}
178                   </span>
179                   <span className="whitespace-nowrap opacity-0 transition-opacity duration-200 group-hove
r:opacity-100 max-[900px]:opacity-100">
180                     {item.label}
181                   </span>
182                 </button>
183               );
184             })}
185           </nav>
186           <div className="flex items-center gap-3 border-t border-white/10 px-3 py-3">
187             <span className="font-chillax flex h-[34px] w-[34px] shrink-0 items-center justify-center rou
nded-full border border-white/15 bg-white/10 text-[0.85rem] font-semibold text-(--railText)">
188               N

```

```
189         </span>
190         <span className="font-vcr whitespace-nowrap text-[9px] tracking-[0.12em] text-white/50 opacity-0 transition-opacity duration-200 group-hover:opacity-100 max-[900px]:opacity-100">
191             AtomX Dashboard v3.0.0
192         </span>
193     </div>
194 </div>
195 </aside>
196 </>
197 );
198 }
```

FILE: apps/dashboard/src/app/device/components/AddPersoDeviceModal.js

```

1  "use client";
2
3  import { useEffect, useMemo, useRef, useState } from "react";
4  import { addPersoDevices, searchDeviceMasterlist } from "../../lib/dashboardApi";
5
6  function CloseIcon() {
7    return (
8      <svg viewBox="0 0 24 24" className="h-4.5 w-4.5" fill="none" stroke="currentColor" strokeWidth="2.3"
9        strokeLinecap="round" strokeLinejoin="round">
10        <path d="M18 6 6 18" />
11        <path d="M6 6 12 12" />
12      </svg>
13    );
14  }
15
16  function SearchIcon() {
17    return (
18      <svg viewBox="0 0 24 24" className="h-4 w-4" fill="none" stroke="currentColor" strokeWidth="2" stroke
19        Linecap="round" strokeLinejoin="round">
20        <circle cx="11" cy="11" r="7" />
21        <path d="m20 20-3.5-3.5" />
22      </svg>
23    );
24  }
25
26  function getKey(device) {
27    return String(device?.id ?? device?.printId ?? device?.androidId ?? device?.hardwareId ?? "");
28  }
29
30  function getDevicePayloadId(device) {
31    const id = Number(device?.id);
32    return Number.isFinite(id) ? id : null;
33  }
34
35  function getDeviceLabel(device) {
36    return String(device?.printId ?? device?.id ?? device?.description ?? "-");
37  }
38
39  function getDeviceMeta(device) {
40    return [device?.description, device?.hardwareId, device?.bank, device?.type]
41      .filter(Boolean)
42      .join(" | ");
43  }
44
45  function SelectedDevicePill({ device, onRemove }) {
46    return (
47      <span className="inline-flex h-8 items-center gap-2 rounded-full border border-[#ded4ff] bg-white pl-
48        3 pr-1.5 text-[0.74rem] font-bold text-[#252525] shadow-[0_0px_14px_rgba(15,23,42,0.05)] max-[640px]:h-7
49        max-[640px]:text-[0.64rem]">
50        {getDeviceLabel(device)}
51        <button
52          type="button"
53          onClick={() => onRemove(device)}
54          className="grid h-5 w-5 place-items-center rounded-full bg-[#1c1c1c] text-white transition hover:
55            bg-[#E04420] max-[640px]:h-4.5 max-[640px]:w-4.5"
56          aria-label={`Remove device ${getDeviceLabel(device)}`}
57        >
58          <CloseIcon />
59        </button>
60      </span>
61    );
62  }
63
64  export default function AddPersoDeviceModal({ eventId, token, onClose, onAdded }) {
65    const overlayRef = useRef(null);
66    const requestIdRef = useRef(0);
67    const [query, setQuery] = useState("");
68    const [results, setResults] = useState([]);
69    const [selectedDevices, setSelectedDevices] = useState([]);

```

```
65 const [isSearching, setIsSearching] = useState(false);
66 const [searchError, setSearchError] = useState("");
67 const [submitError, setSubmitError] = useState("");
68 const [isSubmitting, setIsSubmitting] = useState(false);
69
70 useEffect(() => {
71   const previousOverflow = document.body.style.overflow;
72   const handleKeyDown = (event) => {
73     if (event.key === "Escape") onClose?();
74   };
75
76   document.body.style.overflow = "hidden";
77   document.addEventListener("keydown", handleKeyDown);
78
79   return () => {
80     document.body.style.overflow = previousOverflow;
81     document.removeEventListener("keydown", handleKeyDown);
82   };
83 }, [onClose]);
84
85 useEffect(() => {
86   const search = query.trim();
87   requestIdRef.current += 1;
88   const requestId = requestIdRef.current;
89
90   if (search.length < 3) {
91     setResults([]);
92     setIsSearching(false);
93     setSearchError("");
94     return undefined;
95   }
96
97   setIsSearching(true);
98   setSearchError("");
99
100   const timeoutId = window.setTimeout(async () => {
101     try {
102       const list = await searchDeviceMasterlist({ search, token, dedupe: false });
103       if (requestId === requestIdRef.current) {
104         setResults(Array.isArray(list) ? list : []);
105       }
106     } catch (error) {
107       console.error("Perso device search failed", error);
108       if (requestId === requestIdRef.current) {
109         setResults([]);
110         setSearchError("Unable to search devices.");
111       }
112     } finally {
113       if (requestId === requestIdRef.current) {
114         setIsSearching(false);
115       }
116     }
117   }, 450);
118
119   return () => window.clearTimeout(timeoutId);
120 }, [query, token]);
121
122 const selectedIds = useMemo(
123   () =>
124     selectedDevices
125       .map(getDevicePayloadId)
126       .filter((deviceId) => deviceId !== null && deviceId !== undefined),
127   [selectedDevices]
128 );
129
130 const addDevice = (device) => {
131   const key = getDeviceKey(device);
132   if (!key) return;
133
134   setSelectedDevices((current) => {
```

```

135     if (current.some((item) => getDeviceKey(item) === key)) return current;
136     return [...current, device];
137   });
138   setSubmitError("");
139 };
140
141 const removeDevice = (device) => {
142   const key = getDeviceKey(device);
143   setSelectedDevices((current) => current.filter((item) => getDeviceKey(item) !== key));
144 };
145
146 const handleConfirm = async () => {
147   if (!eventId) {
148     setSubmitError("Missing event.");
149     return;
150   }
151
152   if (selectedIds.length === 0) {
153     setSubmitError("Select at least one device.");
154     return;
155   }
156
157   setIsSubmitting(true);
158   setSubmitError("");
159
160   try {
161     await addPersoDevices({ eventId, token, devices: selectedIds });
162     await onAdded?();
163     onClose?();
164   } catch (error) {
165     console.error("Add perso devices failed", error);
166     setSubmitError("Unable to add selected devices.");
167   } finally {
168     setIsSubmitting(false);
169   }
170 };
171
172 return (
173   <div
174     ref={overlayRef}
175     onClick={event => {
176       if (event.target === overlayRef.current) onClose?();
177     }}
178     className="fixed inset-0 z-[230] flex h-dvh items-center justify-center overflow-hidden bg-[#1c1c1c]
179 /45 px-4 py-8 backdrop-blur-[3px] max-[640px]:px-3 max-[640px]:py-3"
180   >
181     <div className="flex max-h-[calc(100dvh-64px)] w-full max-w-[760px] flex-col overflow-hidden rounded-
182 d-xl border border-[#d5b7ff]/70 bg-white shadow-[0_30px_80px_rgba(15,23,42,0.25)] max-[640px]:max-h-[calc
183 (100dvh-24px)]">
184       <div className="flex shrink-0 items-start justify-between border-b border-[#ececfc] bg-[#1c1c1c]
185 px-6 py-4 text-white max-[640px]:px-4 max-[640px]:py-3">
186         <div className="min-w-0">
187           <p className="m-0 text-[0.62rem] font-bold uppercase tracking-[0.24em] text-[#D5B7FF] max-[64
188 0px]:text-[0.52rem]">
189             Perso Setup
190           </p>
191           <h2 className="m-0 mt-1 truncate text-[1.14rem] font-semibold tracking-wide max-[640px]:text-
192 [0.92rem]">
193             Add Perso Device
194           </h2>
195         </div>
196         <button
197           type="button"
198           onClick={onClose}
199           className="ml-4 grid h-9 w-9 shrink-0 place-items-center rounded-lg border border-white/10 te
200 xt-white/75 transition hover:bg-white/10 hover:text-white max-[640px]:h-8 max-[640px]:w-8"
201           aria-label="Close add perso device modal"
202         >
203           <CloseIcon />
204         </button>
205       </div>
206     </div>
207   )

```

```

198     </div>
199
200     <div className="min-h-0 flex-1 overflow-y-auto px-6 py-5 max-[640px]:px-4 max-[640px]:py-4">
201       <label className="block">
202         <span className="mb-1.5 block text-[0.72rem] font-semibold uppercase tracking-[0.16em] text-[
#858585] max-[640px]:text-[0.58rem]">
203           Devices ({selectedDevices.length})
204         </span>
205         <div className="flex h-11 items-center gap-2 rounded-lg border border-[#dedede] bg-white px-3
.5 text-[#E04420] transition focus-within:border-[#E04420] focus-within:ring-3 focus-within:ring-[#E04420
]/10 max-[640px]:h-9 max-[640px]:px-3">
206           <SearchIcon />
207           <input
208             value={query}
209             onChange={(event) => setQuery(event.target.value)}
210             placeholder="Type 3 digits to search"
211             className="h-full min-w-0 flex-1 bg-transparent text-[0.82rem] font-medium text-[#1c1c1c]
outline-none placeholder:text-[#9ca3af] max-[640px]:text-[0.72rem]"
212           />
213         </div>
214       </label>
215
216       {selectedDevices.length > 0 ? (
217         <div className="mt-3 flex flex-wrap gap-2 rounded-xl border border-[#efeafc] bg-[#fbfbfd] p-2
.5">
218           {selectedDevices.map((device) => (
219             <SelectedDevicePill key={getDeviceKey(device)} device={device} onRemove={removeDevice} />
220           ))}
221         </div>
222       ) : null}
223
224       {query.trim().length >= 3 ? (
225         <div className="mt-3 rounded-xl border border-[#ececfc] bg-[#fafafa] p-3">
226           {isSearching ? (
227             <p className="m-0 text-[0.82rem] font-medium text-[#7b8495]">Searching devices...</p>
228           ) : searchError ? (
229             <p className="m-0 text-[0.82rem] font-semibold text-[#E04420]">{searchError}</p>
230           ) : results.length ? (
231             <div className="grid max-h-[230px] gap-2 overflow-y-auto pr-1 [scrollbar-width:thin] [scr
ollbar-color:#d5b7ff_transparent]">
232               {results.map((device, index) => {
233                 const key = getDeviceKey(device) || `${device?.printId ?? "device"}-${index}`;
234                 const selected = selectedDevices.some((item) => getDeviceKey(item) === getDeviceKey(d
evice));
235                 return (
236                   <button
237                     key={key}
238                     type="button"
239                     onClick={() => addDevice(device)}
240                     className={`rounded-lg border px-3 py-2 text-left transition ${
241                       selected
242                         ? "border-[#D5B7FF] bg-[#f6f2ff]"
243                         : "border-[#e4e4e4] bg-white hover:border-[#d5b7ff]"
244                     }`}
245                   >
246                     <span className="flex items-start justify-between gap-3">
247                       <span className="min-w-0">
248                         <span className="block truncate text-[0.82rem] font-semibold text-[#252525] m
ax-[640px]:text-[0.7rem]">
249                           {getDeviceLabel(device)}
250                         </span>
251                         <span className="mt-0.5 block truncate text-[0.68rem] font-medium text-[#7b84
95] max-[640px]:text-[0.6rem]">
252                           {getDeviceMeta(device) || "Device master record"}
253                         </span>
254                       </span>
255                       <span className="shrink-0 text-[0.68rem] font-bold uppercase tracking-[0.12em]
text-[#E04420] max-[640px]:text-[0.58rem]">
256                         {selected ? "Selected" : "Add"}
257                       </span>

```

```

258             </span>
259         </button>
260     );
261     }}}
262 </div>
263 ) : (
264     <p className="m-0 text-[0.82rem] font-medium text-[#7b8495]">No matching devices found.</
p>
265     )}
266 </div>
267 ) : (
268     <p className="m-0 mt-3 text-[0.72rem] font-medium text-[#8d8d8d]">
269         Enter at least 3 characters to search master devices.
270     </p>
271     )}
272
273     {submitError ? (
274         <p className="m-0 mt-3 rounded-lg bg-[#fff0ec] px-3 py-2 text-[0.78rem] font-semibold text-[#
E04420]">
275             {submitError}
276         </p>
277     ) : null}
278 </div>
279
280     <div className="flex shrink-0 justify-end border-t border-[#eeeeee] bg-[#fbfbfb] px-6 py-4 max-[6
40px]:px-4 max-[640px]:py-3">
281         <button
282             type="button"
283             onClick={handleConfirm}
284             disabled={isSubmitting}
285             className="h-11 rounded-lg bg-[#1c1c1c] px-8 text-[0.9rem] font-semibold text-white shadow-[0
_12px_24px_rgba(28,28,28,0.18)] transition hover:bg-[#E04420] disabled:cursor-not-allowed disabled:opacit
y-60 max-[640px]:h-9 max-[640px]:px-5 max-[640px]:text-[0.72rem]"
286         >
287             {isSubmitting ? "Adding..." : "Confirm"}
288         </button>
289     </div>
290 </div>
291 </div>
292 );
293 }

```

FILE: apps/dashboard/src/app/device/components/DeviceList.js

```

1  "use client";
2
3  import { useCallback, useEffect, useMemo, useState } from "react";
4  import { AtomXLoader } from "@atomx/global-components";
5  import { fetchEventDevices } from "../../lib/dashboardApi";
6  import { useDashboardStore } from "../../store/dashboardStore";
7
8  function SearchIcon() {
9    return (
10     <svg viewBox="0 0 16 16" className="h-3.5 w-3.5 shrink-0 opacity-50" fill="none" stroke="currentColor
" strokeWidth="1.5" strokeLinecap="round">
11       <circle cx="7" cy="7" r="4.6" />
12       <path d="M10.5 10.5 14 14" />
13     </svg>
14   );
15 }
16
17 function DeviceGlyph({ className = "h-3.5 w-3.5" }) {
18   return (
19     <svg viewBox="0 0 16 16" className={className} fill="none" stroke="currentColor" strokeWidth="1.4" st
rokeLinecap="round" strokeLinejoin="round">
20       <rect x="4.4" y="1.9" width="7.2" height="12.2" rx="1.4" />
21       <path d="M6.8 11.9h2.4" />
22     </svg>
23   );
24 }
25
26 function LockIcon() {
27   return (
28     <svg viewBox="0 0 16 16" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="1.4"
strokeLinecap="round" strokeLinejoin="round">
29       <rect x="3.4" y="7" width="9.2" height="6.6" rx="1.4" />
30       <path d="M5.6 7v5.2a2.4 2.4 0 0 1 4.8 0v7" />
31     </svg>
32   );
33 }
34
35 function EditIcon() {
36   return (
37     <svg viewBox="0 0 16 16" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="1.4"
strokeLinecap="round" strokeLinejoin="round">
38       <path d="M11.2 2.4l2.4 2.4-8 8H3.2v-2.4z" />
39     </svg>
40   );
41 }
42
43 function GridIcon() {
44   return (
45     <svg viewBox="0 0 16 16" className="h-4 w-4" fill="currentColor">
46       <rect x="2.4" y="2.4" width="4.6" height="4.6" rx="1.1" />
47       <rect x="9" y="2.4" width="4.6" height="4.6" rx="1.1" />
48       <rect x="2.4" y="9" width="4.6" height="4.6" rx="1.1" />
49       <rect x="9" y="9" width="4.6" height="4.6" rx="1.1" />
50     </svg>
51   );
52 }
53
54 function ListIcon() {
55   return (
56     <svg viewBox="0 0 16 16" className="h-4 w-4" fill="none" stroke="currentColor" strokeWidth="1.5" stro
keLinecap="round">
57       <path d="M5.4 4.4h8.2M5.4 8h8.2M5.4 11.6h8.2M2.6 4.4h.01M2.6 8h.01M2.6 11.6h.01" />
58     </svg>
59   );
60 }
61
62 function formatDeviceDate(value) {
63   if (!value) {
64     return { short: "-", full: "-" };

```



```

65   }
66   const date = new Date(value);
67   if (Number.isNaN(date.getTime())) {
68     const fallback = String(value);
69     return {
70       short: fallback.length > 18 ? `${fallback.slice(0, 18)}...` : fallback,
71       full: fallback
72     };
73   }
74 }
75
76 const formatted = new Intl.DateTimeFormat("en-GB", {
77   year: "numeric",
78   month: "2-digit",
79   day: "2-digit",
80   hour: "2-digit",
81   minute: "2-digit",
82   second: "2-digit",
83   hour12: false
84 })
85   .format(date)
86   .replace(",", "");
87
88 return {
89   short: formatted.length > 18 ? `${formatted.slice(0, 18)}...` : formatted,
90   full: formatted
91 };
92 }
93
94 function formatDeviceType(value) {
95   const type = String(value || "-").trim();
96   if (!type || type === "-") return "-";
97   return type.replace(/-/g, "_").toUpperCase();
98 }
99
100 function normalizeDevice(device, index) {
101   const printId = device?.devices_main_printId ?? device?.printId ?? device?.device_id ?? "-";
102   const loginAt = formatDeviceDate(device?.device_last_login_at);
103   const closedAt = formatDeviceDate(device?.device_day_closed_at ?? device?.device_event_closed_at);
104   const closed = Boolean(device?.device_day_closed || device?.device_event_closed);
105   const printer = String(device?.device_printer || "").trim();
106
107   return {
108     key: `${device?.devices_main_id ?? device?.device_id ?? printId ?? "device"}-${index}`,
109     id: device?.devices_main_id ?? device?.device_id ?? index + 1,
110     code: String(printId ?? "-"),
111     type: formatDeviceType(device?.device_type),
112     vendorTag: printer && printer.toLowerCase() !== "none" ? printer : "",
113     stall: device?.stall_name ?? "-",
114     stallFull: device?.stall_name ?? "-",
115     vendor: device?.vendor_name ?? "-",
116     loginAt: loginAt.short,
117     loginAtFull: loginAt.full,
118     version: device?.device_latest_app_version ?? "-",
119     closedAt: closedAt.full,
120     closed,
121     status: device?.device_status ?? "-",
122     hardwareId: device?.devices_main_hardwareId ?? "",
123     mainType: device?.devices_main_type ?? "",
124     stallId: device?.device_stall ?? "",
125     deviceId: device?.device_id ?? ""
126   };
127 }
128
129 function DeviceEmptyState({ message }) {
130   return (
131     <div className="grid min-h-[150px] place-items-center px-4 py-8 text-center text-[13px] font-medium t
ext-(--muted)">
132       {message}
133     </div>

```

```

134   );
135 }
136
137 function DeviceLoadingState() {
138   return (
139     <div className="grid min-h-[150px] place-items-center py-6">
140       <AtomXLoader label="Loading devices..." />
141     </div>
142   );
143 }
144
145 function ModeTag({ value }) {
146   return (
147     <span className="font-vcr inline-flex items-center whitespace-nowrap rounded-full border border-[rgba(0,169,242,0.4)] bg-[rgba(0,169,242,0.09)] px-2.5 py-1 text-[9px] tracking-[0.1em] text-[#0284c7]">
148       {value}
149     </span>
150   );
151 }
152
153 function ProviderTag({ value }) {
154   if (!value) return null;
155   return (
156     <span className="inline-flex items-center whitespace-nowrap rounded-full border border-[rgba(224,68,32,0.4)] bg-[rgba(224,68,32,0.08)] px-2.5 py-1 text-[10.5px] font-semibold text-(--orange)">
157       {value}
158     </span>
159   );
160 }
161
162 function ClosedBadge({ closed }) {
163   return (
164     <span
165       className={`font-vcr flex items-center justify-center rounded-[8px] px-[13px] py-[5px] text-[10px] tracking-[0.1em] ${
166         closed ? "bg-[rgba(0,169,242,0.11)] text-[#0284c7]" : "bg-[rgba(224,68,32,0.09)] text-(--orange)"
167       }`}
168     >
169       {closed ? "YES" : "NO"}
170     </span>
171   );
172 }
173
174 function MetaField({ label, value }) {
175   return (
176     <div className="min-w-0 flex-1 basis-[88px]">
177       <div className="font-vcr whitespace-nowrap text-[7.5px] tracking-[0.15em] text-(--faint)">{label}</div>
178       <div className="mt-0.5 truncate text-[12.5px] font-semibold">{value || ""}</div>
179     </div>
180   );
181 }
182
183 function DeviceCard({ device }) {
184   return (
185     <div
186       className="flex flex-col gap-[9px] rounded-[12px] border border-transparent p-[11px] shadow-(--shadow) transition-shadow duration-200 hover:shadow-(--shadowUp)"
187       style={{
188         background:
189           "linear-gradient(var(--surface),var(--surface)) padding-box, linear-gradient(135deg, rgba(224,68,32,0.34), rgba(139,92,246,0.26) 52%, rgba(0,169,242,0.22)) border-box"
190       }}
191     >
192       <div className="flex min-w-0 items-center gap-[11px]">
193         <span className="grid h-8 w-8 shrink-0 place-items-center rounded-[9px] bg-[linear-gradient(140deg, #e04420, #8b5cf6)] text-white">
194           <DeviceGlyph />
195         </span>
196         <div className="min-w-0">

```

```

197         <div className="font-vcr text-[9.5px] tracking-[0.08em] text-(--blue)">#{device.id}</div>
198         <div className="font-chillax mt-0.5 truncate text-[15px] font-semibold tracking-[-0.01em]">{dev
ice.code}</div>
199     </div>
200     <span className="ml-auto text-(--faint)">
201         <LockIcon />
202     </span>
203 </div>
204
205 <div className="flex flex-wrap items-center gap-[7px] border-b border-(--line2) pb-[11px]">
206     <ModeTag value={device.type} />
207     <ProviderTag value={device.vendorTag} />
208 </div>
209
210 <div className="flex flex-wrap gap-3">
211     <MetaField label="STALL" value={device.stall} />
212     <MetaField label="VENDOR" value={device.vendor} />
213     <MetaField label="LOGIN AT" value={device.loginAt} />
214     <MetaField label="VERSION" value={device.version} />
215 </div>
216
217 <div className="flex items-center justify-between gap-[10px]">
218     <div className="min-w-0">
219         <div className="font-vcr text-[7.5px] tracking-[0.15em] text-(--faint)">CLOSED AT</div>
220         <div className="mt-0.5 truncate text-[12.5px] font-semibold">{device.closedAt}</div>
221     </div>
222     <ClosedBadge closed={device.closed} />
223 </div>
224 </div>
225 );
226 }
227
228 function DeviceRow({ device }) {
229     return (
230         <div className="flex flex-wrap items-center gap-[14px] border-b border-(--line2) px-1.5 py-[11px] las
t:border-b-0 transition-colors duration-150 hover:bg-(--surface2)">
231             <div className="flex min-w-0 shrink-0 basis-[210px] items-center gap-[11px]">
232                 <span className="grid h-8 w-8 shrink-0 place-items-center rounded-[9px] bg-[linear-gradient(140deg,
g,#e04420,#8b5cf6)] text-white">
233                     <DeviceGlyph />
234                 </span>
235                 <div className="min-w-0">
236                     <div className="font-vcr text-[9.5px] tracking-[0.08em] text-(--blue)">#{device.id}</div>
237                     <div className="font-chillax mt-0.5 truncate text-[15px] font-semibold tracking-[-0.01em]">{dev
ice.code}</div>
238                 </div>
239                 <button
240                     type="button"
241                     className="ml-auto grid h-[30px] w-[30px] shrink-0 place-items-center rounded-[8px] border bord
er-(--line) text-(--muted) transition hover:border-(--orange) hover:text-(--orange)"
242                     aria-label={`Edit device ${device.code}`}
243                 >
244                     <EditIcon />
245                 </button>
246             </div>
247
248             <div className="flex shrink-0 basis-[220px] flex-wrap items-center gap-[7px]">
249                 <ModeTag value={device.type} />
250                 <ProviderTag value={device.vendorTag} />
251             </div>
252
253             <div className="flex min-w-0 flex-1 basis-[340px] flex-wrap gap-3">
254                 <MetaField label="STALL" value={device.stallFull || device.stall} />
255                 <MetaField label="VENDOR" value={device.vendor} />
256                 <MetaField label="LOGIN AT" value={device.loginAtFull || device.loginAt} />
257                 <MetaField label="VERSION" value={device.version} />
258                 <MetaField label="CLOSED AT" value={device.closedAt} />
259             </div>
260
261             <ClosedBadge closed={device.closed} />

```

```

262     </div>
263   );
264 }
265
266 export default function DeviceList() {
267   const token = useDashboardStore((state) => state.token);
268   const eventMeta = useDashboardStore((state) => state.eventMeta);
269   const eventDetails = useDashboardStore((state) => state.eventDetails);
270   const eventId = eventMeta?.eventId ?? eventDetails?.id;
271   const [query, setQuery] = useState("");
272   const [viewMode, setViewMode] = useState("list");
273   const [devices, setDevices] = useState([]);
274   const [isLoading, setIsLoading] = useState(false);
275   const [loadError, setLoadError] = useState("");
276
277   const loadDevices = useCallback(async () => {
278     if (!eventId) {
279       setDevices([]);
280       setLoadError("");
281       setIsLoading(false);
282       return;
283     }
284
285     setIsLoading(true);
286     setLoadError("");
287
288     try {
289       const eventDevices = await fetchEventDevices({ eventId, token });
290       setDevices(Array.isArray(eventDevices) ? eventDevices.map(normalizeDevice) : []);
291     } catch (error) {
292       console.error("Failed to load event devices", error);
293       setDevices([]);
294       setLoadError("Unable to load devices.");
295     } finally {
296       setIsLoading(false);
297     }
298   }, [eventId, token]);
299
300   useEffect(() => {
301     loadDevices();
302   }, [loadDevices]);
303
304   const filteredDevices = useMemo(() => {
305     const normalizedQuery = query.trim().toLowerCase();
306     if (!normalizedQuery) return devices;
307
308     return devices.filter((device) =>
309       [
310         device.id,
311         device.code,
312         device.type,
313         device.vendorTag,
314         device.stall,
315         device.vendor,
316         device.loginAt,
317         device.version,
318         device.closedAt,
319         device.status,
320         device.hardwareId,
321         device.mainType,
322         device.stallId,
323         device.deviceId
324       ]
325       .filter(Boolean)
326       .some((value) => String(value).toLowerCase().includes(normalizedQuery))
327     );
328   }, [devices, query]);
329
330   const renderDeviceContent = () => {
331     if (!eventId) {

```

```

332     return <DeviceEmptyState message="Select an event to load devices." />;
333 }
334
335 if (isLoading) {
336     return <DeviceLoadingState />;
337 }
338
339 if (loadError) {
340     return <DeviceEmptyState message={loadError} />;
341 }
342
343 if (filteredDevices.length === 0) {
344     return <DeviceEmptyState message="No devices found." />;
345 }
346
347 if (viewMode === "grid") {
348     return (
349         <div
350             className="grid gap-3 px-[15px] pb-[15px] pt-1 [grid-template-columns:repeat(auto-fill,minmax(m
in(100%,198px),1fr))]"
351             >
352             {filteredDevices.map((device) => (
353                 <DeviceCard key={device.key} device={device} />
354             ))}
355         </div>
356     );
357 }
358
359 return (
360     <div className="max-h-[520px] overflow-y-auto px-[15px] pb-3 [scrollbar-color:var(--line)_transpare
nt] [scrollbar-width:thin]">
361         {filteredDevices.map((device) => (
362             <DeviceRow key={device.key} device={device} />
363         ))}
364     </div>
365 );
366 };
367
368 return (
369     <section className="mt-[clamp(16px,2vw,22px)] overflow-hidden rounded-[14px] border border-(--line) b
order-l-[3px] border-l-(--orange) bg-(--surface) shadow-(--shadow)">
370         <div className="flex flex-wrap items-center gap-3 px-[15px] py-[14px]">
371             <span className="font-vcr grid h-[38px] w-[38px] shrink-0 place-items-center rounded-[10px] bg-[l
inear-gradient(140deg,#e04420,#8b5cf6)] text-[13px] text-white">
372                 {devices.length}
373             </span>
374             <h2 className="font-chillax whitespace-nowrap text-[18px] font-semibold tracking-[-0.01em]">Devic
e List</h2>
375
376             <label className="flex min-w-[150px] flex-1 items-center gap-2.5 border-b border-(--line) px-0.5
pb-[7px] text-(--muted) focus-within:border-(--orange)">
377                 <SearchIcon />
378                 <input
379                     value={query}
380                     onChange={(event) => setQuery(event.target.value)}
381                     placeholder="Search Device"
382                     className="h-full min-w-0 flex-1 bg-transparent text-[13px] text-(--text) outline-none placeh
older:text-(--faint)"
383                 />
384             </label>
385
386             <div className="flex shrink-0 gap-[3px] rounded-[11px] border border-(--line) bg-(--surface2) p-[
3px]">
387                 <button
388                     type="button"
389                     onClick={() => setViewMode("grid")}
390                     className={`grid h-[30px] w-[34px] place-items-center rounded-[8px] transition ${
391                         viewMode === "grid" ? "bg-(--text) text-(--bg)" : "text-(--muted)"
392                     }`>
393                     aria-label="Card view"

```

```
394         >
395         <GridIcon />
396     </button>
397     <button
398         type="button"
399         onClick={() => setViewMode("list")}
400         className={`grid h-[30px] w-[34px] place-items-center rounded-[8px] transition ${
401             viewMode === "list" ? "bg-(--text) text-(--bg)" : "text-(--muted)"
402         }`}
403         aria-label="List view"
404     >
405         <ListIcon />
406     </button>
407 </div>
408 </div>
409
410     {renderDeviceContent()}
411 </section>
412 );
413 }
```

FILE: apps/dashboard/src/app/device/components/PersoDevicesTable.js

```

1  "use client";
2
3  import { useCallback, useEffect, useMemo, useState } from "react";
4  import { AtomXLoader } from "@atomx/global-components";
5  import { fetchPersoDevices, removePersoDevice } from "../../lib/dashboardApi";
6  import { useDashboardStore } from "../../store/dashboardStore";
7  import AddPersoDeviceModal from "../AddPersoDeviceModal";
8
9  const COLUMNS = [
10   { key: "rowNumber", label: "#", sortable: false, className: "" },
11   { key: "device", label: "DEVICE", sortable: true, className: "" },
12   { key: "mac", label: "MAC", sortable: true, className: "" },
13   { key: "type", label: "TYPE", sortable: true, className: "" },
14   { key: "addedAt", label: "ADDED AT", sortable: true, className: "" },
15   { key: "status", label: "STATUS", sortable: false, className: "" },
16   { key: "action", label: "ADD / REMOVE", sortable: false, className: "text-right" }
17 ];
18
19 const GRID_TEMPLATE = "44px 1.1fr 1.4fr 1fr 1.3fr 0.9fr 96px";
20
21 function SearchIcon() {
22   return (
23     <svg viewBox="0 0 16 16" className="h-3.5 w-3.5 shrink-0 opacity-50" fill="none" stroke="currentColor
24 " strokeWidth="1.5" strokeLinecap="round">
25       <circle cx="7" cy="7" r="4.6" />
26       <path d="M10.5 10.5 14 14" />
27     </svg>
28   );
29 }
30
31 function PlusIcon() {
32   return (
33     <svg viewBox="0 0 14 14" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="1.9"
34     strokeLinecap="round">
35       <path d="M7 2.6v8.8M2.6 7h8.8" />
36     </svg>
37   );
38 }
39
40 function RemoveIcon() {
41   return (
42     <svg viewBox="0 0 14 14" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="1.6"
43     strokeLinecap="round">
44       <path d="M3.5 3.5l7 7M10.5 3.5l-7 7" />
45     </svg>
46   );
47 }
48
49 function SortArrows({ active, dir }) {
50   return (
51     <svg width="9" height="11" viewBox="0 0 9 11" fill="currentColor" className="shrink-0">
52       <path d="M4.5 0l3 3.4h-6z" opacity={active && dir === 1 ? "1" : "0.3"} />
53       <path d="M4.5 11l-3-3.4h6z" opacity={active && dir === -1 ? "1" : "0.3"} />
54     </svg>
55   );
56 }
57
58 function formatPersoDate(value) {
59   if (!value) return "-";
60
61   const date = new Date(value);
62   if (Number.isNaN(date.getTime())) {
63     return String(value);
64   }
65
66   return new Intl.DateTimeFormat("en-GB", {
67     year: "numeric",
68     month: "2-digit",
69     day: "2-digit",

```

```

67     hour: "2-digit",
68     minute: "2-digit",
69     second: "2-digit",
70     hour12: false
71   })
72   .format(date)
73   .replace(", ", "");
74 }
75
76 function normalizePersoDevice(device, index) {
77   const status = String(device?.status ?? "-").toLowerCase();
78
79   return {
80     key: `${device?.id ?? device?.device ?? "perso"}-${index}`,
81     id: device?.id ?? index + 1,
82     rowNumber: index + 1,
83     device: device?.device ?? "-",
84     mac: device?.device_mac ?? "-",
85     type: device?.device_type ?? "-",
86     addedAt: formatPersoDate(device?.created_at),
87     status
88   };
89 }
90
91 function PersoStatusBadge({ status }) {
92   const normalizedStatus = String(status ?? "-").toLowerCase();
93   const isInactive = normalizedStatus === "inactive";
94
95   return (
96     <span
97       className={`font-vcr inline-flex items-center rounded-full px-2.5 py-1 text-[9px] tracking-[0.12em]
98         ${
99           isInactive ? "bg-(--chip) text-(--faint)" : "bg-[rgba(0,169,242,0.1)] text-[#0284c7]"
100         }`
101       >
102       {(normalizedStatus || "-").toUpperCase()}
103     </span>
104   );
105 }
106
107 function PersoEmptyState({ message }) {
108   return (
109     <div className="rounded-[11px] border border-dashed border-(--line) px-4 py-8 text-center text-sm font-medium text-(--muted)">
110       {message}
111     </div>
112   );
113 }
114
115 function PersoLoadingState() {
116   return (
117     <div className="grid min-h-[120px] place-items-center">
118       <AtomXLoader label="Loading perso devices..." size={46} />
119     </div>
120   );
121 }
122
123 function PersoMobileCard({ item, onRemove, isRemoving }) {
124   const isInactive = item.status === "inactive";
125
126   return (
127     <div
128       className="rounded-[12px] border border-transparent p-[11px] shadow-(--shadow)"
129       style={{
130         background:
131           "linear-gradient(var(--surface),var(--surface)) padding-box, linear-gradient(135deg, rgba(224,68,32,0.32), rgba(139,92,246,0.24)) border-box"
132       }}
133     >
134       <div className="flex items-start justify-between gap-3">

```



```

134     <div className="min-w-0">
135       <p className="font-vcr text-[7.5px] tracking-[0.15em] text-(--faint)">DEVICE</p>
136       <p className="font-chillax mt-1 text-[15px] font-semibold">{item.device}</p>
137     </div>
138     <PersoStatusBadge status={item.status} />
139   </div>
140
141   <div className="mt-3 grid grid-cols-2 gap-3">
142     <div className="min-w-0">
143       <p className="font-vcr text-[7.5px] tracking-[0.14em] text-(--faint)">MAC</p>
144       <p className="font-vcr mt-1 truncate text-[12px] text-(--muted)">{item.mac}</p>
145     </div>
146     <div>
147       <p className="font-vcr text-[7.5px] tracking-[0.14em] text-(--faint)">TYPE</p>
148       <p className="mt-1 text-[12.5px] font-semibold">{item.type}</p>
149     </div>
150     <div className="col-span-2">
151       <p className="font-vcr text-[7.5px] tracking-[0.14em] text-(--faint)">ADDED AT</p>
152       <p className="font-vcr mt-1 text-[11.5px]">{item.addedAt}</p>
153     </div>
154   </div>
155
156   <button
157     type="button"
158     onClick={() => onRemove(item)}
159     disabled={isInactive || isRemoving}
160     className="mt-3 inline-flex h-8 items-center gap-2 rounded-[8px] border border-(--line) px-3 text
- [11px] font-semibold text-(--muted) transition hover:border-(--orange) hover:text-(--orange) disabled:cu
rsor-not-allowed disabled:opacity-45"
161     aria-label={`Remove perso device ${item.device}`}
162   >
163     <RemoveIcon />
164     {isRemoving ? "Removing..." : isInactive ? "Inactive" : "Remove"}
165   </button>
166 </div>
167 );
168 }
169
170 export default function PersoDevicesTable() {
171   const token = useDashboardStore((state) => state.token);
172   const eventMeta = useDashboardStore((state) => state.eventMeta);
173   const eventDetails = useDashboardStore((state) => state.eventDetails);
174   const eventId = eventMeta?.eventId ?? eventDetails?.id;
175   const [query, setQuery] = useState("");
176   const [devices, setDevices] = useState([]);
177   const [isLoading, setIsLoading] = useState(false);
178   const [loadError, setLoadError] = useState("");
179   const [isAddModalOpen, setIsAddModalOpen] = useState(false);
180   const [removingId, setRemovingId] = useState(null);
181   const [actionError, setActionError] = useState("");
182   const [sort, setSort] = useState({ key: "rowNumber", dir: 1 });
183
184   const loadPersoDevices = useCallback(async () => {
185     if (!eventId) {
186       setDevices([]);
187       setLoadError("");
188       setIsLoading(false);
189       return;
190     }
191
192     setIsLoading(true);
193     setLoadError("");
194
195     try {
196       const persoDevices = await fetchPersoDevices({ eventId, token });
197       setDevices(Array.isArray(persoDevices) ? persoDevices.map(normalizePersoDevice) : []);
198     } catch (error) {
199       console.error("Failed to load perso devices", error);
200       setDevices([]);
201       setLoadError("Unable to load perso devices.");

```

```

202     } finally {
203       setIsLoading(false);
204     }
205   }, [eventId, token]);
206
207   useEffect(() => {
208     loadPersoDevices();
209   }, [loadPersoDevices]);
210
211   const handleRemoveDevice = useCallback(async (item) => {
212     if (!eventId || !item?.id || item.status === "inactive") {
213       return;
214     }
215
216     setRemovingId(item.id);
217     setActionError("");
218
219     try {
220       await removePersoDevice({ eventId, token, id: item.id });
221       setDevices((currentDevices) =>
222         currentDevices.map((device) =>
223           device.id === item.id ? { ...device, status: "inactive" } : device
224         )
225       );
226       await loadPersoDevices();
227     } catch (error) {
228       console.error("Failed to remove perso device", error);
229       setActionError("Unable to mark perso device inactive.");
230     } finally {
231       setRemovingId(null);
232     }
233   }, [eventId, loadPersoDevices, token]);
234
235   const toggleSort = (key) => {
236     setSort((current) =>
237       current.key === key ? { key, dir: -current.dir } : { key, dir: 1 }
238     );
239   };
240
241   const filteredDevices = useMemo(() => {
242     const normalizedQuery = query.trim().toLowerCase();
243     const base = !normalizedQuery
244       ? devices
245       : devices.filter((item) =>
246         [item.id, item.device, item.mac, item.type, item.addedAt, item.status]
247           .filter(Boolean)
248           .some((value) => String(value).toLowerCase().includes(normalizedQuery))
249       );
250
251     const { key, dir } = sort;
252     return [...base].sort((a, b) => {
253       const av = key === "rowNumber" ? Number(a.rowNumber) : a[key];
254       const bv = key === "rowNumber" ? Number(b.rowNumber) : b[key];
255       if (av > bv) return dir;
256       if (av < bv) return -dir;
257       return 0;
258     });
259   }, [devices, query, sort]);
260
261   const renderDesktopRows = () => {
262     if (!eventId) return <PersoEmptyState message="Select an event to load perso devices." />;
263     if (isLoading) return <PersoLoadingState />;
264     if (loadError) return <PersoEmptyState message={loadError} />;
265     if (filteredDevices.length === 0) return <PersoEmptyState message="No perso devices found." />;
266
267     return (
268       <div className="overflow-x-auto px-[15px] pb-[15px]">
269         <div className="min-w-[640px]">
270           <div
271             className="grid gap-2.5 border-b border-(--line) px-2 pb-2.5"

```

```

272         style={{ gridTemplateColumns: GRID_TEMPLATE }}
273     >
274     {COLUMNS.map((col) => {
275         const active = sort.key === col.key;
276         return (
277             <button
278                 key={col.key}
279                 type="button"
280                 onClick={() => col.sortable && toggleSort(col.key)}
281                 className={`font-vcr flex items-center gap-1.5 whitespace-nowrap text-[9.5px] tracking-
[0.15em] text-(--orange) ${
282                     col.className
283                 } ${col.sortable ? "cursor-pointer" : "cursor-default"}`}
284             >
285                 <span>{col.label}</span>
286                 {col.sortable ? <SortArrows active={active} dir={sort.dir} /> : null}
287             </button>
288         );
289     })}
290 </div>
291
292 <div className="max-h-[360px] overflow-y-auto [scrollbar-color:var(--line)_transparent] [scroll
bar-width:thin]">
293     {filteredDevices.map((item) => (
294         <div
295             key={item.key}
296             className={`grid items-center gap-2.5 border-b border-(--line2) px-2 py-[11px] transition
-colors duration-150 hover:bg-(--surface2) ${
297                 item.status === "inactive" ? "opacity-70" : ""
298             }`}
299             style={{ gridTemplateColumns: GRID_TEMPLATE }}
300         >
301             <div className="font-vcr text-[12px] text-(--muted)">{item.rowNumber}</div>
302             <div className="truncate text-[13.5px] font-semibold">{item.device}</div>
303             <div className="font-vcr truncate text-[12px] text-(--muted)">{item.mac}</div>
304             <div className="text-[13.5px] text-(--muted)">{item.type}</div>
305             <div className="font-vcr whitespace-nowrap text-[11.5px]">{item.addedAt}</div>
306             <div>
307                 <PersoStatusBadge status={item.status} />
308             </div>
309             <div className="flex justify-end">
310                 <button
311                     type="button"
312                     onClick={() => handleRemoveDevice(item)}
313                     disabled={item.status === "inactive" || removingId === item.id}
314                     className="grid h-[30px] w-[30px] place-items-center rounded-[8px] border border-(--l
ine) text-(--muted) transition hover:border-(--orange) hover:text-(--orange) disabled:cursor-not-allowed
disabled:opacity-45"
315                     aria-label={`Remove perso device ${item.device}`}
316                 >
317                     <RemoveIcon />
318                 </button>
319             </div>
320         </div>
321     ))}
322 </div>
323 </div>
324 </div>
325 );
326 };
327
328 const renderMobileRows = () => {
329     if (!eventId) return <PersoEmptyState message="Select an event to load perso devices." />;
330     if (isLoading) return <PersoLoadingState />;
331     if (loadError) return <PersoEmptyState message={loadError} />;
332     if (filteredDevices.length === 0) return <PersoEmptyState message="No perso devices found." />;
333
334     return filteredDevices.map((item) => (
335         <PersoMobileCard
336             key={item.key}

```

```

337     item={item}
338     onRemove={handleRemoveDevice}
339     isRemoving={removingId === item.id}
340   />
341   ));
342 };
343
344   return (
345     <section className="mt-[clamp(14px,1.8vw,20px)] overflow-hidden rounded-[14px] border border-(--line)
border-l-[3px] border-l-(--orange) bg-(--surface) shadow-(--shadow)">
346       <div className="flex flex-wrap items-center gap-3 px-[15px] py-[14px]">
347         <span className="font-vcr grid h-[38px] w-[38px] shrink-0 place-items-center rounded-[10px] bg-[l
inear-gradient(140deg,#e04420,#8b5cf6)] text-[13px] text-white">
348           {devices.length}
349         </span>
350         <div className="min-w-0">
351           <h2 className="font-chillax text-[18px] font-semibold tracking-[-0.01em]">Perso Devices</h2>
352           <p className="font-vcr mt-0.5 text-[9px] tracking-[0.15em] text-(--muted)">
353             PERSO DEVICES ({devices.length})
354           </p>
355         </div>
356         <div className="flex-1" />
357         <button
358           type="button"
359           onClick={() => setIsAddModalOpen(true)}
360           className="inline-flex h-11 shrink-0 items-center justify-center gap-2 rounded-[10px] bg-(--tex
t) px-[18px] text-[13px] font-semibold text-(--bg) transition hover:bg-(--orange) max-[640px]:w-full"
361         >
362           <PlusIcon />
363           Add Device
364         </button>
365       </div>
366
367       <div className="px-[15px] pb-3">
368         <label className="flex h-11 items-center gap-2.5 rounded-[11px] border border-(--line) bg-(--surf
ace2) px-3.5 text-(--muted) focus-within:border-(--orange)">
369           <span className="sr-only">Search perso device</span>
370           <SearchIcon />
371           <input
372             value={query}
373             onChange={(event) => setQuery(event.target.value)}
374             placeholder="Search Device"
375             className="h-full min-w-0 flex-1 bg-transparent text-[13px] text-(--text) outline-none placeh
older:text-(--faint)"
376           />
377         </label>
378       </div>
379
380       {actionError ? (
381         <p className="mx-[15px] mb-2 rounded-[10px] border border-[rgba(224,68,32,0.25)] bg-[rgba(224,68,
32,0.06)] px-3 py-2 text-[12px] font-semibold text-(--orange)">
382           {actionError}
383         </p>
384       ) : null}
385
386       <div className="hidden md:block">{renderDesktopRows()}</div>
387
388       <div className="grid max-h-[510px] gap-2.5 overflow-y-auto px-[15px] pb-[15px] [scrollbar-color:var
(--line)_transparent] [scrollbar-width:thin] md:hidden">
389         {renderMobileRows()}
390       </div>
391
392       {isAddModalOpen ? (
393         <AddPersoDeviceModal
394           eventId={eventId}
395           token={token}
396           onClose={() => setIsAddModalOpen(false)}
397           onAdded={loadPersoDevices}
398         />
399       ) : null}

```

```
400     </section>
401   );
402 }
```

FILE: apps/dashboard/src/app/device/page.js

```
1 import Header from "../components/Header";
2 import DeviceList from "../components/DeviceList";
3 import PersoDevicesTable from "../components/PersoDevicesTable";
4
5 export default function DevicePage() {
6   return (
7     <main
8       className="min-h-screen bg-(--bg) pb-10"
9       style={{ fontFamily: '"Poppins", system-ui, sans-serif' }}
10    >
11      <Header areaLabel="Devices" variant="portal" />
12
13      <div className="mx-auto w-full max-w-[1780px] pr-4 pl-[72px] pt-6 md:pr-7 md:pl-[88px] max-[900px]:
14 px-3">
15        <h1 className="font-chillax text-[clamp(26px,3.2vw,38px)] font-semibold leading-[1.05] tracking-[
16 -0.02em] text-(--text)">
17          Devices
18        </h1>
19        <p className="mt-2 text-[13.5px] font-light text-(--muted)">
20          Monitor device sessions, assigned stalls, wallet values, and close state.
21        </p>
22        <div className="mt-[clamp(16px,2vw,22px)] h-px w-full bg-(--line)" />
23
24        <DeviceList />
25        <PersoDevicesTable />
26      </div>
27    </main>
28  );
29 }
```

FILE: apps/dashboard/src/app/device_masterlist/components/DeviceBankCredentials.js

```
1 "use client";
2
3 import { Field } from "../DeviceFormModal";
4
5 export const BANK_DEFAULT_CREDENTIALS = {
6   "mswipe-atomx": {
7     bank_mswipe_username: "9400100344",
8     bank_mswipe_password: "555555",
9     bank_mswipe_verify_client_code: "ATOMXLIVE",
10    bank_mswipe_verify_user_id: "ATOMXLIVE@SOL",
11    bank_mswipe_verify_password: "ATOMX~DsD@281119",
12    bank_mswipe_upi_api: "sdk"
13  },
14  "mswipe-bms": {
15    bank_mswipe_username: "9300066508",
16    bank_mswipe_password: "555555",
17    bank_mswipe_verify_client_code: "BMSLIVE",
18    bank_mswipe_verify_user_id: "BMSLIVE@SOL",
19    bank_mswipe_verify_password: "BMSLIVE~DsD@061219",
20    bank_mswipe_upi_api: "sdk"
21  },
22  "utap-skypay-dxb": {
23    bank_mswipe_username: "9000036987",
24    bank_mswipe_password: "306090",
25    bank_mswipe_verify_client_code: "ATOMXLIVE",
26    bank_mswipe_verify_user_id: "ATOMXLIVE@SOL",
27    bank_mswipe_verify_password: "ATOMX~DsD@281119",
28    bank_mswipe_upi_api: "sdk"
29  },
30  "cashlez-bms-indonesia-demo": {
31    bank_sunmipay_merchant_name: "The Dragon",
32    bank_sunmipay_transaction_type: "SALE",
33    bank_sunmipay_aggregator_id: "pecinta mi",
34    bank_sunmipay_mobile_user_id: "test03",
35    bank_sunmipay_cashlez_package_name: "com.cashlez.android.garuda.allinone"
36  },
37  "cashlez-bms-indonesia": {
38    bank_sunmipay_merchant_name: "Bookmyshow ID",
39    bank_sunmipay_transaction_type: "SALE",
40    bank_sunmipay_aggregator_id: "BookmyshowID",
41    bank_sunmipay_mobile_user_id: "bookmyshowID",
42    bank_sunmipay_cashlez_package_name: "com.cashlez.android.garuda.allinone"
43  },
44  "worldline-atomx-axis": {
45    bank_worldline_tid: "0447232U",
46    bank_worldline_password: "12345678",
47    bank_worldline_sim_username: "atomx",
48    bank_worldline_sim_password: "U$er#09873",
49    bank_worldline_bank_function_password: "3226",
50    bank_worldline_customer_qr_action: "com.example.menusample.YOUR_ACTION_BQR",
51    bank_worldline_customer_card_action: "cn.desert.newpos.payui.master.YOUR_ACTION"
52  },
53  "mosambee-atomx-sbi": {
54    bank_mosambee_user_name: "5252871758",
55    bank_mosambee_password: "5880",
56    bank_mosambee_app_key: "cGjhE$@fdhj4675riesae",
57    bank_mosambee_merchant_code: "HYLZYQJV6ZXZU052J0K",
58    bank_mosambee_merchant_key: "7DB342C21C56902D31DC166C7B33BBE760AAD7D796B6300A"
59  },
60  "ezetap-atomx-axis": {
61    bank_ezetap_prod_app_key: "730077d9-de69-4961-85bb-6bfcfe3367c7",
62    bank_ezetap_merchant_name: "ATOMX CORPORATION PRIVATE LIMITED",
63    bank_ezetap_user_name: "7875676169"
64  },
65  "pinelabs-atomx-hdfc": {
66    bank_pinelabs_application_id: "43dab288a5d64e40b6b6aba51aaead59",
67    bank_pinelabs_user_id: "1234",
68    bank_pinelabs_smart_action: "com.pinelabs.masterapp.SERVER",
69    bank_pinelabs_smart_package: "com.pinelabs.masterapp"
```

```

70 }
71 };
72
73 export const DEFAULT_DEVICE_TYPES = [
74   ["wisepos+", "Wisepos+"],
75   ["Famoco", "Famoco"],
76   ["serialNFC", "GateX Turnstile Reader"],
77   ["sunmipay", "Cashlez Sunmipay"],
78   ["MF919", "Mosambee MF919"],
79   ["DX8000", "Mosambee | DX8000"],
80   ["ezetap", "Ezetap"],
81   ["worldline", "Worldline"],
82   ["airpay", "Airpay"],
83   ["pinelabsPax", "PinelabsPax"],
84   ["Sunmi", "Sunmi"],
85   ["utapSunmi", "utapSunmi"],
86   ["other", "Other"]
87 ];
88
89 export const DEFAULT_MODEL_TYPES = [
90   ["wisepos+", "Wisepos+"],
91   ["Famoco", "Famoco"],
92   ["gatex", "GateX Turnstile Reader"],
93   ["sunmipay", "Cashlez Sunmipay"],
94   ["MF919", "Mosambee MF919"],
95   ["DX8000", "Mosambee | DX8000"],
96   ["ezetap", "Ezetap"],
97   ["worldline", "Worldline"],
98   ["airpay", "Airpay"],
99   ["pinelabsPax", "PinelabsPax"],
100  ["Sunmi", "Sunmi"],
101  ["utapSunmi", "utapSunmi"],
102  ["other", "Other"]
103 ];
104
105 export const DEFAULT_BANK_TYPES = [
106   ["mswipe-atomx", "MSWIPE ATOMX"],
107   ["mswipe-bms", "MSWIPE BMS"],
108   ["utap-skypay-dxb", "UTAP SKYPAY DXB"],
109   ["cashlez-bms-indonesia", "CASHLEZ BMS INDONESIA"],
110   ["mosambee-atomx-sbi", "MOSAMBEE ATOMX SBI"],
111   ["ezetap-atomx-axis", "EZETAP ATOMX AXIS"],
112   ["worldline-atomx-axis", "WORLDLINE ATOMX AXIS"],
113   ["pinelabs-atomx-hdfe", "PINELABS ATOMX HDFE"],
114   ["airpay-atomx-axis", "AIRPAY ATOMX AXIS"],
115   ["cashlez-bms-indonesia-demo", "CASHLEZ DEMO INDO"],
116   ["none", "NONE"]
117 ];
118
119 export function getDefaultBankData(bank) {
120   return { ...(BANK_DEFAULT_CREDENTIALS[bank] ?? {}) };
121 }
122
123 export function sanitizeBankData(bank, bankData) {
124   const fields = Object.keys(BANK_DEFAULT_CREDENTIALS[bank] ?? {});
125   return Object.fromEntries(
126     fields.map((key) => [
127       key,
128       String(bankData?.[key] ?? BANK_DEFAULT_CREDENTIALS[bank][key] ?? "").trim()
129     ])
130   );
131 }
132
133 function getFieldLabel(key) {
134   return key.replace(/bank_/, "").replaceAll("_", " ").toUpperCase();
135 }
136
137 export default function DeviceBankCredentials({ bank, bankData, onChange }) {
138   const credentialKeys = Object.keys(BANK_DEFAULT_CREDENTIALS[bank] ?? {});
139   if (credentialKeys.length === 0) return null;

```



```
140
141   return (
142     <>
143     <div className="[grid-column:1/-1] mt-1 border-t border-(--line) pt-4">
144       <div className="font-vcr text-[9px] tracking-[0.16em] text-(--orange)">
145         BANK CREDENTIALS
146       </div>
147     </div>
148     {credentialKeys.map((key) => (
149       <Field
150         key={key}
151         label={getFieldLabel(key)}
152         name={key}
153         value={bankData?.[key] ?? ""}
154         onChange={onChange}
155         type={key.includes("password") ? "password" : "text"}
156         placeholder="Not set"
157       </>
158     ))}
159   </>
160 );
161 }
```

FILE: apps/dashboard/src/lib/apiConfig.js

```
1 export const DASHBOARD_API_KEY =  
2   process.env.NEXT_PUBLIC_DASHBOARD_API_KEY ??  
3   "pZebJlF_.dv3_prod.Iu7Zitu3X30C2R6-bVZtRXRu0DeiHY-j";
```

FILE: packages/lib/src/apiConfig.js

```
1 export const DEFAULT_BASE_URL = "https://dapi.atomx.in";
2
3 export function getBaseUrl() {
4   return (process.env.NEXT_PUBLIC_BASE_URL ?? DEFAULT_BASE_URL).replace(/\$/, "");
5 }
```

FILE: apps/dashboard/src/app/device_masterlist/components/AddDeviceModal.js

```
1 "use client";
2
3 import { useState } from "react";
4 import { addDeviceMasterlist } from "../../lib/dashboardApi";
5 import DeviceFormModal, { Field, SelectField } from "./DeviceFormModal";
6 import DeviceBankCredentials, {
7   DEFAULT_BANK_TYPES,
8   DEFAULT_DEVICE_TYPES,
9   DEFAULT_MODEL_TYPES,
10   getDefaultBankData,
11   sanitizeBankData
12 } from "./DeviceBankCredentials";
13
14 const SECTIONS = [
15   {
16     label: "Device Information",
17     hint: "IDENTITY & HARDWARE",
18     title: "DEVICE INFORMATION",
19     sub: "Identity, hardware and configuration for the new device."
20   }
21 ];
22
23 const EMPTY_FORM = {
24   device: "",
25   mac: "",
26   androidId: "",
27   reference: "",
28   description: "",
29   type: "",
30   model: "",
31   bank: "",
32   bankData: {}
33 };
34
35 export default function AddDeviceModal({ token, onClose, onCreated }) {
36   const [form, setForm] = useState(EMPTY_FORM);
37   const [dirty, setDirty] = useState(false);
38   const [saving, setSaving] = useState(false);
39   const [error, setError] = useState("");
40
41   const handleChange = (event) => {
42     const { name, value } = event.target;
43     setForm((current) => ({
44       ...current,
45       [name]: value,
46       ...(name === "bank"
47         ? {
48             bankData: getDefaultBankData(value)
49           }
50         : {}))
51     ));
52     setDirty(true);
53     setError("");
54   };
55
56   const handleBankFieldChange = (event) => {
57     const { name, value } = event.target;
58     setForm((current) => ({
59       ...current,
60       bankData: { ...(current.bankData ?? {}), [name]: value }
61     }));
62     setDirty(true);
63     setError("");
64   };
65
66   const handleSubmit = async (event) => {
67     event.preventDefault();
68
69     if (!String(form.device).trim()) {
```

```

70     setError("Device ID is required.");
71     return;
72 }
73 if (!String(form.mac).trim()) {
74     setError("MAC ID is required.");
75     return;
76 }
77 if (!form.type || !form.model || !form.bank) {
78     setError("Type, device model, and bank are required.");
79     return;
80 }
81
82 const timestamp = new Date().toISOString();
83 const payload = {
84     printId: String(form.device).trim(),
85     hardwareId: String(form.mac).trim(),
86     androidId: String(form.androidId).trim(),
87     type: String(form.type).trim(),
88     bank: String(form.bank).trim(),
89     model: String(form.model).trim(),
90     reference: String(form.reference).trim(),
91     description: String(form.description).trim(),
92     hasNfc: 1,
93     nfcType: "inbuilt",
94     updatedAt: timestamp,
95     createdAt: timestamp,
96     status: "active",
97     bankData: sanitizeBankData(form.bank, form.bankData)
98 };
99
100 setSaving(true);
101 setError("");
102 try {
103     const result = await addDeviceMasterlist({ token, payload });
104     const createdDevice =
105         result?.device ??
106         result?.data?.device ??
107         (result?.data && !Array.isArray(result.data) ? result.data : null) ??
108         (result && typeof result === "object" ? result : null);
109     onCreated?.(createdDevice ? { ...payload, ...createdDevice } : payload);
110 } catch (submitError) {
111     console.error("Failed to add master device", submitError);
112     setError(submitError?.message || "Unable to add device.");
113 } finally {
114     setSaving(false);
115 }
116 };
117
118 return (
119     <DeviceFormModal
120         eyebrow="ADD DEVICE"
121         heading="New Device"
122         sections={SECTIONS}
123         activeSection={0}
124         onSectionChange={() => {}}
125         activeTitle={SECTIONS[0].title}
126         activeSub={SECTIONS[0].sub}
127         dirty={dirty}
128         error={error}
129         saving={saving}
130         submitLabel="Create Device"
131         onClose={onClose}
132         onSubmit={handleSubmit}
133     >
134         <Field label="DEVICE" name="device" value={form.device} onChange={handleChange} placeholder="Not set" required />
135         <Field label="MAC" name="mac" value={form.mac} onChange={handleChange} placeholder="Not set" required />
136         <Field label="ANDROID ID" name="androidId" value={form.androidId} onChange={handleChange} placeholder="Not set" />

```

```
137     <Field label="REFERENCE" name="reference" value={form.reference} onChange={handleChange} placeholder="Not set" />
138     <Field label="DESCRIPTION" name="description" value={form.description} onChange={handleChange} placeholder="Not set" />
139     <SelectField label="TYPE" name="type" value={form.type} onChange={handleChange}>
140       <option value="">Select Type</option>
141       {DEFAULT_DEVICE_TYPES.map(([value, label]) => (
142         <option key={value} value={value}>
143           {label}
144         </option>
145       ))}
146     </SelectField>
147     <SelectField label="DEVICE MODEL" name="model" value={form.model} onChange={handleChange}>
148       <option value="">Select Device Model</option>
149       {DEFAULT_MODEL_TYPES.map(([value, label]) => (
150         <option key={value} value={value}>
151           {label}
152         </option>
153       ))}
154     </SelectField>
155     <SelectField label="BANK" name="bank" value={form.bank} onChange={handleChange}>
156       <option value="">Select Bank</option>
157       {DEFAULT_BANK_TYPES.map(([value, label]) => (
158         <option key={value} value={value}>
159           {label}
160         </option>
161       ))}
162     </SelectField>
163
164     <DeviceBankCredentials
165       bank={form.bank}
166       bankData={form.bankData}
167       onChange={handleBankFieldChange}
168     />
169   </DeviceFormModal>
170 );
171 }
```

FILE: apps/dashboard/src/app/device_masterlist/components/DeviceFormModal.js

```

1  "use client";
2
3  import { useEffect } from "react";
4
5  const INPUT_CLASS =
6    "h-[46px] w-full rounded-[10px] border border-(--line) bg-(--surface) px-3.5 text-[13.5px] font-medium
    text-(--text) outline-none transition focus:border-(--orange) focus:shadow-[0_0_3px_rgba(224,68,32,0.12
    )]";
7
8  const INPUT_DISABLED_CLASS =
9    "h-[46px] w-full rounded-[10px] border border-(--line) bg-(--surface2) px-3.5 text-[13.5px] font-medium
    text-(--faint) outline-none";
10
11 function FieldLabel({ children }) {
12   return (
13     <label className="font-vcr mb-[7px] block text-[8.5px] tracking-[0.16em] text-(--muted)">
14       {children}
15     </label>
16   );
17 }
18
19 export function Field({ label, name, value, onChange, type = "text", placeholder = "", required = false,
    readOnly = false, wide = false }) {
20   return (
21     <div className={`min-w-0 ${wide ? "[grid-column:1/-1]" : ""}`}>
22       <FieldLabel>{label}</FieldLabel>
23       <input
24         name={name}
25         value={value ?? ""}
26         onChange={onChange}
27         type={type}
28         placeholder={placeholder}
29         required={required}
30         readOnly={readOnly}
31         disabled={readOnly}
32         className={readOnly ? INPUT_DISABLED_CLASS : INPUT_CLASS}
33       />
34     </div>
35   );
36 }
37
38 export function SelectField({ label, name, value, onChange, children, wide = false }) {
39   return (
40     <div className={`min-w-0 ${wide ? "[grid-column:1/-1]" : ""}`}>
41       <FieldLabel>{label}</FieldLabel>
42       <div className="relative">
43         <select
44           name={name}
45           value={value}
46           onChange={onChange}
47           className={` ${INPUT_CLASS} cursor-pointer appearance-none pr-9`}>
48           >
49             {children}
50         </select>
51         <svg
52           viewBox="0 0 12 12"
53           className="pointer-events-none absolute right-3.5 top-1/2 h-2.5 w-2.5 -translate-y-1/2 opacity-
54             50"
55           fill="none"
56           stroke="currentColor"
57           strokeWidth="1.5"
58           strokeLinecap="round"
59         >
60           <path d="M2.5 4.5 6 8 3.5 3.5" />
61         </svg>
62       </div>
63     </div>
64   );
65 }

```

```

65
66 function CloseIcon() {
67   return (
68     <svg viewBox="0 0 14 14" className="h-3.5 w-3.5" fill="none" stroke="currentColor" strokeWidth="1.6"
strokeLinecap="round">
69       <path d="M3.5 3.5l7 7" />
70     </svg>
71   );
72 }
73
74 export default function DeviceFormModal({
75   eyebrow,
76   heading,
77   identityName,
78   identityMac,
79   sections,
80   activeSection,
81   onSectionChange,
82   activeTitle,
83   activeSub,
84   dirty = false,
85   error = "",
86   saving = false,
87   submitLabel = "Save Changes",
88   onClose,
89   onSubmit,
90   children
91 }) {
92   useEffect(() => {
93     const previousOverflow = document.body.style.overflow;
94     const handleKeyDown = (event) => {
95       if (event.key === "Escape" && !saving) onClose?();
96     };
97     document.body.style.overflow = "hidden";
98     document.addEventListener("keydown", handleKeyDown);
99     return () => {
100       document.body.style.overflow = previousOverflow;
101       document.removeEventListener("keydown", handleKeyDown);
102     };
103   }, [onClose, saving]);
104
105   return (
106     <div
107       className="fixed inset-0 z-[230] flex items-center justify-center bg-[rgba(12,12,12,0.5)] px-4 py-6
backdrop-blur-[3px] max-[820px]:p-0"
108       onMouseDown={(event) => {
109         if (event.target === event.currentTarget && !saving) onClose?();
110       }}
111     >
112       <div className="flex h-auto max-h-[calc(100dvh-48px)] w-[1040px] max-w-full flex-col overflow-hidde
n rounded-[16px] border-(--line) bg-(--surface) shadow-(--shadowUp) max-[820px]:h-full max-[820px]
: max-h-full max-[820px]:rounded-none md:flex-row">
113         { /* Sidebar */}
114         <aside
115           className="relative flex shrink-0 flex-col gap-[18px] p-[clamp(20px,2.4vw,26px)] text-[#ebebeb]
md:w-[272px]"
116           style={{ background: "linear-gradient(165deg,#1C1C1C 0%,#241C4A 78%,#341CD6 150%)" }}
117         >
118           <div
119             className="pointer-events-none absolute inset-0 opacity-[0.14]"
120             style={{ background: "repeating-linear-gradient(90deg,#fff 0 1px,transparent 1px 34px)" }}
121             aria-hidden
122           />
123
124           <div className="relative flex items-start gap-3">
125             <span className="grid h-11 w-11 shrink-0 place-items-center rounded-[13px] bg-[linear-gradien
t(140deg,#e04420,#8b5cf6)]">
126               <svg viewBox="0 0 16 16" className="h-[19px] w-[19px]" fill="none" stroke="#fff" strokeWidt
h="1.4" strokeLinecap="round" strokeLinejoin="round">
127                 <rect x="4.4" y="1.9" width="7.2" height="12.2" rx="1.4" />

```



```

128         <path d="M6.8 11.9h2.4" />
129     </svg>
130 </span>
131     <div className="min-w-0 flex-1">
132         <div className="font-vcr text-[9px] tracking-[0.2em] text-(--purple)">{eyebrow}</div>
133         <div className="font-chillax mt-1.5 truncate text-[clamp(22px,2.6vw,28px)] font-semibold le
ading-[1.05] tracking-[-0.02em] text-white">
134             {heading}
135         </div>
136     </div>
137     <button
138         type="button"
139         onClick={onClose}
140         disabled={saving}
141         aria-label="Close"
142         className="grid h-8 w-8 shrink-0 place-items-center rounded-[9px] border border-white/20 te
xt-[#ebebcb] transition hover:border-(--orange) hover:bg-(--orange) disabled:opacity-50 md:hidden"
143     >
144         <CloseIcon />
145     </button>
146 </div>
147
148     {identityName ? (
149         <div className="relative rounded-[12px] border border-white/10 bg-white/[0.07] p-[13px_14px]
max-[820px]:hidden">
150             <div className="text-[13px] font-semibold leading-[1.35] text-white [overflow-wrap:anywhere
]">
151                 {identityName}
152             </div>
153             {identityMac ? (
154                 <div className="font-vcr mt-1.5 text-[11px] text-white/60 [overflow-wrap:anywhere]">
155                     {identityMac}
156                 </div>
157             ) : null}
158         </div>
159     ) : null}
160
161     <nav className="relative flex gap-1.5 max-[820px]:overflow-x-auto md:mt-auto md:flex-col">
162         {sections.map((section, index) => {
163             const isActive = index === activeSection;
164             return (
165                 <button
166                     key={section.label}
167                     type="button"
168                     onClick={() => onSectionChange(index)}
169                     className={`flex shrink-0 items-center gap-3 rounded-[11px] p-[11px_12px] text-left tra
nsition ${
170                         isActive ? "bg-white/[0.12] text-white" : "text-white/70 hover:bg-white/[0.09]"
171                     }`}
172                 >
173                     <span
174                         className={`font-vcr grid h-7 w-7 shrink-0 place-items-center rounded-[8px] text-[10p
x] ${
175                             isActive ? "bg-(--orange) text-white" : "bg-white/10 text-white/70"
176                         }`}
177                     >
178                         {String(index + 1).padStart(2, "0")}
179                     </span>
180                     <span className="min-w-0 flex-1">
181                         <span className="block truncate text-[12.5px] font-semibold">{section.label}</span>
182                         <span className="font-vcr mt-0.5 block text-[8px] tracking-[0.14em] opacity-60">{sect
ion.hint}</span>
183                     </span>
184                     {isActive ? <span className="h-6 w-[5px] shrink-0 rounded-full bg-(--orange) max-[820px
]:hidden" /> : null}
185                 </button>
186             );
187         })}
188     </nav>
189 </aside>

```

```

190
191     {/ * Form panel */}
192     <div className="flex min-h-0 min-w-0 flex-1 flex-col">
193       <div className="flex shrink-0 items-start gap-3 border-b border-(--line) px-[clamp(16px,2vw,24p
x)] py-[clamp(14px,1.8vw,20px)]">
194         <div className="min-w-0">
195           <div className="font-vcr text-[10px] tracking-[0.18em] text-(--orange)">{activeTitle}</div>
196           <div className="mt-1 text-[12px] font-light text-(--muted)">{activeSub}</div>
197         </div>
198         <button
199           type="button"
200           onClick={onClose}
201           disabled={saving}
202           aria-label="Close"
203           className="ml-auto grid h-8 w-8 shrink-0 place-items-center rounded-[9px] border border-(--
line) text-(--muted) transition hover:border-(--orange) hover:text-(--orange) disabled:opacity-50 max-[82
0px]:hidden"
204         >
205           <CloseIcon />
206         </button>
207       </div>
208
209       <form onSubmit={onSubmit} className="flex min-h-0 flex-1 flex-col">
210         <div className="min-h-0 flex-1 overflow-y-auto px-[clamp(16px,2vw,24px)] py-[clamp(16px,2vw,2
4px)]">
211           <div className="grid grid-cols-1 gap-[clamp(12px,1.4vw,18px)] md:grid-cols-3">
212             {children}
213           </div>
214           {error ? (
215             <p className="mt-4 rounded-[10px] border border-[rgba(224,68,32,0.25)] bg-[rgba(224,68,32
,0.06)] px-3 py-2 text-[12px] font-semibold text-(--orange)">
216               {error}
217             </p>
218           ) : null}
219         </div>
220
221         <div className="flex shrink-0 flex-col gap-2.5 border-t border-(--line) bg-(--surface2) px-[c
lamp(16px,2vw,24px)] py-[13px] md:flex-row md:items-center">
222           <div
223             className={`inline-flex items-center justify-center gap-1.5 self-start rounded-full px-2.
5 py-1.5 md:self-auto ${
224               dirty ? "bg-[rgba(224,68,32,0.1)] text-(--orange)" : "bg-(--chip) text-(--faint)"
225             }`}
226           >
227             <span className="h-1.5 w-1.5 rounded-full bg-current" />
228             <span className="font-vcr text-[9px] tracking-[0.13em]">
229               {dirty ? "UNSAVED CHANGES" : "NO CHANGES"}
230             </span>
231           </div>
232           <div className="grid grid-cols-2 gap-2.5 md:ml-auto md:flex md:items-center">
233             <button
234               type="button"
235               onClick={onClose}
236               disabled={saving}
237               className="flex h-11 items-center justify-center rounded-[10px] border border-(--line)
bg-(--surface) px-6 text-[13.5px] font-semibold text-(--muted) transition hover:border-(--orange) hover:t
ext-(--orange) disabled:cursor-not-allowed disabled:opacity-50"
238             >
239               Cancel
240             </button>
241             <button
242               type="submit"
243               disabled={saving}
244               className="flex h-11 items-center justify-center whitespace-nowrap rounded-[10px] bg-(
--text) px-7 text-[13.5px] font-semibold text-(--bg) transition hover:bg-(--orange) disabled:cursor-not-al
lowed disabled:opacity-60"
245             >
246               {saving ? "Saving..." : submitLabel}
247             </button>
248           </div>

```

```
249         </div>
250     </form>
251 </div>
252 </div>
253 </div>
254 );
255 }
```

FILE: apps/dashboard/src/app/device_masterlist/components/DeviceMasterIcons.js

```
1 "use client";
2
3 const iconClass = "h-5 w-5";
4
5 export function SearchIcon({ className = iconClass }) {
6   return (
7     <svg viewBox="0 0 24 24" className={className} fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
8       <circle cx="11" cy="11" r="7" />
9       <path d="m20 20-3.5-3.5" />
10    </svg>
11  );
12 }
13
14 export function EditIcon({ className = iconClass }) {
15   return (
16     <svg viewBox="0 0 24 24" className={className} fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
17       <path d="M12 20h9" />
18       <path d="M16.5 3.5a2.1 2.1 0 0 1 3 3L7 19l-4 1 1-4Z" />
19     </svg>
20   );
21 }
22
23 export function ClockIcon({ className = "h-4 w-4" }) {
24   return (
25     <svg viewBox="0 0 24 24" className={className} fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
26       <circle cx="12" cy="12" r="9" />
27       <path d="M12 7v5l3 2" />
28     </svg>
29   );
30 }
31
32 export function PlusIcon({ className = iconClass }) {
33   return (
34     <svg viewBox="0 0 24 24" className={className} fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
35       <path d="M12 5v14" />
36       <path d="M5 12h14" />
37     </svg>
38   );
39 }
```

FILE: apps/dashboard/src/app/device_masterlist/components/DeviceMasterList.js

```

1  "use client";
2
3  import { useEffect, useRef, useState } from "react";
4  import { searchDeviceMasterlist } from "../../lib/dashboardApi";
5  import { useDashboardStore } from "../../store/dashboardStore";
6  import { ClockIcon, EditIcon, PlusIcon, SearchIcon } from "../DeviceMasterIcons";
7  import EditDeviceModal from "../EditDeviceModal";
8  import AddDeviceModal from "../AddDeviceModal";
9
10 const GRID_COLS =
11   "md:grid-cols-[minmax(0,1.5fr)_minmax(0,1.05fr)_minmax(0,1.5fr)_auto]";
12
13 function formatDate(value) {
14   if (!value) return "-";
15   const date = new Date(value);
16   if (Number.isNaN(date.getTime())) return String(value);
17   return new Intl.DateTimeFormat("en-IN", {
18     day: "2-digit",
19     month: "short",
20     year: "numeric",
21     hour: "numeric",
22     minute: "2-digit"
23   }).format(date);
24 }
25
26 function getDeviceKey(device, index) {
27   return device?.id ?? device?.printId ?? device?.androidId ?? device?.hardwareId ?? index;
28 }
29
30 function getDeviceTitle(device) {
31   return device?.description ?? device?.androidId ?? device?.model ?? "-";
32 }
33
34 function getDeviceSerial(device) {
35   return device?.reference ?? device?.serial ?? device?.serialNumber ?? "-";
36 }
37
38 function getNfcLabel(device) {
39   if (device?.nfcType) return device.nfcType;
40   if (device?.hasNfc === 1 || device?.hasNfc === true) return "enabled";
41   if (device?.hasNfc === 0 || device?.hasNfc === false) return "none";
42   return "-";
43 }
44
45 function getInitial(device) {
46   const title = String(getDeviceTitle(device)).trim();
47   return title && title !== "-" ? title.charAt(0).toUpperCase() : "?";
48 }
49
50 function SpecRow({ label, value }) {
51   return (
52     <div className="flex min-w-0 items-baseline gap-2">
53       <span className="font-vcr w-[46px] shrink-0 text-[7.5px] tracking-[0.14em] text(--faint)">
54         {label}
55       </span>
56       <span className="min-w-0 break-words text-[12.5px] font-medium text(--text)">
57         {value || "-"}
58       </span>
59     </div>
60   );
61 }
62
63 function DeviceRow({ device, index, onEdit }) {
64   return (
65     <div
66       className={`grid grid-cols-1 items-start gap-4 border-b border-(--line2) px-4 py-4 transition-color
67 s duration-150 last:border-b-0 hover:bg-(--surface2) md:gap-[18px] md:px-[16px] ${GRID_COLS}`}
68       style={{ animation: `atxRowIn 0.3s both`, animationDelay: `${index * 0.04}s`}
69     >

```

```

69     {/* DEVICE */}
70     <div className="flex min-w-0 items-start gap-3">
71       <span className="font-chillax flex h-[38px] w-[38px] shrink-0 items-center justify-center rounded
-[11px] bg-[linear-gradient(140deg,#e04420,#8b5cf6)] text-[16px] font-semibold leading-none text-white">
72         {getInitial(device)}
73       </span>
74       <div className="min-w-0">
75         <div className="font-chillax text-[15.5px] font-semibold leading-[1.3] tracking-[-0.01em] [over
flow-wrap:anywhere]">
76           {getDeviceTitle(device)}
77         </div>
78         <div className="font-vcr mt-1 text-[11.5px] text-(--blue) [overflow-wrap:anywhere]">
79           {device.hardwareId || "-"}
80         </div>
81       </div>
82     </div>
83
84     {/* IDENTITY */}
85     <div className="min-w-0">
86       <div className="font-vcr text-[7.5px] tracking-[0.15em] text-(--faint)">S/N</div>
87       <div className="mt-1 text-[12.5px] font-semibold [overflow-wrap:anywhere]">
88         {getDeviceSerial(device)}
89       </div>
90       <div className="mt-2.5 flex flex-wrap items-center gap-2">
91         <span className="rounded-full bg-(--text) px-2.5 py-1 text-[11px] font-semibold text-(--bg)">
92           {device.type || "-"}
93         </span>
94         <span className="font-vcr text-[10.5px] text-(--muted)">
95           ID {device.printId ?? device.id ?? "-"}
96         </span>
97       </div>
98     </div>
99
100    {/* SPECS */}
101    <div className="flex min-w-0 flex-col gap-[7px]">
102      <SpecRow label="NFC" value={getNfcLabel(device)} />
103      <SpecRow label="MODEL" value={device.model} />
104      <SpecRow label="BANK" value={device.bank} />
105      <div className="mt-0.5 flex flex-wrap items-center gap-2.5 text-(--muted)">
106        <ClockIcon className="h-3 w-3 shrink-0 opacity-60" />
107        <span className="text-[11.5px] font-light">
108          {formatDateTime(device.updatedAt ?? device.createdAt)}
109        </span>
110        <span className="text-(--line)">|</span>
111        <span className="font-vcr text-[10.5px] [overflow-wrap:anywhere]">
112          {device.androidId || "-"}
113        </span>
114      </div>
115    </div>
116
117    {/* EDIT */}
118    <div className="flex items-start justify-start md:justify-end">
119      <button
120        type="button"
121        onClick={() => onEdit(device)}
122        className="grid h-[34px] w-[34px] place-items-center rounded-[9px] border border-(--line) bg(-
-surface) text-(--orange) transition duration-150 hover:border-(--orange) hover:bg-[rgba(224,68,32,0.08)]
123        aria-label={`Edit device ${device.hardwareId || getDeviceKey(device, index)}`}
124      >
125        <EditIcon className="h-[15px] w-[15px]" />
126      </button>
127    </div>
128  </div>
129  );
130 }
131
132 export default function DeviceMasterList() {
133   const token = useDashboardStore((state) => state.token);
134   const [query, setQuery] = useState("");

```

```

135 const [devices, setDevices] = useState([]);
136 const [loading, setLoading] = useState(false);
137 const [error, setError] = useState("");
138 const [editingDevice, setEditingDevice] = useState(null);
139 const [isAddOpen, setIsAddOpen] = useState(false);
140 const requestIdRef = useRef(0);
141
142 useEffect(() => {
143   const search = query.trim();
144   requestIdRef.current += 1;
145   const requestId = requestIdRef.current;
146
147   if (!search) {
148     setDevices([]);
149     setLoading(false);
150     setError("");
151     return undefined;
152   }
153
154   setLoading(true);
155   setError("");
156
157   const timeoutId = window.setTimeout(async () => {
158     try {
159       const list = await searchDeviceMasterlist({ search, token, dedupe: false });
160       if (requestId === requestIdRef.current) {
161         setDevices(Array.isArray(list) ? list : []);
162       }
163     } catch (searchError) {
164       console.error("Failed to search master devices", searchError);
165       if (requestId === requestIdRef.current) {
166         setDevices([]);
167         setError("Unable to search devices.");
168       }
169     } finally {
170       if (requestId === requestIdRef.current) {
171         setLoading(false);
172       }
173     }
174   }, 450);
175
176   return () => window.clearTimeout(timeoutId);
177 }, [query, token]);
178
179 const handleDeviceSaved = (updatedDevice) => {
180   const updatedKey = getDeviceKey(updatedDevice);
181   setDevices((current) =>
182     current.map((device, index) =>
183       getDeviceKey(device, index) === updatedKey
184         ? { ...device, ...updatedDevice }
185         : device
186     )
187   );
188   setEditingDevice(null);
189 };
190
191 const handleDeviceCreated = (createdDevice) => {
192   if (createdDevice) {
193     setDevices((current) => [createdDevice, ...current]);
194   }
195   setIsAddOpen(false);
196 };
197
198 const hasResults = devices.length > 0;
199
200 return (
201   <div className="mx-auto w-full max-w-[1560px] pb-24">
202     <style>{@keyframes atxRowIn{from{opacity:0;transform:translateY(8px)}to{opacity:1;transform:none}}
203   `}</style>

```

```

204      {/* Title + primary action */}
205      <div className="flex flex-wrap items-end justify-between gap-4 pt-6">
206        <div>
207          <div className="font-vcr text-[9.5px] tracking-[0.2em] text-(--muted)">
208            DEVICES ( {devices.length} )
209          </div>
210          <h1 className="font-chillax mt-1.5 text-[clamp(26px,3.2vw,38px)] font-semibold leading-[1.05] t
racking-[0.02em] text-(--text)">
211            Device Master List
212          </h1>
213        </div>
214        <button
215          type="button"
216          onClick={() => setIsAddOpen(true)}
217          className="inline-flex h-11 shrink-0 items-center gap-2 rounded-[11px] bg-(--text) px-5 text-[1
3.5px] font-semibold text-(--bg) transition duration-150 hover:bg-(--orange)"
218        >
219          <PlusIcon className="h-3.5 w-3.5" />
220          Add Device
221        </button>
222      </div>
223
224      {/* Search */}
225      <div className="mt-[clamp(16px,2vw,22px)]">
226        <label className="flex h-12 items-center gap-3 rounded-[12px] border border-(--line) bg-(--surfac
e) px-[15px] shadow-(--shadow) transition focus-within:border-(--orange)">
227          <SearchIcon className="h-4 w-4 shrink-0 text-(--faint)" />
228          <input
229            value={query}
230            onChange={(event) => setQuery(event.target.value)}
231            placeholder="Search MAC, serial, model, bank"
232            className="h-full min-w-0 flex-1 bg-transparent text-[14px] text-(--text) outline-none placeh
older:text-(--faint)"
233          />
234          {query ? (
235            <button
236              type="button"
237              onClick={() => setQuery("")}
238              aria-label="Clear search"
239              className="grid h-[22px] w-[22px] shrink-0 place-items-center rounded-md text-(--faint) tra
nsition hover:bg-(--chip) hover:text-(--text)"
240            >
241              <svg viewBox="0 0 12 12" className="h-3 w-3" stroke="currentColor" strokeWidth="1.6" stroke
Linecap="round" fill="none">
242                <path d="M3 3l6 6" />
243              </svg>
244            </button>
245          ) : null}
246        </label>
247      </div>
248
249      {/* Table */}
250      <section className="mt-[clamp(14px,1.8vw,20px)] overflow-hidden rounded-[15px] border border-(--lin
e) bg-(--surface) shadow-(--shadow)">
251        <div
252          className={`hidden border-b border-(--line) bg-(--surface2) px-[16px] py-[13px] md:grid md:item
s-center md:gap-[18px] ${GRID_COLS}`}
253        >
254          {[ "DEVICE", "IDENTITY", "SPECS" ].map((label) => (
255            <span key={label} className="font-vcr text-[9.5px] tracking-[0.16em] text-(--orange)">
256              {label}
257            </span>
258          ))}
259          <span className="font-vcr text-right text-[9.5px] tracking-[0.16em] text-(--orange)">
260            EDIT
261          </span>
262        </div>
263
264        {loading ? (
265          <div className="px-6 py-14 text-center text-[13px] font-medium text-(--muted)">

```



```
266         Searching devices...
267     </div>
268     ) : error ? (
269         <div className="px-6 py-14 text-center text-[13px] font-medium text-(--orange)">
270             {error}
271         </div>
272     ) : hasResults ? (
273         devices.map((device, index) => (
274             <DeviceRow
275                 key={getDeviceKey(device, index)}
276                 device={device}
277                 index={index}
278                 onEdit={setEditingDevice}
279             />
280         ))
281     ) : (
282         <div className="px-6 py-14 text-center">
283             <div className="font-chillax text-[17px] font-medium text-(--text)">
284                 {query.trim() ? "No device matches" : "Search the device master"}
285             </div>
286             <div className="mt-1.5 text-[12.5px] text-(--faint)">
287                 Try a MAC address, serial number or bank code.
288             </div>
289         </div>
290     )}
291 </section>
292
293 {editingDevice ? (
294     <EditDeviceModal
295         device={editingDevice}
296         token={token}
297         onClose={() => setEditingDevice(null)}
298         onSave={handleDeviceSaved}
299     />
300 ) : null}
301
302 {isAddOpen ? (
303     <AddDeviceModal
304         token={token}
305         onClose={() => setIsAddOpen(false)}
306         onCreate={handleDeviceCreated}
307     />
308 ) : null}
309 </div>
310 );
311 }
```

FILE: apps/dashboard/src/app/device_masterlist/components/EditDeviceModal.js

```
1 "use client";
2
3 import { useEffect, useMemo, useState } from "react";
4 import { updateDeviceMasterlist } from "../../lib/dashboardApi";
5 import DeviceFormModal, { Field, SelectField } from "./DeviceFormModal";
6 import DeviceBankCredentials, {
7   DEFAULT_BANK_TYPES,
8   DEFAULT_DEVICE_TYPES,
9   DEFAULT_MODEL_TYPES,
10   getDefaultBankData,
11   sanitizeBankData
12 } from "./DeviceBankCredentials";
13
14 const SECTIONS = [
15   {
16     label: "Device Information",
17     hint: "IDENTITY & HARDWARE",
18     title: "DEVICE INFORMATION",
19     sub: "Identity, hardware and NFC configuration for this device."
20   },
21   {
22     label: "Bank Credentials",
23     hint: "PAYMENT CREDENTIALS",
24     title: "BANK CREDENTIALS",
25     sub: "Acquirer credentials used for card and UPI transactions."
26   }
27 ];
28
29 function normalizeBankData(value) {
30   if (value && typeof value === "object" && !Array.isArray(value)) {
31     return value;
32   }
33   return {};
34 }
35
36 function createFormState(device) {
37   const bank = device?.bank ?? "";
38   const bankData = {
39     ...getDefaultBankData(bank),
40     ...normalizeBankData(device?.bankData)
41   };
42   return {
43     id: device?.id ?? "",
44     printId: device?.printId ?? "",
45     hardwareId: device?.hardwareId ?? "",
46     androidId: device?.androidId ?? "",
47     type: device?.type ?? "",
48     bank,
49     model: device?.model ?? "",
50     reference: device?.reference ?? "",
51     description: device?.description ?? "",
52     hasNfc: Number(device?.hasNfc) === 1 ? "1" : "0",
53     nfcType: device?.nfcType ?? "",
54     status: device?.status ?? "active",
55     updatedAt: device?.updatedAt ?? null,
56     createdAt: device?.createdAt ?? null,
57     bankData
58   };
59 }
60
61 function getUpdatedDevice(result, payload) {
62   const directData =
63     result?.data &&
64     !Array.isArray(result.data) &&
65     (result.data.id || result.data.hardwareId)
66     ? result.data
67     : null;
68   const returned =
69     result?.device ??
```

```

70     result?.data?.device ??
71     (result?.id || result?.hardwareId ? result : null) ??
72     directData;
73
74     return returned && typeof returned === "object"
75     ? { ...payload, ...returned, bankData: returned.bankData ?? payload.bankData }
76     : payload;
77 }
78
79 export default function EditDeviceModal({ device, token, onClose, onSave }) {
80     const initialState = useMemo(() => createFormState(device), [device]);
81     const [form, setForm] = useState(initialState);
82     const [saving, setSaving] = useState(false);
83     const [error, setError] = useState("");
84     const [section, setSection] = useState(0);
85     const [dirty, setDirty] = useState(false);
86
87     useEffect(() => {
88         setForm(initialState);
89         setError("");
90         setSection(0);
91         setDirty(false);
92     }, [initialState]);
93
94     const handleChange = (event) => {
95         const { name, value } = event.target;
96         setForm((current) => ({
97             ...current,
98             [name]: value,
99             ...(name === "bank" ? { bankData: getDefaultBankData(value) } : {})
100         }));
101         setDirty(true);
102         setError("");
103     };
104
105     const handleBankChange = (event) => {
106         const { name, value } = event.target;
107         setForm((current) => ({
108             ...current,
109             bankData: { ...current.bankData, [name]: value }
110         }));
111         setDirty(true);
112         setError("");
113     };
114
115     const handleSubmit = async (event) => {
116         event.preventDefault();
117
118         if (!form.id) {
119             setError("Device ID is missing.");
120             return;
121         }
122
123         if (!String(form.hardwareId).trim()) {
124             setError("Hardware ID is required.");
125             return;
126         }
127
128         const payload = {
129             id: Number(form.id),
130             printId: String(form.printId).trim(),
131             hardwareId: String(form.hardwareId).trim(),
132             androidId: String(form.androidId).trim(),
133             type: String(form.type).trim(),
134             bank: String(form.bank).trim(),
135             model: String(form.model).trim(),
136             reference: String(form.reference).trim(),
137             description: String(form.description).trim(),
138             hasNfc: Number(form.hasNfc),
139             nfcType: String(form.nfcType).trim(),

```

```

140     updatedAt: form.updatedAt,
141     createdAt: form.createdAt,
142     status: form.status,
143     bankData: sanitizeBankData(form.bank, form.bankData)
144   };
145
146   setSaving(true);
147   setError("");
148
149   try {
150     const result = await updateDeviceMasterlist({ token, payload });
151     onSave?.(getUpdatedDevice(result, payload));
152   } catch (submitError) {
153     console.error("Failed to update master device", submitError);
154     setError(submitError?.message || "Unable to update device.");
155   } finally {
156     setSaving(false);
157   }
158 };
159
160 return (
161   <DeviceFormModal
162     eyebrow="EDIT DEVICE"
163     heading={form.printId || `#${form.id}`}
164     identityName={form.description || form.androidId || form.model || "Master device"}
165     identityMac={form.hardwareId}
166     sections={SECTIONS}
167     activeSection={section}
168     onSectionChange={setSection}
169     activeTitle={SECTIONS[section].title}
170     activeSub={SECTIONS[section].sub}
171     dirty={dirty}
172     error={error}
173     saving={saving}
174     submitLabel="Save Changes"
175     onClose={onClose}
176     onSubmit={handleSubmit}
177   >
178     {section === 0 ? (
179       <>
180         <Field label="DEVICE ID" name="id" value={form.id} onChange={handleChange} readOnly />
181         <Field label="PRINT ID" name="printId" value={form.printId} onChange={handleChange} required />
182         <Field label="HARDWARE ID" name="hardwareId" value={form.hardwareId} onChange={handleChange} re
183         quired />
184         <Field label="ANDROID ID" name="androidId" value={form.androidId} onChange={handleChange} />
185         <SelectField label="TYPE" name="type" value={form.type} onChange={handleChange}>
186           {form.type && !DEFAULT_DEVICE_TYPES.some((value) => value === form.type) ? (
187             <option value={form.type}>{form.type}</option>
188             ) : null}
189           {DEFAULT_DEVICE_TYPES.map((value, label) => (
190             <option key={value} value={value}>{label}</option>
191           ))}
192         </SelectField>
193         <SelectField label="BANK" name="bank" value={form.bank} onChange={handleChange}>
194           {form.bank && !DEFAULT_BANK_TYPES.some((value) => value === form.bank) ? (
195             <option value={form.bank}>{form.bank}</option>
196             ) : null}
197           {DEFAULT_BANK_TYPES.map((value, label) => (
198             <option key={value} value={value}>{label}</option>
199           ))}
200         </SelectField>
201         <SelectField label="MODEL" name="model" value={form.model} onChange={handleChange}>
202           {form.model && !DEFAULT_MODEL_TYPES.some((value) => value === form.model) ? (
203             <option value={form.model}>{form.model}</option>
204             ) : null}
205           {DEFAULT_MODEL_TYPES.map((value, label) => (
206             <option key={value} value={value}>{label}</option>
207           ))}
208         </SelectField>
209         <Field label="REFERENCE" name="reference" value={form.reference} onChange={handleChange} />

```

```
209     <Field label="NFC TYPE" name="nfcType" value={form.nfcType} onChange={handleChange} />
210     <SelectField label="NFC" name="hasNfc" value={form.hasNfc} onChange={handleChange}>
211       <option value="1">Enabled</option>
212       <option value="0">Disabled</option>
213     </SelectField>
214     <SelectField label="STATUS" name="status" value={form.status} onChange={handleChange}>
215       <option value="active">Active</option>
216       <option value="inactive">Inactive</option>
217     </SelectField>
218     <Field label="DESCRIPTION" name="description" value={form.description} onChange={handleChange}
wide />
219   </>
220   ) : (
221     <DeviceBankCredentials
222       bank={form.bank}
223       bankData={form.bankData}
224       onChange={handleBankChange}
225     />
226   )}
227 </DeviceFormModal>
228 );
229 }
```

FILE: apps/dashboard/src/app/device_masterlist/page.js

```
1 import Header from "../components/Header";
2 import DeviceMasterList from "../components/DeviceMasterList";
3
4 export default function DeviceMasterListPage() {
5   return (
6     <main
7       className="min-h-screen bg-(--bg) pb-8"
8       style={{ fontFamily: '"Poppins", system-ui, sans-serif' }}
9     >
10      <Header areaLabel="Device Master" variant="portal" />
11      <div className="w-full pr-4 pl-[72px] pt-2 md:pr-7 md:pl-[88px] max-[900px]:px-3">
12        <DeviceMasterList />
13      </div>
14    </main>
15  );
16 }
```

FILE: apps/dashboard/src/app/event-edit/components/EventEditContent.js

```
1 "use client";
2
3 import { useEffect, useMemo, useState } from "react";
4 import { useRouter } from "next/navigation";
5 import { AtomXLoader } from "@atomx/global-components";
6 import {
7   closeEventDay,
8   fetchEventDetails,
9   updateEventBalanceSetting,
10  updateEventDetails
11 } from "../../lib/dashboardApi";
12 import { useDashboardStore } from "../../store/dashboardStore";
13 import {
14   CalendarIcon,
15   Field,
16   SearchIcon,
17   SectionCard,
18   SelectField,
19   SettingRow,
20   SubPanel,
21   TopupRow,
22   UploadBox
23 } from "../EventEditPrimitives";
24
25 const emptyEventFields = {
26   "Event Name": "",
27   Country: "",
28   City: "",
29   "Time Zone": "",
30   Venue: "",
31   Organiser: "",
32   Client: "",
33   Currency: "",
34   "Start Date": "",
35   "End Date": "",
36   "ET Code": ""
37 };
38
39 const eventFieldOrder = [
40   "Event Name",
41   "Country",
42   "City",
43   "Time Zone",
44   "Venue",
45   "Organiser",
46   "Client",
47   "Currency",
48   "Start Date",
49   "End Date",
50   "ET Code"
51 ];
52
53 const emptyCardFields = {
54   "Card Fee": "",
55   "1st Topup": "",
56   "Max Wallet": "",
57   "Return Min Amount": "",
58   "Return Max Amount": ""
59 };
60
61 const initialDashFields = {
62   "Dashboard Password": ""
63 };
64
65 const initialMswipeFields = {
66   Username: "",
67   Password: "",
68   "Verify Client Code": "",
69   "Verify User ID": "",
```

```

70   "Verify Password": "",
71   "UPI API": ""
72 };
73
74 const services = [
75   "NFC Cashless",
76   "Use Pin",
77   "Maker Checker",
78   "Online Topup",
79   "Access Control",
80   "Taxation"
81 ];
82
83 const posLeft = ["Happy Hours", "Round Off", "Use Club Card"];
84 const posRight = ["Manual Topup", "Link Mobile"];
85 const returns = [
86   "Return Card Fee",
87   "Return Balance",
88   "Return Bank Card Balance",
89   "Return Card Fee In Cash"
90 ];
91 const dashToggles = ["Show Activation Data", "Show Coupon Data", "Show Comp Data", "Auto Reset Dashboard"];
92
93 function boolMap(labels, value = true) {
94   return labels.reduce((acc, label) => ({ ...acc, [label]: value }), {});
95 }
96
97 function asText(value) {
98   return value === null || value === undefined ? "" : String(value);
99 }
100
101 function asBool(value) {
102   return value === true || value === 1 || value === "1" || value === "true";
103 }
104
105 function formatDate(value) {
106   if (!value) return "";
107   const date = new Date(value);
108   if (Number.isNaN(date.getTime())) return asText(value);
109   return date.toISOString().slice(0, 10);
110 }
111
112 function splitCsv(value) {
113   return asText(value)
114     .split(",")
115     .map((item) => item.trim())
116     .filter(Boolean);
117 }
118
119 function topupsFromEvent(event) {
120   const cashValues = splitCsv(event?.topupValues);
121   const tokenValues = splitCsv(event?.topupNames);
122   const length = Math.max(cashValues.length, tokenValues.length);
123   if (!length) return [];
124   return Array.from({ length }, (_, index) => ({
125     cash: cashValues[index] ?? "",
126     token: tokenValues[index] ?? cashValues[index] ?? ""
127   }));
128 }
129
130 function mapEventToState(event) {
131   return {
132     eventFields: {
133       "Event Name": asText(event?.name),
134       Country: asText(event?.country),
135       City: asText(event?.locationCity),
136       "Time Zone": asText(event?.tz),
137       Venue: asText(event?.venue),
138       Organiser: asText(event?.organizer ?? event?.organiser),

```



```

139     Client: asText(event?.client),
140     Currency: asText(event?.currency).toUpperCase(),
141     "Start Date": formatDate(event?.startAt),
142     "End Date": formatDate(event?.endAt),
143     "ET Code": asText(event?.etCode ?? event?.eventCode ?? event?.locationId)
144 },
145 cardFields: {
146     "Card Fee": asText(event?.cardFee),
147     "1st Topup": asText(event?.minTopup),
148     "Max Wallet": asText(event?.maxTopupWallet),
149     "Return Min Amount": asText(event?.returnMinAmount),
150     "Return Max Amount": asText(event?.returnMaxAmount)
151 },
152 serviceState: {
153     "NFC Cashless": asBool(event?.useNfc),
154     "Use Pin": asBool(event?.usePin),
155     "Maker Checker": asBool(event?.useMakerChecker),
156     "Online Topup": asBool(event?.onlineTopup),
157     "Access Control": asBool(event?.useAccessx),
158     Taxation: asBool(event?.gstSetting)
159 },
160 posState: {
161     "Happy Hours": asBool(event?.happyHour),
162     "Round Off": asBool(event?.roundOff),
163     "Use Club Card": asBool(event?.useClubCard),
164     "Manual Topup": asBool(event?.topupManual),
165     "Link Mobile": asBool(event?.linkUser)
166 },
167 returnState: {
168     "Return Card Fee": asBool(event?.returnCardFee),
169     "Return Balance": asBool(event?.returnCartValue ?? event?.returnBalance),
170     "Return Bank Card Balance": asBool(event?.returnCardFull),
171     "Return Card Fee In Cash": asBool(event?.returnCardFeeCash)
172 },
173 dashFields: {
174     "Dashboard Password": asText(event?.devicePasswordAdmin ?? event?.dashboardPassword)
175 },
176 dashState: {
177     "Show Activation Data": asBool(event?.showActivationData),
178     "Show Coupon Data": asBool(event?.showCouponData),
179     "Show Comp Data": asBool(event?.showCompData),
180     "Auto Reset Dashboard": asBool(event?.autoResetDashboard)
181 },
182 mswipeFields: {
183     Username: asText(event?.bankUsername),
184     Password: asText(event?.bankPassword),
185     "Verify Client Code": asText(event?.verifyClientCode),
186     "Verify User ID": asText(event?.verifyUserId),
187     "Verify Password": asText(event?.verifyPassword),
188     "UPI API": asText(event?.upiApi)
189 },
190 posPasswords: {
191     "Topup Password": asText(event?.devicePasswordTopup),
192     "Sale Password": asText(event?.devicePassword)
193 },
194 printer: asText(event?.printer) || "Select Printer",
195 topups: topupsFromEvent(event)
196 };
197 }
198
199 function includesQuery(query, ...values) {
200     if (!query) return true;
201     return values.join(" ").toLowerCase().includes(query);
202 }
203
204 function BackIcon({ className = "h-4 w-4" }) {
205     return (
206         <svg
207             viewBox="0 0 24 24"
208             className={className}

```

```

209     fill="none"
210     stroke="currentColor"
211     strokeWidth="2.2"
212     strokeLinecap="round"
213     strokeLinejoin="round"
214   >
215     <path d="m15 18-6-6 6-6" />
216   </svg>
217 );
218 }
219
220 function CloseIcon({ className = "h-5 w-5" }) {
221   return (
222     <svg
223       viewBox="0 0 24 24"
224       className={className}
225       fill="none"
226       stroke="currentColor"
227       strokeWidth="2"
228       strokeLinecap="round"
229       strokeLinejoin="round"
230     >
231       <path d="M18 6 6 18" />
232       <path d="m6 6 12 12" />
233     </svg>
234   );
235 }
236
237 function CheckIcon({ className = "h-5 w-5" }) {
238   return (
239     <svg
240       viewBox="0 0 24 24"
241       className={className}
242       fill="none"
243       stroke="currentColor"
244       strokeWidth="3"
245       strokeLinecap="round"
246       strokeLinejoin="round"
247     >
248       <path d="m5 12 4 4L19 6" />
249     </svg>
250   );
251 }
252
253 function BalanceSettingToast({ reference, onClose }) {
254   return (
255     <
256       <div className="fixed inset-0 z-[70] bg-[rgba(12,12,12,0.5)] backdrop-blur-[2px]" />
257       <div className="fixed left-1/2 top-20 z-[80] w-[320px] max-w-[calc(100vw-2rem)] -translate-x-1/2 an
imate-[balanceToastIn_260ms_ease-out] overflow-hidden rounded-[16px] border border-(--line) bg-(--surface
) shadow-(--shadowUp) max-[640px]:top-[72px] max-[640px]:w-[min(320px,calc(100vw-1.5rem))]">
258         <style jsx>{`
259           @keyframes balanceToastIn {
260             from {
261               opacity: 0;
262               transform: translate(-50%, -22px);
263             }
264             to {
265               opacity: 1;
266               transform: translate(-50%, 0);
267             }
268           }
269         `}</style>
270         <div className="h-1 bg-[linear-gradient(90deg,#E04420,#8B5CF6,#341CD6)]" />
271         <div
272           className="flex items-center justify-between gap-3 px-4 py-3"
273           style={{ background: "linear-gradient(135deg,#1C1C1C 0%,#241C4A 74%,#341CD6 155%)" }}
274         >
275           <div className="flex min-w-0 items-center gap-3">
276             <span className="grid h-9 w-9 shrink-0 place-items-center rounded-[11px] bg-[linear-gradient(

```

```

140deg,#e04420,#8b5cf6)] text-white">
277     <CheckIcon className="h-4 w-4" />
278   </span>
279   <div className="min-w-0">
280     <p className="font-vcr text-[8.5px] uppercase tracking-[0.18em] text-(--purple)">
281       Balance Setting
282     </p>
283     <h3 className="font-chillax mt-0.5 text-[17px] font-semibold text-white">Done</h3>
284   </div>
285 </div>
286 <button
287   type="button"
288   onClick={onClose}
289   className="grid h-8 w-8 shrink-0 place-items-center rounded-[9px] border border-white/20 text
-white/70 transition hover:border-(--orange) hover:bg-(--orange) hover:text-white"
290   aria-label="Close balance setting message"
291 >
292   <CloseIcon className="h-3.5 w-3.5" />
293 </button>
294 </div>
295 <div className="p-4">
296   <div className="rounded-[12px] border border-(--line2) bg-(--surface2) p-3">
297     <p className="font-vcr text-[8.5px] uppercase tracking-[0.16em] text-(--muted)">
298       Reference
299     </p>
300     <p className="font-chillax mt-1.5 text-[30px] font-semibold leading-none text-(--orange)">
301       {reference || "-"}
302     </p>
303   </div>
304   <p className="mt-4 text-[12.5px] font-light leading-5 text-(--muted)">
305     Please update all devices for the changes to reflect this reference number.
306   </p>
307 </div>
308 </div>
309 </>
310 );
311 }
312
313 export default function EventEditContent() {
314   const router = useRouter();
315   const token = useDashboardStore((state) => state.token);
316   const eventMeta = useDashboardStore((state) => state.eventMeta);
317   const storedEventDetails = useDashboardStore((state) => state.eventDetails);
318   const setEventDetails = useDashboardStore((state) => state.setEventDetails);
319   const setEventMeta = useDashboardStore((state) => state.setEventMeta);
320   const eventId = eventMeta?.eventId ?? storedEventDetails?.id;
321   const [search, setSearch] = useState("");
322   const [eventFields, setEventFields] = useState(emptyEventFields);
323   const [cardFields, setCardFields] = useState(emptyCardFields);
324   const [dashFields, setDashFields] = useState(initialDashFields);
325   const [mswipeFields, setMswipeFields] = useState(initialMswipeFields);
326   const [posPasswords, setPosPasswords] = useState({
327     "Topup Password": "",
328     "Sale Password": ""
329   });
330   const [printer, setPrinter] = useState("Select Printer");
331   const [serviceState, setServiceState] = useState(() => boolMap(services, false));
332   const [posState, setPosState] = useState(() => boolMap([...posLeft, ...posRight], false));
333   const [returnState, setReturnState] = useState(() => boolMap(returns, false));
334   const [dashState, setDashState] = useState(() => boolMap(dashToggles, false));
335   const [topups, setTopups] = useState([]);
336   const [loading, setLoading] = useState(Boolean(eventId));
337   const [saving, setSaving] = useState(false);
338   const [closingDay, setClosingDay] = useState(false);
339   const [dayCloseMessage, setDayCloseMessage] = useState("");
340   const [updatingBalanceSetting, setUpdatingBalanceSetting] = useState(false);
341   const [balanceToast, setBalanceToast] = useState(null);
342   const [error, setError] = useState("");
343
344   useEffect(() => {

```

```

345     let cancelled = false;
346     async function loadDetails() {
347         if (!eventId) {
348             setLoading(false);
349             return;
350         }
351         setLoading(true);
352         setError("");
353         try {
354             const details = await fetchEventDetails({ eventId, token, dedupe: false });
355             if (cancelled) return;
356             const mapped = mapEventToState(details);
357             setEventDetails(details);
358             setEventMeta({
359                 eventId: details?.id ?? eventId,
360                 eventName: details?.name ?? "",
361                 venue: details?.venue ?? "",
362                 city: details?.locationCity ?? ""
363             });
364             setEventFields(mapped.eventFields);
365             setCardFields(mapped.cardFields);
366             setServiceState(mapped.serviceState);
367             setPosState(mapped.posState);
368             setReturnState(mapped.returnState);
369             setDashFields(mapped.dashFields);
370             setDashState(mapped.dashState);
371             setMswipeFields(mapped.mswipeFields);
372             setPosPasswords(mapped.posPasswords);
373             setPrinter(mapped.printer);
374             setTopups(mapped.topups);
375         } catch (err) {
376             if (!cancelled) {
377                 console.error("Failed to load event details", err);
378                 setError("Unable to load event details.");
379             }
380         } finally {
381             if (!cancelled) setLoading(false);
382         }
383     }
384     loadDetails();
385     return () => {
386         cancelled = true;
387     };
388 }, [eventId, token, setEventDetails, setEventMeta]);
389
390 const query = search.trim().toLowerCase();
391 const visible = useMemo(
392     () => ({
393         details: includesQuery(query, "event details identity location schedule billing poster", ...eventFi
394     eldOrder),
395         pos: includesQuery(query, "pos point of sale topup printer password", ...posLeft, ...posRight),
396         services: includesQuery(query, "active services enable disable", ...services),
397         card: includesQuery(query, "card fee wallet return rules", ...Object.keys(emptyCardFields), ...retu
398     rns),
399         dash: includesQuery(query, "dash settings dashboard password coupon comp activation balance reset",
400     ...dashToggles),
401         mswipe: includesQuery(query, "mswipe payment gateway username password verify upi")
402     }),
403     [query]
404 );
405
406 const setField = (setter, label, value) => {
407     setter((prev) => ({ ...prev, [label]: value }));
408 };
409
410 const toggle = (setter, label) => {
411     setter((prev) => ({ ...prev, [label]: !prev[label] }));
412 };
413
414 const saveChanges = async () => {
415     if (!eventId) return;
416     setSaving(true);

```

```

412     setError("");
413     try {
414         await updateEventDetails({
415             eventId,
416             token,
417             payload: {
418                 name: eventFields["Event Name"],
419                 country: eventFields.Country,
420                 locationCity: eventFields.City,
421                 tz: eventFields["Time Zone"],
422                 venue: eventFields.Venue,
423                 organizer: eventFields.Organiser,
424                 client: eventFields.Client,
425                 currency: eventFields.Currency,
426                 startAt: eventFields["Start Date"] || null,
427                 endAt: eventFields["End Date"] || null,
428                 cardFee: Number(cardFields["Card Fee"]) || 0,
429                 minTopup: Number(cardFields["1st Topup"]) || 0,
430                 maxTopupWallet: Number(cardFields["Max Wallet"]) || 0,
431                 returnMinAmount: Number(cardFields["Return Min Amount"]) || 0,
432                 returnMaxAmount: Number(cardFields["Return Max Amount"]) || 0,
433                 useNfc: serviceState["NFC Cashless"] ? 1 : 0,
434                 usePin: serviceState["Use Pin"] ? 1 : 0,
435                 useMakerChecker: serviceState["Maker Checker"] ? 1 : 0,
436                 onlineTopup: serviceState["Online Topup"] ? 1 : 0,
437                 useAccessx: serviceState["Access Control"] ? 1 : 0,
438                 gstSetting: serviceState.Taxation ? 1 : 0,
439                 happyHour: posState["Happy Hours"] ? 1 : 0,
440                 roundOff: posState["Round Off"] ? 1 : 0,
441                 useClubCard: posState["Use Club Card"] ? 1 : 0,
442                 topupManual: posState["Manual Topup"] ? 1 : 0,
443                 linkUser: posState["Link Mobile"] ? 1 : 0,
444                 returnCardFee: returnState["Return Card Fee"] ? 1 : 0,
445                 returnCartValue: returnState["Return Balance"] ? 1 : 0,
446                 returnCardFull: returnState["Return Bank Card Balance"] ? 1 : 0,
447                 returnCardFeeCash: returnState["Return Card Fee In Cash"] ? 1 : 0,
448                 devicePasswordAdmin: dashFields["Dashboard Password"],
449                 devicePasswordTopup: posPasswords["Topup Password"],
450                 devicePassword: posPasswords["Sale Password"],
451                 printer,
452                 topupValues: topups.map((item) => item.cash).join(","),
453                 topupNames: topups.map((item) => item.token).join(","),
454                 bankUsername: mswipeFields.Username,
455                 bankPassword: mswipeFields.Password
456             }
457         });
458     } catch (err) {
459         console.error("Failed to save event details", err);
460         setError("Unable to save event details.");
461     } finally {
462         setSaving(false);
463     }
464 };
465
466 const handleUpdateBalanceSetting = async () => {
467     if (!eventId) return;
468     setUpdatingBalanceSetting(true);
469     setError("");
470     try {
471         const response = await updateEventBalanceSetting({ token, eventId });
472         const updatedEvent = response?.event;
473         if (updatedEvent) {
474             setEventDetails(updatedEvent);
475             setEventMeta({
476                 eventId: updatedEvent?.id ?? eventId,
477                 eventName: updatedEvent?.name ?? "",
478                 venue: updatedEvent?.venue ?? "",
479                 city: updatedEvent?.locationCity ?? ""
480             });
481         }
482     }

```

```

482     setBalanceToast({
483       reference: updatedEvent?.locationId ?? response?.locationId ?? ""
484     });
485   } catch (err) {
486     console.error("Failed to update balance setting", err);
487     setError("Unable to update balance setting.");
488   } finally {
489     setUpdatingBalanceSetting(false);
490   }
491 };
492
493 const handleDayClose = async () => {
494   if (!eventId) return;
495   setClosingDay(true);
496   setDayCloseMessage("");
497   setError("");
498   try {
499     await closeEventDay({
500       token,
501       eventId,
502       day: "a",
503       volunteerCount: 0
504     });
505     setDayCloseMessage("Event day closed successfully.");
506   } catch (err) {
507     console.error("Failed to close event day", err);
508     setError("Unable to close the event day.");
509   } finally {
510     setClosingDay(false);
511   }
512 };
513
514 return (
515   <
516     <section className="mb-4 max-[640px]:mb-3">
517       <div className="flex flex-col gap-4 xl:flex-row xl:items-end xl:justify-between max-[640px]:gap-3">
518         <div className="min-w-0">
519           <button
520             type="button"
521             onClick={() => router.push("/Config")}
522             className="font-vcr inline-flex max-w-full items-center gap-1.5 text-[9.5px] uppercase tracking-[0.16em] text-(--muted) transition hover:text-(--orange)"
523           >
524             <BackIcon className="h-3 w-3 shrink-0" />
525             <span className="min-w-0 truncate">
526               Events / {eventFields["Event Name"]} || "Selected Event" / Settings
527             </span>
528           </button>
529           <h1 className="font-chillax mt-2 min-w-0 text-[clamp(24px,3vw,32px)] font-semibold leading-[1.05] tracking-[-0.02em] text-(--text) max-[640px]:mt-1.5 max-[640px]:truncate">
530             Edit Event Information
531           </h1>
532           <p className="mt-2 text-[13.5px] font-light text-(--muted) max-[640px]:mt-1.5 max-[640px]:text-[12px]">
533             Identity, services, POS behavior, card limits, and dashboard settings.
534           </p>
535         </div>
536
537         <div className="flex min-w-0 flex-col gap-2.5 md:flex-row md:items-center">
538           <label className="flex h-11 min-w-[280px] items-center gap-2.5 rounded-[10px] border border-(--line) bg-(--surface) px-3.5 text-(--muted) transition focus-within:border-(--orange) focus-within:shadow-[0_0_3px_rgba(224,68,32,0.12)] max-[640px]:h-10 max-[640px]:min-w-0">
539             <SearchIcon className="h-4 w-4 shrink-0 opacity-60" />
540             <input
541               value={search}
542               onChange={(event) => setSearch(event.target.value)}
543               placeholder="Search settings, fields, passwords, services..."
544               className="min-w-0 flex-1 bg-transparent text-[13px] font-medium text-(--text) outline-none placeholder:text-(--faint) max-[640px]:text-[12px]"

```

```

545         />
546     </label>
547
548     <div className="flex flex-wrap items-center gap-2">
549         <button
550             type="button"
551             onClick={saveChanges}
552             disabled={saving || !eventId}
553             className="h-11 shrink-0 rounded-[10px] bg-(--text) px-5 text-[13.5px] font-semibold text
554             -(--bg) transition hover:bg-(--orange) disabled:cursor-not-allowed disabled:opacity-55 max-[640px]:h-10 m
555             ax-[640px]:flex-1 max-[640px]:px-3 max-[640px]:text-[12px]"
556         >
557             {saving ? "Saving..." : "Save"}
558         </button>
559         <button
560             type="button"
561             onClick={handleDayClose}
562             disabled={closingDay || !eventId}
563             className="h-11 shrink-0 truncate rounded-[10px] border border-(--line) bg-(--surface) px-4 text-[
564             13px] font-semibold text-(--muted) transition hover:border-(--orange) hover:text-(--orange) disabled:curs
565             or-not-allowed disabled:opacity-55 max-[640px]:h-10 max-[640px]:flex-1 max-[640px]:px-3 max-[640px]:text-
566             [12px]"
567         >
568             {closingDay ? "Closing..." : "Day Close"}
569         </button>
570         <button
571             type="button"
572             onClick={handleUpdateBalanceSetting}
573             disabled={updatingBalanceSetting || !eventId}
574             className="h-11 shrink-0 truncate rounded-[10px] bg-(--orange) px-5 text-[13.5px] font-se
575             mibold text-white transition hover:bg-(--text) hover:text-(--bg) disabled:cursor-not-allowed disabled:opa
576             city-55 max-[640px]:h-10 max-[640px]:flex-1 max-[640px]:px-3 max-[640px]:text-[12px]"
577         >
578             {updatingBalanceSetting ? "Setting..." : "Set New Balance"}
579         </button>
580     </div>
581 </div>
582 <div className="mt-[clamp(16px,2vw,22px)] h-px w-full bg-(--line)" />
583 </section>
584
585 {error ? (
586     <div className="mb-4 rounded-[10px] border border-[rgba(224,68,32,0.25)] bg-[rgba(224,68,32,0.06)
587     ] px-3.5 py-2.5 text-[12.5px] font-semibold text-(--orange)">
588         {error}
589     </div>
590 ) : null}
591
592 {dayCloseMessage ? (
593     <div className="mb-4 flex items-center justify-between gap-3 rounded-[10px] border border-[rgba(0
594     ,169,242,0.28)] bg-[rgba(0,169,242,0.08)] px-3.5 py-2.5 text-[12.5px] font-semibold text-[#0284c7]">
595         <span>{dayCloseMessage}</span>
596         <button
597             type="button"
598             onClick={() => setDayCloseMessage("")}
599             className="grid h-7 w-7 shrink-0 place-items-center rounded-[8px] transition hover:bg-[rgba(0
600     ,169,242,0.14)]"
601         >
602             aria-label="Close day close message"
603         </button>
604     </div>
605 ) : null}
606
607 {loading ? (
608     <div className="flex min-h-[220px] items-center justify-center rounded-[15px] border border-(--li
609     ne) bg-(--surface) shadow-(--shadow)">
610         <AtomXLoader label="Loading event details..." size={52} />
611     </div>
612 ) : null}

```

```

604
605     {!!loading && !eventId ? (
606       <div className="rounded-[15px] border border-dashed border-(--line) bg-(--surface) px-6 py-14 tex
t-center">
607         <div className="font-chillax text-[17px] font-medium text-(--text)">No event selected</div>
608         <div className="mt-1.5 text-[12.5px] text-(--faint)">
609           Select an event before editing event details.
610         </div>
611       </div>
612     ) : null}
613
614     {!!loading && eventId ? (
615       <div className="grid min-w-0 items-start gap-4 xl:grid-cols-[1.55fr_1fr] max-[640px]:gap-3">
616         <div className="min-w-0 space-y-4 max-[640px]:space-y-3">
617           {visible.details && (
618             <SectionCard
619               index="01"
620               title="Event Details"
621               description="Identity, location, schedule, and billing basics."
622             >
623               <UploadBox />
624               <div className="mt-4 grid gap-4 md:grid-cols-3 max-[640px]:mt-3 max-[640px]:gap-3">
625                 {eventFieldOrder.map((label) => (
626                   <Field
627                     key={label}
628                     label={label}
629                     value={eventFields[label]}
630                     onChange={(value) => setField(setEventFields, label, value)}
631                     placeholder={label.includes("Date") ? "dd/mm/yyyy" : ""}
632                     icon={label.includes("Date") ? <CalendarIcon /> : null}
633                   />
634                 )}}
635               </div>
636             </SectionCard>
637           )}
638
639           {visible.pos && (
640             <SectionCard index="02" title="POS" description="Point-of-sale behavior and topup presets.">
641               <div className="grid gap-4 lg:grid-cols-2 max-[640px]:gap-3">
642                 <div>
643                   {posLeft.map((label) => (
644                     <SettingRow
645                       key={label}
646                       label={label}
647                       checked={posState[label]}
648                       onToggle={() => toggle(setPosState, label)}
649                     />
650                   )}}
651                 </div>
652                 <div>
653                   {posRight.map((label) => (
654                     <SettingRow
655                       key={label}
656                       label={label}
657                       checked={posState[label]}
658                       onToggle={() => toggle(setPosState, label)}
659                     />
660                   )}}
661                 <div className="mt-3">
662                   <SelectField label="Printer" value={printer} onChange={setPrinter} />
663                 </div>
664               </div>
665             </div>
666
667             <SubPanel label="Topup Buttons" className="mt-4 max-[640px]:mt-3">
668               <div className="space-y-3 max-[640px]:space-y-2.5">
669                 {topups.length === 0 ? (
670                   <div className="rounded-[11px] border border-dashed border-(--line) px-4 py-8 text-ce
nter text-[13px] font-medium text-(--muted) max-[640px]:px-3 max-[640px]:py-6 max-[640px]:text-[12px]">
671                     No topup buttons configured.

```



```

672         </div>
673       ) : (
674         topups.map((topup, index) => (
675           <TopupRow
676             key={index}
677             index={String(index + 1).padStart(2, "0")}
678             cash={topup.cash}
679             token={topup.token}
680             onCashChange={(value) =>
681               setTopups((prev) =>
682                 prev.map((item, itemIndex) =>
683                   itemIndex === index ? { ...item, cash: value } : item
684                 )
685               )
686             }
687             onTokenChange={(value) =>
688               setTopups((prev) =>
689                 prev.map((item, itemIndex) =>
690                   itemIndex === index ? { ...item, token: value } : item
691                 )
692               )
693             }
694           />
695         ))
696       )}
697     </div>
698   </SubPanel>
699
700   <div className="mt-4 grid gap-4 md:grid-cols-2 max-[640px]:mt-3 max-[640px]:gap-3">
701     {Object.keys(posPasswords).map((label) => (
702       <Field
703         key={label}
704         label={label}
705         value={posPasswords[label]}
706         onChange={(value) => setField(setPosPasswords, label, value)}
707         placeholder={`Enter ${label.toLowerCase()}`}
708       />
709     ))}
710   </div>
711 </SectionCard>
712 )}
713 </div>
714
715 <aside className="min-w-0 space-y-4 max-[640px]:space-y-3">
716   {visible.services && (
717     <SectionCard
718       index="03"
719       title="Active Services"
720       description="Enable or disable core event capabilities."
721     >
722       <div className="grid gap-x-8 md:grid-cols-2 max-[640px]:gap-x-0">
723         {services.map((label) => (
724           <SettingRow
725             key={label}
726             label={label}
727             checked={serviceState[label]}
728             onToggle={() => toggle(setServiceState, label)}
729           />
730         ))}
731       </div>
732     </SectionCard>
733   )}
734
735   {visible.card && (
736     <SectionCard index="04" title="Card" description="Card fee, wallet limits, and return rules."
737   >
738     <div className="grid gap-4 md:grid-cols-2 max-[640px]:gap-3">
739       {["Card Fee", "1st Topup", "Max Wallet"].map((label) => (
740         <Field

```

```

741         label={label}
742         value={cardFields[label]}
743         onChange={(value) => setField(setCardFields, label, value)}
744     />
745     )})
746 </div>
747
748 <SubPanel label="Returns" className="mt-4 max-[640px]:mt-3">
749   <div className="grid gap-x-8 md:grid-cols-2 max-[640px]:gap-x-0">
750     {returns.map((label) => (
751       <SettingRow
752         key={label}
753         label={label}
754         checked={returnState[label]}
755         onToggle={() => toggle(setReturnState, label)}
756       />
757     )})
758   </div>
759 </SubPanel>
760
761 <div className="mt-4 grid gap-4 md:grid-cols-2 max-[640px]:mt-3 max-[640px]:gap-3">
762   {["Return Min Amount", "Return Max Amount"].map((label) => (
763     <Field
764       key={label}
765       label={label}
766       value={cardFields[label]}
767       onChange={(value) => setField(setCardFields, label, value)}
768     />
769   )})
770 </div>
771 </SectionCard>
772 })
773
774 {visible.dash && (
775   <SectionCard
776     index="05"
777     title="Dash Settings"
778     description="Dashboard security, visibility, and reset behavior."
779   >
780     <div className="grid gap-4 md:grid-cols-2 max-[640px]:gap-3">
781       <Field
782         label="Dashboard Password"
783         value={dashFields["Dashboard Password"]}
784         onChange={(value) => setField(setDashFields, "Dashboard Password", value)}
785         placeholder="Enter dashboard password"
786       />
787       {dashToggles.map((label) => (
788         <SettingRow
789           key={label}
790           label={label}
791           checked={dashState[label]}
792           onToggle={() => toggle(setDashState, label)}
793         />
794       )})
795     </div>
796   </SectionCard>
797 })
798
799 {visible.mswipe && (
800   <SectionCard
801     index="06"
802     title="MSWIPE Details"
803     description="Payment gateway credentials and verification details."
804   >
805     <div className="grid gap-4 md:grid-cols-2 max-[640px]:gap-3">
806       {Object.keys(mswipeFields).map((label) => (
807         <Field
808           key={label}
809           label={label}
810           value={mswipeFields[label]}

```

```
811             onChange={value => setField(setMswipeFields, label, value)}
812             placeholder={`Enter ${label.toLowerCase()}`}
813             />
814         )}}
815     </div>
816 </SectionCard>
817 )}
818 </aside>
819 </div>
820 ) : null}
821
822 {!loading && eventId && query && !Object.values(visible).some(Boolean) ? (
823   <div className="mt-6 rounded-[15px] border border-dashed border-(--line) bg-(--surface) px-6 py-1
2 text-center">
824     <div className="font-chillax text-[17px] font-medium text-(--text)">No settings match</div>
825     <div className="mt-1.5 text-[12.5px] text-(--faint)">
826       Nothing found for &quot;{search}&quot;;. Try a different field or service name.
827     </div>
828   </div>
829 ) : null}
830
831 {balanceToast ? (
832   <BalanceSettingToast
833     reference={balanceToast.reference}
834     onClose={() => setBalanceToast(null)}
835   />
836 ) : null}
837 </>
838 );
839 }
```

FILE: apps/dashboard/src/app/event-edit/components/EventEditPrimitives.js

```

1  "use client";
2
3  const FIELD_CLASS =
4    "h-11 w-full min-w-0 rounded-[10px] border border-(--line) bg-(--surface) px-3.5 text-[13px] font-medium text-(--text) outline-none transition placeholder:text-(--faint) focus:border-(--orange) focus:shadow-[0_0_3px_rgba(224,68,32,0.12)] max-[640px]:h-10";
5
6  export function SearchIcon({ className = "h-4 w-4" }) {
7    return (
8      <svg viewBox="0 0 24 24" className={className} fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
9        <circle cx="11" cy="11" r="7" />
10       <path d="m20 20-3.5-3.5" />
11     </svg>
12   );
13 }
14
15 export function UploadIcon({ className = "h-7 w-7" }) {
16   return (
17     <svg viewBox="0 0 24 24" className={className} fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
18       <path d="M12 4v12" />
19       <path d="m7 9 5-5 5 5" />
20       <path d="M5 20h14" />
21     </svg>
22   );
23 }
24
25 export function CalendarIcon({ className = "h-4 w-4" }) {
26   return (
27     <svg viewBox="0 0 24 24" className={className} fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
28       <rect x="3" y="4" width="18" height="18" rx="2" />
29       <path d="M16 2v4" />
30       <path d="M8 2v4" />
31       <path d="M3 10h18" />
32     </svg>
33   );
34 }
35
36 function ChevronIcon() {
37   return (
38     <svg
39       viewBox="0 0 12 12"
40       className="pointer-events-none absolute right-3.5 top-1/2 h-2.5 w-2.5 -translate-y-1/2 text-(--muted) opacity-50"
41       fill="none"
42       stroke="currentColor"
43       strokeWidth="1.5"
44       strokeLinecap="round"
45       aria-hidden
46     >
47       <path d="M2.5 4.5 6 8 9.5 4.5" />
48     </svg>
49   );
50 }
51
52 export function SectionCard({ index, title, description, children, className = "" }) {
53   return (
54     <section
55       className={`min-w-0 rounded-[15px] border border-(--line) border-l-[3px] border-l-(--orange) bg-(--surface) p-4 shadow-(--shadow) max-[640px]:p-3 ${className}`}
56     >
57       <div className="flex flex-wrap items-center gap-2.5 border-b border-(--line2) pb-3">
58         {index ? (
59           <span className="font-vcr grid h-9 w-9 shrink-0 place-items-center rounded-[10px] bg-[linear-gradient(140deg,#e04420,#8b5cf6)] text-[12px] text-white">
60             {index}
61           </span>

```

```

62     ) : null}
63     <div className="min-w-0">
64         <h2 className="font-chillax m-0 text-[18px] font-semibold tracking-[-0.01em] text-(--text) max-
[640px]:text-[16px]">
65             {title}
66         </h2>
67         {description ? (
68             <p className="m-0 mt-0.5 text-[12.5px] font-light text-(--muted) max-[640px]:text-[11.5px]">
69                 {description}
70             </p>
71         ) : null}
72     </div>
73 </div>
74 <div className="pt-3.5 max-[640px]:pt-3">{children}</div>
75 </section>
76 );
77 }
78
79 export function SubPanel({ label, children, className = "" }) {
80     return (
81         <div className={`rounded-[12px] border border-(--line2) bg-(--surface2) p-3.5 max-[640px]:p-3 ${class
Name}`}>
82             <p className="font-vcr mb-3 text-[8.5px] uppercase tracking-[0.16em] text-(--muted) max-[640px]:mb-
2.5">
83                 {label}
84             </p>
85             {children}
86         </div>
87     );
88 }
89
90 export function Field({
91     label,
92     value = "",
93     onChange,
94     placeholder = "",
95     type = "text",
96     icon = null
97 }) {
98     return (
99         <label className="block min-w-0">
100             <span className="font-vcr mb-1.5 block text-[8.5px] uppercase tracking-[0.16em] text-(--muted)">
101                 {label}
102             </span>
103             <span className="relative flex items-center">
104                 <input
105                     type={type}
106                     value={value}
107                     onChange={(event) => onChange?.(event.target.value)}
108                     placeholder={placeholder}
109                     className={` ${FIELD_CLASS} ${icon ? "pr-10" : ""}`>
110                 />
111                 {icon ? (
112                     <span className="pointer-events-none absolute right-3.5 text-(--muted) opacity-60">{icon}</span>
113                 ) : null}
114             </span>
115         </label>
116     );
117 }
118
119 export function SelectField({
120     label,
121     value = "Select Printer",
122     onChange,
123     options = ["Select Printer", "sprin_3", "USB Printer", "Network Printer"]
124 }) {
125     return (
126         <label className="block min-w-0">
127             <span className="font-vcr mb-1.5 block text-[8.5px] uppercase tracking-[0.16em] text-(--muted)">

```

```

128     {label}
129   </span>
130   <span className="relative block">
131     <select
132       value={value}
133       onChange={(event) => onChange?.(event.target.value)}
134       className={` ${FIELD_CLASS} cursor-pointer appearance-none pr-9`}
135     >
136       {options.map((option) => (
137         <option key={option} value={option}>
138           {option}
139         </option>
140       ))}
141     </select>
142     <ChevronIcon />
143   </span>
144 </label>
145 );
146 }
147
148 export function Toggle({ on = true, onToggle }) {
149   return (
150     <button
151       type="button"
152       onClick={onToggle}
153       aria-pressed={on}
154       className={`flex h-7 w-[54px] shrink-0 items-center rounded-full p-1 transition-all duration-200 ma
x-[640px]:h-6 max-[640px]:w-[46px] ${
155         on ? "bg-[linear-gradient(135deg,#E04420,#341CD6)]" : "bg(--line)"
156       }`}
157     >
158       <span
159         className={`h-5 w-5 rounded-full bg-white shadow-sm transition-transform duration-200 max-[640px]
:h-4 max-[640px]:w-4 ${
160           on ? "translate-x-[26px] max-[640px]:translate-x-[22px]" : "translate-x-0"
161         }`}
162       />
163     </button>
164   );
165 }
166
167 export function SettingRow({ label, checked = true, onToggle, children = null }) {
168   return (
169     <div className="flex min-h-[42px] items-center justify-between gap-4 border-b border-(--line2) py-2 l
ast:border-b-0 max-[640px]:min-h-[38px] max-[640px]:gap-2">
170       <span className="min-w-0 text-[12.5px] font-semibold text-(--text) max-[640px]:text-[11.5px]">
171         {label}
172       </span>
173       {children || <Toggle on={checked} onToggle={onToggle} />}
174     </div>
175   );
176 }
177
178 export function UploadBox() {
179   return (
180     <div className="grid min-h-[135px] place-items-center rounded-[12px] border border-dashed border-[rgb
a(224,68,32,0.4)] bg-[rgba(224,68,32,0.04)] px-4 py-5 text-center max-[640px]:min-h-[104px] max-[640px]:p
x-3 max-[640px]:py-4">
181       <div className="flex flex-col items-center justify-center">
182         <span className="grid h-11 w-11 place-items-center rounded-[13px] bg-[linear-gradient(140deg,#e04
420,#8b5cf6)] text-white max-[640px]:h-9 max-[640px]:w-9">
183           <UploadIcon className="h-[19px] w-[19px] max-[640px]:h-4 max-[640px]:w-4" />
184         </span>
185         <p className="font-chillax mt-3 text-[14.5px] font-semibold text-(--text) max-[640px]:mt-2 max-[6
40px]:text-[13px]">
186           Upload event poster
187         </p>
188         <p className="font-vcr mt-2 text-[8.5px] uppercase tracking-[0.16em] text-(--faint) max-[640px]:m
t-1.5">
189           Drag and drop or browse media

```

```
190     </p>
191   </div>
192 </div>
193   );
194 }
195
196 export function TopupRow({ index, cash, token, onCashChange, onTokenChange }) {
197   return (
198     <div className="grid grid-cols-[42px_minmax(0,1fr)_minmax(0,1fr)] items-end gap-3 max-[640px]:grid-co
199 ls-[28px_minmax(0,1fr)_minmax(0,1fr)] max-[640px]:gap-2">
200       <span className="font-vcr pb-3.5 text-[12px] text-(--orange) max-[640px]:pb-3 max-[640px]:text-[10.
201 5px]">
202         #{index}
203       </span>
204       <div className="min-w-0">
205         <Field label="Cash" value={cash} onChange={onCashChange} />
206       </div>
207       <div className="min-w-0">
208         <Field label="Token" value={token} onChange={onTokenChange} />
209       </div>
210     );
211   }
```

FILE: apps/dashboard/src/app/event-edit/page.js

```
1 import Header from "../../components/Header";
2 import EventEditContent from "../../components/EventEditContent";
3
4 export default function EventEditPage() {
5   return (
6     <main
7       className="min-h-screen w-full max-w-full overflow-x-hidden bg-(--bg) pb-10"
8       style={{ fontFamily: "'Poppins', system-ui, sans-serif" }}
9     >
10      <style>{`
11        @media (max-width: 900px) {
12          html,
13          body {
14            overflow-x: hidden;
15          }
16        }
17      `}</style>
18      <Header areaLabel="Edit Event" variant="portal" />
19
20      <div className="mx-auto w-full max-w-[1780px] overflow-x-hidden pr-4 pl-[72px] pt-6 md:pr-7 md:pl-[
2188px] max-[900px]:px-3 max-[900px]:pt-4">
22        <EventEditContent />
23      </div>
24    </main>
25  );
26 }
```


FILE: apps/dashboard/src/app/globals.css

```
1 @import "tailwindcss";
2
3 /* AtomX primary / headline font */
4 @font-face {
5   font-family: "Chillax";
6   src: url("/shared/fonts/Chillex/Chillax-Regular.woff2") format("woff2");
7   font-weight: 400;
8   font-display: swap;
9 }
10 @font-face {
11   font-family: "Chillax";
12   src: url("/shared/fonts/Chillex/Chillax-Medium.woff2") format("woff2");
13   font-weight: 500;
14   font-display: swap;
15 }
16 @font-face {
17   font-family: "Chillax";
18   src: url("/shared/fonts/Chillex/Chillax-Semibold.woff2") format("woff2");
19   font-weight: 600;
20   font-display: swap;
21 }
22 @font-face {
23   font-family: "Chillax";
24   src: url("/shared/fonts/Chillex/Chillax-Bold.woff2") format("woff2");
25   font-weight: 700;
26   font-display: swap;
27 }
28
29 /* AtomX secondary / body font */
30 @font-face {
31   font-family: "Poppins";
32   src: url("/shared/fonts/Poppins/Poppins-Light.ttf") format("truetype");
33   font-weight: 300;
34   font-display: swap;
35 }
36 @font-face {
37   font-family: "Poppins";
38   src: url("/shared/fonts/Poppins/Poppins-Regular.ttf") format("truetype");
39   font-weight: 400;
40   font-display: swap;
41 }
42 @font-face {
43   font-family: "Poppins";
44   src: url("/shared/fonts/Poppins/Poppins-SemiBold.ttf") format("truetype");
45   font-weight: 600;
46   font-display: swap;
47 }
48 @font-face {
49   font-family: "Poppins";
50   src: url("/shared/fonts/Poppins/Poppins-Bold.ttf") format("truetype");
51   font-weight: 700;
52   font-display: swap;
53 }
54
55 /* AtomX accent / data font */
56 @font-face {
57   font-family: "VCR";
58   src: url("/shared/fonts/VCR/VCR_OSD_MONO_1.001.ttf") format("truetype");
59   font-weight: 400;
60   font-display: swap;
61 }
62
63 :root {
64   --background: #ffffff;
65   --foreground: #171717;
66   --color-teal: 20 149 171;
67   --color-orange: 248 140 67;
68   --color-bg: 243 247 251;
69 }
```

```

70  /* AtomX semantic tokens – light theme */
71  --bg: #f2f1ee;
72  --surface: #ffffff;
73  --surface2: #f8f7f5;
74  --text: #1c1c1c;
75  --muted: #71706e;
76  --faint: #9b9a97;
77  --line: rgba(28, 28, 28, 0.12);
78  --line2: rgba(28, 28, 28, 0.06);
79  --chip: rgba(28, 28, 28, 0.045);
80
81  --orange: #e04420;
82  --blue: #341cd6;
83  --lblue: #00a9f2;
84  --purple: #8b5cf6;
85  --egg: #ebebeb;
86
87  --rail: #1c1c1c;
88  --railText: #ebebeb;
89
90  --shadow: 0 1px 2px rgba(28, 28, 28, 0.04), 0 10px 30px -18px rgba(28, 28, 28, 0.25);
91  --shadowUp: 0 2px 4px rgba(28, 28, 28, 0.05), 0 18px 40px -20px rgba(28, 28, 28, 0.35);
92
93  color-scheme: light;
94 }
95
96 html[data-atx="dark"] {
97   --background: #0c0c0c;
98   --foreground: #ededed;
99   --color-bg: 12 12 12;
100
101   --bg: #0c0c0c;
102   --surface: #181818;
103   --surface2: #212120;
104   --text: #ebebeb;
105   --muted: #8e8d8a;
106   --faint: #6b6a67;
107   --line: rgba(235, 235, 235, 0.14);
108   --line2: rgba(235, 235, 235, 0.07);
109   --chip: rgba(235, 235, 235, 0.06);
110
111   --purple: #d5b7ff;
112   --rail: #181818;
113
114   --shadow: 0 1px 2px rgba(0, 0, 0, 0.5), 0 10px 30px -18px rgba(0, 0, 0, 0.9);
115   --shadowUp: 0 2px 6px rgba(0, 0, 0, 0.6), 0 18px 40px -20px rgba(0, 0, 0, 1);
116
117   color-scheme: dark;
118 }
119
120 @theme inline {
121   --color-background: var(--background);
122   --color-foreground: var(--foreground);
123   --font-sans: var(--font-geist-sans);
124   --font-mono: var(--font-geist-mono);
125 }
126
127 .font-chillax {
128   font-family: "Chillax", "Poppins", "Segoe UI", system-ui, sans-serif;
129 }
130
131 .font-vcr {
132   font-family: "VCR", "VCR OSD Mono", ui-monospace, "SFMono-Regular", monospace;
133 }
134
135 body {
136   background: var(--background);
137   color: var(--foreground);
138   font-family: Arial, Helvetica, sans-serif;
139 }

```

FILE: apps/dashboard/src/app/layout.js

```
1 import { Geist, Geist_Mono } from "next/font/google";
2 import "./globals.css";
3 import SessionGuard from "./components/Session/SessionGuard";
4 import PostHogInit from "./components/PostHogInit";
5
6 const geistSans = Geist({
7   variable: "--font-geist-sans",
8   subsets: ["latin"],
9 });
10
11 const geistMono = Geist_Mono({
12   variable: "--font-geist-mono",
13   subsets: ["latin"],
14 });
15
16 export const metadata = {
17   title: "Create Next App",
18   description: "Generated by create next app",
19 };
20
21 // Applies the saved theme before first paint to avoid a light/dark flash.
22 const THEME_BOOT = `
23 try {
24   var t = localStorage.getItem('atomx.theme');
25   if (t === 'dark') document.documentElement.setAttribute('data-atx', 'dark');
26 } catch (e) {}
27 `;
28
29 export default function RootLayout({ children }) {
30   return (
31     <html lang="en" suppressHydrationWarning>
32       <head>
33         <script suppressHydrationWarning dangerouslySetInnerHTML={{ __html: THEME_BOOT }} />
34       </head>
35       <body
36         className={` ${geistSans.variable} ${geistMono.variable} antialiased`>
37         >
38         <PostHogInit />
39         {children}
40         <SessionGuard />
41       </body>
42     </html>
43   );
44 }
```

FILE: apps/dashboard/src/app/page.js

```
1 import Header from "../components/Header";
2
3 export default function Home() {
4   return (
5     <main className="min-h-screen bg-gradient-to-b from-[#fdf7f2] via-white to-[#eef7fa] pb-10">
6       <Header areaLabel="Access Control" />
7
8       <div className="mx-auto mt-6 flex w-full max-w-6xl flex-col gap-6 pr-4 pl-16 md:pr-8 md:pl-20">
9         <section className="rounded-2xl border border-dashed border-slate-200 bg-white/70 p-6 shadow-sm">
10           <p className="text-sm text-slate-600">
11             Dashboard content goes here. Add widgets, charts, and modules as needed.
12           </p>
13         </section>
14       </div>
15     </main>
16   );
17 }
```

FILE: apps/dashboard/src/app/patcha-ny-track/page.js

```
1 import Header from "../components/Header";
2
3 export default function PatchaNyTrackPage() {
4   return (
5     <main className="min-h-screen bg-[color:rgb(var(--color-bg))] pb-10 font-[Poppins,sans-serif]">
6       <Header areaLabel="Patcha-NY-Track" variant="portal" />
7
8       <div className="w-full pr-4 pl-[72px] pt-5 md:pr-7 md:pl-[88px] max-[900px]:px-3 max-[640px]:pt-4">
9         <section className="border-b border-[#d8d8d8] pb-4">
10          <h1 className="text-[1.75rem] font-semibold leading-none text-[#111827] md:text-[1.9rem] max-[640px]:text-[1.25rem]">
11            Patcha-NY-Track
12          </h1>
13        </section>
14      </div>
15    </main>
16  );
17 }
```

FILE: apps/dashboard/src/app/tapx/components/TapXTransactions.js

```

1  "use client";
2
3  import { useCallback, useMemo, useState } from "react";
4  import {
5    fetchTapXWalletCardList,
6    filterEventTransactions
7  } from "../../lib/dashboardApi";
8  import { useDashboardStore } from "../../store/dashboardStore";
9
10 const FIELDS = [
11   { key: "order", label: "ORDER ID", placeholder: "ORDER ID" },
12   { key: "card", label: "CARD ID", placeholder: "CARD ID" },
13   { key: "mobile", label: "MOBILE", placeholder: "MOBILE" },
14   { key: "email", label: "EMAIL", placeholder: "EMAIL" }
15 ];
16
17 const EMPTY_FILTERS = { order: "", card: "", mobile: "", email: "" };
18
19 const TABS = [
20   { id: "tapx", label: "TapX Transactions" },
21   { id: "wallet", label: "Wallet" }
22 ];
23
24 const HERO_COPY = {
25   tapx: {
26     title: "TapX Transactions",
27     subtitle: "Trace any tap by order, card, mobile or email"
28   },
29   wallet: {
30     title: "Wallet",
31     subtitle: "View wallet balances, top-ups and adjustments"
32   }
33 };
34
35 function toPayloadNumber(value) {
36   if (value === "" || value == null) return "";
37   const numberValue = Number(value);
38   return Number.isNaN(numberValue) ? value : numberValue;
39 }
40
41 function cleanText(value, fallback = "-") {
42   if (value === "" || value == null) return fallback;
43   const text = String(value).trim();
44   return text || fallback;
45 }
46
47 function formatAmount(value) {
48   if (value === "" || value == null) return "-";
49   const numberValue = Number(value);
50   if (Number.isNaN(numberValue)) return `₹ ${value}`;
51   return `₹ ${numberValue.toLocaleString("en-IN")}`;
52 }
53
54 function pad(value) {
55   return String(value).padStart(2, "0");
56 }
57
58 function formatTxnDate(value) {
59   if (!value) return "-";
60   const date = new Date(value);
61   if (Number.isNaN(date.getTime())) return String(value);
62   return `${pad(date.getDate())}/${pad(date.getMonth() + 1)}/${pad(date.getFullYear())} ${pad(date.getHours())}:${pad(date.getMinutes())}:${pad(date.getSeconds())}`;
63 }
64
65 function statusTheme(value) {
66   const status = String(value || "").trim().toLowerCase();
67   if (status === "completed" || status === "success") {
68     return { label: "SUCCESS", fg: "#0e8f62", bg: "rgba(14,143,98,0.1)" };

```

```

69   }
70   if (status.includes("pending")) {
71     return { label: "PENDING", fg: "#e08a20", bg: "rgba(224,138,32,0.12)" };
72   }
73   return { label: status ? status.toUpperCase() : "FAILED", fg: "#e04420", bg: "rgba(224,68,32,0.1)" };
74 }
75
76 const AVATARS = [
77   "linear-gradient(140deg,#341cd6,#00a9f2)",
78   "linear-gradient(140deg,#e04420,#8b5cf6)",
79   "linear-gradient(140deg,#8b5cf6,#341cd6)",
80   "linear-gradient(140deg,#1c1c1c,#e04420)"
81 ];
82
83 function getTxnValue(txn, ...keys) {
84   for (const key of keys) {
85     const value = txn?.[key];
86     if (value !== "" && value !== null) return value;
87   }
88   return null;
89 }
90
91 function TapIcon({ className = "h-4 w-4" }) {
92   return (
93     <svg viewBox="0 0 16 16" className={className} fill="none" stroke="currentColor" strokeWidth="1.4" strokeLinecap="round">
94       <circle cx="6.2" cy="8" r="1.2" fill="currentColor" stroke="none" />
95       <path d="M8.8 5.4a3.7 3.7 0 0 1 0 5.2M11 3.2a6.8 6.8 0 0 1 0 9.6" />
96     </svg>
97   );
98 }
99
100 export default function TapXTransactions() {
101   const token = useDashboardStore((state) => state.token);
102   const eventMeta = useDashboardStore((state) => state.eventMeta);
103   const eventDetails = useDashboardStore((state) => state.eventDetails);
104   const eventId = eventMeta?.eventId ?? eventDetails?.id;
105   const eventName = eventDetails?.name ?? eventMeta?.eventName ?? eventMeta?.name ?? "Selected Event";
106
107   const [view, setView] = useState("tapx");
108   const [filters, setFilters] = useState(EMPTY_FILTERS);
109   const [loading, setLoading] = useState(false);
110   const [error, setError] = useState("");
111   const [results, setResults] = useState(null);
112
113   const setField = (key) => (event) =>
114     setFilters((current) => ({ ...current, [key]: event.target.value }));
115
116   const handleViewChange = useCallback(
117     (nextView) => {
118       setView(nextView);
119
120       if (nextView === "wallet") {
121         fetchTapXWalletCardList({ token }).catch((walletError) => {
122           console.error("Failed to load TapX wallet cards", walletError);
123         });
124       }
125     },
126     [token]
127   );
128
129   const runSearch = useCallback(async () => {
130     if (!eventId) {
131       setError("Select an event before searching transactions.");
132       return;
133     }
134     const order = filters.order.trim();
135     const card = filters.card.trim();
136     const mobile = filters.mobile.trim();
137     const email = filters.email.trim().toLowerCase();

```

```

138
139   setLoading(true);
140   setError("");
141   try {
142     const payload = {
143       cardId: card,
144       mobile,
145       receipt: "",
146       dates: [],
147       dates2: [],
148       type: "",
149       status: "",
150       device: "",
151       txnId: order,
152       settings: {
153         event: true,
154         details: false,
155         cardId: Boolean(card),
156         mobile: Boolean(mobile),
157         receipt: false,
158         dates: false,
159         type: false,
160         status: false,
161         stall: false,
162         vendor: false,
163         device: false,
164         txnId: Boolean(order)
165       },
166       event: toPayloadNumber(eventId),
167       vendor: "",
168       stall: ""
169     };
170     const data = await filterEventTransactions({ token, payload });
171     const list = data?.transactions ?? data?.data?.transactions ?? data?.data ?? data?.list ?? data?.ro
ws ?? [];
172     let rows = Array.isArray(list) ? list : [];
173     // EMAIL is not a server-side filter – narrow client-side when provided.
174     if (email) {
175       rows = rows.filter((txn) =>
176         Object.values(txn || {}).
177           .map((value) => String(value ?? "").toLowerCase())
178           .some((value) => value.includes(email))
179       );
180     }
181     setResults(rows);
182   } catch (searchError) {
183     console.error("Failed to search TapX transactions", searchError);
184     setResults([]);
185     setError("Unable to load transactions for this search.");
186   } finally {
187     setLoading(false);
188   }
189 }, [eventId, filters, token]);
190
191 const handleReset = () => {
192   setFilters(EMPTY_FILTERS);
193   setResults(null);
194   setError("");
195 };
196
197 const rows = results ?? [];
198 const isEmpty = rows.length === 0;
199
200 const displayRows = useMemo(
201   () =>
202     rows.map((txn, index) => {
203       const theme = statusTheme(getTxnValue(txn, "txn_status", "status"));
204       return {
205         key: getTxnValue(txn, "txn_id", "txnId", "id") ?? index,
206         order: cleanText(getTxnValue(txn, "txn_id", "txnId", "id")),

```



```

207     amount: formatAmount(getTxnValue(txn, "txn_amount", "amount")),
208     avatar: AVATARS[index % AVATARS.length],
209     theme,
210     fields: [
211       { k: "CARD ID", v: cleanText(getTxnValue(txn, "txn_card_id", "cardId", "card_id")) },
212       { k: "MOBILE", v: cleanText(getTxnValue(txn, "txn_mobile", "mobile")) },
213       { k: "EMAIL", v: cleanText(getTxnValue(txn, "txn_email", "email", "txn_mail")) },
214       { k: "TIME", v: formatTxnDate(getTxnValue(txn, "txn_at", "txn_created_at", "txn_updated_at"))
215     ]
216   };
217 },
218 [rows]
219 );
220
221 return (
222   <
223     <style>{@keyframes atxTapIn{from{opacity:0;transform:translateY(8px)}to{opacity:1;transform:none}}
224   }</style>
225     { /* Hero */
226     <div
227       className="relative overflow-hidden px-[clamp(16px,2.4vw,28px)] pb-[clamp(58px,6vw,78px)] pt-[cla
228 mp(20px,3vw,34px)]"
229       style={{ background: "linear-gradient(120deg,#1C1C1C 0%,#341CD6 62%,#E04420 130%)" }}
230     >
231       <div
232         className="pointer-events-none absolute inset-0 opacity-[0.12]"
233         style={{ background: "repeating-linear-gradient(90deg,#fff 0 1px,transparent 1px 46px)" }}
234         aria-hidden
235       />
236       <div className="relative mx-auto max-w-[1400px]">
237         <div className="flex flex-wrap items-end justify-between gap-5">
238           <div>
239             <div className="font-vcr text-[9.5px] tracking-[0.2em] text-white/70">CASHLESS PAYMENTS</di
240 v>
241             <h1 className="font-chillax mt-2 text-[clamp(26px,3.6vw,42px)] font-semibold leading-[1.03]
242 tracking-[-0.02em] text-white">
243               {HERO_COPY[view].title}
244             </h1>
245             <div className="mt-1.5 text-[13px] font-light text-white/70">
246               {HERO_COPY[view].subtitle} - {eventName}.
247             </div>
248             <div className="font-chillax text-[clamp(34px,6vw,64px)] font-bold leading-[0.8] tracking-[-0
249 .03em] text-white/15">
250               {eventId ? `#${eventId}` : "-"}
251             </div>
252           </div>
253           <div className="mt-5 inline-flex gap-1 rounded-full border border-white/15 bg-white/10 p-1 back
254 drop-blur">
255             {TABS.map((tab) => {
256               const active = view === tab.id;
257               return (
258                 <button
259                   key={tab.id}
260                   type="button"
261                   onClick={() => handleViewChange(tab.id)}
262                   aria-pressed={active}
263                   className={`rounded-full px-4 py-2 text-[13px] font-semibold transition ${
264                     active ? "bg-white text-[#1c1c1c]" : "text-white/70 hover:text-white"
265                   }`}
266                 >
267                   {tab.label}
268                 </button>
269               );
270             })}
271           </div>
272         </div>
273       </div>
274     </div>

```

```

270     </div>
271
272     <div className="relative z-[1] mx-auto max-w-[1400px] px-[clamp(16px,2.4vw,28px)] pb-24">
273       {view === "tapx" ? (
274         <>
275           {/* Filter card */}
276           <div className="mt-[clamp(38px,4.4vw,54px)] rounded-[16px] border border-(--line) bg-(--surface)
p-[clamp(16px,2vw,22px)] shadow-(--shadowUp)">
277             <div className="font-vcr text-[10px] tracking-[0.16em] text-(--muted)">FILTER</div>
278
279             <div className="mt-3 grid grid-cols-1 gap-[clamp(12px,1.4vw,16px)] md:grid-cols-4">
280               {FIELDS.map((field) => (
281                 <div key={field.key} className="min-w-0">
282                   <label className="font-vcr mb-[7px] block text-[8.5px] tracking-[0.16em] text-(--muted)">
283                     {field.label}
284                   </label>
285                   <input
286                     value={filters[field.key]}
287                     onChange={setField(field.key)}
288                     onKeyDown={(event) => {
289                       if (event.key === "Enter") runSearch();
290                     }}
291                     placeholder={field.placeholder}
292                     className="h-[46px] w-full rounded-[10px] border border-(--line) bg-(--surface) px-3.5
text-[13.5px] text-(--text) outline-none transition placeholder:text-(--faint) focus:border-(--orange) fo
cus:shadow-[0_0_3px_rgba(224,68,32,0.12)]"
293                   />
294                 </div>
295               )]}
296             </div>
297
298             <div className="mt-4 flex flex-wrap items-center justify-end gap-2.5">
299               <button
300                 type="button"
301                 onClick={handleReset}
302                 className="flex h-11 items-center justify-center rounded-[10px] border border-(--line) bg-(
--surface) px-[22px] text-[13.5px] font-semibold text-(--muted) transition hover:border-(--orange) hover:
text-(--orange)"
303               >
304                 Reset
305               </button>
306               <button
307                 type="button"
308                 onClick={runSearch}
309                 disabled={loading}
310                 className="flex h-11 items-center justify-center gap-2 rounded-[10px] bg-(--text) px-[26px]
text-[13.5px] font-semibold text-(--bg) transition hover:bg-(--orange) disabled:cursor-not-allowed disab
led:opacity-60"
311               >
312                 {loading ? (
313                   <span className="h-3.5 w-3.5 animate-spin rounded-full border-2 border-white/35 border-t-
white" />
314                 ) : (
315                   <svg viewBox="0 0 16 16" className="h-3.5 w-3.5" fill="none" stroke="currentColor" stroke
Width="1.6" strokeLinecap="round">
316                     <circle cx="7" cy="7" r="4.6" />
317                     <path d="M10.5 10.5 14 14" />
318                   </svg>
319                 )}
320                 <span>Search</span>
321               </button>
322             </div>
323
324             {error ? (
325               <p className="mt-3 rounded-[10px] border border-[rgba(224,68,32,0.25)] bg-[rgba(224,68,32,0.0
6)] px-3 py-2 text-[12px] font-semibold text-(--orange)">
326                 {error}
327               </p>
328             ) : null}
329           </div>

```

```

330
331     { /* Results */ }
332     <section className="mt-[clamp(16px,2vw,22px)] overflow-hidden rounded-[16px] border border-(--lin
e) bg-(--surface) shadow-(--shadow)">
333         <div className="flex items-center gap-3 border-b border-(--line2) px-4 py-[14px]">
334             <div className="font-vcr whitespace-nowrap text-[11px] tracking-[0.14em] text-(--muted)">
335                 RESULTS ( {rows.length} )
336             </div>
337         </div>
338
339         <div className="flex min-h-[300px] flex-col">
340             {isEmpty ? (
341                 <div className="flex min-h-[300px] flex-1 flex-col items-center justify-center gap-3 px-5 p
y-9 text-center">
342                     <span className="grid h-[54px] w-[54px] place-items-center rounded-[15px] border border-d
ashed border-(--line) text-(--faint)">
343                         <TapIcon className="h-[22px] w-[22px]" />
344                     </span>
345                     <div className="font-chillax text-[17px] font-medium text-(--text)">
346                         {results === null ? "No search yet" : "No matching transaction"}
347                     </div>
348                     <div className="max-w-[320px] text-[12.5px] text-(--faint)">
349                         {results === null
350                             ? "Fill any filter above and hit Search – matching taps will appear here."
351                             : "Check the order ID or try another filter combination."}
352                     </div>
353                 </div>
354             ) : (
355                 displayRows.map((row, index) => (
356                     <div
357                         key={row.key}
358                         className="flex flex-wrap items-center gap-[13px] border-b border-(--line2) px-4 py-3 t
ransition-colors last:border-b-0 hover:bg-(--surface2)"
359                         style={{ animation: "atxTapIn 0.3s both", animationDelay: `${index * 0.035}s` }}
360                     >
361                         <span className="grid h-[38px] w-[38px] shrink-0 place-items-center rounded-[11px] text
-white" style={{ background: row.avatar }}>
362                             <TapIcon />
363                         </span>
364
365                         <div className="min-w-0 flex-[1_1_140px]">
366                             <div className="font-vcr text-[7.5px] tracking-[0.15em] text-(--faint)">ORDER ID</div>
367                             <div className="font-vcr mt-0.5 truncate text-[13px] text-(--orange)">{row.order}</di
v>
368                         </div>
369
370                         {row.fields.map((field) => (
371                             <div key={field.k} className="min-w-0 flex-[1_1_120px]">
372                                 <div className="font-vcr text-[7.5px] tracking-[0.15em] text-(--faint)">{field.k}</
div>
373                                 <div className="mt-0.5 truncate text-[13.5px] font-semibold">{field.v}</div>
374                             </div>
375                         )))
376
377                         <div className="min-w-0 flex-[1_1_90px]">
378                             <div className="font-vcr text-[7.5px] tracking-[0.15em] text-(--faint)">AMOUNT</div>
379                             <div className="font-vcr mt-0.5 whitespace-nowrap text-[14px]">{row.amount}</div>
380                         </div>
381
382                         <div
383                             className="flex shrink-0 items-center gap-1.5 rounded-full px-2.5 py-1.5"
384                             style={{ color: row.theme.fg, background: row.theme.bg }}
385                         >
386                             <span className="h-1.5 w-1.5 rounded-full bg-current" />
387                             <span className="font-vcr text-[9px] tracking-[0.12em]">{row.theme.label}</span>
388                         </div>
389                     </div>
390                 ))
391             )}

```

```
392         </div>
393     </section>
394 </>
395     ) : (
396         <div className="-mt-[clamp(38px,4.4vw,54px)] rounded-[16px] border border-(--line) bg-(--surface) p-[clamp(16px,2vw,22px)] shadow-(--shadowUp)">
397             <div className="flex min-h-[320px] flex-col items-center justify-center gap-3 py-12 text-center">
398                 <span className="grid h-[54px] w-[54px] place-items-center rounded-[15px] border border-dashed border-(--line) text-(--faint)">
399                     <svg viewBox="0 0 24 24" className="h-[22px] w-[22px]" fill="none" stroke="currentColor"
400                       strokeWidth="1.6" strokeLinecap="round" strokeLinejoin="round">
401                         <rect x="3" y="6" width="18" height="12" rx="2" />
402                         <path d="M3 10h18" />
403                         <path d="M16.5 14h.5" />
404                     </svg>
405                 </span>
406                 <div className="font-chillax text-[19px] font-medium text-(--text)">Wallet</div>
407                 <div className="max-w-[360px] text-[12.5px] text-(--faint)">
408                     Wallet balances, top-ups and adjustments will appear here.
409                 </div>
410             </div>
411         </div>
412     )}
413 </div>
414 );
415 }
```

FILE: apps/dashboard/src/app/tapx/page.js

```
1 import Header from "../components/Header";
2 import TapXTransactions from "../components/TapXTransactions";
3
4 export default function TapXPage() {
5   return (
6     <main
7       className="min-h-screen bg-(--bg) pb-10"
8       style={{ fontFamily: '"Poppins", system-ui, sans-serif' }}
9     >
10     <Header areaLabel="TapX" variant="portal" />
11     <div className="pl-[60px] max-[900px]:pl-0">
12       <TapXTransactions />
13     </div>
14   </main>
15 );
16 }
```

FILE: apps/dashboard/src/app/timeline/components/SearchBar.js

```
1 "use client";
2
3 export default function SearchBar() {
4   return (
5     <div className="w-full">
6       <label className="sr-only" htmlFor="timeline-search">
7         Search timeline
8       </label>
9       <div className="flex items-center gap-3 rounded-2xl border border-slate-200 bg-white px-4 py-3 shadow-sm">
10         <svg
11           viewBox="0 0 24 24"
12           className="h-5 w-5 text-slate-400"
13           fill="none"
14           stroke="currentColor"
15           strokeWidth="2"
16           strokeLinecap="round"
17           strokeLinejoin="round"
18         >
19           <circle cx="11" cy="11" r="7" />
20           <path d="M20 20l-3.5-3.5" />
21         </svg>
22         <input
23           id="timeline-search"
24           type="text"
25           placeholder="Search by NAME / BOOKING-ID / AWB / CARD ID"
26           className="w-full border-0 bg-transparent text-sm font-medium text-slate-700 placeholder:text-slate-400 focus:outline-none"
27         />
28       </div>
29     </div>
30   );
31 }
```

FILE: apps/dashboard/src/app/timeline/components/TimelineEmptyState.js

```
1 "use client";
2
3 export default function TimelineEmptyState() {
4   return (
5     <div className="flex flex-col items-center justify-center gap-4 rounded-2xl border border-slate-200 b
6       g-white px-6 py-16 text-center shadow-sm">
7       <div className="flex h-16 w-16 items-center justify-center rounded-full bg-[#eaf8fb] text-[color:rg
8         b(var(--color-teal))]">
9         <svg
10           viewBox="0 0 24 24"
11           className="h-8 w-8"
12           fill="none"
13           stroke="currentColor"
14           strokeWidth="1.6"
15           strokeLinecap="round"
16           strokeLinejoin="round"
17         >
18           <circle cx="12" cy="12" r="9" />
19           <path d="M12 7v5l3 2" />
20           <path d="M12 3v2" />
21         </svg>
22       </div>
23       <div>
24         <h3 className="text-lg font-semibold text-slate-700">
25           No timeline events found
26         </h3>
27         <p className="mt-2 text-sm text-slate-500">
28           Events will appear here when available.
29         </p>
30       </div>
31     </div>
32   );
33 }
```

FILE: apps/dashboard/src/app/timeline/page.js

```
1 import Header from "../components/Header";
2 import SearchBar from "../components/SearchBar";
3 import TimelineEmptyState from "../components/TimelineEmptyState";
4
5 export default function TimelinePage() {
6   return (
7     <main className="min-h-screen bg-[color:rgb(var(--color-bg))] pb-10">
8       <Header areaLabel="Timeline" />
9
10      <div className="w-full pr-4 pl-16 md:pr-8 md:pl-20 mt-6">
11        <div className="w-full rounded-3xl bg-white px-6 py-6 shadow-[0_24px_60px_rgba(15,23,42,0.12)]">
12          <SearchBar />
13        </div>
14
15        <div className="mt-6 w-full rounded-3xl bg-white px-6 py-6 shadow-[0_24px_60px_rgba(15,23,42,0.12)]">
16          <div className="text-sm font-semibold uppercase tracking-[0.2em] text-slate-500">
17            Timeline Events (0)
18          </div>
19          <div className="mt-6">
20            <TimelineEmptyState />
21          </div>
22        </div>
23      </div>
24    </main>
25  );
26 }
```


FILE: apps/dashboard/src/app/transactions/components/TransactionFilters.js

```
1 "use client";
2
3 import { Fragment, useCallback, useEffect, useMemo, useRef, useState } from "react";
4 import {
5   fetchEventTransactionDetails,
6   fetchStalls,
7   fetchVendors,
8   filterEventTransactions,
9   updateEventTransactionStatus
10 } from "../../../lib/dashboardApi";
11 import { useDashboardStore } from "../../../store/dashboardStore";
12
13 const TRANSACTION_TYPES = [
14   { label: "SALE", value: "sale" },
15   { label: "TOPUP", value: "topup" },
16   { label: "RETURN", value: "return" },
17   { label: "REVERSAL TOPUP", value: "reversal_topup" },
18   { label: "REVERSAL SALE", value: "reversal_sale" }
19 ];
20
21 const STATUS_OPTIONS = [
22   { label: "All Status", value: "" },
23   { label: "Completed", value: "completed" },
24   { label: "Void", value: "void" },
25   { label: "Success", value: "success" },
26   { label: "Failed", value: "failed" },
27   { label: "Pending", value: "pending" }
28 ];
29
30 function pad(value) {
31   return String(value).padStart(2, "0");
32 }
33
34 function toDateInputValue(date) {
35   if (!(date instanceof Date) || Number.isNaN(date.getTime())) return "";
36   return `${date.getFullYear()}-${pad(date.getMonth() + 1)}-${pad(date.getDate())}`;
37 }
38
39 function toDateTimeText(dateValue, timeValue) {
40   if (!dateValue) return "";
41   return `${dateValue} ${timeValue || "00:00:00"}`;
42 }
43
44 function toPayloadIsoDate(dateValue, timeValue) {
45   if (!dateValue) return "";
46   const date = new Date(`${dateValue}T${timeValue || "00:00:00"}`);
47   if (Number.isNaN(date.getTime())) return "";
48   return date.toISOString();
49 }
50
51 function toPayloadNumber(value) {
52   if (value === "" || value == null) return "";
53   const numberValue = Number(value);
54   return Number.isNaN(numberValue) ? value : numberValue;
55 }
56
57 function todayDateValue() {
58   return toDateInputValue(new Date());
59 }
60
61 function getVendorId(vendor) {
62   return vendor?.id ?? vendor?.vendorId ?? vendor?.vendor_id ?? vendor?.login ?? vendor?.password ?? "";
63 }
64
65 function getVendorName(vendor) {
66   return vendor?.name ?? vendor?.vendorName ?? vendor?.vendor_name ?? vendor?.label ?? "-";
67 }
68
69 function getStallId(stall) {
```

```
70   return stall?.id ?? stall?.stallId ?? stall?.stall_id ?? "";
71 }
72
73 function getStallName(stall) {
74   return stall?.name ?? stall?.stallName ?? stall?.stall_name ?? "-";
75 }
76
77 function formatMoney(value) {
78   if (value === "" || value == null) return "-";
79   const numberValue = Number(value);
80   if (Number.isNaN(numberValue)) return String(value);
81   return numberValue.toLocaleString("en-IN", {
82     minimumFractionDigits: 2,
83     maximumFractionDigits: 2
84   });
85 }
86
87 function formatTxnDate(value) {
88   if (!value) return "-";
89   const date = new Date(value);
90   if (Number.isNaN(date.getTime())) return String(value);
91   const dateText = `${date.getFullYear()}-${pad(date.getMonth() + 1)}-${pad(date.getDate())}`;
92   const timeText = `${pad(date.getHours())}:${pad(date.getMinutes())}:${pad(date.getSeconds())}`;
93   return `${dateText} ${timeText}`;
94 }
95
96 function cleanText(value, fallback = "-") {
97   if (value === "" || value == null) return fallback;
98   const text = String(value).trim();
99   return text || fallback;
100 }
101
102 function normalizeTxnType(value) {
103   return cleanText(value).replace(/_/g, " ").toUpperCase();
104 }
105
106 function normalizeTxnStatus(value) {
107   return cleanText(value, "").toLowerCase();
108 }
109
110 function getNextTxnStatus(value) {
111   const status = normalizeTxnStatus(value);
112   if (status === "void") return "completed";
113   if (status === "completed") return "void";
114   return null;
115 }
116
117 function getTxnStatusClass(value) {
118   const status = normalizeTxnStatus(value);
119   if (status === "completed") return "text-[#0e8f62]";
120   if (status === "pending" || status.includes("pending")) return "text-[#e08a20]";
121   return "text-(--orange)";
122 }
123
124 function getTransactionId(transaction) {
125   return transaction?.txn_id ?? transaction?.txId ?? transaction?.transactionId ?? transaction?.id;
126 }
127
128 function getTransactionValue(transaction, ...keys) {
129   for (const key of keys) {
130     const value = transaction?.[key];
131     if (value !== "" && value != null) return value;
132   }
133   return null;
134 }
135
136 function mergeTransactionDetails(transaction, details) {
137   if (!details) return transaction;
138
139   return {
```

```

140     ...transaction,
141     ...details,
142     txn_id:
143         details.txn_id ??
144         details.txId ??
145         details.transactionId ??
146         transaction.txn_id ??
147         transaction.txId ??
148         transaction.id,
149     txn_status:
150         transaction.txn_status ??
151         transaction.status ??
152         details.txn_status ??
153         details.status
154 };
155 }
156
157 function ChevronIcon({ open }) {
158     return (
159         <svg
160             viewBox="0 0 24 24"
161             className={`h-4 w-4 transition ${open ? "rotate-90" : ""}`}
162             fill="none"
163             stroke="currentColor"
164             strokeWidth="2.2"
165             strokeLinecap="round"
166             strokeLinejoin="round"
167         >
168             <path d="m9 18 6-6-6-6" />
169         </svg>
170     );
171 }
172
173 function SearchIcon({ className = "h-4 w-4" }) {
174     return (
175         <svg viewBox="0 0 24 24" className={className} fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
176             <circle cx="11" cy="11" r="7" />
177             <path d="m20 20-3.5-3.5" />
178         </svg>
179     );
180 }
181
182 function SortIcon() {
183     return (
184         <span className="inline-grid gap-[2px] text-(--faint)" aria-hidden>
185             <span className="h-0 w-0 border-x-[4px] border-b-[5px] border-x-transparent border-b-current" />
186             <span className="h-0 w-0 border-x-[4px] border-t-[5px] border-x-transparent border-t-current" />
187         </span>
188     );
189 }
190
191 function DownloadIcon({ className = "h-4.5 w-4.5" }) {
192     return (
193         <svg viewBox="0 0 24 24" className={className} fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
194             <path d="M12 3v12" />
195             <path d="m7 10 5 5 5-5" />
196             <path d="M5 21h14" />
197         </svg>
198     );
199 }
200
201 function ClockIcon() {
202     return (
203         <svg viewBox="0 0 24 24" className="h-4.5 w-4.5" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
204             <circle cx="12" cy="12" r="9" />
205             <path d="M12 7v5l3 2" />
206         </svg>

```

```

207   );
208 }
209
210 function EventIcon() {
211   return (
212     <svg viewBox="0 0 24 24" className="h-5 w-5" fill="none" stroke="currentColor" strokeWidth="2" stroke
Lineacap="round" strokeLinejoin="round">
213       <circle cx="12" cy="12" r="9" />
214       <path d="M3 12h18" />
215       <path d="M12 3v18" />
216     </svg>
217   );
218 }
219
220 function SelectArrow() {
221   return (
222     <svg viewBox="0 0 24 24" className="pointer-events-none h-4 w-4 text-(--text)" fill="currentColor" ar
ia-hidden>
223       <path d="M7 9.5h10L12 15z" />
224     </svg>
225   );
226 }
227
228 function Toggle({ checked, onChange, label }) {
229   return (
230     <button
231       type="button"
232       onClick={() => onChange(!checked)}
233       className="flex h-full items-center gap-3 border-r border-(--line) px-3 max-[640px]:gap-2 max-[640p
x]:px-2"
234       aria-pressed={checked}
235     >
236       <span
237         className={`relative h-5 w-9 rounded-full transition max-[640px]:h-4 max-[640px]:w-8 ${
238           checked ? "bg-[#0e8f62]" : "bg-(--surface2)"
239         }`}
240       >
241         <span
242           className={`absolute top-0.5 h-4 w-4 rounded-full bg-(--surface) shadow transition max-[640px]:
h-3 max-[640px]:w-3 ${
243             checked ? "left-[18px] max-[640px]:left-[17px]" : "left-0.5"
244           }`}
245         />
246       </span>
247       <span className="text-[0.7rem] font-bold uppercase tracking-[0.02em] text-(--muted) max-[640px]:tex
t-[0.58rem]">
248         {label}
249       </span>
250     </button>
251   );
252 }
253
254 function FilterCheck({ checked, onChange, label }) {
255   return (
256     <button
257       type="button"
258       onClick={() => onChange(!checked)}
259       className="flex h-full items-center gap-3 border-r border-(--line) px-3 max-[640px]:gap-2 max-[640p
x]:px-2"
260       aria-pressed={checked}
261     >
262       <span
263         className={`grid h-[18px] w-[18px] place-items-center rounded-md border shadow-[0_3px_8px_rgba(15
,23,42,0.04)] transition max-[640px]:h-[14px] max-[640px]:w-[14px] ${
264           checked ? "border-(--orange) bg-(--orange) text-white" : "border-(--line) bg-(--surface) text-t
ransparent"
265         }`}
266       >
267         <svg viewBox="0 0 24 24" className="h-3 w-3" fill="none" stroke="currentColor" strokeWidth="3" st
rokeLinecap="round" strokeLinejoin="round">

```

```

268         <path d="m5 12 4 4L19 6" />
269     </svg>
270 </span>
271 <span className="text-[0.7rem] font-bold uppercase tracking-[0.02em] text-(--muted) max-[640px]:tex
t-[0.58rem]">
272     {label}
273 </span>
274 </button>
275 );
276 }
277
278 function InputField({ label, value, onChange, placeholder, enabled, onEnabledChange }) {
279     return (
280         <label className={`grid h-12 grid-cols-[170px_1fr] items-center overflow-hidden rounded-lg border bg-
(--surface) shadow-[0_8px_20px_rgba(15,23,42,0.025)] transition max-[640px]:h-9 max-[640px]:grid-cols-[96
px_minmax(0,1fr)] ${
281             enabled ? "border-(--line)" : "border-(--line2) bg-(--surface2)"
282         }`>
283             <FilterCheck checked={enabled} onChange={onEnabledChange} label={label} />
284             <input
285                 type="text"
286                 disabled={!enabled}
287                 value={value}
288                 onChange={(event) => onChange(event.target.value)}
289                 placeholder={placeholder}
290                 className="min-w-0 bg-transparent px-4 text-[0.82rem] font-semibold text-(--text) outline-none pl
aceholder:text-(--faint) disabled:text-(--faint) max-[640px]:px-2.5 max-[640px]:text-[0.68rem]"
291             />
292         </label>
293     );
294 }
295
296 function SelectField({ label, value, onChange, options, enabled, onEnabledChange, disabled = false }) {
297     const isDisabled = disabled || !enabled;
298
299     return (
300         <label className={`grid h-12 grid-cols-[170px_1fr] items-center overflow-hidden rounded-lg border bg-
(--surface) shadow-[0_8px_20px_rgba(15,23,42,0.025)] transition max-[640px]:h-9 max-[640px]:grid-cols-[96
px_minmax(0,1fr)] ${
301             enabled ? "border-(--line)" : "border-(--line2) bg-(--surface2)"
302         }`>
303             <FilterCheck checked={enabled} onChange={onEnabledChange} label={label} />
304             <span className="relative flex min-w-0 items-center px-4 max-[640px]:px-2.5">
305                 <select
306                     value={value}
307                     onChange={(event) => onChange(event.target.value)}
308                     disabled={isDisabled}
309                     className="h-full min-w-0 flex-1 appearance-none bg-transparent pr-8 text-[0.82rem] font-semibo
ld text-(--text) outline-none disabled:text-(--faint) max-[640px]:text-[0.68rem]"
310                 >
311                     {options.map((option) => (
312                         <option key={`${label}-${option.value}`} value={option.value}>
313                             {option.label}
314                         </option>
315                     ))}
316                 </select>
317                 <SelectArrow />
318             </span>
319         </label>
320     );
321 }
322
323 function DateRangeField({ enabled, onEnabledChange, range, onChange }) {
324     const [open, setOpen] = useState(false);
325     const pickerRef = useRef(null);
326     const startText = toDateTimeText(range.startDate, range.startTime);
327     const endText = toDateTimeText(range.endDate, range.endTime);
328
329     useEffect(() => {
330         function handleOutsideClick(event) {

```

```

331     if (!pickerRef.current || pickerRef.current.contains(event.target)) return;
332     setOpen(false);
333   }
334
335   document.addEventListener("mousedown", handleOutsideClick);
336   return () => document.removeEventListener("mousedown", handleOutsideClick);
337 }, []);
338
339 const updateRange = (patch) => onChange({ ...range, ...patch });
340
341 const applyQuickRange = (daysBack) => {
342   const end = new Date();
343   const start = new Date();
344   start.setDate(end.getDate() - daysBack);
345   updateRange({
346     startDate: toDateInputValue(start),
347     startTime: "00:00:00",
348     endDate: toDateInputValue(end),
349     endTime: "00:00:00"
350   });
351 };
352
353 return (
354   <div className="relative" ref={pickerRef}>
355     <div
356       className={`grid h-12 grid-cols-[170px_48px_1fr_56px_1fr] items-center overflow-hidden rounded-lg
357       border bg-(--surface) shadow-[0_8px_20px_rgba(15,23,42,0.025)] max-[640px]:h-auto max-[640px]:min-h-9 ma
358       x-[640px]:grid-cols-[96px_28px_minmax(0,1fr)] ${
359         open ? "border-(--orange) shadow-[0_0_3px_rgba(224,68,32,0.12)]" : "border-(--line)"
360       }`}
361     >
362       <Toggle checked={enabled} onChange={onEnabledChange} label="Dates" />
363       <button
364         type="button"
365         onClick={() => setOpen((current) => !current)}
366         disabled={!enabled}
367         className="grid h-full place-items-center border-r border-(--line) text-(--muted)"
368         aria-label="Open date range picker"
369       >
370         <ClockIcon />
371       </button>
372       <button
373         type="button"
374         onClick={() => setOpen((current) => !current)}
375         disabled={!enabled}
376         className="min-w-0 bg-transparent px-5 text-left text-[0.92rem] font-normal text-(--muted) outl
377         ine-none max-[640px]:px-2 max-[640px]:text-[0.66rem]"
378       >
379         <span className="block truncate">{startText || "Start"}</span>
380       </button>
381       <button
382         type="button"
383         onClick={() => setOpen((current) => !current)}
384         disabled={!enabled}
385         className="text-center text-[1rem] text-(--muted) max-[640px]:hidden"
386       >
387         -
388       </button>
389       <button
390         type="button"
391         onClick={() => setOpen((current) => !current)}
392         disabled={!enabled}
393         className="min-w-0 bg-transparent px-5 text-left text-[0.92rem] font-normal text-(--muted) outl
394         ine-none max-[640px]:col-span-3 max-[640px]:border-t max-[640px]:border-(--line) max-[640px]:px-2 max-[64
395         0px]:py-2 max-[640px]:text-[0.66rem]"

```

```

396     {open && enabled ? (
397         <div className="absolute left-0 top-[calc(100%+8px)] z-30 grid w-full min-w-[720px] grid-cols-[15
0px_1fr] overflow-hidden rounded-lg border border-(--line) bg-(--surface) shadow-[0_22px_60px_rgba(15,23,
42,0.18)] max-[900px]:min-w-0 max-[900px]:grid-cols-1">
398             <div className="border-r border-(--line) p-4 text-[0.84rem] font-normal text-(--muted) max-[900
px]:flex max-[900px]:gap-2 max-[900px]:overflow-x-auto max-[900px]:border-b max-[900px]:border-r-0 max-[9
00px]:p-2">
399                 <button type="button" onClick={() => applyQuickRange(7)} className="block rounded-md px-2 py-
2 text-left hover:bg-(--surface2) max-[900px]:min-w-fit">
400                     Last week
401                 </button>
402                 <button type="button" onClick={() => applyQuickRange(30)} className="block rounded-md px-2 py
-2 text-left hover:bg-(--surface2) max-[900px]:min-w-fit">
403                     Last month
404                 </button>
405                 <button type="button" onClick={() => applyQuickRange(90)} className="block rounded-md px-2 py
-2 text-left hover:bg-(--surface2) max-[900px]:min-w-fit">
406                     Last 3 months
407                 </button>
408             </div>
409
410             <div>
411                 <div className="grid grid-cols-[1fr_28px_1fr] gap-3 border-b border-(--line) p-3 max-[640px]:
grid-cols-1">
412                     <div className="grid grid-cols-[1fr_120px] gap-2 max-[640px]:grid-cols-1">
413                         <input
414                             type="date"
415                             value={range.startDate}
416                             onChange={(event) => updateRange({ startDate: event.target.value })}
417                             className="h-10 rounded-md border border-(--line) px-3 text-[0.86rem] text-(--muted) ou
tline-none focus:border-(--orange)"
418                         />
419                         <input
420                             type="time"
421                             step="1"
422                             value={range.startTime}
423                             onChange={(event) => updateRange({ startTime: event.target.value })}
424                             className="h-10 rounded-md border border-(--line) px-3 text-[0.86rem] text-(--muted) ou
tline-none focus:border-(--orange)"
425                         />
426                     </div>
427                     <span className="grid place-items-center text-xl text-(--text) max-[640px]:hidden"></span>
428                     <div className="grid grid-cols-[1fr_120px] gap-2 max-[640px]:grid-cols-1">
429                         <input
430                             type="date"
431                             value={range.endDate}
432                             onChange={(event) => updateRange({ endDate: event.target.value })}
433                             className="h-10 rounded-md border border-(--line) px-3 text-[0.86rem] text-(--muted) ou
tline-none focus:border-(--orange)"
434                         />
435                         <input
436                             type="time"
437                             step="1"
438                             value={range.endTime}
439                             onChange={(event) => updateRange({ endTime: event.target.value })}
440                             className="h-10 rounded-md border border-(--line) px-3 text-[0.86rem] text-(--muted) ou
tline-none focus:border-(--orange)"
441                         />
442                     </div>
443                 </div>
444
445                 <div className="flex justify-end gap-3 p-3">
446                     <button
447                         type="button"
448                         onClick={() =>
449                             updateRange({
450                                 startDate: todayDateValue(),
451                                 startTime: "00:00:00",
452                                 endDate: todayDateValue(),
453                                 endTime: "00:00:00"

```

```

454         })
455     }
456     className="h-9 px-3 text-[0.82rem] font-semibold text-(--orange)"
457 >
458     Clear
459 </button>
460 <button
461     type="button"
462     onClick={() => setOpen(false)}
463     className="h-9 rounded-md border border-(--line) bg-(--surface) px-4 text-[0.82rem] font-
semibold text-(--muted) hover:border-(--orange)"
464 >
465     OK
466 </button>
467 </div>
468 </div>
469 </div>
470 ) : null}
471 </div>
472 );
473 }
474
475 function TransactionStatusAction({ transaction, onUpdateStatus }) {
476     const [confirming, setConfirming] = useState(false);
477     const [updating, setUpdating] = useState(false);
478     const [error, setError] = useState("");
479     const nextStatus = getNextTxnStatus(transaction.txn_status);
480     const txId = transaction.txn_id ?? transaction.txId ?? transaction.id;
481
482     if (!nextStatus || txId === "" || txId == null) return null;
483
484     const actionLabel = nextStatus === "completed" ? "Mark Completed" : "Mark Void";
485
486     const handleConfirm = async () => {
487         setUpdating(true);
488         setError("");
489         try {
490             await onUpdateStatus({ txId, status: nextStatus });
491             setConfirming(false);
492         } catch {
493             setError("Status could not be updated. Please try again.");
494         } finally {
495             setUpdating(false);
496         }
497     };
498
499     return (
500         <div className="mt-4 border-t border-(--line2) pt-3">
501             {confirming ? (
502                 <div className="flex flex-wrap items-center justify-between gap-3 rounded-lg bg-(--surface2) px-3
py-2.5">
503                     <p className="text-[0.72rem] font-semibold text-(--muted)">
504                         Change this transaction to <span className="capitalize text-(--text)">{nextStatus}</span>?
505                     </p>
506                     <div className="flex items-center gap-2">
507                         <button
508                             type="button"
509                             onClick={() => {
510                                 setConfirming(false);
511                                 setError("");
512                             }}
513                             disabled={updating}
514                             className="h-8 rounded-md border border-(--line) bg-(--surface) px-3 text-[0.7rem] font-sem
ibold text-(--muted) transition hover:border-(--muted) disabled:opacity-50"
515                         >
516                             Cancel
517                         </button>
518                         <button
519                             type="button"
520                             onClick={handleConfirm}

```



```

521         disabled={updating}
522         className={`h-8 rounded-md px-3 text-[0.7rem] font-bold text-(--bg) transition disabled:cur
sor-not-allowed disabled:opacity-55 ${
523             nextStatus === "completed"
524             ? "bg-(--text) hover:bg-[#0e8f62]"
525             : "bg-(--orange) hover:bg-(--orange)"
526         }`}
527     >
528         {updating ? "Updating..." : `Confirm ${nextStatus === "completed" ? "Completed" : "Void"}`}
529     </button>
530 </div>
531 </div>
532 ) : (
533     <div className="flex justify-end">
534         <button
535             type="button"
536             onClick={() => setConfirming(true)}
537             className={`h-9 rounded-lg px-4 text-[0.74rem] font-bold transition ${
538                 nextStatus === "completed"
539                 ? "bg-(--text) text-(--bg) hover:bg-[#0e8f62]"
540                 : "border border-(--orange) bg-[rgba(224,68,32,0.06)] text-(--orange) hover:bg-(--orange)
541                 hover:text-white"
542             }`}
543             >
544                 {actionLabel}
545             </button>
546         </div>
547     )}
548     {error ? <p className="mt-2 text-right text-[0.68rem] font-semibold text-(--orange)">{error}</p> :
549     null}
550 </div>
551 );
552 }
553
554 function TransactionDetailContent({
555     transaction,
556     loading,
557     error,
558     onRetry,
559     onUpdateStatus
560 }) {
561     if (loading) {
562         return (
563             <div className="grid min-h-36 place-items-center rounded-xl border border-(--line2) bg-(--surface)
564             p-4 text-center shadow-[0_14px_35px_rgba(15,23,42,0.05)]">
565                 <div>
566                     <span className="mx-auto block h-7 w-7 animate-spin rounded-full border-2 border-(--line) borde
567                     r-t-[#E04420]" />
568                     <p className="mt-3 text-[0.74rem] font-semibold text-(--muted)">Loading transaction details...<
569                     /p>
570                 </div>
571             </div>
572         );
573     }
574     if (error) {
575         return (
576             <div className="grid min-h-32 place-items-center rounded-xl border border-[rgba(224,68,32,0.25)] bg
577             -[rgba(224,68,32,0.06)] p-4 text-center">
578                 <div>
579                     <p className="text-[0.74rem] font-semibold text-(--orange)">{error}</p>
580                     <button
581                         type="button"
582                         onClick={onRetry}
583                         className="mt-3 h-8 rounded-md bg-(--text) px-3 text-[0.7rem] font-bold text-(--bg) transitio
584                         n hover:bg-(--orange)"
585                     >
586                         Retry
587                     </button>
588                 </div>
589             </div>
590         );
591     }
592 }

```

```

583     </div>
584   );
585 }
586
587 const details = [
588   ["Card ID", cleanText(getTransactionValue(transaction, "txn_card_id", "cardId", "card_id"))],
589   ["Card UID", cleanText(getTransactionValue(transaction, "txn_card_uid", "cardUid", "card_uid"))],
590   ["TXN Counter", cleanText(getTransactionValue(transaction, "txn_counter", "txnCounter", "counter"))],
591   ["TXN ID", cleanText(getTransactionId(transaction))],
592   ["Receipt", cleanText(getTransactionValue(transaction, "txn_receipt", "receipt"))],
593   ["Invoice", cleanText(getTransactionValue(transaction, "txn_invoice", "invoice"))],
594   ["Balance Before", formatMoney(getTransactionValue(transaction, "txn_bal_before", "balanceBefore", "balanceBefore"))],
595   ["Balance After", formatMoney(getTransactionValue(transaction, "txn_bal_after", "balanceAfter", "balanceAfter"))],
596   ["Mobile", cleanText(getTransactionValue(transaction, "txn_mobile", "mobile"))],
597   ["Name", cleanText(getTransactionValue(transaction, "txn_name", "name"))],
598   ["Reference", cleanText(getTransactionValue(transaction, "txn_reference", "reference"))],
599   ["App Version", cleanText(getTransactionValue(transaction, "txn_app_version", "appVersion", "version"))]
600 ];
601
602 return (
603   <div className="mx-auto max-w-[860px] rounded-xl border border-(--line2) bg-(--surface) p-4 shadow-[0 14px 35px rgba(15,23,42,0.05)]">
604     <div className="mb-3 flex items-center justify-center gap-2">
605       <span className="h-px flex-1 bg-(--surface2)" />
606       <span className="text-[0.68rem] font-semibold uppercase tracking-[0.08em] text-(--faint)">Transaction Info</span>
607     <div className="h-px flex-1 bg-(--surface2)" />
608   </div>
609   <dl className="grid grid-cols-2 gap-x-7 gap-y-3 max-[760px]:grid-cols-1">
610     {details.map(([label, value]) => (
611       <div key={label} className="grid grid-cols-[130px_1fr] items-center gap-3 border-b border-(--line2) pb-2 last:border-b-0 max-[520px]:grid-cols-1 max-[520px]:gap-1">
612         <dt className="text-[0.66rem] font-semibold uppercase tracking-[0.05em] text-(--faint)">{label}</dt>
613         <dd className="min-w-0 break-words text-[0.9rem] font-medium text-(--muted)">{value}</dd>
614       </div>
615     ))}
616   </dl>
617   <TransactionStatusAction transaction={transaction} onUpdateStatus={onUpdateStatus} />
618 </div>
619 );
620 }
621
622 function TransactionDetailRow(props) {
623   return (
624     <tr className="bg-(--surface2)">
625       <td colSpan={10} className="border-b border-(--line2) px-4 py-5">
626         <TransactionDetailContent {...props} />
627       </td>
628     </tr>
629   );
630 }
631
632 function TransactionResults({ transactions, token, onUpdateStatus }) {
633   const [query, setQuery] = useState("");
634   const [expandedTxnId, setExpandedTxnId] = useState(null);
635   const [detailsByTxnId, setDetailsByTxnId] = useState({});
636
637   const loadTransactionDetails = useCallback(
638     async (transaction, rowKey, force = false) => {
639       const cacheKey = String(rowKey);
640       const txId = getTransactionId(transaction);
641       if (txId === "" || txId == null) {
642         setDetailsByTxnId((current) => ({
643           ...current,
644           [cacheKey]: {
645             data: null,

```

```
646         loading: false,
647         error: "This transaction does not include a transaction ID."
648     }
649     }));
650     return;
651 }
652
653 if (!force && detailsByTxnId[cacheKey]?.data) return;
654
655 setDetailsByTxnId((current) => ({
656     ...current,
657     [cacheKey]: { ...current[cacheKey], loading: true, error: "" }
658 }));
659
660 try {
661     const details = await fetchEventTransactionDetails({
662         token,
663         txId,
664         dedupe: !force
665     });
666     setDetailsByTxnId((current) => ({
667         ...current,
668         [cacheKey]: {
669             data: details,
670             loading: false,
671             error: details ? "" : "No transaction details were returned."
672         }
673     }));
674 } catch (detailError) {
675     console.error("Failed to load transaction details", detailError);
676     setDetailsByTxnId((current) => ({
677         ...current,
678         [cacheKey]: {
679             data: current[cacheKey]?.data ?? null,
680             loading: false,
681             error: "Unable to load transaction details."
682         }
683     }));
684 }
685 },
686 [detailsByTxnId, token]
687 );
688
689 const toggleTransactionDetails = useCallback(
690     (transaction, rowKey) => {
691         if (expandedTxnId === rowKey) {
692             setExpandedTxnId(null);
693             return;
694         }
695
696         setExpandedTxnId(rowKey);
697         loadTransactionDetails(transaction, rowKey);
698     },
699     [expandedTxnId, loadTransactionDetails]
700 );
701
702 const updateExpandedTransactionStatus = useCallback(
703     async ({ txId, status }) => {
704         await onUpdateStatus({ txId, status });
705         setDetailsByTxnId((current) => {
706             const next = { ...current };
707             Object.keys(next).forEach((key) => {
708                 const details = next[key]?.data;
709                 if (String(getTransactionId(details)) === String(txId)) {
710                     next[key] = {
711                         ...next[key],
712                         data: { ...details, txn_status: status, status }
713                     };
714                 }
715             });
716         });
717     }
718 );
```

```

716     return next;
717   });
718 },
719 [onUpdateStatus]
720 );
721
722 const getDetailProps = useCallback(
723   (transaction, rowKey) => {
724     const detailState = detailsByTxnId[String(rowKey)] ?? {};
725     return {
726       transaction: mergeTransactionDetails(transaction, detailState.data),
727       loading: Boolean(detailState.loading),
728       error: detailState.error ?? "",
729       onRetry: () => loadTransactionDetails(transaction, rowKey, true),
730       onUpdateStatus: updateExpandedTransactionStatus
731     };
732   },
733   [detailsByTxnId, loadTransactionDetails, updateExpandedTransactionStatus]
734 );
735
736 const filteredTransactions = useMemo(() => {
737   const searchText = query.trim().toLowerCase();
738   if (!searchText) return transactions;
739
740   return transactions.filter((transaction) =>
741     [
742       transaction.txn_id,
743       transaction.txn_type,
744       transaction.txn_invoice,
745       transaction.txn_receipt,
746       transaction.txn_card_id,
747       transaction.txn_card_uid,
748       transaction.txn_device_print_id,
749       transaction.txn_mode,
750       transaction.txn_mode_info,
751       transaction.txn_mobile,
752       transaction.stall_name,
753       transaction.vendor_name,
754       transaction.event_name,
755       transaction.txn_status
756     ]
757     .map((value) => cleanText(value, ""))
758     .join(" ")
759     .toLowerCase()
760     .includes(searchText)
761   );
762 }, [query, transactions]);
763
764 return (
765   <section className="mt-3 rounded-xl border border-(--line) border-l-[4px] border-l-[#E04420] bg-(--surface) shadow-[0_16px_45px_rgba(15,23,42,0.08)] max-[640px]:rounded-lg">
766     <div className="flex flex-col gap-3 border-b border-(--line2) px-4 py-3 md:flex-row md:items-center md:justify-between max-[640px]:px-3 max-[640px]:py-2.5">
767       <div>
768         <p className="text-[0.68rem] font-bold uppercase tracking-[0.12em] text-(--orange)">Transaction
769         Results</p>
770         <h3 className="mt-1 text-[1rem] font-semibold text-(--text) max-[640px]:text-[0.9rem]">
771           {filteredTransactions.length} listed
772         </h3>
773       </div>
774       <label className="flex h-10 min-w-[320px] items-center gap-3 rounded-lg border border-(--line) bg-(--surface) px-3 shadow-[0_8px_18px_rgba(15,23,42,0.035)] focus-within:border-(--orange) focus-within:shadow-[0_0_0_3px_rgba(224,68,32,0.12)] max-[640px]:h-9 max-[640px]:min-w-0 max-[640px]:w-full">
775         <SearchIcon className="h-4 w-4 text-(--faint)" />
776         <input
777           type="search"
778           value={query}
779           onChange={(event) => setQuery(event.target.value)}
780           placeholder="Search transactions"
781           className="min-w-0 flex-1 bg-transparent text-[0.78rem] font-medium text-(--text) outline-non

```

```

    e placeholder:text-(--faint) max-[640px]:text-[0.68rem]"
781     />
782   </label>
783 </div>
784
785   {filteredTransactions.length ? (
786     <
787       <div className="overflow-x-auto max-[760px]:hidden">
788         <table className="min-w-[1180px] w-full border-collapse">
789           <thead>
790             <tr className="text-left">
791               <th className="w-12 border-b border-(--line2) px-4 py-3 text-[0.68rem] font-semibold uppercase tracking-[0.08em] text-(--orange)" />
792               {["Type", "Txn ID | Ref", "Vendor | Stall", "Device", "Card ID", "Txn Counter", "Mode",
793                 "Amount", "Time"].map((heading) => (
794                 <th key={heading} className="border-b border-(--line2) px-4 py-3 text-[0.68rem] font-
795                 semibold uppercase tracking-[0.08em] text-(--orange)">
796                   <span className="inline-flex items-center gap-2">
797                     {heading}
798                     <SortIcon />
799                   </span>
800                 </th>
801               )]}
802             </tr>
803           </thead>
804           <tbody>
805             {filteredTransactions.map((transaction, index) => {
806               const rowKey = transaction.txn_id ?? `${transaction.txn_receipt}-${index}`;
807               const isOpen = expandedTxnId === rowKey;
808               return (
809                 <Fragment key={rowKey}>
810                   <tr
811                     <td className="border-b border-(--line2) px-4 py-3">
812                       <button
813                         type="button"
814                         onClick={(event) => {
815                           event.stopPropagation();
816                           toggleTransactionDetails(transaction, rowKey);
817                         }}
818                         className="grid h-8 w-8 place-items-center rounded-md text-(--muted) hover:bg
819                         -(--surface2) hover:text-(--orange)"
820                         aria-label={isOpen ? "Hide transaction details" : "Show transaction details"}
821                       >
822                         <ChevronIcon open={isOpen} />
823                       </button>
824                     </td>
825                     <td className="border-b border-(--line2) px-4 py-3">
826                       <span className="text-[0.82rem] font-bold text-(--blue)">{normalizeTxnType(transaction.txn_type)}</span>
827                     </td>
828                     <td className="border-b border-(--line2) px-4 py-3">
829                       <div className="text-[0.76rem] font-medium text-(--muted)">{cleanText(transaction.txn_id)}</div>
830                       <div className="mt-1 text-[0.72rem] font-semibold text-(--orange)">{cleanText(transaction.txn_receipt)}</div>
831                     </td>
832                     <td className="border-b border-(--line2) px-4 py-3">
833                       <div className="text-[0.76rem] font-semibold text-(--muted)">{cleanText(transaction.vendor_name)}</div>
834                       <div className="mt-1 text-[0.76rem] font-semibold text-(--muted)">{cleanText(transaction.stall_name)}</div>
835                     </td>
836                     <td className="border-b border-(--line2) px-4 py-3 text-[0.76rem] font-medium text-(--muted)">
837                       {cleanText(transaction.txn_device_print_id)}
838                       {transaction.txn_app_version ? <span className="text-(--faint)"> | {transaction

```

```

    .txn_app_version}</span> : null}
839         </td>
840         <td className="border-b border-(--line2) px-4 py-3">
841             <div className="text-[0.9rem] font-medium text-(--muted)">{cleanText(transaction
n.txn_card_id)}</div>
842             <div className="mt-1 text-[0.72rem] font-medium text-(--muted) underline">{clea
nText(transaction.txn_card_uid)}</div>
843         </td>
844         <td className="border-b border-(--line2) px-4 py-3">
845             <div className="text-[0.76rem] font-medium text-(--muted)">{cleanText(transacti
on.txn_counter)}</div>
846             <div className="mt-1 text-[0.72rem] font-medium text-(--muted) underline">{clea
nText(transaction.txn_invoice)}</div>
847         </td>
848         <td className="border-b border-(--line2) px-4 py-3">
849             <div className="text-[0.72rem] font-medium uppercase text-(--faint)">{cleanText
(transaction.txn_mode)}</div>
850             <div className="mt-1 text-[0.74rem] font-medium text-(--muted)">{cleanText(tran
saction.txn_mode_info)}</div>
851         </td>
852         <td className="border-b border-(--line2) px-4 py-3 text-[0.8rem] font-semibold te
xt-(--muted)">
853             {formatMoney(transaction.txn_amount)}
854         </td>
855         <td className="border-b border-(--line2) px-4 py-3">
856             <span className={`text-[0.72rem] font-bold uppercase ${getTxnStatusClass(transa
ction.txn_status)}`}>
857                 {cleanText(transaction.txn_status)}
858             </span>
859             <div className="mt-1 text-[0.72rem] font-medium text-(--muted)">{formatTxnDate(
transaction.txn_at ?? transaction.txn_created_at ?? transaction.txn_updated_at)}</div>
860         </td>
861     </tr>
862     {isOpen ? <TransactionDetailRow {...getDetailProps(transaction, rowKey)} /> : null}
863 </Fragment>
864 );
865 }}
866 </tbody>
867 </table>
868 </div>
869
870 <div className="hidden divide-y divide-[#eef0f4] max-[760px]:block">
871     {filteredTransactions.map((transaction, index) => {
872         const rowKey = transaction.txn_id ?? `${transaction.txn_receipt}-${index}`;
873         const isOpen = expandedTxnId === rowKey;
874         return (
875             <article key={rowKey} className="p-3">
876                 <button
877                     type="button"
878                     onClick={() => toggleTransactionDetails(transaction, rowKey)}
879                     className="flex w-full items-start justify-between gap-3 text-left"
880                 >
881                     <div>
882                         <p className="text-[0.78rem] font-bold text-(--blue)">{normalizeTxnType(transaction
.txn_type)}</p>
883                         <p className="mt-1 text-[0.72rem] font-semibold text-(--text)">#{cleanText(transact
ion.txn_id)}</p>
884                     </div>
885                     <div className="text-right">
886                         <p className="text-[0.74rem] font-bold text-(--text)">{formatMoney(transaction.txn_
amount)}</p>
887                         <p className={`mt-1 text-[0.62rem] font-bold uppercase ${getTxnStatusClass(transact
ion.txn_status)}`}>
888                             {cleanText(transaction.txn_status)}
889                         </p>
890                     </div>
891                 </button>
892                 <div className="mt-3 grid grid-cols-2 gap-2 text-[0.68rem]">
893                     <div>
894                         <span className="block font-semibold uppercase tracking-[0.06em] text-(--faint)">Ve

```

```

    ndor</span>
895         <span className="font-semibold text-(--muted)">{cleanText(transaction.vendor_name)}
</span>
896     </div>
897     <div>
898         <span className="block font-semibold uppercase tracking-[0.06em] text-(--faint)">St
all</span>
899         <span className="font-semibold text-(--muted)">{cleanText(transaction.stall_name)}<
/s>span>
900     </div>
901     <div>
902         <span className="block font-semibold uppercase tracking-[0.06em] text-(--faint)">De
vice</span>
903         <span className="font-semibold text-(--muted)">{cleanText(transaction.txn_device_pr
int_id)}</span>
904     </div>
905     <div>
906         <span className="block font-semibold uppercase tracking-[0.06em] text-(--faint)">Ti
me</span>
907         <span className="font-semibold text-(--muted)">{formatTxnDate(transaction.txn_at ??
transaction.txn_created_at ?? transaction.txn_updated_at)}</span>
908     </div>
909 </div>
910 {isOpen ? (
911     <div className="mt-3 rounded-lg border border-(--line2) bg-(--surface2) p-3 text-[0.6
8rem]">
912         <TransactionDetailContent {...getDetailProps(transaction, rowKey)} />
913     </div>
914     ) : null}
915 </article>
916 );
917 }}
918 </div>
919 </>
920 ) : (
921     <div className="px-4 py-8 text-center text-[0.82rem] font-medium text-(--faint) max-[640px]:px-3
max-[640px]:py-6 max-[640px]:text-[0.68rem]">
922         No transactions found for this search.
923     </div>
924 )}
925 </section>
926 );
927 }
928
929 export function DownloadDumpButton({ variant = "light" }) {
930     const isDark = variant === "dark";
931     return (
932         <button
933             type="button"
934             className={`inline-flex h-10 items-center justify-center gap-3 rounded-lg px-5 text-[0.86rem] font-
semibold shadow-[0_8px_20px_rgba(15,23,42,0.06)] transition duration-200 max-[640px]:h-9 max-[640px]:gap-
2 max-[640px]:px-3 max-[640px]:text-[0.72rem]`} ${
935                 isDark
936                 ? "bg-(--text) text-(--bg) hover:bg-(--orange)"
937                 : "border border-(--line) bg-(--surface) text-(--text) hover:border-(--orange) hover:text-(--or
ange)"
938             }`
939         >
940             <DownloadIcon />
941             Download Dump
942         </button>
943     );
944 }
945
946 export default function TransactionFilters() {
947     const token = useDashboardStore((state) => state.token);
948     const eventMeta = useDashboardStore((state) => state.eventMeta);
949     const eventDetails = useDashboardStore((state) => state.eventDetails);
950     const cachedVendors = useDashboardStore((state) => state.vendorsByEventId?.[eventMeta?.eventId ?? event
Details?.id]);

```

```

951   const cachedStalls = useDashboardStore((state) => state.stallsByEventId?.[eventMeta?.eventId ?? eventDe
tails?.id]);
952   const setVendorsForEvent = useDashboardStore((state) => state.setVendorsForEvent);
953   const setStallsForEvent = useDashboardStore((state) => state.setStallsForEvent);
954   const eventId = eventMeta?.eventId ?? eventDetails?.id;
955   const eventName = eventDetails?.name ?? eventMeta?.eventName ?? eventMeta?.name ?? "Selected Event";
956   const today = useMemo(() => todayDateValue(), []);
957   const [filterEnabled, setFilterEnabled] = useState({
958     dates: false,
959     type: false,
960     status: false,
961     vendor: false,
962     stall: false,
963     cardId: false,
964     receipt: false,
965     device: false,
966     txnId: false,
967     mobile: false
968   });
969   const [range, setRange] = useState({
970     startDate: today,
971     startTime: "00:00:00",
972     endDate: today,
973     endTime: "00:00:00"
974   });
975   const [type, setType] = useState("");
976   const [status, setStatus] = useState("");
977   const [vendorId, setVendorId] = useState("");
978   const [stallId, setStallId] = useState("");
979   const [mobile, setMobile] = useState("");
980   const [cardId, setCardId] = useState("");
981   const [txnId, setTxnId] = useState("");
982   const [receipt, setReceipt] = useState("");
983   const [device, setDevice] = useState("");
984   const [vendors, setVendors] = useState(() => cachedVendors || []);
985   const [stalls, setStalls] = useState(() => cachedStalls || []);
986   const [loadingLists, setLoadingLists] = useState(false);
987   const [transactions, setTransactions] = useState([]);
988   const [filterError, setFilterError] = useState("");
989   const [isFiltering, setIsFiltering] = useState(false);
990
991   const setFilterFlag = useCallback((key, value) => {
992     setFilterEnabled((current) => ({ ...current, [key]: value }));
993   }, []);
994
995   const loadFilterLists = useCallback(async () => {
996     if (!eventId) {
997       setVendors([]);
998       setStalls([]);
999       return;
1000     }
1001
1002     if (cachedVendors?.length) setVendors(cachedVendors);
1003     if (cachedStalls?.length) setStalls(cachedStalls);
1004
1005     setLoadingLists(true);
1006     try {
1007       const [vendorList, stallList] = await Promise.all([
1008         cachedVendors?.length ? Promise.resolve(cachedVendors) : fetchVendors({ eventId, token }),
1009         cachedStalls?.length ? Promise.resolve(cachedStalls) : fetchStalls({ eventId, token })
1010       ]);
1011       const normalizedVendors = Array.isArray(vendorList) ? vendorList : [];
1012       const normalizedStalls = Array.isArray(stallList) ? stallList : [];
1013       setVendors(normalizedVendors);
1014       setStalls(normalizedStalls);
1015       setVendorsForEvent(eventId, normalizedVendors);
1016       setStallsForEvent(eventId, normalizedStalls);
1017     } catch (error) {
1018       console.error("Failed to load transaction filters", error);
1019     } finally {

```



```
1020     setLoadingLists(false);
1021   }
1022 }, [cachedStalls, cachedVendors, eventId, setStallsForEvent, setVendorsForEvent, token]);
1023
1024 useEffect(() => {
1025   loadFilterLists();
1026 }, [loadFilterLists]);
1027
1028 const vendorOptions = useMemo(
1029   () => [
1030     { label: loadingLists ? "Loading vendors..." : "Select Vendor", value: "" },
1031     ...vendors.map((vendor) => ({
1032       label: getVendorName(vendor),
1033       value: String(getVendorId(vendor))
1034     }))
1035   ],
1036   [loadingLists, vendors]
1037 );
1038
1039 const stallOptions = useMemo(
1040   () => [
1041     { label: loadingLists ? "Loading stalls..." : "Select Stall", value: "" },
1042     ...stalls.map((stall) => ({
1043       label: getStallName(stall),
1044       value: String(getStallId(stall))
1045     }))
1046   ],
1047   [loadingLists, stalls]
1048 );
1049
1050 const buildFilterPayload = useCallback(() => {
1051   const startIso = toPayloadIsoDate(range.startDate, range.startTime);
1052   const endIso = toPayloadIsoDate(range.endDate, range.endTime);
1053   const dates = filterEnabled.dates && startIso ? [startIso, startIso] : [];
1054   const dates2 = filterEnabled.dates && endIso ? [endIso, endIso] : [];
1055
1056   return {
1057     cardId: filterEnabled.cardId ? cardId.trim() : "",
1058     mobile: filterEnabled.mobile ? mobile.trim() : "",
1059     receipt: filterEnabled.receipt ? receipt.trim() : "",
1060     dates,
1061     dates2,
1062     type: filterEnabled.type ? type : "",
1063     status: filterEnabled.status ? status : "",
1064     device: filterEnabled.device ? device.trim() : "",
1065     txnId: filterEnabled.txnId ? txnId.trim() : "",
1066     settings: {
1067       event: true,
1068       details: false,
1069       cardId: filterEnabled.cardId,
1070       mobile: filterEnabled.mobile,
1071       receipt: filterEnabled.receipt,
1072       dates: filterEnabled.dates,
1073       type: filterEnabled.type,
1074       status: filterEnabled.status,
1075       stall: filterEnabled.stall,
1076       vendor: filterEnabled.vendor,
1077       device: filterEnabled.device,
1078       txnId: filterEnabled.txnId
1079     },
1080     event: toPayloadNumber(eventId),
1081     vendor: filterEnabled.vendor ? toPayloadNumber(vendorId) : "",
1082     stall: filterEnabled.stall ? toPayloadNumber(stallId) : ""
1083   };
1084 }, [cardId, device, eventId, filterEnabled, range, receipt, stallId, status, txnId, type, vendorId]);
1085
1086 const handleApplyFilters = useCallback(async () => {
1087   if (!eventId) {
1088     setFilterError("Select an event before applying transaction filters.");
1089     return;
```

```

1090     }
1091
1092     setIsFiltering(true);
1093     setFilterError("");
1094
1095     try {
1096         const data = await filterEventTransactions({ token, payload: buildFilterPayload() });
1097         const list = data?.transactions ?? data?.data?.transactions ?? data?.data ?? data?.list ?? data?.ro
ws ?? [];
1098         setTransactions(Array.isArray(list) ? list : []);
1099     } catch (error) {
1100         console.error("Failed to filter transactions", error);
1101         setTransactions([]);
1102         setFilterError("Unable to load transactions for selected filters.");
1103     } finally {
1104         setIsFiltering(false);
1105     }
1106 }, [buildFilterPayload, eventId, token]);
1107
1108 const handleUpdateTransactionStatus = useCallback(
1109     async ({ txId, status: nextStatus }) => {
1110         await updateEventTransactionStatus({
1111             token,
1112             txId,
1113             status: nextStatus,
1114             reason: `aml ${nextStatus}`
1115         });
1116
1117         setTransactions((current) =>
1118             current.map((transaction) => {
1119                 const transactionId = transaction.txn_id ?? transaction.txId ?? transaction.id;
1120                 return String(transactionId) === String(txId)
1121                     ? { ...transaction, txn_status: nextStatus }
1122                     : transaction;
1123             })
1124         );
1125     },
1126     [token]
1127 );
1128
1129 return (
1130     <
1131         <section className="rounded-xl border border-(--line) border-l-[4px] border-l-[#E04420] bg-(--surfa
ce) p-4 shadow-[0_18px_52px_rgba(15,23,42,0.09)] max-[640px]:rounded-lg max-[640px]:p-3">
1132             <div className="flex flex-col gap-3 border-b border-(--line) pb-3 lg:flex-row lg:items-center lg:
justify-between max-[640px]:gap-2 max-[640px]:pb-2.5">
1133                 <div className="flex items-center gap-2.5">
1134                     <span className="grid h-9 w-9 place-items-center rounded-lg bg-[linear-gradient(135deg,#E0442
0_0%,#A9379E_48%,#341CD6_100%)] text-[0.8rem] font-bold text-white shadow-[0_10px_22px_rgba(52,28,214,0.2
0)] max-[640px]:h-7 max-[640px]:w-7 max-[640px]:text-[0.66rem]">
1135                         01
1136                     </span>
1137                     <h2 className="text-[1.05rem] font-semibold text-(--text) max-[640px]:text-[0.95rem]">Filter<
/h2>
1138                 </div>
1139
1140                 <div className="inline-flex h-10 items-center gap-3 rounded-full border border-(--line) bg-(--s
urface) px-5 shadow-[0_8px_18px_rgba(15,23,42,0.04)] max-[640px]:h-8 max-[640px]:gap-2 max-[640px]:px-3">
1141                     <EventIcon />
1142                     <span className="text-[0.68rem] font-bold uppercase tracking-[0.08em] text-(--muted) max-[640
px]:text-[0.56rem]">
1143                         Event
1144                     </span>
1145                     <span className="max-w-[220px] truncate text-[0.8rem] font-bold uppercase text-(--text) max-[
640px]:max-w-[150px] max-[640px]:text-[0.66rem]">
1146                         {eventName}
1147                     </span>
1148                 </div>
1149             </div>
1150

```

```

1151     <div className="pt-4 max-[640px]:pt-3">
1152       <DateRangeField
1153         enabled={filterEnabled.dates}
1154         onEnabledChange={(value) => setFilterFlag("dates", value)}
1155         range={range}
1156         onChange={setRange}
1157       />
1158     <div className="mt-3 grid gap-3 lg:grid-cols-2 max-[640px]:gap-2">
1159       <SelectField
1160         label="Type"
1161         value={type}
1162         onChange={setType}
1163         enabled={filterEnabled.type}
1164         onEnabledChange={(value) => setFilterFlag("type", value)}
1165         options={[{ label: "Select Type", value: "" }, ...TRANSACTION_TYPES]}
1166       />
1167       <SelectField
1168         label="Status"
1169         value={status}
1170         onChange={setStatus}
1171         enabled={filterEnabled.status}
1172         onEnabledChange={(value) => setFilterFlag("status", value)}
1173         options={STATUS_OPTIONS}
1174       />
1175       <SelectField
1176         label="Vendors"
1177         value={vendorId}
1178         onChange={setVendorId}
1179         enabled={filterEnabled.vendor}
1180         onEnabledChange={(value) => setFilterFlag("vendor", value)}
1181         options={vendorOptions}
1182         disabled={!eventId}
1183       />
1184       <SelectField
1185         label="Stalls"
1186         value={stallId}
1187         onChange={setStallId}
1188         enabled={filterEnabled.stall}
1189         onEnabledChange={(value) => setFilterFlag("stall", value)}
1190         options={stallOptions}
1191         disabled={!eventId}
1192       />
1193       <InputField label="Mobile" value={mobile} onChange={setMobile} placeholder="Mobile" enabled={
filterEnabled.mobile} onEnabledChange={(value) => setFilterFlag("mobile", value)} />
1194       <InputField label="Card ID" value={cardId} onChange={setCardId} placeholder="Card" enabled={f
ilterEnabled.cardId} onEnabledChange={(value) => setFilterFlag("cardId", value)} />
1195       <InputField label="TXN-ID" value={txnId} onChange={setTxnId} placeholder="TXN-ID" enabled={fi
lterEnabled.txnId} onEnabledChange={(value) => setFilterFlag("txnId", value)} />
1196       <InputField label="Receipt" value={receipt} onChange={setReceipt} placeholder="Receipt" enabl
ed={filterEnabled.receipt} onEnabledChange={(value) => setFilterFlag("receipt", value)} />
1197       <InputField label="Device" value={device} onChange={setDevice} placeholder="Device" enabled={
filterEnabled.device} onEnabledChange={(value) => setFilterFlag("device", value)} />
1198       <button
1199         type="button"
1200         onClick={handleApplyFilters}
1201         disabled={isFiltering}
1202         className="flex h-12 items-center justify-center gap-3 rounded-lg border border-(--orange)
bg-[rgba(224,68,32,0.06)] px-5 text-[0.86rem] font-bold text-(--orange) shadow-[0_10px_22px_rgba(224,68,3
2,0.10)] transition duration-200 hover:bg-(--orange) hover:text-white disabled:cursor-not-allowed disable
d:opacity-60 max-[640px]:h-9 max-[640px]:gap-2 max-[640px]:px-3 max-[640px]:text-[0.72rem]"
1203     >
1204         <SearchIcon />
1205         {isFiltering ? "Applying..." : "Apply Filters"}
1206       </button>
1207     </div>
1208     {filterError ? (
1209       <p className="mt-3 rounded-lg border border-[rgba(224,68,32,0.25)] bg-[rgba(224,68,32,0.06)]
px-3 py-2 text-[0.72rem] font-semibold text-(--orange)">
1210         {filterError}
1211       </p>

```

```
1212         ) : null}
1213     </div>
1214 </section>
1215
1216     <TransactionResults
1217         transactions={transactions}
1218         token={token}
1219         onUpdateStatus={handleUpdateTransactionStatus}
1220     />
1221 </>
1222 );
1223 }
```

FILE: apps/dashboard/src/app/transactions/page.js

```

1 import Header from "../components/Header";
2 import TransactionFilters, { DownloadDumpButton } from "../components/TransactionFilters";
3
4 export default function TransactionsPage() {
5   return (
6     <main
7       className="min-h-screen bg-(--bg) pb-10"
8       style={{ fontFamily: '"Poppins", system-ui, sans-serif' }}
9     >
10      <Header areaLabel="Transactions" variant="portal" />
11
12      <div className="mx-auto w-full max-w-[1780px] pr-4 pl-[72px] pt-6 md:pr-7 md:pl-[88px] max-[900px]:
13 px-3">
14        <div className="flex flex-col gap-4 sm:flex-row sm:items-start sm:justify-between">
15          <div>
16            <h1 className="font-chillax text-[clamp(24px,3vw,32px)] font-semibold leading-[1.05] tracking
17 -[-0.02em] text-(--text)">
18              Transactions
19            </h1>
20            <p className="mt-2 text-[13.5px] font-light text-(--muted)">
21              Filter event transactions by date, type, vendor, stall, card, receipt, or device.
22            </p>
23          </div>
24
25          <DownloadDumpButton />
26
27          <div className="mt-[clamp(16px,2vw,22px)] h-px w-full bg-(--line)" />
28
29          <div className="mt-[clamp(16px,2vw,22px)]">
30            <TransactionFilters />
31          </div>
32        </div>
33      </main>
34    );
35  }

```

FILE: apps/dashboard/src/app/whitelist/components/WhitelistContent.js

```
1  "use client";
2
3  import { useState } from "react";
4  import { useDashboardStore } from "../../store/dashboardStore";
5  import {
6    fetchAccessXWhitelistLogs,
7    searchAccessXWhitelist
8  } from "../../lib/dashboardApi";
9
10 function SearchIcon({ className = "h-4 w-4" }) {
11   return (
12     <svg
13       viewBox="0 0 24 24"
14       className={className}
15       fill="none"
16       stroke="currentColor"
17       strokeWidth="2"
18       strokeLinecap="round"
19       aria-hidden="true"
20     >
21       <circle cx="10.5" cy="10.5" r="6.5" />
22       <path d="m16 16 4.5 4.5" />
23     </svg>
24   );
25 }
26
27 function HistoryIcon() {
28   return (
29     <svg
30       viewBox="0 0 24 24"
31       className="h-4 w-4"
32       fill="none"
33       stroke="currentColor"
34       strokeWidth="2"
35       strokeLinecap="round"
36       strokeLinejoin="round"
37       aria-hidden="true"
38     >
39       <path d="M3 12a9 9 0 1 0 3-6.7L3 8" />
40       <path d="M3 3v5h5" />
41       <path d="M12 7v5l3 2" />
42     </svg>
43   );
44 }
45
46 function CategoryIcon() {
47   return (
48     <svg
49       viewBox="0 0 24 24"
50       className="h-4 w-4"
51       fill="none"
52       stroke="currentColor"
53       strokeWidth="2"
54       strokeLinecap="round"
55       strokeLinejoin="round"
56       aria-hidden="true"
57     >
58       <path d="M20 13 11 22l-9-9V2h11l7 7Z" />
59       <circle cx="7.5" cy="7.5" r="1.5" />
60     </svg>
61   );
62 }
63
64 function cleanValue(value, fallback = "-") {
65   if (value === "" || value == null) return fallback;
66   return String(value);
67 }
68
69 export default function WhitelistContent() {
```

```

70  const token = useDashboardStore((state) => state.token);
71  const eventMeta = useDashboardStore((state) => state.eventMeta);
72  const eventDetails = useDashboardStore((state) => state.eventDetails);
73  const eventId = eventMeta?.eventId ?? eventDetails?.id ?? null;
74  const eventName =
75    eventDetails?.name ??
76    eventMeta?.eventName ??
77    eventMeta?.name ??
78    "Selected Event";
79  const [query, setQuery] = useState("");
80  const [submittedQuery, setSubmittedQuery] = useState("");
81  const [results, setResults] = useState([]);
82  const [searching, setSearching] = useState(false);
83  const [searchError, setSearchError] = useState("");
84  const hasSearched = submittedQuery.trim().length > 0;
85
86  const runSearch = async (value = query) => {
87    const normalized = value.trim();
88    setQuery(value);
89    setSubmittedQuery(normalized);
90    setSearchError("");
91
92    if (!normalized) {
93      setResults([]);
94      return;
95    }
96    if (eventId === "" || eventId == null) {
97      setSearchError("Event ID is unavailable.");
98      return;
99    }
100
101    setSearching(true);
102    try {
103      const response = await searchAccessXWhitelist({
104        eventId,
105        search: normalized,
106        token
107      });
108      const users = response?.users ?? response?.data?.users ?? [];
109      setResults(Array.isArray(users) ? users : []);
110    } catch (error) {
111      console.error("Failed to search AccessX whitelist", error);
112      setResults([]);
113      setSearchError("Unable to search the whitelist.");
114    } finally {
115      setSearching(false);
116    }
117  };
118
119  const loadHistory = async (whitelistUserId) => {
120    try {
121      await fetchAccessXWhitelistLogs({
122        eventId,
123        wid: whitelistUserId,
124        token
125      });
126    } catch (error) {
127      console.error("Failed to load AccessX whitelist history", error);
128    }
129  };
130
131  return (
132    <div className="ml-[60px] w-[calc(100vw-60px)] min-w-0 overflow-x-hidden max-[900px]:ml-0 max-[900px]:w-screen">
133      <section className="relative overflow-hidden bg-[#241a57] px-6 pb-[72px] pt-8 md:px-8 md:pb-[82px] md:pt-9">
134        <div
135          className="pointer-events-none absolute inset-0 opacity-100"
136          style={{
137            background:

```

```

138         "linear-gradient(115deg, #1c1c1c 0%, #341cd6 64%, #e04420 145%)"
139     }}
140     aria-hidden="true"
141 />
142 <div
143     className="pointer-events-none absolute inset-0 opacity-[0.12]"
144     style={{
145         backgroundImage:
146             "repeating-linear-gradient(90deg, #ffffff 0 1px, transparent 1px 46px)"
147     }}
148     aria-hidden="true"
149 />
150
151     <div className="relative mx-auto flex w-full max-w-[1400px] items-end justify-between gap-5">
152         <div>
153             <p className="whitelist-mono text-[0.64rem] uppercase tracking-[0.2em] text-white/65">
154                 Access Control
155             </p>
156             <h1 className="whitelist-display mt-2 text-[2.55rem] font-semibold leading-none text-white ma
x-[640px]:text-[2rem]">
157                 Whitelist
158             </h1>
159             <p className="mt-3 text-[0.82rem] font-light text-white/70 max-[640px]:max-w-[280px] max-[640
px]:text-[0.72rem]">
160                 Look up a guest and grant or revoke event access.
161             </p>
162         </div>
163         <span className="whitelist-display text-[4rem] font-bold leading-none text-white/15 max-[640px]
:text-[2.5rem]">
164             {eventId ?? "-"}
165         </span>
166     </div>
167 </section>
168
169     <div className="relative mx-auto w-full max-w-[1448px] px-5 pb-14 md:px-7 max-[640px]:w-[calc(100vw
-24px)] max-[640px]:max-w-[calc(100vw-24px)] max-[640px]:px-0">
170         <section className="-mt-[48px] w-full max-w-[720px] rounded-lg border border-black/10 bg-white p-
5 shadow-[0_20px_44px_-24px_rgba(28,28,28,0.45)] max-[640px]:-mt-[42px] max-[640px]:p-4">
171             <form
172                 onSubmit={(event) => {
173                     event.preventDefault();
174                     runSearch();
175                 }}
176             >
177                 <div className="flex h-12 items-center gap-3 rounded-lg border border-[#dedddb] bg-[#f8f7f5]
px-4 text-[#8c8b88] transition focus-within:border-[#b8aef7] focus-within:bg-white">
178                     <SearchIcon />
179                     <input
180                         value={query}
181                         onChange={(event) => setQuery(event.target.value)}
182                         placeholder="Search"
183                         className="h-full min-w-0 flex-1 border-0 bg-transparent text-[0.9rem] text-[#1c1c1c] out
line-none placeholder:text-[#8f8e8b]"
184                     />
185                     {query ? (
186                         <button
187                             type="button"
188                             onClick={() => {
189                                 setQuery("");
190                                 setSubmittedQuery("");
191                                 setResults([]);
192                                 setSearchError("");
193                             }}
194                         className="grid h-7 w-7 place-items-center rounded-md text-[#969592] transition hover:b
g-black/5 hover:text-[#1c1c1c]"
195                         aria-label="Clear whitelist search"
196                     >
197                         <span className="text-lg leading-none">x</span>
198                     </button>
199                 ) : null}

```



```

200         </div>
201
202         <p className="whitelist-mono mt-2.5 text-[0.58rem] uppercase tracking-[0.13em] text-[#777672]
">
203             Search by name / booking ID / AWB / card ID
204         </p>
205
206         <div className="mt-4 flex flex-wrap items-center gap-2.5">
207             <button
208                 type="submit"
209                 disabled={searching}
210                 className="h-10 rounded-md bg-[#1c1c1c] px-6 text-[0.78rem] font-semibold text-white tran
sition hover:bg-[#e04420] disabled:cursor-wait disabled:opacity-60"
211             >
212                 {searching ? "Searching..." : "Search"}
213             </button>
214             {[ "VIP", "CREW", "PENDING" ].map((filter) => (
215                 <button
216                     key={filter}
217                     type="button"
218                     onClick={() => runSearch(filter)}
219                     className="whitelist-mono h-9 rounded-md border border-[#dedddb] bg-white px-4 text-[0.
58rem] uppercase tracking-[0.11em] text-[#777672] transition hover:border-[#e04420] hover:text-[#e04420]"
220                 >
221                     {filter}
222                 </button>
223             )]}
224         </div>
225
226         {searchError ? (
227             <p className="mt-3 text-[0.68rem] font-medium text-[#e04420]" role="alert">
228                 {searchError}
229             </p>
230         ) : null}
231     </form>
232 </section>
233
234     <section className="mt-5 overflow-hidden rounded-lg border border-black/10 bg-white shadow-[0_12p
x_32px_-24px_rgba(28,28,28,0.4)]">
235         <header className="flex min-h-12 items-center gap-3 border-b border-black/[0.06] px-4 py-3">
236             <span className="whitelist-mono text-[0.66rem] uppercase tracking-[0.14em] text-[#71706e]">
237                 Users ({results.length})
238             </span>
239             <span className="ml-auto max-w-[50%] truncate text-[0.7rem] font-semibold text-[#8b8a87] max-
[640px]:hidden">
240                 {eventName}
241             </span>
242         </header>
243
244         {results.length ? (
245             <div className="grid grid-cols-1 gap-2.5 p-3 md:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4 max
-[640px]:p-2.5">
246                 {results.map((user, index) => {
247                     const name = cleanValue(user.name, "Unnamed guest");
248                     const bookingReference = cleanValue(user.bookingReference);
249                     const resultEventName = cleanValue(user.eventName, eventName);
250                     const pax = cleanValue(user.pax, "0");
251                     return (
252                         <article
253                             key={user.id ?? `${bookingReference}-${index}`}
254                             className="overflow-hidden rounded-lg border border-[#e2e0dc] bg-[#fbfaf8] transition
duration-200 hover:-translate-y-0.5 hover:border-[#cfc9f8] hover:shadow-[0_14px_28px_-22px_rgba(28,28,28
,0.55)]"
255                         >
256                             <div className="flex items-start gap-2.5 border-b border-black/[0.06] p-2.5">
257                                 <div
258                                     className={`whitelist-display grid h-9 w-9 shrink-0 place-items-center rounded-md
text-[0.78rem] font-semibold text-white ${
259                                     index % 2 ? "bg-[#341cd6]" : "bg-[#e04420]"
260                                 }`}

```

```

261         >
262         {name.charAt(0).toUpperCase()}
263     </div>
264     <div className="min-w-0 flex-1">
265         <h2 className="truncate text-[0.78rem] font-semibold text-[#1c1c1c]">{name}</h2>
266         <p className="mt-0.5 truncate text-[0.61rem] text-[#777672]">{resultEventName}</p>
267     >
268     </div>
269     <span className="whitelist-mono shrink-0 rounded-md bg-[#eeeafd] px-1.5 py-1 text-[
0.46rem] uppercase tracking-[0.08em] text-[#341cd6]">
270         Total pax {pax}
271     </span>
272 </div>
273     <div className="grid grid-cols-2 gap-x-3 p-2.5">
274         <div className={user.courierBarcode ? "" : "col-span-2"}>
275             <p className="whitelist-mono text-[0.44rem] uppercase tracking-[0.12em] text-[#9b
9a97]">Booking reference</p>
276             <p className="mt-0.5 truncate text-[0.65rem] font-semibold text-[#353535]">{booki
ngReference}</p>
277         </div>
278         {user.courierBarcode ? (
279             <div>
280                 <p className="whitelist-mono text-[0.44rem] uppercase tracking-[0.12em] text-[#
9b9a97]">Courier barcode</p>
281                 <p className="mt-0.5 truncate text-[0.65rem] font-semibold text-[#353535]">{use
r.courierBarcode}</p>
282             </div>
283         ) : null}
284     </div>
285     <div className="flex gap-1.5 border-t border-black/[0.06] p-2">
286         <button
287             type="button"
288             onClick={() => loadHistory(user.id)}
289             className="flex h-7 flex-1 items-center justify-center gap-1.5 rounded-md border
border-[#dedddb] bg-white px-2 text-[0.6rem] font-semibold text-[#555451] transition hover:border-[#341cd
6] hover:text-[#341cd6]"
290         >
291             <HistoryIcon />
292             History
293         </button>
294         <button
295             type="button"
296             className="flex h-7 flex-1 items-center justify-center gap-1.5 rounded-md bg-[#1c
1c1c] px-2 text-[0.6rem] font-semibold text-white transition hover:bg-[#e04420]"
297         >
298             <CategoryIcon />
299             Category
300         </button>
301     </div>
302 </article>
303 </div>
304 );
305 }}
306 </div>
307 ) : (
308     <div className="flex min-h-[340px] flex-col items-center justify-center gap-3 px-5 py-10 text
-center">
309         <div className="grid h-14 w-14 place-items-center rounded-lg border border-dashed border-[#
d8d7d4] text-[#9b9a97]">
310             <SearchIcon className="h-6 w-6" />
311         </div>
312         <h2 className="whitelist-display text-[1rem] font-semibold text-[#1c1c1c]">
313             {hasSearched ? "No matching guest" : "Nothing searched yet"}
314         </h2>
315         <p className="max-w-[350px] text-[0.72rem] leading-5 text-[#9b9a97]">
316             {hasSearched
317                 ? "Check the spelling or try a booking ID instead."
318                 : "Search a name, booking ID, AWB or card ID. Matching guests will appear here."}
319         </p>

```

```
320         </div>
321     })
322     </section>
323 </div>
324 </div>
325 );
326 }
```

FILE: apps/dashboard/src/app/whitelist/page.js

```

1 import Header from "../../components/Header";
2 import WhitelistContent from "../../components/WhitelistContent";
3
4 export default function WhitelistPage() {
5   return (
6     <main
7       className="whitelist-page-font min-h-screen w-full overflow-x-hidden bg-[#f2f1ee] pb-12"
8       style={{ fontFamily: '"AtomX Whitelist Poppins", Poppins, sans-serif' }}
9     >
10    <style>{`
11      @font-face {
12        font-family: "AtomX Whitelist Poppins";
13        src: url("/shared/fonts/Poppins/Poppins-Light.ttf") format("truetype");
14        font-weight: 300;
15        font-style: normal;
16        font-display: swap;
17      }
18      @font-face {
19        font-family: "AtomX Whitelist Poppins";
20        src: url("/shared/fonts/Poppins/Poppins-Regular.ttf") format("truetype");
21        font-weight: 400;
22        font-style: normal;
23        font-display: swap;
24      }
25      @font-face {
26        font-family: "AtomX Whitelist Poppins";
27        src: url("/shared/fonts/Poppins/Poppins-SemiBold.ttf") format("truetype");
28        font-weight: 600;
29        font-style: normal;
30        font-display: swap;
31      }
32      @font-face {
33        font-family: "AtomX Whitelist Poppins";
34        src: url("/shared/fonts/Poppins/Poppins-Bold.ttf") format("truetype");
35        font-weight: 700;
36        font-style: normal;
37        font-display: swap;
38      }
39      @font-face {
40        font-family: "AtomX Whitelist Chillax";
41        src: url("/shared/fonts/Chillex/Chillax-Semibold.woff2") format("woff2");
42        font-weight: 600;
43        font-style: normal;
44        font-display: swap;
45      }
46      @font-face {
47        font-family: "AtomX Whitelist VCR";
48        src: url("/shared/fonts/VCR/VCR_OSD_MONO_1.001.ttf") format("truetype");
49        font-weight: 400;
50        font-style: normal;
51        font-display: swap;
52      }
53      .whitelist-page-font,
54      .whitelist-page-font * {
55        font-family: "AtomX Whitelist Poppins", Poppins, sans-serif;
56      }
57      .whitelist-page-font .whitelist-display {
58        font-family: "AtomX Whitelist Chillax", "AtomX Whitelist Poppins", sans-serif;
59      }
60      .whitelist-page-font .whitelist-mono {
61        font-family: "AtomX Whitelist VCR", monospace;
62      }
63    `}</style>
64
65    <Header areaLabel="Whitelist" variant="portal" />
66    <WhitelistContent />
67  </main>
68  );
69 }

```

FILE: apps/dashboard/src/lib/apiClient.js

```

1 import { DASHBOARD_API_KEY } from "./apiConfig";
2
3 // -----
4 // TEMPORARY TEST SWITCH - cookie-only session
5 //
6 // false = the frontend does NOT send Authorization: Bearer at all. Every request
7 //         still goes out with `credentials: "include"`, so the ONLY credential
8 //         reaching the API is the session cookie the browser stores after
9 //         /auth/select on the access page and re-attaches automatically.
10 //
11 // Set back to true to restore the previous Bearer + cookie behavior.
12 // Note: the token is still read from the URL and stored, because the header,
13 // profile menu and admin role linking read name/email/role/adminId out of it.
14 // This switch only controls whether the token is SENT.
15 // -----
16 const SEND_BEARER_TOKEN = false;
17
18 const STATUS_MESSAGES = {
19   400: "The request could not be processed.",
20   401: "Your session has expired. Please sign in again.",
21   403: "You do not have permission to perform this action.",
22   404: "The requested resource was not found.",
23   409: "The request conflicts with the current data.",
24   422: "Please check the submitted information.",
25   429: "Too many requests. Please wait and try again."
26 };
27
28 export class ApiError extends Error {
29   constructor(
30     message,
31     {
32       status = 0,
33       code = null,
34       details = null,
35       serverMessage = "",
36       url = "",
37       method = "GET",
38       cause = null
39     } = {}
40   ) {
41     super(message);
42     this.name = "ApiError";
43     this.status = status;
44     this.code = code;
45     this.details = details;
46     this.serverMessage = serverMessage;
47     this.url = url;
48     this.method = method;
49
50     if (cause) {
51       this.cause = cause;
52     }
53   }
54 }
55
56 function isJsonBody(body) {
57   if (body == null || typeof body !== "object") {
58     return false;
59   }
60
61   return !(
62     body instanceof FormData ||
63     body instanceof URLSearchParams ||
64     body instanceof Blob ||
65     body instanceof ArrayBuffer
66   );
67 }
68
69 function buildHeaders({ token, headers, jsonBody }) {

```

```
70   const requestHeaders = new Headers(headers || {});
71
72   if (!requestHeaders.has("Accept")) {
73     requestHeaders.set("Accept", "application/json");
74   }
75   if (jsonBody && !requestHeaders.has("Content-Type")) {
76     requestHeaders.set("Content-Type", "application/json");
77   }
78   if (DASHBOARD_API_KEY && !requestHeaders.has("x-api-key")) {
79     requestHeaders.set("x-api-key", DASHBOARD_API_KEY);
80   }
81   if (SEND_BEARER_TOKEN && token && !requestHeaders.has("Authorization")) {
82     requestHeaders.set("Authorization", `Bearer ${token}`);
83   }
84
85   return requestHeaders;
86 }
87
88 async function parseResponseBody(response) {
89   if (response.status === 204 || response.status === 205) {
90     return null;
91   }
92
93   const text = await response.text();
94   if (!text) {
95     return null;
96   }
97
98   try {
99     return JSON.parse(text);
100  } catch {
101    return text;
102  }
103 }
104
105 function extractServerMessage(payload) {
106   if (typeof payload === "string") {
107     return payload.trim();
108   }
109
110   if (!payload || typeof payload !== "object") {
111     return "";
112   }
113
114   const message =
115     payload.message ??
116     payload.error?.message ??
117     payload.error ??
118     payload.msg ??
119     payload.detail ??
120     payload.title;
121
122   return typeof message === "string" ? message.trim() : "";
123 }
124
125 function getStatusMessage(status, serverMessage) {
126   if (status >= 500) {
127     return "The server could not complete the request. Please try again.";
128   }
129
130   return serverMessage || STATUS_MESSAGES[status] || `Request failed (${status}).`;
131 }
132
133 function getErrorCode(payload) {
134   if (!payload || typeof payload !== "object") {
135     return null;
136   }
137
138   return payload.code ?? payload.error?.code ?? null;
139 }
```

```
140
141 function serializeBody(body) {
142   return isJsonBody(body) ? JSON.stringify(body) : body;
143 }
144
145 async function sendRequest({
146   url,
147   method,
148   token,
149   body,
150   headers,
151   credentials,
152   cache,
153   signal
154 }) {
155   const jsonBody = isJsonBody(body);
156
157   try {
158     return await fetch(url, {
159       method,
160       headers: buildHeaders({ token, headers, jsonBody }),
161       credentials,
162       ...(cache ? { cache } : {}),
163       ...(signal ? { signal } : {}),
164       ...(body == null ? {} : { body: serializeBody(body) })
165     });
166   } catch (error) {
167     if (error?.name === "AbortError") {
168       throw error;
169     }
170
171     throw new ApiError("Unable to connect to the server. Please try again.", {
172       status: 0,
173       url,
174       method,
175       cause: error
176     });
177   }
178 }
179
180 export async function apiRequest({
181   url,
182   method = "GET",
183   token,
184   body,
185   headers,
186   credentials = "include",
187   cache,
188   signal,
189   retryWithoutToken = false
190 }) {
191   const normalizedMethod = String(method || "GET").toUpperCase();
192   const request = (requestToken) =>
193     sendRequest({
194       url,
195       method: normalizedMethod,
196       token: requestToken,
197       body,
198       headers,
199       credentials,
200       cache: cache ?? (normalizedMethod === "GET" ? "no-store" : undefined),
201       signal
202     });
203
204   let response = await request(token);
205
206   // With SEND_BEARER_TOKEN = false no Authorization header was sent in the first
207   // place, so this cookie-only retry never triggers – the first attempt already
208   // was the cookie-only attempt.
209   if (
```

```
210     SEND_BEARER_TOKEN &&
211     retryWithoutToken &&
212     token &&
213     (response.status === 401 || response.status === 403)
214 ) {
215     response = await request(null);
216 }
217
218 const payload = await parseResponseBody(response);
219
220 if (!response.ok) {
221     const serverMessage = extractServerMessage(payload);
222     throw new ApiError(getStatusMessage(response.status, serverMessage), {
223         status: response.status,
224         code: getErrorCode(payload),
225         details: payload,
226         serverMessage,
227         url,
228         method: normalizedMethod
229     });
230 }
231
232 return payload;
233 }
234
235 export function getApiErrorMessage(error, fallback = "Something went wrong. Please try again.") {
236     if (error instanceof ApiError || error instanceof Error) {
237         return error.message || fallback;
238     }
239
240     return fallback;
241 }
```


FILE: apps/dashboard/src/lib/dashboardApi.js

```
1 import { getBaseUrl } from "@atomx/lib";
2 import { apiRequest } from "../apiClient";
3
4 const inFlightGetRequests = new Map();
5
6 function buildGetRequestKey(url, token) {
7   return `${url}::${token || "cookie-session"}`;
8 }
9
10 async function fetchGetJson({ url, token }) {
11   return apiRequest({
12     url,
13     method: "GET",
14     token,
15     retryWithoutToken: true
16   });
17 }
18
19 async function fetchGetJsonDeduped({ url, token, dedupe = true }) {
20   if (!dedupe) {
21     return fetchGetJson({ url, token });
22   }
23
24   const key = buildGetRequestKey(url, token);
25   const existing = inFlightGetRequests.get(key);
26   if (existing) {
27     return existing;
28   }
29
30   const request = fetchGetJson({ url, token }).finally(() => {
31     inFlightGetRequests.delete(key);
32   });
33
34   inFlightGetRequests.set(key, request);
35   return request;
36 }
37
38 export async function linkRole({ token, payload }) {
39   const baseUrl = getBaseUrl();
40   return apiRequest({
41     url: `${baseUrl}/v1/Operators/Link`,
42     method: "POST",
43     token,
44     body: payload
45   });
46 }
47
48 export async function linkAdmin({ email, adminId, token }) {
49   return linkRole({
50     token,
51     payload: {
52       email,
53       adminId,
54       type: "admin"
55     }
56   });
57 }
58
59 export async function linkOperator({ email, eventId, type, token }) {
60   return linkRole({
61     token,
62     payload: {
63       email,
64       eventId,
65       type
66     }
67   });
68 }
69
```

```
70 export async function fetchOperatorsList({ token, dedupe = true }) {
71   const baseUrl = getBaseUrl();
72   const data = await fetchGetJsonDeduped({
73     url: `${baseUrl}/v1/Operators/List`,
74     token,
75     dedupe
76   });
77
78   const candidates = [
79     data?.operatorLinks,
80     data?.operators,
81     data?.data?.operatorLinks,
82     data?.data?.operators,
83     data?.data?.rows,
84     data?.result?.operatorLinks,
85     data?.result?.operators,
86     data?.result,
87     data?.rows,
88     data?.list,
89     data?.data
90   ];
91
92   return candidates.find(Array.isArray) ?? [];
93 }
94
95 export async function fetchReportsList({ eventId, token, dedupe = true }) {
96   if (eventId === "" || eventId == null) {
97     throw new Error("Missing eventId");
98   }
99
100   const baseUrl = getBaseUrl();
101   return fetchGetJsonDeduped({
102     url: `${baseUrl}/v1/Reports/List?eventId=${encodeURIComponent(eventId)}`,
103     token,
104     dedupe
105   });
106 }
107
108 export async function startReportBuild({
109   eventId,
110   token,
111   dates,
112   days = [0],
113   idType,
114   id,
115   type,
116   requestId
117 }) {
118   if (eventId === "" || eventId == null) {
119     throw new Error("Missing eventId");
120   }
121
122   if (!Array.isArray(dates) || dates.length !== 2 || dates.some((date) => !date)) {
123     throw new Error("Select a start and end date");
124   }
125
126   // `id` is the report target: the event id for an event report, or the
127   // selected vendor's id for a vendor report. Defaults to the event id.
128   const targetId = id === "" || id == null ? eventId : id;
129   const numericTargetId = Number(targetId);
130   const baseUrl = getBaseUrl();
131   return apiRequest({
132     url: `${baseUrl}/v1/Reports/Build/Start`,
133     method: "POST",
134     token,
135     body: {
136       dates,
137       days,
138       idType,
139       id: Number.isNaN(numericTargetId) ? targetId : numericTargetId,
```

```
140     type,
141     requestId
142   }
143 });
144 }
145
146 export async function fetchEventDetails({ eventId, token, dedupe = true }) {
147   if (!eventId) {
148     throw new Error("Missing eventId");
149   }
150   const baseUrl = getBaseUrl();
151   const data = await fetchGetJsonDeduped({
152     url: `${baseUrl}/v1/Events/Details/${encodeURIComponent(eventId)}`,
153     token,
154     dedupe
155   });
156   return data?.event ?? data?.data?.event ?? null;
157 }
158
159 export async function fetchEventsList({ token, dedupe = true }) {
160   const baseUrl = getBaseUrl();
161   const data = await fetchGetJsonDeduped({
162     url: `${baseUrl}/v1/Events/List`,
163     token,
164     dedupe
165   });
166   return data?.events ?? data?.data?.events ?? data?.data ?? data?.list ?? [];
167 }
168
169 export async function updateEventDetails({ eventId, token, payload }) {
170   if (!eventId) {
171     throw new Error("Missing eventId");
172   }
173   const baseUrl = getBaseUrl();
174   return apiRequest({
175     url: `${baseUrl}/v1/Events/Edit/${encodeURIComponent(eventId)}`,
176     method: "PATCH",
177     token,
178     body: payload
179   });
180 }
181
182 export async function createVendor({ token, vendor }) {
183   const baseUrl = getBaseUrl();
184   return apiRequest({
185     url: `${baseUrl}/v1/Vendors/Create`,
186     method: "POST",
187     token,
188     body: { vendor }
189   });
190 }
191
192 export async function fetchVendors({ eventId, token, dedupe = true }) {
193   if (!eventId) {
194     throw new Error("Missing eventId");
195   }
196   const baseUrl = getBaseUrl();
197   const data = await fetchGetJsonDeduped({
198     url: `${baseUrl}/v1/Vendors/List/${encodeURIComponent(eventId)}`,
199     token,
200     dedupe
201   });
202   return data?.vendors ?? data?.data?.vendors ?? data?.data ?? data?.list ?? [];
203 }
204
205 export async function updateVendor({ vendorId, token, payload }) {
206   if (!vendorId) {
207     throw new Error("Missing vendorId");
208   }
209   const baseUrl = getBaseUrl();
```

```
210   return apiRequest({
211     url: `${baseUrl}/v1/Vendors/Edit/${encodeURIComponent(vendorId)}`,
212     method: "PATCH",
213     token,
214     body: payload
215   });
216 }
217
218 export async function createStall({ token, stall }) {
219   const baseUrl = getBaseUrl();
220   return apiRequest({
221     url: `${baseUrl}/v1/Stalls/Create`,
222     method: "POST",
223     token,
224     body: { stall }
225   });
226 }
227
228 export async function fetchStalls({ eventId, token, dedupe = true }) {
229   if (!eventId) {
230     throw new Error("Missing eventId");
231   }
232   const baseUrl = getBaseUrl();
233   const data = await fetchGetJsonDeduped({
234     url: `${baseUrl}/v1/Stalls/List/Eventwise/${encodeURIComponent(eventId)}`,
235     token,
236     dedupe
237   });
238   return data?.stalls ?? data?.data?.stalls ?? data?.data ?? data?.list ?? [];
239 }
240
241 export async function fetchStallItems({ stallId, token, dedupe = true }) {
242   if (!stallId) {
243     throw new Error("Missing stallId");
244   }
245
246   const baseUrl = getBaseUrl();
247   return fetchGetJsonDeduped({
248     url: `${baseUrl}/v1/Items/List/${encodeURIComponent(stallId)}`,
249     token,
250     dedupe
251   });
252 }
253
254 async function fetchAccessXList({ path, eventId, token, dedupe = true }) {
255   if (!eventId) {
256     throw new Error("Missing eventId");
257   }
258
259   const baseUrl = getBaseUrl();
260   return fetchGetJsonDeduped({
261     url: `${baseUrl}${path}?code=${encodeURIComponent(eventId)}`,
262     token,
263     dedupe
264   });
265 }
266
267 export function fetchAccessXCategories({ eventId, token, dedupe = true }) {
268   return fetchAccessXList({
269     path: "/v1/AccessX/Categories/List",
270     eventId,
271     token,
272     dedupe
273   });
274 }
275
276 export function fetchAccessXGateMasters({ eventId, token, dedupe = true }) {
277   return fetchAccessXList({
278     path: "/v1/AccessX/GatesMaster/List",
279     eventId,
```

```
280     token,
281     dedupe
282   });
283 }
284
285 export async function createAccessXGateMaster({ token, gateMaster }) {
286   if (!gateMaster?.eventId) {
287     throw new Error("Missing eventId");
288   }
289   if (!String(gateMaster?.name || "").trim()) {
290     throw new Error("Missing gate master name");
291   }
292
293   const baseUrl = getBaseUrl();
294   return apiRequest({
295     url: `${baseUrl}/v1/AccessX/GatesMaster/Create`,
296     method: "POST",
297     token,
298     body: gateMaster
299   });
300 }
301
302 export async function updateAccessXGateMaster({ token, gateMaster }) {
303   if (!gateMaster?.id) {
304     throw new Error("Missing gate master id");
305   }
306   if (!gateMaster?.eventId) {
307     throw new Error("Missing eventId");
308   }
309
310   const baseUrl = getBaseUrl();
311   return apiRequest({
312     url: `${baseUrl}/v1/AccessX/GatesMaster/Edit`,
313     method: "POST",
314     token,
315     body: gateMaster
316   });
317 }
318
319 export function fetchAccessXGates({ eventId, token, dedupe = true }) {
320   return fetchAccessXList({
321     path: "/v1/AccessX/Gates/List",
322     eventId,
323     token,
324     dedupe
325   });
326 }
327
328 export async function searchAccessXWhitelist({ eventId, search, token }) {
329   if (eventId === "" || eventId == null) {
330     throw new Error("Missing eventId");
331   }
332
333   const normalizedSearch = String(search || "").trim();
334   if (!normalizedSearch) {
335     throw new Error("Missing whitelist search value");
336   }
337
338   const baseUrl = getBaseUrl();
339   return fetchGetJson({
340     url: `${baseUrl}/v1/Whitelist/Search?code=${encodeURIComponent(eventId)}&search=${encodeURIComponent(normalizedSearch)}`,
341     token
342   });
343 }
344
345 export async function fetchAccessXWhitelistLogs({ eventId, wid, token }) {
346   if (eventId === "" || eventId == null) {
347     throw new Error("Missing eventId");
348   }
349 }
```

```

349
350   const normalizedWid = String(wid || "").trim();
351   if (!normalizedWid) {
352     throw new Error("Missing whitelist user id");
353   }
354
355   const baseUrl = getBaseUrl();
356   return fetchGetJson({
357     url: `${baseUrl}/v1/Whitelist/Logs?eventId=${encodeURIComponent(eventId)}&wid=${encodeURIComponent(normalizedWid)}`,
358     token
359   });
360 }
361
362 export async function createAccessXCategory({ token, category }) {
363   if (!category?.eventId) {
364     throw new Error("Missing eventId");
365   }
366   if (!String(category?.name || "").trim()) {
367     throw new Error("Missing category name");
368   }
369
370   const baseUrl = getBaseUrl();
371   return apiRequest({
372     url: `${baseUrl}/v1/AccessX/Category/Create`,
373     method: "POST",
374     token,
375     body: category
376   });
377 }
378
379 export async function updateAccessXCategory({ token, category }) {
380   if (!category?.id) {
381     throw new Error("Missing category id");
382   }
383   if (!category?.eventId) {
384     throw new Error("Missing eventId");
385   }
386
387   const baseUrl = getBaseUrl();
388   return apiRequest({
389     url: `${baseUrl}/v1/AccessX/Category/Edit`,
390     method: "POST",
391     token,
392     body: category
393   });
394 }
395
396 export async function fetchEventDevices({ eventId, token, type = "event-wise", dedupe = true }) {
397   if (!eventId) {
398     throw new Error("Missing eventId");
399   }
400
401   const baseUrl = getBaseUrl();
402   const data = await fetchGetJsonDeduped({
403     url: `${baseUrl}/v1/Devices/List?code=${encodeURIComponent(eventId)}&type=${encodeURIComponent(type)}`,
404     token,
405     dedupe
406   });
407
408   return data?.devices ?? data?.data?.devices ?? data?.data ?? data?.list ?? [];
409 }
410
411 export async function fetchPersoDevices({ eventId, token, dedupe = true }) {
412   return fetchEventDevices({ eventId, token, type: "perso-wise", dedupe });
413 }
414
415 export async function searchDeviceMasterlist({ search, token, dedupe = true }) {
416   const query = String(search ?? "").trim();

```

```
417   if (!query) {
418     return [];
419   }
420
421   const baseUrl = getBaseUrl();
422   const data = await fetchGetJsonDeduped({
423     url: `${baseUrl}/v1/Devices/Masterlist/Search?search=${encodeURIComponent(query)}`,
424     token,
425     dedupe
426   });
427
428   return data?.devices ?? data?.data?.devices ?? data?.data ?? data?.list ?? [];
429 }
430
431 export async function addDeviceMasterlist({ token, payload }) {
432   if (!payload?.printId) {
433     throw new Error("Missing device print ID");
434   }
435   if (!payload?.hardwareId) {
436     throw new Error("Missing device hardware ID");
437   }
438
439   const baseUrl = getBaseUrl();
440   return apiRequest({
441     url: `${baseUrl}/v1/Devices/Masterlist/add`,
442     method: "POST",
443     token,
444     body: payload
445   });
446 }
447
448 export async function updateDeviceMasterlist({ token, payload }) {
449   if (!payload?.id) {
450     throw new Error("Missing device id");
451   }
452
453   const baseUrl = getBaseUrl();
454   return apiRequest({
455     url: `${baseUrl}/v1/Devices/Masterlist/edit`,
456     method: "POST",
457     token,
458     body: payload
459   });
460 }
461
462 export async function addDevicesToStall({ token, payload }) {
463   const baseUrl = getBaseUrl();
464   return apiRequest({
465     url: `${baseUrl}/v1/Devices/AddToStall`,
466     method: "POST",
467     token,
468     body: payload
469   });
470 }
471
472 export async function addPersoDevices({ eventId, token, devices }) {
473   if (!eventId) {
474     throw new Error("Missing eventId");
475   }
476
477   const baseUrl = getBaseUrl();
478   return apiRequest({
479     url: `${baseUrl}/v1/Devices/Perso/Add?code=${encodeURIComponent(eventId)}`,
480     method: "POST",
481     token,
482     body: { devices }
483   });
484 }
485
486 export async function removePersoDevice({ eventId, token, id }) {
```

```
487   if (!eventId) {
488     throw new Error("Missing eventId");
489   }
490
491   if (!id) {
492     throw new Error("Missing perso device id");
493   }
494
495   const baseUrl = getBaseUrl();
496   return apiRequest({
497     url: `${baseUrl}/v1/Devices/Perso/Remove?code=${encodeURIComponent(eventId)}`,
498     method: "POST",
499     token,
500     body: { id }
501   });
502 }
503
504 export async function filterEventTransactions({ token, payload }) {
505   const baseUrl = getBaseUrl();
506   return apiRequest({
507     url: `${baseUrl}/v1/EventTransactions/Filter`,
508     method: "POST",
509     token,
510     body: payload
511   });
512 }
513
514 export async function fetchTapXWalletCardList({ token }) {
515   const baseUrl = getBaseUrl();
516   return apiRequest({
517     url: `${baseUrl}/v2/WalletTapX/card-list`,
518     token
519   });
520 }
521
522 export async function fetchEventTransactionDetails({ token, txId, dedupe = true }) {
523   if (txId === "" || txId == null) {
524     throw new Error("Missing transaction id");
525   }
526
527   const baseUrl = getBaseUrl();
528   const data = await fetchGetJsonDeduped({
529     url: `${baseUrl}/v1/EventTransactions/Details/${encodeURIComponent(txId)}`,
530     token,
531     dedupe
532   });
533   const details =
534     data?.transaction ??
535     data?.data?.transaction ??
536     data?.data?.details ??
537     data?.details ??
538     data?.result?.transaction ??
539     data?.result?.details ??
540     data?.result ??
541     data?.data ??
542     data;
543
544   if (Array.isArray(details)) {
545     return details[0] ?? null;
546   }
547
548   return details && typeof details === "object" ? details : null;
549 }
550
551 export async function updateEventTransactionStatus({ token, txId, status, reason }) {
552   if (txId === "" || txId == null) {
553     throw new Error("Missing transaction id");
554   }
555
556   const normalizedStatus = String(status || "").trim().toLowerCase();
```



```
557   if (!["completed", "void"].includes(normalizedStatus)) {
558     throw new Error("Unsupported transaction status");
559   }
560
561   const numericTxId = Number(txId);
562   const payload = {
563     txId: Number.isNaN(numericTxId) ? txId : numericTxId,
564     status: normalizedStatus,
565     reason: String(reason || `aml ${normalizedStatus}`).trim()
566   };
567   const baseUrl = getBaseUrl();
568   return apiRequest({
569     url: `${baseUrl}/v1/EventTransactions/UpdateStatus`,
570     method: "POST",
571     token,
572     body: payload
573   });
574 }
575
576 export async function updateEventBalanceSetting({ token, eventId }) {
577   if (!eventId) {
578     throw new Error("Missing eventId");
579   }
580
581   const baseUrl = getBaseUrl();
582   return apiRequest({
583     url: `${baseUrl}/v1/Events/update-balance-setting`,
584     method: "POST",
585     token,
586     body: { eventId: Number(eventId) }
587   });
588 }
589
590 export async function closeEventDay({
591   token,
592   eventId,
593   day = "a",
594   volunteerCount = 0
595 }) {
596   if (!eventId) {
597     throw new Error("Missing eventId");
598   }
599
600   const numericEventId = Number(eventId);
601   const baseUrl = getBaseUrl();
602   return apiRequest({
603     url: `${baseUrl}/v1/Events/day-close`,
604     method: "POST",
605     token,
606     body: {
607       eventId: Number.isNaN(numericEventId) ? eventId : numericEventId,
608       day,
609       volunteerCount: Number(volunteerCount) || 0
610     }
611   });
612 }
```

FILE: apps/dashboard/src/lib/idb.js

```
1  const DB_NAME = "atomx.drafts";
2  const STORE_NAME = "drafts";
3  const DB_VERSION = 1;
4
5  let dbPromise = null;
6
7  function openDb() {
8    if (dbPromise) return dbPromise;
9    dbPromise = new Promise((resolve, reject) => {
10      if (typeof indexedDB === "undefined") {
11        reject(new Error("IndexedDB not available"));
12        return;
13      }
14      const request = indexedDB.open(DB_NAME, DB_VERSION);
15      request.onupgradeneeded = () => {
16        const db = request.result;
17        if (!db.objectStoreNames.contains(STORE_NAME)) {
18          db.createObjectStore(STORE_NAME, { keyPath: "key" });
19        }
20      };
21      request.onsuccess = () => resolve(request.result);
22      request.onerror = () => reject(request.error);
23    });
24    return dbPromise;
25  }
26
27  export async function idbGet(key) {
28    const db = await openDb();
29    return new Promise((resolve, reject) => {
30      const tx = db.transaction(STORE_NAME, "readonly");
31      const store = tx.objectStore(STORE_NAME);
32      const req = store.get(key);
33      req.onsuccess = () => resolve(req.result || null);
34      req.onerror = () => reject(req.error);
35    });
36  }
37
38  export async function idbSet(key, value) {
39    const db = await openDb();
40    return new Promise((resolve, reject) => {
41      const tx = db.transaction(STORE_NAME, "readwrite");
42      const store = tx.objectStore(STORE_NAME);
43      const req = store.put({ key, ...value });
44      req.onsuccess = () => resolve(true);
45      req.onerror = () => reject(req.error);
46    });
47  }
48
49  export async function idbDelete(key) {
50    const db = await openDb();
51    return new Promise((resolve, reject) => {
52      const tx = db.transaction(STORE_NAME, "readwrite");
53      const store = tx.objectStore(STORE_NAME);
54      const req = store.delete(key);
55      req.onsuccess = () => resolve(true);
56      req.onerror = () => reject(req.error);
57    });
58  }
```

FILE: apps/dashboard/src/lib/useFormAutosave.js

```
1 import { useCallback, useEffect, useMemo, useRef } from "react";
2 import { idbDelete, idbGet, idbSet } from "../idb";
3
4 export function useFormAutosave({
5   formRef,
6   draftKey,
7   getValues,
8   onRestore,
9   enabled = true,
10  debounceMs = 800,
11  watch = []
12 }) {
13   const saveTimerRef = useRef(null);
14   const restoringRef = useRef(false);
15   const watchKey = useMemo(() => {
16     if (!watch || watch.length === 0) return "";
17     try {
18       return JSON.stringify(watch);
19     } catch (err) {
20       console.error("Failed to serialize watch values", err);
21       return String(Date.now());
22     }
23   }, [watch]);
24
25   const clearDraft = useCallback(async () => {
26     if (!draftKey) return;
27     try {
28       await idbDelete(draftKey);
29     } catch (err) {
30       console.error("Failed to clear draft", err);
31     }
32   }, [draftKey]);
33
34   useEffect(() => {
35     if (!enabled || !draftKey) return undefined;
36     let active = true;
37
38     (async () => {
39       try {
40         const draft = await idbGet(draftKey);
41         if (!active || !draft?.values) return;
42         restoringRef.current = true;
43         try {
44           onRestore?.(draft.values, draft.updatedAt);
45         } catch (restoreError) {
46           console.error("Failed to restore draft", restoreError);
47           try {
48             await idbDelete(draftKey);
49           } catch (cleanupError) {
50             console.error("Failed to remove invalid draft", cleanupError);
51           }
52         }
53       } catch (err) {
54         console.error("Failed to load draft", err);
55       } finally {
56         restoringRef.current = false;
57       }
58     })();
59
60     return () => {
61       active = false;
62     };
63   }, [draftKey, enabled, onRestore]);
64
65   const scheduleSave = useCallback(() => {
66     if (restoringRef.current) return;
67     if (!enabled || !draftKey) return;
68     if (saveTimerRef.current) {
69       clearTimeout(saveTimerRef.current);
```

```
70     }
71     saveTimerRef.current = setTimeout(async () => {
72       try {
73         const values = getValues ? getValues() : {};
74         if (!values || typeof values !== "object") return;
75         await idbSet(draftKey, { values, updatedAt: Date.now() });
76       } catch (err) {
77         console.error("Failed to save draft", err);
78       }
79     }, debounceMs);
80   }, [debounceMs, draftKey, enabled, getValues]);
81
82   useEffect(() => {
83     if (!enabled || !draftKey || !formRef?.current) return undefined;
84     const form = formRef.current;
85
86     const handler = () => {
87       scheduleSave();
88     };
89
90     form.addEventListener("input", handler);
91     form.addEventListener("change", handler);
92
93     return () => {
94       form.removeEventListener("input", handler);
95       form.removeEventListener("change", handler);
96       if (saveTimerRef.current) {
97         clearTimeout(saveTimerRef.current);
98       }
99     };
100   }, [draftKey, enabled, formRef, scheduleSave]);
101
102   useEffect(() => {
103     if (!enabled || !draftKey) return;
104     if (!watchKey) return;
105     scheduleSave();
106   }, [watchKey, enabled, draftKey, scheduleSave]);
107
108   return { clearDraft };
109 }
```

FILE: apps/dashboard/src/store/dashboardStore.js

```
1  "use client";
2
3  import { create } from "zustand";
4  import { createJSONStorage, persist } from "zustand/middleware";
5  import { decodeJwt } from "@atomx/lib";
6
7  function normalizeService(value) {
8    return String(value || "").
9      .trim()
10     .toLowerCase()
11     .replace(/[_\s]+/g, "-");
12 }
13
14 function safeDecode(token) {
15   try {
16     return decodeJwt(token);
17   } catch (error) {
18     console.error("Failed to decode dashboard token", error);
19     return null;
20   }
21 }
22
23 function shallowEqual(a, b) {
24   if (a === b) return true;
25   if (!a || !b) return false;
26   const aKeys = Object.keys(a);
27   const bKeys = Object.keys(b);
28   if (aKeys.length !== bKeys.length) return false;
29   return aKeys.every((key) => a[key] === b[key]);
30 }
31
32 export const useDashboardStore = create(
33   persist(
34     (set, get) => ({
35       token: null,
36       profile: null,
37       eventMeta: null,
38       eventDetails: null,
39       selectedService: null,
40       vendorsByEventId: {},
41       stallsByEventId: {},
42       setToken: (token) => {
43         if (!token) {
44           set({ token: null, profile: null });
45           return;
46         }
47         const decoded = safeDecode(token);
48         if (get().token === token) {
49           set({ profile: decoded });
50           return;
51         }
52         set({ token, profile: decoded });
53       },
54       setEventMeta: (meta) => {
55         if (!meta) return;
56         set((state) => {
57           const next = { ...(state.eventMeta ?? {}), ...meta };
58           if (shallowEqual(state.eventMeta ?? {}, next)) {
59             return state;
60           }
61           return { eventMeta: next };
62         });
63       },
64       setEventDetails: (details) => {
65         if (!details) return;
66         set({ eventDetails: details });
67       },
68       setSelectedService: (service) => {
69         const normalized = normalizeService(service);
```

```

70     if (get().selectedService === (normalized || null)) return;
71     set({ selectedService: normalized || null });
72 },
73 setVendorsForEvent: (eventId, vendors) => {
74     if (!eventId) return;
75     const normalized = Array.isArray(vendors) ? vendors : [];
76     set((state) => {
77         const current = state.vendorsByEventId?.[eventId];
78         if (current === normalized) return state;
79         if (Array.isArray(current) && current.length === 0 && normalized.length === 0) {
80             return state;
81         }
82         return {
83             vendorsByEventId: {
84                 ...(state.vendorsByEventId ?? {}),
85                 [eventId]: normalized
86             }
87         };
88     });
89 },
90 setStallsForEvent: (eventId, stalls) => {
91     if (!eventId) return;
92     const normalized = Array.isArray(stalls) ? stalls : [];
93     set((state) => {
94         const current = state.stallsByEventId?.[eventId];
95         if (current === normalized) return state;
96         if (Array.isArray(current) && current.length === 0 && normalized.length === 0) {
97             return state;
98         }
99         return {
100             stallsByEventId: {
101                 ...(state.stallsByEventId ?? {}),
102                 [eventId]: normalized
103             }
104         };
105     });
106 }
107 },
108 {
109     name: "atomx.dashboard.store",
110     partialize: (state) => ({
111         token: state.token,
112         profile: state.profile,
113         eventMeta: state.eventMeta,
114         eventDetails: state.eventDetails,
115         selectedService: state.selectedService,
116         vendorsByEventId: state.vendorsByEventId,
117         stallsByEventId: state.stallsByEventId
118     }),
119     storage: createJSONStorage(() =>
120         typeof window !== "undefined" ? window.localStorage : undefined
121     )
122 }
123 )
124 );

```

FILE: apps/tag_series/api/api.js

```

1 import { getBaseUrl } from "@atomx/lib";
2
3 const BASE_URL = getBaseUrl();
4 const CARD_CLIENTS_ENDPOINT = `${BASE_URL}/v1/TagSeries/CardClients`;
5 const SERIES_ENDPOINT = `${BASE_URL}/v1/TagSeries/Series`;
6 const LOGS_ENDPOINT = `${BASE_URL}/v1/TagSeries/Logs`;
7 const BATCH_RECORDS_ENDPOINT = `${BASE_URL}/v1/TagSeries/BatchRecords`;
8 const EVENTS_ENDPOINT = `${BASE_URL}/v1/TagSeries/Events`;
9 const TAG_SERIES_API_KEY =
10   process.env.NEXT_PUBLIC_TAG_SERIES_API_KEY ??
11   process.env.NEXT_PUBLIC_DASHBOARD_API_KEY ??
12   "pZebJlF_.dv3_prod.Iu7Zitu3X30C2R6-bVZtRXRu0DeiHY-j";
13
14 function buildHeaders(token) {
15   return {
16     "x-api-key": TAG_SERIES_API_KEY
17   };
18 }
19
20 export async function fetchCardClients(token) {
21   const response = await fetch(CARD_CLIENTS_ENDPOINT, {
22     method: "GET",
23     headers: buildHeaders(token),
24     credentials: "include",
25     cache: "no-store"
26   });
27
28   if (!response.ok) {
29     const errorMessage = await response.text().catch(() => response.statusText);
30     throw new Error(
31       `CardClients request failed (${response.status}): ${errorMessage || "Unknown error"}`
32     );
33   }
34
35   return response.json();
36 }
37
38 export async function fetchSeriesMeta(token, { eventId, adminId, yearSeries }) {
39   if (!eventId || !adminId || !yearSeries) {
40     throw new Error("Missing required parameters for Series request");
41   }
42
43   const searchParams = new URLSearchParams({
44     eventId: String(eventId),
45     adminId: String(adminId),
46     yearSeries: String(yearSeries)
47   });
48
49   const response = await fetch(`${SERIES_ENDPOINT}?${searchParams.toString()}`, {
50     method: "GET",
51     headers: buildHeaders(token),
52     credentials: "include",
53     cache: "no-store"
54   });
55
56   if (!response.ok) {
57     const errorMessage = await response.text().catch(() => response.statusText);
58     throw new Error(
59       `Series request failed (${response.status}): ${errorMessage || "Unknown error"}`
60     );
61   }
62
63   return response.json();
64 }
65
66 export async function createTagSeriesLog(token, payload) {
67   if (!payload) {
68     throw new Error("Missing payload for Tag Series log request");
69   }

```

```
70
71 const response = await fetch(LOGS_ENDPOINT, {
72   method: "POST",
73   headers: {
74     ...buildHeaders(token),
75     "Content-Type": "application/json"
76   },
77   credentials: "include",
78   cache: "no-store",
79   body: JSON.stringify(payload)
80 });
81
82 if (!response.ok) {
83   const errorMessage = await response.text().catch(() => response.statusText);
84   throw new Error(
85     `Logs request failed (${response.status}): ${errorMessage || "Unknown error"}`
86   );
87 }
88
89 return response.json();
90 }
91
92 export async function fetchBatchRecords(token, { eventId, adminId }) {
93   if (!eventId || !adminId) {
94     throw new Error("Missing eventId or adminId for BatchRecords request");
95   }
96
97   const searchParams = new URLSearchParams({
98     eventId: String(eventId),
99     adminId: String(adminId)
100  });
101
102   const response = await fetch(`${BATCH_RECORDS_ENDPOINT}?${searchParams.toString()}`, {
103     method: "GET",
104     headers: buildHeaders(token),
105     credentials: "include",
106     cache: "no-store"
107   });
108
109   if (!response.ok) {
110     const errorMessage = await response.text().catch(() => response.statusText);
111     throw new Error(
112       `BatchRecords request failed (${response.status}): ${errorMessage || "Unknown error"}`
113     );
114   }
115
116   return response.json();
117 }
118
119 export async function fetchTagSeriesEvents(token) {
120   const response = await fetch(EVENTS_ENDPOINT, {
121     method: "GET",
122     headers: buildHeaders(token),
123     credentials: "include",
124     cache: "no-store"
125   });
126
127   if (!response.ok) {
128     const errorMessage = await response.text().catch(() => response.statusText);
129     throw new Error(
130       `Events request failed (${response.status}): ${errorMessage || "Unknown error"}`
131     );
132   }
133
134   return response.json();
135 }
```


FILE: apps/tag_series/app/(auth)/login/page.js

```
1 import { Suspense } from "react";
2 import { UniversalLoginPage } from "@atomx/global-components";
3
4 export const metadata = {
5   title: "Tag Series Login",
6   description: "Secure console access for the Tag Series fleet."
7 };
8
9 export default function LoginPage() {
10   return (
11     <Suspense fallback={<div className="flex min-h-screen items-center justify-center bg-[#f6f7fb]">Loadi
ng...</div>}>
12       <UniversalLoginPage appId="tag-series" redirectPath="/" />
13     </Suspense>
14   );
15 }
```

FILE: apps/tag_series/app/Admin/AddFormFactor/page.js

```

1 export default function AddFormFactorPage() {
2   return (
3     <section className="mx-auto w-full max-w-2xl space-y-6 px-4 py-10">
4       <header className="space-y-2">
5
6         <h1 className="text-3xl font-bold tracking-tight text-gray-900 ">Create New Formfactor</h1>
7
8       </header>
9
10      <form className="space-y-4 rounded-2xl border border-gray-200 bg-white p-6 shadow-card">
11        <div className="space-y-2">
12          <label className="label" htmlFor="formfactor-name">Formfactor Name</label>
13          <input
14            id="formfactor-name"
15            name="formfactor-name"
16            className="input"
17            placeholder="e.g. Wristband"
18            type="text"
19          />
20        </div>
21
22        <div className="space-y-2">
23          <label className="label" htmlFor="formfactor-number">Number</label>
24          <input
25            id="formfactor-number"
26            name="formfactor-number"
27            className="input"
28            type="text"
29            inputMode="numeric"
30            placeholder="e.g. 7"
31          />
32        </div>
33
34        <div className="space-y-2">
35          <label className="label" htmlFor="sheet-qty">Sheet Qty</label>
36          <input
37            id="sheet-qty"
38            name="sheet-qty"
39            className="input"
40            type="text"
41            inputMode="numeric"
42            placeholder="e.g. 250"
43          />
44        </div>
45
46        <button
47          type="button"
48          className="inline-flex w-full items-center justify-center rounded-full bg-emerald-500 px-6 py-3
49          text-sm font-semibold text-white shadow-sm transition hover:bg-emerald-600"
50        >
51          Add
52        </button>
53      </form>
54    </section>
55  );
56 }

```

FILE: apps/tag_series/app/Admin/AddProduct/page.js

```
1 export default function AddProductPage() {
2   return (
3     <section className="mx-auto w-full max-w-2xl space-y-6 px-4 py-10">
4       <header className="space-y-2">
5
6         <h1 className="text-3xl font-bold tracking-tight text-gray-900">Create New Product</h1>
7
8       </header>
9
10      <form className="space-y-4 rounded-2xl border border-gray-200 bg-white p-6 shadow-card">
11        <div className="space-y-2">
12          <label className="label" htmlFor="product-name">Product Name</label>
13          <input
14            id="product-name"
15            name="product-name"
16            className="input"
17            placeholder="e.g. NFC Sticker"
18            type="text"
19          />
20        </div>
21
22        <div className="space-y-2">
23          <label className="label" htmlFor="product-number">Number</label>
24          <input
25            id="product-number"
26            name="product-number"
27            className="input"
28            type="text"
29            inputMode="numeric"
30            placeholder="e.g. 12"
31          />
32        </div>
33
34        <button
35          type="button"
36          className="inline-flex w-full items-center justify-center rounded-full bg-emerald-500 px-6 py-3
37          text-sm font-semibold text-white shadow-sm transition hover:bg-emerald-600"
38          >
39          Add
40        </button>
41      </form>
42    </section>
43  );
44 }
```

FILE: apps/tag_series/app/Admin/View/page.js

```

1 function DownloadCloudIcon(props) {
2   return (
3     <svg
4       viewBox="0 0 24 24"
5       fill="none"
6       stroke="currentColor"
7       strokeWidth="1.8"
8       strokeLinecap="round"
9       strokeLinejoin="round"
10      aria-hidden="true"
11      {...props}
12    >
13      <path d="M8 17a4 4 0 1 1 0-8 5.5 5.5 0 0 1 10.4 1.66A3.5 3.5 0 1 1 19 17" />
14      <path d="M12 12v9" />
15      <path d="m8 16 4 4 4-4" />
16    </svg>
17  );
18 }
19
20 const RECORDS = [
21   {
22     id: "1",
23     loginEmail: "admin@atomx.in",
24     eventId: "333",
25     yearSeries: "25",
26     client: "00",
27     eventSeries: "00",
28     quantityGenerated: 998000,
29     quantityExtra: 2000,
30     formType: "1 : Card",
31     createdAt: "2024-10-28T10:32:00Z",
32     mailOwner: "Omkar A.",
33     mailTo: ["events@brand.com"],
34     mailCc: ["ops@atomx.in", "support@atomx.in"],
35     mailMessage: "Attached: QR redirects for AtomX card batch.",
36     includesUrls: true,
37     tagsRange: { first: "250000000001", last: "250000999999" },
38   },
39   {
40     id: "2",
41     loginEmail: "ops@atomx.in",
42     eventId: "782",
43     yearSeries: "24",
44     client: "01",
45     eventSeries: "12",
46     quantityGenerated: 15000,
47     quantityExtra: 0,
48     formType: "4 : Tag",
49     createdAt: "2024-10-18T08:05:00Z",
50     mailOwner: "Priya S.",
51     mailTo: ["client.ops@brand.com"],
52     mailCc: ["qa@atomx.in"],
53     mailMessage: "Deliverables ready. Please sync printing line.",
54     includesUrls: false,
55     tagsRange: { first: "240112000001", last: "240112015000" },
56   },
57   {
58     id: "3",
59     loginEmail: "support@atomx.in",
60     eventId: "987",
61     yearSeries: "23",
62     client: "00",
63     eventSeries: "45",
64     quantityGenerated: 64000,
65     quantityExtra: 0,
66     formType: "7 : Bamboo-card",
67     createdAt: "2024-09-15T15:45:00Z",
68     mailOwner: "Nikita R.",
69     mailTo: ["green.events@brand.com"],

```

```

70     mailCc: ["ops@atomx.in", "finance@atomx.in"],
71     mailMessage: "Eco batch ready for QA upload.",
72     includesUrls: true,
73     tagsRange: { first: "230045000001", last: "230045064000" },
74   },
75 ];
76
77 function formatDate(isoString) {
78   const date = new Date(isoString);
79   return date.toLocaleString("en-IN", {
80     dateStyle: "medium",
81     timeStyle: "short",
82   });
83 }
84
85 function formatNumber(value) {
86   return value.toLocaleString("en-IN");
87 }
88
89 function formatEmails(list) {
90   if (!Array.isArray(list) || list.length === 0) return "-";
91   return list.join(", ");
92 }
93
94 function formatTagRange(range) {
95   if (!range || !range.first || !range.last) return "-";
96   return `From: ${range.first} → To: ${range.last}`;
97 }
98
99 const COLUMNS = [
100   { key: "loginEmail", label: "Login Email" },
101   { key: "eventId", label: "Event ID" },
102   { key: "yearSeries", label: "Year Series" },
103   { key: "client", label: "Client" },
104   { key: "eventSeries", label: "Event Series" },
105   { key: "quantityGenerated", label: "Generated Qty", formatter: formatNumber },
106   { key: "quantityExtra", label: "Extra Qty", formatter: formatNumber },
107   { key: "tagsRange", label: "Generated Card IDs", formatter: formatTagRange },
108   { key: "createdAt", label: "Created", formatter: formatDate },
109   { key: "mailOwner", label: "Mail" },
110 ];
111
112 export default function AdminViewPage() {
113   return (
114     <section className="w-full self-start px-6 pt-6 pb-10">
115       <h1 className="mb-6 text-3xl font-bold tracking-tight text-gray-900">View Tag Batches</h1>
116
117       <div className="relative rounded-3xl border border-gray-200 shadow-card">
118         <div className="w-full">
119           <table className="min-w-full text-sm">
120             <thead className="bg-gradient-to-r from-gray-100 to-gray-200">
121               <tr>
122                 {COLUMNS.map((column) => (
123                   <th
124                     key={column.key}
125                     scope="col"
126                     className="whitespace-nowrap px-4 py-3 text-left text-xs font-bold uppercase tracking
127 -wider text-gray-700"
128                   >
129                     {column.label}
130                   </th>
131                 ))}
132                 <th className="px-6 py-3 text-right text-xs font-bold uppercase tracking-wider text-gray-
133 700">Actions</th>
134             </thead>
135             <tbody>
136               {RECORDS.map((record, idx) => {
137                 const isEven = idx % 2 === 0;
138                 return (

```

```

138         <tr
139             key={record.id}
140             className={` ${isEven ? "bg-white" : "bg-gray-50"} border-t border-gray-100 transition
141             hover:bg-[#fff5f2]`}
142         >
143             {COLUMNS.map((column) => {
144                 const rawValue = record[column.key];
145                 const displayValue = column.formatter ? column.formatter(rawValue) : rawValue;
146
147                 if (column.key === "loginEmail") {
148                     return (
149                         <td key={column.key} className="px-4 py-3 text-gray-900">
150                             <div className="flex items-center gap-3">
151                                 <span className="inline-flex h-8 w-8 items-center justify-center rounded-fu
152                                 ll bg-[#e04420]/10 text-xs font-semibold text-[#e04420]">
153                                     {record.mailOwner
154                                         .split(" ")
155                                         .map((part) => part[0])
156                                         .join("")}
157                                 </span>
158                                 <div className="min-w-[12rem]">
159                                     <p className="font-semibold">{displayValue}</p>
160                                     <p className="text-xs text-gray-500">Owned by {record.mailOwner}</p>
161                                 </div>
162                             </div>
163                         </td>
164                     );
165                 }
166
167                 if (column.key === "mailOwner") {
168                     return (
169                         <td key={column.key} className="px-4 py-3 text-gray-800">
170                             <div className="group relative inline-flex">
171                                 <button
172                                     type="button"
173                                     className="inline-flex h-9 w-9 items-center justify-center rounded-full b
174                                     order border-gray-300 text-lg text-gray-600 transition hover:border-[#e04420] hover:text-[#e04420] focus:
175                                     border-[#e04420] focus:text-[#e04420]"
176                                     aria-haspopup="true"
177                                     aria-label={`Mail details shared by ${record.mailOwner}`}
178                                 >
179                                     :
180                                 </button>
181                                 <div
182                                     className="pointer-events-none absolute right-0 top-full z-20 mt-2 w-60 r
183                                     ounded-2xl border border-gray-200 bg-white p-4 text-xs text-gray-600 shadow-xl opacity-0 transition group
184                                     -hover:pointer-events-auto group-hover:opacity-100 group-focus-within:pointer-events-auto group-focus-wit
185                                     hin:opacity-100"
186                                 >
187                                     <p className="text-sm font-semibold text-gray-900">Shared by {record.mail
188                                     Owner}</p>
189
190                                     <div className="mt-3 space-y-2">
191                                         <div>
192                                             <p className="font-medium text-gray-700">To</p>
193                                             <p className="text-gray-600">{formatEmails(record.mailTo)}</p>
194                                         </div>
195                                         <div>
196                                             <p className="font-medium text-gray-700">CC</p>
197                                             <p className="text-gray-600">{formatEmails(record.mailCc)}</p>
198                                         </div>
199                                         <div>
200                                             <p className="font-medium text-gray-700">Message</p>
201                                             <p className="text-gray-600">{record.mailMessage}</p>
202                                         </div>
203                                         <div>
204                                             <p className="font-medium text-gray-700">Form Type</p>
205                                             <p className="text-gray-600">{record.formType}</p>
206                                         </div>
207                                         <div className="flex items-center gap-2 text-gray-700">
208                                             <span className={`h-2 w-2 rounded-full ${record.includesUrls ? "bg-em

```

```

    erald-500" : "bg-slate-400"}` } />
200         <span>{record.includesUrls ? "URLs included" : "IDs only"}</span>
201         </div>
202     </div>
203 </div>
204 </div>
205 </td>
206     );
207 }
208
209     return (
210         <td key={column.key} className="px-4 py-3 text-gray-800">
211             {displayValue}
212         </td>
213     );
214     })}
215
216     <td className="px-6 py-3">
217         <div className="flex items-center justify-end">
218             <button
219                 type="button"
220                 className="inline-flex items-center gap-2 rounded-full bg-[#fde6de] px-4 py-2 t
ext-xs font-semibold text-[#c23a19] shadow-sm transition hover:brightness-95"
221                 >
222                 <DownloadCloudIcon className="h-4 w-4" />
223
224             </button>
225         </div>
226     </td>
227 </tr>
228 );
229 })}
230 </tbody>
231 </table>
232 </div>
233 </div>
234 </section>
235 );
236 }

```

FILE: apps/tag_series/app/generate/page.js

```

1  "use client";
2
3  import { Suspense, useCallback, useEffect, useMemo, useState } from "react";
4  import { useSearchParams, useRouter } from "next/navigation";
5  import * as XLSX from "xlsx";
6  import { createTagSeriesLog, fetchBatchRecords } from "../../api/api";
7  import { getStepOneState } from "../../lib/setupStorage";
8  import { capturePostHogEvent } from "@atomx/global-components";
9
10 const LOG_SERIAL_WIDTH = 5;
11 const MAX_QTY = 1_000_000;
12
13 // Form (f)
14 const FORM_TYPES = [
15   { value: 1, label: "Card" },
16   { value: 2, label: "Paper-card" },
17   { value: 3, label: "Sticker" },
18   { value: 4, label: "Tag" },
19   { value: 5, label: "Band" },
20   { value: 6, label: "Bracelet" },
21   { value: 7, label: "Bamboo-card" },
22   { value: 8, label: "Bamboo-tag-square" },
23   { value: 9, label: "Accred" },
24   { value: 10, label: "Powerbank station" },
25 ];
26 const EXTRA_QTY_DIVISORS = {
27   1: 28,
28   4: 28,
29   9: 16
30 };
31
32 // Product (p) - only 1: event
33 const PRODUCT_VALUE = 1;
34 const BASE_URL = "https://tags.atomx.in/redirect/v1?scanType=QR";
35
36 function pad(num, width) {
37   const s = String(num);
38   return s.length >= width ? s : "0".repeat(width - s.length) + s;
39 }
40
41 function buildTag(prefix, serial) {
42   return `${prefix}${pad(serial, LOG_SERIAL_WIDTH)}`;
43 }
44
45 function buildTagUrl({ eventId, formType, tag }) {
46   return `${BASE_URL}&f=${formType}&p=${PRODUCT_VALUE}&c=${eventId}&tag=${tag}`;
47 }
48
49 function getTagSeriesToken() {
50   if (typeof window === "undefined") return null;
51   const canonical = window.localStorage.getItem("atomx.auth.tag-series");
52   if (canonical) return canonical;
53
54   const legacy = window.localStorage.getItem("atomx.auth.tag_series");
55   if (legacy) {
56     // Migrate old key to canonical key to keep all apps in sync.
57     window.localStorage.setItem("atomx.auth.tag-series", legacy);
58     return legacy;
59   }
60   return null;
61 }
62
63 function normalizeBatchRecords(response) {
64   if (!response) return [];
65   if (typeof response === "string") {
66     try {
67       return normalizeBatchRecords(JSON.parse(response));
68     } catch (error) {
69       return [];
70     }
71   }

```



```
70     }
71   }
72   if (Array.isArray(response)) return response;
73
74   const candidates = [
75     response.records,
76     response.data?.records,
77     response.seriesLogs,
78     response.data?.seriesLogs,
79     response.batchRecords,
80     response.data?.batchRecords,
81     response.data?.batches,
82     response.batches,
83     response.record,
84     response.data?.record,
85     response.batch,
86     response.data?.batch,
87     response.data,
88     response
89   ];
90
91   for (const candidate of candidates) {
92     if (!candidate) continue;
93     if (Array.isArray(candidate)) return candidate;
94     if (typeof candidate === "object") {
95       const values = Object.values(candidate);
96       if (values.length && values.every((value) => typeof value === "object")) {
97         return values;
98       }
99       return [candidate];
100    }
101  }
102
103  return [];
104 }
105
106 function normalizeSeriesList(response) {
107   if (!response) return [];
108   if (typeof response === "string") {
109     try {
110       return normalizeSeriesList(JSON.parse(response));
111     } catch (error) {
112       return [];
113     }
114   }
115   if (Array.isArray(response)) return response;
116
117   const candidates = [
118     response.tagSeries_existing,
119     response.data?.tagSeries_existing,
120     response.tagSeriesExisting,
121     response.data?.tagSeriesExisting,
122     response.tag_series_existing,
123     response.data?.tag_series_existing,
124     response.series,
125     response.data?.series,
126     response.tagSeries,
127     response.data?.tagSeries,
128     response.seriesList,
129     response.data?.seriesList,
130     response.records,
131     response.data?.records,
132     response.data,
133     response
134   ];
135
136   for (const candidate of candidates) {
137     if (!candidate) continue;
138     if (Array.isArray(candidate)) return candidate;
139     if (typeof candidate === "object") {
```

```
140     const values = Object.values(candidate);
141     if (values.length && values.every((value) => typeof value === "object")) {
142         return values;
143     }
144     return [candidate];
145 }
146 }
147
148 return [];
149 }
150
151 function isSeriesEntry(value) {
152     return (
153         value &&
154         typeof value === "object" &&
155         (value.series !== undefined || value.tagSeries !== undefined) &&
156         (value.id !== undefined ||
157             value.eventwiseId !== undefined ||
158             value.tagSeriesId !== undefined ||
159             value.tagSeriesExistingId !== undefined)
160     );
161 }
162
163 function collectSeriesEntries(value, seen = new Set()) {
164     if (!value) return [];
165     if (typeof value === "string") {
166         try {
167             return collectSeriesEntries(JSON.parse(value), seen);
168         } catch (error) {
169             return [];
170         }
171     }
172     if (typeof value !== "object" || seen.has(value)) return [];
173     seen.add(value);
174
175     if (Array.isArray(value)) {
176         return value.flatMap((item) => collectSeriesEntries(item, seen));
177     }
178
179     const direct = isSeriesEntry(value) ? [value] : [];
180     return direct.concat(
181         Object.values(value).flatMap((item) => collectSeriesEntries(item, seen))
182     );
183 }
184
185 function getSeriesEntryKeys(entry) {
186     return [
187         entry?.id,
188         entry?.eventwiseId,
189         entry?.eventwise_id,
190         entry?.tagSeriesId,
191         entry?.tag_series_id,
192         entry?.tagSeriesExistingId,
193         entry?.tag_series_existing_id
194     ]
195     .map((value) => Number(value))
196     .filter((value) => Number.isFinite(value));
197 }
198
199 function buildSeriesMap(seriesList = []) {
200     const map = new Map();
201     seriesList.forEach((entry) => {
202         getSeriesEntryKeys(entry).forEach((key) => {
203             map.set(key, entry);
204         });
205     });
206     return map;
207 }
208
209 function getSeriesMetaForLog(seriesMap, log) {
```

```
210 const keys = [
211   log?.eventId,
212   log?.eventwise_id,
213   log?.tagSeriesId,
214   log?.tag_series_id,
215   log?.tagSeriesExistingId,
216   log?.tag_series_existing_id
217 ]
218   .map((value) => Number(value))
219   .filter((value) => Number.isFinite(value));
220
221 for (const key of keys) {
222   const seriesMeta = seriesMap?.get?.(key);
223   if (seriesMeta) return seriesMeta;
224 }
225
226 return null;
227 }
228
229 function getBatchRecords(response) {
230   const normalized = normalizeBatchRecords(response);
231   if (normalized.length) return normalized;
232
233   const fallbacks = [
234     response?.seriesLogs,
235     response?.data?.seriesLogs,
236     response?.series_logs,
237     response?.data?.series_logs,
238     response?.result?.seriesLogs,
239     response?.data?.result?.seriesLogs
240   ];
241
242   for (const fallback of fallbacks) {
243     const next = normalizeBatchRecords(fallback);
244     if (next.length) return next;
245   }
246
247   return [];
248 }
249
250 function formatRequestTime({ requestId, createdAt }) {
251   const ts = Number(requestId);
252   const date = Number.isFinite(ts) && ts > 0 ? new Date(ts * 1000) : new Date(createdAt);
253   if (!Number.isFinite(date.getTime())) return "-";
254   return new Intl.DateTimeFormat("en-IN", {
255     dateStyle: "medium",
256     timeStyle: "short",
257     timeZone: "Asia/Kolkata"
258   }).format(date);
259 }
260
261 function formatRanges(logs = [], options = {}) {
262   const { seriesMap, fallbackYear, fallbackBrand } = options;
263   const list = Array.isArray(logs)
264     ? logs
265     : typeof logs === "object" && logs
266       ? Object.values(logs)
267       : [];
268   if (!list.length) return "-";
269
270   return list
271     .map((log) => {
272       const seriesMeta = getSeriesMetaForLog(seriesMap, log);
273       const seriesSource = seriesMeta?.series ?? seriesMeta?.tagSeries ?? log.series;
274       const yearSource = seriesMeta?.yearSeries ?? seriesMeta?.year ?? fallbackYear;
275       const brandSource = seriesMeta?.brand ?? seriesMeta?.brandSeries ?? fallbackBrand ?? "0";
276       const hasSeries =
277         seriesSource !== undefined && seriesSource !== null && String(seriesSource).trim() !== "";
278       const hasYear =
279         yearSource !== undefined && yearSource !== null && String(yearSource).trim() !== "";
```

```

280     const hasBrand =
281       brandSource !== undefined && brandSource !== null && String(brandSource).trim() !== "";
282     if (!hasSeries || !hasYear || !hasBrand) return null;
283
284     const seriesValue = String(seriesSource).padStart(2, "0");
285     const yearValue = String(
286       seriesMeta?.yearSeries ?? seriesMeta?.year ?? fallbackYear ?? ""
287     ).padStart(2, "0");
288     const brandValue = String(brandSource);
289     const prefix = `${yearValue}${brandValue}${seriesValue}`;
290     const startTag = buildTag(prefix, log.start);
291     const endTag = buildTag(prefix, log.end);
292     return `${startTag}–${endTag}`;
293   })
294   .filter(Boolean)
295   .join(", ") || "-";
296 }
297
298 function getRecordLogs(record) {
299   if (!record) return [];
300   return (
301     record.tagSeriesLogs ||
302     record.seriesLogs ||
303     record.logs ||
304     record.tag_series_logs ||
305     []
306   );
307 }
308
309 function formatCount(value) {
310   const next = Number(value);
311   return Number.isFinite(next) ? next.toLocaleString("en-IN") : "-";
312 }
313
314 function getRecordKey(record, index) {
315   return String(record.id ?? record.requestId ?? `batch-${index}`);
316 }
317
318 function getRangeFormType(record, seriesMeta, fallbackFormType) {
319   const value =
320     record?.formFactor ??
321     record?.form_factor ??
322     seriesMeta?.formFactor ??
323     seriesMeta?.form_factor ??
324     fallbackFormType;
325   const next = Number(value);
326   return Number.isFinite(next) ? next : fallbackFormType;
327 }
328
329 function buildRangesFromLogs(logs = [], options = {}) {
330   const {
331     seriesMap,
332     fallbackYear,
333     fallbackBrand,
334     fallbackFormType,
335     eventId,
336     requireSeriesMeta = false
337   } = options;
338   const list = Array.isArray(logs)
339     ? logs
340     : typeof logs === "object" && logs
341       ? Object.values(logs)
342       : [];
343
344   return list
345     .map((log, index) => {
346       const seriesMeta = getSeriesMetaForLog(seriesMap, log);
347       const seriesSource =
348         seriesMeta?.series ??
349         seriesMeta?.tagSeries ??

```

```

350     log.series;
351
352     if (requireSeriesMeta && (seriesSource === undefined || seriesSource === null || seriesSource === "
353 "")) {
354         return null;
355     }
356
357     const seriesValue = String(seriesSource ?? "").padStart(2, "0");
358     const yearValue = String(
359         seriesMeta?.yearSeries ?? seriesMeta?.year ?? log.yearSeries ?? fallbackYear ?? ""
360     ).padStart(2, "0");
361     const brandValue = String(
362         seriesMeta?.clientSeries ?? seriesMeta?.brandSeries ?? log.brandSeries ?? fallbackBrand ?? "0"
363     );
364     const start = Number(log.start);
365     const end = Number(log.end);
366     const hasPrefix =
367         seriesValue.trim() &&
368         yearValue.trim() &&
369         brandValue.trim() &&
370         Number.isFinite(start) &&
371         Number.isFinite(end);
372
373     if (!hasPrefix) return null;
374
375     const prefix = `${yearValue}${brandValue}${seriesValue}`;
376     const formTypeValue = getRangeFormType(log, seriesMeta, fallbackFormType);
377     const startTag = buildTag(prefix, start);
378     const endTag = buildTag(prefix, end);
379
380     return {
381         id: log.id ?? index,
382         eventwiseId: log.eventwiseId,
383         series: seriesSource,
384         start,
385         end,
386         count: Number(log.count ?? 0),
387         prefix,
388         startTag,
389         endTag,
390         formType: formTypeValue,
391         firstUrl: buildTagUrl({ eventId, formType: formTypeValue, tag: startTag }),
392         lastUrl: buildTagUrl({ eventId, formType: formTypeValue, tag: endTag })
393     };
394 }
395 .filter(Boolean);
396 }
397
398 function buildExcelRows(ranges, includeUrls, eventId, fallbackFormType) {
399     const rows = [];
400
401     ranges.forEach((range) => {
402         for (let serial = range.start; serial <= range.end; serial += 1) {
403             const tag = buildTag(range.prefix, serial);
404             const digits = tag.split("").map((char) => Number(char));
405             const code = digits.reduce((sum, value) => sum + value, 0);
406             const codeSquared = code * code;
407             const lastDigit = digits[digits.length - 1] ?? 0;
408             const adjusted = codeSquared - lastDigit;
409             const finalCode = String(Math.abs(adjusted) % 100).padStart(2, "0");
410             const row = {
411                 "Final Series": `${tag} / ${finalCode}`
412             };
413             if (includeUrls) {
414                 row.URL = buildTagUrl({
415                     eventId,
416                     formType: range.formType ?? fallbackFormType,
417                     tag
418                 });
419             }
420         }
421     });
422 }

```

```

419     rows.push(row);
420   }
421   });
422
423   return rows;
424 }
425
426 function downloadRangesExcel({ ranges, eventId, formType, includeUrls, fileLabel = "Tag" }) {
427   if (!ranges?.length) return null;
428
429   const headers = includeUrls ? ["Final Series", "URL"] : ["Final Series"];
430   const rows = buildExcelRows(ranges, includeUrls, eventId, formType);
431   const worksheet = XLSX.utils.json_to_sheet(rows, { header: headers });
432   const workbook = XLSX.utils.book_new();
433   XLSX.utils.book_append_sheet(workbook, worksheet, "Tags");
434
435   const safeLabel = fileLabel.replace(/[\:\?*>|]/g, "-");
436   const allTags = ranges.flatMap((range) => [range.startTag, range.endTag]);
437   const minMax = allTags.reduce(
438     (acc, tag) => {
439       if (!acc[0] || BigInt(tag) < BigInt(acc[0])) acc[0] = tag;
440       if (!acc[1] || BigInt(tag) > BigInt(acc[1])) acc[1] = tag;
441       return acc;
442     },
443     [null, null]
444   );
445   const [minTag, maxTag] = minMax;
446   const fileName = `${safeLabel}-Series ${minTag} to ${maxTag}.xlsx`;
447
448   XLSX.writeFile(workbook, fileName);
449
450   return {
451     fileName,
452     rowCount: rows.length
453   };
454 }
455
456 function getRequestId() {
457   return Math.floor(Date.now() / 1000);
458 }
459
460 function generatePseudoId(length = 12) {
461   const alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";
462   let result = "";
463   for (let i = 0; i < length; i += 1) {
464     result += alphabet[Math.floor(Math.random() * alphabet.length)];
465   }
466   return result;
467 }
468
469 function calculateExtraQty(quantity, formType) {
470   const value = Number(quantity);
471   if (!Number.isFinite(value) || value <= 0) return "";
472   const divisor = EXTRA_QTY_DIVISORS[formType];
473   if (!divisor) return "0";
474   const rounded = Math.ceil(value / divisor) * divisor;
475   return String(Math.max(rounded - value, 0));
476 }
477
478 function GenerateForm() {
479   const router = useRouter();
480   const searchParams = useSearchParams();
481   const eventId = searchParams.get("c") || "";
482
483   const defaultYY = useMemo(() => new Date().getFullYear().toString().slice(-2), []);
484
485   const [yy, setYy] = useState(defaultYY); // XX - Year
486   const [brand, setBrand] = useState("0"); // Y - 0=AtomX, 1=Client
487   const [series, setSeries] = useState("00"); // ZZ - 2 digits
488   const [qty, setQty] = useState(""); // Quantity

```

```

489 const [extraQty, setExtraQty] = useState(""); // Extra Quantity
490 const [formType, setFormType] = useState(1); // f
491 const [note, setNote] = useState("");
492 const [out, setOut] = useState(null);
493 const [err, setErr] = useState("");
494 const [includeLinks, setIncludeLinks] = useState(true);
495 const [recentIncludeLinks, setRecentIncludeLinks] = useState({});
496 const [clientMeta, setClientMeta] = useState(null);
497 const [batchRecords, setBatchRecords] = useState([]);
498 const [batchLoading, setBatchLoading] = useState(false);
499 const [batchError, setBatchError] = useState("");
500 const [batchSeriesMap, setBatchSeriesMap] = useState(() => new Map());
501
502 const fetchAndSetBatchRecords = useCallback(async () => {
503   if (!eventId || !clientMeta?.clientId || typeof window === "undefined") return;
504   setBatchLoading(true);
505   setBatchError("");
506   try {
507     const token = getTagSeriesToken();
508     const response = await fetchBatchRecords(token, {
509       eventId,
510       adminId: clientMeta.clientId
511     });
512     const recordList = getBatchRecords(response);
513     const seriesList = [
514       ...normalizeSeriesList(response),
515       ...collectSeriesEntries(response)
516     ];
517     if (seriesList.length) {
518       setBatchSeriesMap(buildSeriesMap(seriesList));
519     }
520     const sorted = recordList
521       .slice()
522       .sort((a, b) => Number(b.requestId ?? 0) - Number(a.requestId ?? 0));
523     setBatchRecords(sorted.slice(0, 4));
524   } catch (error) {
525     setBatchError(error.message || "Unable to load batch records.");
526   } finally {
527     setBatchLoading(false);
528   }
529 }, [eventId, clientMeta?.clientId, yy]);
530
531 useEffect(() => {
532   if (!/^d+$/.test(eventId)) {
533     router.replace("/");
534   }
535 }, [eventId, router]);
536
537 useEffect(() => {
538   const state = getStepOneState();
539   if (state && state.eventId === eventId) {
540     setClientMeta(state);
541   }
542 }, [eventId]);
543
544 useEffect(() => {
545   setExtraQty(calculateExtraQty(qty, formType));
546 }, [qty, formType]);
547
548 useEffect(() => {
549   fetchAndSetBatchRecords();
550 }, [fetchAndSetBatchRecords]);
551
552 useEffect(() => {
553   if (clientMeta?.yearSeries) {
554     setYy(clientMeta.yearSeries);
555   }
556   if (clientMeta?.clientSeries !== undefined && clientMeta?.clientSeries !== null) {
557     setBrand(String(clientMeta.clientSeries));
558   }

```

```

559     if (clientMeta?.eventSeries) {
560         setSeries(clientMeta.eventSeries);
561     }
562 }, [clientMeta]);
563
564 const handleGenerate = async (e) => {
565     e.preventDefault();
566     setErr("");
567     setOut(null);
568
569     if (!clientMeta?.clientId) {
570         setErr("Client information is missing. Please complete Step 1 again.");
571         return;
572     }
573
574     if (!/^d{2}$/.test(yy)) return setErr("Year Series must be exactly 2 digits (e.g., 25).");
575     if (!/^01$/.test(brand)) return setErr("Client must be AtomX (0) or Client (1).");
576     if (!/^d{2}$/.test(series)) return setErr("Event Series must be exactly 2 digits (00-99).");
577     if (!/^d+$/.test(qty)) return setErr("Quantity must be a positive number.");
578     const fallbackExtraQty =
579         extraQty === "" ? calculateExtraQty(qty, formType) : extraQty;
580     if (extraQty === "" && fallbackExtraQty !== "") {
581         setExtraQty(fallbackExtraQty);
582     }
583     if (!/^d+$/.test(fallbackExtraQty)) {
584         return setErr("Extra Quantity must be a non-negative number.");
585     }
586     if (note.length > 50) return setErr("Note must be 50 characters or fewer.");
587     const q = parseInt(qty, 10);
588     const spare = parseInt(fallbackExtraQty, 10);
589     const totalCount = q + spare;
590     if (q < 1) return setErr("Quantity must be at least 1.");
591     if (q > MAX_QTY) return setErr(`Max quantity is ${MAX_QTY.toLocaleString()}`);
592
593     const token = getTagSeriesToken();
594     const payload = {
595         eventId: Number(eventId),
596         adminId: Number(clientMeta.clientId),
597         yearSeries: Number(yy),
598         scanType: "tagQR",
599         formFactor: formType,
600         product: PRODUCT_VALUE,
601         requestId: getRequestId(),
602         userPseudoId: generatePseudoId(),
603         count: totalCount,
604         batchNote: note,
605         spare
606     };
607
608     try {
609         const response = await createTagSeriesLog(token, payload);
610         const logs = Array.isArray(response?.logs) ? response.logs : [];
611         const seriesList = Array.isArray(response?.series) ? response.series : [];
612         if (!logs.length) {
613             setErr("No log ranges returned from the server.");
614             return;
615         }
616         if (!seriesList.length) {
617             setErr("No series metadata returned from the server.");
618             return;
619         }
620
621         const seriesMap = buildSeriesMap(seriesList);
622
623         const ranges = logs.map((log, index) => {
624             const seriesMeta = getSeriesMetaForLog(seriesMap, log);
625             if (!seriesMeta) {
626                 throw new Error(`Missing series metadata for eventwiseId ${log.eventwiseId}`);
627             }
628         });

```



```

629     const seriesValue = String(seriesMeta.series ?? "").padStart(2, "0");
630     const yearValue = String(seriesMeta.yearSeries ?? yy).padStart(2, "0");
631     const prefix = `${yearValue}${brand}${seriesValue}`;
632     const start = Number(log.start);
633     const end = Number(log.end);
634     const startTag = buildTag(prefix, start);
635     const endTag = buildTag(prefix, end);
636     return {
637         id: log.id ?? index,
638         eventId: log.eventId,
639         series: seriesMeta.series,
640         start,
641         end,
642         count: Number(log.count ?? 0),
643         prefix,
644         startTag,
645         endTag,
646         firstUrl: buildTagUrl({ eventId, formType, tag: startTag }),
647         lastUrl: buildTagUrl({ eventId, formType, tag: endTag })
648     };
649 };
650
651 setOut({
652     ranges,
653     params: {
654         formType,
655         eventId
656     }
657 });
658 capturePostHogEvent("operation_tag_batch_generated", {
659     app: "tag_series",
660     event_id: eventId,
661     form_type: formType,
662     form_type_label: FORM_TYPES.find((ft) => ft.value === formType)?.label,
663     quantity: q,
664     extra_quantity: spare,
665     total_count: totalCount,
666     year_series: yy,
667     brand_series: brand,
668     client_id: clientMeta?.clientId,
669     client_name: clientMeta?.clientName,
670     range_count: ranges.length
671 });
672 await fetchAndSetBatchRecords();
673 } catch (error) {
674     capturePostHogEvent("$exception", {
675         app: "tag_series",
676         $exception_message: error.message,
677         $exception_source: "tag_batch_generated"
678     });
679     setErr(error.message || "Unable to save tag series log.");
680     return;
681 }
682 };
683
684 const handleDownloadExcel = () => {
685     if (!out) return;
686
687     const { ranges, params } = out;
688     const formLabel =
689         FORM_TYPES.find((ft) => ft.value === params.formType)?.label ?? "Tag";
690     const result = downloadRangesExcel({
691         ranges,
692         eventId: params.eventId,
693         formType: params.formType,
694         includeUrls: includeLinks,
695         fileLabel: formLabel
696     });
697     if (!result) return;
698 }

```

```

699     capturePostHogEvent("operation_tag_excel_downloaded", {
700         app: "tag_series",
701         event_id: params.eventId,
702         form_type: params.formType,
703         form_type_label: formLabel,
704         include_urls: includeLinks,
705         row_count: result.rowCount,
706         file_name: result.fileName
707     });
708 };
709
710 const handleDownloadRecentBatch = (record, index) => {
711     const key = getRecordKey(record, index);
712     const includeUrls = recentIncludeLinks[key] ?? true;
713     const logs = getRecordLogs(record);
714     const ranges = buildRangesFromLogs(logs, {
715         seriesMap: batchSeriesMap,
716         fallbackYear: yy,
717         fallbackBrand: brand,
718         fallbackFormType: formType,
719         eventId,
720         requireSeriesMeta: true
721     });
722     const selectedFormType = ranges[0]?.formType ?? formType;
723     const formLabel =
724         FORM_TYPES.find((ft) => ft.value === selectedFormType)?.label ?? "Tag";
725     const result = downloadRangesExcel({
726         ranges,
727         eventId,
728         formType: selectedFormType,
729         includeUrls,
730         fileLabel: formLabel
731     });
732     if (!result) return;
733
734     capturePostHogEvent("operation_tag_recent_batch_downloaded", {
735         app: "tag_series",
736         event_id: eventId,
737         form_type: selectedFormType,
738         form_type_label: formLabel,
739         include_urls: includeUrls,
740         row_count: result.rowCount,
741         file_name: result.fileName,
742         request_id: record.requestId ?? null
743     });
744 };
745
746 return (
747     <div className="min-h-[100dvh] w-full bg-gradient-to-b from-gray-50 to-white px-4 py-8">
748         <div className="mx-auto w-full max-w-2xl rounded-2xl border border-gray-200 bg-white p-6 shadow-xl
749         shadow-gray-100/60 md:p-8">
750             <div className="mb-6 text-center">
751                 <h1 className="text-3xl font-semibold tracking-tight text-slate-900">
752                     AtomX <span className="text-[#e04420]"></span> Tag Generator
753                 </h1>
754                 <p className="mt-2 text-sm text-gray-600">
755                     Event ID: <span className="font-semibold text-gray-900">{eventId}</span>
756                 </p>
757             </div>
758             <div className="mb-4 rounded-2xl border border-amber-200 bg-amber-50 px-3 py-2 text-sm text-amb
759             er-800">
760                 Client information is missing. Please complete Step 1 again if values look incorrect.
761             </div>
762         </div>
763     </div>
764     <div className="mb-6 mt-6 rounded-2xl border border-slate-200 bg-white p-4 shadow-sm">
765         <div className="mb-4 flex flex-wrap items-center justify-between gap-2">
766             <h2 className="text-base font-semibold text-slate-900">Recent Batch Records</h2>

```

```

767         <span className="text-xs text-slate-500">Last 4 batches</span>
768     </div>
769
770     {batchLoading && (
771         <div className="rounded-xl border border-slate-200 bg-slate-50 px-4 py-3 text-sm text-slate-6
00">
772         Loading batch records...
773     </div>
774     )}
775
776     {batchError && !batchLoading && (
777         <div className="rounded-xl border border-red-200 bg-red-50 px-4 py-3 text-sm text-red-700">
778         {batchError}
779     </div>
780     )}
781
782     {!batchLoading && !batchError && batchRecords.length === 0 && (
783         <div className="rounded-xl border border-slate-200 bg-slate-50 px-4 py-3 text-sm text-slate-6
00">
784         No batch records available yet.
785     </div>
786     )}
787
788     {!batchLoading && !batchError && batchRecords.length > 0 && (
789         <div className="overflow-x-auto">
790         <table className="min-w-full text-left text-sm">
791             <thead className="border-b border-slate-200 text-xs uppercase tracking-wide text-slate-50
0">
792                 <tr>
793                     <th className="py-2 pr-4">Request Time</th>
794                     <th className="py-2 pr-4">Count</th>
795                     <th className="py-2 pr-4">Start - End</th>
796                     <th className="py-2 pr-4">Spare</th>
797                     <th className="py-2 pr-4">Batch Note</th>
798                     <th className="py-2">Download</th>
799                 </tr>
800             </thead>
801             <tbody className="divide-y divide-slate-100">
802                 {batchRecords.map((record, index) => {
803                     const recordKey = getRecordKey(record, index);
804                     const recordLogs = getRecordLogs(record);
805                     const includeRecentLinks = recentIncludeLinks[recordKey] ?? true;
806                     const downloadableRanges = buildRangesFromLogs(recordLogs, {
807                         seriesMap: batchSeriesMap,
808                         fallbackYear: yy,
809                         fallbackBrand: brand,
810                         fallbackFormType: formType,
811                         eventId,
812                         requireSeriesMeta: true
813                     });
814                     const canDownload = downloadableRanges.length > 0;
815
816                     return (
817                         <tr key={recordKey}>
818                             <td className="py-3 pr-4 font-medium text-slate-900">
819                                 {formatRequestTime(record)}
820                             </td>
821                             <td className="py-3 pr-4 text-slate-700">
822                                 {formatCount(record.count)}
823                             </td>
824                             <td className="py-3 pr-4 text-slate-700">
825                                 {formatRanges(recordLogs, {
826                                     seriesMap: batchSeriesMap,
827                                     fallbackYear: yy,
828                                     fallbackBrand: brand
829                                 })}
830                             </td>
831                             <td className="py-3 pr-4 text-slate-700">
832                                 {formatCount(record.spare)}
833                             </td>

```

```

834         <td className="py-3 pr-4 text-slate-700">
835             {record.batchNote || "-"}
836         </td>
837         <td className="py-3">
838             <div className="flex min-w-[9rem] flex-col gap-2">
839                 <button
840                     type="button"
841                     role="switch"
842                     aria-checked={includeRecentLinks}
843                     onClick={() =>
844                         setRecentIncludeLinks((prev) => ({
845                             ...prev,
846                             [recordKey]: !(prev[recordKey] ?? true)
847                         }))
848                     }
849                 >
850                     0"
851                 <span
852                     className={`relative inline-flex h-5 w-9 items-center rounded-full transi
853                         tion ${
854                             includeRecentLinks ? "bg-[#e04420]" : "bg-gray-300"
855                         }`}
856                 >
857                     <span
858                         className={`inline-block h-4 w-4 transform rounded-full bg-white transi
859                             tion ${
860                                 includeRecentLinks ? "translate-x-4" : "translate-x-1"
861                             }`}
862                     />
863                 </span>
864                 URLs
865             </button>
866             <button
867                 type="button"
868                 onClick={() => handleDownloadRecentBatch(record, index)}
869                 disabled={!canDownload}
870                 className={`inline-flex items-center justify-center rounded-lg px-3 py-2 te
871                     xt-xs font-semibold shadow-sm ring-1 ring-inset transition ${
872                         canDownload
873                             ? "bg-white text-[#e04420] ring-[#e04420]/40 hover:bg-[#e04420] hover:t
874                             ext-white"
875                             : "cursor-not-allowed bg-slate-100 text-slate-400 ring-slate-200"
876                         }`}
877             >
878                 Download
879             </button>
880         </div>
881     </td>
882 </tr>
883 </tbody>
884 </table>
885 </div>
886
887 <form onSubmit={handleGenerate} className="grid grid-cols-1 gap-4 md:grid-cols-2">
888     {clientMeta && (
889         <div className="md:col-span-2">
890             <label className="mb-1 block text-sm font-medium text-gray-700">Client</label>
891             <input
892                 readOnly
893                 value={clientMeta.clientName || `Admin #${clientMeta.clientId}`}
894                 className="w-full rounded-xl border-0 ring-1 ring-inset ring-gray-200 bg-gray-50 px-3 py-
895                 2 text-gray-900"
896             />
897         </div>
898     )}

```

```

898     { /* Year */ }
899     <div>
900         <label className="mb-1 block text-sm font-medium text-gray-700">Year Series</label>
901         <input
902             className="w-full rounded-xl border-0 ring-1 ring-inset ring-gray-200 bg-white px-3 py-2 text-gray-900 placeholder:text-gray-400 shadow-sm focus:outline-none focus:ring-2 focus:ring-[#e04420]"
903             inputMode="numeric"
904             maxLength={2}
905             value={yy}
906             onChange={(e) => setYy(e.target.value.replace(/\D/g, "").slice(0, 2))}
907             placeholder={defaultYY}
908         />
909     </div>
910
911     { /* Quantity */ }
912     <div>
913         <label className="mb-1 block text-sm font-medium text-gray-700">Quantity</label>
914         <input
915             className="w-full rounded-xl border-0 ring-1 ring-inset ring-gray-200 bg-white px-3 py-2 text-gray-900 placeholder:text-gray-400 shadow-sm focus:outline-none focus:ring-2 focus:ring-[#e04420]"
916             inputMode="numeric"
917             value={qty}
918             onChange={(e) => setQty(e.target.value.replace(/\D/g, ""))}
919             placeholder="e.g. 45678"
920         />
921     </div>
922
923     { !clientMeta && (
924         <div className="md:col-span-2">
925             <label className="mb-1 block text-sm font-medium text-gray-700">Client</label>
926             <select
927                 className="w-full rounded-xl border-0 ring-1 ring-inset ring-gray-200 bg-white px-3 py-2 text-gray-900 shadow-sm focus:outline-none focus:ring-2 focus:ring-[#e04420]"
928                 value={brand}
929                 onChange={(e) => setBrand(e.target.value)}
930             >
931                 <option value="0">AtomX (0)</option>
932                 <option value="1">Client Branded (1)</option>
933             </select>
934         </div>
935     ) }
936
937     { /* Form Type */ }
938     <div>
939         <label className="mb-1 block text-sm font-medium text-gray-700">Form Type (f)</label>
940         <select
941             className="w-full rounded-xl border-0 ring-1 ring-inset ring-gray-200 bg-white px-3 py-2 text-gray-900 shadow-sm focus:outline-none focus:ring-2 focus:ring-[#e04420]"
942             value={formType}
943             onChange={(e) => setFormType(parseInt(e.target.value, 10))}
944         >
945             { FORM_TYPES.map((ft) => (
946                 <option key={ft.value} value={ft.value}>
947                     {ft.value} : {ft.label}
948                 </option>
949             )) }
950         </select>
951     </div>
952
953     { /* Extra Quantity */ }
954     <div>
955         <label className="mb-1 block text-sm font-medium text-gray-700">Extra Quantity</label>
956         <input
957             className="w-full rounded-xl border-0 ring-1 ring-inset ring-gray-200 bg-white px-3 py-2 text-gray-900 placeholder:text-gray-400 shadow-sm focus:outline-none focus:ring-2 focus:ring-[#e04420]"
958             inputMode="numeric"
959             value={extraQty}
960             onChange={(e) => setExtraQty(e.target.value.replace(/\D/g, ""))}
961             placeholder="Auto-calculated"
962         />

```

```

963         </div>
964
965         { /* Note */ }
966         <div className="md:col-span-2">
967             <label className="mb-1 block text-sm font-medium text-gray-700">Note</label>
968             <input
969                 className="w-full rounded-xl border-0 ring-1 ring-inset ring-gray-200 bg-white px-3 py-2 text-gray-900 placeholder:text-gray-400 shadow-sm focus:outline-none focus:ring-2 focus:ring-[#e04420]"
970                 maxLength={50}
971                 value={note}
972                 onChange={(e) => setNote(e.target.value)}
973                 placeholder="Add a short note (max 50 characters)"
974             />
975         </div>
976
977         {err && (
978             <div className="md:col-span-2">
979                 <div className="rounded-xl border border-red-200 bg-red-50 px-3 py-2 text-sm text-red-700">
980                     {err}
981                 </div>
982             </div>
983         )}
984
985         {!out && (
986             <div className="md:col-span-2 flex gap-3 pt-2">
987                 <button
988                     type="submit"
989                     className="inline-flex w-full items-center justify-center rounded-xl bg-[#e04420] px-4 py-2 text-sm font-semibold text-white shadow-sm transition hover:brightness-95 active:scale-[.99] md:w-1/2">
990                     >
991                     Generate
992                 </button>
993                 <button
994                     type="button"
995                     className="inline-flex w-full items-center justify-center rounded-xl border border-gray-300 bg-white px-4 py-2 text-sm font-semibold text-gray-700 shadow-sm transition hover:bg-gray-50 active:scale-[.99] md:w-1/2">
996                     onClick={() => router.push("/")}>
997                     >
998                     Back
999                 </button>
1000             </div>
1001         )}
1002     </form>
1003
1004     {out && (
1005         <div className="mt-8 rounded-2xl border border-green-200 bg-green-50 p-4">
1006             <p className="mb-2 text-sm font-semibold text-gray-800">Generated Card IDs:</p>
1007             <div className="space-y-4 text-sm">
1008                 {out.ranges.map((range, index) => (
1009                     <div key={range.id ?? index} className="space-y-2">
1010                         <p className="font-mono text-sm text-green-800">
1011                             From: <span className="font-bold">{range.startTag}</span>
1012                             <span className="mx-2">→</span>
1013                             To: <span className="font-bold">{range.endTag}</span>
1014                         </p>
1015                         <div className="text-gray-600">First URL:</div>
1016                         <div className="break-all font-mono text-slate-700">{range.firstUrl}</div>
1017                         <div className="mt-2 text-gray-600">Last URL:</div>
1018                         <div className="break-all font-mono text-slate-700">{range.lastUrl}</div>
1019                     </div>
1020                 ))}
1021             </div>
1022
1023             <div className="mt-4 flex flex-col gap-3 md:flex-row md:items-center md:justify-between">
1024                 <button
1025                     type="button"
1026                     onClick={handleDownloadExcel}
1027                     className="inline-flex items-center justify-center rounded-lg bg-white px-4 py-2 text-sm font-semibold text-[#e04420] shadow-sm ring-1 ring-inset ring-[#e04420]/40 transition hover:bg-[#e04420]"

```

```

    hover:text-white"
1028         >
1029         Download Excel (.xlsx)
1030     </button>
1031
1032     <div className="flex items-center gap-3 text-sm text-gray-700">
1033         <span>Include URLs in Excel</span>
1034         <button
1035             type="button"
1036             role="switch"
1037             aria-checked={includeLinks}
1038             onClick={() => setIncludeLinks((prev) => !prev)}
1039             className={`relative inline-flex h-6 w-11 items-center rounded-full transition ${include
eLinks ? "bg-[#e04420]" : "bg-gray-300"}`}
1040         >
1041             <span className="sr-only">Toggle URL column</span>
1042             <span
1043                 className={`inline-block h-5 w-5 transform rounded-full bg-white transition ${include
Links ? "translate-x-5" : "translate-x-1"}`}
1044             />
1045         </button>
1046     </div>
1047 </div>
1048
1049
1050 </div>
1051 )}
1052 </div>
1053 </div>
1054 );
1055 }
1056
1057 export default function GeneratePage() {
1058     return (
1059         <Suspense fallback={<div className="flex min-h-screen items-center justify-center bg-gradient-to-b fr
om-gray-50 to-white text-sm text-gray-500">Loading generator...</div>}>
1060             <GenerateForm />
1061         </Suspense>
1062     );
1063 }

```

FILE: apps/tag_series/app/globals.css

```
1 @import "tailwindcss";
2
3 @font-face {
4   font-family: "Poppins";
5   src: url("/shared/fonts/Poppins-Thin.ttf") format("truetype");
6   font-display: swap;
7   font-weight: 100;
8   font-style: normal;
9 }
10
11 @font-face {
12   font-family: "Poppins";
13   src: url("/shared/fonts/Poppins-Light.ttf") format("truetype");
14   font-display: swap;
15   font-weight: 300;
16   font-style: normal;
17 }
18
19 @font-face {
20   font-family: "Poppins";
21   src: url("/shared/fonts/Poppins-Regular.ttf") format("truetype");
22   font-display: swap;
23   font-weight: 400;
24   font-style: normal;
25 }
26
27 @font-face {
28   font-family: "Poppins";
29   src: url("/shared/fonts/Poppins-MediumItalic.ttf") format("truetype");
30   font-display: swap;
31   font-weight: 500;
32   font-style: italic;
33 }
34
35 @font-face {
36   font-family: "Poppins";
37   src: url("/shared/fonts/Poppins-SemiBold.ttf") format("truetype");
38   font-display: swap;
39   font-weight: 600;
40   font-style: normal;
41 }
42
43 @font-face {
44   font-family: "Poppins";
45   src: url("/shared/fonts/Poppins-Bold.ttf") format("truetype");
46   font-display: swap;
47   font-weight: 700;
48   font-style: normal;
49 }
50
51 :root {
52   --background: #030712;
53   --foreground: #f8fafc;
54   --accent: #0ea5e9;
55   --font-base: "Poppins", "Inter", "Segoe UI", system-ui, -apple-system,
56     BlinkMacSystemFont, "Helvetica Neue", Arial, sans-serif;
57 }
58
59 @theme inline {
60   --color-background: var(--background);
61   --color-foreground: var(--foreground);
62   --font-sans: var(--font-base);
63 }
64
65 body {
66   background: radial-gradient(circle at 10% 20%, #0f172a, #020617 65%);
67   color: var(--foreground);
68   font-family: var(--font-base);
69 }
```


FILE: apps/tag_series/app/layout.js

```
1 import { Suspense } from "react";
2 import "../globals.css";
3 import { TokenGate } from "@atomx/global-components";
4 import { PostHogInit } from "../components/PostHogInit";
5
6 export const metadata = {
7   title: "Tag Series | AtomX Platform",
8   description: "IoT analytics and command center for connected tags."
9 };
10
11 export default function RootLayout({ children }) {
12   return (
13     <html lang="en">
14       <body className="antialiased">
15         <PostHogInit />
16         <Suspense fallback=<div className="flex min-h-screen items-center justify-center bg-[#f6f7fb] text-sm text-slate-500">Checking access...</div>>
17           <TokenGate appId="tag-series" publicRoutes={["/login"]}>
18             {children}
19           </TokenGate>
20         </Suspense>
21       </body>
22     </html>
23   );
24 }
```

FILE: apps/tag_series/app/page.js

```

1  "use client";
2
3  import { useRouter } from "next/navigation";
4  import { useEffect, useRef, useState } from "react";
5  import { fetchCardClients, fetchSeriesMeta, fetchTagSeriesEvents } from "../api/api";
6  import { getStepOneState, saveStepOneState } from "../lib/setupStorage";
7  import AddFormFactorPage from "../Admin/AddFormFactor/page";
8  import AddProductPage from "../Admin/AddProduct/page";
9  import ViewPage from "../Admin/View/page";
10 import {
11   capturePostHogEvent,
12   identifyPostHogUser
13 } from "@atomx/global-components";
14
15 function getTagSeriesAuthToken() {
16   if (typeof window === "undefined") return null;
17   const canonical = window.localStorage.getItem("atomx.auth.tag-series");
18   if (canonical) return canonical;
19
20   const legacy = window.localStorage.getItem("atomx.auth.tag_series");
21   if (legacy) {
22     // Migrate old key to canonical key to keep all apps in sync.
23     window.localStorage.setItem("atomx.auth.tag-series", legacy);
24     return legacy;
25   }
26   return null;
27 }
28
29 export default function EventIdPage() {
30   const router = useRouter();
31   const [eventId, setEventId] = useState("");
32   const [error, setError] = useState("");
33   const [menuOpen, setMenuOpen] = useState(false);
34   const [cardClientsError, setCardClientsError] = useState(null);
35   const [yearSeries, setYearSeries] = useState(() => new Date().getFullYear().toString().slice(-2));
36   const [clients, setClients] = useState([]);
37   const [clientsLoading, setClientsLoading] = useState(false);
38   const [selectedClient, setSelectedClient] = useState("");
39   const [events, setEvents] = useState([]);
40   const [eventsLoading, setEventsLoading] = useState(false);
41   const [eventsError, setEventsError] = useState(null);
42   const [eventDropdownOpen, setEventDropdownOpen] = useState(false);
43   const [submitError, setSubmitError] = useState(null);
44   const [submitting, setSubmitting] = useState(false);
45
46   const menuRef = useRef(null);
47   const eventDropdownRef = useRef(null);
48   const savedStateRef = useRef(null);
49
50   const adminQuickLinks = [
51     { label: "View", href: "/Admin/View", component: ViewPage },
52     { label: "Add Formfactor", href: "/Admin/AddFormFactor", component: AddFormFactorPage },
53     { label: "Add Product", href: "/Admin/AddProduct", component: AddProductPage }
54   ];
55
56   useEffect(() => {
57     if (!menuOpen) return undefined;
58
59     const handleClickOutside = (event) => {
60       if (menuRef.current && !menuRef.current.contains(event.target)) {
61         setMenuOpen(false);
62       }
63     };
64
65     document.addEventListener("mousedown", handleClickOutside);
66     return () => document.removeEventListener("mousedown", handleClickOutside);
67   }, [menuOpen]);
68
69   useEffect(() => {

```

```

70   if (!eventDropdownOpen) return undefined;
71
72   const handleClickOutside = (event) => {
73     if (eventDropdownRef.current && !eventDropdownRef.current.contains(event.target)) {
74       setEventDropdownOpen(false);
75     }
76   };
77
78   document.addEventListener("mousedown", handleClickOutside);
79   return () => document.removeEventListener("mousedown", handleClickOutside);
80 }, [eventDropdownOpen]);
81
82 useEffect(() => {
83   // Prefill form from the last completed Step 1 (saved in sessionStorage)
84   const saved = getStepOneState();
85   savedStateRef.current = saved;
86   if (saved) {
87     setEventId(saved.eventId ? String(saved.eventId) : "");
88     if (saved.yearSeries) {
89       setYearSeries(String(saved.yearSeries).slice(0, 2));
90     }
91     if (saved.clientId) {
92       setSelectedClient(String(saved.clientId));
93     }
94   }
95 }, []);
96
97 useEffect(() => {
98   let cancelled = false;
99
100  async function loadEvents() {
101    if (typeof window === "undefined") return;
102    setEventsLoading(true);
103    try {
104      const token = getTagSeriesAuthToken();
105      const response = await fetchTagSeriesEvents(token);
106      const raw = response?.events ?? response?.data ?? response?.result ?? response;
107      const list = Array.isArray(raw) ? raw : [];
108      const mapped = list.map((item) => {
109        if (typeof item === "string" || typeof item === "number") {
110          return { id: String(item), name: "" };
111        }
112        const id =
113          item?.eventId ??
114          item?.event_id ??
115          item?.id ??
116          item?.eventID ??
117          item?.eventCode ??
118          "";
119        const name =
120          item?.eventName ??
121          item?.name ??
122          item?.title ??
123          item?.event ??
124          item?.eventTitle ??
125          "";
126        return { id: String(id), name: String(name || "") };
127      }).filter((item) => item.id);
128
129      if (!cancelled) {
130        setEvents(mapped);
131        const preferredId = savedStateRef.current?.eventId?.toString();
132        const hasPreferred = preferredId
133          ? mapped.some((event) => event.id === preferredId)
134          : false;
135        setEventId((prev) =>
136          prev || (hasPreferred ? preferredId : savedStateRef.current?.eventId?.toString()) ?? ""
137        );
138        setEventsError(null);
139      }
140    }
141  }

```

```

140     } catch (err) {
141       if (!cancelled) {
142         console.error("Failed to fetch events", err);
143         setEvents([]);
144         setEventsError("Unable to load events. Try again later.");
145       }
146     } finally {
147       if (!cancelled) {
148         setEventsLoading(false);
149       }
150     }
151   }
152
153   loadEvents();
154
155   return () => {
156     cancelled = true;
157   };
158 }, []);
159
160 useEffect(() => {
161   let cancelled = false;
162
163   async function loadCardClients() {
164     if (typeof window === "undefined") return;
165     setClientsLoading(true);
166     try {
167       const token = getTagSeriesAuthToken();
168       const response = await fetchCardClients(token);
169       if (!cancelled) {
170         const fetchedClients = response?.cardClients ?? [];
171         const preferredId = savedStateRef.current?.clientId?.toString();
172         const hasPreferred = preferredId
173           ? fetchedClients.some((client) => client.id?.toString() === preferredId)
174           : false;
175         setClients(fetchedClients);
176         setSelectedClient(
177           hasPreferred
178             ? preferredId
179             : fetchedClients[0]?.id?.toString() ?? ""
180         );
181         setCardClientsError(null);
182       }
183     } catch (err) {
184       if (!cancelled) {
185         console.error("Failed to fetch CardClients", err);
186         setClients([]);
187         setSelectedClient("");
188         setCardClientsError("Unable to sync Card Clients. Try again later.");
189       }
190     } finally {
191       if (!cancelled) {
192         setClientsLoading(false);
193       }
194     }
195   }
196
197   loadCardClients();
198
199   return () => {
200     cancelled = true;
201   };
202 }, []);
203
204 const selectedClientMeta = clients.find(
205   (client) => client.id?.toString() === selectedClient
206 );
207
208 const onSubmit = async (e) => {
209   e.preventDefault();

```

```
210     setSubmitError(null);
211
212     if (!eventId) {
213         setError("Please select an event.");
214         return;
215     }
216
217     if (!/^\\d+$/\\.test(eventId)) {
218         setError("Event ID must be digits only.");
219         return;
220     }
221
222     if (!selectedClient) {
223         setSubmitError("Please select a client.");
224         return;
225     }
226
227     try {
228         setSubmitting(true);
229         const token = getTagSeriesAuthToken();
230         const seriesPayload = await fetchSeriesMeta(token, {
231             eventId,
232             adminId: selectedClient,
233             yearSeries
234         });
235
236         const lastSeries = Number(seriesPayload?.tagSeries_last ?? 0);
237         const nextSeries = String((Number.isNaN(lastSeries) ? 0 : lastSeries) + 1).padStart(2, "0");
238
239         saveStepOneState({
240             eventId,
241             yearSeries,
242             clientId: selectedClient,
243             clientName: selectedClientMeta?.name ?? "",
244             clientSeries: selectedClientMeta?.series ?? null,
245             eventSeries: nextSeries,
246             timestamp: Date.now()
247         });
248
249         identifyPostHogUser(selectedClient, {
250             app: "tag_series",
251             client_name: selectedClientMeta?.name ?? ""
252         });
253         capturePostHogEvent("operation_event_setup_submitted", {
254             app: "tag_series",
255             event_id: eventId,
256             year_series: yearSeries,
257             client_id: selectedClient,
258             client_name: selectedClientMeta?.name ?? "",
259             next_series: nextSeries
260         });
261
262         router.push(`/generate?c=${eventId}`);
263     } catch (err) {
264         console.error("Failed to validate series", err);
265         capturePostHogEvent("$exception", {
266             app: "tag_series",
267             $exception_message: err.message,
268             $exception_source: "event_setup_submitted"
269         });
270         setSubmitError("Unable to validate this event. Please try again.");
271     } finally {
272         setSubmitting(false);
273     }
274 };
275
276 const isContinueDisabled = !eventId || !selectedClient || submitting;
277 const filteredEvents = events.filter((event) => {
278     if (!eventId) return true;
279     const query = eventId.toLowerCase();
```

```

280     return event.id.toLowerCase().includes(query) || event.name.toLowerCase().includes(query);
281   });
282
283   return (
284     <div className="relative min-h-[100dvh] w-full bg-gradient-to-br from-[#fef3ec] via-white to-[#eff8ff]
285 ] px-4 py-16">
286     <div className="pointer-events-none absolute inset-0 bg-[radial-gradient(circle_at_top,_rgba(224,68
287 ,32,0.08),_transparent_45%)]" />
288     <div ref={menuRef} className="fixed right-6 top-6 z-20">
289       <div className="relative">
290         <button
291           type="button"
292           onClick={() => setMenuOpen((prev) => !prev)}
293           className="flex h-12 w-12 items-center justify-center rounded-full bg-[#e04420] text-sm font-
294 semibold uppercase text-white shadow-lg shadow-[#e04420]/30 transition hover:brightness-95"
295           aria-haspopup="menu"
296           aria-expanded={menuOpen}
297         >
298           OM
299           <span className="sr-only">Open user menu</span>
300         </button>
301
302         {menuOpen && (
303           <div
304             role="menu"
305             className="absolute right-0 z-10 mt-3 w-60 overflow-hidden rounded-2xl border border-gray-2
306 00 bg-white shadow-xl shadow-gray-200/70"
307           >
308             <div className="border-b border-gray-100 bg-gray-50 px-4 py-3">
309               <p className="text-sm font-semibold text-gray-900">omkar@atomx.in</p>
310               <p className="mt-1 text-xs text-gray-500">Role: Admin</p>
311             </div>
312             <nav className="py-2" aria-label="Quick links">
313               {adminQuickLinks.map(({ label, href, component: Component }) => (
314                 <button
315                   key={label}
316                   type="button"
317                   onClick={() => {
318                     setMenuOpen(false);
319                     router.push(href);
320                   }}
321                   className="flex w-full items-center justify-start px-4 py-2 text-sm font-medium text-
322 gray-700 transition hover:bg-gray-100"
323                   role="menuitem"
324                   aria-label={`Go to ${label} (${Component.name})`}
325                 >
326                   {label}
327                 </button>
328               ))}
329             </nav>
330           </div>
331         )}
332       </div>
333     </div>
334
335     <div className="relative mx-auto w-full max-w-xl rounded-[32px] border border-white/60 bg-white/80
336 p-8 text-gray-900 shadow-[0_35px_80px_rgba(15,23,42,0.12)] backdrop-blur">
337       <div className="text-center">
338         <p className="text-xs font-semibold uppercase tracking-[0.45em] text-[#e04420]">AtomX Platform<
339 /p>
340         <h1 className="mt-2 text-3xl font-semibold text-slate-900">Tag Generator</h1>
341         <p className="mt-1 text-sm text-slate-500">Step 1 of 2 • Enter event inputs</p>
342       </div>
343
344       {cardClientsError && (
345         <div className="mt-6 rounded-2xl border border-amber-200 bg-amber-50 px-4 py-2 text-sm text-amb
346 er-900">
347           {cardClientsError}
348         </div>
349       )}
350     </div>

```

```

342
343     <form onSubmit={onSubmit} className="mt-6 space-y-5">
344       <div ref={eventDropdownRef}>
345         <label className="mb-2 block text-sm font-semibold text-slate-600">Event</label>
346         <div className="relative">
347           <input
348             className="w-full rounded-2xl border border-slate-200 bg-white/70 px-4 py-3 text-base font-medium text-slate-900 shadow-inner shadow-slate-100 outline-none ring-0 placeholder:text-slate-300 focus:ring-2 focus:ring-[#e04420]/30"
349             inputMode="numeric"
350             pattern="\d*"
351             placeholder="Select event"
352             value={eventId}
353             onFocus={() => setEventDropdownOpen(true)}
354             onChange={(e) => {
355               setError("");
356               setEventId(e.target.value.replace(/\D/g, ""));
357               setEventDropdownOpen(true);
358             }}
359           />
360           <button
361             type="button"
362             onClick={() => setEventDropdownOpen((prev) => !prev)}
363             className="absolute inset-y-0 right-3 flex items-center text-slate-400"
364             aria-label="Toggle event dropdown"
365           >
366             {eventDropdownOpen ? "▲" : "▼"}
367           </button>
368
369           {eventDropdownOpen && (
370             <div className="absolute z-20 mt-2 w-full max-h-64 overflow-auto rounded-2xl border border-slate-200 bg-white shadow-lg">
371               {eventsLoading && (
372                 <div className="px-4 py-3 text-sm text-slate-500">Loading events...</div>
373               )}
374               {!eventsLoading && !filteredEvents.length && (
375                 <div className="px-4 py-3 text-sm text-slate-500">No matching events.</div>
376               )}
377               {!eventsLoading &&
378                 filteredEvents.map((event) => (
379                   <button
380                     key={event.id}
381                     type="button"
382                     onClick={() => {
383                       setEventId(event.id);
384                       setEventDropdownOpen(false);
385                     }}
386                     className="flex w-full items-center justify-between px-4 py-2 text-left text-sm text-slate-700 hover:bg-slate-50"
387                   >
388                     <span className="font-semibold">{event.id}</span>
389                     <span className="text-slate-500">{event.name || ""}</span>
390                   </button>
391                 ))}
392             </div>
393           )}
394         </div>
395         {error && <p className="mt-1 text-sm text-red-600">{error}</p>}
396         {eventsError && <p className="mt-1 text-sm text-red-600">{eventsError}</p>}
397       </div>
398
399       <div className="grid gap-4 md:grid-cols-2">
400         <div>
401           <label className="mb-2 block text-sm font-semibold text-slate-600">Year Series</label>
402           <input
403             value={yearSeries}
404             onChange={(e) => setYearSeries(e.target.value.replace(/\D/g, "").slice(0, 2))}
405             inputMode="numeric"
406             maxLength={2}
407             className="w-full rounded-2xl border border-slate-200 bg-white/70 px-4 py-3 text-base font

```

```

    t-semibold text-slate-700 shadow-inner shadow-slate-100 outline-none focus:border-[#e04420] focus:ring-2
    focus:ring-[#e04420]/30"
408      />
409    </div>
410    <div>
411      <label className="mb-2 block text-sm font-semibold text-slate-600">Client</label>
412      <div className="relative">
413        <select
414          className="w-full appearance-none rounded-2xl border border-slate-200 bg-white/80 px-4
py-3 text-base font-medium text-slate-900 shadow-inner outline-none focus:border-[#1495ab] focus:ring-2 f
ocus:ring-[#1495ab]/30"
415          value={selectedClient}
416          onChange={(e) => setSelectedClient(e.target.value)}
417          disabled={clientsLoading || !clients.length}
418        >
419          {clientsLoading && <option>Loading clients...</option>}
420          {!clientsLoading && !clients.length && <option>No clients available</option>}
421          {!clientsLoading &&
            clients.map((client) => (
422              <option key={client.id} value={client.id}>
423                {client.name}
424              </option>
425            ))}
426          </select>
427          <span className="pointer-events-none absolute inset-y-0 right-4 flex items-center text-sl
ate-400">
428            ▼
429          </span>
430        </div>
431      {selectedClientMeta && (
432        <p className="mt-1 text-xs text-slate-500">
433          Series ID: {selectedClientMeta.series ?? "-"}
434        </p>
435      )}
436    </div>
437  </div>
438 </div>
439
440  <button
441    className={`w-full rounded-2xl px-4 py-3 text-base font-semibold text-white transition ${
442      isContinueDisabled
443        ? "cursor-not-allowed bg-slate-200 text-slate-400"
444        : "bg-[#e04420] shadow-lg shadow-[#e04420]/30 hover:brightness-95"
445    }`}
446    type="submit"
447    disabled={isContinueDisabled}
448  >
449    {submitting ? "Validating..." : "Continue"}
450  </button>
451
452  {submitError && (
453    <p className="text-center text-sm font-medium text-red-600">{submitError}</p>
454  )}
455 </form>
456 </div>
457 </div>
458 );
459 }

```


FILE: apps/tag_series/components/AnalyticsList.js

```
1 export function AnalyticsList({ metrics }) {
2   return (
3     <dl className="grid gap-4 sm:grid-cols-2">
4       {metrics.map((metric) => (
5         <div
6           key={metric.label}
7           className="rounded-2xl border border-black/10 bg-white/70 p-4 shadow-sm dark:border-white/10 dark:bg-white/10"
8         >
9           <dt className="text-sm text-black/60 dark:text-white/60">
10             {metric.label}
11           </dt>
12           <dd className="text-3xl font-semibold text-black dark:text-white">
13             {metric.value}
14           </dd>
15         </div>
16       )]}
17     </dl>
18   );
19 }
```

FILE: apps/tag_series/components/PostHogInit.js

```
1 "use client";
2
3 import { useEffect } from "react";
4 import { initPostHog } from "@atomx/global-components";
5
6 export function PostHogInit() {
7   useEffect(() => {
8     initPostHog("tag_series");
9   }, []);
10
11   return null;
12 }
```

FILE: apps/tag_series/lib/devices.js

```
1 import { createApiClient, apiRoutes } from "@atomx/api-client";
2
3 const client = createApiClient({
4   baseUrl: apiRoutes.tag_series.baseUrl,
5   project: "tag_series"
6 });
7
8 export async function getDeviceSummary() {
9   return {
10     active: 1260,
11     offline: 24,
12     batteriesLow: 18
13   };
14 }
15
16 export async function getRecentEvents() {
17   return [
18     { id: "evt-204", message: "Tag #2245 left safe zone", severity: "alert" },
19     { id: "evt-205", message: "Batch 91 synced, 420 records", severity: "info" },
20     { id: "evt-206", message: "Firmware 4.12 rolled out", severity: "success" }
21   ];
22 }
23
24 export async function pingHealth() {
25   try {
26     await client.health();
27     return { ok: true };
28   } catch (error) {
29     return { ok: false, message: error.message };
30   }
31 }
```

FILE: apps/tag_series/lib/setupStorage.js

```
1 export const STEP_ONE_STATE_KEY = "atomx.tag_series.step1";
2
3 export function saveStepOneState(state) {
4   if (typeof window === "undefined") return;
5   window.sessionStorage.setItem(STEP_ONE_STATE_KEY, JSON.stringify(state));
6 }
7
8 export function getStepOneState() {
9   if (typeof window === "undefined") return null;
10  const raw = window.sessionStorage.getItem(STEP_ONE_STATE_KEY);
11  if (!raw) return null;
12  try {
13    return JSON.parse(raw);
14  } catch (error) {
15    console.error("Failed to parse step one state", error);
16    return null;
17  }
18 }
19
20 export function clearStepOneState() {
21   if (typeof window === "undefined") return;
22   window.sessionStorage.removeItem(STEP_ONE_STATE_KEY);
23 }
```

FILE: apps/tag_series/next.config.mjs

```
1 import path from "node:path";
2 import dotenv from "dotenv";
3
4 dotenv.config({ path: path.resolve(process.cwd(), "..", "..", ".env") });
5
6 /** @type {import('next').NextConfig} */
7 const config = {
8   reactCompiler: true,
9   output: "export",
10  basePath: "/tag_series",
11  assetPrefix: "/tag_series",
12  images: {
13    unoptimized: true
14  },
15  trailingSlash: true,
16  transpilePackages: [
17    "@atomx/shared-ui",
18    "@atomx/global-components",
19    "@atomx/auth",
20    "@atomx/api-client",
21    "@atomx/lib",
22    "@atomx/utils"
23  ],
24  experimental: {
25    externalDir: true
26  }
27 };
28
29 export default config;
```

FILE: apps/tag_series/package.json

```
1 {
2   "name": "@atomx/tag_series",
3   "private": true,
4   "scripts": {
5     "dev": "next dev -p 3002",
6     "build": "next build",
7     "start": "next start"
8   },
9   "dependencies": {
10    "@atomx/api-client": "0.1.0",
11    "@atomx/auth": "0.1.0",
12    "@atomx/global-components": "0.1.0",
13    "@atomx/lib": "0.1.0",
14    "@atomx/shared-ui": "0.1.0",
15    "@atomx/utils": "0.1.0",
16    "next": "16.2.6",
17    "react": "19.2.0",
18    "react-dom": "19.2.0",
19    "xlsx": "^0.18.5"
20  },
21  "devDependencies": {
22    "@tailwindcss/postcss": "^4",
23    "tailwindcss": "^4"
24  }
25 }
```

FILE: apps/tag_series/postcss.config.mjs

```
1 const config = {  
2   plugins: {  
3     "@tailwindcss/postcss": {}  
4   }  
5 };  
6  
7 export default config;
```

FILE: apps/tag_series/utils/thresholds.js

```
1 export const energyThreshold = 40;
2
3 export function getHealthState(percentage) {
4   if (percentage > 80) return "healthy";
5   if (percentage > 50) return "observe";
6   if (percentage > 20) return "warning";
7   return "critical";
8 }
```


FILE: packages/api/package.json

```
1 {  
2   "name": "@atomx/api-client",  
3   "version": "0.1.0",  
4   "private": true,  
5   "type": "module",  
6   "main": "src/index.js"  
7 }
```

FILE: packages/api/src/createApiClient.js

```
1 export function createApiClient({ baseUrl, project }) {
2   const endpoint = (path) => `${baseUrl.replace(/\$/ , "").}/${path.replace(/^\//, "")}`;
3   const shouldMock =
4     process.env.MOCK_API !== "false" && baseUrl.includes("example.com");
5
6   async function request(path, options = {}) {
7     if (shouldMock) {
8       return Promise.resolve({
9         mocked: true,
10        project,
11        path,
12        timestamp: new Date().toISOString()
13      });
14    }
15
16    const url = endpoint(path);
17    const result = await fetch(url, {
18      ...options,
19      headers: {
20        "Content-Type": "application/json",
21        "x-atomx-project": project,
22        ...(options.headers ?? {})
23      }
24    });
25
26    if (!result.ok) {
27      const text = await result.text();
28      throw new Error(`Request failed (${result.status}): ${text}`);
29    }
30
31    if (result.status === 204) return null;
32    return result.json();
33  }
34
35  return {
36    get(path) {
37      return request(path, { method: "GET" });
38    },
39    post(path, body) {
40      return request(path, { method: "POST", body: JSON.stringify(body) });
41    },
42    health() {
43      return request("health");
44    }
45  };
46 }
```

FILE: packages/api/src/index.js

```
1 export * from "./createApiClient.js";  
2 export * from "./routes.js";
```

FILE: packages/api/src/routes.js

```
1 export const apiRoutes = {
2   livelink: {
3     baseUrl: "https://api.example.com/livelink"
4   },
5   tag_series: {
6     baseUrl: "https://api.example.com/tag-series"
7   }
8 };
```

FILE: packages/auth/package.json

```
1 {  
2   "name": "@atomx/auth",  
3   "version": "0.1.0",  
4   "private": true,  
5   "type": "module",  
6   "main": "src/index.js"  
7 }
```

FILE: packages/auth/src/config.js

```
1 export const authMatrix = {
2   livelink: {
3     audience: "livelink-users",
4     loginRoute: "/login",
5     cookieName: "livelink-token"
6   },
7   tag_series: {
8     audience: "tag-series-admin",
9     loginRoute: "/login",
10    cookieName: "tag-series-token"
11  }
12 };
13
14 export function getAuthConfig(app) {
15   return authMatrix[app] ?? {
16     audience: "public",
17     loginRoute: "/login",
18     cookieName: "atomx-token"
19   };
20 }
```

FILE: packages/auth/src/createAuthClient.js

```
1  const memoryStore = new Map();
2
3  function setSession(key, value) {
4    memoryStore.set(key, {
5      value,
6      updatedAt: Date.now()
7    });
8  }
9
10 function getSession(key) {
11   return memoryStore.get(key);
12 }
13
14 export function createAuthClient({ audience, project }) {
15   const namespace = `${project}:${audience}`;
16
17   return {
18     async login({ email }) {
19       const token = `${namespace}:${email}:${Date.now()}`;
20       setSession(namespace, { email, token });
21       return { token, email };
22     },
23     async logout() {
24       memoryStore.delete(namespace);
25     },
26     getSession() {
27       return getSession(namespace);
28     }
29   };
30 }
```

FILE: packages/auth/src/index.js

```
1 export * from "./createAuthClient.js";  
2 export * from "./config.js";
```


FILE: packages/global-components/loaders/AtomXLoader.js

```
1 "use client";
2
3 export function AtomXLoader({
4   label = "Loading...",
5   size = 72,
6   fullScreen = false,
7   className = ""
8 }) {
9   const wrapperClasses = fullScreen
10     ? "flex min-h-screen w-full items-center justify-center"
11     : "flex w-full items-center justify-center py-6";
12
13   return (
14     <div className={` ${wrapperClasses} ${className}`.trim()}>
15       <div className="flex flex-col items-center gap-3">
16         
22         {label ? (
23           <p className="m-0 text-sm font-medium text-slate-600">{label}</p>
24         ) : null}
25       </div>
26     </div>
27   );
28 }
```

FILE: packages/global-components/loaders/index.js

```
1 export * from "./AtomXLoader.js";
```

FILE: packages/global-components/package.json

```
1 {
2   "name": "@atomx/global-components",
3   "version": "0.1.0",
4   "private": true,
5   "type": "module",
6   "main": "src/index.js",
7   "dependencies": {
8     "@atomx/lib": "0.1.0",
9     "@atomx/public-assets": "0.1.0",
10    "@atomx/shared-ui": "0.1.0",
11    "posthog-js": "^1.363.0"
12  }
13 }
```

FILE: packages/global-components/src/AppShell.js

```
1 import { SharedCard } from "@atomx/shared-ui";
2
3 export function AppShell({ title, description, hero, children }) {
4   return (
5     <div className="min-h-screen bg-zinc-50 font-sans text-black dark:bg-black dark:text-white">
6       <div className="mx-auto flex max-w-6xl flex-col gap-10 px-6 py-12">
7         <header className="flex flex-col gap-4 rounded-3xl border border-black/5 bg-white/90 p-8 shadow-l
8 g backdrop-blur dark:border-white/10 dark:bg-white/5">
9           <div className="flex flex-col gap-3 md:flex-row md:items-center md:justify-between">
10             <div>
11               <p className="text-sm uppercase tracking-[0.4em] text-black/40 dark:text-white/60">
12                 {hero?.eyebrow ?? "ATOMX PLATFORM"}
13               </p>
14               <h1 className="text-4xl font-semibold">{title}</h1>
15               <p className="mt-2 text-base text-black/70 dark:text-white/70">
16                 {description}
17               </p>
18             </div>
19             {hero?.cta}
20           </div>
21           {hero?.meta && (
22             <div className="grid gap-4 sm:grid-cols-2 lg:grid-cols-3">
23               {hero.meta.map((item) => (
24                 <SharedCard key={item.label} title={item.value} subtitle={item.label}>
25                   {item.description}
26                 </SharedCard>
27               ))}
28             </div>
29           )}
30         </header>
31         <main className="grid gap-6 lg:grid-cols-[2fr,1fr]">{children}</main>
32       </div>
33     </div>
34   );
35 }
```

FILE: packages/global-components/src/AuthPanel.js

```
1 import Link from "next/link";
2 import { SharedButton, SharedCard } from "@atomx/shared-ui";
3
4 export function AuthPanel({ title, description, actionLabel, actionHref = "/auth" }) {
5   return (
6     <SharedCard title={title} subtitle="Authentication">
7       <p className="mb-4 text-sm text-black/70 dark:text-white/70">
8         {description}
9       </p>
10      <Link href={actionHref}>
11        <SharedButton className="w-full">{actionLabel}</SharedButton>
12      </Link>
13    </SharedCard>
14  );
15 }
```

FILE: packages/global-components/src/GlobalFooter.js

```
1 import Link from "next/link";
2
3 export function GlobalFooter() {
4   return (
5     <footer className="rounded-3xl border border-black/5 bg-white/80 p-6 text-sm text-black/70 shadow-sm
6       dark:border-white/10 dark:bg-white/5 dark:text-white/70">
7       <div className="flex flex-col gap-2 sm:flex-row sm:items-center sm:justify-between">
8         <p>© {new Date().getFullYear()} AtomX Platform</p>
9         <nav className="flex gap-4 text-black/60 dark:text-white/60">
10          <Link href="/legal/privacy">Privacy</Link>
11          <Link href="/legal/terms">Terms</Link>
12          <Link href="mailto:support@atomx.example">Support</Link>
13        </nav>
14      </div>
15    </footer>
16  );
17 }
```

FILE: packages/global-components/src/TokenGate.js

```
1 "use client";
2
3 import { useEffect, useMemo, useState } from "react";
4 import { usePathname, useSearchParams } from "next/navigation";
5
6 const DEFAULT_FALLBACK = (
7   <div className="flex min-h-screen items-center justify-center bg-[#f6f7fb] text-sm text-slate-500">
8     Checking your access...
9   </div>
10 );
11
12 export function TokenGate({
13   appId,
14   children,
15   publicRoutes = ["/login"],
16   fallback = DEFAULT_FALLBACK
17 }) {
18   const pathname = usePathname();
19   const searchParams = useSearchParams();
20   const [ready, setReady] = useState(false);
21
22   const isPublicRoute = useMemo(() => {
23     if (!pathname) return false;
24     return publicRoutes.some((route) => pathname.startsWith(route));
25   }, [pathname, publicRoutes]);
26
27   useEffect(() => {
28     if (!appId || isPublicRoute) {
29       setReady(true);
30       return;
31     }
32     setReady(true);
33     if (searchParams.get("token") && typeof window !== "undefined") {
34       const url = new URL(window.location.href);
35       url.searchParams.delete("token");
36       // Avoid router basePath re-prefix issues by rewriting URL in place.
37       window.history.replaceState(
38         window.history.state,
39         "",
40         `${url.pathname}${url.search}${url.hash}`
41       );
42     }
43   }, [appId, isPublicRoute, searchParams]);
44
45   if (isPublicRoute || ready) {
46     return children;
47   }
48
49   return fallback;
50 }
```

FILE: packages/global-components/src/UniversalLoginPage.js

```

1  "use client";
2
3  import { useCallback, useEffect, useMemo, useState } from "react";
4  import { useRouter, useSearchParams } from "next/navigation";
5  import { decodeJwt } from "@atomx/lib";
6
7  const DEFAULT_AUTH_URL = `${
8    (process.env.NEXT_PUBLIC_BASE_URL ?? "https://dapi.atomx.in").replace(/\/$/, "")
9  }/auth/google/start`;
10
11 function buildAuthUrl(authUrl, appId, redirect) {
12   try {
13     const url = new URL(authUrl);
14     if (redirect) url.searchParams.set("redirect", redirect);
15     if (appId) url.searchParams.set("app", appId);
16     return url.toString();
17   } catch {
18     return authUrl;
19   }
20 }
21
22 export function UniversalLoginPage({
23   appId = "atomx",
24   redirectPath = "/",
25   title = "Sign in to AtomX Portal",
26   description = "Access your event dashboard, manage cashless payments, and view real-time analytics.",
27   authUrl = DEFAULT_AUTH_URL
28 }) {
29   const router = useRouter();
30   const searchParams = useSearchParams();
31   const [status, setStatus] = useState("idle");
32   const [error, setError] = useState(null);
33   const [profile, setProfile] = useState(null);
34   const [loginUrl, setLoginUrl] = useState(authUrl);
35   const [devToken, setDevToken] = useState(null);
36
37   useEffect(() => {
38     if (typeof window === "undefined") return;
39
40     let redirectTarget = redirectPath || "/";
41     if (!/^https?:\/\/\./i.test(redirectTarget)) {
42       try {
43         redirectTarget = new URL(redirectTarget, window.location.origin).toString();
44       } catch {
45         redirectTarget = window.location.href;
46       }
47     }
48
49     const nextUrl = buildAuthUrl(authUrl, appId, redirectTarget);
50     setLoginUrl(nextUrl);
51   }, [appId, authUrl, redirectPath]);
52
53   const processToken = useCallback(
54     (token, delay = 1200) => {
55       if (!token) return;
56       try {
57         const decoded = decodeJwt(token);
58         setProfile(decoded);
59         setStatus("success");
60         if (typeof window !== "undefined") {
61           window.localStorage.setItem(`atomx.auth.${appId}`, token);
62           if (appId === "portal") {
63             window.localStorage.setItem("atomx.portal.token", token);
64           }
65
66           const url = new URL(window.location.href);
67           url.searchParams.delete("token");
68           window.history.replaceState(
69             window.history.state,

```



```
70         "",
71         `${url.pathname}${url.search}${url.hash}`
72     );
73
74     window.setTimeout(() => {
75         if (/^https?:\/\//i.test(redirectPath)) {
76             window.location.assign(redirectPath);
77         } else {
78             router.push(redirectPath || "/");
79         }
80     }, delay);
81 }
82 } catch (err) {
83     console.error(err);
84     setError("We could not verify your login token. Please try again.");
85     setStatus("error");
86 }
87 },
88 [appId, redirectPath, router]
89 );
90
91 const handleGoogleSignIn = useCallback(
92     (event) => {
93         event.preventDefault();
94         if (status === "loading" || status === "success") return;
95         setError(null);
96         setStatus("loading");
97         if (typeof window !== "undefined") {
98             window.setTimeout(() => {
99                 window.location.assign(loginUrl);
100             }, 120);
101         }
102     },
103     [loginUrl, status]
104 );
105
106 useEffect(() => {
107     if (
108         typeof window === "undefined" ||
109         process.env.NODE_ENV !== "development" ||
110         searchParams.get("token") ||
111         status !== "idle"
112     ) {
113         return;
114     }
115
116     setDevToken(process.env.NEXT_PUBLIC_DEV_PORTAL_TOKEN || null);
117 }, [searchParams, status]);
118
119 useEffect(() => {
120     const token = searchParams.get("token");
121     if (!token) return;
122     processToken(token);
123 }, [searchParams, processToken]);
124
125 const showDevButton =
126     process.env.NODE_ENV === "development" &&
127     process.env.NEXT_PUBLIC_DEV_TOKEN_BUTTON === "true" &&
128     devToken &&
129     status === "idle" &&
130     !profile;
131
132 const handleDevTokenLogin = () => {
133     if (!showDevButton) return;
134     processToken(devToken, 600);
135 };
136
137 const statusLabel = useMemo(() => {
138     switch (status) {
139         case "success":
```

```

140     return "Authenticated. Redirecting...";
141     case "loading":
142         return "Verifying...";
143     case "error":
144         return error;
145     default:
146         return "Continue with Google";
147     }
148 }, [status, error]);
149
150 return (
151     <main className="relative min-h-screen overflow-hidden bg-[#031026]">
152         <div className="absolute inset-0 bg-[radial-gradient(circle_at_22%_14%,rgba(20,149,171,0.46)_0%,rgb
153 a(20,149,171,0.14)_28%,transparent_56%),radial-gradient(circle_at_78%_78%,rgba(248,140,67,0.34)_0%,rgba(2
154 48,140,67,0.12)_34%,transparent_62%),linear-gradient(120deg,#041329_0%,#0a2745_50%,#0f3759_100%)]" />
155         <section className="relative z-10 flex min-h-screen items-center justify-center px-5 py-8 md:justif
156 y-end md:px-8">
157             <div className="w-full max-w-[460px]">
158                 <div className="flex rounded-[10px] border border-white/28 bg-black/34 p-7 shadow-[0_26px_60px_
159 rgba(2,8,20,0.46)] backdrop-blur-[10px] md:min-h-[600px] md:px-9 md:py-10">
160                     <div className="flex w-full flex-col">
161                         
166                         <div className="mt-2 h-px w-full max-w-[210px] bg-[linear-gradient(90deg,#F88C43_0%,#1495AB
167 _52%,#FFFFFF_100%)]" />
168                         <h1 className="mt-7 text-[2.2rem] font-semibold leading-[1.08] text-white md:text-[2.8rem]"
169 >
170                             {title}
171                         </h1>
172                         <p className="mt-5 text-[1.04rem] leading-relaxed text-white/70">{description}</p>
173                         {status === "error" ? (
174                             <div className="mt-6 rounded-2xl border border-red-300/40 bg-red-400/10 px-4 py-3 text-ba
175 se text-red-100">
176                                 {error}
177                             </div>
178                         ) : null}
179                         {profile ? (
180                             <div className="mt-7 rounded-2xl border border-white/25 bg-white/10 px-4 py-3 text-base t
181 ext-white/90">
182                                 Authenticated as <span className="font-semibold text-white">{profile.email}</span>. Red
183 irecting...
184                             </div>
185                         ) : (
186                             <a
187                                 href={loginUrl}
188                                 onClick={handleGoogleSignIn}
189                                 className={`mt-8 flex h-14 w-full items-center justify-center gap-3 rounded-[26px] bg-[
190 #f88c43] px-7 text-[1.2rem] font-semibold text-white [text-shadow:0_1px_2px_rgba(0,0,0,0.28)] shadow-[0_1
191 8px_38px_rgba(248,140,67,0.45)] transition focus-visible:outline-none focus-visible:ring-2 focus-visible:
192 ring-[#f88c43]/45 ${
193                                     status === "loading" || status === "success"
194                                     ? "pointer-events-none opacity-95"
195                                     : "hover:brightness-105"
196                                 }`}
197                             >
198                                 <svg viewBox="0 0 24 24" aria-hidden className="h-7 w-7">
199                                     <path
200                                         fill="#4285F4"
201                                         d="M23.49 12.27c0-.79-.07-1.54-.2-2.27H12v4.3h6.45a5.52 5.52 0 01-2.39 3.62v2.99h3.
202 86c2.26-2.08 3.57-5.14 3.57-8.64z"
203                                     />
204                                     <path
205                                         fill="#34A853"

```

```

197         d="M12 24c3.24 0 5.96-1.07 7.95-2.9l-3.86-2.99c-1.07.72-2.44 1.15-4.09 1.15-3.14 0-
198 5.8-2.12-6.75-4.98H1.26v3.11A12 12 0 0012 24z"
199     />
200     <path
201         fill="#FBBC05"
202         d="M5.25 14.27a7.22 7.22 0 010-4.54V6.62H1.26a12 12 0 000 10.76l3.99-3.11z"
203     />
204     <path
205         fill="#EA4335"
206         d="M12 4.74c1.76 0 3.34.61 4.58 1.8l3.44-3.44C17.95 1.19 15.24 0 12 0 7.31 0 3.23 2
207 .69 1.26 6.62l3.99 3.11c.95-2.86 3.61-4.99 6.75-4.99z"
208     />
209 </svg>
210 <span className="leading-none">{statusLabel}</span>
211 </a>
212 })
213
214 {showDevButton ? (
215     <button
216         type="button"
217         onClick={handleDevTokenLogin}
218         className="mt-3 w-full rounded-full border border-dashed border-[#ffbb92] bg-[#fff3ea]
219 px-4 py-2 text-sm font-semibold text-[#d24f10]"
220     >
221         Use dev session token
222     </button>
223 ) : null}
224
225 <p className="mt-auto pt-9 text-[0.96rem] leading-relaxed text-white/66 md:text-[1rem]">
226     You&apos;ll be redirected through Google OAuth. Grant access to continue into AtomX.
227 </p>
228 </div>
229 </div>
230 </section>
231 </main>
232 );
233 }

```

FILE: packages/global-components/src/index.js

```
1 export * from "./AppShell.js";
2 export * from "./GlobalFooter.js";
3 export * from "./AuthPanel.js";
4 export * from "./UniversalLoginPage.js";
5 export * from "./TokenGate.js";
6 export * from "../loaders/index.js";
7 export * from "./posthog.js";
```

FILE: packages/global-components/src/posthog.js

```
1 import posthog from "posthog-js";
2
3 let posthogReady = false;
4 let gaConfigured = false;
5
6 const FALLBACK_GA_MEASUREMENT_ID = "G-HMEGGM1C7N";
7 const ANALYTICS_CONSENT_KEY = "atomx.analytics.consent.v1";
8
9 function getConsentPayload() {
10   if (typeof window === "undefined") return null;
11   try {
12     const raw = window.localStorage.getItem(ANALYTICS_CONSENT_KEY);
13     if (!raw) return null;
14     const parsed = JSON.parse(raw);
15     if (!parsed || typeof parsed !== "object") return null;
16     return parsed;
17   } catch {
18     return null;
19   }
20 }
21
22 function getEnv() {
23   return {
24     key: process.env.NEXT_PUBLIC_POSTHOG_KEY,
25     host: process.env.NEXT_PUBLIC_POSTHOG_HOST,
26     gaMeasurementId:
27       process.env.NEXT_PUBLIC_GA_MEASUREMENT_ID || FALLBACK_GA_MEASUREMENT_ID
28   };
29 }
30
31 function hasAnalyticsConsent() {
32   const payload = getConsentPayload();
33   return payload?.analytics === true;
34 }
35
36 function applyGaConsent(granted) {
37   if (typeof window === "undefined") return;
38   const { gaMeasurementId } = getEnv();
39   if (!gaMeasurementId) return;
40
41   window[`ga-disable-${gaMeasurementId}`] = !granted;
42   if (typeof window.gtag === "function") {
43     window.gtag("consent", "update", {
44       analytics_storage: granted ? "granted" : "denied"
45     });
46   }
47 }
48
49 function initGoogleAnalytics(app = "portal") {
50   if (typeof window === "undefined") return false;
51   if (!hasAnalyticsConsent()) return false;
52   const { gaMeasurementId } = getEnv();
53   if (!gaMeasurementId) return false;
54
55   if (!window.dataLayer) {
56     window.dataLayer = [];
57   }
58   if (typeof window.gtag !== "function") {
59     window.gtag = function gtag() {
60       window.dataLayer.push(arguments);
61     };
62   }
63
64   if (!gaConfigured) {
65     window.gtag("js", new Date());
66     window.gtag("config", gaMeasurementId, {
67       app_name: app
68     });
69     gaConfigured = true;
```

```
70   }
71
72   if (!document.getElementById("atomx-ga4-script")) {
73     const script = document.createElement("script");
74     script.id = "atomx-ga4-script";
75     script.async = true;
76     script.src = `https://www.googletagmanager.com/gtag/js?id=${encodeURIComponent(gaMeasurementId)}`;
77     document.head.appendChild(script);
78   }
79
80   return true;
81 }
82
83 function buildGaPayload(properties = {}) {
84   const payload = {};
85   Object.entries(properties || {}).forEach(([key, value]) => {
86     if (value === undefined || value === null) return;
87     // Avoid sending email/PII-like fields into GA4 custom events.
88     if (key.toLowerCase().includes("email")) return;
89     if (
90       typeof value === "string" ||
91       typeof value === "number" ||
92       typeof value === "boolean"
93     ) {
94       payload[key] = value;
95     }
96   });
97   return payload;
98 }
99
100 export function initPostHog(app = "portal") {
101   if (!hasAnalyticsConsent()) return false;
102   initGoogleAnalytics(app);
103   if (typeof window === "undefined") return false;
104   if (posthogReady || posthog.__loaded) {
105     posthogReady = true;
106     return true;
107   }
108
109   const { key, host } = getEnv();
110   if (!key || !host) return false;
111
112   posthog.init(key, {
113     api_host: host,
114     capture_pageview: true,
115     capture_pageleave: true,
116     persistence: "localStorage+cookie",
117     loaded: () => {
118       posthogReady = true;
119       posthog.register({ app });
120     }
121   });
122
123   return true;
124 }
125
126 export function capturePostHogEvent(eventName, properties = {}) {
127   if (typeof window === "undefined") return;
128   if (!hasAnalyticsConsent()) return;
129   initPostHog();
130   const gaPayload = buildGaPayload(properties);
131   if (typeof window.gtag === "function") {
132     window.gtag("event", eventName, gaPayload);
133   }
134   if (!posthog.__loaded) return;
135   posthog.capture(eventName, properties);
136 }
137
138 export function identifyPostHogUser(distinctId, properties = {}) {
139   if (typeof window === "undefined" || !distinctId) return;
```

```
140   if (!hasAnalyticsConsent()) return;
141   initPostHog();
142   if (!posthog.__loaded) return;
143   posthog.identify(String(distinctId), properties);
144 }
145
146 export function resetPostHogUser() {
147   if (typeof window === "undefined") return;
148   if (!posthog.__loaded) return;
149   posthog.reset();
150 }
151
152 export function getAnalyticsConsent() {
153   const payload = getConsentPayload();
154   if (!payload) return null;
155   return payload.analytics === true ? "granted" : "denied";
156 }
157
158 export function setAnalyticsConsent(granted, app = "portal") {
159   if (typeof window === "undefined") return;
160
161   const payload = {
162     version: 1,
163     analytics: Boolean(granted),
164     updatedAt: Date.now()
165   };
166   try {
167     window.localStorage.setItem(ANALYTICS_CONSENT_KEY, JSON.stringify(payload));
168   } catch {
169     // ignore storage failures
170   }
171
172   applyGaConsent(Boolean(granted));
173
174   if (granted) {
175     initPostHog(app);
176     if (posthog.__loaded) {
177       posthog.opt_in_capturing();
178     }
179     return;
180   }
181
182   if (posthog.__loaded) {
183     posthog.opt_out_capturing();
184     posthog.reset();
185   }
186 }
```

FILE: packages/lib/package.json

```
1 {  
2   "name": "@atomx/lib",  
3   "version": "0.1.0",  
4   "private": true,  
5   "type": "module",  
6   "main": "src/index.js"  
7 }
```


FILE: packages/lib/src/environment.js

```
1 export function getEnvironment() {
2   return {
3     node: process.version,
4     mode: process.env.NODE_ENV || "development",
5     timestamp: new Date().toISOString()
6   };
7 }
```

FILE: packages/lib/src/index.js

```
1 export * from "./environment.js";
2 export * from "./projects.js";
3 export * from "./token.js";
4 export * from "./apiConfig.js";
```

FILE: packages/lib/src/projects.js

```
1 export const projects = {
2   livelink: {
3     displayName: "LiveLink",
4     tagline: "Real-time content orchestration"
5   },
6   tag_series: {
7     displayName: "Tag Series",
8     tagline: "Advanced analytics for IoT tags"
9   }
10 };
11
12 export function getProjectCopy(key) {
13   return (
14     projects[key] ?? {
15       displayName: "AtomX",
16       tagline: "Unified operations"
17     }
18   );
19 }
```

FILE: packages/lib/src/token.js

```
1 function normalizeBase64(segment = "") {
2   const normalized = segment.replace(/-/g, "+").replace(/_/g, "/");
3   const padding = normalized.length % 4;
4   if (padding === 2) return normalized + "==";
5   if (padding === 3) return normalized + "=";
6   if (padding !== 0) return normalized + "==";
7   return normalized;
8 }
9
10 export function decodeJwt(token) {
11   if (!token) throw new Error("Missing token");
12   const parts = token.split(".");
13   if (parts.length < 2) throw new Error("Invalid JWT format");
14   const payload = normalizeBase64(parts[1]);
15   const decoded =
16     typeof window === "undefined"
17       ? Buffer.from(payload, "base64").toString("utf-8")
18       : atob(payload);
19   return JSON.parse(decoded);
20 }
21
22 export function getInitials(name = "") {
23   if (!name) return "AO";
24   return name
25     .split(" ")
26     .map((part) => part[0])
27     .join("")
28     .slice(0, 2)
29     .toUpperCase();
30 }
```

FILE: packages/public-assets/package.json

```
1 {  
2   "name": "@atomx/public-assets",  
3   "version": "0.1.0",  
4   "private": true,  
5   "type": "module",  
6   "main": "src/index.js"  
7 }
```

FILE: packages/public-assets/src/index.js

```
1 import path from "node:path";
2 import { fileURLToPath } from "node:url";
3
4 const packageRoot = path.join(path.dirname(fileURLToPath(import.meta.url)), "..");
5 const assetRoot = path.join(packageRoot, "assets");
6 const fontRoot = path.join(packageRoot, "fonts");
7 const logoRoot = path.join(packageRoot, "logos");
8
9 export const assetManifest = [
10   "file.svg",
11   "globe.svg",
12   "next.svg",
13   "vercel.svg",
14   "window.svg"
15 ];
16
17 export const fontManifest = [
18   { file: "Poppins-Thin.ttf", weight: 100, style: "normal" },
19   { file: "Poppins-Light.ttf", weight: 300, style: "normal" },
20   { file: "Poppins-Regular.ttf", weight: 400, style: "normal" },
21   { file: "Poppins-MediumItalic.ttf", weight: 500, style: "italic" },
22   { file: "Poppins-SemiBold.ttf", weight: 600, style: "normal" },
23   { file: "Poppins-Bold.ttf", weight: 700, style: "normal" }
24 ];
25
26 export const sharedLogoFile = "AtomX_Logo.svg";
27 export const sharedLogoPublicPath = `/shared/logos/${sharedLogoFile}`;
28
29 export function getAssetPath(asset) {
30   if (!assetManifest.includes(asset)) {
31     throw new Error(`Unknown shared asset: ${asset}`);
32   }
33
34   return path.join(assetRoot, asset);
35 }
36
37 export function getFontPath(fontFile) {
38   const match = fontManifest.find((font) => font.file === fontFile);
39   if (!match) {
40     throw new Error(`Unknown Poppins font: ${fontFile}`);
41   }
42   return {
43     filePath: path.join(fontRoot, fontFile),
44     meta: match
45   };
46 }
47
48 export function getFontPublicPath(fontFile) {
49   const match = fontManifest.find((font) => font.file === fontFile);
50   if (!match) {
51     throw new Error(`Unknown Poppins font: ${fontFile}`);
52   }
53   return {
54     href: `/shared/fonts/${fontFile}`,
55     meta: match
56   };
57 }
58
59 export function getLogoPath() {
60   return path.join(logoRoot, sharedLogoFile);
61 }
62
63 export function getLogoPublicPath() {
64   return sharedLogoPublicPath;
65 }
```

FILE: packages/shared-ui/package.json

```
1 {  
2   "name": "@atomx/shared-ui",  
3   "version": "0.1.0",  
4   "private": true,  
5   "type": "module",  
6   "main": "src/index.js"  
7 }
```

FILE: packages/shared-ui/src/Button.js

```
1 'use client';
2
3 export function SharedButton({
4   children,
5   variant = "solid",
6   className = "",
7   ...rest
8 }) {
9   const base =
10     "inline-flex items-center justify-center rounded-full px-5 py-3 text-sm font-semibold transition-colors focus-visible:outline focus-visible:outline-2 focus-visible:outline-offset-2";
11   const variants = {
12     solid: "bg-black text-white hover:bg-zinc-800 dark:bg-white dark:text-black",
13     outline:
14       "border border-current text-black hover:bg-black/5 dark:text-white dark:hover:bg-white/10",
15     ghost:
16       "text-black/70 hover:text-black dark:text-white/70 dark:hover:text-white"
17   };
18   const variantClasses = variants[variant] ?? variants.solid;
19
20   return (
21     <button className={` ${base} ${variantClasses} ${className}`} {...rest}>
22       {children}
23     </button>
24   );
25 }
26 }
```


FILE: packages/shared-ui/src/Card.js

```
1 export function SharedCard({ title, subtitle, children }) {
2   return (
3     <section className="rounded-3xl border border-black/10 bg-white/80 p-6 shadow-sm backdrop-blur dark:b
order-white/10 dark:bg-white/5">
4       <header className="mb-4 space-y-1">
5         <p className="text-xs uppercase tracking-[0.3em] text-black/50 dark:text-white/60">
6           {subtitle}
7         </p>
8         <h2 className="text-2xl font-semibold text-black dark:text-white">
9           {title}
10        </h2>
11      </header>
12      <div className="text-sm text-black/70 dark:text-white/80">{children}</div>
13    </section>
14  );
15 }
```

FILE: packages/shared-ui/src/SectionHeading.js

```
1 export function SectionHeading({ eyebrow, title, description }) {
2   return (
3     <div className="space-y-2">
4       {eyebrow && (
5         <p className="text-xs uppercase tracking-[0.3em] text-black/60 dark:text-white/60">
6           {eyebrow}
7         </p>
8       )}
9       <h2 className="text-3xl font-semibold text-black dark:text-white">
10        {title}
11      </h2>
12      {description && (
13        <p className="text-base text-black/70 dark:text-white/70">
14          {description}
15        </p>
16      )}
17    </div>
18  );
19 }
```

FILE: packages/shared-ui/src/index.js

```
1 export * from "./Button.js";  
2 export * from "./Card.js";  
3 export * from "./SectionHeading.js";
```

FILE: packages/utils/package.json

```
1 {  
2   "name": "@atomx/utils",  
3   "version": "0.1.0",  
4   "private": true,  
5   "type": "module",  
6   "main": "src/index.js"  
7 }
```

FILE: packages/utils/src/featureFlags.js

```
1  const cache = new Map();
2
3  export function setFeatureFlag(flag, value) {
4    cache.set(flag, value);
5  }
6
7  export function getFeatureFlag(flag, fallback = false) {
8    return cache.has(flag) ? cache.get(flag) : fallback;
9  }
10
11 export function getAllFeatureFlags() {
12   return Object.fromEntries(cache.entries());
13 }
```

FILE: packages/utils/src/formatters.js

```
1 export function formatNumber(value) {
2   return new Intl.NumberFormat("en-US", {
3     maximumFractionDigits: 1
4   }).format(value);
5 }
6
7 export function formatPercentage(value) {
8   return `${value.toFixed(1)}%`;
9 }
```

FILE: packages/utils/src/index.js

```
1 export * from "./formatters.js";  
2 export * from "./featureFlags.js";
```

FILE: scripts/export-all.js

```

1  const { spawnSync } = require("node:child_process");
2  const fs = require("node:fs");
3  const path = require("node:path");
4
5  const apps = [
6    { name: "Tag Series", dir: "apps/tag_series", outLabel: "tag_series" },
7    { name: "Access Portal", dir: "apps/access_portal", outLabel: "access_portal" },
8    { name: "Dashboard", dir: "apps/dashboard", outLabel: "dashboard" }
9  ];
10
11 function run(command, args, cwd, label) {
12   console.log(`\n→ ${label}`);
13   const result = spawnSync(command, args, {
14     cwd,
15     stdio: "inherit",
16     shell: process.platform === "win32"
17   });
18
19   if (result.status !== 0) {
20     throw new Error(`Command failed: ${command} ${args.join(" ")}`);
21   }
22 }
23
24 function main() {
25   const root = process.cwd();
26   const outRoot = path.join(root, "out");
27   fs.rmSync(outRoot, { recursive: true, force: true });
28   fs.mkdirSync(outRoot, { recursive: true });
29
30   apps.forEach((app) => {
31     const appDir = path.join(root, app.dir);
32     if (!fs.existsSync(appDir)) {
33       console.warn(`⚠ Skipping ${app.name} – missing directory ${app.dir}`);
34       return;
35     }
36
37     const pkg = path.join(appDir, "package.json");
38     if (!fs.existsSync(pkg)) {
39       console.warn(`⚠ Skipping ${app.name} – missing package.json in ${app.dir}`);
40       return;
41     }
42
43     // Ensure stale out folders don't interfere with fresh exports.
44     fs.rmSync(path.join(appDir, "out"), { recursive: true, force: true });
45
46     // Force webpack build to avoid Turbopack port-binding panic in this environment.
47     run("npm", ["next", "build", "--webpack"], appDir, `Building ${app.name}`);
48
49     let sourceOut = path.join(appDir, "out");
50     const nestedOut = path.join(sourceOut, app.outLabel);
51     if (fs.existsSync(nestedOut)) {
52       sourceOut = nestedOut;
53     }
54     const targetOut = path.join(outRoot, app.outLabel);
55     fs.rmSync(targetOut, { recursive: true, force: true });
56     fs.mkdirSync(targetOut, { recursive: true });
57     fs.cpSync(sourceOut, targetOut, { recursive: true });
58     console.log(`✓ Copied ${app.name} export to ${path.relative(root, targetOut)}`);
59   });
60
61   const portalOut = path.join(outRoot, "access_portal");
62   if (fs.existsSync(portalOut)) {
63     console.log("\n→ Syncing access portal export to root /out for S3 index");
64     for (const entry of fs.readdirSync(portalOut)) {
65       const from = path.join(portalOut, entry);
66       const to = path.join(outRoot, entry);
67       fs.rmSync(to, { recursive: true, force: true });
68       fs.cpSync(from, to, { recursive: true });
69     }

```



```
70     console.log("✓ Root /out now serves the access portal build (index.html, assets, etc.)");
71 }
72
73 const dashboardOut = path.join(outRoot, "dashboard");
74 if (fs.existsSync(dashboardOut)) {
75     console.log("\n→ Overlaying dashboard routes at root /out (for /admin and /Config)");
76     for (const entry of fs.readdirSync(dashboardOut)) {
77         // Keep access portal shell at root while exposing dashboard routes at root.
78         if (entry === "index.html" || entry === "404.html") continue;
79         const from = path.join(dashboardOut, entry);
80         const to = path.join(outRoot, entry);
81         if (entry === "_next" && fs.existsSync(to)) {
82             // Merge chunk trees so assets from both apps are available.
83             fs.cpSync(from, to, { recursive: true, force: true });
84             continue;
85         }
86         fs.rmSync(to, { recursive: true, force: true });
87         fs.cpSync(from, to, { recursive: true, force: true });
88     }
89     console.log("✓ Root /out now includes dashboard routes (/admin, /Config, etc.)");
90 }
91
92 console.log(`\nAll exports completed. Combined output lives in ${path.relative(root, outRoot)}\`);
93 }
94
95 main();
```

FILE: scripts/sync-public-assets.js

```
1  const fs = require("node:fs");
2  const path = require("node:path");
3
4  const root = process.cwd();
5  const packagesRoot = path.join(root, "packages", "public-assets");
6  const appsDir = path.join(root, "apps");
7
8  const shareMap = [
9    { source: "assets", target: "" },
10   { source: "fonts", target: "fonts" },
11   { source: "logos", target: "logos" }
12 ];
13
14 if (!fs.existsSync(packagesRoot)) {
15   console.warn("Shared assets package missing. Skipping sync.");
16   process.exit(0);
17 }
18
19 const apps = fs
20   .readdirSync(appsDir, { withFileTypes: true })
21   .filter((entry) => entry.isDirectory())
22   .map((entry) => entry.name);
23
24 function copyBucket(appName, sourceDir, targetDir) {
25   if (!fs.existsSync(sourceDir)) {
26     return false;
27   }
28
29   fs.rmSync(targetDir, { recursive: true, force: true });
30   fs.mkdirSync(targetDir, { recursive: true });
31   fs.cpSync(sourceDir, targetDir, { recursive: true });
32   return true;
33 }
34
35 apps.forEach((appName) => {
36   shareMap.forEach(({ source, target }) => {
37     const sourceDir = path.join(packagesRoot, source);
38     const targetDir = path.join(appsDir, appName, "public", "shared", target);
39     if (copyBucket(appName, sourceDir, targetDir)) {
40       console.log(
41         `✓ ${source} synced to ${path.relative(root, targetDir || path.join(appsDir, appName, "public", "
42 shared"))}`
43       );
44     }
45   });
46 });
```